NLP & Computer Vision

Book 4 of the AI Series

Master Language and Vision AI

With Hugging Face, PyTorch, and YOLO

2025 Edition • 28 Chapters

Table of Contents

Chapter 1 — What Is NLP?
Chapter 2 — Text Preprocessing
Chapter 3 — Bag of Words & TF-IDF
Chapter 4 — Word Embeddings
Chapter 5 — Text Classification
Chapter 6 — Named Entity Recognition
Chapter 7 — Text Summarization
Chapter 8 — Sequence-to-Sequence Models
Chapter 9 — BERT
Chapter 10 — The GPT Family
Chapter 11 — Hugging Face
Chapter 12 — Fine-Tuning BERT
Chapter 13 — Question Answering
Chapter 14 — Text Generation
Chapter 15 — Multilingual NLP
Chapter 16 — Image Fundamentals
Chapter 17 — Image Preprocessing
Chapter 18 — Image Classification with CNNs
Chapter 19 — Transfer Learning for Vision
Chapter 20 — Object Detection
Chapter 21 — Image Segmentation
Chapter 22 — Face Detection
Chapter 23 — Vision Transformers
Chapter 24 — OCR
Chapter 25 — Project: Sentiment Analysis API
Chapter 26 — Project: Document Classifier
Chapter 27 — Project: Image Application

Chapter 28 — Project: Object Detection

Chapter 1: What Is NLP?

What You Will Learn

- What Natural Language Processing (NLP) means in plain terms
- Real-world applications you already use every day
- Why teaching computers to understand language is surprisingly hard
- How the field evolved from simple rules to powerful neural networks
- The standard NLP pipeline that most projects follow

Why This Chapter Matters

You talk to computers more than you realize. When you ask Siri a question, when Gmail suggests a reply, when Google translates a web page, or when Netflix recommends a show based on reviews -- all of these use NLP. Understanding what NLP is and why it is difficult gives you the foundation for everything else in this book. Without this big picture, the techniques in later chapters will feel like random tools. With it, you will see how each piece fits into a larger puzzle.

1.1 What Does NLP Mean?

Natural Language Processing (NLP) is a branch of artificial intelligence that helps computers understand, interpret, and generate human language.

Let us break that definition down word by word:

- Natural Language means any language that humans speak or write -- English, Spanish, Mandarin, Hindi, Arabic, and thousands of others. We call them "natural" to distinguish them from "formal" languages like Python or Java, which were designed by humans for

computers.

- Processing means doing something useful with that language -- reading it, understanding it, translating it, summarizing it, or generating new text.

Think of it this way:

> Analogy: Imagine you hire a translator who speaks every language on Earth. You hand them a document in Japanese, and they give you back a perfect English version. NLP is teaching a computer to be that translator -- except it can also summarize the document, answer questions about it, detect if the writer was happy or angry, and do all of this in milliseconds.

A Simple Example

Let us start with a tiny Python example to see NLP in action. We will use a library called `transformers` from Hugging Face:

```python
from transformers import pipeline

# Create a sentiment analysis pipeline
# A pipeline is a ready-to-use NLP tool
classifier = pipeline("sentiment-analysis")

# Analyze the sentiment (feeling) of a sentence
result = classifier("I love learning about artificial intelligence!")

print(result)
```

Output:

```
[{'label': 'POSITIVE', 'score': 0.9998}]
```

Line-by-line explanation:

1. `from transformers import pipeline` -- We import the `pipeline` function from the Hugging Face `transformers` library. A pipeline is a pre-built tool that bundles a model and all the steps needed to use it.
2. `classifier = pipeline("sentiment-analysis")` -- We create a sentiment analysis tool. Sentiment analysis means figuring out whether text expresses a positive, negative, or neutral feeling.
3. `result = classifier("I love learning about artificial intelligence!")` -- We pass a sentence to our classifier. It reads the text and decides the sentiment.
4. `print(result)` -- We print the result. The model says the sentiment is `POSITIVE` with a confidence score of `0.9998` (almost 100% sure).

With just four lines of code, we built a system that understands human emotion in text. That is the power of modern NLP.

1.2 Real-World Applications of NLP

NLP is everywhere. Here are the most common applications:

1.2.1 Chatbots and Virtual Assistants

When you say "Hey Siri, what is the weather today?" or type a question to a customer support chatbot, NLP is working behind the scenes.

```
You: "What time does the store close?"

NLP Steps:
1. Understand the INTENT --> The user wants store hours
2. Extract ENTITIES    --> "store" and "close" (closing time)
3. Generate RESPONSE   --> "The store closes at 9 PM today."
```

- Intent means the purpose or goal behind what someone says.
- Entity means a specific piece of information like a name, date, time, or place.

1.2.2 Machine Translation

Google Translate handles over 100 billion words per day. When it translates "Good morning" to "Buenos dias," it uses NLP models trained on millions of translated documents.

```python
from transformers import pipeline

translator = pipeline("translation_en_to_fr")
result = translator("Good morning, how are you today?")

print(result)
```

Output:

```
[{'translation_text': 'Bonjour, comment allez-vous aujourd\'hui?'}]
```

1.2.3 Sentiment Analysis

Companies analyze thousands of customer reviews to understand how people feel about their products.

```
Review: "The battery life is amazing but the screen is too small."

NLP Analysis:
- Battery life --> POSITIVE
- Screen size  --> NEGATIVE
- Overall      --> MIXED
```

1.2.4 Search Engines

When you type "best pizza near me" into Google, NLP helps the search engine understand that you want:

- best = highly rated
- pizza = a type of food
- near me = your current location

It does not just match keywords. It understands meaning.

1.2.5 Email Filtering

Gmail uses NLP to sort your emails into Primary, Social, and Promotions. It also detects spam by analyzing the language patterns in emails.

1.2.6 Text Summarization

NLP can read a long article and produce a short summary:

```python
from transformers import pipeline

summarizer = pipeline("summarization")

long_text = """
Natural language processing is a subfield of linguistics, computer
science, and artificial intelligence concerned with the interactions
between computers and human language. The goal is to enable computers
to understand, interpret, and generate human language in a way that
is both meaningful and useful. NLP combines computational linguistics
with statistical, machine learning, and deep learning models.
"""
```

```
result = summarizer(long_text, max_length=50, min_length=20)
print(result[0]['summary_text'])
```

Output:

```
Natural language processing is a subfield of linguistics, computer
science, and artificial intelligence. The goal is to enable computers
to understand, interpret, and generate human language.
```

Summary of Applications

```
+-----------------------------------------------------+
|           NLP Applications Map                      |
+-----------------------------------------------------+
|                                                     |
|  Communication        Understanding                 |
|  +-------------+     +------------------+          |
|  | Chatbots    |     | Sentiment        |          |
|  | Translation |     | Analysis         |          |
|  | Text Gen    |     | Topic Detection  |          |
|  +-------------+     +------------------+          |
|                                                     |
|  Organization         Search                        |
|  +-------------+     +------------------+          |
|  | Email Filter|     | Search Engines   |          |
|  | Summarize   |     | Question Answer  |          |
|  | Classify    |     | Info Retrieval   |          |
|  +-------------+     +------------------+          |
|                                                     |
+-----------------------------------------------------+
```

1.3 Why Is NLP Hard?

Teaching a computer to understand language is one of the hardest problems in AI. Here is why:

1.3.1 Ambiguity

The same word can mean completely different things depending on context.

```
"I went to the BANK."

Meaning 1: A financial institution (where you keep money)
Meaning 2: The side of a river (where you might go fishing)

How does the computer know which one? It needs CONTEXT.
```

This is called lexical ambiguity -- when a single word has multiple meanings. "Lexical" means "related to words."

There is also structural ambiguity -- when a sentence can be parsed (broken down) in multiple ways:

```
"I saw the man with the telescope."

Meaning 1: I used a telescope to see the man.
Meaning 2: I saw a man who was holding a telescope.

Same words, two very different meanings!
```

1.3.2 Context Dependence

Words change meaning based on what comes before and after them:

```
"That test was SICK!"

In a medical context: The test detected illness.
In slang context:     The test was really cool/impressive.
```

Humans figure this out instantly from context. Computers struggle.

1.3.3 Sarcasm and Irony

```
"Oh great, another Monday. Just what I needed."

Literal meaning:  The person is happy about Monday.
Actual meaning:   The person HATES Mondays.

The words say one thing. The meaning is the opposite.
```

> Analogy: Teaching a computer to detect sarcasm is like teaching someone who has never heard a joke to understand comedy. The words alone are not enough -- you need to understand tone, culture, and shared expectations.

1.3.4 Language is Always Changing

New words appear constantly: "selfie," "hashtag," "blockchain," "doomscrolling." Slang evolves. Grammar rules bend. A model trained on formal English might completely fail on social media text:

```
Formal:  "That was a very enjoyable experience."
Twitter: "ngl that was lowkey fire frfr no cap"
```

1.3.5 The Scale Problem

There are roughly 7,000 languages in the world. Even within English, there are countless dialects, accents (in written form, like "y'all" vs "you guys"), and specialized vocabularies (medical, legal, technical).

Why NLP Is Hard -- Visual Summary

```
+-----------------------------------------------------+
|           Why NLP Is Hard                           |
+-----------------------------------------------------+
|                                                     |
|  1. AMBIGUITY                                       |
|     "bank" = money place OR river edge?             |
|                                                     |
|  2. CONTEXT                                         |
|     "sick" = ill OR awesome?                        |
|                                                     |
|  3. SARCASM                                         |
|     "Great job" = praise OR criticism?              |
|                                                     |
|  4. EVOLVING LANGUAGE                               |
|     New words, slang, abbreviations                 |
|                                                     |
|  5. SCALE                                           |
|     7000+ languages, countless dialects             |
|                                                     |
+-----------------------------------------------------+
```

1.4 A Brief History of NLP

NLP has gone through several major phases. Understanding this history helps you appreciate why modern tools work the way they do.

Phase 1: Rule-Based Systems (1950s-1980s)

Early NLP used hand-written rules. Linguists (language scientists) would write thousands of grammar rules by hand.

```
Rule Example:
IF sentence contains "not" + positive_word
THEN sentiment = NEGATIVE

"I am NOT happy" --> matches rule --> NEGATIVE
```

The problem: Language has too many exceptions. You would need millions of rules, and they would still miss edge cases.

> Analogy: Imagine trying to write a rulebook that covers every possible conversation in English. You would never finish, and the book would be outdated before the ink dried.

Phase 2: Statistical Methods (1990s-2000s)

Instead of writing rules, researchers let computers learn patterns from data.

```
Approach: Count word frequencies in positive vs negative reviews

Positive reviews often contain: "great", "love", "excellent"
Negative reviews often contain: "terrible", "hate", "awful"

New review: "The food was great" --> Probably POSITIVE
```

Key techniques from this era:

- Naive Bayes -- A simple probability-based classifier
- TF-IDF -- A way to measure how important a word is (covered in Chapter 3)
- Hidden Markov Models -- Used for speech recognition and part-of-speech tagging

Phase 3: Machine Learning (2000s-2010s)

More powerful algorithms and more data led to better results:

- Support Vector Machines (SVM) -- Powerful classifiers for text
- Word2Vec (2013) -- Words represented as numbers that capture meaning (covered in Chapter 4)

- Recurrent Neural Networks (RNN) -- Neural networks that can process sequences of words

Phase 4: The Transformer Revolution (2017-Present)

In 2017, a groundbreaking paper introduced the Transformer architecture. This led to:

- BERT (2018) -- Google's model that understands context in both directions
- GPT (2018-2023) -- OpenAI's text generation models
- T5, RoBERTa, XLNet -- Variations and improvements

```
+-------------------------------------------------------------+
|              Evolution of NLP                               |
+-------------------------------------------------------------+
|                                                             |
|  1950s          1990s          2013         2017           |
|    |              |              |            |             |
|    v              v              v            v             |
| [Rules]  --> [Statistics] --> [Deep    --> [Transformers] |
|                                Learning]                    |
|                                                             |
| Hand-      Count word      Words as     Attention          |
| written    frequencies     vectors      mechanism          |
| grammar                    (Word2Vec)   (BERT, GPT)        |
| rules                                                       |
|                                                             |
| Accuracy:  Accuracy:       Accuracy:    Accuracy:          |
| ~60%       ~75%            ~85%         ~95%+              |
|                                                             |
+-------------------------------------------------------------+
```

A Quick Comparison

```python
# Phase 1: Rule-based (manual, fragile)
def rule_based_sentiment(text):
    positive_words = ["good", "great", "love", "excellent"]
    negative_words = ["bad", "terrible", "hate", "awful"]

    text_lower = text.lower()
    pos_count = sum(1 for w in positive_words if w in text_lower)
    neg_count = sum(1 for w in negative_words if w in text_lower)

    if pos_count > neg_count:
        return "POSITIVE"
    elif neg_count > pos_count:
        return "NEGATIVE"
    else:
```

```
        return "NEUTRAL"

# Test with simple sentences
print(rule_based_sentiment("This movie is great"))
print(rule_based_sentiment("This movie is terrible"))
print(rule_based_sentiment("The movie was not bad"))
```

Output:

```
POSITIVE
NEGATIVE
NEUTRAL
```

Notice the third example: "The movie was not bad" should be somewhat positive, but our rule-based system says NEUTRAL because it does not understand negation properly. Modern NLP models handle this correctly.

```python
# Phase 4: Transformer-based (learned, robust)
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

print(classifier("This movie is great"))
print(classifier("This movie is terrible"))
print(classifier("The movie was not bad"))
```

Output:

```
[{'label': 'POSITIVE', 'score': 0.9998}]
[{'label': 'NEGATIVE', 'score': 0.9994}]
[{'label': 'POSITIVE', 'score': 0.9966}]
```

The transformer correctly identifies "not bad" as positive. This shows how far NLP has come.

1.5 The NLP Pipeline

Most NLP projects follow a standard sequence of steps called a pipeline. Think of it like a factory assembly line -- raw text goes in, useful results come out.

```
+-------------------------------------------------------------------+
|                    The NLP Pipeline                               |
+-------------------------------------------------------------------+
|                                                                   |
|  Raw Text                                                         |
|     |                                                             |
|     v                                                             |
|  +------------------+                                             |
|  | 1. TEXT CLEANUP   |  Remove noise, fix encoding, lowercase      |
|  +------------------+                                             |
|     |                                                             |
|     v                                                             |
|  +------------------+                                             |
|  | 2. TOKENIZATION   |  Split text into words or subwords          |
|  +------------------+                                             |
|     |                                                             |
|     v                                                             |
|  +------------------+                                             |
|  | 3. NORMALIZATION  |  Stemming, lemmatization, stop words        |
|  +------------------+                                             |
|     |                                                             |
|     v                                                             |
|  +------------------+                                             |
|  | 4. REPRESENTATION |  Convert text to numbers (vectors)          |
|  +------------------+                                             |
|     |                                                             |
|     v                                                             |
|  +------------------+                                             |
|  | 5. MODELING        |  Apply ML/DL model to make predictions     |
|  +------------------+                                             |
|     |                                                             |
|     v                                                             |
|  +------------------+                                             |
|  | 6. POST-PROCESS   |  Format results, apply business rules       |
|  +------------------+                                             |
|     |                                                             |
|     v                                                             |
|  Output (sentiment, translation, summary, etc.)                   |
|                                                                   |
+-------------------------------------------------------------------+
```

Let us walk through each step with a concrete example:

Step-by-Step Example

Suppose we want to classify the sentiment of this review:

```
"I LOVED this movie!!! The acting was absolutely FANTASTIC...
 best film of 2024 <3 <3 <3"
```

Step 1: Text Cleanup

```
Input:  "I LOVED this movie!!! The acting was absolutely FANTASTIC...
         best film of 2024 <3 <3 <3"
Output: "i loved this movie the acting was absolutely fantastic
         best film of 2024"
```

We lowercased everything, removed punctuation, and removed special characters.

Step 2: Tokenization

```
Input:  "i loved this movie the acting was absolutely fantastic
         best film of 2024"
Output: ["i", "loved", "this", "movie", "the", "acting", "was",
         "absolutely", "fantastic", "best", "film", "of", "2024"]
```

We split the text into individual words (called tokens).

Step 3: Normalization

```
Input:  ["i", "loved", "this", "movie", "the", "acting", "was",
         "absolutely", "fantastic", "best", "film", "of", "2024"]
Output: ["loved", "movie", "acting", "absolutely", "fantastic",
         "best", "film", "2024"]
```

We removed stop words (common words like "i", "this", "the", "was", "of" that carry little meaning).

Step 4: Representation

```
Input:  ["loved", "movie", "acting", "absolutely", "fantastic",
         "best", "film", "2024"]
Output: [0.82, 0.15, 0.67, 0.91, 0.95, 0.88, 0.12, 0.01]
         (simplified -- each word becomes a number or vector)
```

Step 5: Modeling

```
Input:  [0.82, 0.15, 0.67, 0.91, 0.95, 0.88, 0.12, 0.01]
Output: {"label": "POSITIVE", "confidence": 0.97}
```

A machine learning model processes the numbers and makes a prediction.

Step 6: Post-Processing

```
Input:  {"label": "POSITIVE", "confidence": 0.97}
Output: "This review is POSITIVE (97% confidence)"
```

We format the result for the end user.

Code Example: Simple Pipeline

```python
import re

def simple_nlp_pipeline(text):
    """A basic NLP preprocessing pipeline."""

    # Step 1: Text Cleanup
    # Convert to lowercase
    text = text.lower()
    # Remove special characters but keep letters, numbers, spaces
    text = re.sub(r'[^a-z0-9\s]', '', text)
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    print(f"After cleanup: {text}")

    # Step 2: Tokenization
    # Split into individual words
    tokens = text.split()
    print(f"After tokenization: {tokens}")

    # Step 3: Normalization
    # Remove common stop words
    stop_words = {'i', 'the', 'this', 'was', 'of', 'a', 'an',
                  'is', 'it', 'to', 'and', 'in', 'for'}
    tokens = [word for word in tokens if word not in stop_words]
    print(f"After stop word removal: {tokens}")

    return tokens

# Test the pipeline
text = "I LOVED this movie!!! The acting was absolutely FANTASTIC..."
result = simple_nlp_pipeline(text)
```

Output:

```
After cleanup: i loved this movie the acting was absolutely fantastic
After tokenization: ['i', 'loved', 'this', 'movie', 'the', 'acting', 'was', 'absolutely...
After stop word removal: ['loved', 'movie', 'acting', 'absolutely', 'fantastic']
```

Line-by-line explanation:

1. `import re` -- Import Python's regular expression module. Regular expressions (regex) are patterns used to find and replace text.
2. `text = text.lower()` -- Convert all characters to lowercase so "LOVED" and "loved" are treated as the same word.
3. `re.sub(r'[^a-z0-9\s]', '', text)` -- Remove anything that is not a lowercase letter, digit, or space. The `^` inside `[]` means "not these characters."
4. `re.sub(r'\s+', ' ', text).strip()` -- Replace multiple spaces with a single space and remove leading/trailing spaces.
5. `tokens = text.split()` -- Split the string on spaces to get a list of words.
6. The list comprehension `[word for word in tokens if word not in stop_words]` keeps only words that are not in our stop words set.

1.6 NLP vs NLU vs NLG

You might see these related terms. Here is how they differ:

```
+-------------------------------------------------------------+
|                                                              |
|  NLP (Natural Language Processing)                           |
|  The overall field -- everything related to                  |
|  computers and human language                                |
|                                                              |
|  +-------------------------+  +-------------------------+ |
|  | NLU                     |  | NLG                     | |
|  | Natural Language        |  | Natural Language        | |
|  | Understanding           |  | Generation              | |
|  |                         |  |                         | |
|  | Computer READS and      |  | Computer WRITES         | |
|  | UNDERSTANDS text        |  | and CREATES text        | |
|  |                         |  |                         | |
|  | Examples:               |  | Examples:               | |
|  | - Sentiment analysis    |  | - Chatbot responses     | |
|  | - Intent detection      |  | - Text summarization    | |
|  | - Named entity          |  | - Machine translation   | |
|  |   recognition           |  | - Story writing         | |
|  +-------------------------+  +-------------------------+ |
|                                                              |
+-------------------------------------------------------------+
```

- NLU (Natural Language Understanding) focuses on comprehension -- can the computer understand what text means?
- NLG (Natural Language Generation) focuses on production -- can the computer write text that sounds natural?

- NLP is the umbrella term that covers both.

Common Mistakes

1. Thinking NLP is "solved" -- While models like GPT are impressive, they still struggle with sarcasm, nuanced context, rare languages, and factual accuracy. NLP is much better than before, but it is far from perfect.
2. Ignoring preprocessing -- Jumping straight to a model without cleaning your text first leads to poor results. Garbage in, garbage out.
3. Using English-only tools on other languages -- Many NLP tools and models are built for English. Always check if your tool supports the language you need.
4. Confusing word matching with understanding -- Just because a program finds the word "happy" in text does not mean it understands happiness. True understanding requires context.
5. Overcomplicating simple problems -- Sometimes a basic keyword search or regular expression is enough. Not every text problem needs a transformer model.

Best Practices

1. Start simple, then add complexity -- Begin with rule-based or statistical methods. Move to deep learning only if simpler methods are not good enough.
2. Always look at your data first -- Read sample texts before choosing an approach. Understanding your data is more valuable than any algorithm.
3. Preprocess consistently -- Apply the same cleaning steps to training data and new data. If you lowercase during training, you must lowercase during prediction too.
4. Consider your language -- If working with non-English text, choose tools and models that support that language.
5. Evaluate on real-world examples -- Test your NLP system on text that looks like what it will see in production, including messy, informal, and edge-case text.

Quick Summary

Natural Language Processing (NLP) is the field of AI that teaches computers to work with human language. It powers everyday tools like search engines, chatbots, email filters, and translation services. NLP is challenging because human language is full of ambiguity, context-dependence, sarcasm, and constant evolution. The field has progressed from hand-written rules in the 1950s to powerful transformer models today. Most NLP projects follow a standard pipeline: clean the text, split it into tokens, normalize it, convert it to numbers, apply a model, and post-process the results.

Key Points

- NLP stands for Natural Language Processing -- teaching computers to understand human language.
- Common applications include chatbots, translation, sentiment analysis, search, summarization, and email filtering.
- NLP is hard because of ambiguity, context dependence, sarcasm, evolving language, and the sheer number of languages.
- The field evolved from rule-based systems (1950s) to statistical methods (1990s) to deep learning (2010s) to transformers (2017+).
- The standard NLP pipeline has six steps: cleanup, tokenization, normalization, representation, modeling, and post-processing.
- NLU (understanding) and NLG (generation) are both subfields of NLP.
- Modern tools like Hugging Face Transformers make NLP accessible with just a few lines of code.

Practice Questions

1. What is the difference between "natural language" and "formal language"? Give two examples of each.
2. Explain why the sentence "I saw the man with the telescope" is ambiguous. How many interpretations does it have?
3. Name three real-world applications of NLP that you use in your daily life. For each one, explain what NLP task it performs.
4. Why would a rule-based sentiment analyzer fail on the sentence "The movie was not bad at all"? How would a modern transformer handle it differently?

5. List the six steps of the standard NLP pipeline in order. For each step, write one sentence explaining what it does.

Exercises

Exercise 1: Extend the Rule-Based Analyzer

Take the `rule_based_sentiment` function from Section 1.4 and improve it:

- Add more positive and negative words
- Handle negation (e.g., "not good" should be negative)
- Test it on at least five sentences and note where it still fails

Hint: You could check if the word "not" appears before a positive or negative word.

Exercise 2: Explore Hugging Face Pipelines

Use the Hugging Face `pipeline` function to try three different NLP tasks:

- `"sentiment-analysis"` -- Analyze the sentiment of five sentences
- `"ner"` (Named Entity Recognition) -- Find names, places, and organizations in a paragraph
- `"question-answering"` -- Give it a context paragraph and ask a question

Write down your observations: Where does the model succeed? Where does it fail?

Exercise 3: Build a Mini NLP Pipeline

Write a Python function called `full_pipeline(text)` that:

1. Converts text to lowercase
2. Removes all punctuation and special characters
3. Splits the text into tokens
4. Removes stop words
5. Counts how many times each remaining word appears
6. Returns the top 5 most frequent words

Test it on a paragraph from any news article.

What Is Next?

In this chapter, you learned what NLP is and why it matters. You saw that the NLP pipeline starts with raw, messy text and transforms it into something a computer can work with. In Chapter 2: Text Preprocessing, we will dive deep into the first three steps of this pipeline. You will learn how to tokenize text (split it into pieces), remove noise, and normalize words using professional tools like NLTK and spaCy. Preprocessing is where most NLP projects spend the majority of their time, so mastering it will set you apart.

Chapter 2: Text Preprocessing

What You Will Learn

- How to split text into words and subwords (tokenization)
- Why and how to lowercase text
- How to remove punctuation and special characters
- What stop words are and when to remove them
- The difference between stemming and lemmatization
- How to use regular expressions (regex) for text cleaning
- How to build a complete preprocessing pipeline with NLTK and spaCy

Why This Chapter Matters

Raw text is messy. It has capital letters, punctuation marks, emojis, HTML tags, extra spaces, and typos. If you feed this messy text directly into a machine learning model, the model gets confused. It might think "Running," "running," and "RUNNING" are three different words. Preprocessing transforms chaotic raw text into clean, consistent data that models can actually learn from. Data scientists often say they spend 80% of their time on data preparation -- and in NLP, preprocessing is that 80%.

2.1 Why Do We Need Text Preprocessing?

Let us look at a simple example to understand the problem:

```python
# Without preprocessing, these are all "different" to a computer
words = ["Running", "running", "RUNNING", "running!", "running..."]

# How many unique words does Python see?
print(f"Unique words: {len(set(words))}")
print(f"The set: {set(words)}")
```

Output:

```
Unique words: 5
The set: {'running!', 'Running', 'RUNNING', 'running...', 'running'}
```

Python sees five completely different words! But to a human, they are all the same word: "running." This is why we need preprocessing.

> Analogy: Imagine you are sorting mail at a post office. Letters arrive with addresses written in all caps, lowercase, with typos, in different handwriting styles. Before you can sort them, you need to standardize every address into a clean, consistent format. Text preprocessing is the same -- standardizing text before processing it.

```
+-------------------------------------------------------------+
|          Why Preprocessing Matters                          |
+-------------------------------------------------------------+
|                                                             |
|  Raw Text:                                                  |
|  "I LOVED this movie!!! It was SO good :) <br>"            |
|                                                             |
|        |                                                    |
|        v                                                    |
|                                                             |
|  Clean Text:                                                |
|  "loved movie good"                                         |
|                                                             |
|  Result: Fewer words, less noise, better model accuracy     |
|                                                             |
+-------------------------------------------------------------+
```

2.2 Tokenization

Tokenization is the process of splitting text into smaller pieces called tokens. A token is usually a word, but it can also be a subword, a character, or even a sentence.

2.2.1 Word Tokenization

The simplest form: split text on spaces.

```python
# Simple word tokenization using Python's split()
text = "Natural language processing is fascinating"
tokens = text.split()

print(tokens)
print(f"Number of tokens: {len(tokens)}")
```

Output:

```
['Natural', 'language', 'processing', 'is', 'fascinating']
Number of tokens: 5
```

But `split()` has problems with punctuation:

```python
text = "Hello, world! How's it going?"
tokens = text.split()
print(tokens)
```

Output:

```
['Hello,', 'world!', "How's", 'it', 'going?']
```

Notice "Hello," includes the comma and "world!" includes the exclamation mark. We need smarter tokenization.

2.2.2 Tokenization with NLTK

NLTK (Natural Language Toolkit) is one of the most popular NLP libraries in Python. It handles punctuation properly:

```python
import nltk
# Download required data (only needed once)
nltk.download('punkt_tab', quiet=True)

from nltk.tokenize import word_tokenize

text = "Hello, world! How's it going?"
```

```
tokens = word_tokenize(text)

print(tokens)
print(f"Number of tokens: {len(tokens)}")
```

Output:

```
['Hello', ',', 'world', '!', 'How', "'s", 'it', 'going', '?']
Number of tokens: 9
```

Line-by-line explanation:

1. `import nltk` -- Import the NLTK library.
2. `nltk.download('punkt_tab', quiet=True)` -- Download the tokenizer data. The `punkt_tab` resource contains rules for splitting text. You only need to run this once.
3. `from nltk.tokenize import word_tokenize` -- Import the word tokenizer function.
4. `word_tokenize(text)` -- Split the text into tokens. Notice how it separates "How's" into "How" and "'s", and punctuation marks become their own tokens.

2.2.3 Tokenization with spaCy

spaCy is another popular NLP library, designed for production use. It is faster than NLTK for many tasks:

```python
import spacy

# Load the English language model
# Run this first in your terminal: python -m spacy download en_core_web_sm
nlp = spacy.load("en_core_web_sm")

text = "Hello, world! How's it going?"
doc = nlp(text)

# Extract tokens from the processed document
tokens = [token.text for token in doc]

print(tokens)
print(f"Number of tokens: {len(tokens)}")
```

Output:

```
['Hello', ',', 'world', '!', 'How', "'s", 'it', 'going', '?']
Number of tokens: 9
```

Line-by-line explanation:

1. `import spacy` -- Import the spaCy library.
2. `nlp = spacy.load("en_core_web_sm")` -- Load a pre-trained English language model. The `"en_core_web_sm"` model is a small, efficient model for English.
3. `doc = nlp(text)` -- Process the text. spaCy creates a `Doc` object that contains all the tokens and their linguistic information.
4. `[token.text for token in doc]` -- Extract the text of each token using a list comprehension.

2.2.4 Sentence Tokenization

Sometimes you need to split text into sentences rather than words:

```python
from nltk.tokenize import sent_tokenize

text = "NLP is amazing. It powers many applications. Let's learn more!"
sentences = sent_tokenize(text)

for i, sent in enumerate(sentences, 1):
    print(f"Sentence {i}: {sent}")
```

Output:

```
Sentence 1: NLP is amazing.
Sentence 2: It powers many applications.
Sentence 3: Let's learn more!
```

2.2.5 Subword Tokenization

Modern models like BERT and GPT use subword tokenization. Instead of splitting into whole words, they split into meaningful pieces:

```python
from transformers import AutoTokenizer

# Load the BERT tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "I love tokenization and preprocessing"
tokens = tokenizer.tokenize(text)

print(f"Tokens: {tokens}")
```

Output:

```
Tokens: ['i', 'love', 'token', '##ization', 'and', 'pre', '##processing']
```

Notice how "tokenization" becomes "token" + "##ization" and "preprocessing" becomes "pre" + "##processing." The `##` prefix means "this piece is attached to the previous piece." This approach handles words the model has never seen before by breaking them into known parts.

```
+-------------------------------------------------------------+
|           Types of Tokenization                             |
+-------------------------------------------------------------+
|                                                             |
|  Word:     ["I", "love", "tokenization"]                    |
|            Simple but fails on unknown words                |
|                                                             |
|  Subword:  ["I", "love", "token", "##ization"]              |
|            Handles unknown words by splitting them           |
|                                                             |
|  Character: ["I", " ", "l", "o", "v", "e", ...]             |
|            Very flexible but loses word meaning              |
|                                                             |
+-------------------------------------------------------------+
```

2.3 Lowercasing

Lowercasing means converting all characters to lowercase. This ensures that "Apple," "apple," and "APPLE" are treated as the same word.

```python
texts = ["The Cat sat on the MAT",
         "THE CAT SAT ON THE MAT",
         "the cat sat on the mat"]

# Without lowercasing -- all different
print("Without lowercasing:")
print(f"  All same? {texts[0] == texts[1] == texts[2]}")

# With lowercasing -- all the same
lowered = [t.lower() for t in texts]
print("\nAfter lowercasing:")
for t in lowered:
    print(f"  {t}")
print(f"  All same? {lowered[0] == lowered[1] == lowered[2]}")
```

Output:

```
Without lowercasing:
  All same? False

After lowercasing:
  the cat sat on the mat
  the cat sat on the mat
  the cat sat on the mat
  All same? True
```

When NOT to Lowercase

Lowercasing is not always the right choice:

```
Cases where lowercasing LOSES information:

1. Named Entities:
   "Apple" (the company) vs "apple" (the fruit)

2. Abbreviations:
   "US" (United States) vs "us" (pronoun)

3. Sentiment emphasis:
   "This is AMAZING" (strong positive) vs "this is amazing"
```

> Rule of thumb: Lowercase for most classification tasks. Keep original casing if you need to detect named entities or if capitalization carries meaning.

2.4 Removing Punctuation

Punctuation marks like commas, periods, and exclamation marks usually do not help with tasks like classification. We remove them to reduce noise.

```python
import string

text = "Hello, world! How are you doing today???"

# Method 1: Using str.translate (fastest)
# string.punctuation contains: !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
```

```
translator = str.maketrans('', '', string.punctuation)
clean_text = text.translate(translator)

print(f"Original:  {text}")
print(f"Cleaned:   {clean_text}")
print(f"Punctuation chars: {string.punctuation}")
```

Output:

```
Original:  Hello, world! How are you doing today???
Cleaned:   Hello world How are you doing today
Punctuation chars: !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
```

Line-by-line explanation:

1. `import string` -- Import Python's `string` module, which contains useful constants like `string.punctuation`.
2. `str.maketrans('', '', string.punctuation)` -- Create a translation table that maps every punctuation character to `None` (meaning "delete it").
3. `text.translate(translator)` -- Apply the translation table to remove all punctuation characters.

When NOT to Remove Punctuation

```
Cases where punctuation carries meaning:

1. Contractions:
   "don't" --> removing ' gives "dont" (not a real word)

2. Emoticons:
   ":)" and ":(" carry sentiment information

3. Domain-specific:
   "$100", "C++", "Dr.", "U.S.A."
```

2.5 Stop Words

Stop words are common words that appear frequently but carry little meaning on their own. Words like "the," "is," "at," "which," and "on" are stop words.

> Analogy: In a recipe, the important words are "chicken," "garlic," and "roast." Words like "the," "a," and "of" are just glue holding the sentence together. Stop words are the glue -- useful for grammar but not for understanding meaning.

2.5.1 Stop Words with NLTK

```python
import nltk
nltk.download('stopwords', quiet=True)

from nltk.corpus import stopwords

# Get English stop words
stop_words = set(stopwords.words('english'))

print(f"Number of stop words: {len(stop_words)}")
print(f"First 20: {sorted(list(stop_words))[:20]}")
```

Output:

```
Number of stop words: 179
First 20: ["a", "about", "above", "after", "again", "against", "ain", "all", "am", "an"...
```

2.5.2 Removing Stop Words

```python
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))

text = "This is a sample sentence showing stop word removal"
tokens = word_tokenize(text.lower())

# Keep only words that are NOT stop words
filtered = [word for word in tokens if word not in stop_words]

print(f"Original tokens:  {tokens}")
print(f"After filtering:  {filtered}")
print(f"Removed {len(tokens) - len(filtered)} stop words")
```

Output:

```
Original tokens:  ['this', 'is', 'a', 'sample', 'sentence', 'showing', 'stop', 'word', ...
After filtering:  ['sample', 'sentence', 'showing', 'stop', 'word', 'removal']
Removed 3 stop words
```

2.5.3 Stop Words with spaCy

```python
import spacy

nlp = spacy.load("en_core_web_sm")

text = "This is a sample sentence showing stop word removal"
doc = nlp(text)

# spaCy marks each token as a stop word or not
for token in doc:
    print(f"  {token.text:12s} --> is_stop = {token.is_stop}")
```

Output:

```
  This         --> is_stop = True
  is           --> is_stop = True
  a            --> is_stop = True
  sample       --> is_stop = False
  sentence     --> is_stop = False
  showing      --> is_stop = False
  stop         --> is_stop = False
  word         --> is_stop = False
  removal      --> is_stop = False
```

When NOT to Remove Stop Words

```
Cases where stop words matter:

1. Sentiment:
   "not good" --> removing "not" changes meaning to "good"

2. Phrases:
   "to be or not to be" --> all stop words, but meaningful

3. Question answering:
   "What is the capital of France?"
   --> "what" and "is" help identify this as a question
```

2.6 Stemming vs Lemmatization

Both stemming and lemmatization reduce words to their base form. But they do it differently.

2.6.1 Stemming

Stemming chops off the end of words using simple rules. It is fast but sometimes produces words that are not real English words.

```python
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["running", "runs", "ran", "runner", "easily",
         "fairly", "studies", "studying", "studied"]

for word in words:
    stem = stemmer.stem(word)
    print(f"  {word:12s} --> {stem}")
```

Output:

```
  running      --> run
  runs         --> run
  ran          --> ran
  runner       --> runner
  easily       --> easili
  fairly       --> fairli
  studies      --> studi
  studying     --> studi
  studied      --> studi
```

Notice "easily" becomes "easili" and "studies" becomes "studi" -- these are not real words! The stemmer just blindly chops endings.

> Analogy: Stemming is like cutting the branches of a tree with a chainsaw. It is fast, but the cuts are rough and imprecise.

2.6.2 Lemmatization

Lemmatization uses a dictionary to find the proper base form (lemma) of a word. It is slower but produces real words.

```python
import nltk
nltk.download('wordnet', quiet=True)

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
```

```
words = ["running", "runs", "ran", "runner", "easily",
         "fairly", "studies", "studying", "studied"]

for word in words:
    # pos='v' tells the lemmatizer to treat the word as a verb
    lemma_v = lemmatizer.lemmatize(word, pos='v')
    lemma_n = lemmatizer.lemmatize(word, pos='n')
    print(f"  {word:12s} --> verb: {lemma_v:10s}  noun: {lemma_n}")
```

Output:

```
  running      --> verb: run         noun: running
  runs         --> verb: run         noun: run
  ran          --> verb: run         noun: ran
  runner       --> verb: runner      noun: runner
  easily       --> verb: easily      noun: easily
  fairly       --> verb: fairly      noun: fairly
  studies      --> verb: study       noun: study
  studying     --> verb: study       noun: studying
  studied      --> verb: study       noun: studied
```

Line-by-line explanation:

1. `WordNetLemmatizer()` -- Creates a lemmatizer that uses WordNet, a large English dictionary database.
2. `lemmatizer.lemmatize(word, pos='v')` -- Find the base form of the word, assuming it is a verb. The `pos` parameter stands for part of speech (noun, verb, adjective, etc.).
3. Notice that the results depend on the part of speech. "running" as a verb becomes "run," but as a noun it stays "running."

> Analogy: Lemmatization is like pruning a tree with garden shears and a guidebook. It is slower, but every cut is precise and intentional.

2.6.3 Lemmatization with spaCy

spaCy performs lemmatization automatically when you process text:

```python
import spacy

nlp = spacy.load("en_core_web_sm")

text = "The children were running and playing in the studies"
doc = nlp(text)
```

```
for token in doc:
    print(f"  {token.text:12s} --> lemma: {token.lemma_:12s}  POS: {token.pos_}")
```

Output:

```
  The          --> lemma: the           POS: DET
  children     --> lemma: child         POS: NOUN
  were         --> lemma: be            POS: AUX
  running      --> lemma: run           POS: VERB
  and          --> lemma: and           POS: CCONJ
  playing      --> lemma: play          POS: VERB
  in           --> lemma: in            POS: ADP
  the          --> lemma: the           POS: DET
  studies      --> lemma: study         POS: NOUN
```

spaCy automatically detects the part of speech and applies the correct lemmatization. "children" becomes "child," "were" becomes "be," and "running" becomes "run."

Comparison Table

```
+-------------------------------------------------------------+
|         Stemming vs Lemmatization                           |
+-------------------------------------------------------------+
|                                                             |
|  Feature        Stemming          Lemmatization             |
|  ---------      --------          -------------             |
|  Speed           Fast              Slower                   |
|  Accuracy        Rough             Precise                  |
|  Real words?     Not always        Always                   |
|  Needs POS?      No                Yes (for best results)   |
|  Example:                                                   |
|  "studies"       "studi"           "study"                  |
|  "better"        "better"          "good"                   |
|                                                             |
|  Use stemming when: speed matters, search engines           |
|  Use lemmatization when: accuracy matters, chatbots         |
|                                                             |
+-------------------------------------------------------------+
```

2.7 Regular Expressions (Regex) for Text Cleaning

Regular expressions (regex) are patterns that describe text. They are incredibly powerful for finding and replacing text patterns.

> Analogy: A regex is like a search-and-replace feature on steroids. Instead of searching for exact text, you search for patterns. It is like telling someone, "Find me all phone numbers in this document" instead of "Find me 555-1234."

2.7.1 Regex Basics

```python
import re

# \d   matches any digit (0-9)
# \w   matches any letter, digit, or underscore
# \s   matches any whitespace (space, tab, newline)
# .    matches any character except newline
# *    means "zero or more of the previous character"
# +    means "one or more of the previous character"
# []   matches any character inside the brackets
# ^    at the start means "beginning of string"
# $    at the end means "end of string"

text = "Call me at 555-1234 or email john@example.com"

# Find all numbers
numbers = re.findall(r'\d+', text)
print(f"Numbers found: {numbers}")

# Find email addresses
emails = re.findall(r'\S+@\S+', text)
print(f"Emails found: {emails}")
```

Output:

```
Numbers found: ['555', '1234']
Emails found: ['john@example.com']
```

Line-by-line explanation:

1. `import re` -- Import Python's regular expression module.
2. `re.findall(r'\d+', text)` -- Find all sequences of one or more digits. `\d` matches a digit, `+` means "one or more."
3. `re.findall(r'\S+@\S+', text)` -- Find patterns that look like email addresses. `\S` matches any non-whitespace character.

2.7.2 Common Text Cleaning with Regex

```python
import re

def clean_text(text):
    """Clean text using regular expressions."""

    # Remove HTML tags
    text = re.sub(r'<[^>]+>', '', text)

    # Remove URLs
    text = re.sub(r'https?://\S+|www\.\S+', '', text)

    # Remove email addresses
    text = re.sub(r'\S+@\S+', '', text)

    # Remove numbers
    text = re.sub(r'\d+', '', text)

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Test with messy text
messy = """
<p>Check out https://example.com for details!</p>
Contact us at info@company.com or call 555-1234.
Visit <a href="link">our site</a> for 50% off!!!
"""

cleaned = clean_text(messy)
print(f"Original:\n{messy}")
print(f"Cleaned:\n{cleaned}")
```

Output:

```
Original:

<p>Check out https://example.com for details!</p>
Contact us at info@company.com or call 555-1234.
Visit <a href="link">our site</a> for 50% off!!!

Cleaned:
Check out for details! Contact us at or call -. Visit our site for % off!!!
```

Line-by-line explanation:

1. `re.sub(r'<[^>]+>', '', text)` -- Find anything between `<` and `>` (HTML tags) and replace with nothing. `[^>]+` means "one or more characters that are not `>`."
2. `re.sub(r'https?://\S+|www\.\S+', '', text)` -- Find URLs starting with "http://" or "https://" or "www." and remove them. The `?` after `s` makes it optional. The `|` means "or."

3. `re.sub(r'\S+@\S+', '', text)` -- Find and remove email addresses.
4. `re.sub(r'\d+', '', text)` -- Find and remove all sequences of digits.
5. `re.sub(r'\s+', ' ', text).strip()` -- Replace multiple whitespace characters with a single space.

2.7.3 Regex Quick Reference

```
+------------------------------------------------------------+
|           Regex Quick Reference                            |
+------------------------------------------------------------+
|                                                            |
|  Pattern    Meaning              Example                   |
|  -------    -------              -------                   |
|  \d         Any digit            "3" matches \d            |
|  \D         Any non-digit        "a" matches \D            |
|  \w         Letter/digit/_       "a" matches \w            |
|  \W         Non-word char        "!" matches \W            |
|  \s         Whitespace           " " matches \s            |
|  \S         Non-whitespace       "a" matches \S            |
|  .          Any character        "x" matches .             |
|  *          Zero or more         "ab*" matches "a", "abbb"|
|  +          One or more          "ab+" matches "ab","abbb"|
|  ?          Zero or one          "ab?" matches "a", "ab"  |
|  [abc]      Any of a, b, c      "b" matches [abc]        |
|  [^abc]     Not a, b, or c      "d" matches [^abc]       |
|  ^          Start of string      "^Hello" matches start   |
|  $          End of string        "end$" matches end       |
|                                                            |
+------------------------------------------------------------+
```

2.8 Complete Preprocessing Pipeline

Now let us put everything together into a complete, reusable preprocessing pipeline.

2.8.1 Pipeline with NLTK

```python
import re
import nltk
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
```

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

def preprocess_nltk(text):
    """Complete text preprocessing pipeline using NLTK."""

    # Step 1: Lowercase
    text = text.lower()

    # Step 2: Remove HTML tags
    text = re.sub(r'<[^>]+>', '', text)

    # Step 3: Remove URLs
    text = re.sub(r'https?://\S+|www\.\S+', '', text)

    # Step 4: Remove special characters (keep letters and spaces)
    text = re.sub(r'[^a-z\s]', '', text)

    # Step 5: Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # Step 6: Tokenize
    tokens = word_tokenize(text)

    # Step 7: Remove stop words
    stop_words = set(stopwords.words('english'))
    tokens = [t for t in tokens if t not in stop_words]

    # Step 8: Lemmatize
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(t, pos='v') for t in tokens]

    return tokens

# Test the pipeline
texts = [
    "I absolutely LOVED this movie!!! Best film ever <3",
    "The acting was terrible... worst $15 I've ever spent :(",
    "Check out https://example.com for more reviews!!! #movies",
]

for text in texts:
    result = preprocess_nltk(text)
    print(f"Original: {text}")
    print(f"Cleaned:  {result}")
    print()
```

Output:

```
Original: I absolutely LOVED this movie!!! Best film ever <3
Cleaned:  ['absolutely', 'love', 'movie', 'best', 'film', 'ever']
```

```
Original: The acting was terrible... worst $15 I've ever spent :(
Cleaned:  ['act', 'terrible', 'worst', 'ever', 'spend']

Original: Check out https://example.com for more reviews!!! #movies
Cleaned:  ['check', 'review', 'movie']
```

2.8.2 Pipeline with spaCy

```python
import re
import spacy

nlp = spacy.load("en_core_web_sm")

def preprocess_spacy(text):
    """Complete text preprocessing pipeline using spaCy."""

    # Step 1: Remove HTML tags
    text = re.sub(r'<[^>]+>', '', text)

    # Step 2: Remove URLs
    text = re.sub(r'https?://\S+|www\.\S+', '', text)

    # Step 3: Process with spaCy
    doc = nlp(text.lower())

    # Step 4: Lemmatize, remove stop words and punctuation
    tokens = [
        token.lemma_           # Use the lemma (base form)
        for token in doc
        if not token.is_stop   # Skip stop words
        and not token.is_punct # Skip punctuation
        and not token.is_space # Skip whitespace
        and len(token.text) > 1 # Skip single characters
    ]

    return tokens

# Test the pipeline
texts = [
    "I absolutely LOVED this movie!!! Best film ever <3",
    "The acting was terrible... worst $15 I've ever spent :(",
    "The children were running and playing happily in the garden",
]

for text in texts:
    result = preprocess_spacy(text)
    print(f"Original: {text}")
    print(f"Cleaned:  {result}")
    print()
```

Output:

```
Original: I absolutely LOVED this movie!!! Best film ever <3
Cleaned:  ['absolutely', 'love', 'movie', 'good', 'film']

Original: The acting was terrible... worst $15 I've ever spent :(
Cleaned:  ['acting', 'terrible', 'bad', '15', 'spend']

Original: The children were running and playing happily in the garden
Cleaned:  ['child', 'run', 'play', 'happily', 'garden']
```

Notice how spaCy's lemmatizer correctly converts "children" to "child" and "Best" to "good" -- it understands that "best" is the superlative form of "good."

2.8.3 Processing Multiple Documents

In real projects, you often need to preprocess many documents at once:

```python
import re
import spacy

nlp = spacy.load("en_core_web_sm")

def preprocess_batch(texts):
    """Preprocess a batch of texts efficiently with spaCy."""

    # Clean all texts first
    cleaned = []
    for text in texts:
        text = re.sub(r'<[^>]+>', '', text)
        text = re.sub(r'https?://\S+|www\.\S+', '', text)
        cleaned.append(text.lower())

    # Use spaCy's pipe for efficient batch processing
    # pipe() processes multiple texts much faster than one at a time
    results = []
    for doc in nlp.pipe(cleaned, batch_size=50):
        tokens = [
            token.lemma_
            for token in doc
            if not token.is_stop
            and not token.is_punct
            and not token.is_space
            and len(token.text) > 1
        ]
        results.append(tokens)

    return results

# Example with multiple reviews
reviews = [
```

```
    "Great product! Works perfectly.",
    "Terrible quality. Very disappointed.",
    "It's okay, nothing special really.",
    "Absolutely amazing! Best purchase ever!",
    "Broke after two days. Want my money back.",
]

processed = preprocess_batch(reviews)
for original, cleaned in zip(reviews, processed):
    print(f"Original: {original}")
    print(f"Cleaned:  {cleaned}")
    print()
```

Output:

```
Original: Great product! Works perfectly.
Cleaned:  ['great', 'product', 'work', 'perfectly']

Original: Terrible quality. Very disappointed.
Cleaned:  ['terrible', 'quality', 'disappointed']

Original: It's okay, nothing special really.
Cleaned:  ['okay', 'special', 'really']

Original: Absolutely amazing! Best purchase ever!
Cleaned:  ['absolutely', 'amazing', 'good', 'purchase']

Original: Broke after two days. Want my money back.
Cleaned:  ['break', 'day', 'want', 'money']
```

Line-by-line explanation:

1. `nlp.pipe(cleaned, batch_size=50)` -- Process texts in batches of 50 for efficiency. `pipe()` is much faster than calling `nlp()` on each text individually because it processes them in parallel.
2. The rest of the pipeline is the same as before -- lemmatize and filter tokens.

2.9 The Complete Preprocessing Pipeline Diagram

```
+-------------------------------------------------------------------+
|               Complete Text Preprocessing Pipeline                |
+-------------------------------------------------------------------+
|                                                                    |
|  Raw Text: "I LOVED this movie!!! <br> Check https://t.co/abc"    |
```

```
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 1. Remove HTML Tags  |  "I LOVED this movie!!! Check ..."      |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 2. Remove URLs       |  "I LOVED this movie!!!"                |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 3. Lowercase         |  "i loved this movie!!!"                |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 4. Remove Punctuation|  "i loved this movie"                   |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 5. Tokenize          |  ["i", "loved", "this", "movie"]        |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 6. Remove Stop Words |  ["loved", "movie"]                     |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  +---------------------+                                               |
|  | 7. Lemmatize         |  ["love", "movie"]                      |
|  +---------------------+                                               |
|     |                                                                  |
|     v                                                                  |
|  Clean Tokens: ["love", "movie"]                                       |
|                                                                        |
+------------------------------------------------------------------------+
```

Common Mistakes

1. Applying preprocessing steps in the wrong order -- Always remove HTML and URLs before tokenizing. If you tokenize first, HTML tags become garbage tokens.
2. Removing stop words for every task -- For sentiment analysis, "not" is critical. For topic modeling, stop words are noise. Think about your task before deciding.

3. Using stemming when you need real words -- If you need to show results to users, use lemmatization. Stemmed words like "studi" confuse people.
4. Forgetting to apply the same preprocessing to test data -- If you lowercase and lemmatize during training, you must do the exact same steps on new data at prediction time.
5. Over-preprocessing -- Removing too much can destroy useful signal. For example, removing all numbers loses information in reviews like "This laptop only lasted 2 days."

Best Practices

1. Document your preprocessing steps -- Write down exactly what you do and in what order. This makes your work reproducible.
2. Use spaCy for production, NLTK for learning -- spaCy is faster and designed for real applications. NLTK has more educational tools and flexibility.
3. Keep the original text -- Always store the raw text alongside your cleaned version. You might need to go back to it later.
4. Test on edge cases -- Try your pipeline on emojis, numbers, URLs, HTML, non-English characters, and empty strings. Make sure it handles them without crashing.
5. Consider your domain -- Medical text needs different preprocessing than social media text. Customize your pipeline for your specific use case.

Quick Summary

Text preprocessing transforms raw, messy text into clean, consistent data for NLP models. The key steps are: lowercasing (making everything lowercase), removing noise (HTML, URLs, punctuation), tokenization (splitting text into words or subwords), removing stop words (dropping common words like "the" and "is"), and stemming or lemmatization (reducing words to their base forms). Regular expressions are a powerful tool for pattern-based text cleaning. NLTK and spaCy are the two main Python libraries for text preprocessing, each with their own strengths.

Key Points

- Tokenization splits text into tokens (words, subwords, or sentences). NLTK, spaCy, and Hugging Face tokenizers each handle this differently.
- Lowercasing ensures "Apple" and "apple" are treated as the same word, but loses information about proper nouns.
- Punctuation removal reduces noise but should preserve contractions and domain-specific symbols when needed.
- Stop words are common words that add little meaning. Remove them for most tasks, but keep them when negation or phrases matter.
- Stemming is fast but crude -- it chops word endings and may produce non-words.
- Lemmatization is slower but accurate -- it uses a dictionary to find proper base forms.
- Regular expressions are pattern-matching tools essential for removing HTML, URLs, emails, and other noise.
- Always apply the same preprocessing steps to both training and test data.

Practice Questions

1. What is the difference between word tokenization and subword tokenization? When would you use each one?
2. Given the sentence "The children are NOT enjoying their studies," walk through each preprocessing step (lowercase, tokenize, remove stop words, lemmatize) and show the output at each stage. What potential problem do you see with removing stop words here?
3. Write a regular expression that matches phone numbers in the format "XXX-XXX-XXXX" where X is a digit.
4. Explain the difference between stemming and lemmatization using the words "better," "running," and "studies" as examples.
5. Why is the order of preprocessing steps important? What would go wrong if you tokenized before removing HTML tags?

Exercises

Exercise 1: Custom Preprocessing Pipeline

Write a function `custom_preprocess(text, options)` that accepts an `options` dictionary to control which steps are applied:

```python
options = {
    'lowercase': True,
    'remove_html': True,
    'remove_urls': True,
    'remove_numbers': False,
    'remove_punctuation': True,
    'remove_stopwords': True,
    'lemmatize': True
}
```

Test it on five different types of text: a tweet, an email, an HTML snippet, a product review, and a scientific abstract.

Exercise 2: Compare Preprocessing Methods

Take a set of 10 product reviews and preprocess them three different ways:

1. Only lowercase and tokenize
2. Lowercase, tokenize, and remove stop words
3. Full pipeline (lowercase, remove punctuation, tokenize, remove stop words, lemmatize)

For each method, count the total number of unique tokens. How much does each step reduce the vocabulary?

Exercise 3: Regex Challenge

Write regular expressions to extract the following from a block of text:

1. All email addresses
2. All dates in the format "MM/DD/YYYY"
3. All prices in the format "$XX.XX"
4. All hashtags (words starting with #)
5. All mentions (words starting with @)

Test your patterns on a paragraph that contains examples of each.

What Is Next?

Now that you know how to clean and prepare text, the next question is: how do we convert words into numbers that a machine learning model can understand? In Chapter 3: Text Representation -- Bag of Words and TF-IDF, you will learn two foundational methods for turning text into numerical vectors. These methods are simple, powerful, and still used widely in industry today.

Chapter 3: Text Representation -- Bag of Words and TF-IDF

What You Will Learn

- Why computers need numbers, not words
- How Bag of Words (BoW) converts text into number vectors by counting words
- How TF-IDF improves on BoW by weighting words based on importance
- How to use sklearn's CountVectorizer and TfidfVectorizer
- How to measure document similarity using cosine similarity
- The limitations of BoW and TF-IDF and when they fall short

Why This Chapter Matters

Machine learning models understand numbers, not words. You cannot feed the sentence "I love this movie" directly into a neural network or a logistic regression model. You must first convert it into a list of numbers -- a vector. This chapter teaches you two foundational ways to do this conversion: Bag of Words and TF-IDF. These methods are decades old, but they remain widely used in industry because they are simple, fast, and surprisingly effective. Every NLP practitioner should understand them before moving to more advanced representations like word embeddings.

3.1 Why Computers Need Numbers, Not Words

Let us make this concrete with a simple example:

```python
# This will NOT work -- you cannot do math on strings
sentence1 = "I love cats"
sentence2 = "I love dogs"

# How similar are these sentences?
# We cannot compute similarity on raw text
# sentence1 - sentence2 = ???  <-- Makes no sense
```

Computers do math. They add, multiply, and compare numbers. To process text, we must first convert every document into a numerical representation called a vector.

> Analogy: Imagine you are a judge at a cooking competition, but you cannot taste the food. Instead, each dish is described by numbers: spiciness level (1-10), sweetness level (1-10), and presentation score (1-10). You compare dishes using these numbers. Similarly, NLP converts text into numbers so algorithms can "compare" and "judge" documents.

```
+-------------------------------------------------------------+
|        Text to Numbers: The Core Challenge                  |
+-------------------------------------------------------------+
|                                                             |
|  "I love cats"  -->  ???  -->  [1, 1, 1, 0]                |
|  "I love dogs"  -->  ???  -->  [1, 1, 0, 1]                |
|                                                             |
|  Now we can compare:                                        |
|  Similarity([1,1,1,0], [1,1,0,1]) = 0.67                   |
|                                                             |
|  The computer can see: these sentences are similar!         |
|                                                             |
+-------------------------------------------------------------+
```

3.2 Bag of Words (BoW)

Bag of Words is the simplest way to convert text into numbers. The idea is straightforward: count how many times each word appears in a document.

3.2.1 The Concept

> Analogy: Imagine dumping all the words of a sentence into a bag, like Scrabble tiles. You shake the bag so the word order is lost. Then you count how many of each word tile you have. That count is your numerical representation.

```
Document: "the cat sat on the mat"

Step 1: Dump words into a bag (lose order):
        {the, cat, sat, on, the, mat}

Step 2: Count each unique word:
        the = 2, cat = 1, sat = 1, on = 1, mat = 1

Step 3: Create a vector:
        [2, 1, 1, 1, 1]
```

3.2.2 Building BoW by Hand

```python
from collections import Counter

documents = [
    "I love cats",
    "I love dogs",
    "cats and dogs are great",
]

# Step 1: Build vocabulary (all unique words)
vocabulary = set()
for doc in documents:
    words = doc.lower().split()
    vocabulary.update(words)

vocabulary = sorted(vocabulary)
print(f"Vocabulary: {vocabulary}")
print(f"Vocabulary size: {len(vocabulary)}")

# Step 2: Count word frequencies for each document
print("\nBag of Words vectors:")
for doc in documents:
    words = doc.lower().split()
    counts = Counter(words)
    # Create a vector: count of each vocabulary word
    vector = [counts.get(word, 0) for word in vocabulary]
    print(f"  '{doc}'")
    print(f"  Vector: {vector}")
    print()

# Show which number corresponds to which word
print("Index mapping:")
for i, word in enumerate(vocabulary):
    print(f"  Position {i}: '{word}'")
```

Output:

```
Vocabulary: ['and', 'are', 'cats', 'dogs', 'great', 'i', 'love']
Vocabulary size: 7

Bag of Words vectors:
  'I love cats'
  Vector: [0, 0, 1, 0, 0, 1, 1]

  'I love dogs'
  Vector: [0, 0, 0, 1, 0, 1, 1]

  'cats and dogs are great'
  Vector: [1, 1, 1, 1, 1, 0, 0]

Index mapping:
  Position 0: 'and'
  Position 1: 'are'
  Position 2: 'cats'
  Position 3: 'dogs'
  Position 4: 'great'
  Position 5: 'i'
  Position 6: 'love'
```

Line-by-line explanation:

1. We create a vocabulary -- the set of all unique words across all documents.
2. `Counter(words)` counts how many times each word appears in a document.
3. `counts.get(word, 0)` gets the count for each vocabulary word. If the word is not in the document, the count is 0.
4. The resulting vector has one number for each word in the vocabulary.

3.2.3 Using sklearn's CountVectorizer

In practice, we use sklearn's `CountVectorizer` which does all of this automatically:

```python
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "I love cats",
    "I love dogs",
    "cats and dogs are great",
]

# Create the vectorizer
vectorizer = CountVectorizer()

# Fit (learn vocabulary) and transform (create vectors)
bow_matrix = vectorizer.fit_transform(documents)
```

```
# Show the vocabulary
print("Vocabulary:")
print(f"  {vectorizer.get_feature_names_out()}")

# Show the BoW matrix
print("\nBag of Words matrix:")
print(bow_matrix.toarray())

# Show each document's vector with labels
print("\nDetailed view:")
feature_names = vectorizer.get_feature_names_out()
for i, doc in enumerate(documents):
    print(f"\n  Document: '{doc}'")
    vector = bow_matrix.toarray()[i]
    for word, count in zip(feature_names, vector):
        if count > 0:
            print(f"    {word}: {count}")
```

Output:

```
Vocabulary:
  ['and' 'are' 'cats' 'dogs' 'great' 'love']

Bag of Words matrix:
[[0 0 1 0 0 1]
 [0 0 0 1 0 1]
 [1 1 1 1 1 0]]

Detailed view:

  Document: 'I love cats'
    cats: 1
    love: 1

  Document: 'I love dogs'
    dogs: 1
    love: 1

  Document: 'cats and dogs are great'
    and: 1
    are: 1
    cats: 1
    dogs: 1
    great: 1
```

Line-by-line explanation:

1. `CountVectorizer()` -- Creates a vectorizer object. By default, it converts to lowercase and removes single-character words (which is why "I" is not in the vocabulary).
2. `vectorizer.fit_transform(documents)` -- Two steps in one: `fit` learns the vocabulary from the documents, and `transform` converts each document into a count vector.

3. `vectorizer.get_feature_names_out()` -- Returns the vocabulary (all unique words the vectorizer learned).
4. `bow_matrix.toarray()` -- Converts the sparse matrix to a regular array for display. sklearn uses sparse matrices (a memory-efficient format) because most values are 0.

3.2.4 CountVectorizer Options

```python
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "The quick brown fox jumps over the lazy dog",
    "The dog chased the fox around the park",
    "Quick brown foxes are very lazy animals",
]

# Option 1: Limit vocabulary size
vec1 = CountVectorizer(max_features=5)
result1 = vec1.fit_transform(documents)
print("Top 5 features only:")
print(f"  Words: {vec1.get_feature_names_out()}")
print(f"  Matrix:\n{result1.toarray()}")

# Option 2: Use n-grams (word pairs, triples, etc.)
vec2 = CountVectorizer(ngram_range=(1, 2))
result2 = vec2.fit_transform(documents)
print(f"\nWith bigrams (1-2 word combinations):")
print(f"  Features: {vec2.get_feature_names_out()}")

# Option 3: Remove common and rare words
vec3 = CountVectorizer(min_df=2, max_df=0.9)
result3 = vec3.fit_transform(documents)
print(f"\nFiltered vocabulary:")
print(f"  Words: {vec3.get_feature_names_out()}")
```

Output:

```
Top 5 features only:
  Words: ['brown' 'dog' 'fox' 'lazy' 'the']
  Matrix:
[[1 1 1 1 2]
 [0 1 1 0 3]
 [1 0 0 1 0]]

With bigrams (1-2 word combinations):
  Features: ['animals' 'are' 'are very' 'around' 'around the' 'brown' 'brown fox' 'brow...

Filtered vocabulary:
  Words: ['brown' 'dog' 'fox' 'lazy' 'the']
```

Key parameters explained:

- `max_features=5` -- Keep only the top 5 most frequent words.
- `ngram_range=(1, 2)` -- Include single words (unigrams) and word pairs (bigrams). For example, "brown fox" is a bigram. This captures some word order information.
- `min_df=2` -- A word must appear in at least 2 documents to be included. Removes very rare words.
- `max_df=0.9` -- A word must appear in at most 90% of documents. Removes very common words (similar to stop word removal).

```
+-----------------------------------------------------------+
|        N-grams: Capturing Word Order                      |
+-----------------------------------------------------------+
|                                                           |
|  Text: "New York is great"                                |
|                                                           |
|  Unigrams (n=1): "New", "York", "is", "great"            |
|  Bigrams  (n=2): "New York", "York is", "is great"       |
|  Trigrams (n=3): "New York is", "York is great"           |
|                                                           |
|  Unigrams lose "New York" as a phrase.                    |
|  Bigrams capture it!                                      |
|                                                           |
+-----------------------------------------------------------+
```

3.3 The Problem with Bag of Words

BoW has a major flaw: it treats all words equally. The word "the" and the word "revolutionary" get the same treatment, even though "the" appears in almost every document and tells us nothing, while "revolutionary" is rare and informative.

```python
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "the movie was the best movie the director ever made",
    "the food at the restaurant was the worst",
]

vectorizer = CountVectorizer()
bow = vectorizer.fit_transform(documents)

print("Vocabulary:", vectorizer.get_feature_names_out())
```

```
print("\nBoW matrix:")
print(bow.toarray())
print("\nNotice: 'the' has the highest counts (3 and 3)")
print("But 'the' tells us nothing about what the document is about!")
```

Output:

```
Vocabulary: ['at' 'best' 'director' 'ever' 'food' 'made' 'movie' 'restaurant' 'the' 'wa...

BoW matrix:
[[0 1 1 1 0 1 2 0 3 1 0]
 [1 0 0 0 1 0 0 1 3 1 1]]

Notice: 'the' has the highest counts (3 and 3)
But 'the' tells us nothing about what the document is about!
```

This is where TF-IDF comes in.

3.4 TF-IDF: Term Frequency-Inverse Document Frequency

TF-IDF solves the problem above by giving less weight to common words and more weight to rare, informative words.

3.4.1 The Two Parts

TF-IDF is made up of two measurements multiplied together:

TF (Term Frequency): How often a word appears in a specific document. More occurrences means higher TF.

```
TF(word, document) = (Number of times word appears in document)
                     / (Total number of words in document)
```

IDF (Inverse Document Frequency): How rare a word is across ALL documents. Rarer words get higher IDF.

```
IDF(word) = log(Total number of documents
               / Number of documents containing the word)
```

TF-IDF = TF x IDF

> Analogy: Imagine you are reading restaurant reviews. The word "food" appears in almost every review -- it is not helpful for distinguishing reviews. But the word "rancid" appears in only one review. "Rancid" is much more informative because it is rare. TF-IDF is like a scoring system that says: "Common words are boring. Rare words are interesting."

3.4.2 Calculating TF-IDF by Hand

```python
import math

documents = [
    "the cat sat on the mat",
    "the dog sat on the log",
    "cats and dogs are friends",
]

# Step 1: Calculate TF for "the" in document 1
doc1_words = documents[0].lower().split()
tf_the = doc1_words.count("the") / len(doc1_words)
print(f"TF('the', doc1) = {doc1_words.count('the')}/{len(doc1_words)} = {tf_the:.4f}")

# Step 2: Calculate IDF for "the" across all documents
total_docs = len(documents)
docs_with_the = sum(1 for doc in documents if "the" in doc.lower())
idf_the = math.log(total_docs / docs_with_the)
print(f"IDF('the') = log({total_docs}/{docs_with_the}) = {idf_the:.4f}")

# Step 3: TF-IDF
tfidf_the = tf_the * idf_the
print(f"TF-IDF('the', doc1) = {tf_the:.4f} * {idf_the:.4f} = {tfidf_the:.4f}")

print("\n--- Now for a rare word: 'friends' ---")

# TF for "friends" in document 3
doc3_words = documents[2].lower().split()
tf_friends = doc3_words.count("friends") / len(doc3_words)
print(f"TF('friends', doc3) = {doc3_words.count('friends')}/{len(doc3_words)} = {tf_fri...

# IDF for "friends"
docs_with_friends = sum(1 for doc in documents if "friends" in doc.lower())
idf_friends = math.log(total_docs / docs_with_friends)
print(f"IDF('friends') = log({total_docs}/{docs_with_friends}) = {idf_friends:.4f}")

# TF-IDF
tfidf_friends = tf_friends * idf_friends
print(f"TF-IDF('friends', doc3) = {tf_friends:.4f} * {idf_friends:.4f} = {tfidf_friends...

print(f"\n'friends' TF-IDF ({tfidf_friends:.4f}) >> 'the' TF-IDF ({tfidf_the:.4f})")
print("Rare words get higher scores!")
```

Output:

```
TF('the', doc1) = 2/6 = 0.3333
IDF('the') = log(3/2) = 0.4055
TF-IDF('the', doc1) = 0.3333 * 0.4055 = 0.1352

--- Now for a rare word: 'friends' ---
TF('friends', doc3) = 1/5 = 0.2000
IDF('friends') = log(3/1) = 1.0986
TF-IDF('friends', doc3) = 0.2000 * 1.0986 = 0.2197

'friends' TF-IDF (0.2197) >> 'the' TF-IDF (0.1352)
Rare words get higher scores!
```

3.4.3 Visual Explanation

```
+-------------------------------------------------------------+
|           TF-IDF: How It Works                              |
+-------------------------------------------------------------+
|                                                             |
|  Word: "the"                                                |
|  TF:  High (appears often in the document)                  |
|  IDF: Low  (appears in almost ALL documents)                |
|  TF-IDF = High * Low = LOW SCORE                           |
|  --> "the" is not important                                 |
|                                                             |
|  Word: "revolutionary"                                      |
|  TF:  Medium (appears a few times)                          |
|  IDF: High (appears in very FEW documents)                  |
|  TF-IDF = Medium * High = HIGH SCORE                       |
|  --> "revolutionary" is important!                          |
|                                                             |
+-------------------------------------------------------------+
```

3.4.4 Using sklearn's TfidfVectorizer

```python
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "the cat sat on the mat",
    "the dog sat on the log",
    "cats and dogs are friends",
]

# Create TF-IDF vectorizer
vectorizer = TfidfVectorizer()
```

```
# Fit and transform
tfidf_matrix = vectorizer.fit_transform(documents)

# Show vocabulary
feature_names = vectorizer.get_feature_names_out()
print("Vocabulary:", feature_names)

# Show TF-IDF matrix (rounded for readability)
import numpy as np
print("\nTF-IDF matrix (rounded to 2 decimals):")
print(np.round(tfidf_matrix.toarray(), 2))

# Show scores for each document
print("\nDetailed scores:")
for i, doc in enumerate(documents):
    print(f"\n  Document: '{doc}'")
    vector = tfidf_matrix.toarray()[i]
    # Sort by score (highest first)
    scored_words = sorted(
        zip(feature_names, vector),
        key=lambda x: x[1],
        reverse=True
    )
    for word, score in scored_words:
        if score > 0:
            print(f"    {word:12s}: {score:.4f}")
```

Output:

```
Vocabulary: ['and' 'are' 'cat' 'cats' 'dog' 'dogs' 'friends' 'log' 'mat' 'on' 'sat' 'the']

TF-IDF matrix (rounded to 2 decimals):
[[0.   0.   0.47 0.   0.   0.   0.   0.   0.47 0.3  0.3  0.61]
 [0.   0.   0.   0.   0.47 0.   0.   0.47 0.   0.3  0.3  0.61]
 [0.45 0.45 0.   0.45 0.   0.45 0.45 0.   0.   0.   0.   0.  ]]

Detailed scores:

  Document: 'the cat sat on the mat'
    the         : 0.6133
    cat         : 0.4681
    mat         : 0.4681
    on          : 0.2985
    sat         : 0.2985

  Document: 'the dog sat on the log'
    the         : 0.6133
    dog         : 0.4681
    log         : 0.4681
    on          : 0.2985
    sat         : 0.2985
```

```
Document: 'cats and dogs are friends'
  and         : 0.4472
  are         : 0.4472
  cats        : 0.4472
  dogs        : 0.4472
  friends     : 0.4472
```

Line-by-line explanation:

1. `TfidfVectorizer()` -- Creates a TF-IDF vectorizer. It automatically handles tokenization, counting, and TF-IDF calculation.
2. `vectorizer.fit_transform(documents)` -- Learns the vocabulary and computes TF-IDF scores for all documents.
3. The output shows that common words shared across documents (like "on" and "sat") get lower scores, while unique words (like "cat," "mat," "log") get higher scores.

> Note: sklearn's TF-IDF formula is slightly different from the textbook formula. It uses smoothed IDF and L2 normalization. The concept is the same -- common words get lower scores.

3.5 Document Similarity with Cosine Similarity

Now that we have numerical vectors for our documents, we can measure how similar they are using cosine similarity.

3.5.1 What Is Cosine Similarity?

Cosine similarity measures the angle between two vectors. If two vectors point in the same direction, they are similar (cosine similarity = 1). If they point in perpendicular directions, they are unrelated (cosine similarity = 0).

```
+-------------------------------------------------------------+
|        Cosine Similarity Visualized                         |
+-------------------------------------------------------------+
|                                                             |
|  Similar documents (small angle):                           |
|                                                             |
|           B                                                 |
|          /                                                  |
|         / ) small angle = HIGH similarity                   |
|        /                                                    |
```

```
|       A                                                    |
|                                                            |
|  Different documents (large angle):                        |
|                                                            |
|       B                                                    |
|       |                                                    |
|       | ) large angle = LOW similarity                     |
|       |                                                    |
|       -------A                                             |
|                                                            |
|  Cosine Similarity ranges from 0 to 1:                     |
|    1.0 = identical direction (very similar)                |
|    0.0 = perpendicular (completely different)              |
|                                                            |
+------------------------------------------------------------+
```

3.5.2 Computing Cosine Similarity

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "I love machine learning",
    "I love deep learning",
    "The weather is sunny today",
    "Machine learning is a subset of artificial intelligence",
]

# Create TF-IDF vectors
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

# Compute cosine similarity between all pairs
similarity_matrix = cosine_similarity(tfidf_matrix)

# Display results
print("Documents:")
for i, doc in enumerate(documents):
    print(f"  D{i}: {doc}")

print("\nCosine Similarity Matrix:")
print("        D0      D1      D2      D3")
for i, row in enumerate(similarity_matrix):
    scores = "  ".join(f"{score:.4f}" for score in row)
    print(f"  D{i}  {scores}")
```

Output:

```
Documents:
  D0: I love machine learning
  D1: I love deep learning
  D2: The weather is sunny today
  D3: Machine learning is a subset of artificial intelligence

Cosine Similarity Matrix:
        D0      D1      D2      D3
  D0  1.0000  0.5765  0.0000  0.2480
  D1  0.5765  1.0000  0.0000  0.1228
  D2  0.0000  0.0000  1.0000  0.0880
  D3  0.2480  0.1228  0.0880  1.0000
```

Interpretation:

- D0 and D1 are most similar (0.5765) -- both talk about "love" and "learning."
- D0 and D2 have zero similarity -- they share no words at all.
- D0 and D3 are somewhat similar (0.2480) -- they share "machine" and "learning."
- Every document is perfectly similar to itself (1.0000).

3.5.3 Finding the Most Similar Document

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Our document collection (like a mini search engine)
documents = [
    "Python is a great programming language",
    "Machine learning uses algorithms to learn from data",
    "Deep learning is a subset of machine learning",
    "Python has many libraries for data science",
    "The weather today is warm and sunny",
]

# Build TF-IDF vectors for all documents
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

# A new query
query = "Python for machine learning"
query_vector = vectorizer.transform([query])

# Compute similarity between query and all documents
similarities = cosine_similarity(query_vector, tfidf_matrix)[0]

# Rank documents by similarity
print(f"Query: '{query}'\n")
```

```
print("Results (ranked by similarity):")
ranked_indices = np.argsort(similarities)[::-1]
for rank, idx in enumerate(ranked_indices, 1):
    print(f"  {rank}. (score: {similarities[idx]:.4f}) {documents[idx]}")
```

Output:

```
Query: 'Python for machine learning'

Results (ranked by similarity):
  1. (score: 0.4649) Machine learning uses algorithms to learn from data
  2. (score: 0.3731) Python is a great programming language
  3. (score: 0.3473) Deep learning is a subset of machine learning
  4. (score: 0.2897) Python has many libraries for data science
  5. (score: 0.0000) The weather today is warm and sunny
```

Line-by-line explanation:

1. `vectorizer.transform([query])` -- Note we use `transform` (not `fit_transform`) because we want to use the same vocabulary learned from our documents. If we used `fit_transform`, it would create a new vocabulary based only on the query.
2. `cosine_similarity(query_vector, tfidf_matrix)[0]` -- Compute similarity between the query and every document. `[0]` extracts the first (and only) row of results.
3. `np.argsort(similarities)[::-1]` -- Sort document indices by similarity score in descending order.

This is essentially how basic search engines work!

3.6 Comparing BoW and TF-IDF

Let us compare both methods on the same documents:

```python
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
import numpy as np

documents = [
    "the the the cat sat",
    "the dog chased the cat",
    "a bird flew over the rainbow",
]
```

```
# Bag of Words
bow_vec = CountVectorizer()
bow_matrix = bow_vec.fit_transform(documents)

# TF-IDF
tfidf_vec = TfidfVectorizer()
tfidf_matrix = tfidf_vec.fit_transform(documents)

print("=== Bag of Words ===")
print(f"Features: {bow_vec.get_feature_names_out()}")
print(f"Matrix:\n{bow_matrix.toarray()}")

print("\n=== TF-IDF ===")
print(f"Features: {tfidf_vec.get_feature_names_out()}")
print(f"Matrix:\n{np.round(tfidf_matrix.toarray(), 3)}")

print("\nKey difference:")
print("  BoW: 'the' in doc 1 has count 3 (highest value)")
print("  TF-IDF: 'cat' and 'sat' in doc 1 have higher scores than 'the'")
print("  TF-IDF correctly downweights the common word 'the'")
```

Output:

```
=== Bag of Words ===
Features: ['bird' 'cat' 'chased' 'dog' 'flew' 'over' 'rainbow' 'sat' 'the']
Matrix:
[[0 1 0 0 0 0 0 1 3]
 [0 1 1 1 0 0 0 0 2]
 [1 0 0 0 1 1 1 0 1]]

=== TF-IDF ===
Features: ['bird' 'cat' 'chased' 'dog' 'flew' 'over' 'rainbow' 'sat' 'the']
Matrix:
[[0.    0.318 0.    0.    0.    0.    0.    0.497 0.808]
 [0.    0.315 0.492 0.492 0.    0.    0.    0.    0.529]
 [0.423 0.    0.    0.    0.423 0.423 0.423 0.    0.36 ]]

Key difference:
  BoW: 'the' in doc 1 has count 3 (highest value)
  TF-IDF: 'cat' and 'sat' in doc 1 have higher scores than 'the'
  TF-IDF correctly downweights the common word 'the'
```

3.7 Limitations of BoW and TF-IDF

Despite their usefulness, these methods have significant limitations:

3.7.1 No Word Order

```python
from sklearn.feature_extraction.text import CountVectorizer

# These sentences have very different meanings
# but identical BoW vectors!
docs = [
    "the dog bit the man",
    "the man bit the dog",
]

vectorizer = CountVectorizer()
bow = vectorizer.fit_transform(docs)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("Vectors:")
print(bow.toarray())
print("\nBoth vectors are IDENTICAL!")
print("BoW cannot tell that these sentences mean different things.")
```

Output:

```
Vocabulary: ['bit' 'dog' 'man' 'the']
Vectors:
[[1 1 1 2]
 [1 1 1 2]]

Both vectors are IDENTICAL!
BoW cannot tell that these sentences mean different things.
```

3.7.2 No Semantic Understanding

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# These sentences mean the same thing but use different words
docs = [
    "The movie was excellent",    # positive
    "The film was wonderful",     # positive (same meaning!)
    "The movie was terrible",     # negative
]

vectorizer = TfidfVectorizer()
tfidf = vectorizer.fit_transform(docs)

sim_01 = cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0]
sim_02 = cosine_similarity(tfidf[0:1], tfidf[2:3])[0][0]
```

```
print(f"'excellent movie' vs 'wonderful film': {sim_01:.4f}")
print(f"'excellent movie' vs 'terrible movie': {sim_02:.4f}")
print(f"\nProblem: 'terrible movie' is MORE similar than 'wonderful film'!")
print("TF-IDF matches words, not meanings.")
```

Output:

```
'excellent movie' vs 'wonderful film': 0.1218
'excellent movie' vs 'terrible movie': 0.4640

Problem: 'terrible movie' is MORE similar than 'wonderful film'!
TF-IDF matches words, not meanings.
```

This is because "excellent movie" and "terrible movie" share the word "movie," while "excellent movie" and "wonderful film" share no words at all. BoW and TF-IDF do not understand that "excellent" and "wonderful" are synonyms, or that "movie" and "film" mean the same thing.

3.7.3 High Dimensionality

```python
from sklearn.feature_extraction.text import CountVectorizer

# Even a small collection creates a large vocabulary
documents = [
    "Natural language processing enables computers to understand text",
    "Machine learning algorithms can classify documents automatically",
    "Deep neural networks have revolutionized artificial intelligence",
    "Text classification is a fundamental NLP task",
    "Sentiment analysis determines whether text is positive or negative",
]

vectorizer = CountVectorizer()
bow = vectorizer.fit_transform(documents)

print(f"Number of documents: {bow.shape[0]}")
print(f"Vocabulary size: {bow.shape[1]}")
print(f"Matrix size: {bow.shape[0]} x {bow.shape[1]} = {bow.shape[0] * bow.shape[1]} ce...
print(f"Non-zero cells: {bow.nnz}")
print(f"Sparsity: {(1 - bow.nnz / (bow.shape[0] * bow.shape[1])) * 100:.1f}%")
print("\nWith real datasets (millions of documents), the vocabulary")
print("can be hundreds of thousands of words. Most vectors are mostly zeros.")
```

Output:

```
Number of documents: 5
Vocabulary size: 29
Matrix size: 5 x 29 = 145 cells
Non-zero cells: 34
Sparsity: 76.6%
```

Summary of Limitations

```
+-------------------------------------------------------------+
|        Limitations of BoW and TF-IDF                        |
+-------------------------------------------------------------+
|                                                             |
|  1. NO WORD ORDER                                           |
|     "dog bit man" == "man bit dog" (same vector!)           |
|                                                             |
|  2. NO SEMANTIC UNDERSTANDING                               |
|     "excellent" and "wonderful" are treated as              |
|     completely unrelated words                              |
|                                                             |
|  3. HIGH DIMENSIONALITY                                     |
|     Large vocabularies create sparse, high-dimensional      |
|     vectors that waste memory                               |
|                                                             |
|  4. NO CONTEXT                                              |
|     "bank" always gets the same representation              |
|     whether it means river bank or money bank               |
|                                                             |
|  Solution: Word Embeddings (Chapter 4)                      |
|                                                             |
+-------------------------------------------------------------+
```

Common Mistakes

1. Using `fit_transform` on test data -- Always use `fit_transform` on training data and `transform` on test/new data. If you use `fit_transform` on test data, it creates a different vocabulary and your model will not work.
2. Ignoring the sparsity issue -- BoW and TF-IDF vectors are mostly zeros. Use sklearn's sparse matrices instead of converting to dense arrays (with `.toarray()`) unless you need to for display purposes.
3. Not setting `max_features` -- With large datasets, the vocabulary can grow to hundreds of thousands of words. Use `max_features` to limit it to a manageable size.

4. Forgetting to preprocess first -- If you do not lowercase and clean your text before vectorizing, "Great" and "great" become separate features. CountVectorizer does lowercase by default, but be aware of this.
5. Assuming high cosine similarity means documents are about the same topic -- Two documents might share common words without being semantically related.

Best Practices

1. Use TF-IDF over raw BoW for most tasks -- TF-IDF almost always outperforms raw word counts because it downweights common, uninformative words.
2. Experiment with n-grams -- Adding bigrams (`ngram_range=(1, 2)`) often improves performance by capturing some word order.
3. Set `min_df` and `max_df` -- Remove very rare words (`min_df=2` or `min_df=5`) and very common words (`max_df=0.95`) to keep a clean vocabulary.
4. Use `max_features` for large datasets -- Limiting to the top 10,000 or 50,000 features keeps your vectors manageable without losing much signal.
5. Always split data before fitting the vectorizer -- Fit the vectorizer on training data only. Transform test data using the same fitted vectorizer.

Quick Summary

Bag of Words (BoW) converts text into numerical vectors by counting how many times each word appears. TF-IDF improves on this by giving higher scores to rare, informative words and lower scores to common words. Both methods use sklearn's CountVectorizer and TfidfVectorizer. Cosine similarity measures how similar two document vectors are. While BoW and TF-IDF are simple and effective for many tasks, they have important limitations: they ignore word order, do not understand word meaning, and create high-dimensional sparse vectors.

Key Points

- Computers need numbers, not words. Vectorization is the process of converting text to numbers.
- Bag of Words counts word occurrences. Simple but treats all words equally.
- TF-IDF weighs words by their importance: rare words get high scores, common words get low scores.
- TF (Term Frequency) measures how often a word appears in a document.
- IDF (Inverse Document Frequency) measures how rare a word is across all documents.
- CountVectorizer creates BoW vectors; TfidfVectorizer creates TF-IDF vectors.
- Cosine similarity measures how similar two vectors are, from 0 (unrelated) to 1 (identical).
- BoW and TF-IDF ignore word order, do not understand synonyms, and create sparse vectors.
- Always use `fit_transform` on training data and `transform` on new/test data.

Practice Questions

1. Explain in your own words why TF-IDF gives the word "the" a low score. What does "Inverse Document Frequency" mean intuitively?
2. Given the documents "I like cats" and "I like dogs," draw the Bag of Words representation. What is the vocabulary? What is each document's vector?
3. Why does cosine similarity show that "the movie was terrible" is more similar to "the movie was excellent" than "the film was wonderful" is? What fundamental limitation does this reveal?
4. What is the difference between `fit_transform` and `transform`? Why is it important to use the right one on test data?
5. How do bigrams help address one of BoW's limitations? Give an example where unigrams would fail but bigrams would succeed.

Exercises

Exercise 1: Build a Simple Search Engine

Create a basic search engine using TF-IDF and cosine similarity:

1. Define a collection of at least 10 documents (use short paragraphs about different topics).
2. Build a TF-IDF matrix from the collection.
3. Write a function `search(query, n=3)` that takes a search query and returns the top `n` most similar documents.
4. Test it with at least 5 different queries.

Exercise 2: Compare BoW, TF-IDF, and N-grams

Using a collection of at least 20 product reviews:

1. Create BoW vectors (unigrams only)
2. Create TF-IDF vectors (unigrams only)
3. Create TF-IDF vectors with bigrams (`ngram_range=(1, 2)`)
4. For each method, find the top 10 most important features. Which method produces the most meaningful features?

Exercise 3: Document Clustering

Using TF-IDF vectors and sklearn's KMeans:

1. Create 15 short documents about 3 different topics (5 each about sports, technology, and cooking).
2. Convert them to TF-IDF vectors.
3. Apply KMeans clustering with `n_clusters=3`.
4. Check if the clusters align with the topics. Print each cluster's documents and top TF-IDF terms.

What Is Next?

In this chapter, you learned that BoW and TF-IDF convert text into numbers by counting words. But you also saw their biggest weakness: they treat every word as independent and cannot understand that "excellent" and "wonderful" mean similar things. In Chapter 4: Word Embeddings, you will discover a revolutionary approach that represents words as dense vectors in a mathematical space where similar words are close together. This idea -- that meaning can be captured in geometry -- is one of the most important breakthroughs in NLP history.

Chapter 4: Word Embeddings

What You Will Learn

- Why Bag of Words and TF-IDF fail to capture word meaning
- What word embeddings are and why they are revolutionary
- How Word2Vec works (Skip-gram and CBOW, explained simply)
- What GloVe is and how it differs from Word2Vec
- How to use pre-trained word embeddings with gensim
- How to visualize word embeddings in 2D using t-SNE
- How to perform word analogy tasks (king - man + woman = queen)

Why This Chapter Matters

In the previous chapter, you learned that BoW and TF-IDF treat every word as an independent, unrelated entity. The word "happy" and the word "joyful" are as different as "happy" and "volcano." This is obviously wrong -- "happy" and "joyful" are almost identical in meaning. Word embeddings fix this by placing words in a mathematical space where similar words are close together. This single idea -- representing meaning as geometry -- transformed NLP and laid the groundwork for modern language models like BERT and GPT. If you understand word embeddings, you understand the foundation that the entire field is built on.

4.1 Why BoW and TF-IDF Fail

Let us revisit the problem from Chapter 3:

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "The dog ran quickly",      # About a fast dog
    "The canine sprinted fast", # Same meaning, different words!
    "The cat slept quietly",    # Different meaning
]

vectorizer = TfidfVectorizer()
tfidf = vectorizer.fit_transform(docs)

sim_01 = cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0]
sim_02 = cosine_similarity(tfidf[0:1], tfidf[2:3])[0][0]

print(f"'dog ran quickly' vs 'canine sprinted fast': {sim_01:.4f}")
print(f"'dog ran quickly' vs 'cat slept quietly':    {sim_02:.4f}")
```

Output:

```
'dog ran quickly' vs 'canine sprinted fast': 0.0000
'dog ran quickly' vs 'cat slept quietly':    0.1345
```

The first two sentences mean the same thing, but TF-IDF gives them ZERO similarity because they share no words. Meanwhile, the third sentence (with a completely different meaning) scores higher because it shares the word "the." This is fundamentally broken.

The Three Big Problems

```
+-------------------------------------------------------------+
|    Why BoW/TF-IDF Fails at Meaning                          |
+-------------------------------------------------------------+
|                                                             |
|  Problem 1: NO SYNONYMS                                     |
|  "happy" and "joyful" are treated as unrelated              |
|                                                             |
|  Problem 2: NO RELATIONSHIPS                                |
|  "king" and "queen" are as different as "king" and "car"    |
|                                                             |
|  Problem 3: HUGE VECTORS                                    |
|  A vocabulary of 100,000 words means vectors of             |
|  length 100,000, mostly filled with zeros                   |
|                                                             |
|  Solution: Word Embeddings!                                 |
|  Represent each word as a SHORT, DENSE vector               |
|  where similar words have similar vectors                   |
|                                                             |
+-------------------------------------------------------------+
```

4.2 What Are Word Embeddings?

A word embedding is a way to represent a word as a list of numbers (a vector), typically with 100 to 300 numbers. The key insight is that these numbers are learned from data so that words with similar meanings end up with similar vectors.

> Analogy: Imagine a city map where every restaurant has a location (latitude, longitude). Italian restaurants cluster together in Little Italy. Chinese restaurants cluster together in Chinatown. Sushi places are near each other. You do not need to read the menu to know a restaurant serves Italian food -- you just look at where it sits on the map. Word embeddings are like a map for words. Similar words are close together, and you can tell what a word means by looking at its neighbors.

BoW Vector vs Word Embedding Vector

```
+-----------------------------------------------------------+
|     BoW Vector vs Word Embedding                          |
+-----------------------------------------------------------+
|                                                           |
|  BoW Vector for "cat" (vocabulary of 10,000 words):       |
|  [0, 0, 0, ..., 1, ..., 0, 0, 0]                         |
|  Length: 10,000 (mostly zeros) -- SPARSE                  |
|                                                           |
|  Word Embedding for "cat" (300 dimensions):               |
|  [0.23, -0.41, 0.87, 0.15, -0.62, 0.33, ...]             |
|  Length: 300 (no zeros, every number matters) -- DENSE     |
|                                                           |
|  Key Difference:                                          |
|  BoW:       cat = [0,0,0,1,0,0,0,0,0,0]  (one-hot)       |
|  Embedding: cat = [0.23, -0.41, 0.87, ...]  (meaningful)  |
|                                                           |
|  In embeddings, "cat" and "kitten" have SIMILAR vectors   |
|  because they appear in similar contexts                  |
|                                                           |
+-----------------------------------------------------------+
```

The Distributional Hypothesis

Word embeddings are based on a simple but powerful idea called the distributional hypothesis:

> "You shall know a word by the company it keeps." -- J.R. Firth, 1957

This means: words that appear in similar contexts have similar meanings.

```
Context examples:

"I adopted a cute _____ from the shelter"
"The _____ chased the mouse"
"I fed my _____ this morning"

Words that fit: cat, dog, kitten, puppy

These words appear in similar contexts,
so they must have similar meanings!
```

4.3 Word2Vec: Words as Vectors

Word2Vec is an algorithm developed by researchers at Google in 2013 that learns word embeddings from large amounts of text. It has two variants:

4.3.1 Skip-gram: Predict Context from a Word

Given a center word, predict the words around it.

```
+-------------------------------------------------------------+
|        Skip-gram: Predict context from center word          |
+-------------------------------------------------------------+
|                                                             |
|  Sentence: "The cat sat on the mat"                         |
|                                                             |
|  Window size = 2 (look 2 words left and right)              |
|                                                             |
|  Center word: "sat"                                         |
|  Context words: "The", "cat", "on", "the"                   |
|                                                             |
|  Training examples:                                         |
|  Input: "sat" --> Predict: "The"                            |
|  Input: "sat" --> Predict: "cat"                            |
|  Input: "sat" --> Predict: "on"                             |
|  Input: "sat" --> Predict: "the"                            |
|                                                             |
|  The neural network learns to predict context words.        |
|  The hidden layer weights BECOME the word embeddings.       |
|                                                             |
+-------------------------------------------------------------+
```

4.3.2 CBOW: Predict a Word from Context

CBOW (Continuous Bag of Words) works in the opposite direction: given the context words, predict the center word.

```
+-----------------------------------------------------------+
|        CBOW: Predict center word from context             |
+-----------------------------------------------------------+
|                                                           |
|  Sentence: "The cat sat on the mat"                       |
|                                                           |
|  Context words: "The", "cat", "on", "the"                 |
|  Center word: "sat"                                       |
|                                                           |
|  Training:                                                |
|  Input: ["The", "cat", "on", "the"] --> Predict: "sat"    |
|                                                           |
+-----------------------------------------------------------+
```

4.3.3 How Training Works (Simplified)

```
+-----------------------------------------------------------+
|        Word2Vec Training (Simplified)                     |
+-----------------------------------------------------------+
|                                                           |
|  Step 1: Start with random vectors for each word          |
|                                                           |
|    "cat"  = [0.1, -0.3, 0.5, ...]  (random)              |
|    "dog"  = [0.8, 0.2, -0.1, ...]   (random)             |
|    "sat"  = [-0.4, 0.7, 0.3, ...]   (random)             |
|                                                           |
|  Step 2: Read millions of sentences                       |
|                                                           |
|    "The cat sat on the mat"                               |
|    "The dog sat on the rug"                               |
|    "A cat chased a mouse"                                 |
|    "A dog chased a ball"                                  |
|    ... millions more ...                                  |
|                                                           |
|  Step 3: Adjust vectors so context predictions work       |
|                                                           |
|    "cat" and "dog" appear in similar contexts             |
|    --> their vectors move closer together                 |
|                                                           |
|    "cat" and "airplane" appear in different contexts      |
|    --> their vectors stay far apart                       |
|                                                           |
|  Step 4: After training, similar words have similar       |
|          vectors automatically!                           |
|                                                           |
```

```
|    "cat"  = [0.62, -0.15, 0.48, ...]                      |
|    "dog"  = [0.58, -0.22, 0.51, ...]  (close to cat!)    |
|    "car"  = [-0.31, 0.78, -0.12, ...] (far from cat)     |
|                                                           |
+-----------------------------------------------------------+
```

> Analogy: Imagine you are new at a company and you do not know anyone. After a year, you notice that Alice and Bob always eat lunch together, attend the same meetings, and work on the same projects. Even without being told, you can guess they work in the same department. Word2Vec does the same thing -- it watches which words "hang out together" and groups them accordingly.

4.4 GloVe: Global Vectors for Word Representation

GloVe (Global Vectors) is another popular word embedding method developed at Stanford in 2014. While Word2Vec learns from local context windows, GloVe learns from global word co-occurrence statistics.

How GloVe Differs from Word2Vec

```
+-----------------------------------------------------------+
|        Word2Vec vs GloVe                                  |
+-----------------------------------------------------------+
|                                                           |
|  Word2Vec:                                                |
|  - Reads text one window at a time                        |
|  - Predicts context words (local)                         |
|  - Like reading a book word by word                       |
|                                                           |
|  GloVe:                                                   |
|  - First counts how often every pair of words             |
|    appears together across the ENTIRE corpus              |
|  - Then learns vectors from these counts (global)         |
|  - Like first making a frequency table, then learning     |
|                                                           |
|  Both produce similar quality embeddings.                 |
|  In practice, the choice often does not matter much.      |
|                                                           |
+-----------------------------------------------------------+
```

GloVe creates a large co-occurrence matrix -- a table that counts how often each pair of words appears near each other. Then it factorizes this matrix to produce the word vectors.

```
Co-occurrence matrix (simplified):

           ice    steam   solid   gas    water
ice        -       0       3      0       5
steam      0       -       0      4       4
solid      3       0       -      0       1
gas        0       4       0      -       1
water      5       4       1      1       -

GloVe learns vectors so that:
- "ice" and "solid" are close (high co-occurrence)
- "steam" and "gas" are close (high co-occurrence)
- "ice" and "gas" are far apart (low co-occurrence)
```

4.5 Using Pre-trained Embeddings with Gensim

Training word embeddings from scratch requires massive amounts of text and computing power. Fortunately, you can download pre-trained embeddings that others have already trained on billions of words.

Gensim is a Python library specifically designed for working with word embeddings.

4.5.1 Loading Pre-trained Word2Vec

```python
import gensim.downloader as api

# Download pre-trained Word2Vec (trained on Google News, 3 billion words)
# This model has 300-dimensional vectors for 3 million words
# Note: First download is ~1.7 GB and takes a few minutes
print("Loading pre-trained Word2Vec model...")
model = api.load("word2vec-google-news-300")
print("Model loaded!")

# Get the vector for a word
vector = model['computer']
print(f"\nVector for 'computer':")
print(f"  Dimensions: {len(vector)}")
print(f"  First 10 values: {vector[:10]}")
```

Output:

```
Loading pre-trained Word2Vec model...
Model loaded!
```

```
Vector for 'computer':
  Dimensions: 300
  First 10 values: [ 0.07617188  0.01416016 -0.14160156  0.18554688 -0.08398438
  -0.01367188  0.09960938  0.12695312 -0.09765625  0.04296875]
```

4.5.2 Finding Similar Words

```python
import gensim.downloader as api

model = api.load("word2vec-google-news-300")

# Find words most similar to "king"
similar_words = model.most_similar("king", topn=10)
print("Words most similar to 'king':")
for word, score in similar_words:
    print(f"  {word:20s} {score:.4f}")
```

Output:

```
Words most similar to 'king':
  kings                0.7138
  queen                0.6511
  monarch              0.6413
  crown_prince         0.6204
  prince               0.6160
  sultan               0.5866
  ruler                0.5797
  Queen_Consort        0.5367
  queens               0.5311
  princess             0.5153
```

The model correctly identifies that "kings," "queen," "monarch," and "prince" are similar to "king" -- all related to royalty.

4.5.3 Word Similarity Scores

```python
import gensim.downloader as api

model = api.load("word2vec-google-news-300")

# Compare word pairs
pairs = [
    ("cat", "dog"),        # Both animals
```

```
    ("cat", "kitten"),     # Very related
    ("cat", "car"),        # Unrelated
    ("happy", "joyful"),   # Synonyms
    ("happy", "sad"),      # Antonyms
    ("king", "queen"),     # Related concepts
    ("paris", "france"),   # Capital and country
]

print("Word Similarity Scores:")
print(f"{'Word 1':15s} {'Word 2':15s} {'Similarity':>10s}")
print("-" * 42)
for w1, w2 in pairs:
    try:
        sim = model.similarity(w1, w2)
        print(f"{w1:15s} {w2:15s} {sim:10.4f}")
    except KeyError as e:
        print(f"{w1:15s} {w2:15s} {'Not found':>10s}")
```

Output:

```
Word Similarity Scores:
Word 1          Word 2           Similarity
------------------------------------------
cat             dog                  0.7609
cat             kitten               0.7840
cat             car                  0.1164
happy           joyful               0.5715
happy           sad                  0.3848
king            queen                0.6511
paris           france               0.4543
```

Interpretation:

- "cat" and "kitten" (0.78) are more similar than "cat" and "dog" (0.76), which makes sense.
- "cat" and "car" (0.12) are barely similar -- they just happen to share letters.
- "happy" and "joyful" (0.57) are recognized as synonyms.
- "happy" and "sad" (0.38) have some similarity because they both relate to emotions, even though they are opposites.

4.5.4 Using a Smaller Model (GloVe)

If you want a faster download, use the smaller GloVe model:

```python
import gensim.downloader as api

# GloVe model (smaller, faster to download)
```

```
# 50-dimensional vectors trained on Wikipedia + Gigaword
model = api.load("glove-wiki-gigaword-50")

# Find similar words
print("Words similar to 'python':")
for word, score in model.most_similar("python", topn=5):
    print(f"  {word:15s} {score:.4f}")

print("\nWords similar to 'university':")
for word, score in model.most_similar("university", topn=5):
    print(f"  {word:15s} {score:.4f}")
```

Output:

```
Words similar to 'python':
  perl            0.7873
  snake           0.6928
  scripting       0.6623
  java            0.6519
  ruby            0.6408

Words similar to 'university':
  college         0.9218
  school          0.8032
  campus          0.7927
  graduate        0.7596
  universities    0.7566
```

Notice how "python" finds both programming languages (perl, java, ruby) and snake-related words -- it captures multiple meanings.

4.6 Word Analogies

One of the most impressive properties of word embeddings is their ability to solve analogies using simple vector arithmetic.

4.6.1 The King - Man + Woman = Queen Example

The most famous example:

```
king - man + woman = ???

In vector space:
```

```
vector("king") - vector("man") + vector("woman") ≈ vector("queen")
```

> Analogy: Think of it this way. If you take the concept of "king" and remove the "maleness" from it, then add "femaleness," you get "queen." The embedding vectors capture these abstract concepts as directions in space.

```
+-----------------------------------------------------------+
|        Word Analogy in Vector Space                       |
+-----------------------------------------------------------+
|                                                           |
|             queen *                                       |
|            /     |                                        |
|           /      | "royalty"                              |
|          /       |    direction                           |
|    woman *-------*                                        |
|         |       king                                      |
|         |      /                                          |
|  "gender"|    /                                           |
| direction|   /                                            |
|         |  /                                              |
|     man *                                                 |
|                                                           |
|  king - man = "royalty" direction                         |
|  "royalty" direction + woman = queen                      |
|                                                           |
+-----------------------------------------------------------+
```

4.6.2 Code Example

```python
import gensim.downloader as api

model = api.load("word2vec-google-news-300")

# king - man + woman = ?
result = model.most_similar(
    positive=["king", "woman"],
    negative=["man"],
    topn=3
)

print("king - man + woman = ?")
for word, score in result:
    print(f"  {word:15s} {score:.4f}")
```

Output:

```
king - man + woman = ?
  queen           0.7118
  monarch         0.6189
  princess         0.5902
```

4.6.3 More Analogy Examples

```python
import gensim.downloader as api

model = api.load("word2vec-google-news-300")

analogies = [
    # (word1, word2, word3) --> word4
    # word1 is to word2 as word3 is to ???
    ("Paris", "France", "Tokyo", "Japan"),
    ("big", "bigger", "small", "smaller"),
    ("man", "doctor", "woman", "doctor/nurse"),
    ("walk", "walked", "run", "ran"),
]

print("Word Analogies:")
print("=" * 60)
for w1, w2, w3, expected in analogies:
    try:
        result = model.most_similar(
            positive=[w2, w3],
            negative=[w1],
            topn=3
        )
        print(f"\n{w1} is to {w2} as {w3} is to ???")
        print(f"Expected: {expected}")
        print("Got:")
        for word, score in result:
            print(f"  {word:20s} {score:.4f}")
    except KeyError as e:
        print(f"Word not found: {e}")
```

Output:

```
Word Analogies:
============================================================

Paris is to France as Tokyo is to ???
Expected: Japan
Got:
  Japan                0.6697
  Japanese             0.5970
  Osaka                0.5387
```

```
big is to bigger as small is to ???
Expected: smaller
Got:
  smaller              0.6852
  larger               0.6478
  Bigger               0.5847

man is to doctor as woman is to ???
Expected: doctor/nurse
Got:
  physician            0.5895
  gynecologist         0.5627
  nurse                0.5490

walk is to walked as run is to ???
Expected: ran
Got:
  ran                  0.6095
  running              0.5359
  walks                0.5044
```

Line-by-line explanation:

1. `model.most_similar(positive=[w2, w3], negative=[w1], topn=3)` -- This computes `w2 - w1 + w3` and finds the closest words to the result.
2. `positive` means "add these vectors" and `negative` means "subtract these vectors."
3. The analogy `Paris:France :: Tokyo:???` becomes `France - Paris + Tokyo = ???`, and the answer is Japan.

4.7 Visualizing Embeddings with t-SNE

Word embeddings typically have 100-300 dimensions. We cannot visualize that many dimensions, but we can use t-SNE (t-distributed Stochastic Neighbor Embedding) to compress them down to 2 dimensions for plotting.

> Analogy: Imagine you have a 3D globe with cities marked on it. You want to draw a flat 2D map. You will lose some accuracy (distances will be slightly off), but the general relationships are preserved -- cities that are close on the globe will be close on the map. t-SNE does this for high-dimensional word vectors.

```python
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.manifold import TSNE
import gensim.downloader as api

# Load a pre-trained model
model = api.load("glove-wiki-gigaword-50")

# Choose words from different categories
words = {
    'Animals': ['cat', 'dog', 'horse', 'fish', 'bird', 'mouse'],
    'Countries': ['france', 'germany', 'japan', 'china', 'brazil', 'india'],
    'Colors': ['red', 'blue', 'green', 'yellow', 'purple', 'orange'],
    'Numbers': ['one', 'two', 'three', 'four', 'five', 'six'],
}

# Get vectors for all words
all_words = []
all_vectors = []
all_categories = []
colors_map = {
    'Animals': 'red',
    'Countries': 'blue',
    'Colors': 'green',
    'Numbers': 'purple'
}

for category, word_list in words.items():
    for word in word_list:
        if word in model:
            all_words.append(word)
            all_vectors.append(model[word])
            all_categories.append(category)

vectors_array = np.array(all_vectors)

# Apply t-SNE to reduce to 2 dimensions
tsne = TSNE(n_components=2, random_state=42, perplexity=5)
vectors_2d = tsne.fit_transform(vectors_array)

# Plot
plt.figure(figsize=(12, 8))
for category in words.keys():
    indices = [i for i, c in enumerate(all_categories) if c == category]
    x = vectors_2d[indices, 0]
    y = vectors_2d[indices, 1]
    plt.scatter(x, y, c=colors_map[category], label=category, s=100)

    for i in indices:
        plt.annotate(
            all_words[i],
            (vectors_2d[i, 0], vectors_2d[i, 1]),
            fontsize=10,
            ha='center',
            va='bottom'
        )
```

```
plt.title("Word Embeddings Visualized with t-SNE")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("word_embeddings_tsne.png", dpi=100)
plt.show()
print("Plot saved as 'word_embeddings_tsne.png'")
```

Expected behavior: The plot will show four distinct clusters. Animal words cluster together, country words cluster together, color words cluster together, and number words cluster together. This visually proves that word embeddings capture meaning.

Line-by-line explanation:

1. We select words from four categories to see if embeddings group them correctly.
2. `model[word]` retrieves the pre-trained vector for each word.
3. `TSNE(n_components=2)` reduces 50-dimensional vectors to 2 dimensions.
4. `tsne.fit_transform(vectors_array)` applies the dimensionality reduction.
5. We plot each category in a different color to see the clustering.

```
+-------------------------------------------------------------+
|        What the t-SNE Plot Looks Like                       |
+-------------------------------------------------------------+
|                                                             |
|         * france                                            |
|     * germany   * japan                                     |
|       * india  * china                                      |
|          * brazil                                           |
|                                                             |
|                        * one                                |
|                    * two  * three                           |
|                      * four * five                          |
|    * cat                * six                               |
|  * dog  * horse                                             |
|    * mouse  * bird                                          |
|      * fish            * red                                |
|                      * blue * green                         |
|                        * yellow * purple                    |
|                          * orange                           |
|                                                             |
|  Similar words cluster together automatically!              |
|                                                             |
+-------------------------------------------------------------+
```

4.8 Using Word Embeddings in Practice

4.8.1 Computing Document Vectors

To represent an entire document, you can average the word vectors of all its words:

```python
import numpy as np
import gensim.downloader as api
from sklearn.metrics.pairwise import cosine_similarity

model = api.load("glove-wiki-gigaword-50")

def document_vector(text, model):
    """Compute the average word vector for a document."""
    words = text.lower().split()
    vectors = []
    for word in words:
        if word in model:
            vectors.append(model[word])

    if len(vectors) == 0:
        return np.zeros(model.vector_size)

    return np.mean(vectors, axis=0)

# Compare documents
docs = [
    "The cat chased the mouse",
    "A kitten ran after a small rodent",
    "The stock market crashed today",
]

print("Document Similarity using Word Embeddings:")
print("=" * 50)

# Compute vectors for all documents
doc_vectors = np.array([document_vector(doc, model) for doc in docs])

# Compare all pairs
for i in range(len(docs)):
    for j in range(i + 1, len(docs)):
        sim = cosine_similarity(
            doc_vectors[i].reshape(1, -1),
            doc_vectors[j].reshape(1, -1)
        )[0][0]
        print(f"\nDoc {i}: '{docs[i]}'")
        print(f"Doc {j}: '{docs[j]}'")
        print(f"Similarity: {sim:.4f}")
```

Output:

```
Document Similarity using Word Embeddings:
==================================================

Doc 0: 'The cat chased the mouse'
Doc 1: 'A kitten ran after a small rodent'
Similarity: 0.8924

Doc 0: 'The cat chased the mouse'
Doc 2: 'The stock market crashed today'
Similarity: 0.5013

Doc 1: 'A kitten ran after a small rodent'
Doc 2: 'The stock market crashed today'
Similarity: 0.3618
```

Now "The cat chased the mouse" and "A kitten ran after a small rodent" are correctly identified as highly similar (0.89), even though they share almost no words. This is a massive improvement over TF-IDF.

4.8.2 Finding Odd One Out

```python
import gensim.downloader as api

model = api.load("word2vec-google-news-300")

# Which word does not belong?
groups = [
    ["cat", "dog", "horse", "computer"],
    ["happy", "joyful", "glad", "table"],
    ["paris", "london", "berlin", "banana"],
]

print("Odd One Out:")
for group in groups:
    odd = model.doesnt_match(group)
    print(f"  {group} --> '{odd}' does not belong")
```

Output:

```
Odd One Out:
  ['cat', 'dog', 'horse', 'computer'] --> 'computer' does not belong
  ['happy', 'joyful', 'glad', 'table'] --> 'table' does not belong
  ['paris', 'london', 'berlin', 'banana'] --> 'banana' does not belong
```

4.9 Word Embeddings with PyTorch

In deep learning models, you use an embedding layer that learns embeddings during training:

```python
import torch
import torch.nn as nn

# Create an embedding layer
# 10000 = vocabulary size (number of unique words)
# 100 = embedding dimension (length of each vector)
embedding_layer = nn.Embedding(num_embeddings=10000, embedding_dim=100)

# Look up embeddings for some word indices
# Suppose: 0=the, 1=cat, 2=sat, 3=on, 4=mat
word_indices = torch.tensor([1, 2, 3])  # "cat sat on"

# Get embeddings
embedded = embedding_layer(word_indices)

print(f"Input shape:  {word_indices.shape}")
print(f"Output shape: {embedded.shape}")
print(f"\nEmbedding for word index 1 ('cat'):")
print(f"  First 10 values: {embedded[0][:10].detach().numpy()}")
```

Output:

```
Input shape:  torch.Size([3])
Output shape: torch.Size([3, 100])

Embedding for word index 1 ('cat'):
  First 10 values: [-0.5246  1.3406 -0.1488  0.7630  0.9472
  -0.2115  1.0423  0.1535  0.7291 -0.3845]
```

Line-by-line explanation:

1. `nn.Embedding(num_embeddings=10000, embedding_dim=100)` -- Creates a lookup table of 10,000 words, each represented by a 100-dimensional vector. Initially, these vectors are random.
2. `word_indices = torch.tensor([1, 2, 3])` -- Words are represented by their index in the vocabulary. Index 1 might be "cat," index 2 might be "sat," etc.
3. `embedding_layer(word_indices)` -- Looks up the vector for each word index. Input is 3 word indices, output is 3 vectors of 100 dimensions each.
4. During training, these embedding vectors are updated by backpropagation, just like other neural network weights.

Loading Pre-trained Embeddings into PyTorch

```python
import torch
import torch.nn as nn
import numpy as np
import gensim.downloader as api

# Load pre-trained GloVe
model = api.load("glove-wiki-gigaword-50")

# Create vocabulary from your data
vocab = ["cat", "dog", "bird", "fish", "car", "bus"]

# Build embedding matrix
embedding_dim = 50
vocab_size = len(vocab)
embedding_matrix = np.zeros((vocab_size, embedding_dim))

for i, word in enumerate(vocab):
    if word in model:
        embedding_matrix[i] = model[word]
    else:
        # Random vector for unknown words
        embedding_matrix[i] = np.random.normal(0, 0.1, embedding_dim)

# Create PyTorch embedding layer with pre-trained weights
embedding_layer = nn.Embedding.from_pretrained(
    torch.FloatTensor(embedding_matrix),
    freeze=False  # Allow fine-tuning during training
)

print(f"Embedding matrix shape: {embedding_matrix.shape}")
print(f"Embedding layer weight shape: {embedding_layer.weight.shape}")

# Verify: cat (index 0) and dog (index 1) should be similar
cat_vec = embedding_layer(torch.tensor(0))
dog_vec = embedding_layer(torch.tensor(1))
car_vec = embedding_layer(torch.tensor(4))

cat_dog_sim = torch.cosine_similarity(cat_vec.unsqueeze(0), dog_vec.unsqueeze(0))
cat_car_sim = torch.cosine_similarity(cat_vec.unsqueeze(0), car_vec.unsqueeze(0))

print(f"\nSimilarity (cat, dog): {cat_dog_sim.item():.4f}")
print(f"Similarity (cat, car): {cat_car_sim.item():.4f}")
```

Output:

```
Embedding matrix shape: (6, 50)
Embedding layer weight shape: torch.Size([6, 50])

Similarity (cat, dog): 0.9218
Similarity (cat, car): 0.4517
```

Common Mistakes

1. Training embeddings on too little data -- Word2Vec needs millions of sentences to learn good embeddings. For small datasets, use pre-trained embeddings instead.
2. Comparing embeddings from different models -- A vector from Word2Vec and a vector from GloVe live in different spaces. Never mix them.
3. Ignoring out-of-vocabulary (OOV) words -- Pre-trained models do not have vectors for every word. Always check if a word exists in the model before accessing its vector.
4. Using embeddings without fine-tuning -- Pre-trained embeddings are good starting points, but fine-tuning them on your specific task usually improves performance.
5. Overinterpreting individual dimensions -- Unlike BoW where each dimension corresponds to a word, individual dimensions of word embeddings do not have clear interpretations.

Best Practices

1. Use pre-trained embeddings as a starting point -- GloVe (glove-wiki-gigaword-50 or glove-wiki-gigaword-300) and Word2Vec (word2vec-google-news-300) are good defaults.
2. Choose the right dimensionality -- 50 dimensions for small projects, 100-300 for most tasks. More dimensions can capture more nuance but need more data.
3. Handle unknown words gracefully -- Use the average of all word vectors, or a special random vector, for words not in your vocabulary.
4. Use averaging for simple tasks, weighted averaging for better results -- TF-IDF-weighted word vectors often outperform simple averaging for document representation.
5. Visualize your embeddings -- Use t-SNE to check that your embeddings make sense. If "cat" and "banana" are close together, something is wrong.

Quick Summary

Word embeddings represent words as dense vectors where similar words have similar vectors. Word2Vec learns embeddings by predicting context words (Skip-gram) or center

words (CBOW) from large text corpora. GloVe learns from global word co-occurrence statistics. Pre-trained embeddings can be loaded using gensim and used immediately for tasks like finding similar words, solving analogies, and computing document similarity. The t-SNE algorithm can compress high-dimensional embeddings to 2D for visualization. In PyTorch, the nn.Embedding layer creates trainable embedding vectors that can be initialized with pre-trained weights.

Key Points

- Word embeddings represent words as dense vectors (100-300 numbers) where meaning is encoded in the numbers.
- Distributional hypothesis: Words in similar contexts have similar meanings and therefore similar vectors.
- Word2Vec has two architectures: Skip-gram (predict context from center word) and CBOW (predict center word from context).
- GloVe learns from global co-occurrence statistics rather than local context windows.
- Pre-trained embeddings (gensim) let you use high-quality vectors without training from scratch.
- Word embeddings can solve analogies: king - man + woman = queen.
- t-SNE reduces high-dimensional embeddings to 2D for visualization.
- In PyTorch, `nn.Embedding` creates an embedding lookup table that can be initialized with pre-trained weights.
- Average word vectors to represent entire documents.

Practice Questions

1. Why do word embeddings represent "cat" and "kitten" as similar vectors but "cat" and "car" as different vectors? What principle makes this work?
2. Explain the difference between Skip-gram and CBOW in Word2Vec. Which one is generally considered better for rare words and why?
3. The analogy "king - man + woman = queen" works because of a mathematical property of word vectors. Explain in simple terms what this means geometrically.
4. What is the difference between Word2Vec and GloVe? When might you choose one over the other?

5. Why is t-SNE useful for word embeddings? What information might be lost when reducing from 300 dimensions to 2 dimensions?

Exercises

Exercise 1: Explore Word Relationships

Using gensim with a pre-trained model:

1. Find the 10 most similar words to "computer," "doctor," "university," and "music."
2. Try at least 5 word analogies of your own (e.g., "man is to king as woman is to ???").
3. Find 3 analogies that work well and 3 that fail. Explain why some might fail.

Exercise 2: Build a Synonym Finder

Write a function `find_synonyms(word, n=5)` that:

1. Takes a word and returns its `n` most similar words from a pre-trained model.
2. Filters out words that are just different forms of the same word (e.g., "running" should not be a synonym of "run").
3. Test it on 10 different words across different categories (emotions, animals, verbs, adjectives).

Exercise 3: Document Similarity with Embeddings vs TF-IDF

Create a set of 10 document pairs where each pair consists of sentences with the same meaning but different words (paraphrases). Compare:

1. TF-IDF cosine similarity
2. Average word embedding cosine similarity

Which method correctly identifies more paraphrases? By how much?

What Is Next?

Now that you understand how to represent words and documents as numbers -- from simple counts (BoW) to intelligent vectors (embeddings) -- you are ready to build real NLP

applications. In Chapter 5: Text Classification, you will combine preprocessing, text representation, and machine learning to build systems that can automatically categorize text. You will build a spam detector, a sentiment analyzer, and learn how to evaluate these systems properly.

Chapter 5: Text Classification

What You Will Learn

- What text classification is and why it is one of the most important NLP tasks
- How to combine TF-IDF with traditional machine learning models (Logistic Regression, Naive Bayes, SVM)
- How to use sklearn's Pipeline to build clean, reproducible text classifiers
- How to evaluate text classifiers with accuracy, precision, recall, and F1-score
- How to build a complete spam detection system from scratch
- How to build a complete sentiment analysis system from scratch

Why This Chapter Matters

Text classification is the bread and butter of applied NLP. Every time Gmail sorts your email into "Primary" or "Spam," every time a company analyzes whether customer reviews are positive or negative, every time a news website automatically tags articles by topic -- that is text classification. It is often the first NLP task companies implement because it delivers immediate business value. In this chapter, you will build two complete, real-world systems: a spam detector and a sentiment analyzer. These are not toy examples -- they follow the same patterns used in production.

5.1 What Is Text Classification?

Text classification (also called text categorization) is the task of assigning a category or label to a piece of text.

```
+-----------------------------------------------------------+
|           Text Classification Examples                    |
+-----------------------------------------------------------+
|                                                           |
|  Input Text                    --> Category               |
|  ------------------------          --------               |
|  "You won a free iPhone!"     --> SPAM                    |
|  "Meeting at 3pm tomorrow"    --> NOT SPAM                |
|                                                           |
|  "I love this product!"       --> POSITIVE                |
|  "Worst purchase ever"        --> NEGATIVE                |
|                                                           |
|  "Lakers beat Celtics 110-98" --> SPORTS                  |
|  "New AI model breaks record" --> TECHNOLOGY              |
|                                                           |
|  "How do I reset my password?"--> ACCOUNT HELP            |
|  "Package not delivered yet"  --> SHIPPING                |
|                                                           |
+-----------------------------------------------------------+
```

> Analogy: Text classification is like hiring a sorting clerk at a post office. The clerk reads each letter and puts it in the right bin -- bills in one bin, personal letters in another, junk mail in the trash. We are training a computer to be that clerk.

The Text Classification Pipeline

```
+-------------------------------------------------------------------+
|            Text Classification Pipeline                           |
+-------------------------------------------------------------------+
|                                                                   |
|  "I love this movie!"                                             |
|     |                                                             |
|     v                                                             |
|  +---------------------+                                          |
|  | 1. PREPROCESS        |  Lowercase, clean, tokenize              |
|  +---------------------+                                          |
|     |                                                             |
|     v                                                             |
|  +---------------------+                                          |
|  | 2. VECTORIZE         |  TF-IDF converts text to numbers         |
|  +---------------------+                                          |
|     |                                                             |
|     v                                                             |
|  +---------------------+                                          |
|  | 3. CLASSIFY          |  ML model predicts the label             |
|  +---------------------+                                          |
|     |                                                             |
|     v                                                             |
|  Label: POSITIVE (confidence: 0.94)                               |
|                                                                   |
```

```
+-----------------------------------------------------------------+
```

5.2 The Three Classic ML Models for Text

5.2.1 Naive Bayes

Naive Bayes is a simple, fast classifier based on probability. It calculates the probability that a document belongs to each category and picks the most probable one.

> Analogy: Imagine you are at a fruit stand. Someone hands you a round, orange, fist-sized object. Based on your experience, you know: 90% of round-orange-fist-sized objects are oranges, 8% are tangerines, 2% are something else. So you classify it as an orange. Naive Bayes does the same thing with words -- it calculates the probability of each category based on the words in the text.

Why "naive"? Because it assumes all words are independent of each other. The presence of "great" does not affect the probability of seeing "movie." This is obviously false (a review saying "great" is likely about something), but the assumption works surprisingly well in practice.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

# Training data
texts = [
    "I love this movie",
    "Great film, highly recommend",
    "Terrible movie, waste of time",
    "Awful acting, boring plot",
    "Amazing performance, loved it",
    "Worst movie I have ever seen",
]
labels = ["positive", "positive", "negative", "negative", "positive", "negative"]

# Create a pipeline: TF-IDF + Naive Bayes
model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('classifier', MultinomialNB()),
])

# Train the model
```

```
model.fit(texts, labels)

# Predict on new text
test_texts = ["This film was great", "Horrible movie"]
predictions = model.predict(test_texts)

for text, pred in zip(test_texts, predictions):
    print(f"  '{text}' --> {pred}")
```

Output:

```
  'This film was great' --> positive
  'Horrible movie' --> negative
```

5.2.2 Logistic Regression

Logistic Regression learns weights for each word that indicate how much that word pushes toward each category. It is often the best starting point for text classification.

> Analogy: Imagine each word is a voter, and each voter has a "vote strength." The word "excellent" might have a +5 vote for positive. The word "terrible" might have a -5 vote. Logistic Regression adds up all the votes and decides: if the total is positive, the document is positive; if negative, it is negative.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Same training data
texts = [
    "I love this movie",
    "Great film, highly recommend",
    "Terrible movie, waste of time",
    "Awful acting, boring plot",
    "Amazing performance, loved it",
    "Worst movie I have ever seen",
]
labels = ["positive", "positive", "negative", "negative", "positive", "negative"]

# Create pipeline: TF-IDF + Logistic Regression
model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('classifier', LogisticRegression()),
])

# Train
```

```
model.fit(texts, labels)

# Predict with probabilities
test_texts = ["This film was great", "Horrible movie", "It was okay"]
for text in test_texts:
    prediction = model.predict([text])[0]
    probabilities = model.predict_proba([text])[0]
    class_names = model.classes_
    print(f"  '{text}'")
    print(f"    Prediction: {prediction}")
    for cls, prob in zip(class_names, probabilities):
        print(f"    P({cls}): {prob:.4f}")
    print()
```

Output:

```
  'This film was great'
    Prediction: positive
    P(negative): 0.2847
    P(positive): 0.7153

  'Horrible movie'
    Prediction: negative
    P(negative): 0.7306
    P(positive): 0.2694

  'It was okay'
    Prediction: negative
    P(negative): 0.5120
    P(positive): 0.4880
```

Notice how "It was okay" is almost 50/50 -- the model is uncertain because it is a neutral statement.

5.2.3 Support Vector Machine (SVM)

Support Vector Machine (SVM) finds the best boundary (hyperplane) that separates different categories with the widest possible margin.

> Analogy: Imagine two groups of students standing on a field -- math students on the left and art students on the right. An SVM draws a line between them that maximizes the distance to the nearest students on each side. This line is the decision boundary, and the nearest students are the "support vectors."

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
```

```
from sklearn.pipeline import Pipeline

texts = [
    "I love this movie",
    "Great film, highly recommend",
    "Terrible movie, waste of time",
    "Awful acting, boring plot",
    "Amazing performance, loved it",
    "Worst movie I have ever seen",
]
labels = ["positive", "positive", "negative", "negative", "positive", "negative"]

# Create pipeline: TF-IDF + SVM
model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('classifier', LinearSVC()),
])

model.fit(texts, labels)

test_texts = ["This film was great", "Horrible movie", "It was okay"]
predictions = model.predict(test_texts)

for text, pred in zip(test_texts, predictions):
    print(f"  '{text}' --> {pred}")
```

Output:

```
  'This film was great' --> positive
  'Horrible movie' --> negative
  'It was okay' --> positive
```

Comparison of the Three Models

```
+-------------------------------------------------------------+
|    Comparing Text Classification Models                     |
+-------------------------------------------------------------+
|                                                             |
|  Model              Strengths          Weaknesses           |
|  -----              ---------          ----------           |
|  Naive Bayes        Very fast          Assumes word         |
|                     Good with small    independence         |
|                     data               (oversimplified)     |
|                                                             |
|  Logistic           Good all-around    Slower than NB       |
|  Regression         Gives probability  on very large        |
|                     scores             datasets             |
|                                                             |
|  SVM                Best accuracy      No probability       |
```

```
|                     for many text      scores (by         |
|                     problems           default)           |
|                                                           |
|  Recommendation: Start with Logistic Regression.          |
|  Try Naive Bayes if speed matters.                        |
|  Try SVM if accuracy is critical.                         |
|                                                           |
+-----------------------------------------------------------+
```

5.3 sklearn Pipeline for Text Classification

The sklearn Pipeline chains multiple steps together into a single object. This is essential for text classification because you always need at least two steps: vectorization and classification.

5.3.1 Why Pipelines Matter

Without a pipeline, you have to manually keep track of multiple objects and call them in the right order. This leads to bugs:

```python
# WITHOUT pipeline (error-prone):
# Must remember to call transform (not fit_transform) on test data
# Must keep vectorizer and model in sync

# WITH pipeline (clean and safe):
# One object does everything
# Cannot accidentally fit on test data
```

5.3.2 Building a Pipeline

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Sample dataset
texts = [
    "I absolutely love this product",
    "Best purchase I ever made",
```

```
    "Works great, highly recommend",
    "This is amazing quality",
    "Wonderful experience overall",
    "Very satisfied with this item",
    "Terrible quality, broke in a day",
    "Worst product ever, do not buy",
    "Complete waste of money",
    "Very disappointed with this",
    "Awful experience, returning it",
    "Poor quality, falls apart easily",
]
labels = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
# 1 = positive, 0 = negative

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=42
)

# Create the pipeline
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=5000,       # Keep top 5000 words
        ngram_range=(1, 2),      # Use unigrams and bigrams
        stop_words='english',    # Remove English stop words
    )),
    ('classifier', LogisticRegression(
        max_iter=1000,           # Allow enough iterations
        random_state=42,
    )),
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Predict on test data
predictions = pipeline.predict(X_test)

# Evaluate
print("Classification Report:")
print(classification_report(
    y_test,
    predictions,
    target_names=["Negative", "Positive"]
))
```

Output:

```
Classification Report:
              precision    recall  f1-score   support

    Negative       1.00      1.00      1.00         2
    Positive       1.00      1.00      1.00         1
```

```
    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

Line-by-line explanation:

1. `Pipeline([...])` -- Creates a pipeline with named steps. Each step is a tuple of (name, object).
2. `TfidfVectorizer(max_features=5000, ngram_range=(1, 2), stop_words='english')` -- Converts text to TF-IDF vectors with at most 5000 features, including both single words and word pairs.
3. `LogisticRegression(max_iter=1000)` -- The classifier. `max_iter=1000` gives it enough iterations to converge.
4. `pipeline.fit(X_train, y_train)` -- Trains both the vectorizer and the classifier in one step.
5. `pipeline.predict(X_test)` -- Vectorizes and classifies test data in one step.
6. `classification_report` -- Shows precision, recall, and F1-score for each class.

5.4 Evaluation Metrics for Text Classification

Understanding how to measure your model's performance is critical. Here are the key metrics:

5.4.1 The Confusion Matrix

```
+-------------------------------------------------------------+
|           Confusion Matrix Explained                        |
+-------------------------------------------------------------+
|                                                             |
|                    Predicted                                |
|                 Positive  Negative                          |
|  Actual  +------+--------+---------+                        |
|  Positive| TP   |  True  | False   |                        |
|          |      | Pos.   | Neg.    |                        |
|          +------+--------+---------+                        |
|  Negative| FP   | False  | True    |                        |
|          |      | Pos.   | Neg.    |                        |
|          +------+--------+---------+                        |
|                                                             |
|  TP: Correctly predicted positive (GOOD)                    |
|  TN: Correctly predicted negative (GOOD)                    |
```

```
|  FP: Incorrectly predicted positive (BAD - false alarm)   |
|  FN: Incorrectly predicted negative (BAD - missed it)     |
|                                                           |
+-----------------------------------------------------------+
```

5.4.2 Precision, Recall, and F1-Score

```python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

# Simulated predictions
y_true = [1, 1, 1, 1, 0, 0, 0, 0, 1, 0]
y_pred = [1, 1, 0, 1, 0, 1, 0, 0, 1, 0]
# Labels: 1 = spam, 0 = not spam

# Calculate metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

print("Evaluation Metrics:")
print(f"  Accuracy:  {accuracy:.4f}")
print(f"  Precision: {precision:.4f}")
print(f"  Recall:    {recall:.4f}")
print(f"  F1-Score:  {f1:.4f}")

print(f"\nConfusion Matrix:")
cm = confusion_matrix(y_true, y_pred)
print(f"  {cm}")

print(f"\nDetailed Report:")
print(classification_report(
    y_true, y_pred,
    target_names=["Not Spam", "Spam"]
))
```

Output:

```
Evaluation Metrics:
  Accuracy:  0.8000
  Precision: 0.8000
  Recall:    0.8000
  F1-Score:  0.8000

Confusion Matrix:
  [[4 1]
```

```
   [1 4]]

Detailed Report:
              precision    recall  f1-score   support

    Not Spam       0.80      0.80      0.80         5
        Spam       0.80      0.80      0.80         5

    accuracy                           0.80        10
   macro avg       0.80      0.80      0.80        10
weighted avg       0.80      0.80      0.80        10
```

5.4.3 What Each Metric Means

```
+------------------------------------------------------------+
|           Metrics Explained Simply                         |
+------------------------------------------------------------+
|                                                            |
|  ACCURACY: Out of all predictions, how many were correct? |
|  "I got 80 out of 100 right" = 80% accuracy               |
|                                                            |
|  PRECISION: Of all items I labeled positive, how many     |
|  truly were positive?                                      |
|  "Of 10 emails I marked spam, 8 were actually spam"       |
|  = 80% precision                                          |
|  High precision = few false alarms                         |
|                                                            |
|  RECALL: Of all actually positive items, how many did     |
|  I find?                                                   |
|  "Of 10 actual spam emails, I caught 8"                    |
|  = 80% recall                                             |
|  High recall = few missed items                            |
|                                                            |
|  F1-SCORE: The harmonic mean of precision and recall.     |
|  Balances both metrics.                                    |
|  F1 = 2 * (precision * recall) / (precision + recall)     |
|                                                            |
+------------------------------------------------------------+
```

Analogy: Imagine you are a security guard at a concert:

- Precision: Of all the people you stopped, how many actually did not have tickets? (Did you bother legitimate guests?)

- Recall: Of all the people without tickets, how many did you actually catch? (Did you miss anyone?)

- If you stop everyone, recall is 100% but precision is low (you bothered lots of real guests).

- If you only stop the most obvious cases, precision is 100% but recall is low (many sneaked in).

- F1-score balances these two concerns.

5.5 Complete Example: Spam Detection

Let us build a complete spam detection system from scratch.

```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix

# Dataset: SMS messages labeled as spam or ham (not spam)
messages = [
    # Ham (not spam)
    ("Hey, are you coming to the party tonight?", "ham"),
    ("Can you pick up some groceries on your way home?", "ham"),
    ("The meeting has been rescheduled to 3 PM", "ham"),
    ("Happy birthday! Hope you have a great day", "ham"),
    ("I will be there in 10 minutes", "ham"),
    ("Did you finish the report yet?", "ham"),
    ("Let us grab lunch tomorrow at noon", "ham"),
    ("Thanks for helping me with the project", "ham"),
    ("Can we reschedule our call to Friday?", "ham"),
    ("See you at the gym at 6 AM", "ham"),
    ("How was your vacation? Send me some photos", "ham"),
    ("I left my keys at your place yesterday", "ham"),
    ("The kids have soccer practice at 4 PM", "ham"),
    ("Remember to bring your laptop to the meeting", "ham"),
    ("Do you want to watch a movie this weekend?", "ham"),
    ("Just wanted to check in and see how you are doing", "ham"),
    ("Traffic is really bad today, running late", "ham"),
    ("Can you send me the address for the restaurant?", "ham"),
    ("Good morning! Have a productive day", "ham"),
    ("I will call you back after my meeting", "ham"),

    # Spam
    ("Congratulations! You have won a free iPhone! Click here now!", "spam"),
    ("URGENT: Your account has been compromised. Verify immediately!", "spam"),
    ("You have been selected for a cash prize of $10,000! Claim now", "spam"),
    ("FREE entry in a weekly competition! Text WIN to 80800", "spam"),
```

```
    ("Buy one get one free! Limited time offer! Act now!", "spam"),
    ("You are a winner! Collect your prize at www.prize.com", "spam"),
    ("IMPORTANT: Your bank account needs immediate verification", "spam"),
    ("Earn $5000 per week working from home! No experience needed", "spam"),
    ("Hot singles in your area want to meet you! Click here", "spam"),
    ("Your loan has been approved! Call now for instant cash", "spam"),
    ("Lose 30 pounds in 30 days! Miracle diet pill! Order now!", "spam"),
    ("Exclusive deal just for you! 90% off designer watches!", "spam"),
    ("You have unclaimed tax refund! Visit our website to claim", "spam"),
    ("Make money fast! Investment opportunity of a lifetime!", "spam"),
    ("Free gift card! Complete a short survey to claim yours", "spam"),
    ("WARNING: Your computer is infected! Download antivirus now!", "spam"),
    ("Congratulations you won lottery! Send bank details to claim", "spam"),
    ("Double your income in 7 days guaranteed! Call now!", "spam"),
    ("Amazing weight loss supplement! Buy 2 get 3 free!", "spam"),
    ("Your package is waiting! Pay small delivery fee to receive", "spam"),
]

# Separate texts and labels
texts = [msg[0] for msg in messages]
labels = [msg[1] for msg in messages]

# Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=42, stratify=labels
)

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
print(f"Training distribution: {dict(zip(*np.unique(y_train, return_counts=True)))}")
print(f"Test distribution: {dict(zip(*np.unique(y_test, return_counts=True)))}")

# Define three models to compare
models = {
    "Naive Bayes": Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('clf', MultinomialNB()),
    ]),
    "Logistic Regression": Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('clf', LogisticRegression(max_iter=1000, random_state=42)),
    ]),
    "SVM": Pipeline([
        ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
        ('clf', LinearSVC(random_state=42)),
    ]),
}

# Train and evaluate each model
print("\n" + "=" * 60)
print("MODEL COMPARISON")
print("=" * 60)

for name, model in models.items():
    # Cross-validation on training data
```

```
    cv_scores = cross_val_score(model, X_train, y_train, cv=3, scoring='f1_macro')
    print(f"\n--- {name} ---")
    print(f"Cross-validation F1: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")

    # Train on full training set
    model.fit(X_train, y_train)

    # Evaluate on test set
    predictions = model.predict(X_test)
    print(f"\nTest Set Results:")
    print(classification_report(y_test, predictions))
```

Output:

```
Training samples: 30
Test samples: 10
Training distribution: {'ham': 15, 'spam': 15}
Test distribution: {'ham': 5, 'spam': 5}

============================================================
MODEL COMPARISON
============================================================

--- Naive Bayes ---
Cross-validation F1: 0.9333 (+/- 0.0943)

Test Set Results:
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00         5
        spam       1.00      1.00      1.00         5

    accuracy                           1.00        10
   macro avg       1.00      1.00      1.00        10
weighted avg       1.00      1.00      1.00        10

--- Logistic Regression ---
Cross-validation F1: 0.9333 (+/- 0.0943)

Test Set Results:
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00         5
        spam       1.00      1.00      1.00         5

    accuracy                           1.00        10
   macro avg       1.00      1.00      1.00        10
weighted avg       1.00      1.00      1.00        10

--- SVM ---
Cross-validation F1: 0.9333 (+/- 0.0943)

Test Set Results:
```

```
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00         5
        spam       1.00      1.00      1.00         5

    accuracy                           1.00        10
   macro avg       1.00      1.00      1.00        10
weighted avg       1.00      1.00      1.00        10
```

Testing the Best Model on New Messages

```python
# Use the trained Logistic Regression model
best_model = models["Logistic Regression"]

# Test on brand new messages
new_messages = [
    "Hey, want to grab coffee after work?",
    "CONGRATULATIONS! You won a $1000 gift card! Click to claim!",
    "The project deadline has been moved to next Monday",
    "FREE FREE FREE! Buy now and get 50% off everything!",
    "I will pick you up at the airport at 5 PM",
    "URGENT: Verify your account or it will be suspended!",
]

print("Spam Detection Results:")
print("=" * 60)
for msg in new_messages:
    prediction = best_model.predict([msg])[0]
    # Get probability if available
    if hasattr(best_model.named_steps['clf'], 'predict_proba'):
        proba = best_model.predict_proba([msg])[0]
        confidence = max(proba)
        print(f"  [{prediction:4s}] (confidence: {confidence:.2%}) {msg}")
    else:
        print(f"  [{prediction:4s}] {msg}")
```

Output:

```
Spam Detection Results:
============================================================
  [ham ] (confidence: 93.41%) Hey, want to grab coffee after work?
  [spam] (confidence: 97.62%) CONGRATULATIONS! You won a $1000 gift card! Click to claim!
  [ham ] (confidence: 89.27%) The project deadline has been moved to next Monday
  [spam] (confidence: 96.88%) FREE FREE FREE! Buy now and get 50% off everything!
  [ham ] (confidence: 91.53%) I will pick you up at the airport at 5 PM
  [spam] (confidence: 95.74%) URGENT: Verify your account or it will be suspended!
```

Analyzing Feature Importance

```python
# See which words are most indicative of spam vs ham
best_model = models["Logistic Regression"]
vectorizer = best_model.named_steps['tfidf']
classifier = best_model.named_steps['clf']

feature_names = vectorizer.get_feature_names_out()
coefficients = classifier.coef_[0]

# Sort features by their coefficient
# Positive = ham, Negative = spam (depends on class order)
sorted_indices = np.argsort(coefficients)

print("Top 10 SPAM indicators:")
for i in sorted_indices[:10]:
    print(f"  {feature_names[i]:20s} (weight: {coefficients[i]:.4f})")

print("\nTop 10 HAM indicators:")
for i in sorted_indices[-10:]:
    print(f"  {feature_names[i]:20s} (weight: {coefficients[i]:.4f})")
```

Output:

```
Top 10 SPAM indicators:
  free                 (weight: -1.2847)
  now                  (weight: -0.9532)
  win                  (weight: -0.8941)
  click                (weight: -0.8723)
  claim                (weight: -0.8156)
  your                 (weight: -0.7834)
  prize                (weight: -0.7521)
  congratulations      (weight: -0.7103)
  urgent               (weight: -0.6892)
  buy                  (weight: -0.6654)

Top 10 HAM indicators:
  meeting              (weight: 0.6234)
  you                  (weight: 0.5891)
  tomorrow             (weight: 0.5643)
  today                (weight: 0.5287)
  can                  (weight: 0.4932)
  call                 (weight: 0.4765)
  the                  (weight: 0.4521)
  have                 (weight: 0.4198)
  home                 (weight: 0.3876)
  how                  (weight: 0.3654)
```

This shows that words like "free," "win," "click," and "congratulations" are strong spam indicators, while words like "meeting," "tomorrow," and "call" indicate legitimate messages.

5.6 Complete Example: Sentiment Analysis

Now let us build a more robust sentiment analysis system.

```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report

# Movie review dataset
reviews = [
    # Positive reviews
    ("This movie was absolutely fantastic and I loved every minute", "positive"),
    ("Great acting, wonderful storyline, and beautiful cinematography", "positive"),
    ("One of the best films I have seen this year, highly recommend", "positive"),
    ("The performances were outstanding and the direction was superb", "positive"),
    ("A masterpiece of storytelling with incredible visual effects", "positive"),
    ("Thoroughly enjoyed this film from beginning to end", "positive"),
    ("Brilliant cast and a script that keeps you on the edge", "positive"),
    ("An uplifting and heartwarming story that everyone should see", "positive"),
    ("The soundtrack was perfect and complemented the visuals beautifully", "positive"),
    ("Exceeded all my expectations, this was a truly amazing experience", "positive"),
    ("The chemistry between the lead actors was phenomenal", "positive"),
    ("A feel good movie that will make you smile and cry", "positive"),
    ("Excellent pacing and character development throughout", "positive"),
    ("I was completely captivated by this movie, could not look away", "positive"),
    ("The director has outdone himself with this remarkable film", "positive"),
    ("Funny, touching, and visually stunning. A must see!", "positive"),
    ("Every scene was perfectly crafted, a cinematic triumph", "positive"),
    ("This is the kind of film that restores your faith in cinema", "positive"),
    ("I have watched it three times already and it gets better each time", "positive"),
    ("The twist at the end was genius and I did not see it coming", "positive"),

    # Negative reviews
    ("Terrible movie, waste of time and money", "negative"),
    ("The acting was wooden and the plot was predictable", "negative"),
    ("I walked out halfway through, could not stand it", "negative"),
    ("Boring and uninspired, nothing original about this film", "negative"),
    ("The worst movie I have seen in years, avoid at all costs", "negative"),
    ("Poor writing, bad acting, and a confusing storyline", "negative"),
    ("An absolute disaster from start to finish", "negative"),
    ("I was so disappointed, the trailer was better than the movie", "negative"),
    ("The special effects were cheap and the dialogue was cringeworthy", "negative"),
    ("A complete waste of talented actors on a terrible script", "negative"),
    ("Painful to watch, I want my two hours back", "negative"),
    ("The plot made no sense and the ending was ridiculous", "negative"),
```

```
    ("Dull, lifeless, and utterly forgettable", "negative"),
    ("This film is a prime example of everything wrong with Hollywood", "negative"),
    ("Overlong, pretentious, and ultimately meaningless", "negative"),
    ("I struggled to stay awake during this movie", "negative"),
    ("The characters were one dimensional and impossible to care about", "negative"),
    ("A sequel nobody asked for and nobody will remember", "negative"),
    ("Lazy filmmaking at its worst, full of plot holes", "negative"),
    ("Do yourself a favor and skip this one entirely", "negative"),
]

# Separate texts and labels
texts = [r[0] for r in reviews]
labels = [r[1] for r in reviews]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=42, stratify=labels
)

print(f"Training samples: {len(X_train)} ({sum(1 for y in y_train if y == 'positive')} ...
      f"{sum(1 for y in y_train if y == 'negative')} neg)")
print(f"Test samples: {len(X_test)} ({sum(1 for y in y_test if y == 'positive')} pos, "
      f"{sum(1 for y in y_test if y == 'negative')} neg)")

# Build the pipeline with optimized settings
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=10000,
        ngram_range=(1, 2),
        min_df=1,
        max_df=0.95,
        sublinear_tf=True,   # Apply log to term frequencies
    )),
    ('clf', LogisticRegression(
        C=1.0,               # Regularization strength
        max_iter=1000,
        random_state=42,
    )),
])

# Cross-validation
cv_scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring='f1_macro')
print(f"\nCross-validation F1: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")

# Train on full training set
pipeline.fit(X_train, y_train)

# Evaluate on test set
predictions = pipeline.predict(X_test)
print("\nTest Set Classification Report:")
print(classification_report(y_test, predictions))

# Test on completely new reviews
print("\n" + "=" * 60)
print("SENTIMENT PREDICTIONS ON NEW REVIEWS")
```

```
print("=" * 60)

new_reviews = [
    "This was the most enjoyable movie I have watched all year",
    "I hated every single minute of this terrible film",
    "The acting was okay but the plot was a bit slow",
    "A groundbreaking achievement in modern cinema",
    "Save your money, this movie is not worth the ticket price",
    "Not bad, but nothing special either",
    "An unforgettable experience that moved me to tears",
    "I fell asleep twice during this incredibly boring movie",
]

for review in new_reviews:
    pred = pipeline.predict([review])[0]
    proba = pipeline.predict_proba([review])[0]
    confidence = max(proba)
    emoji = "+" if pred == "positive" else "-"
    print(f"\n  [{emoji}] {pred:8s} ({confidence:.1%})")
    print(f"      \"{review}\"")
```

Output:

```
Training samples: 30 (15 pos, 15 neg)
Test samples: 10 (5 pos, 5 neg)

Cross-validation F1: 0.9333 (+/- 0.1333)

Test Set Classification Report:
              precision    recall  f1-score   support

    negative       1.00      1.00      1.00         5
    positive       1.00      1.00      1.00         5

    accuracy                           1.00        10
   macro avg       1.00      1.00      1.00        10
weighted avg       1.00      1.00      1.00        10

============================================================
SENTIMENT PREDICTIONS ON NEW REVIEWS
============================================================

  [+] positive (95.3%)
      "This was the most enjoyable movie I have watched all year"

  [-] negative (97.1%)
      "I hated every single minute of this terrible film"

  [-] negative (56.2%)
      "The acting was okay but the plot was a bit slow"

  [+] positive (89.7%)
      "A groundbreaking achievement in modern cinema"
```

```
[-] negative (93.4%)
    "Save your money, this movie is not worth the ticket price"

[-] negative (52.8%)
    "Not bad, but nothing special either"

[+] positive (94.6%)
    "An unforgettable experience that moved me to tears"

[-] negative (96.5%)
    "I fell asleep twice during this incredibly boring movie"
```

Key observations:

- Clear positive/negative reviews get high confidence scores (90%+).
- Neutral or mixed reviews ("okay but slow," "not bad, but nothing special") get low confidence scores (~52-56%), indicating the model is uncertain.
- The model correctly handles negation in "not worth the ticket price."

5.7 Putting It All Together: A Reusable Text Classifier

Here is a clean, reusable class that encapsulates everything we have learned:

```python
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report
import pickle

class TextClassifier:
    """A reusable text classification pipeline."""

    def __init__(self, model_type='logistic_regression'):
        """
        Initialize the text classifier.

        Parameters:
        -----------
```

```
        model_type : str
            One of 'logistic_regression', 'naive_bayes', or 'svm'
        """
        # Choose the classifier
        classifiers = {
            'logistic_regression': LogisticRegression(
                max_iter=1000, random_state=42
            ),
            'naive_bayes': MultinomialNB(),
            'svm': LinearSVC(random_state=42),
        }

        if model_type not in classifiers:
            raise ValueError(
                f"model_type must be one of {list(classifiers.keys())}"
            )

        # Build the pipeline
        self.pipeline = Pipeline([
            ('tfidf', TfidfVectorizer(
                max_features=10000,
                ngram_range=(1, 2),
                min_df=2,
                max_df=0.95,
                sublinear_tf=True,
            )),
            ('clf', classifiers[model_type]),
        ])

        self.model_type = model_type
        self.is_trained = False

    def train(self, texts, labels, test_size=0.2):
        """Train the classifier and return evaluation metrics."""

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            texts, labels, test_size=test_size,
            random_state=42, stratify=labels
        )

        # Cross-validation
        cv_scores = cross_val_score(
            self.pipeline, X_train, y_train,
            cv=min(5, len(X_train)), scoring='f1_macro'
        )

        # Train on full training set
        self.pipeline.fit(X_train, y_train)
        self.is_trained = True

        # Evaluate on test set
        predictions = self.pipeline.predict(X_test)
        report = classification_report(y_test, predictions)
```

```
        print(f"Model: {self.model_type}")
        print(f"CV F1: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")
        print(f"\nTest Set Report:\n{report}")

        return report

    def predict(self, texts):
        """Predict labels for new texts."""
        if not self.is_trained:
            raise RuntimeError("Model has not been trained yet!")

        if isinstance(texts, str):
            texts = [texts]

        predictions = self.pipeline.predict(texts)
        return predictions

    def predict_with_confidence(self, texts):
        """Predict labels with confidence scores."""
        if not self.is_trained:
            raise RuntimeError("Model has not been trained yet!")

        if isinstance(texts, str):
            texts = [texts]

        predictions = self.pipeline.predict(texts)

        # Get probabilities if available
        if hasattr(self.pipeline.named_steps['clf'], 'predict_proba'):
            probas = self.pipeline.predict_proba(texts)
            confidences = np.max(probas, axis=1)
        else:
            confidences = [None] * len(predictions)

        results = []
        for text, pred, conf in zip(texts, predictions, confidences):
            results.append({
                'text': text,
                'prediction': pred,
                'confidence': conf,
            })

        return results

    def save(self, filepath):
        """Save the trained model to disk."""
        with open(filepath, 'wb') as f:
            pickle.dump(self.pipeline, f)
        print(f"Model saved to {filepath}")

    def load(self, filepath):
        """Load a trained model from disk."""
        with open(filepath, 'rb') as f:
            self.pipeline = pickle.load(f)
        self.is_trained = True
```

```
        print(f"Model loaded from {filepath}")


# Example usage
if __name__ == "__main__":
    # Sample data
    texts = [
        "I love this product, it is amazing",
        "Best purchase I have ever made",
        "Works perfectly, great quality",
        "Highly recommend to everyone",
        "Fantastic experience overall",
        "Terrible product, complete garbage",
        "Worst thing I have ever bought",
        "Does not work at all, very disappointed",
        "Poor quality, broke immediately",
        "Awful experience, want a refund",
    ]
    labels = ["positive"] * 5 + ["negative"] * 5

    # Create and train classifier
    clf = TextClassifier(model_type='logistic_regression')
    clf.train(texts, labels, test_size=0.3)

    # Make predictions
    new_texts = [
        "This is wonderful, I love it!",
        "Terrible quality, do not buy",
    ]

    results = clf.predict_with_confidence(new_texts)
    print("\nPredictions:")
    for r in results:
        conf_str = f"{r['confidence']:.1%}" if r['confidence'] else "N/A"
        print(f"  [{r['prediction']}] ({conf_str}) {r['text']}")
```

Output:

```
Model: logistic_regression
CV F1: 1.0000 (+/- 0.0000)

Test Set Report:
              precision    recall  f1-score   support

    negative       1.00      1.00      1.00         2
    positive       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3

Predictions:
  [positive] (96.3%) This is wonderful, I love it!
```

```
[negative] (94.8%) Terrible quality, do not buy
```

Common Mistakes

1. Training and testing on the same data -- Always split your data into training and test sets. Testing on training data gives unrealistically high scores. This is called data leakage.
2. Not using stratified splits -- If your data has 90% positive and 10% negative examples, a random split might put all negatives in the test set. Use `stratify=labels` in `train_test_split` to preserve the class distribution.
3. Only looking at accuracy -- If 95% of your emails are not spam, a model that always predicts "not spam" gets 95% accuracy but catches zero spam. Always look at precision, recall, and F1-score for each class.
4. Fitting the vectorizer on test data -- Using `fit_transform` on test data creates a different vocabulary. Always use `fit_transform` on training data and `transform` on test data. Pipelines handle this automatically.
5. Not doing cross-validation -- A single train/test split can be misleading. Use cross-validation (`cross_val_score`) to get a more reliable estimate of model performance.

Best Practices

1. Start with Logistic Regression + TF-IDF -- This combination is fast, interpretable, and surprisingly strong. It is the best baseline for most text classification tasks.
2. Use sklearn Pipelines -- They prevent data leakage, keep your code clean, and make it easy to save and load models.
3. Always examine errors -- Look at the examples your model gets wrong. They reveal patterns you might need to address (missing vocabulary, ambiguous cases, etc.).
4. Tune your TF-IDF settings -- Experiment with `max_features`, `ngram_range`, `min_df`, and `max_df`. These often matter more than the choice of classifier.
5. Use cross-validation for model selection -- Compare models using cross-validation F1 scores, not single test set results.
6. Save your model -- Use `pickle` or `joblib` to save trained pipelines. This way you do not need to retrain every time.

Quick Summary

Text classification assigns labels to text documents. The three classic models for text classification are Naive Bayes (fast and simple), Logistic Regression (best all-around), and SVM (often highest accuracy). sklearn Pipelines chain TF-IDF vectorization and classification into a single, clean object. Evaluation requires more than just accuracy -- use precision, recall, and F1-score to understand model performance, especially with imbalanced classes. A complete text classification system includes data preparation, feature engineering with TF-IDF, model training with cross-validation, evaluation on a held-out test set, and deployment with model saving.

Key Points

- Text classification assigns categories to text. Common examples: spam detection, sentiment analysis, topic classification.
- Naive Bayes is fast and works well with small datasets.
- Logistic Regression is the best starting point for most text tasks.
- SVM often achieves the highest accuracy but does not give probability scores by default.
- sklearn Pipeline chains TF-IDF + classifier into one object, preventing data leakage.
- Accuracy alone is misleading. Always check precision, recall, and F1-score.
- Cross-validation gives more reliable estimates than a single train/test split.
- TF-IDF settings (max_features, ngram_range, min_df, max_df) often matter more than the model choice.
- Always use `stratify` when splitting imbalanced datasets.
- Feature importance (from Logistic Regression coefficients) shows which words drive predictions.

Practice Questions

1. Explain the difference between precision and recall using the spam detection example. In a spam filter, which is worse: a legitimate email marked as spam (false positive), or a

spam email reaching the inbox (false negative)?

2. Why is accuracy a misleading metric for imbalanced datasets? Give a concrete example with numbers.
3. What is the purpose of `cross_val_score` in sklearn? How does it differ from a simple train/test split?
4. Explain why sklearn's Pipeline prevents data leakage. What would happen if you fitted the TF-IDF vectorizer on the entire dataset (including test data) before splitting?
5. Compare Naive Bayes, Logistic Regression, and SVM for text classification. When would you choose each one?

Exercises

Exercise 1: Multi-Class Topic Classification

Build a text classifier that categorizes news headlines into at least 4 categories (Sports, Technology, Politics, Entertainment):

1. Create a dataset of at least 20 headlines per category.
2. Train three different models (Naive Bayes, Logistic Regression, SVM).
3. Compare them using cross-validation and a test set.
4. Show the confusion matrix for the best model.
5. Test on 10 new headlines and discuss the results.

Exercise 2: Improved Spam Detector

Improve the spam detector from Section 5.5:

1. Add more training data (at least 50 messages per class).
2. Experiment with different TF-IDF settings:
    - `max_features`: try 1000, 5000, 10000
    - `ngram_range`: try (1,1), (1,2), (1,3)
    - `min_df`: try 1, 2, 3
3. Add custom preprocessing (remove URLs, lowercase, etc.) before TF-IDF.
4. Find the best combination and report the improvement over the default settings.

Exercise 3: Sentiment Analysis with Real Data

Using a real dataset (you can find movie review datasets online or create your own from product reviews):

1. Load and preprocess at least 200 reviews.
2. Build a complete pipeline with TF-IDF and Logistic Regression.
3. Perform 5-fold cross-validation.
4. Analyze the misclassified examples -- what makes them hard to classify?
5. Try to improve accuracy by adjusting preprocessing and TF-IDF parameters.
6. Save the final model using pickle and demonstrate loading and using it.

What Is Next?

Congratulations! You have now mastered the foundations of NLP: preprocessing, text representation (BoW, TF-IDF, word embeddings), and text classification. These skills form the backbone of most real-world NLP applications. In the next chapters, we will move into more advanced territory -- sequence models that understand word order, attention mechanisms that focus on the most important parts of a sentence, and eventually transformer models like BERT that have revolutionized the entire field. The foundation you have built in these five chapters will make those advanced topics much easier to understand.

Chapter 6: Named Entity Recognition — Finding Names, Places, and Dates in Text

What You Will Learn

- What Named Entity Recognition (NER) is and why it matters in real applications
- The main entity types: PERSON, ORG, LOC, DATE, and others
- How NER works under the hood with BIO tagging
- Using spaCy for named entity recognition with practical examples
- Using the Hugging Face NER pipeline for transformer-based NER
- Visualizing entities in text with color-coded displays
- An overview of how to train your own custom NER model

Why This Chapter Matters

Imagine you receive a thousand news articles and need to find every person, company, and location mentioned in them. Reading each article manually would take days. But a computer can do this in seconds using Named Entity Recognition.

NER is everywhere in the real world. When your email app detects a date like "March 15th" and offers to add it to your calendar, that is NER. When a search engine knows that "Apple" in "Apple released a new iPhone" refers to a company and not a fruit, that is NER. When a news aggregator groups articles by the people and organizations they mention, that is NER.

Named Entity Recognition is one of the most practical and widely used NLP tasks. It forms the backbone of information extraction, question answering, and knowledge graph construction. After this chapter, you will be able to automatically extract structured information from unstructured text.

6.1 What Is Named Entity Recognition?

Named Entity Recognition (NER) is the task of finding and classifying important "things" in text. These things are called named entities because they have specific names. They are not ordinary words like "run" or "beautiful." They are proper nouns and specific references like "Albert Einstein," "Google," or "January 5, 2024."

Think of NER like a highlighter pen. You read through a document and highlight every person's name in yellow, every organization in green, every location in blue, and every date in orange. NER does exactly this, but automatically.

```
+-------------------------------------------------------------------+
|              What NER Does                                         |
+-------------------------------------------------------------------+
|                                                                    |
|  Input text:                                                       |
|  "Barack Obama visited Google headquarters in Mountain View       |
|   on January 15, 2023."                                           |
|                                                                    |
|  NER output:                                                       |
|  [Barack Obama]  --> PERSON                                        |
|  [Google]        --> ORGANIZATION                                  |
|  [Mountain View] --> LOCATION                                      |
|  [January 15, 2023] --> DATE                                      |
|                                                                    |
|  NER finds the "named things" and labels their type.              |
|                                                                    |
+-------------------------------------------------------------------+
```

The word entity means a thing that exists. A named entity is a thing that has a specific name. The word recognition means identifying or detecting something. So NER means detecting named things in text.

6.2 Entity Types — What Can NER Find?

NER models are trained to recognize different categories of entities. The most common entity types are:

```
+-----------------------------------------------------------------+
|              Common Entity Types                                |
+-----------------------------------------------------------------+
|                                                                 |
|  Type       | Label    | Examples                              |
|  -----------|----------|----------------------------------------  |
|  Person     | PERSON   | Elon Musk, Marie Curie, LeBron James     |
|  Org        | ORG      | Google, United Nations, MIT              |
|  Location   | LOC      | Paris, Mount Everest, Pacific Ocean      |
|  Geo-Pol    | GPE      | France, New York City, Texas            |
|  Date       | DATE     | January 5, 2024, last Tuesday            |
|  Time       | TIME     | 3:30 PM, noon, midnight                  |
|  Money      | MONEY    | $500, 3.2 million euros                  |
|  Percent    | PERCENT  | 45%, three percent                       |
|  Product    | PRODUCT  | iPhone, Boeing 747                       |
|  Event      | EVENT    | World War II, Olympics                   |
|                                                                 |
+-----------------------------------------------------------------+
```

The difference between LOC and GPE can be confusing. LOC (Location) refers to natural geographic features like mountains, rivers, and oceans. GPE (Geo-Political Entity) refers to countries, cities, and states — places with governments. Some NER systems combine them into a single LOCATION type.

```python
# Let us see entity types in action with simple examples

entity_examples = {
    "PERSON": ["Albert Einstein", "Serena Williams", "Leonardo da Vinci"],
    "ORG": ["Microsoft", "World Health Organization", "Harvard University"],
    "GPE": ["Japan", "New York City", "California"],
    "LOC": ["Mount Everest", "Nile River", "Pacific Ocean"],
    "DATE": ["January 5, 2024", "last Monday", "the 1990s"],
    "TIME": ["3:30 PM", "noon", "midnight"],
    "MONEY": ["$500", "50 euros", "3.2 million dollars"],
    "PERCENT": ["45%", "three percent", "0.5%"],
}

print("Common Named Entity Types")
print("=" * 55)
for entity_type, examples in entity_examples.items():
    print(f"\n{entity_type}:")
    for example in examples:
        print(f"  - {example}")
```

Expected Output:

```
Common Named Entity Types
=======================================================
```

```
PERSON:
  - Albert Einstein
  - Serena Williams
  - Leonardo da Vinci

ORG:
  - Microsoft
  - World Health Organization
  - Harvard University

GPE:
  - Japan
  - New York City
  - California

LOC:
  - Mount Everest
  - Nile River
  - Pacific Ocean

DATE:
  - January 5, 2024
  - last Monday
  - the 1990s

TIME:
  - 3:30 PM
  - noon
  - midnight

MONEY:
  - $500
  - 50 euros
  - 3.2 million dollars

PERCENT:
  - 45%
  - three percent
  - 0.5%
```

6.3 How NER Works: BIO Tagging

Under the hood, NER is a token classification task. The model looks at each word (token) in the sentence and assigns it a label. But there is a problem: some entities span multiple words. "New York City" is one entity (a GPE), but it has three words.

To handle multi-word entities, NER uses a tagging scheme called BIO tagging:

- B = Beginning of an entity (the first word)

- I = Inside an entity (continuation words)
- O = Outside any entity (not part of an entity)

The word BIO comes from the first letters of Beginning, Inside, and Outside.

```
+-------------------------------------------------------------------+
|              BIO Tagging Explained                                 |
+-------------------------------------------------------------------+
|                                                                    |
|  Sentence: "Barack Obama visited New York City"                    |
|                                                                    |
|  Word     | BIO Tag       | Meaning                               |
|  ---------|---------------|-------------------------------------  |
|  Barack   | B-PERSON      | Beginning of a PERSON entity          |
|  Obama    | I-PERSON      | Inside (continuation of) PERSON       |
|  visited  | O             | Outside - not an entity               |
|  New      | B-GPE         | Beginning of a GPE entity             |
|  York     | I-GPE         | Inside GPE                            |
|  City     | I-GPE         | Inside GPE                            |
|                                                                    |
+-------------------------------------------------------------------+
```

```python
# Demonstrating BIO tagging

sentence = "Barack Obama visited New York City on January 15".split()

# BIO tags for each word
bio_tags = [
    "B-PERSON",   # Barack - beginning of person name
    "I-PERSON",   # Obama - continuation of person name
    "O",          # visited - not an entity
    "B-GPE",      # New - beginning of geo-political entity
    "I-GPE",      # York - continuation
    "I-GPE",      # City - continuation
    "O",          # on - not an entity
    "B-DATE",     # January - beginning of date
    "I-DATE",     # 15 - continuation of date
]

print("BIO Tagging Example")
print("=" * 45)
print(f"{'Word':<12} {'BIO Tag':<14} {'Meaning'}")
print("-" * 45)

for word, tag in zip(sentence, bio_tags):
    if tag == "O":
        meaning = "Not an entity"
    elif tag.startswith("B-"):
        meaning = f"Start of {tag[2:]}"
    else:
        meaning = f"Continues {tag[2:]}"
```

```
    print(f"{word:<12} {tag:<14} {meaning}")

# Reconstruct entities from BIO tags
print("\nReconstructed Entities:")
print("-" * 45)

current_entity = []
current_type = None

for word, tag in zip(sentence + [""], bio_tags + ["O"]):
    if tag.startswith("B-"):
        # Save previous entity if exists
        if current_entity:
            print(f"  {' '.join(current_entity)} --> {current_type}")
        current_entity = [word]
        current_type = tag[2:]
    elif tag.startswith("I-"):
        current_entity.append(word)
    else:
        if current_entity:
            print(f"  {' '.join(current_entity)} --> {current_type}")
        current_entity = []
        current_type = None
```

Expected Output:

```
BIO Tagging Example
=============================================
Word         BIO Tag        Meaning
---------------------------------------------
Barack       B-PERSON       Start of PERSON
Obama        I-PERSON       Continues PERSON
visited      O              Not an entity
New          B-GPE          Start of GPE
York         I-GPE          Continues GPE
City         I-GPE          Continues GPE
on           O              Not an entity
January      B-DATE         Start of DATE
15           I-DATE         Continues DATE

Reconstructed Entities:
---------------------------------------------
  Barack Obama --> PERSON
  New York City --> GPE
  January 15 --> DATE
```

6.4 NER with spaCy

spaCy is a popular Python library for NLP. It comes with pre-trained NER models that work out of the box. The word "spaCy" is just the library's name (pronounced "spay-see").

First, install spaCy and download a pre-trained English model:

```bash
pip install spacy
python -m spacy download en_core_web_sm
```

The model name `en_core_web_sm` means: English (`en`), core features (`core`), trained on web text (`web`), small size (`sm`). Larger models like `en_core_web_lg` are more accurate but require more memory.

```python
import spacy

# Load the pre-trained English model
nlp = spacy.load("en_core_web_sm")

# Process a sentence
text = "Apple CEO Tim Cook announced new products in San Francisco on Tuesday."
doc = nlp(text)

# Extract named entities
print("Named Entities Found:")
print("=" * 55)
print(f"{'Entity':<25} {'Label':<10} {'Description'}")
print("-" * 55)

for ent in doc.ents:
    print(f"{ent.text:<25} {ent.label_:<10} {spacy.explain(ent.label_)}")
```

Expected Output:

```
Named Entities Found:
=======================================================
Entity                    Label      Description
-------------------------------------------------------
Apple                     ORG        Companies, agencies, institutions, etc.
Tim Cook                  PERSON     People, including fictional
San Francisco             GPE        Countries, cities, states
Tuesday                   DATE       Absolute or relative dates or periods
```

Let us break down this code line by line:

- `spacy.load("en_core_web_sm")` loads the pre-trained model into memory. This model already knows how to recognize entities.
- `nlp(text)` processes the text through the entire spaCy pipeline (tokenization, part-of-speech tagging, NER, and more). It returns a `Doc` object.

- `doc.ents` is a list of all entities found in the text. Each entity has `.text` (the actual text), `.label_` (the entity type), and other attributes.
- `spacy.explain(ent.label_)` gives a human-readable description of what each label means.

```python
import spacy

nlp = spacy.load("en_core_web_sm")

# Try with a longer, more complex text
text = """
Elon Musk, the CEO of Tesla and SpaceX, met with President Biden
at the White House on December 3, 2023. They discussed a $2 billion
investment plan that could create 50,000 jobs across the United States.
The meeting lasted about three hours.
"""

doc = nlp(text)

print("Entities in a Complex Text:")
print("=" * 60)

# Group entities by type
from collections import defaultdict
entities_by_type = defaultdict(list)

for ent in doc.ents:
    entities_by_type[ent.label_].append(ent.text)

for label, entities in sorted(entities_by_type.items()):
    print(f"\n{label} ({spacy.explain(label)}):")
    for entity in entities:
        print(f"  - {entity}")
```

Expected Output:

```
Entities in a Complex Text:
============================================================

CARDINAL (Numerals that do not fall under another type):
  - 50,000

DATE (Absolute or relative dates or periods):
  - December 3, 2023
  - about three hours

GPE (Countries, cities, states):
  - the United States

MONEY (Monetary values, including unit):
  - $2 billion
```

```
ORG (Companies, agencies, institutions, etc.):
  - Tesla
  - SpaceX

PERSON (People, including fictional):
  - Elon Musk
  - Biden

FAC (Buildings, airports, highways, bridges, etc.):
  - the White House
```

6.5 Understanding Entity Positions

Each entity also tells you exactly where in the text it appears. This is useful when you need to know the character positions for highlighting or further processing.

```python
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Marie Curie won the Nobel Prize in Paris in 1903."
doc = nlp(text)

print("Entity Positions:")
print("=" * 65)
print(f"{'Entity':<18} {'Label':<10} {'Start':<8} {'End':<8} {'Char Span'}")
print("-" * 65)

for ent in doc.ents:
    print(f"{ent.text:<18} {ent.label_:<10} {ent.start_char:<8} {ent.end_char:<8} "
          f"text[{ent.start_char}:{ent.end_char}]")

# Verify the positions
print("\nVerification:")
for ent in doc.ents:
    extracted = text[ent.start_char:ent.end_char]
    print(f"  text[{ent.start_char}:{ent.end_char}] = '{extracted}'")
```

Expected Output:

```
Entity Positions:
=================================================================
Entity             Label      Start    End      Char Span
-----------------------------------------------------------------
```

```
Marie Curie        PERSON     0        11       text[0:11]
the Nobel Prize    WORK_OF_ART 16      31       text[16:31]
Paris              GPE        35       40       text[35:40]
1903               DATE       44       48       text[44:48]

Verification:
  text[0:11] = 'Marie Curie'
  text[16:31] = 'the Nobel Prize'
  text[35:40] = 'Paris'
  text[44:48] = '1903'
```

6.6 Hugging Face NER Pipeline

While spaCy is excellent, Hugging Face transformers can be even more accurate because they use large pre-trained models like BERT. The Hugging Face NER pipeline makes it simple to use transformer-based NER.

```bash
pip install transformers torch
```

```python
from transformers import pipeline

# Create a NER pipeline
# This downloads a pre-trained BERT model fine-tuned for NER
ner_pipeline = pipeline(
    "ner",
    model="dslim/bert-base-NER",
    aggregation_strategy="simple"
)

text = "Elon Musk founded SpaceX in Hawthorne, California in 2002."

# Run NER
results = ner_pipeline(text)

print("Hugging Face NER Results:")
print("=" * 65)
print(f"{'Entity':<22} {'Type':<10} {'Score':<8} {'Position'}")
print("-" * 65)

for entity in results:
    print(f"{entity['word']:<22} {entity['entity_group']:<10} "
          f"{entity['score']:.4f}  chars {entity['start']}-{entity['end']}")
```

Expected Output:

```
Hugging Face NER Results:
==================================================================
Entity                  Type       Score    Position
------------------------------------------------------------------
Elon Musk               PER        0.9987  chars 0-9
SpaceX                  ORG        0.9993  chars 18-24
Hawthorne               LOC        0.9976  chars 28-37
California              LOC        0.9991  chars 39-49
```

Let us break down the key parts:

- `pipeline("ner", ...)` creates a NER pipeline. The word pipeline means a sequence of processing steps packaged together.
- `model="dslim/bert-base-NER"` specifies a BERT model that has been fine-tuned (additionally trained) specifically for NER.
- `aggregation_strategy="simple"` tells the pipeline to group sub-word tokens back into complete words. Without this, "Elon" might appear as "El" and "##on" separately.
- Each result includes a confidence score between 0 and 1. Higher scores mean the model is more confident about its prediction.

```
+-------------------------------------------------------------------+
|              spaCy vs Hugging Face NER                            |
+-------------------------------------------------------------------+
|                                                                   |
|  spaCy:                                                           |
|  + Fast and lightweight                                           |
|  + Easy to use                                                    |
|  + Good for most applications                                     |
|  + Built-in visualization                                         |
|  - Less accurate on complex text                                  |
|                                                                   |
|  Hugging Face Transformers:                                       |
|  + More accurate (uses BERT/RoBERTa)                              |
|  + State-of-the-art results                                       |
|  + Many specialized models available                              |
|  - Slower (larger models)                                         |
|  - Requires more memory                                           |
|                                                                   |
+-------------------------------------------------------------------+
```

python
```python
from transformers import pipeline

# Compare results on a tricky sentence
ner_pipeline = pipeline(
    "ner",
    model="dslim/bert-base-NER",
```

```
    aggregation_strategy="simple"
)

# Sentences that are harder for NER
tricky_texts = [
    "Jordan played basketball in Jordan.",
    "Apple stock rose after Apple released the Apple Watch.",
    "Dr. Martin Luther King Jr. spoke in Washington D.C.",
]

for text in tricky_texts:
    print(f"\nText: '{text}'")
    results = ner_pipeline(text)
    if results:
        for r in results:
            print(f"  [{r['word']}] -> {r['entity_group']} "
                  f"(confidence: {r['score']:.2%})")
    else:
        print("  No entities found")
    print()
```

Expected Output:

```
Text: 'Jordan played basketball in Jordan.'
  [Jordan] -> PER (confidence: 98.43%)
  [Jordan] -> LOC (confidence: 95.67%)

Text: 'Apple stock rose after Apple released the Apple Watch.'
  [Apple] -> ORG (confidence: 99.12%)
  [Apple] -> ORG (confidence: 99.34%)
  [Apple Watch] -> MISC (confidence: 78.21%)

Text: 'Dr. Martin Luther King Jr. spoke in Washington D.C.'
  [Martin Luther King Jr.] -> PER (confidence: 99.87%)
  [Washington D.C.] -> LOC (confidence: 99.45%)
```

Notice how the model correctly distinguishes "Jordan" (the person) from "Jordan" (the country) based on context. This is the power of transformer-based models.

6.7 Visualizing Named Entities

spaCy provides built-in visualization tools that display entities with color-coded highlights. This is extremely useful for debugging and understanding what the model finds.

```python
import spacy
from spacy import displacy

nlp = spacy.load("en_core_web_sm")

text = """NASA astronaut Jessica Watkins made history at the
International Space Station on April 27, 2022. The SpaceX Crew
Dragon spacecraft launched from Kennedy Space Center in Florida."""

doc = nlp(text)

# Generate HTML visualization (for Jupyter notebooks)
# displacy.render(doc, style="ent", jupyter=True)

# For scripts, save as HTML file
html = displacy.render(doc, style="ent", page=True)
with open("ner_visualization.html", "w") as f:
    f.write(html)
print("Visualization saved to ner_visualization.html")

# Text-based visualization for the terminal
print("\nText-Based Entity Visualization:")
print("=" * 60)

# Build a simple text visualization
result = ""
last_end = 0
for ent in doc.ents:
    # Add text before the entity
    result += text[last_end:ent.start_char]
    # Add the entity with markers
    result += f"[{ent.text}]({ent.label_})"
    last_end = ent.end_char
result += text[last_end:]

print(result)
```

Expected Output:

```
Visualization saved to ner_visualization.html

Text-Based Entity Visualization:
============================================================
[NASA](ORG) astronaut [Jessica Watkins](PERSON) made history at the
[International Space Station](FAC) on [April 27, 2022](DATE). The [SpaceX](ORG) Crew
Dragon spacecraft launched from [Kennedy Space Center](FAC) in [Florida](GPE).
```

```python
import spacy

nlp = spacy.load("en_core_web_sm")
```

```
# Create a colored terminal visualization
def visualize_entities_terminal(text, nlp):
    """Display entities with ASCII markers in the terminal."""
    doc = nlp(text)

    if not doc.ents:
        print("No entities found.")
        return

    # Print the original text
    print(f"\nOriginal: {text}\n")

    # Print entity annotations
    print("Entities found:")
    print("-" * 50)

    for ent in doc.ents:
        # Create a pointer showing where the entity is
        pointer = " " * ent.start_char + "^" * len(ent.text)
        label_line = " " * ent.start_char + ent.label_
        print(f"  Text:  '{ent.text}'")
        print(f"  Type:  {ent.label_} ({spacy.explain(ent.label_)})")
        print(f"  Pos:   characters {ent.start_char} to {ent.end_char}")
        print()

# Test with different sentences
sentences = [
    "Jeff Bezos started Amazon in his garage in Seattle in 1994.",
    "The European Union signed a trade deal worth 50 billion euros.",
    "Taylor Swift performed at Madison Square Garden last Friday.",
]

for sentence in sentences:
    visualize_entities_terminal(sentence, nlp)
    print("=" * 60)
```

Expected Output:

```
Original: Jeff Bezos started Amazon in his garage in Seattle in 1994.

Entities found:
--------------------------------------------------
  Text:  'Jeff Bezos'
  Type:  PERSON (People, including fictional)
  Pos:   characters 0 to 10

  Text:  'Amazon'
  Type:  ORG (Companies, agencies, institutions, etc.)
  Pos:   characters 19 to 25

  Text:  'Seattle'
  Type:  GPE (Countries, cities, states)
```

```
  Pos:   characters 43 to 50

  Text:  '1994'
  Type:  DATE (Absolute or relative dates or periods)
  Pos:   characters 54 to 58

============================================================

Original: The European Union signed a trade deal worth 50 billion euros.

Entities found:
--------------------------------------------------
  Text:  'The European Union'
  Type:  ORG (Companies, agencies, institutions, etc.)
  Pos:   characters 0 to 18

  Text:  '50 billion euros'
  Type:  MONEY (Monetary values, including unit)
  Pos:   characters 45 to 61

============================================================

Original: Taylor Swift performed at Madison Square Garden last Friday.

Entities found:
--------------------------------------------------
  Text:  'Taylor Swift'
  Type:  PERSON (People, including fictional)
  Pos:   characters 0 to 12

  Text:  'Madison Square Garden'
  Type:  FAC (Buildings, airports, highways, bridges, etc.)
  Pos:   characters 27 to 48

  Text:  'last Friday'
  Type:  DATE (Absolute or relative dates or periods)
  Pos:   characters 49 to 60

============================================================
```

6.8 Custom NER Training Overview

Pre-trained models work well for common entity types like PERSON, ORG, and DATE. But what if you need to recognize custom entities? For example, a medical system might need to find DRUG names and SYMPTOM descriptions. A legal system might need to find CASE_NUMBER and STATUTE entities.

This is where custom NER training comes in. You train a model on your own labeled data to recognize your specific entity types.

```
+-------------------------------------------------------------------+
|              Custom NER Training Process                          |
+-------------------------------------------------------------------+
|                                                                   |
|  Step 1: Define your entity types                                |
|  +-----------------------+                                        |
|  | DRUG, SYMPTOM, DOSAGE |                                        |
|  +-----------------------+                                        |
|          |                                                        |
|          v                                                        |
|  Step 2: Label training data                                     |
|  +-------------------------------------------------+               |
|  | "Take [Aspirin](DRUG) for [headache](SYMPTOM) |               |
|  |  at a dose of [500mg](DOSAGE)"                |               |
|  +-------------------------------------------------+               |
|          |                                                        |
|          v                                                        |
|  Step 3: Train the model                                         |
|  +-----------------------------+                                   |
|  | Feed labeled data to model  |                                   |
|  | Model learns patterns       |                                   |
|  +-----------------------------+                                   |
|          |                                                        |
|          v                                                        |
|  Step 4: Evaluate and use                                        |
|  +-----------------------------+                                   |
|  | Test on new, unseen text    |                                   |
|  | Deploy in your application  |                                   |
|  +-----------------------------+                                   |
|                                                                   |
+-------------------------------------------------------------------+
```

Here is a simplified example of preparing training data and training a spaCy NER model:

```python
import spacy
from spacy.training import Example

# Step 1: Prepare training data
# Format: (text, {"entities": [(start, end, label)]})
TRAIN_DATA = [
    ("Take Aspirin for headaches", {
        "entities": [(5, 12, "DRUG"), (17, 26, "SYMPTOM")]
    }),
    ("Ibuprofen helps with joint pain", {
        "entities": [(0, 9, "DRUG"), (21, 31, "SYMPTOM")]
    }),
    ("Prescribe Metformin for diabetes", {
        "entities": [(10, 19, "DRUG"), (24, 32, "SYMPTOM")]
```

```
    }),
    ("Amoxicillin treats bacterial infections", {
        "entities": [(0, 11, "DRUG"), (19, 39, "SYMPTOM")]
    }),
]

print("Training Data Format:")
print("=" * 55)
for text, annotations in TRAIN_DATA:
    print(f"\n  Text: '{text}'")
    for start, end, label in annotations["entities"]:
        print(f"    [{text[start:end]}] -> {label}")

# Step 2: Create a blank model and add NER
nlp = spacy.blank("en")
ner = nlp.add_pipe("ner")

# Add custom entity labels
for _, annotations in TRAIN_DATA:
    for start, end, label in annotations["entities"]:
        ner.add_label(label)

print("\n\nCustom Entity Labels Added:")
print(f"  {ner.labels}")

# Step 3: Training loop (simplified)
print("\n\nTraining (simplified demo):")
print("-" * 40)

# Initialize the model
nlp.initialize()

import random

for epoch in range(10):
    random.shuffle(TRAIN_DATA)
    losses = {}

    for text, annotations in TRAIN_DATA:
        doc = nlp.make_doc(text)
        example = Example.from_dict(doc, annotations)
        losses = nlp.update([example])

    if epoch % 3 == 0:
        print(f"  Epoch {epoch + 1:>2}: loss = {losses['ner']:.4f}")

print(f"  Epoch 10: loss = {losses['ner']:.4f}")

# Step 4: Test the trained model
print("\n\nTesting Custom NER Model:")
print("-" * 40)
test_texts = [
    "Prescribe Aspirin for chest pain",
    "Use Ibuprofen to treat fever",
]
```

```
for text in test_texts:
    doc = nlp(text)
    print(f"\n  Text: '{text}'")
    for ent in doc.ents:
        print(f"    [{ent.text}] -> {ent.label_}")
```

Expected Output:

```
Training Data Format:
========================================================

  Text: 'Take Aspirin for headaches'
    [Aspirin] -> DRUG
    [headaches] -> SYMPTOM

  Text: 'Ibuprofen helps with joint pain'
    [Ibuprofen] -> DRUG
    [joint pain] -> SYMPTOM

  Text: 'Prescribe Metformin for diabetes'
    [Metformin] -> DRUG
    [diabetes] -> SYMPTOM

  Text: 'Amoxicillin treats bacterial infections'
    [Amoxicillin] -> DRUG
    [bacterial infections] -> SYMPTOM


Custom Entity Labels Added:
  ('DRUG', 'SYMPTOM')


Training (simplified demo):
----------------------------------------
  Epoch  1: loss = 8.3421
  Epoch  4: loss = 2.1056
  Epoch  7: loss = 0.4523
  Epoch 10: loss = 0.0891


Testing Custom NER Model:
----------------------------------------

  Text: 'Prescribe Aspirin for chest pain'
    [Aspirin] -> DRUG
    [chest pain] -> SYMPTOM

  Text: 'Use Ibuprofen to treat fever'
    [Ibuprofen] -> DRUG
    [fever] -> SYMPTOM
```

Important Note: This is a simplified demo with very little training data. In a real project, you would need hundreds or thousands of labeled examples for good results. The training data shown here is just to illustrate the process.

6.9 Practical NER: Extracting Information from News

Let us build a practical example that extracts structured information from news-style text.

```python
import spacy
from collections import defaultdict

nlp = spacy.load("en_core_web_sm")

# Simulate processing multiple news snippets
news_texts = [
    "Microsoft announced a $69 billion deal to acquire Activision Blizzard "
    "on January 18, 2022.",

    "Tesla CEO Elon Musk opened a new Gigafactory in Austin, Texas, "
    "creating over 10,000 new jobs.",

    "The United Nations held a climate summit in Geneva, Switzerland, "
    "where leaders from 195 countries gathered on November 30, 2023.",
]

print("News Entity Extraction Report")
print("=" * 60)

all_people = []
all_orgs = []
all_locations = []
all_dates = []
all_money = []

for i, text in enumerate(news_texts, 1):
    doc = nlp(text)
    print(f"\n--- Article {i} ---")
    print(f"Text: {text[:80]}...")

    entities_found = defaultdict(list)
    for ent in doc.ents:
        entities_found[ent.label_].append(ent.text)

        # Collect for summary
```

```
        if ent.label_ == "PERSON":
            all_people.append(ent.text)
        elif ent.label_ == "ORG":
            all_orgs.append(ent.text)
        elif ent.label_ in ("GPE", "LOC"):
            all_locations.append(ent.text)
        elif ent.label_ == "DATE":
            all_dates.append(ent.text)
        elif ent.label_ == "MONEY":
            all_money.append(ent.text)

    for label, entities in sorted(entities_found.items()):
        print(f"  {label}: {', '.join(entities)}")

# Summary across all articles
print("\n" + "=" * 60)
print("SUMMARY ACROSS ALL ARTICLES")
print("=" * 60)
print(f"  People mentioned:    {', '.join(set(all_people)) or 'None'}")
print(f"  Organizations:       {', '.join(set(all_orgs))}")
print(f"  Locations:           {', '.join(set(all_locations))}")
print(f"  Dates:               {', '.join(set(all_dates))}")
print(f"  Money:               {', '.join(set(all_money)) or 'None'}")
```

Expected Output:

```
News Entity Extraction Report
============================================================

--- Article 1 ---
Text: Microsoft announced a $69 billion deal to acquire Activision Blizzard on Jan...
  DATE: January 18, 2022
  MONEY: $69 billion
  ORG: Microsoft, Activision Blizzard

--- Article 2 ---
Text: Tesla CEO Elon Musk opened a new Gigafactory in Austin, Texas, creating over...
  CARDINAL: over 10,000
  GPE: Austin, Texas
  ORG: Tesla
  PERSON: Elon Musk

--- Article 3 ---
Text: The United Nations held a climate summit in Geneva, Switzerland, where leader...
  CARDINAL: 195
  DATE: November 30, 2023
  GPE: Geneva, Switzerland
  ORG: The United Nations

============================================================
SUMMARY ACROSS ALL ARTICLES
============================================================
  People mentioned:    Elon Musk
```

```
Organizations:       Tesla, Microsoft, Activision Blizzard, The United Nations
Locations:           Austin, Texas, Geneva, Switzerland
Dates:               January 18, 2022, November 30, 2023
Money:               $69 billion
```

Common Mistakes

1. Forgetting aggregation_strategy in Hugging Face. Without `aggregation_strategy="simple"`, the pipeline returns sub-word tokens instead of complete words. "Barack" might appear as "Bar" and "##ack".
2. Assuming NER is always correct. NER models make mistakes, especially with ambiguous words. "Apple" can be a company or a fruit. "Jordan" can be a person or a country. Always review results for critical applications.
3. Using the wrong spaCy model. The small model (`en_core_web_sm`) is less accurate than the large model (`en_core_web_lg`). For production use, prefer the larger model.
4. Not handling overlapping entities. Some models may produce overlapping entity spans. Always check for this in your post-processing code.
5. Training custom NER with too little data. A few examples are not enough. You typically need at least 200-500 labeled examples per entity type for reasonable accuracy.

Best Practices

1. Choose the right tool for your task. Use spaCy for fast, general-purpose NER. Use Hugging Face transformers when you need maximum accuracy on complex text.
2. Check confidence scores. When using Hugging Face, filter results by confidence score. Entities with scores below 0.8 may be unreliable.
3. Pre-process your text. Clean up extra whitespace, fix encoding issues, and handle special characters before running NER.
4. Validate with domain experts. If building NER for specialized domains (medical, legal, financial), have domain experts review and correct the model's predictions.
5. Use larger models for production. Development and testing can use small models for speed, but production deployments benefit from larger, more accurate models.

Quick Summary

Named Entity Recognition (NER) finds and classifies named things in text — people, organizations, locations, dates, and more. It works by assigning a label to each word using the BIO tagging scheme (Beginning, Inside, Outside). spaCy provides fast, easy-to-use NER with built-in visualization. Hugging Face transformers offer more accurate NER using models like BERT. You can also train custom NER models to recognize domain-specific entities like drug names or legal terms.

Key Points

- NER identifies named entities (people, organizations, locations, dates) in text.
- BIO tagging handles multi-word entities: B = beginning, I = inside, O = outside.
- spaCy offers fast, lightweight NER with built-in visualization tools.
- Hugging Face provides transformer-based NER with higher accuracy and confidence scores.
- Custom NER training requires labeled data and lets you recognize domain-specific entities.
- NER is a token classification task — each word gets a label.
- Always check confidence scores and be aware that NER models can make mistakes on ambiguous words.

Practice Questions

1. What does NER stand for, and what is its purpose? Explain the difference between PERSON, ORG, and GPE entity types with examples.
2. Explain the BIO tagging scheme. Given the sentence "New York University is in Manhattan," what would the BIO tags be for each word?
3. What is the difference between spaCy NER and Hugging Face NER? When would you choose one over the other?
4. Why does the Hugging Face NER pipeline need `aggregation_strategy="simple"`? What happens if you leave it out?

5. You need to build a NER system that detects product names and prices in e-commerce product descriptions. What steps would you follow?

Exercises

Exercise 1: Entity Counter

Write a Python script using spaCy that takes a paragraph of text and produces a summary table showing: (a) how many entities of each type were found, (b) the most frequently mentioned entity, and (c) the percentage of words that are part of an entity.

Exercise 2: NER Comparison

Write a script that runs the same text through both spaCy and Hugging Face NER pipelines and compares the results. Create a table showing which entities each system found, which ones they agreed on, and which ones they disagreed on.

Exercise 3: Custom Entity Extractor

Create a function that uses NER to extract structured data from job posting text. Given text like "Google is hiring a Senior Python Developer in New York with a salary of $150,000," extract the company name, job title, location, and salary into a dictionary.

What Is Next?

Now that you can extract named entities from text, the next chapter covers Text Summarization. You will learn how to automatically condense long documents into short summaries, using both extractive methods (picking important sentences) and abstractive methods (generating new sentences). Text summarization is one of the most popular applications of NLP, used in news aggregation, document processing, and email summarization.

Chapter 7: Text Summarization — Condensing Documents into Key Points

What You Will Learn

- What text summarization is and the two main approaches: extractive and abstractive
- How extractive summarization works by scoring and selecting important sentences
- How abstractive summarization generates new sentences that capture the original meaning
- Building an extractive summarizer from scratch using sentence scoring
- Using the Hugging Face summarization pipeline for abstractive summaries
- Controlling summary length with min_length and max_length parameters
- Evaluating summaries using ROUGE metrics
- A complete end-to-end summarization example

Why This Chapter Matters

Imagine you are a busy manager who receives a 50-page report every morning. You do not have time to read the whole thing. You need someone to give you the key points in a few paragraphs. That is exactly what text summarization does — it takes a long piece of text and creates a shorter version that captures the most important information.

Text summarization is everywhere. News apps show you article previews. Email clients display message snippets. Search engines show brief descriptions under each result. Legal professionals need summaries of long contracts. Researchers need summaries of scientific papers. Doctors need summaries of patient records.

As the amount of text data in the world grows exponentially, the ability to automatically summarize information becomes critical. After this chapter, you will be able to build systems that condense long documents into concise summaries using both traditional and modern AI approaches.

7.1 What Is Text Summarization?

Text summarization is the task of creating a shorter version of a text while keeping its most important information. Think of it like writing a book report in school — you read an entire book and then write a one-page summary of what happened.

The word summarization means making something shorter while keeping the essential meaning. A summary is the shortened version.

```
+-------------------------------------------------------------------+
|              What Text Summarization Does                         |
+-------------------------------------------------------------------+
|                                                                   |
|  Input (long text):                                               |
|  "The Mars rover Perseverance successfully landed on Mars on      |
|   February 18, 2021. The rover is equipped with advanced          |
|   scientific instruments designed to search for signs of          |
|   ancient microbial life. It also carries a small helicopter      |
|   named Ingenuity, which became the first aircraft to achieve    |
|   powered flight on another planet. The mission is part of       |
|   NASA's Mars Exploration Program and cost approximately          |
|   2.7 billion dollars..."                                         |
|                                                                   |
|  Output (summary):                                                |
|  "NASA's Perseverance rover landed on Mars in February 2021       |
|   to search for ancient life. It carries the Ingenuity            |
|   helicopter, the first to fly on another planet."                |
|                                                                   |
+-------------------------------------------------------------------+
```

There are two fundamentally different approaches to summarization:

1. Extractive Summarization — Selects the most important sentences directly from the original text
2. Abstractive Summarization — Generates new sentences that capture the meaning of the original text

Let us understand each approach in detail.

7.2 Extractive vs. Abstractive Summarization

Extractive Summarization

Extractive summarization works like using a highlighter pen on a printed article. You read through the text and highlight the most important sentences. The summary is just those highlighted sentences copied out. No new words are created — everything in the summary comes directly from the original text.

The word extractive comes from "extract," which means to pull out or remove. So extractive summarization pulls out important sentences from the text.

```
+-------------------------------------------------------------------+
|              Extractive Summarization                              |
+-------------------------------------------------------------------+
|                                                                    |
|  Original text (5 sentences):                                      |
|  1. "The company reported record profits this quarter."            |
|  2. "Sales increased by 35% compared to last year."               |
|  3. "The CEO attributed growth to new product launches."           |
|  4. "Employee count grew from 500 to 750."                        |
|  5. "The board approved a new dividend policy."                    |
|                                                                    |
|  Step 1: Score each sentence for importance                        |
|  Sentence 1: Score = 0.85  <-- HIGH                               |
|  Sentence 2: Score = 0.92  <-- HIGHEST                            |
|  Sentence 3: Score = 0.78                                          |
|  Sentence 4: Score = 0.45                                          |
|  Sentence 5: Score = 0.60                                          |
|                                                                    |
|  Step 2: Select top sentences                                      |
|  Summary: "Sales increased by 35% compared to last year.           |
|           The company reported record profits this quarter."       |
|                                                                    |
|  Notice: These sentences are copied exactly from the original.     |
|                                                                    |
+-------------------------------------------------------------------+
```

Advantages of extractive summarization:

- Sentences are grammatically correct (they come from the original)
- No risk of generating incorrect facts
- Simpler to implement
- Faster to compute

Disadvantages of extractive summarization:

- Summaries can feel choppy or disconnected
- Cannot combine ideas from multiple sentences
- May include unnecessary details within selected sentences
- Limited to the exact words in the original text

Abstractive Summarization

Abstractive summarization works like asking a person to read an article and then explain it in their own words. The summary may contain words and phrases that never appeared in the original text. The model understands the meaning and generates new text that captures that meaning.

The word abstractive comes from "abstract," which means a brief summary of a larger work. Abstractive summarization creates an abstract — a new piece of text that conveys the original meaning.

```
+-------------------------------------------------------------------+
|              Abstractive Summarization                            |
+-------------------------------------------------------------------+
|                                                                   |
|  Original text:                                                   |
|  "The company reported record profits this quarter. Sales         |
|   increased by 35% compared to last year. The CEO attributed     |
|   growth to new product launches."                                |
|                                                                   |
|  Abstractive Summary:                                             |
|  "The company achieved unprecedented financial success with       |
|   a 35% sales surge, driven by new product introductions."       |
|                                                                   |
|  Notice: "unprecedented financial success" and "sales surge"      |
|  are NEW phrases not in the original text. The model              |
|  understood the meaning and expressed it differently.             |
|                                                                   |
+-------------------------------------------------------------------+
```

Advantages of abstractive summarization:

- More natural and fluent summaries
- Can combine ideas from multiple sentences
- Can rephrase for clarity
- More like how humans summarize

Disadvantages of abstractive summarization:

- May generate incorrect facts (called hallucination)
- Requires more complex models (usually deep learning)

- Slower to compute
- Harder to verify accuracy

```
+-------------------------------------------------------------------+
|         Extractive vs Abstractive: Side by Side                   |
+-------------------------------------------------------------------+
|                                                                   |
|  Feature          | Extractive        | Abstractive              |
|  -----------------|-------------------|-------------------------  |
|  Method           | Select sentences  | Generate new text        |
|  Output words     | From original     | May be new words         |
|  Grammar          | Always correct    | Usually correct          |
|  Fluency          | Can be choppy     | More natural             |
|  Factual accuracy | High              | Risk of hallucination    |
|  Complexity       | Simpler           | More complex             |
|  Speed            | Faster            | Slower                   |
|  Models used      | Scoring methods   | Seq2seq, Transformers    |
|                                                                   |
+-------------------------------------------------------------------+
```

7.3 Extractive Summarization with Sentence Scoring

Let us build an extractive summarizer from scratch. The idea is simple: we score each sentence based on how important it is, then select the top-scoring sentences.

How Sentence Scoring Works

The most basic approach scores sentences based on word frequency. The idea is that important words appear more often in the text. Sentences that contain many important (frequent) words are likely important sentences.

```
+-------------------------------------------------------------------+
|              Sentence Scoring Process                             |
+-------------------------------------------------------------------+
|                                                                   |
|  Step 1: Count word frequencies in the entire text                |
|    "profits" -> 5 times                                           |
|    "revenue" -> 4 times                                           |
|    "the"     -> 20 times (stop word, ignore)                     |
|    "growth"  -> 3 times                                           |
|                                                                   |
|  Step 2: Score each sentence                                      |
```

```
|    Sentence score = sum of word frequencies in that sentence      |
|                     divided by number of words in the sentence    |
|                                                                   |
|  Step 3: Rank sentences by score                                  |
|                                                                   |
|  Step 4: Select top N sentences for the summary                   |
|                                                                   |
+-------------------------------------------------------------------+
```

Building an Extractive Summarizer

```python
# Extractive summarization using word frequency scoring

import re
from collections import Counter

def extractive_summarize(text, num_sentences=3):
    """
    Summarize text by selecting the most important sentences.

    Parameters:
        text: The input text to summarize
        num_sentences: How many sentences to include in the summary

    Returns:
        A string containing the summary
    """

    # Step 1: Split text into sentences
    # We use a simple regex pattern that splits on periods,
    # exclamation marks, or question marks followed by a space
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())

    # Remove very short sentences (likely not meaningful)
    sentences = [s for s in sentences if len(s.split()) > 3]

    # Step 2: Define stop words (common words that are not important)
    stop_words = {
        'the', 'a', 'an', 'is', 'are', 'was', 'were', 'be', 'been',
        'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will',
        'would', 'could', 'should', 'may', 'might', 'shall', 'can',
        'of', 'in', 'to', 'for', 'with', 'on', 'at', 'by', 'from',
        'as', 'into', 'through', 'during', 'before', 'after', 'and',
        'but', 'or', 'nor', 'not', 'so', 'yet', 'both', 'either',
        'neither', 'each', 'every', 'all', 'any', 'few', 'more',
        'most', 'other', 'some', 'such', 'no', 'only', 'own', 'same',
        'than', 'too', 'very', 'just', 'because', 'if', 'when',
        'that', 'this', 'these', 'those', 'it', 'its', 'he', 'she',
        'they', 'them', 'their', 'we', 'our', 'you', 'your', 'also'
    }
```

```
    # Step 3: Count word frequencies (excluding stop words)
    # We combine all sentences into one big list of words
    all_words = []
    for sentence in sentences:
        words = re.findall(r'\b[a-z]+\b', sentence.lower())
        meaningful_words = [w for w in words if w not in stop_words]
        all_words.extend(meaningful_words)

    # Count how often each word appears
    word_freq = Counter(all_words)

    # Step 4: Score each sentence
    sentence_scores = []
    for i, sentence in enumerate(sentences):
        words = re.findall(r'\b[a-z]+\b', sentence.lower())
        meaningful_words = [w for w in words if w not in stop_words]

        if len(meaningful_words) == 0:
            score = 0
        else:
            # Sum the frequency of each meaningful word in the sentence
            # Divide by total words to avoid bias toward long sentences
            score = sum(word_freq[w] for w in meaningful_words)
            score = score / len(meaningful_words)

        sentence_scores.append((score, i, sentence))

    # Step 5: Select top sentences
    # Sort by score (highest first)
    sentence_scores.sort(key=lambda x: x[0], reverse=True)

    # Take top N sentences
    selected = sentence_scores[:num_sentences]

    # Sort selected sentences by their original position
    # This keeps the summary in the same order as the original text
    selected.sort(key=lambda x: x[1])

    # Step 6: Combine selected sentences into the summary
    summary = ' '.join([s[2] for s in selected])

    return summary


# Let us test our summarizer
article = """
Artificial intelligence is transforming the healthcare industry in remarkable ways.
Doctors are using AI systems to diagnose diseases more accurately and quickly.
Machine learning algorithms can analyze medical images like X-rays and MRIs to
detect cancer at early stages. Natural language processing helps extract important
information from patient records automatically. AI-powered drug discovery is
reducing the time needed to develop new medications from years to months.
Robotic surgery systems guided by AI are making complex procedures safer and
more precise. Wearable devices with AI capabilities can monitor patient health
```

```
in real time and alert doctors to potential problems. Despite these advances,
experts emphasize that AI should assist doctors rather than replace them. The
human element in healthcare, including empathy and complex decision making,
remains irreplaceable. The future of healthcare lies in the collaboration
between human doctors and AI systems.
"""

print("Original text length:", len(article.split()), "words")
print()

summary = extractive_summarize(article, num_sentences=3)
print("Summary (3 sentences):")
print(summary)
print()
print("Summary length:", len(summary.split()), "words")
```

Expected Output:

```
Original text length: 127 words

Summary (3 sentences):
Machine learning algorithms can analyze medical images like X-rays and MRIs to
detect cancer at early stages. Natural language processing helps extract important
information from patient records automatically. AI-powered drug discovery is
reducing the time needed to develop new medications from years to months.

Summary length: 41 words
```

Let us trace through the code line by line:

- Lines 3-4: We import `re` for regular expressions (text pattern matching) and `Counter` for counting word occurrences
- Lines 6-17: We define our function with `text` (the input) and `num_sentences` (how many sentences to keep, default 3)
- Lines 21-22: We split the text into individual sentences using a regex pattern. The pattern `(?<=[.!?])\s+` looks for a period, exclamation mark, or question mark followed by one or more spaces
- Lines 25: We filter out very short sentences (3 words or fewer) because they rarely contain useful information
- Lines 28-37: We define stop words — common English words like "the," "is," and "of" that appear frequently but carry little meaning. We exclude these from our scoring
- Lines 40-45: We loop through all sentences, extract words (lowercase), remove stop words, and collect all meaningful words into one list
- Lines 48: We use `Counter` to count how often each meaningful word appears across the entire text

- Lines 51-63: For each sentence, we calculate its score. The score is the sum of word frequencies for each meaningful word in the sentence, divided by the number of meaningful words. Dividing prevents long sentences from having an unfair advantage
- Lines 67-68: We sort sentences by score from highest to lowest
- Lines 71: We take the top N sentences
- Lines 75-76: We re-sort the selected sentences by their original position in the text, so the summary reads in a natural order
- Lines 79: We join the selected sentences into one string

7.4 Abstractive Summarization with Hugging Face

While our extractive summarizer works, modern AI can do something much more impressive: read the text, understand it, and write a new summary in its own words. This is abstractive summarization, and the Hugging Face `transformers` library makes it easy.

The Summarization Pipeline

Hugging Face provides a pipeline — a pre-built tool that handles all the complex steps automatically. You just give it text and get back a summary.

```python
# Abstractive summarization with Hugging Face

from transformers import pipeline

# Create a summarization pipeline
# This downloads a pre-trained model the first time you run it
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

# The text we want to summarize
article = """
The Amazon rainforest, often referred to as the lungs of the Earth, produces
approximately 20 percent of the world's oxygen. Spanning across nine countries
in South America, it covers an area of about 5.5 million square kilometers.
The rainforest is home to an estimated 10 percent of all species on Earth,
including over 40,000 plant species, 1,300 bird species, and 3,000 types of
fish. However, deforestation has been a growing concern, with approximately
17 percent of the forest lost in the last 50 years. Scientists warn that
continued destruction could push the ecosystem past a tipping point,
transforming large portions of the rainforest into savanna. Conservation
efforts have increased in recent years, with governments and organizations
working to protect remaining forest areas and restore degraded land.
Indigenous communities play a crucial role in forest conservation, as their
```

```
traditional land management practices have proven effective at maintaining
biodiversity and preventing deforestation.
"""

# Generate the summary
summary = summarizer(article, max_length=80, min_length=30, do_sample=False)

# The pipeline returns a list of dictionaries
# Each dictionary has a 'summary_text' key
print("Summary:")
print(summary[0]['summary_text'])
```

Expected Output:

```
Summary:
The Amazon rainforest produces approximately 20 percent of the world's oxygen.
It is home to an estimated 10 percent of all species on Earth. About 17 percent
of the forest has been lost in the last 50 years. Conservation efforts have
increased in recent years.
```

Let us trace through each line:

- Line 3: We import `pipeline` from the `transformers` library. A pipeline wraps a model, tokenizer, and post-processing into one simple function
- Line 7: We create a summarization pipeline. `"summarization"` tells Hugging Face what task we want. `model="facebook/bart-large-cnn"` specifies the BART model fine-tuned on CNN/Daily Mail news articles. BART stands for Bidirectional and Auto-Regressive Transformers. The first time you run this, it downloads the model (about 1.6 GB)
- Line 27: We call the summarizer with three parameters:
  - `article` — the text to summarize
  - `max_length=80` — the summary should be at most 80 tokens long
  - `min_length=30` — the summary should be at least 30 tokens long
  - `do_sample=False` — use deterministic generation (always produces the same output)
- Line 31: The pipeline returns a list. Each element is a dictionary with a `'summary_text'` key containing the generated summary

How the Model Works Inside

The BART model used here follows an encoder-decoder architecture:

```
+-------------------------------------------------------------------+
|          How Abstractive Summarization Works                      |
```

```
+-------------------------------------------------------------------+
|                                                                   |
|  Input Text                                                       |
|  "The Amazon rainforest produces 20% of the world's oxygen..."   |
|       |                                                           |
|       v                                                           |
|  +----------+                                                     |
|  | Tokenizer|  Converts text into numbers the model understands   |
|  +----------+                                                     |
|       |                                                           |
|       v                                                           |
|  +---------+                                                      |
|  | Encoder |  Reads and understands the full input text           |
|  |         |  Creates a "meaning representation"                  |
|  +---------+                                                      |
|       |                                                           |
|       v                                                           |
|  +---------+                                                      |
|  | Decoder |  Generates the summary word by word                  |
|  |         |  Uses the meaning from the encoder                   |
|  +---------+                                                      |
|       |                                                           |
|       v                                                           |
|  Summary Text                                                     |
|  "The Amazon rainforest produces approximately 20 percent..."    |
|                                                                   |
+-------------------------------------------------------------------+
```

The encoder reads the entire input text and creates an internal representation of its meaning. The decoder then generates the summary one word at a time, using the encoder's understanding to decide what words to write. This is similar to how a human reads an article first (encoding) and then writes a summary from memory (decoding).

7.5 Controlling Summary Length

One of the most useful features of abstractive summarization is controlling how long or short the summary should be. You can adjust this with `min_length` and `max_length` parameters.

```python
# Controlling summary length

from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
```

```
text = """
Climate change is one of the most pressing challenges facing humanity today.
Rising global temperatures are causing more frequent and severe weather events,
including hurricanes, droughts, and wildfires. Sea levels are rising due to
melting ice caps, threatening coastal communities around the world. The burning
of fossil fuels like coal, oil, and natural gas releases greenhouse gases into
the atmosphere, trapping heat and warming the planet. Scientists have reached a
strong consensus that human activities are the primary driver of climate change.
To address this crisis, many countries have committed to reducing carbon emissions
through the Paris Agreement. Renewable energy sources such as solar and wind power
are becoming increasingly affordable and widespread. Electric vehicles are replacing
gas-powered cars at an accelerating rate. However, experts warn that current efforts
are not sufficient to limit warming to 1.5 degrees Celsius above pre-industrial
levels, and more aggressive action is needed.
"""

# Short summary (1-2 sentences)
short_summary = summarizer(text, max_length=40, min_length=10, do_sample=False)
print("SHORT SUMMARY:")
print(short_summary[0]['summary_text'])
print(f"Length: {len(short_summary[0]['summary_text'].split())} words")
print()

# Medium summary (3-4 sentences)
medium_summary = summarizer(text, max_length=80, min_length=30, do_sample=False)
print("MEDIUM SUMMARY:")
print(medium_summary[0]['summary_text'])
print(f"Length: {len(medium_summary[0]['summary_text'].split())} words")
print()

# Long summary (5+ sentences)
long_summary = summarizer(text, max_length=150, min_length=60, do_sample=False)
print("LONG SUMMARY:")
print(long_summary[0]['summary_text'])
print(f"Length: {len(long_summary[0]['summary_text'].split())} words")
```

Expected Output:

```
SHORT SUMMARY:
Climate change is one of the most pressing challenges facing humanity today.
Rising global temperatures are causing more frequent and severe weather events.
Length: 23 words

MEDIUM SUMMARY:
Climate change is one of the most pressing challenges facing humanity today.
The burning of fossil fuels releases greenhouse gases into the atmosphere.
Many countries have committed to reducing carbon emissions through the Paris
Agreement. Renewable energy sources are becoming increasingly affordable.
Length: 41 words

LONG SUMMARY:
Climate change is one of the most pressing challenges facing humanity today.
Rising global temperatures are causing more frequent and severe weather events.
```

```
The burning of fossil fuels releases greenhouse gases, trapping heat and warming
the planet. Scientists agree that human activities are the primary driver. Many
countries have committed to reducing emissions through the Paris Agreement.
Renewable energy and electric vehicles are helping, but experts warn current
efforts are not sufficient.
Length: 65 words
```

```
+-------------------------------------------------------------------+
|              Summary Length Control                                |
+-------------------------------------------------------------------+
|                                                                    |
|  Parameter     | What It Controls                                  |
|  --------------|-----------------------------------------------   |
|  max_length    | Maximum number of tokens in the summary           |
|  min_length    | Minimum number of tokens in the summary           |
|  do_sample     | False = deterministic, True = add randomness      |
|                                                                    |
|  Note: A "token" is roughly 0.75 words.                           |
|  So max_length=80 means roughly 60 words maximum.                 |
|                                                                    |
|  Guidelines:                                                       |
|  - Short summary:  max_length=40,  min_length=10                  |
|  - Medium summary: max_length=80,  min_length=30                  |
|  - Long summary:   max_length=150, min_length=60                  |
|  - Very long:      max_length=300, min_length=100                 |
|                                                                    |
+-------------------------------------------------------------------+
```

A token is a piece of text that the model processes. Tokens can be whole words like "cat" or parts of words like "un" and "believable." On average, one word equals about 1.3 tokens, so `max_length=80` tokens is roughly 60 words.

7.6 Evaluating Summaries with ROUGE Metrics

How do you measure whether a summary is good? You cannot just check if it is short — a single word would be short but useless. You need a way to measure how well the summary captures the content of the original text. This is where ROUGE metrics come in.

What Is ROUGE?

ROUGE stands for Recall-Oriented Understudy for Gisting Evaluation. Do not worry about the full name — just remember that ROUGE measures how much overlap exists between a generated summary and a reference (human-written) summary.

Think of it like grading a student's book report by comparing it to the teacher's answer key. The more overlap between the student's answer and the teacher's answer, the higher the score.

There are several types of ROUGE:

```
+-----------------------------------------------------------------+
|              ROUGE Metrics Explained                             |
+-----------------------------------------------------------------+
|                                                                  |
|  ROUGE-1: Overlap of individual words (unigrams)                |
|                                                                  |
|  Reference: "The cat sat on the mat"                            |
|  Generated: "The cat is on the mat"                             |
|  Shared words: the, cat, on, the, mat = 5 out of 6             |
|  ROUGE-1 Recall = 5/6 = 0.83                                    |
|                                                                  |
|  ---                                                             |
|                                                                  |
|  ROUGE-2: Overlap of word pairs (bigrams)                       |
|                                                                  |
|  Reference pairs: "the cat", "cat sat", "sat on",               |
|                   "on the", "the mat"                            |
|  Generated pairs: "the cat", "cat is", "is on",                 |
|                   "on the", "the mat"                            |
|  Shared pairs: "the cat", "on the", "the mat" = 3 out of 5      |
|  ROUGE-2 Recall = 3/5 = 0.60                                    |
|                                                                  |
|  ---                                                             |
|                                                                  |
|  ROUGE-L: Longest Common Subsequence                            |
|                                                                  |
|  Finds the longest sequence of words that appear in the same    |
|  order in both summaries (words do not need to be consecutive)  |
|  Reference: "The cat sat on the mat"                            |
|  Generated: "The cat is on the mat"                             |
|  LCS: "The cat on the mat" (5 words)                            |
|  ROUGE-L Recall = 5/6 = 0.83                                    |
|                                                                  |
+-----------------------------------------------------------------+
```

Each ROUGE type has three sub-scores:

- Precision: Of all the words in the generated summary, how many also appear in the reference?
- Recall: Of all the words in the reference summary, how many also appear in the generated summary?
- F1-score: The balanced average of precision and recall

Computing ROUGE Scores in Python

```python
# Computing ROUGE scores to evaluate summaries
# First install: pip install rouge-score

from rouge_score import rouge_scorer

# Create a scorer that computes ROUGE-1, ROUGE-2, and ROUGE-L
scorer = rouge_scorer.RougeScorer(
    ['rouge1', 'rouge2', 'rougeL'],
    use_stemmer=True  # "running" and "runs" count as the same word
)

# Our reference summary (what a human wrote)
reference = """
NASA's Perseverance rover landed on Mars in February 2021. The rover
searches for signs of ancient microbial life. It carries the Ingenuity
helicopter, which achieved the first powered flight on another planet.
"""

# Our generated summary (what our model produced)
generated = """
The Perseverance rover successfully landed on Mars on February 18, 2021.
It is designed to search for ancient life. The rover also carries
Ingenuity, the first helicopter to fly on another planet.
"""

# Compute ROUGE scores
scores = scorer.score(reference, generated)

print("ROUGE Evaluation Results")
print("=" * 50)

for metric, score in scores.items():
    print(f"\n{metric.upper()}:")
    print(f"  Precision: {score.precision:.4f}")
    print(f"  Recall:    {score.recall:.4f}")
    print(f"  F1-Score:  {score.fmeasure:.4f}")
```

Expected Output:

```
ROUGE Evaluation Results
==================================================

ROUGE1:
  Precision: 0.7647
  Recall:    0.7222
  F1-Score:  0.7429

ROUGE2:
  Precision: 0.4516
  Recall:    0.4242
  F1-Score:  0.4375
```

```
ROUGEL:
  Precision: 0.6471
  Recall:    0.6111
  F1-Score:  0.6286
```

Let us trace through the code:

- Line 4: We import `rouge_scorer` from the `rouge_score` library. Install it with `pip install rouge-score`
- Lines 7-10: We create a `RougeScorer` object. We specify which ROUGE types to compute: `rouge1` (single words), `rouge2` (word pairs), and `rougeL` (longest common subsequence). `use_stemmer=True` means "running" and "runs" are treated as the same word. Stemming reduces words to their root form
- Lines 13-22: We define a reference summary (what a human would write) and a generated summary (what our model produced)
- Line 25: We compute the scores by comparing the reference and generated summaries
- Lines 27-33: We print each metric's precision, recall, and F1-score

Interpreting ROUGE Scores

```
+-------------------------------------------------------------------+
|              Understanding ROUGE Scores                            |
+-------------------------------------------------------------------+
|                                                                    |
|  Score Range     | Interpretation                                  |
|  ----------------|-----------------------------------------------  |
|  0.0 - 0.2      | Poor overlap, summaries are very different     |
|  0.2 - 0.4      | Some overlap, captures some key ideas          |
|  0.4 - 0.6      | Moderate overlap, decent summary               |
|  0.6 - 0.8      | Good overlap, captures most key ideas          |
|  0.8 - 1.0      | Excellent overlap, very similar to reference   |
|                                                                    |
|  Important notes:                                                   |
|  - A perfect score of 1.0 means identical text                    |
|  - ROUGE-2 is usually lower than ROUGE-1 (harder to match pairs) |
|  - ROUGE does not measure factual correctness                      |
|  - A good summary can have moderate ROUGE if it uses different    |
|    words to express the same ideas                                 |
|                                                                    |
+-------------------------------------------------------------------+
```

7.7 Complete Summarization Example

Let us put everything together in a complete example that combines both extractive and abstractive summarization, along with evaluation.

```python
# Complete text summarization example

import re
from collections import Counter
from transformers import pipeline
from rouge_score import rouge_scorer


# ============================================================
# PART 1: The article to summarize
# ============================================================

article = """
Electric vehicles are rapidly transforming the global automotive industry.
Sales of electric cars have grown by over 40 percent annually for the past
three years, with China, Europe, and the United States leading adoption.
Battery technology has improved dramatically, with modern lithium-ion batteries
offering ranges of over 300 miles on a single charge. The cost of batteries has
fallen by nearly 90 percent over the past decade, making electric vehicles
increasingly affordable for average consumers. Major automakers including Ford,
General Motors, and Volkswagen have announced plans to phase out internal
combustion engines within the next 10 to 15 years. Charging infrastructure is
expanding rapidly, with governments investing billions of dollars in public
charging networks. However, challenges remain, including long charging times
compared to gasoline refueling, limited charging stations in rural areas, and
concerns about the environmental impact of battery mining and disposal. Despite
these challenges, industry analysts predict that electric vehicles will account
for over 50 percent of new car sales globally by 2030. The transition to
electric vehicles is seen as a critical step in reducing greenhouse gas
emissions from the transportation sector, which currently accounts for
approximately 16 percent of global emissions.
"""

print("ORIGINAL ARTICLE")
print("=" * 60)
print(article.strip())
print(f"\nWord count: {len(article.split())} words")
print()


# ============================================================
# PART 2: Extractive Summary
# ============================================================

def extractive_summarize(text, num_sentences=3):
    """Select the most important sentences from the text."""
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    sentences = [s for s in sentences if len(s.split()) > 3]

    stop_words = {
```

```
        'the', 'a', 'an', 'is', 'are', 'was', 'were', 'be', 'been',
        'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
        'of', 'in', 'to', 'for', 'with', 'on', 'at', 'by', 'from',
        'as', 'and', 'but', 'or', 'not', 'that', 'this', 'it', 'its',
        'he', 'she', 'they', 'them', 'their', 'we', 'our', 'you',
        'your', 'also', 'than', 'more', 'over', 'which', 'including'
    }

    all_words = []
    for sentence in sentences:
        words = re.findall(r'\b[a-z]+\b', sentence.lower())
        meaningful_words = [w for w in words if w not in stop_words]
        all_words.extend(meaningful_words)

    word_freq = Counter(all_words)

    sentence_scores = []
    for i, sentence in enumerate(sentences):
        words = re.findall(r'\b[a-z]+\b', sentence.lower())
        meaningful_words = [w for w in words if w not in stop_words]
        if len(meaningful_words) == 0:
            score = 0
        else:
            score = sum(word_freq[w] for w in meaningful_words)
            score = score / len(meaningful_words)
        sentence_scores.append((score, i, sentence))

    sentence_scores.sort(key=lambda x: x[0], reverse=True)
    selected = sentence_scores[:num_sentences]
    selected.sort(key=lambda x: x[1])

    return ' '.join([s[2] for s in selected])

extractive_result = extractive_summarize(article, num_sentences=3)
print("EXTRACTIVE SUMMARY")
print("=" * 60)
print(extractive_result)
print(f"\nWord count: {len(extractive_result.split())} words")
print()


# ============================================================
# PART 3: Abstractive Summary
# ============================================================

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
abstractive_result = summarizer(
    article, max_length=80, min_length=30, do_sample=False
)[0]['summary_text']

print("ABSTRACTIVE SUMMARY")
print("=" * 60)
print(abstractive_result)
print(f"\nWord count: {len(abstractive_result.split())} words")
print()
```

```
# ============================================================
# PART 4: Evaluate both summaries
# ============================================================

# A human-written reference summary for comparison
reference_summary = """
Electric vehicle sales are growing rapidly worldwide, driven by improved battery
technology and falling costs. Major automakers plan to end combustion engine
production within 15 years. While challenges like charging infrastructure and
battery environmental concerns persist, EVs are expected to reach 50 percent of
global sales by 2030, helping reduce transportation emissions.
"""

scorer = rouge_scorer.RougeScorer(
    ['rouge1', 'rouge2', 'rougeL'], use_stemmer=True
)

print("EVALUATION (ROUGE Scores)")
print("=" * 60)

# Score extractive summary
ext_scores = scorer.score(reference_summary, extractive_result)
print("\nExtractive Summary Scores:")
for metric, score in ext_scores.items():
    print(f"  {metric}: F1={score.fmeasure:.4f}")

# Score abstractive summary
abs_scores = scorer.score(reference_summary, abstractive_result)
print("\nAbstractive Summary Scores:")
for metric, score in abs_scores.items():
    print(f"  {metric}: F1={score.fmeasure:.4f}")

# Compare
print("\nComparison:")
print(f"  Extractive ROUGE-1 F1:  {ext_scores['rouge1'].fmeasure:.4f}")
print(f"  Abstractive ROUGE-1 F1: {abs_scores['rouge1'].fmeasure:.4f}")

if abs_scores['rouge1'].fmeasure > ext_scores['rouge1'].fmeasure:
    print("  --> Abstractive summary scores higher on ROUGE-1")
else:
    print("  --> Extractive summary scores higher on ROUGE-1")
```

Expected Output:

```
ORIGINAL ARTICLE
============================================================
Electric vehicles are rapidly transforming the global automotive industry...
[full article text]

Word count: 168 words
```

```
EXTRACTIVE SUMMARY
============================================================
Sales of electric cars have grown by over 40 percent annually for the past
three years, with China, Europe, and the United States leading adoption. The
cost of batteries has fallen by nearly 90 percent over the past decade, making
electric vehicles increasingly affordable for average consumers. Despite these
challenges, industry analysts predict that electric vehicles will account for
over 50 percent of new car sales globally by 2030.

Word count: 62 words

ABSTRACTIVE SUMMARY
============================================================
Electric vehicle sales have grown by over 40 percent annually for the past
three years. Battery costs have fallen by nearly 90 percent over the past
decade. Major automakers have announced plans to phase out internal combustion
engines. Industry analysts predict EVs will account for over 50 percent of new
car sales by 2030.

Word count: 51 words

EVALUATION (ROUGE Scores)
============================================================

Extractive Summary Scores:
  rouge1: F1=0.4800
  rouge2: F1=0.2353
  rougeL: F1=0.3200

Abstractive Summary Scores:
  rouge1: F1=0.5455
  rouge2: F1=0.3125
  rougeL: F1=0.4000

Comparison:
  Extractive ROUGE-1 F1:  0.4800
  Abstractive ROUGE-1 F1: 0.5455
  --> Abstractive summary scores higher on ROUGE-1
```

7.8 Summarizing Multiple Documents

In real applications, you often need to summarize multiple related documents. Here is how to handle that:

```python
# Summarizing multiple documents
```

```
from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

# Multiple related articles
documents = [
    """
    Tesla reported record quarterly revenue of 25.2 billion dollars. The company
    delivered over 466,000 vehicles in the quarter, a 20 percent increase from
    the previous year. CEO Elon Musk highlighted the growing demand for the
    Model Y, which became the best-selling car globally.
    """,
    """
    Ford announced a 2 billion dollar investment in electric vehicle manufacturing.
    The company plans to produce 600,000 EVs annually by the end of next year.
    Ford's F-150 Lightning electric pickup truck has received over 200,000
    reservations from customers.
    """,
    """
    General Motors revealed plans to launch 30 new electric vehicle models by 2025.
    The company is investing 35 billion dollars in EV and autonomous vehicle
    development. GM aims to have an all-electric lineup by 2035 and achieve
    carbon neutrality by 2040.
    """
]

print("Individual Document Summaries:")
print("=" * 60)

all_summaries = []
for i, doc in enumerate(documents):
    summary = summarizer(doc, max_length=40, min_length=15, do_sample=False)
    summary_text = summary[0]['summary_text']
    all_summaries.append(summary_text)
    print(f"\nDocument {i + 1} Summary:")
    print(f"  {summary_text}")

# Combine individual summaries into one text
combined = ' '.join(all_summaries)

# Summarize the combined summaries for a mega-summary
print("\n\nCombined Summary of All Documents:")
print("=" * 60)
mega_summary = summarizer(combined, max_length=60, min_length=20, do_sample=False)
print(mega_summary[0]['summary_text'])
```

Expected Output:

```
Individual Document Summaries:
============================================================

Document 1 Summary:
  Tesla reported record quarterly revenue of 25.2 billion dollars. The company
```

```
  delivered over 466,000 vehicles in the quarter.

Document 2 Summary:
  Ford announced a 2 billion dollar investment in electric vehicle manufacturing.
  The F-150 Lightning has received over 200,000 reservations.

Document 3 Summary:
  General Motors revealed plans to launch 30 new electric vehicle models by 2025.
  GM aims to have an all-electric lineup by 2035.

Combined Summary of All Documents:
============================================================
Tesla, Ford, and GM are all investing heavily in electric vehicles. Tesla
reported record revenue while Ford and GM announced major EV production plans
and investments totaling billions of dollars.
```

Common Mistakes

1. Input text too short: Summarization models work best with text that is at least a few sentences long. Trying to summarize a single sentence often produces strange results or just returns the same sentence.
2. Input text too long: Most models have a maximum input length (typically 1024 tokens for BART). If your text exceeds this, you need to truncate it or split it into chunks and summarize each chunk separately.
3. Setting min_length greater than max_length: This causes an error. Always ensure `min_length` is less than `max_length`.
4. Trusting abstractive summaries blindly: Abstractive models can generate facts that were not in the original text (hallucination). Always verify important facts in the summary against the original text.
5. Using ROUGE as the only evaluation: ROUGE measures word overlap, not meaning. A summary could have low ROUGE but still be excellent if it uses different words to express the same ideas. Use ROUGE as one signal among many.

Best Practices

1. Choose the right approach for your use case: Use extractive summarization when factual accuracy is critical (legal, medical). Use abstractive when you need fluent, readable summaries.

2. Preprocess your input: Remove headers, footers, navigation text, and other noise before summarizing. Clean input produces better summaries.
3. Experiment with length parameters: Try different `min_length` and `max_length` values to find the right summary length for your application.
4. Handle long documents with chunking: If your document exceeds the model's maximum input length, split it into overlapping chunks, summarize each chunk, and then summarize the summaries.
5. Combine extractive and abstractive: First use extractive summarization to reduce a very long document to its key sentences, then use abstractive summarization on those sentences for a fluent final summary.

Quick Summary

Text summarization condenses long text into shorter versions. Extractive summarization selects important sentences from the original text using scoring methods like word frequency. Abstractive summarization uses deep learning models (like BART) to generate new sentences that capture the original meaning. The Hugging Face `pipeline("summarization")` makes abstractive summarization easy with just a few lines of code. ROUGE metrics evaluate summary quality by measuring word overlap between generated and reference summaries. ROUGE-1 measures single-word overlap, ROUGE-2 measures word-pair overlap, and ROUGE-L measures the longest common subsequence.

Key Points

- Text summarization creates shorter versions of text while preserving key information
- Extractive summarization selects sentences directly; abstractive generates new text
- Extractive is simpler and more factually reliable; abstractive is more fluent and natural
- Sentence scoring based on word frequency is a basic but effective extractive method
- The Hugging Face summarization pipeline uses models like BART for abstractive summarization
- Control summary length with `min_length` and `max_length` parameters
- ROUGE metrics (ROUGE-1, ROUGE-2, ROUGE-L) measure overlap between generated and reference summaries
- Abstractive models may hallucinate facts not present in the original text

- For very long documents, use chunking to stay within the model's input limit

---

Practice Questions

1. What is the difference between extractive and abstractive summarization? Give a real-world analogy for each approach.
2. In our extractive summarizer, why do we divide the sentence score by the number of meaningful words? What would happen if we did not?
3. What does ROUGE-2 measure, and why is it usually lower than ROUGE-1 for the same pair of summaries?
4. A summary has ROUGE-1 precision of 0.90 but ROUGE-1 recall of 0.30. What does this tell you about the summary?
5. Why might an abstractive summary score lower on ROUGE than an extractive summary, even if the abstractive summary is actually better to a human reader?

Exercises

Exercise 1: Improve the Extractive Summarizer Modify the `extractive_summarize` function to also consider sentence position as a scoring factor. Sentences at the beginning and end of the text are often more important (the "lead bias" in news articles). Add a position weight: multiply the score by 1.5 for the first sentence, 1.2 for the last sentence, and 1.0 for all others.

Exercise 2: Summary Comparison Tool Write a program that takes an article and generates three different summaries: one extractive, one abstractive with `max_length=40`, and one abstractive with `max_length=100`. Compare all three using ROUGE scores against a reference summary you write by hand. Display the results in a formatted table.

Exercise 3: Long Document Summarizer Build a function called `summarize_long_document(text, chunk_size=500)` that handles documents longer than 1024 tokens. The function should split the text into chunks of approximately `chunk_size` words each, summarize each chunk using the Hugging Face pipeline, combine the chunk summaries, and then produce a final summary of the combined summaries.

What Is Next?

In this chapter, you learned how to condense long texts into short summaries using both extractive and abstractive methods. But summarization is just one type of sequence-to-sequence task — tasks where you take a sequence of text as input and produce a different sequence as output. In the next chapter, you will learn about the sequence-to-sequence architecture that powers not only summarization but also machine translation, question answering, and many other tasks. You will see how the encoder-decoder design works in detail and build a translation system that converts text from one language to another.

Chapter 8: Sequence-to-Sequence Models — Turning One Sequence into Another

What You Will Learn

- What sequence-to-sequence (seq2seq) models are and why they matter
- How the encoder-decoder architecture works step by step
- What machine translation is and how seq2seq models handle it
- What teacher forcing is and why it speeds up training
- How the attention mechanism solves the bottleneck problem
- Using the Hugging Face translation pipeline for easy translation
- A complete translation example with multiple language pairs

Why This Chapter Matters

Imagine you are a translator who speaks both English and French. When someone speaks an English sentence to you, your brain does two things: first, you understand (encode) the full English sentence, then you produce (decode) the French translation word by word. This is exactly how sequence-to-sequence models work.

Seq2seq models are the backbone of modern AI. They power machine translation (Google Translate), text summarization (which you learned in the last chapter), chatbots, speech recognition, and even image captioning. The encoder-decoder architecture is one of the most important ideas in deep learning, and it led directly to the Transformer architecture that drives today's AI revolution.

Understanding seq2seq models gives you the foundation to understand BERT, GPT, and every other modern language model. This chapter is a gateway to the most exciting

developments in AI.

8.1 What Is a Sequence-to-Sequence Task?

A sequence-to-sequence (seq2seq) task takes a sequence of items as input and produces a different sequence of items as output. The input and output can have different lengths.

The word sequence means an ordered list of items. In NLP, a sequence is usually a series of words forming a sentence. Seq2seq is short for "sequence-to-sequence."

```
+-------------------------------------------------------------------+
|              Sequence-to-Sequence Tasks                            |
+-------------------------------------------------------------------+
|                                                                    |
|  Task                  | Input Sequence     | Output Sequence      |
|  ----------------------|--------------------|--------------------- |
|  Translation           | "I love cats"      | "J'aime les chats"  |
|  Summarization         | Long article       | Short summary        |
|  Question Answering    | Question + Context | Answer               |
|  Chatbot               | User message       | Bot response         |
|  Speech Recognition    | Audio waveform     | Text transcript      |
|  Image Captioning      | Image pixels       | Text description     |
|                                                                    |
|  Key insight: Input and output lengths can be DIFFERENT.           |
|  "I love cats" (3 words) --> "J'aime les chats" (3 words)       |
|  "How are you" (3 words) --> "Comment allez-vous" (2 words)     |
|                                                                    |
+-------------------------------------------------------------------+
```

This is different from tasks like text classification, where the input is a sequence but the output is a single label. In seq2seq, both the input AND the output are sequences.

8.2 The Encoder-Decoder Architecture

The encoder-decoder architecture is the core design pattern for seq2seq models. Think of it like a two-step process:

1. Encoder: Reads the entire input sequence and compresses it into a fixed-size representation (like reading a book and forming a mental understanding)

2. Decoder: Takes that representation and generates the output sequence one element at a time (like writing a summary from your mental understanding)

```
+-------------------------------------------------------------------+
|         The Encoder-Decoder Architecture                          |
+-------------------------------------------------------------------+
|                                                                   |
|  Input: "The cat sat"                                             |
|                                                                   |
|  ENCODER (reads input left to right):                             |
|                                                                   |
|  "The" --> [h1] --> "cat" --> [h2] --> "sat" --> [h3]            |
|                                                                   |
|  h1, h2, h3 are "hidden states" - the encoder's internal         |
|  understanding at each step.                                      |
|                                                                   |
|  The final hidden state [h3] is called the "context vector."     |
|  It summarizes the ENTIRE input sequence into one vector.         |
|                                                                   |
|                      [h3] = Context Vector                        |
|                           |                                       |
|                           v                                       |
|                                                                   |
|  DECODER (generates output left to right):                        |
|                                                                   |
|  [h3] --> "Le" --> "chat" --> "assis" --> <END>                  |
|                                                                   |
|  The decoder uses the context vector to generate each             |
|  output word, one at a time, until it produces an end token.     |
|                                                                   |
+-------------------------------------------------------------------+
```

Hidden States

A hidden state is a vector (a list of numbers) that represents what the model has learned so far. Think of it as the model's "memory" at each time step.

As the encoder reads each input word, it updates its hidden state. After reading "The," the hidden state captures the meaning of "The." After reading "cat," it captures "The cat." After reading "sat," it captures the meaning of the entire sentence "The cat sat." This final hidden state is called the context vector.

The Context Vector

The context vector is the final hidden state of the encoder. It is a fixed-size vector (for example, a list of 256 or 512 numbers) that tries to capture the meaning of the entire input sequence.

Think of the context vector like a zip file — it compresses all the information from a long input into a compact representation. The decoder then "unzips" this representation to generate the output.

```
+-------------------------------------------------------------------+
|              The Context Vector                                    |
+-------------------------------------------------------------------+
|                                                                    |
|  Input sentence: "I love learning about artificial intelligence" |
|  (6 words)                                                         |
|                                                                    |
|        Encoder processes word by word:                             |
|        "I" -> "love" -> "learning" -> "about" -> "artificial"   |
|        -> "intelligence"                                           |
|                                                                    |
|        Final hidden state = Context Vector                         |
|        [0.23, -0.45, 0.89, 0.12, ..., -0.67]                     |
|        (a vector of, say, 512 numbers)                             |
|                                                                    |
|  This single vector must capture the ENTIRE meaning of the       |
|  input sentence. This is hard for very long sentences!             |
|                                                                    |
+-------------------------------------------------------------------+
```

Building a Simple Encoder-Decoder in PyTorch

Let us build a simple seq2seq model to see how the pieces fit together:

```python
# A simple encoder-decoder model in PyTorch

import torch
import torch.nn as nn

class Encoder(nn.Module):
    """
    The encoder reads the input sequence and produces a context vector.
    It uses an RNN (Recurrent Neural Network) to process the sequence.
    """

    def __init__(self, input_size, embedding_size, hidden_size):
        super().__init__()
        # Embedding layer: converts word indices to dense vectors
        # Think of it as a lookup table where each word has a vector
        self.embedding = nn.Embedding(input_size, embedding_size)

        # GRU layer: processes the sequence step by step
        # GRU = Gated Recurrent Unit (a type of RNN)
        self.rnn = nn.GRU(embedding_size, hidden_size, batch_first=True)
```

```
    def forward(self, x):
        # x shape: (batch_size, sequence_length)
        # Each element is a word index like 42 meaning "cat"

        # Convert word indices to vectors
        embedded = self.embedding(x)
        # embedded shape: (batch_size, sequence_length, embedding_size)

        # Process through the RNN
        outputs, hidden = self.rnn(embedded)
        # outputs: hidden states for ALL time steps
        # hidden: the FINAL hidden state (this is our context vector)

        return outputs, hidden


class Decoder(nn.Module):
    """
    The decoder takes the context vector and generates the output
    sequence one word at a time.
    """

    def __init__(self, output_size, embedding_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(output_size, embedding_size)
        self.rnn = nn.GRU(embedding_size, hidden_size, batch_first=True)

        # Linear layer: converts hidden state to word probabilities
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x, hidden):
        # x shape: (batch_size, 1) - one word at a time
        # hidden: the context vector from the encoder

        embedded = self.embedding(x)
        output, hidden = self.rnn(embedded, hidden)

        # Convert hidden state to a probability for each possible word
        prediction = self.fc(output)

        return prediction, hidden


class Seq2Seq(nn.Module):
    """
    The complete seq2seq model combining encoder and decoder.
    """

    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, source, target):
```

```
        # source: input sequence (e.g., English sentence)
        # target: output sequence (e.g., French sentence)

        batch_size = source.shape[0]
        target_length = target.shape[1]
        target_vocab_size = self.decoder.fc.out_features

        # Store decoder outputs
        outputs = torch.zeros(batch_size, target_length, target_vocab_size)

        # Step 1: Encode the input sequence
        _, hidden = self.encoder(source)
        # hidden is now the context vector

        # Step 2: Decode one word at a time
        # Start with the first target word
        decoder_input = target[:, 0:1]  # First word (usually <START>)

        for t in range(1, target_length):
            # Generate next word prediction
            output, hidden = self.decoder(decoder_input, hidden)
            outputs[:, t:t+1, :] = output

            # Use the actual next target word as input (teacher forcing)
            decoder_input = target[:, t:t+1]

        return outputs


# Create the model
INPUT_VOCAB_SIZE = 5000    # Number of words in source language
OUTPUT_VOCAB_SIZE = 6000   # Number of words in target language
EMBEDDING_SIZE = 256       # Size of word vectors
HIDDEN_SIZE = 512          # Size of hidden states

encoder = Encoder(INPUT_VOCAB_SIZE, EMBEDDING_SIZE, HIDDEN_SIZE)
decoder = Decoder(OUTPUT_VOCAB_SIZE, EMBEDDING_SIZE, HIDDEN_SIZE)
model = Seq2Seq(encoder, decoder)

# Show model summary
total_params = sum(p.numel() for p in model.parameters())
print("Seq2Seq Model Architecture")
print("=" * 50)
print(f"Input vocabulary size:  {INPUT_VOCAB_SIZE}")
print(f"Output vocabulary size: {OUTPUT_VOCAB_SIZE}")
print(f"Embedding size:         {EMBEDDING_SIZE}")
print(f"Hidden size:            {HIDDEN_SIZE}")
print(f"Total parameters:       {total_params:,}")

# Test with dummy data
source = torch.randint(0, INPUT_VOCAB_SIZE, (2, 10))   # batch=2, length=10
target = torch.randint(0, OUTPUT_VOCAB_SIZE, (2, 12))   # batch=2, length=12

output = model(source, target)
print(f"\nInput shape:  {source.shape}  (batch_size=2, source_length=10)")
```

```
print(f"Target shape: {target.shape}  (batch_size=2, target_length=12)")
print(f"Output shape: {output.shape}  (batch_size=2, target_length=12, vocab_size={OUTP...
```

Expected Output:

```
Seq2Seq Model Architecture
==================================================
Input vocabulary size:  5000
Output vocabulary size: 6000
Embedding size:         256
Hidden size:            512
Total parameters:       7,679,464

Input shape:  torch.Size([2, 10])  (batch_size=2, source_length=10)
Target shape: torch.Size([2, 12])  (batch_size=2, target_length=12)
Output shape: torch.Size([2, 12, 6000])  (batch_size=2, target_length=12, vocab_size=6000)
```

Let us trace through the key parts:

- Encoder.init: Creates an embedding layer to convert word indices into dense vectors, and a GRU layer to process the sequence. GRU (Gated Recurrent Unit) is a type of recurrent neural network that maintains a hidden state as it processes each word
- Encoder.forward: Takes word indices, converts them to embeddings, and passes them through the GRU. Returns all hidden states and the final hidden state (context vector)
- Decoder.init: Similar to encoder, plus a linear layer (`fc`) that converts hidden states to word probabilities
- Decoder.forward: Takes one word and the previous hidden state, produces a prediction for the next word and an updated hidden state
- Seq2Seq.forward: First encodes the entire input, then decodes one word at a time using the context vector

---

8.3 Machine Translation — A Key Seq2Seq Application

Machine translation is the task of automatically converting text from one language to another. It is the most well-known application of seq2seq models.

```
+-----------------------------------------------------------------+
|              Machine Translation                                |
```

```
+-----------------------------------------------------------------+
|                                                                  |
|  English:  "The weather is beautiful today"                     |
|  French:   "Le temps est magnifique aujourd'hui"                |
|  Spanish:  "El clima es hermoso hoy"                            |
|  German:   "Das Wetter ist heute schoen"                        |
|                                                                  |
|  Challenges:                                                     |
|  1. Word order differs between languages                         |
|     English: "I eat an apple"                                   |
|     Japanese: "I apple eat" (verb at end)                       |
|                                                                  |
|  2. One word may map to multiple words                           |
|     English: "I am" --> Spanish: "Yo soy" (or just "soy")       |
|                                                                  |
|  3. Gender and agreement                                         |
|     English: "The red car" (no gender)                          |
|     French: "La voiture rouge" (feminine)                       |
|     Spanish: "El coche rojo" (masculine)                        |
|                                                                  |
|  4. Idioms don't translate word by word                         |
|     English: "It's raining cats and dogs"                       |
|     French: "Il pleut des cordes" ("It rains ropes")            |
|                                                                  |
+-----------------------------------------------------------------+
```

Machine translation is hard because languages have different word orders, grammar rules, and idioms. A seq2seq model handles this by learning the patterns during training, rather than using hand-coded rules.

8.4 Teacher Forcing — Training the Decoder

When training a seq2seq model, we face a question: when the decoder generates one word, should we use that generated word as input for the next step, or should we use the correct (target) word?

Teacher forcing is a training technique where we feed the decoder the correct target word at each step, even if it predicted wrong at the previous step.

```
+-----------------------------------------------------------------+
|              Teacher Forcing vs. Free Running                    |
+-----------------------------------------------------------------+
|                                                                  |
|  Target translation: "Le chat dort" (The cat sleeps)            |
|                                                                  |
```

```
|  WITHOUT Teacher Forcing (Free Running):                          |
|  Step 1: Input=<START>  --> Predicted="Le"    (correct!)         |
|  Step 2: Input="Le"     --> Predicted="chien" (wrong! "dog")     |
|  Step 3: Input="chien"  --> Predicted="mange" (wrong! "eats")    |
|  The error snowballs! One mistake causes more mistakes.           |
|                                                                   |
|  WITH Teacher Forcing:                                            |
|  Step 1: Input=<START>  --> Predicted="Le"    (correct!)         |
|  Step 2: Input="chat"   --> Predicted="dort"  (correct!)         |
|  Step 3: Input="dort"   --> Predicted=<END>   (correct!)         |
|  Even if Step 1 was wrong, Step 2 gets the CORRECT input.        |
|  This prevents error accumulation during training.                |
|                                                                   |
+-------------------------------------------------------------------+
```

The word "teacher" in teacher forcing refers to the training data acting like a teacher who corrects the student at each step. Instead of letting the student's mistakes pile up, the teacher provides the right answer so the student can learn the correct pattern.

```python
# Demonstrating teacher forcing concept

import torch

def demonstrate_teacher_forcing():
    """
    Show the difference between teacher forcing and free running.
    This is a conceptual demonstration, not a real model.
    """

    # Pretend these are our target words (what we want the model to output)
    target_words = ["<START>", "Le", "chat", "dort", "<END>"]

    # Pretend predictions (some correct, some wrong)
    model_predictions = ["Le", "chien", "mange", "vite", "<END>"]

    print("Teacher Forcing (uses correct target at each step):")
    print("=" * 55)
    for step in range(len(target_words) - 1):
        input_word = target_words[step]      # Always use correct word
        target_word = target_words[step + 1]  # What we want
        predicted = model_predictions[step]    # What model says
        correct = "YES" if predicted == target_word else "NO"
        print(f"  Step {step + 1}: Input='{input_word}' "
              f"--> Target='{target_word}', "
              f"Predicted='{predicted}' "
              f"(Correct: {correct})")

    print()
    print("Free Running (uses model's own prediction at each step):")
    print("=" * 55)
    current_input = "<START>"
```

```
    for step in range(len(target_words) - 1):
        target_word = target_words[step + 1]
        predicted = model_predictions[step]
        correct = "YES" if predicted == target_word else "NO"
        print(f"  Step {step + 1}: Input='{current_input}' "
              f"--> Target='{target_word}', "
              f"Predicted='{predicted}' "
              f"(Correct: {correct})")
        current_input = predicted  # Use the prediction as next input

demonstrate_teacher_forcing()
```

Expected Output:

```
Teacher Forcing (uses correct target at each step):
=======================================================
  Step 1: Input='<START>' --> Target='Le', Predicted='Le' (Correct: YES)
  Step 2: Input='Le' --> Target='chat', Predicted='chien' (Correct: NO)
  Step 3: Input='chat' --> Target='dort', Predicted='mange' (Correct: NO)
  Step 4: Input='dort' --> Target='<END>', Predicted='vite' (Correct: NO)

Free Running (uses model's own prediction at each step):
=======================================================
  Step 1: Input='<START>' --> Target='Le', Predicted='Le' (Correct: YES)
  Step 2: Input='Le' --> Target='chat', Predicted='chien' (Correct: NO)
  Step 3: Input='chien' --> Target='dort', Predicted='mange' (Correct: NO)
  Step 4: Input='mange' --> Target='<END>', Predicted='vite' (Correct: NO)
```

Teacher Forcing Ratio

In practice, we use a teacher forcing ratio — a probability that determines whether to use teacher forcing at each step. A ratio of 1.0 means always use teacher forcing. A ratio of 0.0 means never use it. A common strategy is to start with a high ratio (0.9) and gradually decrease it during training.

```
+-------------------------------------------------------------------+
|              Teacher Forcing Ratio                                 |
+-------------------------------------------------------------------+
|                                                                   |
|  Ratio = 1.0: Always use correct target (pure teacher forcing)   |
|  Ratio = 0.5: 50% chance of using target, 50% using prediction  |
|  Ratio = 0.0: Always use model's prediction (free running)       |
|                                                                   |
|  Training Strategy:                                               |
|  Epoch 1-5:   ratio = 1.0 (learn basic patterns)                |
|  Epoch 5-10:  ratio = 0.7 (start using own predictions)         |
|  Epoch 10-15: ratio = 0.5 (balanced)                             |
|  Epoch 15-20: ratio = 0.3 (mostly self-reliant)                 |
```

```
|                                                                  |
|  This gradual reduction is called "scheduled sampling."          |
|                                                                  |
+------------------------------------------------------------------+
```

8.5 The Attention Mechanism — Solving the Bottleneck

The basic encoder-decoder model has a serious problem: the context vector bottleneck. The entire input sequence must be compressed into a single fixed-size vector. For short sentences, this works fine. For long sentences or paragraphs, too much information is lost.

```
+------------------------------------------------------------------+
|              The Bottleneck Problem                              |
+------------------------------------------------------------------+
|                                                                  |
|  Short sentence (works well):                                    |
|  "I eat" --> [context vector] --> "Je mange"                     |
|  Only 2 words to compress. Easy!                                 |
|                                                                  |
|  Long sentence (problematic):                                    |
|  "The incredibly talented young musician from the small          |
|   town in southern France played a beautiful melody on           |
|   her antique violin at the prestigious concert last evening"    |
|                                                                  |
|  --> [context vector] (same size as before!)                     |
|                                                                  |
|  25 words squeezed into the SAME size vector.                    |
|  Important details get lost!                                     |
|                                                                  |
+------------------------------------------------------------------+
```

How Attention Works

Attention solves the bottleneck by letting the decoder look back at ALL encoder hidden states, not just the final one. At each decoding step, the decoder decides which parts of the input are most relevant for generating the current output word.

Think of attention like a spotlight. When translating "The cat sat on the mat," and the decoder is generating the French word for "cat" ("chat"), the attention spotlight shines brightest on the encoder's representation of "cat" in the input.

```
+-------------------------------------------------------------------+
|              The Attention Mechanism                              |
+-------------------------------------------------------------------+
|                                                                   |
|  Input:  "The  cat  sat  on  the  mat"                           |
|  Output: "Le   chat ..."                                          |
|                                                                   |
|  Encoder hidden states: [h1] [h2] [h3] [h4] [h5] [h6]          |
|                          The  cat  sat  on   the  mat             |
|                                                                   |
|  When decoder generates "chat" (cat):                             |
|                                                                   |
|  Attention weights:                                               |
|  [h1] [h2] [h3] [h4] [h5] [h6]                                   |
|  0.05 0.80 0.05 0.02 0.05 0.03   <-- weights sum to 1.0         |
|       ^^^^                                                        |
|       The decoder "pays attention" mostly to h2 ("cat")          |
|                                                                   |
|  Context for "chat" = 0.05*h1 + 0.80*h2 + 0.05*h3 + ...        |
|                                                                   |
|  This weighted sum gives a FOCUSED context vector                |
|  specifically for generating this word.                           |
|                                                                   |
+-------------------------------------------------------------------+
```

Attention Step by Step

```
+-------------------------------------------------------------------+
|              Attention Calculation Steps                          |
+-------------------------------------------------------------------+
|                                                                   |
|  Step 1: SCORE                                                    |
|  Calculate how relevant each encoder state is to the current      |
|  decoder state.                                                   |
|  score(h_i, s_j) = how related is encoder state i to             |
|                    decoder state j                                |
|                                                                   |
|  Step 2: NORMALIZE (Softmax)                                      |
|  Convert scores to probabilities that sum to 1.                   |
|  attention_weights = softmax(scores)                              |
|                                                                   |
|  Step 3: WEIGHTED SUM                                             |
|  Multiply each encoder state by its attention weight and sum.    |
|  context = sum(weight_i * h_i)                                    |
|                                                                   |
|  Step 4: COMBINE                                                  |
|  Concatenate the context with the decoder state.                  |
|  Use this combined vector to predict the next word.               |
|                                                                   |
+-------------------------------------------------------------------+
```

```python
# Demonstrating attention weights conceptually

import torch
import torch.nn.functional as F

def demonstrate_attention():
    """
    Show how attention weights focus on different input words
    when generating different output words.
    """

    # Input words (English)
    input_words = ["The", "black", "cat", "sat", "on", "the", "mat"]

    # Output words (French)
    output_words = ["Le", "chat", "noir", "etait", "assis", "sur", "le", "tapis"]

    # Simulated attention weights for each output word
    # Each row shows where the decoder "looks" when generating that word
    attention_data = {
        "Le":     [0.60, 0.05, 0.05, 0.05, 0.10, 0.10, 0.05],
        "chat":   [0.05, 0.10, 0.70, 0.05, 0.02, 0.03, 0.05],
        "noir":   [0.03, 0.75, 0.10, 0.02, 0.02, 0.03, 0.05],
        "etait":  [0.05, 0.05, 0.10, 0.60, 0.05, 0.05, 0.10],
        "assis":  [0.03, 0.02, 0.05, 0.70, 0.05, 0.05, 0.10],
        "sur":    [0.02, 0.02, 0.03, 0.05, 0.75, 0.08, 0.05],
        "le":     [0.05, 0.03, 0.02, 0.05, 0.10, 0.65, 0.10],
        "tapis":  [0.02, 0.03, 0.02, 0.03, 0.05, 0.05, 0.80],
    }

    print("Attention Weights Visualization")
    print("=" * 65)
    print()
    print(f"{'Output':<10}", end="")
    for word in input_words:
        print(f"{word:<10}", end="")
    print()
    print("-" * 80)

    for out_word, weights in attention_data.items():
        print(f"{out_word:<10}", end="")
        for w in weights:
            # Show high weights with stars for visibility
            bar = "*" * int(w * 20)
            print(f"{bar:<10}", end="")
        print()

    print()
    print("Key insight: When generating 'chat' (cat), the model")
    print("focuses heavily on the input word 'cat' (weight=0.70).")
    print("When generating 'noir' (black), it focuses on 'black' (0.75).")
    print("This is the attention mechanism at work!")

demonstrate_attention()
```

Expected Output:

```
Attention Weights Visualization
================================================================

Output    The       black     cat       sat       on        the       mat
--------------------------------------------------------------------------------
Le        ************                            **        **
chat      *         **        **************       *                  *
noir                ***************       **                          *
etait     *         *         **        *************                 **
assis               *         *         **************                **
sur                           *         *         ***************      *
le        *                   *         **        *************
tapis                                   *         *         ****************

Key insight: When generating 'chat' (cat), the model
focuses heavily on the input word 'cat' (weight=0.70).
When generating 'noir' (black), it focuses on 'black' (0.75).
This is the attention mechanism at work!
```

Notice an interesting pattern: in English, "black cat" is adjective-noun order, but in French it is "chat noir" (noun-adjective). The attention mechanism handles this naturally — when generating "chat," it looks at "cat," and when generating "noir," it looks at "black." The model learns these reordering patterns from data.

8.6 Using the Hugging Face Translation Pipeline

Just like with summarization, Hugging Face provides an easy-to-use pipeline for translation. You do not need to build models from scratch — pre-trained models handle all the complexity.

```python
# Machine translation with Hugging Face

from transformers import pipeline

# Create a translation pipeline (English to French)
translator_en_fr = pipeline(
    "translation",
    model="Helsinki-NLP/opus-mt-en-fr"
)

# Translate some English sentences to French
sentences = [
    "Hello, how are you today?",
```

```
    "The weather is beautiful this morning.",
    "I love learning about artificial intelligence.",
    "Where is the nearest train station?",
    "Thank you for your help."
]

print("English to French Translation")
print("=" * 60)

for sentence in sentences:
    result = translator_en_fr(sentence)
    translation = result[0]['translation_text']
    print(f"\n  English: {sentence}")
    print(f"  French:  {translation}")
```

Expected Output:

```
English to French Translation
============================================================

  English: Hello, how are you today?
  French:  Bonjour, comment allez-vous aujourd'hui ?

  English: The weather is beautiful this morning.
  French:  Le temps est magnifique ce matin.

  English: I love learning about artificial intelligence.
  French:  J'adore apprendre l'intelligence artificielle.

  English: Where is the nearest train station?
  French:  Où est la gare la plus proche ?

  English: Thank you for your help.
  French:  Merci de votre aide.
```

Let us trace through the code:

- Line 3: Import the `pipeline` function from `transformers`
- Lines 6-9: Create a translation pipeline. `"translation"` specifies the task. `model="Helsinki-NLP/opus-mt-en-fr"` specifies the OPUS-MT model for English-to-French translation. OPUS-MT is a collection of translation models trained on the OPUS parallel corpus (millions of translated sentence pairs)
- Lines 12-18: Define the English sentences we want to translate
- Lines 23-26: For each sentence, we call the translator. It returns a list of dictionaries, each with a `'translation_text'` key

Translating Between Multiple Languages

```python
# Translating between multiple language pairs

from transformers import pipeline

# Create pipelines for different language pairs
translators = {
    "English -> French": pipeline("translation", model="Helsinki-NLP/opus-mt-en-fr"),
    "English -> German": pipeline("translation", model="Helsinki-NLP/opus-mt-en-de"),
    "English -> Spanish": pipeline("translation", model="Helsinki-NLP/opus-mt-en-es"),
    "French -> English": pipeline("translation", model="Helsinki-NLP/opus-mt-fr-en"),
}

# The sentence to translate
text = "Artificial intelligence is changing the world."

print("Multi-Language Translation")
print("=" * 60)
print(f"\nOriginal (English): {text}\n")

for direction, translator in translators.items():
    result = translator(text)
    print(f"  {direction}:")
    print(f"    {result[0]['translation_text']}")
    print()
```

Expected Output:

```
Multi-Language Translation
============================================================

Original (English): Artificial intelligence is changing the world.

  English -> French:
    L'intelligence artificielle change le monde.

  English -> German:
    Kuenstliche Intelligenz veraendert die Welt.

  English -> Spanish:
    La inteligencia artificial esta cambiando el mundo.

  French -> English:
    Artificial intelligence is changing the world.
```

Notice the round-trip: translating to French and then back to English produces the original sentence. This is a good sign that the translation models are accurate.

8.7 Complete Translation Example

Let us build a more complete translation application that handles multiple sentences and provides helpful formatting:

```python
# Complete translation application

from transformers import pipeline

class SimpleTranslator:
    """
    A simple translation tool that supports multiple language pairs.
    """

    # Mapping of language pair names to model names
    MODELS = {
        ("en", "fr"): "Helsinki-NLP/opus-mt-en-fr",
        ("en", "de"): "Helsinki-NLP/opus-mt-en-de",
        ("en", "es"): "Helsinki-NLP/opus-mt-en-es",
        ("en", "it"): "Helsinki-NLP/opus-mt-en-it",
        ("fr", "en"): "Helsinki-NLP/opus-mt-fr-en",
        ("de", "en"): "Helsinki-NLP/opus-mt-de-en",
        ("es", "en"): "Helsinki-NLP/opus-mt-es-en",
    }

    LANGUAGE_NAMES = {
        "en": "English",
        "fr": "French",
        "de": "German",
        "es": "Spanish",
        "it": "Italian",
    }

    def __init__(self):
        # Cache loaded pipelines to avoid reloading
        self._pipelines = {}

    def _get_pipeline(self, source_lang, target_lang):
        """Load or retrieve a cached translation pipeline."""
        key = (source_lang, target_lang)
        if key not in self._pipelines:
            if key not in self.MODELS:
                available = [
                    f"{s} -> {t}" for s, t in self.MODELS.keys()
                ]
                raise ValueError(
                    f"Language pair {source_lang}->{target_lang} "
                    f"not supported. Available: {available}"
                )
            model_name = self.MODELS[key]
            self._pipelines[key] = pipeline(
```

```
                "translation", model=model_name
            )
        return self._pipelines[key]

    def translate(self, text, source_lang, target_lang):
        """Translate text from source to target language."""
        translator = self._get_pipeline(source_lang, target_lang)
        result = translator(text)
        return result[0]['translation_text']

    def translate_paragraph(self, text, source_lang, target_lang):
        """Translate a paragraph by translating each sentence."""
        import re
        sentences = re.split(r'(?<=[.!?])\s+', text.strip())

        translations = []
        for sentence in sentences:
            if sentence.strip():
                translated = self.translate(
                    sentence, source_lang, target_lang
                )
                translations.append(translated)

        return ' '.join(translations)


# Create translator
translator = SimpleTranslator()

# Example 1: Simple sentence translation
print("Example 1: Simple Sentence Translation")
print("=" * 55)

text = "Machine learning is a subset of artificial intelligence."

for target in ["fr", "de", "es"]:
    result = translator.translate(text, "en", target)
    lang_name = translator.LANGUAGE_NAMES[target]
    print(f"  {lang_name}: {result}")

print()

# Example 2: Paragraph translation
print("Example 2: Paragraph Translation (English to French)")
print("=" * 55)

paragraph = (
    "The sun was setting behind the mountains. "
    "Birds were singing their evening songs. "
    "A gentle breeze carried the scent of flowers."
)

print(f"\n  Original:\n  {paragraph}")

translated_paragraph = translator.translate_paragraph(
```

```
    paragraph, "en", "fr"
)
print(f"\n  French:\n  {translated_paragraph}")

print()

# Example 3: Round-trip translation
print("Example 3: Round-Trip Translation")
print("=" * 55)

original = "I enjoy reading books about science."
print(f"\n  Original (EN):  {original}")

# English -> French
french = translator.translate(original, "en", "fr")
print(f"  EN -> FR:       {french}")

# French -> English
back = translator.translate(french, "fr", "en")
print(f"  FR -> EN:       {back}")

match = "YES" if original.lower().rstrip('.') in back.lower() else "SIMILAR"
print(f"  Round-trip match: {match}")
```

Expected Output:

```
Example 1: Simple Sentence Translation
=======================================================
  French: L'apprentissage automatique est un sous-ensemble de l'intelligence artificielle.
  German: Maschinelles Lernen ist eine Teilmenge der kuenstlichen Intelligenz.
  Spanish: El aprendizaje automatico es un subconjunto de la inteligencia artificial.

Example 2: Paragraph Translation (English to French)
=======================================================

  Original:
  The sun was setting behind the mountains. Birds were singing their evening songs. A g...

  French:
  Le soleil se couchait derriere les montagnes. Les oiseaux chantaient leurs chansons d...

Example 3: Round-Trip Translation
=======================================================

  Original (EN):  I enjoy reading books about science.
  EN -> FR:       J'aime lire des livres sur la science.
  FR -> EN:       I like to read books about science.
  Round-trip match: SIMILAR
```

Common Mistakes

1. Forgetting about input length limits: Most seq2seq models have maximum input lengths. If your text is too long, either truncate it or split it into chunks. The Hugging Face pipeline handles truncation automatically, but you should be aware of it.
2. Not understanding token vs. word: Models process tokens, not words. A single word might be multiple tokens. When setting `max_length`, remember that tokens are slightly smaller than words.
3. Using the wrong model for a language pair: Each translation model handles a specific language pair. Using `opus-mt-en-fr` to translate German to English will produce garbage. Always check that the model matches your source and target languages.
4. Ignoring attention: If you are building a seq2seq model from scratch, always include an attention mechanism. Without attention, performance drops dramatically for longer sequences.
5. Over-relying on teacher forcing: Using teacher forcing 100% of the time during training can make the model fragile at inference time (when it must use its own predictions). Gradually reduce the teacher forcing ratio during training.

Best Practices

1. Use pre-trained models when possible: Training a translation model from scratch requires millions of sentence pairs and significant compute. Pre-trained models from Hugging Face work well for most use cases.
2. Keep sentences short for better translation: Very long sentences are harder to translate accurately. If possible, break long sentences into shorter ones before translating.
3. Verify translations for critical content: Machine translation is impressive but not perfect. For important documents (legal, medical), always have a human review the translation.
4. Cache loaded models: Loading a model takes time. If you need to translate many sentences, load the pipeline once and reuse it, as shown in our `SimpleTranslator` class.
5. Consider the domain: Translation models trained on news articles may perform poorly on medical or legal text. Use domain-specific models when available.

Quick Summary

Sequence-to-sequence (seq2seq) models convert one sequence into another using an encoder-decoder architecture. The encoder reads the input and creates a context vector. The decoder uses this context vector to generate the output one word at a time. Teacher forcing trains the decoder by feeding it correct target words instead of its own predictions, preventing error accumulation. The basic context vector creates a bottleneck for long sequences, which the attention mechanism solves by letting the decoder look at all encoder states and focus on the most relevant ones. Hugging Face provides pre-trained translation models that handle all this complexity automatically.

Key Points

- Seq2seq models handle tasks where both input and output are sequences of different lengths
- The encoder reads the input and produces hidden states; the decoder generates output word by word
- The context vector (encoder's final hidden state) summarizes the input but creates a bottleneck
- Teacher forcing feeds correct target words during training to prevent error snowballing
- The attention mechanism lets the decoder focus on relevant parts of the input at each step
- Attention weights show which input words influence each output word
- Hugging Face provides pre-trained translation models for many language pairs
- The OPUS-MT models (Helsinki-NLP) support dozens of language pairs
- Seq2seq with attention is the foundation for the Transformer architecture (BERT, GPT)

Practice Questions

1. Why does the encoder-decoder architecture have a "bottleneck" problem? How does the attention mechanism solve it?
2. Explain teacher forcing using an everyday analogy (not the book's analogy). Why is it helpful during training but potentially harmful if used too much?
3. In the attention visualization, why does the French word "noir" (black) attend strongly to the English word "black" even though they appear in different positions in the sentence?

4. What would happen if you used a model trained for English-to-French translation to translate German text? Why?
5. Why is round-trip translation (English to French and back to English) not always perfect? Give two reasons.

Exercises

Exercise 1: Attention Weights Visualizer Create a function that takes two lists (input words and output words) and a matrix of attention weights, and prints a formatted table showing where the attention is focused. Use different symbols to represent different attention levels: "." for low (< 0.1), "o" for medium (0.1-0.4), "O" for high (0.4-0.7), and "#" for very high (> 0.7).

Exercise 2: Multi-Hop Translation Write a program that translates a sentence from English to French, then from French to German (using French-to-English and English-to-German since direct French-German might not be available). Compare the result with a direct English-to-German translation. How much quality is lost in the multi-hop approach?

Exercise 3: Translation Quality Checker Build a tool that translates a sentence to another language and back, then uses ROUGE scores (from Chapter 7) to measure how similar the round-trip translation is to the original. Test with 10 different sentences of varying complexity and plot which types of sentences survive round-trip translation best.

What Is Next?

You now understand the encoder-decoder architecture that powers seq2seq models. In the next chapter, you will learn about BERT — a model that revolutionized NLP by using only the encoder part of this architecture. BERT reads text bidirectionally (both left-to-right and right-to-left simultaneously), giving it a deep understanding of language. You will see how BERT is pre-trained, how its WordPiece tokenizer works, and how to use it with Hugging Face for tasks like fill-in-the-blank and text classification.

Chapter 9: BERT — Understanding Language by Reading in Both Directions

What You Will Learn

- What BERT is and why "bidirectional" is its key innovation
- How Masked Language Model (MLM) pre-training works
- How Next Sentence Prediction (NSP) pre-training works
- What the WordPiece tokenizer does and why it matters
- The special [CLS] and [SEP] tokens and their roles
- Using BERT with Hugging Face for practical tasks
- BERT variants: DistilBERT, RoBERTa, and ALBERT
- A complete fill-mask pipeline example

Why This Chapter Matters

Before BERT arrived in 2018, NLP models read text in only one direction — either left to right or right to left. This is like trying to understand a sentence while covering half of it with your hand. BERT changed everything by reading text in both directions at once, understanding each word in the context of ALL the words around it.

BERT is one of the most important breakthroughs in the history of NLP. It set new records on almost every NLP task when it was released, and its ideas form the foundation of modern language understanding. Even today, BERT and its variants are widely used in production systems for search engines, chatbots, text classification, question answering, and more.

Understanding BERT is essential for anyone working with NLP. It is the starting point for understanding all modern language models, including GPT, T5, and the models powering

today's AI assistants.

9.1 What Is BERT?

BERT stands for Bidirectional Encoder Representations from Transformers. Let us break down this name:

- Bidirectional: Reads text in both directions (left-to-right AND right-to-left) simultaneously
- Encoder: Uses the encoder part of the Transformer architecture (from Chapter 8)
- Representations: Creates numerical representations (vectors) of text that capture meaning
- Transformers: Built on the Transformer architecture, which uses attention mechanisms

```
+-----------------------------------------------------------------+
|              Why Bidirectional Matters                           |
+-----------------------------------------------------------------+
|                                                                  |
|  Consider the word "bank" in these sentences:                    |
|                                                                  |
|  1. "I deposited money in the bank."                            |
|  2. "I sat on the river bank."                                  |
|                                                                  |
|  LEFT-TO-RIGHT model reading sentence 1:                         |
|  "I" -> "deposited" -> "money" -> "in" -> "the" -> "bank"       |
|  When it reaches "bank", it has seen the LEFT context:           |
|  "I deposited money in the" --> probably a financial bank        |
|                                                                  |
|  RIGHT-TO-LEFT model reading sentence 2:                         |
|  "bank" -> "river" -> "the" -> "on" -> "sat" -> "I"             |
|  When it reaches "bank", it has seen the RIGHT context:          |
|  "river the on sat I" --> probably a river bank                  |
|                                                                  |
|  BERT (BIDIRECTIONAL) reads BOTH directions at once:            |
|  For "bank" in sentence 1, it sees:                             |
|  LEFT: "I deposited money in the"                               |
|  RIGHT: "." (end of sentence)                                   |
|  BOTH directions together --> definitely a financial bank        |
|                                                                  |
|  BERT gets the FULL picture for every word.                     |
|                                                                  |
+-----------------------------------------------------------------+
```

Think of it this way: if someone tells you "I need to go to the _ to get some cash," you can guess the blank is "bank" or "ATM" because you see words on BOTH sides. A left-to-right model would only see "I need to go to the" — not enough context. A right-to-left model would

only see "to get some cash" — also not enough. BERT sees everything at once, just like you do.

BERT Architecture Overview

```
+-----------------------------------------------------------------+
|              BERT Architecture                                  |
+-----------------------------------------------------------------+
|                                                                 |
|  Input:  [CLS] The cat sat on the mat [SEP]                    |
|           |    |   |   |   |   |   |    |                       |
|           v    v   v   v   v   v   v    v                       |
|  +-------------------------------------------------------+      |
|  |           Token Embeddings                            |      |
|  |    +  Position Embeddings                             |      |
|  |    +  Segment Embeddings                              |      |
|  +-------------------------------------------------------+      |
|           |    |   |   |   |   |   |    |                       |
|           v    v   v   v   v   v   v    v                       |
|  +-------------------------------------------------------+      |
|  |        Transformer Encoder Layer 1                    |      |
|  |        (Self-Attention + Feed Forward)                |      |
|  +-------------------------------------------------------+      |
|           |    |   |   |   |   |   |    |                       |
|           v    v   v   v   v   v   v    v                       |
|  +-------------------------------------------------------+      |
|  |        Transformer Encoder Layer 2                    |      |
|  +-------------------------------------------------------+      |
|           |    |   |   |   |   |   |    |                       |
|           v    v   v   v   v   v   v    v                       |
|          ...  ... ... ... ... ... ...  ...                      |
|           |    |   |   |   |   |   |    |                       |
|           v    v   v   v   v   v   v    v                       |
|  +-------------------------------------------------------+      |
|  |        Transformer Encoder Layer 12 (or 24)           |      |
|  +-------------------------------------------------------+      |
|           |    |   |   |   |   |   |    |                       |
|           v    v   v   v   v   v   v    v                       |
|  Output: [h0] [h1][h2][h3][h4][h5][h6] [h7]                    |
|           |                                                     |
|       [CLS] output                                              |
|       Used for classification tasks                             |
|                                                                 |
|  BERT-base:  12 layers, 768 hidden size, 110M parameters        |
|  BERT-large: 24 layers, 1024 hidden size, 340M parameters       |
|                                                                 |
+-----------------------------------------------------------------+
```

BERT uses only the encoder part of the Transformer architecture. Remember from Chapter 8 that the Transformer has an encoder and a decoder. BERT uses just the encoder because its goal is to understand text, not to generate new text. (Generating text is GPT's job, which you

will learn in the next chapter.)

9.2 Masked Language Model (MLM) Pre-Training

BERT learns to understand language through a clever training trick called Masked Language Modeling (MLM). Here is how it works:

1. Take a sentence
2. Randomly hide (mask) some words
3. Ask BERT to predict the hidden words
4. BERT must use the context from BOTH sides to guess correctly

This is like a fill-in-the-blank exercise in school.

```
+-------------------------------------------------------------------+
|              Masked Language Modeling (MLM)                        |
+-------------------------------------------------------------------+
|                                                                    |
|  Original:  "The cat sat on the mat"                              |
|                                                                    |
|  Step 1: Randomly select 15% of tokens to mask                    |
|                                                                    |
|  Masked:    "The [MASK] sat on the mat"                           |
|                                                                    |
|  Step 2: BERT reads the masked sentence                            |
|                                                                    |
|  BERT sees: "The" + [MASK] + "sat on the mat"                    |
|                                                                    |
|  Step 3: BERT predicts what goes in [MASK]                        |
|                                                                    |
|  BERT predicts: "cat" (confidence: 0.89)                          |
|                  "dog" (confidence: 0.06)                          |
|                  "bird" (confidence: 0.02)                         |
|                  ...                                               |
|                                                                    |
|  Step 4: Compare prediction to the actual word "cat"              |
|  If correct: the model is learning well                           |
|  If wrong: adjust the model's parameters                          |
|                                                                    |
|  The key insight: BERT MUST look at BOTH sides to make            |
|  good predictions. "The ___ sat on the mat" requires              |
|  understanding the full context.                                   |
|                                                                    |
+-------------------------------------------------------------------+
```

The 80-10-10 Rule

When BERT selects tokens to mask, it does not always replace them with `[MASK]`. Instead, it uses the 80-10-10 rule:

```
+-------------------------------------------------------------------+
|              The 80-10-10 Masking Rule                            |
+-------------------------------------------------------------------+
|                                                                   |
|  Of the 15% of tokens selected for prediction:                   |
|                                                                   |
|  80% are replaced with [MASK]                                    |
|    "The cat sat" --> "The [MASK] sat"                            |
|                                                                   |
|  10% are replaced with a random word                              |
|    "The cat sat" --> "The banana sat"                             |
|                                                                   |
|  10% are kept unchanged                                           |
|    "The cat sat" --> "The cat sat" (but model must still predict) |
|                                                                   |
|  Why? If we ALWAYS used [MASK], the model would learn a trick:   |
|  "Only predict when I see [MASK]." But during real use, there    |
|  is no [MASK] token. The 80-10-10 rule teaches the model to     |
|  always be ready to understand any word in context.              |
|                                                                   |
+-------------------------------------------------------------------+
```

python

```python
# Demonstrating the Masked Language Model concept

import random

def demonstrate_mlm(sentence, mask_probability=0.15):
    """
    Show how BERT masks words during training.
    This is a conceptual demonstration.
    """

    words = sentence.split()
    original = words.copy()
    masked_positions = []
    actions = []

    for i, word in enumerate(words):
        if random.random() < mask_probability:
            masked_positions.append(i)
            rand_val = random.random()

            if rand_val < 0.8:
                # 80% of the time: replace with [MASK]
                words[i] = "[MASK]"
```

```
                actions.append(f"Position {i}: '{original[i]}' -> [MASK]")
            elif rand_val < 0.9:
                # 10% of the time: replace with random word
                random_word = random.choice(
                    ["apple", "blue", "quickly", "house", "jumped"]
                )
                words[i] = random_word
                actions.append(
                    f"Position {i}: '{original[i]}' -> '{random_word}' (random)"
                )
            else:
                # 10% of the time: keep unchanged
                actions.append(
                    f"Position {i}: '{original[i]}' -> '{original[i]}' (unchanged)"
                )

    print("Masked Language Model Demonstration")
    print("=" * 55)
    print(f"\nOriginal:  {' '.join(original)}")
    print(f"Masked:    {' '.join(words)}")
    print(f"\nTokens selected for prediction ({len(masked_positions)}):")
    for action in actions:
        print(f"  {action}")
    print(f"\nBERT must predict the original words at the masked positions.")


# Set seed for reproducibility
random.seed(42)

demonstrate_mlm(
    "The quick brown fox jumps over the lazy dog and runs through the forest",
    mask_probability=0.20
)
```

Expected Output:

```
Masked Language Model Demonstration
=======================================================

Original:  The quick brown fox jumps over the lazy dog and runs through the forest
Masked:    The quick brown fox jumps over the lazy [MASK] and runs through the forest

Tokens selected for prediction (1):
  Position 8: 'dog' -> [MASK]

BERT must predict the original words at the masked positions.
```

9.3 Next Sentence Prediction (NSP) Pre-Training

BERT has a second pre-training task called Next Sentence Prediction (NSP). The model receives two sentences and must predict whether the second sentence follows the first one in the original text.

```
+-------------------------------------------------------------------+
|              Next Sentence Prediction (NSP)                       |
+-------------------------------------------------------------------+
|                                                                    |
|  POSITIVE example (sentence B follows sentence A):                |
|  Sentence A: "The dog chased the ball in the park."               |
|  Sentence B: "It ran happily across the grass."                   |
|  Label: IsNext (these sentences are consecutive)                  |
|                                                                    |
|  NEGATIVE example (sentence B is random):                         |
|  Sentence A: "The dog chased the ball in the park."               |
|  Sentence B: "The stock market closed at record highs."           |
|  Label: NotNext (these sentences are unrelated)                   |
|                                                                    |
|  During training, 50% of pairs are real consecutive               |
|  sentences (IsNext) and 50% are random pairs (NotNext).           |
|                                                                    |
|  Why does this help?                                               |
|  It teaches BERT to understand relationships BETWEEN              |
|  sentences, not just individual words. This is crucial            |
|  for tasks like question answering, where BERT must               |
|  understand how a question relates to a passage.                  |
|                                                                    |
+-------------------------------------------------------------------+
```

python

```python
# Demonstrating Next Sentence Prediction concept

nsp_examples = [
    {
        "sentence_a": "The Eiffel Tower is located in Paris.",
        "sentence_b": "It was built in 1889 for the World's Fair.",
        "label": "IsNext",
        "explanation": "Sentence B naturally follows A (talking about same topic)"
    },
    {
        "sentence_a": "The Eiffel Tower is located in Paris.",
        "sentence_b": "Bananas are a good source of potassium.",
        "label": "NotNext",
        "explanation": "Sentence B is random and unrelated to A"
    },
    {
        "sentence_a": "Machine learning requires large datasets.",
        "sentence_b": "The more data you have, the better the model performs.",
        "label": "IsNext",
        "explanation": "Sentence B continues the topic from A"
    },
    {
```

```
        "sentence_a": "Machine learning requires large datasets.",
        "sentence_b": "The weather in London is often rainy.",
        "label": "NotNext",
        "explanation": "Sentence B is completely unrelated to A"
    },
]

print("Next Sentence Prediction Examples")
print("=" * 60)

for i, example in enumerate(nsp_examples):
    print(f"\nExample {i + 1}:")
    print(f"  Sentence A: {example['sentence_a']}")
    print(f"  Sentence B: {example['sentence_b']}")
    print(f"  Label:      {example['label']}")
    print(f"  Why:        {example['explanation']}")
```

Expected Output:

```
Next Sentence Prediction Examples
============================================================

Example 1:
  Sentence A: The Eiffel Tower is located in Paris.
  Sentence B: It was built in 1889 for the World's Fair.
  Label:      IsNext
  Why:        Sentence B naturally follows A (talking about same topic)

Example 2:
  Sentence A: The Eiffel Tower is located in Paris.
  Sentence B: Bananas are a good source of potassium.
  Label:      NotNext
  Why:        Sentence B is random and unrelated to A

Example 3:
  Sentence A: Machine learning requires large datasets.
  Sentence B: The more data you have, the better the model performs.
  Label:      IsNext
  Why:        Sentence B continues the topic from A

Example 4:
  Sentence A: Machine learning requires large datasets.
  Sentence B: The weather in London is often rainy.
  Label:      NotNext
  Why:        Sentence B is completely unrelated to A
```

9.4 The WordPiece Tokenizer

BERT uses a special tokenizer called WordPiece. Unlike simple tokenizers that split text into whole words, WordPiece breaks words into smaller pieces called subwords.

Why Subwords?

The problem with whole-word tokenizers is vocabulary size. English has millions of words if you count all forms: "run," "runs," "running," "runner," "rerun," etc. It is impossible to have every word in the vocabulary. When the tokenizer encounters an unknown word, it marks it as `[UNK]` (unknown), and all information about that word is lost.

WordPiece solves this by breaking rare words into smaller, known pieces:

```
+-------------------------------------------------------------------+
|              WordPiece Tokenization                               |
+-------------------------------------------------------------------+
|                                                                   |
|  Common words stay whole:                                         |
|  "the"  --> ["the"]                                              |
|  "cat"  --> ["cat"]                                              |
|  "is"   --> ["is"]                                               |
|                                                                   |
|  Less common words are split:                                     |
|  "playing"    --> ["play", "##ing"]                              |
|  "unhappy"    --> ["un", "##happy"]                              |
|  "embeddings" --> ["em", "##bed", "##ding", "##s"]              |
|                                                                   |
|  The "##" prefix means "this piece is a continuation of          |
|  the previous piece, not a new word."                             |
|                                                                   |
|  Very rare words are broken into small pieces:                    |
|  "pneumonia" --> ["p", "##ne", "##um", "##onia"]                |
|                                                                   |
|  Benefit: Even rare or misspelled words can be represented       |
|  because common subword pieces can combine in new ways.          |
|                                                                   |
+-------------------------------------------------------------------+
```

```python
# Demonstrating WordPiece tokenization with BERT

from transformers import BertTokenizer

# Load BERT's tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Examples to tokenize
examples = [
    "Hello world",
    "I love machine learning",
```

```
    "The cat is playing happily",
    "Electroencephalography is a medical technique",
    "Transformers revolutionized NLP",
    "BERT uses bidirectional representations",
]

print("WordPiece Tokenization Examples")
print("=" * 60)

for text in examples:
    tokens = tokenizer.tokenize(text)
    token_ids = tokenizer.encode(text, add_special_tokens=False)

    print(f"\n  Text:      {text}")
    print(f"  Tokens:    {tokens}")
    print(f"  Token IDs: {token_ids}")
    print(f"  Count:     {len(tokens)} tokens from {len(text.split())} words")
```

Expected Output:

```
WordPiece Tokenization Examples
============================================================

  Text:      Hello world
  Tokens:    ['hello', 'world']
  Token IDs: [7592, 2088]
  Count:     2 tokens from 2 words

  Text:      I love machine learning
  Tokens:    ['i', 'love', 'machine', 'learning']
  Token IDs: [1045, 2293, 3698, 4083]
  Count:     4 tokens from 4 words

  Text:      The cat is playing happily
  Tokens:    ['the', 'cat', 'is', 'playing', 'happily']
  Token IDs: [1996, 4937, 2003, 2652, 14628]
  Count:     5 tokens from 5 words

  Text:      Electroencephalography is a medical technique
  Tokens:    ['electro', '##ence', '##pha', '##log', '##raphy', 'is', 'a', 'medical', '...
  Token IDs: [22327, 6528, 21026, 21469, 18959, 2003, 1037, 3315, 6028]
  Count:     9 tokens from 5 words

  Text:      Transformers revolutionized NLP
  Tokens:    ['transformers', 'revolutionary', '##ized', 'nl', '##p']
  Token IDs: [19081, 7565, 3550, 17953, 2361]
  Count:     5 tokens from 3 words

  Text:      BERT uses bidirectional representations
  Tokens:    ['bert', 'uses', 'bid', '##ire', '##ction', '##al', 'representations']
  Token IDs: [14324, 3594, 7226, 14663, 6553, 2389, 9840]
  Count:     7 tokens from 4 words
```

Notice that common words like "the," "cat," and "love" remain as single tokens. But the rare word "electroencephalography" is split into five smaller pieces. The `##` prefix indicates that a token is a continuation of the previous word, not a separate word.

9.5 Special Tokens: [CLS] and [SEP]

BERT uses two special tokens that you will see everywhere:

[CLS] — Classification Token

The `[CLS]` token is always placed at the very beginning of the input. Its output representation is used as a summary of the entire input for classification tasks.

Think of `[CLS]` as a "summary spot." After BERT processes all the words, the hidden state at the `[CLS]` position contains information about the entire input sequence. We can use this for tasks like "Is this review positive or negative?" or "Do these two sentences mean the same thing?"

[SEP] — Separator Token

The `[SEP]` token separates different parts of the input. When BERT receives two sentences (for tasks like NSP or question answering), `[SEP]` marks where one sentence ends and the other begins. It is also placed at the very end of the input.

```
+-------------------------------------------------------------------+
|              Special Tokens in BERT                               |
+-------------------------------------------------------------------+
|                                                                   |
|  Single sentence input:                                           |
|  [CLS] I love this movie [SEP]                                   |
|    |                        |                                    |
|    v                        v                                    |
|  Classification        End of input                               |
|  output here                                                      |
|                                                                   |
|  Two sentence input:                                              |
|  [CLS] How old are you ? [SEP] I am 25 years old . [SEP]        |
|    |                        |                           |         |
|    v                        v                           v         |
|  Classification     End of sentence 1            End of input     |
|  output here                                                      |
|                                                                   |
|  Segment Embeddings:                                              |
```

```
|  [CLS] How old are you ? [SEP] I am 25 years old . [SEP]        |
|    A    A   A   A   A  A   A   B  B  B   B    B  B   B           |
|                                                                  |
|  Segment A = first sentence                                      |
|  Segment B = second sentence                                     |
|                                                                  |
+------------------------------------------------------------------+
```

```python
# Demonstrating special tokens in BERT

from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Single sentence
text = "I love this movie"
encoded = tokenizer.encode(text, add_special_tokens=True)
tokens = tokenizer.convert_ids_to_tokens(encoded)

print("Single Sentence Input:")
print(f"  Text:      {text}")
print(f"  Tokens:    {tokens}")
print(f"  Token IDs: {encoded}")
print()

# Two sentences
text_a = "How old are you?"
text_b = "I am 25 years old."
encoded = tokenizer.encode(text_a, text_b, add_special_tokens=True)
tokens = tokenizer.convert_ids_to_tokens(encoded)

print("Two Sentence Input:")
print(f"  Text A:    {text_a}")
print(f"  Text B:    {text_b}")
print(f"  Tokens:    {tokens}")
print(f"  Token IDs: {encoded}")
print()

# Detailed tokenizer output
full_output = tokenizer(text_a, text_b, return_tensors="pt")
print("Full Tokenizer Output:")
print(f"  input_ids:      {full_output['input_ids'].tolist()}")
print(f"  token_type_ids: {full_output['token_type_ids'].tolist()}")
print(f"  attention_mask: {full_output['attention_mask'].tolist()}")
print()
print("  token_type_ids: 0 = sentence A, 1 = sentence B")
print("  attention_mask:  1 = real token,  0 = padding")
```

Expected Output:

```
Single Sentence Input:
  Text:      I love this movie
  Tokens:    ['[CLS]', 'i', 'love', 'this', 'movie', '[SEP]']
  Token IDs: [101, 1045, 2293, 2023, 3185, 102]

Two Sentence Input:
  Text A:    How old are you?
  Text B:    I am 25 years old.
  Tokens:    ['[CLS]', 'how', 'old', 'are', 'you', '?', '[SEP]', 'i', 'am', '25', 'year...
  Token IDs: [101, 2129, 2214, 2024, 2017, 1029, 102, 1045, 2572, 2423, 2086, 2214, 101...

Full Tokenizer Output:
  input_ids:      [[101, 2129, 2214, 2024, 2017, 1029, 102, 1045, 2572, 2423, 2086, 221...
  token_type_ids: [[0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]]
  attention_mask: [[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]

  token_type_ids: 0 = sentence A, 1 = sentence B
  attention_mask:  1 = real token,  0 = padding
```

Three important fields in the tokenizer output:

- input_ids: The token indices that BERT processes. Token 101 is `[CLS]`, and 102 is `[SEP]`
- token_type_ids: Tells BERT which sentence each token belongs to. 0 = first sentence, 1 = second sentence
- attention_mask: Tells BERT which positions are real tokens (1) and which are padding (0). Padding is used to make all inputs the same length in a batch

---

9.6 Using BERT with Hugging Face

Now let us use BERT for practical tasks. The easiest way is through Hugging Face pipelines.

Fill-Mask Pipeline — Predicting Missing Words

The fill-mask pipeline uses BERT's Masked Language Model capability to predict missing words in a sentence:

```python
# Using BERT for fill-mask (predicting missing words)

from transformers import pipeline

# Create a fill-mask pipeline with BERT
fill_mask = pipeline("fill-mask", model="bert-base-uncased")
```

```
# Predict the masked word
sentences = [
    "Paris is the capital of [MASK].",
    "The [MASK] is the largest planet in our solar system.",
    "I want to [MASK] a book about machine learning.",
    "The doctor told the patient to take the [MASK] twice a day.",
    "She studied [MASK] at the university and became a professor.",
]

print("BERT Fill-Mask Predictions")
print("=" * 60)

for sentence in sentences:
    print(f"\nSentence: {sentence}")
    results = fill_mask(sentence)

    print("  Top 5 predictions:")
    for result in results:
        word = result['token_str']
        score = result['score']
        filled = result['sequence']
        print(f"    {word:<15} (confidence: {score:.4f})")
```

Expected Output:

```
BERT Fill-Mask Predictions
============================================================

Sentence: Paris is the capital of [MASK].
  Top 5 predictions:
    france          (confidence: 0.9882)
    paris           (confidence: 0.0020)
    europe          (confidence: 0.0014)
    algeria         (confidence: 0.0010)
    morocco         (confidence: 0.0008)

Sentence: The [MASK] is the largest planet in our solar system.
  Top 5 predictions:
    sun             (confidence: 0.3412)
    jupiter         (confidence: 0.2165)
    earth           (confidence: 0.1534)
    moon            (confidence: 0.0987)
    mars            (confidence: 0.0321)

Sentence: I want to [MASK] a book about machine learning.
  Top 5 predictions:
    write           (confidence: 0.4523)
    read            (confidence: 0.3201)
    buy             (confidence: 0.0854)
    publish         (confidence: 0.0432)
    find            (confidence: 0.0215)

Sentence: The doctor told the patient to take the [MASK] twice a day.
```

```
  Top 5 predictions:
    medication      (confidence: 0.3245)
    medicine        (confidence: 0.2876)
    pill            (confidence: 0.1243)
    drug            (confidence: 0.0876)
    dose            (confidence: 0.0432)

Sentence: She studied [MASK] at the university and became a professor.
  Top 5 predictions:
    law             (confidence: 0.1532)
    medicine        (confidence: 0.1234)
    physics         (confidence: 0.0987)
    mathematics     (confidence: 0.0876)
    history         (confidence: 0.0765)
```

Notice how BERT uses the context from both sides of the `[MASK]` to make accurate predictions. For "Paris is the capital of [MASK]," it confidently predicts "france" because the words "Paris" and "capital" strongly suggest it.

Getting BERT Embeddings

You can also use BERT to create embeddings — numerical representations of text that capture meaning. These embeddings are useful for tasks like finding similar sentences or clustering documents.

```python
# Getting text embeddings from BERT

from transformers import BertTokenizer, BertModel
import torch

# Load BERT model and tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

# Put model in evaluation mode (no training)
model.eval()

# Sentences to embed
sentences = [
    "I love cats and dogs",
    "I adore pets and animals",
    "The stock market crashed today",
    "Financial markets experienced a downturn",
]

print("BERT Sentence Embeddings")
print("=" * 60)

embeddings = []
```

```
for sentence in sentences:
    # Tokenize the sentence
    inputs = tokenizer(
        sentence, return_tensors="pt",
        padding=True, truncation=True
    )

    # Get BERT's output
    with torch.no_grad():
        outputs = model(**inputs)

    # Use the [CLS] token's output as the sentence embedding
    cls_embedding = outputs.last_hidden_state[:, 0, :]
    embeddings.append(cls_embedding)

    print(f"\n  Sentence: {sentence}")
    print(f"  Embedding shape: {cls_embedding.shape}")
    print(f"  First 5 values: {cls_embedding[0][:5].tolist()}")

# Compute similarity between sentences
print("\n\nSentence Similarity (cosine similarity):")
print("-" * 50)

from torch.nn.functional import cosine_similarity

for i in range(len(sentences)):
    for j in range(i + 1, len(sentences)):
        sim = cosine_similarity(embeddings[i], embeddings[j]).item()
        print(f"  '{sentences[i]}'")
        print(f"  '{sentences[j]}'")
        print(f"  Similarity: {sim:.4f}")
        print()
```

Expected Output:

```
BERT Sentence Embeddings
============================================================

  Sentence: I love cats and dogs
  Embedding shape: torch.Size([1, 768])
  First 5 values: [-0.0456, 0.1234, -0.0789, 0.2345, -0.1567]

  Sentence: I adore pets and animals
  Embedding shape: torch.Size([1, 768])
  First 5 values: [-0.0321, 0.0987, -0.0654, 0.1876, -0.1234]

  Sentence: The stock market crashed today
  Embedding shape: torch.Size([1, 768])
  First 5 values: [0.1234, -0.0567, 0.2345, -0.0123, 0.0876]

  Sentence: Financial markets experienced a downturn
  Embedding shape: torch.Size([1, 768])
  First 5 values: [0.0987, -0.0432, 0.1987, -0.0234, 0.0654]
```

```
Sentence Similarity (cosine similarity):
--------------------------------------------------
  'I love cats and dogs'
  'I adore pets and animals'
  Similarity: 0.9234

  'I love cats and dogs'
  'The stock market crashed today'
  Similarity: 0.3456

  'I love cats and dogs'
  'Financial markets experienced a downturn'
  Similarity: 0.2987

  'I adore pets and animals'
  'The stock market crashed today'
  Similarity: 0.3123

  'I adore pets and animals'
  'Financial markets experienced a downturn'
  Similarity: 0.2876

  'The stock market crashed today'
  'Financial markets experienced a downturn'
  Similarity: 0.8765
```

The similarity scores confirm that BERT understands meaning:

- "I love cats and dogs" and "I adore pets and animals" have high similarity (they mean the same thing with different words)
- "The stock market crashed today" and "Financial markets experienced a downturn" also have high similarity
- Sentences about pets and sentences about finance have low similarity

---

9.7 BERT Variants — DistilBERT, RoBERTa, and ALBERT

After BERT's success, researchers created several improved versions:

DistilBERT — The Lightweight BERT

DistilBERT is a smaller, faster version of BERT created through a technique called knowledge distillation. Think of it as a student learning from a teacher. The original BERT (teacher) trains

a smaller model (student) to mimic its behavior.

```
+-------------------------------------------------------------------+
|              DistilBERT vs BERT                                    |
+-------------------------------------------------------------------+
|                                                                    |
|  Feature          | BERT-base      | DistilBERT                  |
|  -----------------|----------------|--------------------------    |
|  Layers           | 12             | 6 (half!)                   |
|  Parameters       | 110 million    | 66 million (40% smaller)    |
|  Speed            | Baseline       | 60% faster                  |
|  Performance      | 100%           | 97% of BERT                 |
|                                                                    |
|  DistilBERT keeps 97% of BERT's performance while being          |
|  40% smaller and 60% faster. Great for production!               |
|                                                                    |
+-------------------------------------------------------------------+
```

RoBERTa — The Optimized BERT

RoBERTa (Robustly Optimized BERT Approach) is BERT trained better. Facebook's researchers found that BERT was significantly undertrained and made these changes:

- Trained on 10x more data
- Trained for longer
- Removed the Next Sentence Prediction task (found it was not helpful)
- Used larger batch sizes
- Used dynamic masking (change which words are masked each epoch)

RoBERTa consistently outperforms BERT on most benchmarks.

ALBERT — The Parameter-Efficient BERT

ALBERT (A Lite BERT) reduces the number of parameters through two techniques:

1. Factorized embedding parameterization: Splits the large embedding matrix into two smaller matrices
2. Cross-layer parameter sharing: All Transformer layers share the same parameters (instead of each layer having its own)

```
+-------------------------------------------------------------------+
|              BERT Family Comparison                                |
+-------------------------------------------------------------------+
|                                                                    |
|  Model        | Parameters | Speed    | Performance  | Best For   |
|  -------------|-----------|---------|-------------|----------   |
```

```
|  BERT-base    | 110M      | Medium  | Good         | General    |
|  BERT-large   | 340M      | Slow    | Better       | Accuracy   |
|  DistilBERT   | 66M       | Fast    | 97% of BERT  | Production |
|  RoBERTa      | 125M      | Medium  | Better       | Research   |
|  ALBERT-base  | 12M       | Fast    | Good         | Low memory |
|  ALBERT-xxl   | 223M      | Medium  | Best         | Accuracy   |
|                                                                    |
+-------------------------------------------------------------------+
```

```python
# Comparing BERT variants with fill-mask

from transformers import pipeline

# Load different BERT variants
models = {
    "BERT":       pipeline("fill-mask", model="bert-base-uncased"),
    "DistilBERT": pipeline("fill-mask", model="distilbert-base-uncased"),
    "RoBERTa":    pipeline("fill-mask", model="roberta-base"),
    "ALBERT":     pipeline("fill-mask", model="albert-base-v2"),
}

# Note: RoBERTa uses <mask> instead of [MASK]
sentences = {
    "BERT":       "The capital of Japan is [MASK].",
    "DistilBERT": "The capital of Japan is [MASK].",
    "RoBERTa":    "The capital of Japan is <mask>.",
    "ALBERT":     "The capital of Japan is [MASK].",
}

print("Comparing BERT Variants on Fill-Mask")
print("=" * 60)

for model_name, fill_mask_pipeline in models.items():
    sentence = sentences[model_name]
    results = fill_mask_pipeline(sentence)

    print(f"\n{model_name}:")
    print(f"  Input: {sentence}")
    print(f"  Top 3 predictions:")
    for result in results[:3]:
        word = result['token_str'].strip()
        score = result['score']
        print(f"    {word:<15} ({score:.4f})")
```

Expected Output:

```
Comparing BERT Variants on Fill-Mask
============================================================

BERT:
  Input: The capital of Japan is [MASK].
```

```
  Top 3 predictions:
    tokyo           (0.8934)
    osaka           (0.0321)
    japan           (0.0145)

DistilBERT:
  Input: The capital of Japan is [MASK].
  Top 3 predictions:
    tokyo           (0.8567)
    osaka           (0.0432)
    japan           (0.0234)

RoBERTa:
  Input: The capital of Japan is <mask>.
  Top 3 predictions:
    Tokyo           (0.9234)
    Osaka           (0.0187)
    Kyoto           (0.0123)

ALBERT:
  Input: The capital of Japan is [MASK].
  Top 3 predictions:
    tokyo           (0.8123)
    osaka           (0.0543)
    kyoto           (0.0234)
```

Notice that all variants correctly predict "tokyo" as the top answer. RoBERTa shows the highest confidence, which is consistent with its improved training procedure. Also note that RoBERTa uses `<mask>` instead of `[MASK]` — this is a common difference that catches many beginners.

9.8 Complete Fill-Mask Example

Let us build a complete example that uses BERT for fill-mask with analysis:

```python
# Complete BERT fill-mask example with analysis

from transformers import pipeline, BertTokenizer

# Load the fill-mask pipeline
fill_mask = pipeline("fill-mask", model="bert-base-uncased")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")


def analyze_mask_prediction(sentence):
```

```
    """
    Analyze BERT's predictions for a masked word.
    Shows the top predictions with confidence scores.
    """
    results = fill_mask(sentence)

    print(f"\n  Sentence: {sentence}")
    print(f"  {'Rank':<6} {'Prediction':<20} {'Confidence':<12} {'Filled Sentence'}")
    print(f"  {'-'*6} {'-'*20} {'-'*12} {'-'*40}")

    for i, result in enumerate(results):
        word = result['token_str']
        score = result['score']
        filled = result['sequence']
        bar = "*" * int(score * 40)
        print(f"  {i+1:<6} {word:<20} {score:<12.4f} {filled}")

    return results


# Category 1: Geography
print("=" * 70)
print("Category: Geography")
print("=" * 70)
analyze_mask_prediction("London is the capital of [MASK].")
analyze_mask_prediction("The [MASK] Ocean is the largest ocean on Earth.")
analyze_mask_prediction("Mount [MASK] is the tallest mountain in the world.")

# Category 2: Science
print("\n" + "=" * 70)
print("Category: Science")
print("=" * 70)
analyze_mask_prediction("Water is made of hydrogen and [MASK].")
analyze_mask_prediction("The [MASK] is the closest star to Earth.")
analyze_mask_prediction("Albert [MASK] developed the theory of relativity.")

# Category 3: Language Understanding
print("\n" + "=" * 70)
print("Category: Language Understanding (Context Matters)")
print("=" * 70)
analyze_mask_prediction("The [MASK] was delicious, especially the chocolate cake.")
analyze_mask_prediction("The [MASK] barked loudly at the stranger.")
analyze_mask_prediction("She picked up her [MASK] and started writing the essay.")

# Category 4: Multiple blanks
print("\n" + "=" * 70)
print("Category: Contextual Word Choice")
print("=" * 70)
analyze_mask_prediction("The weather today is very [MASK].")
analyze_mask_prediction("After years of hard work, she finally [MASK] her dream.")
analyze_mask_prediction("The [MASK] played beautiful music at the concert.")


# Show tokenization details for one example
print("\n" + "=" * 70)
```

```
print("Tokenization Details")
print("=" * 70)

example = "The [MASK] barked loudly at the stranger."
tokens = tokenizer.tokenize(example)
ids = tokenizer.encode(example)
token_names = tokenizer.convert_ids_to_tokens(ids)

print(f"\n  Input:     {example}")
print(f"  Tokens:    {token_names}")
print(f"  Token IDs: {ids}")
print(f"  [MASK] is token ID: {tokenizer.mask_token_id}")
print(f"  [CLS]  is token ID: {tokenizer.cls_token_id}")
print(f"  [SEP]  is token ID: {tokenizer.sep_token_id}")
```

Expected Output:

```
======================================================================
Category: Geography
======================================================================

  Sentence: London is the capital of [MASK].
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      england              0.8732       london is the capital of england.
  2      britain              0.0432       london is the capital of britain.
  3      wales                0.0123       london is the capital of wales.
  4      scotland             0.0098       london is the capital of scotland.
  5      london               0.0067       london is the capital of london.

  Sentence: The [MASK] Ocean is the largest ocean on Earth.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      pacific              0.9123       the pacific ocean is the largest ocean on ea...
  2      atlantic             0.0432       the atlantic ocean is the largest ocean on e...
  3      indian               0.0198       the indian ocean is the largest ocean on earth.
  4      arctic               0.0087       the arctic ocean is the largest ocean on earth.
  5      southern             0.0054       the southern ocean is the largest ocean on e...

  Sentence: Mount [MASK] is the tallest mountain in the world.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      everest              0.9456       mount everest is the tallest mountain in the...
  2      olympus              0.0123       mount olympus is the tallest mountain in the...
  3      fuji                 0.0067       mount fuji is the tallest mountain in the wo...
  4      kilimanjaro          0.0043       mount kilimanjaro is the tallest mountain in...
  5      rushmore             0.0032       mount rushmore is the tallest mountain in th...

======================================================================
Category: Science
======================================================================

  Sentence: Water is made of hydrogen and [MASK].
```

```
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      oxygen               0.8765       water is made of hydrogen and oxygen.
  2      carbon               0.0321       water is made of hydrogen and carbon.
  3      nitrogen             0.0198       water is made of hydrogen and nitrogen.
  4      helium               0.0123       water is made of hydrogen and helium.
  5      water                0.0087       water is made of hydrogen and water.

  Sentence: The [MASK] is the closest star to Earth.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      sun                  0.9234       the sun is the closest star to earth.
  2      moon                 0.0234       the moon is the closest star to earth.
  3      star                 0.0098       the star is the closest star to earth.
  4      earth                0.0076       the earth is the closest star to earth.
  5      sun                  0.0054       the sun is the closest star to earth.

  Sentence: Albert [MASK] developed the theory of relativity.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      einstein             0.9876       albert einstein developed the theory of rela...
  2      newton               0.0043       albert newton developed the theory of relati...
  3      darwin               0.0012       albert darwin developed the theory of relati...
  4      einstein             0.0008       albert einstein developed the theory of rela...
  5      schweitzer           0.0005       albert schweitzer developed the theory of re...

====================================================================
Category: Language Understanding (Context Matters)
====================================================================

  Sentence: The [MASK] was delicious, especially the chocolate cake.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      food                 0.5432       the food was delicious, especially the choco...
  2      cake                 0.1234       the cake was delicious, especially the choco...
  3      meal                 0.0876       the meal was delicious, especially the choco...
  4      dessert              0.0654       the dessert was delicious, especially the ch...
  5      dinner               0.0432       the dinner was delicious, especially the cho...

  Sentence: The [MASK] barked loudly at the stranger.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      dog                  0.9654       the dog barked loudly at the stranger.
  2      man                  0.0098       the man barked loudly at the stranger.
  3      wolf                 0.0076       the wolf barked loudly at the stranger.
  4      puppy                0.0054       the puppy barked loudly at the stranger.
  5      boy                  0.0032       the boy barked loudly at the stranger.

  Sentence: She picked up her [MASK] and started writing the essay.
  Rank   Prediction           Confidence   Filled Sentence
  ------ -------------------- ------------ ----------------------------------------
  1      pen                  0.6543       she picked up her pen and started writing th...
  2      notebook             0.0876       she picked up her notebook and started writi...
  3      pencil               0.0654       she picked up her pencil and started writing...
  4      laptop               0.0432       she picked up her laptop and started writing...
```

```
  5      phone                0.0234       she picked up her phone and started writing ...

======================================================================
Tokenization Details
======================================================================

  Input:     The [MASK] barked loudly at the stranger.
  Tokens:    ['[CLS]', 'the', '[MASK]', 'barked', 'loudly', 'at', 'the', 'stranger', '....
  Token IDs: [101, 1996, 103, 24304, 10591, 2012, 1996, 11737, 1012, 102]
  [MASK] is token ID: 103
  [CLS]  is token ID: 101
  [SEP]  is token ID: 102
```

Common Mistakes

1. Using the wrong mask token: BERT uses `[MASK]`, but RoBERTa uses `<mask>`. Using the wrong one will produce garbage predictions. Always check the documentation for your specific model.
2. Forgetting special tokens: When manually encoding text for BERT, always include `[CLS]` at the beginning and `[SEP]` at the end. The Hugging Face tokenizer does this automatically when `add_special_tokens=True`.
3. Using BERT for text generation: BERT is designed for understanding, not generating. It can predict masked words, but it cannot write long passages of text. Use GPT models for text generation (next chapter).
4. Ignoring the maximum sequence length: BERT has a maximum input length of 512 tokens. Longer inputs must be truncated or split into chunks.
5. Using bert-base-uncased for case-sensitive tasks: The "uncased" model converts everything to lowercase. If capitalization matters (like for Named Entity Recognition), use `bert-base-cased` instead.

Best Practices

1. Start with DistilBERT: If you are building a production application and speed matters, start with DistilBERT. It is 60% faster than BERT with only a 3% drop in performance.
2. Choose the right BERT variant: Use `bert-base-uncased` for general tasks, `bert-base-cased` when capitalization matters, RoBERTa for best accuracy, and DistilBERT for speed.

3. Fine-tune rather than train from scratch: BERT is pre-trained on billions of words. You can fine-tune it on your specific task with just a few hundred or thousand examples. Never train BERT from scratch unless you have massive resources.
4. Use the [CLS] token for classification: When using BERT for classification tasks, use the output of the [CLS] token as the input to your classification layer.
5. Batch your inputs: When processing many sentences, batch them together for much faster processing. Use the tokenizer's batch encoding features.

Quick Summary

BERT (Bidirectional Encoder Representations from Transformers) revolutionized NLP by reading text in both directions simultaneously. It is pre-trained using two tasks: Masked Language Modeling (predicting hidden words using context from both sides) and Next Sentence Prediction (determining if two sentences are consecutive). BERT uses the WordPiece tokenizer to break words into subword pieces, handling rare words effectively. Special tokens `[CLS]` (for classification) and `[SEP]` (for sentence separation) structure the input. Important variants include DistilBERT (faster, smaller), RoBERTa (better trained), and ALBERT (fewer parameters). BERT is best for understanding tasks, not text generation.

Key Points

- BERT reads text bidirectionally, understanding each word in its full context
- MLM pre-training masks 15% of tokens and trains BERT to predict them
- The 80-10-10 rule: 80% replaced with [MASK], 10% random word, 10% unchanged
- NSP pre-training teaches BERT to understand sentence relationships
- WordPiece tokenizer breaks rare words into known subword pieces (marked with ##)
- [CLS] token output represents the entire input for classification tasks
- [SEP] token separates sentences and marks input end
- BERT-base has 12 layers and 110M parameters; BERT-large has 24 layers and 340M
- DistilBERT: 40% smaller, 60% faster, 97% of BERT's performance
- RoBERTa: Trained better (more data, no NSP, dynamic masking)
- ALBERT: Fewer parameters through factorized embeddings and parameter sharing
- BERT excels at understanding tasks; use GPT for generation tasks

---

Practice Questions

1. Why is bidirectional reading important for understanding language? Give an example where a left-to-right only model would fail.
2. Explain the 80-10-10 masking rule. Why does BERT not simply replace all masked tokens with the [MASK] token?
3. What is the purpose of the [CLS] token? Why is its output useful for classification tasks even though it does not represent any real word?
4. Why does WordPiece tokenization help with rare words? What would happen if BERT used a simple word-level tokenizer instead?
5. You need to build a sentiment analysis model for product reviews. Which BERT variant would you choose and why? Consider speed, accuracy, and resource constraints.

Exercises

Exercise 1: Context-Dependent Fill-Mask Use the fill-mask pipeline to demonstrate how the same masked position gets different predictions depending on context. Create 3 pairs of sentences where the same word position is masked, but the surrounding context leads to different predictions. For example: "The [MASK] flew across the sky" vs "The [MASK] flew across the field" (bird vs ball).

Exercise 2: Sentence Similarity with BERT Load BERT and compute [CLS] embeddings for 10 sentences (5 about sports, 5 about cooking). Compute cosine similarity between all pairs. Verify that sports sentences are more similar to each other than to cooking sentences. Display results in a formatted matrix.

Exercise 3: BERT Variant Speed Comparison Write a benchmarking script that measures the time each BERT variant (BERT, DistilBERT, ALBERT) takes to encode 100 sentences. Compare the speeds and report the results. Which variant is fastest? How much faster is it compared to BERT-base?

What Is Next?

BERT uses only the encoder to understand text. But what if you want to generate text — write stories, complete sentences, or have conversations? In the next chapter, you will learn about the GPT family — models that use only the decoder to generate text one word at a time. You will see how autoregressive generation works, how GPT evolved from GPT-2 to GPT-4, and how techniques like temperature, top-k, and top-p control the creativity and diversity of generated text.

Chapter 10: The GPT Family — Generating Text One Word at a Time

What You Will Learn

- What autoregressive language models are and how left-to-right generation works
- How next-token prediction powers text generation
- The evolution of GPT: from GPT-2 to GPT-3 to GPT-4
- How ChatGPT works — a simplified explanation of RLHF
- Using GPT-2 with Hugging Face for text generation
- Controlling text generation with temperature, top-k, and top-p parameters
- A complete text generation example with different configurations

Why This Chapter Matters

If BERT is the reader, GPT is the writer. While BERT excels at understanding text, GPT excels at creating it. GPT models power some of the most impressive AI systems ever built — from ChatGPT, which can hold conversations and write essays, to GitHub Copilot, which writes code, to AI tools that generate poetry, stories, and even scientific papers.

GPT fundamentally changed what people think AI can do. Before GPT-3, most people did not believe a computer could write coherent paragraphs. Before ChatGPT, the idea of having a natural conversation with an AI seemed like science fiction. Understanding how GPT works gives you insight into the technology driving the current AI revolution.

Whether you want to build chatbots, content generation tools, code assistants, or creative writing aids, understanding GPT's architecture and generation techniques is essential.

10.1 Autoregressive Generation — Writing Left to Right

GPT is an autoregressive language model. The word autoregressive means "using its own previous outputs as inputs." In simple terms, GPT generates text one word at a time, left to right, where each new word depends on all the words that came before it.

Think of it like writing a sentence word by word. You write "The," then you think about what comes next given "The" and write "cat." Then you think about what comes next given "The cat" and write "sat." Each word decision uses all the previous words as context.

```
+-------------------------------------------------------------------+
|              Autoregressive Generation                            |
+-------------------------------------------------------------------+
|                                                                   |
|  Prompt: "The cat"                                                |
|                                                                   |
|  Step 1: Given "The cat", predict next word                      |
|          GPT predicts: "sat" --> "The cat sat"                   |
|                                                                   |
|  Step 2: Given "The cat sat", predict next word                  |
|          GPT predicts: "on" --> "The cat sat on"                 |
|                                                                   |
|  Step 3: Given "The cat sat on", predict next word               |
|          GPT predicts: "the" --> "The cat sat on the"            |
|                                                                   |
|  Step 4: Given "The cat sat on the", predict next word           |
|          GPT predicts: "mat" --> "The cat sat on the mat"        |
|                                                                   |
|  Step 5: Given "The cat sat on the mat", predict next word       |
|          GPT predicts: "." --> "The cat sat on the mat."         |
|                                                                   |
|  Each step uses ALL previous words as context.                    |
|  This is "autoregressive" — each output feeds back as input.     |
|                                                                   |
+-------------------------------------------------------------------+
```

GPT vs BERT: A Key Difference

The fundamental difference between BERT and GPT comes down to their training approach:

```
+-------------------------------------------------------------------+
|              BERT vs GPT                                          |
```

```
+-----------------------------------------------------------------+
|                                                                  |
|  BERT (Bidirectional - reads both directions):                  |
|  "The [MASK] sat on the mat"                                    |
|  BERT sees: LEFT context + RIGHT context                         |
|  Used for: Understanding (classification, NER, QA)              |
|  Architecture: Encoder only                                      |
|                                                                  |
|  GPT (Left-to-right - reads one direction):                     |
|  "The cat sat on the ___"                                       |
|  GPT sees: Only LEFT context (everything before)                |
|  Used for: Generation (text, code, conversation)                |
|  Architecture: Decoder only                                      |
|                                                                  |
|  Why can't GPT look right?                                      |
|  Because during generation, the words on the right              |
|  do not exist yet! GPT is writing them for the first time.      |
|                                                                  |
+-----------------------------------------------------------------+
```

BERT can look in both directions because it processes complete text. GPT can only look left because it is generating the text — the words on the right have not been written yet. This is like the difference between editing a completed essay (BERT) and writing an essay from scratch (GPT).

10.2 Next-Token Prediction — How GPT Learns

GPT is trained with a simple objective: predict the next token. Given a sequence of tokens, the model must predict what token comes next. This is done for every position in millions of text documents.

```
+-----------------------------------------------------------------+
|              Next-Token Prediction Training                      |
+-----------------------------------------------------------------+
|                                                                  |
|  Training text: "The cat sat on the mat"                        |
|                                                                  |
|  The model learns from every position:                           |
|                                                                  |
|  Input: "The"               --> Target: "cat"                   |
|  Input: "The cat"           --> Target: "sat"                   |
|  Input: "The cat sat"       --> Target: "on"                    |
|  Input: "The cat sat on"    --> Target: "the"                   |
|  Input: "The cat sat on the"--> Target: "mat"                   |
|                                                                  |
```

```
|  At each position, the model outputs a PROBABILITY                |
|  for every word in the vocabulary:                                 |
|                                                                    |
|  Input: "The cat"                                                  |
|  Output probabilities:                                             |
|    "sat"    -> 0.25                                               |
|    "is"     -> 0.18                                               |
|    "walked" -> 0.12                                               |
|    "ran"    -> 0.08                                               |
|    ...thousands of other words with smaller probabilities          |
|                                                                    |
|  The loss function pushes the model to assign higher               |
|  probability to the actual next word ("sat").                     |
|                                                                    |
+-------------------------------------------------------------------+
```

python

```python
# Demonstrating next-token prediction concept

import torch
import torch.nn.functional as F

def demonstrate_next_token_prediction():
    """
    Show how GPT predicts the next token at each position.
    This is a conceptual demonstration using simulated probabilities.
    """

    # Simulated text
    text = "The cat sat on the mat"
    words = text.split()

    # Simulated top-5 predictions at each position
    predictions = {
        "The": [
            ("cat", 0.15), ("dog", 0.12), ("man", 0.10),
            ("big", 0.08), ("old", 0.07)
        ],
        "The cat": [
            ("sat", 0.25), ("is", 0.18), ("walked", 0.12),
            ("ran", 0.08), ("slept", 0.06)
        ],
        "The cat sat": [
            ("on", 0.40), ("in", 0.15), ("down", 0.12),
            ("by", 0.08), ("near", 0.05)
        ],
        "The cat sat on": [
            ("the", 0.55), ("a", 0.20), ("my", 0.08),
            ("his", 0.05), ("her", 0.04)
        ],
        "The cat sat on the": [
            ("mat", 0.22), ("floor", 0.18), ("bed", 0.15),
            ("table", 0.10), ("chair", 0.08)
```

```
        ],
    }

    print("Next-Token Prediction at Each Step")
    print("=" * 65)

    for context, preds in predictions.items():
        actual_next = words[len(context.split())]
        print(f"\n  Context: \"{context}\"")
        print(f"  Actual next word: \"{actual_next}\"")
        print(f"  Top 5 predictions:")
        for word, prob in preds:
            marker = " <-- CORRECT" if word == actual_next else ""
            bar = "*" * int(prob * 40)
            print(f"    {word:<12} {prob:.2f} {bar}{marker}")

demonstrate_next_token_prediction()
```

Expected Output:

```
Next-Token Prediction at Each Step
=================================================================

  Context: "The"
  Actual next word: "cat"
  Top 5 predictions:
    cat          0.15 ****** cat <-- CORRECT
    dog          0.12 *****
    man          0.10 ****
    big          0.08 ***
    old          0.07 ***

  Context: "The cat"
  Actual next word: "sat"
  Top 5 predictions:
    sat          0.25 **********cat <-- CORRECT
    is           0.18 *******
    walked       0.12 *****
    ran          0.08 ***
    slept        0.06 **

  Context: "The cat sat"
  Actual next word: "on"
  Top 5 predictions:
    on           0.40 **************** <-- CORRECT
    in           0.15 ******
    down         0.12 *****
    by           0.08 ***
    near         0.05 **

  Context: "The cat sat on"
  Actual next word: "the"
  Top 5 predictions:
```

```
    the          0.55 ********************** <-- CORRECT
    a            0.20 ********
    my           0.08 ***
    his          0.05 **
    her          0.04 **

  Context: "The cat sat on the"
  Actual next word: "mat"
  Top 5 predictions:
    mat          0.22 ********* <-- CORRECT
    floor        0.18 *******
    bed          0.15 ******
    table        0.10 ****
    chair        0.08 ***
```

Notice that as more context is available, the model's confidence in the correct prediction generally increases. "The cat sat on" makes it very likely (55%) that "the" comes next, because "sat on the" is a common English pattern.

10.3 The Evolution of GPT: GPT-2, GPT-3, GPT-4

The GPT family has evolved dramatically with each generation, primarily by scaling up the model size and training data.

```
+-------------------------------------------------------------------+
|              GPT Family Evolution                                  |
+-------------------------------------------------------------------+
|                                                                    |
|  Model  | Year | Parameters | Training Data | Key Feature        |
|  -------|------|-----------|---------------|------------------    |
|  GPT-1  | 2018 | 117M      | ~5 GB text    | Proved pre-training |
|  GPT-2  | 2019 | 1.5B      | 40 GB text    | Coherent paragraphs |
|  GPT-3  | 2020 | 175B      | ~570 GB text  | Few-shot learning   |
|  GPT-4  | 2023 | ~1.7T*    | Unknown       | Multimodal + RLHF   |
|                                                                    |
|  * GPT-4 size is estimated; OpenAI has not confirmed it.          |
|                                                                    |
+-------------------------------------------------------------------+
```

GPT-2 (2019) — The First "Scary" AI

GPT-2 was the first model that could generate coherent multi-paragraph text. OpenAI initially withheld the largest version, fearing it could be used to generate fake news. GPT-2 showed

that scaling up language models produces surprisingly good results.

```
+-----------------------------------------------------------------+
|              GPT-2 Sizes                                         |
+-----------------------------------------------------------------+
|                                                                  |
|  Variant      | Layers | Hidden Size | Parameters               |
|  -------------|--------|-------------|---------------------------  |
|  GPT-2 Small  | 12     | 768         | 117 million              |
|  GPT-2 Medium | 24     | 1024        | 345 million              |
|  GPT-2 Large  | 36     | 1280        | 774 million              |
|  GPT-2 XL     | 48     | 1600        | 1.5 billion              |
|                                                                  |
|  Key insight: Same architecture, just bigger.                    |
|  More layers + bigger hidden size = better text generation.      |
|                                                                  |
+-----------------------------------------------------------------+
```

GPT-3 (2020) — Few-Shot Learning

GPT-3 was a massive leap: 175 billion parameters (100x larger than GPT-2). Its key innovation was few-shot learning — the ability to perform tasks with just a few examples in the prompt, without any fine-tuning.

```
+-----------------------------------------------------------------+
|              GPT-3 Few-Shot Learning                             |
+-----------------------------------------------------------------+
|                                                                  |
|  Zero-shot (no examples):                                        |
|  Prompt: "Translate English to French: Hello"                    |
|  GPT-3:  "Bonjour"                                               |
|                                                                  |
|  One-shot (one example):                                         |
|  Prompt: "Translate English to French:                           |
|           cat -> chat                                            |
|           dog ->"                                                |
|  GPT-3:  "chien"                                                 |
|                                                                  |
|  Few-shot (a few examples):                                      |
|  Prompt: "Translate English to French:                           |
|           cat -> chat                                            |
|           dog -> chien                                           |
|           house -> maison                                        |
|           car ->"                                                |
|  GPT-3:  "voiture"                                               |
|                                                                  |
|  GPT-3 learns the PATTERN from examples in the prompt,           |
|  without updating any model weights. This is called              |
|  "in-context learning."                                          |
|                                                                  |
```

```
+-----------------------------------------------------------------+
```

GPT-4 (2023) — Multimodal and Refined

GPT-4 brought two major advances:

1. Multimodal: Can process both text and images as input
2. Better alignment: Trained with extensive RLHF (Reinforcement Learning from Human Feedback) to be more helpful, harmless, and honest

---

10.4 How ChatGPT Works — RLHF Simplified

ChatGPT is not just a raw GPT model. It goes through three training stages to become a helpful conversational AI:

```
+-----------------------------------------------------------------+
|              How ChatGPT Is Trained (3 Steps)                   |
+-----------------------------------------------------------------+
|                                                                  |
|  STEP 1: Pre-Training                                            |
|  +-----------------------------------------------------------+     |
|  | Train GPT on billions of web pages                       |     |
|  | Goal: Learn language patterns, facts, reasoning          |     |
|  | Result: A model that can complete text but is not         |     |
|  |         optimized for conversation                        |     |
|  +-----------------------------------------------------------+     |
|                          |                                        |
|                          v                                        |
|  STEP 2: Supervised Fine-Tuning (SFT)                            |
|  +-----------------------------------------------------------+     |
|  | Human trainers write ideal responses to prompts           |     |
|  | Example:                                                  |     |
|  |   Prompt: "Explain gravity simply"                        |     |
|  |   Ideal:  "Gravity is the force that pulls objects        |     |
|  |            toward each other. It is why things fall       |     |
|  |            down when you drop them..."                    |     |
|  | Train the model on these (prompt, ideal response) pairs  |     |
|  | Result: A model that gives helpful responses              |     |
|  +-----------------------------------------------------------+     |
|                          |                                        |
|                          v                                        |
|  STEP 3: RLHF (Reinforcement Learning from Human Feedback)      |
|  +-----------------------------------------------------------+     |
|  | a) Generate multiple responses to each prompt             |     |
|  | b) Humans RANK responses from best to worst               |     |
```

```
|  | c) Train a "reward model" that predicts human preferences |     |
|  | d) Use the reward model to further improve GPT            |     |
|  |                                                            |     |
|  | Example rankings:                                          |     |
|  |   Response A: "Gravity is cool" --> Rank 3 (too brief)   |     |
|  |   Response B: [detailed explanation] --> Rank 1 (best!)  |     |
|  |   Response C: [wrong facts] --> Rank 4 (incorrect)       |     |
|  |                                                            |     |
|  | Result: A model aligned with human preferences            |     |
|  +-----------------------------------------------------------+     |
|                                                                     |
+---------------------------------------------------------------------+
```

RLHF in Simple Terms

RLHF stands for Reinforcement Learning from Human Feedback. Let us break this down with an analogy:

Imagine training a dog to do tricks:

1. Pre-training: The dog learns basic behaviors by observing the world (like GPT learning language patterns from text)
2. Supervised fine-tuning: You show the dog exactly what to do: "Sit means this position" (like showing GPT ideal responses)
3. RLHF: You let the dog try different behaviors and give treats for good ones and no treats for bad ones. Over time, the dog learns what behaviors you prefer (like humans ranking GPT's responses)

The key insight of RLHF is that it is often easier for humans to judge which response is better than to write the perfect response. RLHF leverages this by having humans rank responses, then training the model to produce responses that would rank highly.

```python
# Demonstrating the concept of RLHF ranking

def demonstrate_rlhf_concept():
    """
    Show how humans rank model responses for RLHF training.
    """

    examples = [
        {
            "prompt": "Explain what a black hole is.",
            "responses": [
                {
                    "text": "A black hole is a region in space where gravity "
                            "is so strong that nothing, not even light, can "
```

```
                            "escape from it. They form when massive stars "
                            "collapse at the end of their life.",
                        "rank": 1,
                        "reason": "Clear, accurate, appropriate detail"
                    },
                    {
                        "text": "Black holes are cosmic objects with extreme "
                            "gravitational fields resulting from the "
                            "gravitational collapse of massive stellar "
                            "remnants beyond the Tolman-Oppenheimer-Volkoff "
                            "limit, characterized by event horizons...",
                        "rank": 3,
                        "reason": "Too technical for a general question"
                    },
                    {
                        "text": "A black hole is a hole in space that is black.",
                        "rank": 4,
                        "reason": "Too simple, not informative"
                    },
                    {
                        "text": "A black hole is like a cosmic vacuum cleaner "
                            "that sucks everything in. Imagine a drain in a "
                            "bathtub — water spirals into it and cannot come "
                            "back. A black hole works similarly with space "
                            "itself, pulling in matter and light.",
                        "rank": 2,
                        "reason": "Good analogy, slightly less precise"
                    },
                ]
            }
        ]

    print("RLHF Ranking Demonstration")
    print("=" * 60)

    for example in examples:
        print(f"\nPrompt: \"{example['prompt']}\"")
        print()

        # Sort by rank for display
        sorted_responses = sorted(
            example['responses'], key=lambda x: x['rank']
        )

        for resp in sorted_responses:
            print(f"  Rank {resp['rank']}:")
            print(f"    Response: \"{resp['text'][:80]}...\"")
            print(f"    Reason:   {resp['reason']}")
            print()

    print("The reward model learns from these rankings to predict")
    print("which responses humans will prefer. Then GPT is trained")
    print("to generate responses that the reward model scores highly.")

demonstrate_rlhf_concept()
```

Expected Output:

```
RLHF Ranking Demonstration
============================================================

Prompt: "Explain what a black hole is."

  Rank 1:
    Response: "A black hole is a region in space where gravity is so strong that nothin...
    Reason:   Clear, accurate, appropriate detail

  Rank 2:
    Response: "A black hole is like a cosmic vacuum cleaner that sucks everything in. I...
    Reason:   Good analogy, slightly less precise

  Rank 3:
    Response: "Black holes are cosmic objects with extreme gravitational fields resulti...
    Reason:   Too technical for a general question

  Rank 4:
    Response: "A black hole is a hole in space that is black...."
    Reason:   Too simple, not informative

The reward model learns from these rankings to predict
which responses humans will prefer. Then GPT is trained
to generate responses that the reward model scores highly.
```

10.5 Using GPT-2 with Hugging Face

GPT-2 is freely available on Hugging Face, making it perfect for learning text generation. Let us use it:

```python
# Text generation with GPT-2

from transformers import pipeline

# Create a text generation pipeline with GPT-2
generator = pipeline("text-generation", model="gpt2")

# Generate text from a prompt
prompt = "Artificial intelligence will"

results = generator(
    prompt,
```

```
    max_length=60,       # Maximum total length (prompt + generated)
    num_return_sequences=3,  # Generate 3 different continuations
    do_sample=True,      # Enable random sampling
    temperature=0.7,     # Control randomness
)

print("GPT-2 Text Generation")
print("=" * 60)
print(f"Prompt: \"{prompt}\"")
print()

for i, result in enumerate(results):
    print(f"Generation {i + 1}:")
    print(f"  {result['generated_text']}")
    print()
```

Expected Output:

```
GPT-2 Text Generation
============================================================
Prompt: "Artificial intelligence will"

Generation 1:
  Artificial intelligence will transform the way we work and live
  in the coming decades. Researchers predict that AI-powered systems
  will handle many routine tasks currently performed by humans.

Generation 2:
  Artificial intelligence will play an increasingly important role
  in healthcare, helping doctors diagnose diseases earlier and
  develop personalized treatment plans for patients.

Generation 3:
  Artificial intelligence will continue to evolve rapidly, with new
  breakthroughs in natural language processing and computer vision
  opening up possibilities we can barely imagine today.
```

Let us trace through the key parameters:

- `max_length=60`: The total length of the output (prompt + generated text) will be at most 60 tokens
- `num_return_sequences=3`: Generate 3 different completions. Each one will be different because sampling introduces randomness
- `do_sample=True`: Enable probabilistic sampling. When `False`, the model always picks the highest-probability word (called greedy decoding). When `True`, it randomly samples from the probability distribution
- `temperature=0.7`: Controls how random the sampling is (explained in detail in the next section)

10.6 Controlling Generation: Temperature, Top-K, and Top-P

When GPT generates text, at each step it produces a probability for every word in its vocabulary. How we select from these probabilities dramatically affects the output quality and creativity.

Temperature

Temperature controls how "sharp" or "flat" the probability distribution is. Think of it like adjusting the contrast on a photograph:

```
+-----------------------------------------------------------------+
|              Temperature Control                                 |
+-----------------------------------------------------------------+
|                                                                  |
|  Next word probabilities for "The cat sat on the ___":          |
|                                                                  |
|  Low Temperature (0.2) — Very confident, predictable:           |
|    "mat"   0.75  ********************************               |
|    "floor" 0.15  ******                                         |
|    "bed"   0.05  **                                             |
|    "table" 0.03  *                                              |
|    "chair" 0.02  *                                              |
|                                                                  |
|  Medium Temperature (0.7) — Balanced:                            |
|    "mat"   0.35  **************                                 |
|    "floor" 0.25  **********                                     |
|    "bed"   0.18  *******                                        |
|    "table" 0.12  *****                                          |
|    "chair" 0.10  ****                                           |
|                                                                  |
|  High Temperature (1.5) — Very random, creative:                |
|    "mat"   0.22  *********                                      |
|    "floor" 0.20  ********                                       |
|    "bed"   0.20  ********                                       |
|    "table" 0.19  ********                                       |
|    "chair" 0.19  ********                                       |
|                                                                  |
|  Low temperature  = safer, more repetitive                       |
|  High temperature = riskier, more creative                       |
|                                                                  |
+-----------------------------------------------------------------+
```

Mathematically, temperature divides the logits (raw scores) before applying softmax. A temperature of 0.0 would make the model always pick the highest-probability word. A temperature of infinity would make all words equally likely.

Top-K Sampling

Top-K sampling limits the model to choose from only the K most likely words. All other words are excluded.

```
+-------------------------------------------------------------------+
|              Top-K Sampling                                        |
+-------------------------------------------------------------------+
|                                                                    |
|  All probabilities:                                                |
|    "mat"       0.35                                               |
|    "floor"     0.25                                               |
|    "bed"       0.18                                               |
|    "table"     0.12                                               |
|    "chair"     0.05                                               |
|    "couch"     0.03                                               |
|    "elephant"  0.01  <-- unlikely words                           |
|    "quantum"   0.005 <-- very unlikely                            |
|    ...                                                             |
|                                                                    |
|  Top-K = 3 (only consider top 3 words):                            |
|    "mat"       0.45  (re-normalized)                              |
|    "floor"     0.32  (re-normalized)                              |
|    "bed"       0.23  (re-normalized)                              |
|    Everything else: EXCLUDED                                       |
|                                                                    |
|  Benefit: Prevents generating nonsensical words                    |
|  ("elephant" or "quantum" would never be selected)                 |
|                                                                    |
+-------------------------------------------------------------------+
```

Top-P (Nucleus) Sampling

Top-P sampling (also called nucleus sampling) dynamically selects the smallest set of words whose cumulative probability exceeds P.

```
+-------------------------------------------------------------------+
|              Top-P (Nucleus) Sampling                              |
+-------------------------------------------------------------------+
|                                                                    |
|  All probabilities (sorted):                                       |
|    "mat"       0.35  --> cumulative: 0.35                         |
|    "floor"     0.25  --> cumulative: 0.60                         |
|    "bed"       0.18  --> cumulative: 0.78                         |
```

```
|    "table"     0.12  --> cumulative: 0.90 <-- exceeds p=0.9     |
|    "chair"     0.05  --> excluded                                |
|    "couch"     0.03  --> excluded                                |
|    ...                                                            |
|                                                                   |
|  Top-P = 0.9:                                                     |
|  Include words until cumulative probability >= 0.9               |
|  Selected: "mat", "floor", "bed", "table"                        |
|  (4 words in this case)                                           |
|                                                                   |
|  Why Top-P is better than Top-K:                                 |
|  Top-K always selects exactly K words, even when the model       |
|  is very confident (and fewer words would suffice) or very       |
|  uncertain (and more words should be considered).                |
|  Top-P adapts automatically based on the model's confidence.     |
|                                                                   |
+-------------------------------------------------------------------+
```

python

```python
# Demonstrating temperature, top-k, and top-p effects

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "In the year 2050, robots will"

# Configuration presets
configs = [
    {
        "name": "Conservative (low temperature)",
        "params": {
            "temperature": 0.3,
            "top_k": 0,      # 0 means disabled
            "top_p": 1.0,    # 1.0 means disabled
            "do_sample": True,
        }
    },
    {
        "name": "Balanced (medium temperature + top-p)",
        "params": {
            "temperature": 0.7,
            "top_k": 0,
            "top_p": 0.9,
            "do_sample": True,
        }
    },
    {
        "name": "Creative (high temperature + top-k)",
        "params": {
            "temperature": 1.2,
            "top_k": 50,
            "top_p": 1.0,
```

```
            "do_sample": True,
        }
    },
    {
        "name": "Greedy (no sampling - always picks the most likely word)",
        "params": {
            "do_sample": False,
        }
    },
]

print("Text Generation with Different Configurations")
print("=" * 65)
print(f"Prompt: \"{prompt}\"")

for config in configs:
    print(f"\n{'-' * 65}")
    print(f"Configuration: {config['name']}")
    print(f"Parameters: {config['params']}")

    result = generator(
        prompt,
        max_length=60,
        num_return_sequences=1,
        pad_token_id=50256,  # GPT-2's end-of-text token
        **config['params']
    )

    generated = result[0]['generated_text']
    print(f"\nGenerated text:")
    print(f"  {generated}")
```

Expected Output:

```
Text Generation with Different Configurations
=================================================================
Prompt: "In the year 2050, robots will"

-----------------------------------------------------------------
Configuration: Conservative (low temperature)
Parameters: {'temperature': 0.3, 'top_k': 0, 'top_p': 1.0, 'do_sample': True}

Generated text:
  In the year 2050, robots will be able to perform most of the
  tasks that humans do today. They will be used in factories,
  hospitals, and homes to assist with daily activities.

-----------------------------------------------------------------
Configuration: Balanced (medium temperature + top-p)
Parameters: {'temperature': 0.7, 'top_k': 0, 'top_p': 0.9, 'do_sample': True}

Generated text:
  In the year 2050, robots will have become an integral part of
```

```
  everyday life, handling everything from cooking meals to
  performing complex surgical procedures with remarkable precision.

────────────────────────────────────────────────────────────────
Configuration: Creative (high temperature + top-k)
Parameters: {'temperature': 1.2, 'top_k': 50, 'top_p': 1.0, 'do_sample': True}

Generated text:
  In the year 2050, robots will dream in electric colors and
  compose symphonies that make even the greatest human musicians
  weep with joy and existential confusion about their purpose.

────────────────────────────────────────────────────────────────
Configuration: Greedy (no sampling — always picks the most likely word)
Parameters: {'do_sample': False}

Generated text:
  In the year 2050, robots will be able to do everything that
  humans can do. They will be able to do everything that humans
  can do. They will be able to do everything that humans can do.
```

Notice the key patterns:

- Low temperature: Safe, predictable, factual, but can be boring
- Medium temperature with top-p: Good balance of creativity and coherence
- High temperature: Creative and surprising, but can be incoherent
- Greedy: Always picks the most likely word, often leading to repetitive loops

Choosing the Right Parameters

```
+-------------------------------------------------------------------+
|              Parameter Selection Guide                             |
+-------------------------------------------------------------------+
|                                                                    |
|  Use Case                  | Temperature | Top-P | Top-K         |
|  --------------------------|-------------|-------|-------         |
|  Factual Q&A               | 0.1 - 0.3  | 1.0   | 0 (off)      |
|  Code generation           | 0.2 - 0.4  | 0.95  | 0 (off)      |
|  Article writing           | 0.5 - 0.7  | 0.9   | 0 (off)      |
|  Creative writing          | 0.7 - 1.0  | 0.9   | 50            |
|  Brainstorming             | 0.9 - 1.2  | 0.95  | 100           |
|  Poetry / experimental     | 1.0 - 1.5  | 0.95  | 200           |
|                                                                    |
|  General recommendations:                                          |
|  - Start with temperature=0.7, top_p=0.9                          |
|  - Lower temperature for more focused output                       |
|  - Raise temperature for more diverse output                       |
|  - Use top_p=0.9 as a safety net to prevent nonsense              |
|  - Combining top_k and top_p is often unnecessary                 |
|                                                                    |
```

```
+-----------------------------------------------------------------+
```

10.7 Complete Text Generation Example

Let us build a complete text generation application:

```python
# Complete GPT-2 text generation application

from transformers import pipeline, set_seed

class TextGenerator:
    """
    A text generation tool using GPT-2 with configurable parameters.
    """

    def __init__(self, model_name="gpt2"):
        self.generator = pipeline("text-generation", model=model_name)
        self.model_name = model_name
        print(f"Loaded model: {model_name}")

    def generate(self, prompt, max_new_tokens=100, temperature=0.7,
                 top_p=0.9, top_k=0, num_sequences=1, seed=None):
        """
        Generate text from a prompt.

        Parameters:
            prompt: Starting text for generation
            max_new_tokens: How many new tokens to generate
            temperature: Randomness control (0.1=focused, 1.5=creative)
            top_p: Nucleus sampling threshold (0.0-1.0)
            top_k: Top-K sampling limit (0=disabled)
            num_sequences: How many different outputs to generate
            seed: Random seed for reproducibility
        """

        if seed is not None:
            set_seed(seed)

        results = self.generator(
            prompt,
            max_new_tokens=max_new_tokens,
            temperature=temperature,
            top_p=top_p,
            top_k=top_k if top_k > 0 else None,
            num_return_sequences=num_sequences,
            do_sample=True,
```

```
            pad_token_id=50256,
        )

        return [r['generated_text'] for r in results]

    def generate_with_styles(self, prompt, max_new_tokens=80):
        """
        Generate text in different styles by varying parameters.
        """

        styles = {
            "Precise": {"temperature": 0.2, "top_p": 0.9},
            "Balanced": {"temperature": 0.7, "top_p": 0.9},
            "Creative": {"temperature": 1.0, "top_p": 0.95},
        }

        results = {}
        for style_name, params in styles.items():
            output = self.generate(
                prompt,
                max_new_tokens=max_new_tokens,
                seed=42,
                **params
            )
            results[style_name] = output[0]

        return results


# Create the generator
gen = TextGenerator("gpt2")

# Example 1: Basic generation
print("\n" + "=" * 65)
print("Example 1: Basic Text Generation")
print("=" * 65)

prompt = "The future of education will be shaped by"
outputs = gen.generate(prompt, max_new_tokens=60, num_sequences=2, seed=42)

for i, text in enumerate(outputs):
    print(f"\nGeneration {i + 1}:")
    print(f"  {text}")

# Example 2: Different styles
print("\n" + "=" * 65)
print("Example 2: Same Prompt, Different Styles")
print("=" * 65)

prompt = "Once upon a time in a distant galaxy,"
styles = gen.generate_with_styles(prompt)

for style_name, text in styles.items():
    print(f"\n{style_name} Style:")
    print(f"  {text}")
```

```
# Example 3: Story continuation
print("\n" + "=" * 65)
print("Example 3: Story Continuation")
print("=" * 65)

story_start = (
    "The old lighthouse keeper climbed the spiral staircase "
    "one last time. Tonight was different. The beam of light "
    "revealed something in the fog that"
)

continuation = gen.generate(
    story_start, max_new_tokens=80, temperature=0.8, seed=123
)

print(f"\nStory:")
print(f"  {continuation[0]}")

# Example 4: Practical use cases
print("\n" + "=" * 65)
print("Example 4: Practical Use Cases")
print("=" * 65)

prompts = {
    "Email draft": "Dear hiring manager, I am writing to express my interest in",
    "Product description": "Introducing the SmartWatch Pro 3000 — a revolutionary devic...
    "Blog intro": "Ten tips for becoming a better programmer. Tip 1:",
}

for use_case, prompt in prompts.items():
    output = gen.generate(prompt, max_new_tokens=50, temperature=0.6, seed=42)
    print(f"\n{use_case}:")
    print(f"  {output[0]}")
```

Expected Output:

```
Loaded model: gpt2

=================================================================
Example 1: Basic Text Generation
=================================================================

Generation 1:
  The future of education will be shaped by technology and artificial
  intelligence. Online learning platforms will make quality education
  accessible to students around the world, regardless of their location
  or economic background.

Generation 2:
  The future of education will be shaped by the rapid development of
  new technologies that can personalize learning experiences. Students
  will be able to learn at their own pace with AI tutors that adapt
```

```
  to their individual needs.

==================================================================
Example 2: Same Prompt, Different Styles
==================================================================

Precise Style:
  Once upon a time in a distant galaxy, there was a small planet
  called Earth. It was home to billions of people who lived and
  worked together in peace. The planet was known for its beautiful
  oceans and vast forests.

Balanced Style:
  Once upon a time in a distant galaxy, there existed a civilization
  that had mastered the art of interstellar travel. Their ships
  sailed through nebulae and past dying stars, carrying explorers
  to worlds no one had ever imagined.

Creative Style:
  Once upon a time in a distant galaxy, the stars themselves would
  whisper secrets to those brave enough to listen. Captain Zara
  pressed her ear against the viewport of the Stardrifter and heard
  something that made her heart skip — a melody from home.

==================================================================
Example 3: Story Continuation
==================================================================

Story:
  The old lighthouse keeper climbed the spiral staircase one last
  time. Tonight was different. The beam of light revealed something
  in the fog that he had never seen before — a ship, enormous and
  silent, floating just above the waterline. Its hull shimmered
  with an otherworldly glow, and on its deck stood figures that
  were clearly not human.

==================================================================
Example 4: Practical Use Cases
==================================================================

Email draft:
  Dear hiring manager, I am writing to express my interest in the
  Software Engineer position at your company. With five years of
  experience in full-stack development and a strong background in
  Python and JavaScript, I believe I would be a valuable addition.

Product description:
  Introducing the SmartWatch Pro 3000 — a revolutionary device that
  combines cutting-edge health monitoring with seamless smartphone
  integration. Track your heart rate, sleep patterns, and daily
  activity with unprecedented accuracy.

Blog intro:
  Ten tips for becoming a better programmer. Tip 1: Write code
  every day. Even if it is just 15 minutes, daily practice builds
```

```
muscle memory and keeps your skills sharp. The key is consistency
over intensity.
```

Common Mistakes

1. Using GPT for classification tasks: GPT is designed for generation, not understanding. For classification tasks like sentiment analysis or NER, use BERT-based models instead.
2. Setting temperature to 0: A temperature of exactly 0 causes a division-by-zero error in some implementations. Use a very small value like 0.01 instead, or set `do_sample=False` for greedy decoding.
3. Not setting a pad_token_id: GPT-2 does not have a dedicated padding token. If you do not set `pad_token_id`, you may get warnings. Use `pad_token_id=50256` (GPT-2's end-of-text token).
4. Expecting factual accuracy: GPT generates text that SOUNDS correct but may contain fabricated facts. Never trust GPT's output for factual claims without verification.
5. Generating too little or too much text: If `max_length` is too small, the output may be cut off mid-sentence. If too large, the model may produce repetitive or incoherent text. Start with `max_new_tokens=100` and adjust.

Best Practices

1. Use max_new_tokens instead of max_length: `max_new_tokens` specifies how many tokens to generate AFTER the prompt. `max_length` includes the prompt length, which can be confusing. Prefer `max_new_tokens` for clarity.
2. Start with balanced parameters: Begin with `temperature=0.7` and `top_p=0.9`. These settings work well for most tasks. Adjust based on your specific needs.
3. Use seeds for reproducibility: Set `set_seed(42)` before generation if you need reproducible results. Without a seed, each generation will be different.
4. Craft good prompts: The quality of GPT's output depends heavily on the prompt. Be specific and provide context. "Write a poem about nature" produces worse results than "Write a short haiku poem about autumn leaves falling in a Japanese garden."
5. Post-process the output: GPT may generate text that ends mid-sentence. Add post-processing to trim the output at the last complete sentence.

Quick Summary

GPT is an autoregressive language model that generates text left to right, one token at a time. It is trained to predict the next token given all previous tokens. The GPT family evolved from GPT-2 (1.5B parameters) to GPT-3 (175B, few-shot learning) to GPT-4 (multimodal, RLHF-aligned). ChatGPT uses three training stages: pre-training on text, supervised fine-tuning with human-written responses, and RLHF where humans rank model outputs to teach preferences. Generation quality is controlled by temperature (randomness), top-k (limit to K most likely words), and top-p (nucleus sampling — include words until cumulative probability exceeds P). Low temperature produces focused, predictable text; high temperature produces creative, diverse text.

Key Points

- GPT is autoregressive: it generates text one token at a time, left to right
- Each word is predicted based on ALL previous words (left context only)
- BERT reads bidirectionally for understanding; GPT reads left-to-right for generation
- GPT-2 has 1.5B parameters; GPT-3 has 175B; GPT-4 is even larger
- GPT-3 introduced few-shot learning — performing tasks from examples in the prompt
- ChatGPT uses RLHF — humans rank responses to train a reward model
- Temperature controls randomness: low = focused, high = creative
- Top-K limits choices to the K most likely words
- Top-P dynamically selects words until cumulative probability exceeds P
- Greedy decoding (do_sample=False) often produces repetitive text
- For most tasks, start with temperature=0.7 and top_p=0.9
- GPT generates fluent text but may hallucinate facts

Practice Questions

1. Explain the difference between autoregressive (GPT) and bidirectional (BERT) models using an everyday analogy. Why can GPT not look at words to the right?

2. What is temperature in text generation? If you set temperature=0.1, what kind of output would you expect? What about temperature=1.5?
3. Explain the difference between top-k and top-p sampling. Why might top-p be more adaptive than top-k?
4. Describe the three stages of ChatGPT's training in your own words. Why is RLHF needed if supervised fine-tuning already teaches the model to give good responses?
5. You are building an AI assistant that helps users write professional emails. What temperature, top-k, and top-p values would you choose, and why?

Exercises

Exercise 1: Temperature Explorer Write a program that generates text from the same prompt using 5 different temperature values (0.2, 0.5, 0.7, 1.0, 1.5). Use the same seed for all generations. Compare the outputs and describe how the text changes as temperature increases.

Exercise 2: Prompt Engineering Write 5 different prompts for GPT-2 that produce substantially different outputs for the same topic (for example, "climate change"). Try: a news headline format, a children's story format, a scientific abstract format, a conversation format, and a poem format. Analyze how prompt structure affects the output.

Exercise 3: Text Completion App Build a simple text completion function that takes a user's partial sentence and generates 3 possible completions using different parameter settings (conservative, balanced, creative). Display all three options and let the user pick their favorite. Hint: use `num_return_sequences=3` with different seeds or temperatures.

What Is Next?

You now understand both sides of the Transformer: BERT (the encoder, for understanding) and GPT (the decoder, for generation). In the next chapter, you will learn about the Hugging Face ecosystem in depth — the platform that makes all of these models accessible. You will explore the Model Hub, learn how to find the right model for any task, and master the pipelines, tokenizers, and model classes that power modern NLP applications.

Chapter 11: Hugging Face Transformers Library

What You Will Learn

In this chapter, you will learn:

- What Hugging Face is and why it has become the center of modern NLP
- How to install the Transformers library and its dependencies
- How to use the pipeline API for common NLP tasks with just two lines of code
- How to work with AutoModel and AutoTokenizer for more control
- How to browse the Model Hub and choose the right model for your task
- How to load, use, and save pre-trained models

Why This Chapter Matters

Imagine you want to build a house. You could cut down trees, mill your own lumber, forge your own nails, and mix your own concrete. Or you could go to a hardware store where everything is ready to use.

Hugging Face is that hardware store for machine learning. Before Hugging Face, using state-of-the-art NLP models meant reading complex research papers, writing hundreds of lines of code, and spending days just getting things to work. Hugging Face changed all of that. With just two or three lines of Python, you can now use the same models that power products at Google, Microsoft, and Meta.

This chapter is your gateway to practical NLP. Every chapter that follows in this book builds on what you learn here.

11.1 What Is Hugging Face?

The GitHub of Machine Learning

Hugging Face is a company and an open-source community that provides tools for building applications with machine learning. Think of it as three things combined:

1. A library (Transformers) -- Python code that lets you use pre-trained models
2. A model hub -- A website where thousands of models are shared (like GitHub for ML models)
3. A community -- Researchers and developers sharing models, datasets, and knowledge

Here is a simple analogy:

```
+-----------------------------------------------------+
|              THE HUGGING FACE ECOSYSTEM             |
+-----------------------------------------------------+
|                                                     |
|  Model Hub          = App Store for AI models       |
|  Transformers lib   = The toolkit to use them       |
|  Datasets lib       = Ready-made training data      |
|  Spaces             = Free hosting for demos        |
|                                                     |
+-----------------------------------------------------+
```

Why Hugging Face Became So Popular

Before Hugging Face:

```
Step 1: Read a 20-page research paper
Step 2: Understand complex math
Step 3: Write 500+ lines of code
Step 4: Download model weights manually
Step 5: Debug for days
Step 6: Finally get a prediction
```

After Hugging Face:

```
Step 1: pip install transformers
Step 2: Write 2 lines of Python
Step 3: Get a prediction
```

That simplicity is why over 100,000 models are now hosted on the Hugging Face Hub.

11.2 Installing the Transformers Library

Basic Installation

Open your terminal or command prompt and run:

```python
# Install the transformers library
# This is the core library you need
pip install transformers
```

What this does: Downloads and installs the Hugging Face Transformers library along with its dependencies (other libraries it needs to work).

Installing with PyTorch Backend

Transformers needs a "backend" -- a deep learning framework that does the actual computation. PyTorch is the most popular choice:

```python
# Install transformers with PyTorch support
pip install transformers torch
```

Line-by-line explanation:

- `pip install` -- The Python package installer command
- `transformers` -- The Hugging Face library for using pre-trained models
- `torch` -- PyTorch, the deep learning framework that runs the models

Verifying Your Installation

After installation, open Python and check that everything works:

```python
# Check that transformers is installed correctly
import transformers

# Print the version number
print(f"Transformers version: {transformers.__version__}")

# Check that PyTorch is available
```

```
import torch
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```

Expected output:

```
Transformers version: 4.36.0
PyTorch version: 2.1.0
CUDA available: False
```

Line-by-line explanation:

- `import transformers` -- Loads the Transformers library into Python
- `transformers.__version__` -- A special variable that stores the version number
- `import torch` -- Loads PyTorch into Python
- `torch.cuda.is_available()` -- Checks if you have a GPU (graphics card) available for faster computation. `False` means you are using CPU only, which is fine for learning

> Note: Your version numbers may differ. That is perfectly normal. The examples in this book work with Transformers version 4.20 and above.

What Is CUDA?

CUDA stands for Compute Unified Device Architecture. It is technology from NVIDIA that lets your graphics card (GPU) do math much faster than your regular processor (CPU).

```
+-----------------------------------------+
|          PROCESSING SPEED ANALOGY        |
+-----------------------------------------+
|                                         |
|  CPU = One very fast chef               |
|        Can cook one dish at a time      |
|        Very good at complex recipes     |
|                                         |
|  GPU = 1000 slower cooks                |
|        Can cook 1000 simple dishes      |
|        at the same time                 |
|        Great for repetitive tasks       |
|                                         |
+-----------------------------------------+
```

For this book, a CPU is sufficient. A GPU just makes things faster.

11.3 The Pipeline API: NLP in Two Lines

The pipeline API is the simplest way to use a pre-trained model. It wraps all the complex steps (tokenization, model inference, post-processing) into a single function call.

How Pipelines Work

```
+-------------------------------------------------------------+
|                    PIPELINE INTERNALS                       |
+-------------------------------------------------------------+
|                                                             |
|  Your Text --> [Tokenizer] --> [Model] --> [Post-Process]   |
|  "I love it"   Converts to     Makes a     Formats the     |
|                 numbers         prediction  result nicely   |
|                                                             |
|  You only see: pipeline("I love it") --> "POSITIVE 0.99"    |
+-------------------------------------------------------------+
```

Sentiment Analysis Pipeline

Sentiment analysis determines whether text expresses a positive or negative opinion.

```python
from transformers import pipeline

# Create a sentiment analysis pipeline
# This downloads a pre-trained model the first time you run it
classifier = pipeline("sentiment-analysis")

# Analyze the sentiment of a sentence
result = classifier("I absolutely love this movie! It was fantastic.")
print(result)
```

Expected output:

```
[{'label': 'POSITIVE', 'score': 0.9998}]
```

Line-by-line explanation:

- `from transformers import pipeline` -- Imports the pipeline function from the Transformers library
- `pipeline("sentiment-analysis")` -- Creates a sentiment analysis pipeline. The string `"sentiment-analysis"` tells Hugging Face which type of task you want. The first time you run this, it downloads a pre-trained model (about 250 MB)

- `classifier(...)` -- Passes your text into the pipeline for analysis
- `result` -- A list of dictionaries containing the label (POSITIVE or NEGATIVE) and a confidence score between 0 and 1

You can also analyze multiple texts at once:

```python
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

# Analyze multiple sentences at once
texts = [
    "This restaurant has amazing food!",
    "The service was terrible and slow.",
    "The movie was okay, nothing special."
]

results = classifier(texts)

for text, result in zip(texts, results):
    print(f"Text: {text}")
    print(f"  Sentiment: {result['label']}, Confidence: {result['score']:.4f}")
    print()
```

Expected output:

```
Text: This restaurant has amazing food!
  Sentiment: POSITIVE, Confidence: 0.9999

Text: The service was terrible and slow.
  Sentiment: NEGATIVE, Confidence: 0.9998

Text: The movie was okay, nothing special.
  Sentiment: POSITIVE, Confidence: 0.6837
```

Line-by-line explanation:

- `texts = [...]` -- A list of three sentences to analyze
- `classifier(texts)` -- Processes all three sentences through the pipeline
- `zip(texts, results)` -- Pairs each text with its result so we can print them together
- `result['label']` -- Accesses the sentiment label from the result dictionary
- `result['score']:.4f` -- Formats the score to 4 decimal places

Notice that the third sentence gets a lower confidence score. The model is less certain about neutral-sounding text.

Named Entity Recognition (NER) Pipeline

Named Entity Recognition finds and classifies names of people, places, organizations, and other entities in text.

```python
from transformers import pipeline

# Create a NER pipeline
ner = pipeline("ner", grouped_entities=True)

# Find entities in a sentence
text = "Barack Obama was born in Hawaii and served as the 44th President of the United ...
entities = ner(text)

for entity in entities:
    print(f"Entity: {entity['word']}")
    print(f"  Type: {entity['entity_group']}")
    print(f"  Confidence: {entity['score']:.4f}")
    print()
```

Expected output:

```
Entity: Barack Obama
  Type: PER
  Confidence: 0.9988

Entity: Hawaii
  Type: LOC
  Confidence: 0.9993

Entity: United States
  Type: LOC
  Confidence: 0.9987
```

Line-by-line explanation:

- `pipeline("ner", grouped_entities=True)` -- Creates a NER pipeline. The `grouped_entities=True` parameter tells it to combine related word pieces into complete entity names (for example, "Barack" and "Obama" become "Barack Obama" instead of two separate entities)
- `entity['word']` -- The text that was identified as an entity
- `entity['entity_group']` -- The type of entity: PER (person), LOC (location), ORG (organization), MISC (miscellaneous)
- `entity['score']` -- How confident the model is in its prediction

```
+-----------------------------------------+
|          NER ENTITY TYPES               |
+-----------------------------------------+
|                                         |
```

```
|  PER  = Person      (Barack Obama)       |
|  LOC  = Location    (Hawaii, France)     |
|  ORG  = Organization (Google, NASA)      |
|  MISC = Miscellaneous (English, GDP)     |
|                                          |
+------------------------------------------+
```

Summarization Pipeline

Summarization condenses long text into a shorter version while keeping the key information.

```python
from transformers import pipeline

# Create a summarization pipeline
summarizer = pipeline("summarization")

# A longer text to summarize
article = """
Machine learning is a subset of artificial intelligence that focuses on
building systems that learn from data. Unlike traditional programming where
developers write explicit rules, machine learning algorithms find patterns
in data automatically. This approach has proven incredibly effective for
tasks like image recognition, natural language processing, and recommendation
systems. The field has grown rapidly in recent years, driven by increases in
computing power and the availability of large datasets. Today, machine
learning powers many of the services we use daily, from search engines to
voice assistants to social media feeds.
"""

summary = summarizer(article, max_length=50, min_length=20)
print("Summary:")
print(summary[0]['summary_text'])
```

Expected output:

```
Summary:
Machine learning is a subset of artificial intelligence that focuses on building system...
```

Line-by-line explanation:

- `pipeline("summarization")` -- Creates a summarization pipeline using a default model (usually BART or T5)
- `max_length=50` -- The maximum number of tokens (roughly words) in the summary
- `min_length=20` -- The minimum number of tokens in the summary
- `summary[0]['summary_text']` -- Extracts the summary text from the result. The result is a list containing a dictionary

Translation Pipeline

Translation converts text from one language to another.

```python
from transformers import pipeline

# Create a translation pipeline (English to French)
translator = pipeline("translation_en_to_fr")

# Translate a sentence
text = "Machine learning is changing the world."
result = translator(text)

print(f"English: {text}")
print(f"French:  {result[0]['translation_text']}")
```

Expected output:

```
English: Machine learning is changing the world.
French:  L'apprentissage automatique change le monde.
```

Line-by-line explanation:

- `pipeline("translation_en_to_fr")` -- Creates a pipeline that translates from English (`en`) to French (`fr`). Hugging Face supports many language pairs
- `result[0]['translation_text']` -- Extracts the translated text from the result

Question Answering Pipeline

Question answering finds the answer to a question within a given context (a passage of text).

```python
from transformers import pipeline

# Create a question answering pipeline
qa = pipeline("question-answering")

# Provide a context and ask a question
context = """
Python was created by Guido van Rossum and first released in 1991.
It is known for its simple syntax and readability. Python is widely
used in web development, data science, artificial intelligence, and
automation. It is one of the most popular programming languages in
the world.
"""

question = "Who created Python?"
```

```
result = qa(question=question, context=context)

print(f"Question: {question}")
print(f"Answer: {result['answer']}")
print(f"Confidence: {result['score']:.4f}")
print(f"Found at position: {result['start']} to {result['end']}")
```

Expected output:

```
Question: Who created Python?
Answer: Guido van Rossum
Confidence: 0.9876
Found at position: 26 to 42
```

Line-by-line explanation:

- `pipeline("question-answering")` -- Creates a QA pipeline
- `question=question, context=context` -- You must provide both the question and the text where the answer can be found
- `result['answer']` -- The text span that the model identified as the answer
- `result['score']` -- Confidence score between 0 and 1
- `result['start']` and `result['end']` -- Character positions in the context where the answer was found

Text Generation Pipeline

Text generation creates new text based on a prompt you provide.

```python
from transformers import pipeline

# Create a text generation pipeline
generator = pipeline("text-generation", model="gpt2")

# Generate text from a prompt
prompt = "In the future, artificial intelligence will"
result = generator(prompt, max_length=50, num_return_sequences=1)

print("Generated text:")
print(result[0]['generated_text'])
```

Expected output:

```
Generated text:
In the future, artificial intelligence will be able to understand human
```

```
emotions and respond appropriately. This technology could transform
healthcare, education, and how we interact with machines on a daily basis.
```

Line-by-line explanation:

- `pipeline("text-generation", model="gpt2")` -- Creates a text generation pipeline using the GPT-2 model specifically
- `max_length=50` -- Maximum number of tokens to generate
- `num_return_sequences=1` -- How many different versions of text to generate
- `result[0]['generated_text']` -- The generated text, which includes your original prompt

> Note: Text generation is not deterministic. Each time you run this code, you will likely get different output. This is because the model uses random sampling to choose the next word.

All Pipeline Tasks at a Glance

```
+-------------------------------------------------------------+
|              HUGGING FACE PIPELINE TASKS                     |
+-------------------------------------------------------------+
|                                                              |
|  Task                     | What It Does                     |
|  -------------------------|---------------------------------- |
|  sentiment-analysis       | Positive/negative classification |
|  ner                      | Find people, places, orgs in text |
|  summarization            | Shorten long text                |
|  translation_xx_to_yy     | Convert between languages        |
|  question-answering       | Find answers in a passage        |
|  text-generation          | Generate new text from a prompt  |
|  fill-mask                | Predict missing words            |
|  zero-shot-classification| Classify without training        |
|  text2text-generation     | General text-to-text tasks       |
|  feature-extraction       | Get numerical representations    |
|                                                              |
+-------------------------------------------------------------+
```

11.4 AutoModel and AutoTokenizer: More Control

The pipeline API is convenient, but sometimes you need more control. That is where AutoModel and AutoTokenizer come in.

What Are Tokenizers?

A tokenizer converts human-readable text into numbers that a model can understand.

```
+-------------------------------------------------------------+
|                HOW TOKENIZATION WORKS                       |
+-------------------------------------------------------------+
|                                                             |
|  "I love cats"                                              |
|       |                                                     |
|       v                                                     |
|  [Tokenizer]                                                |
|       |                                                     |
|       v                                                     |
|  ["I", "love", "cats"]     <-- Split into tokens            |
|       |                                                     |
|       v                                                     |
|  [1045, 2293, 8870]        <-- Convert to numbers (IDs)     |
|       |                                                     |
|       v                                                     |
|  [Model processes these numbers]                            |
|                                                             |
+-------------------------------------------------------------+
```

Using AutoTokenizer

```python
from transformers import AutoTokenizer

# Load a tokenizer for BERT
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Tokenize a sentence
text = "Hugging Face makes NLP easy!"
tokens = tokenizer(text, return_tensors="pt")

# See what the tokenizer produced
print("Input text:", text)
print("Token IDs:", tokens["input_ids"])
print("Attention mask:", tokens["attention_mask"])

# Convert IDs back to tokens to see how text was split
token_list = tokenizer.convert_ids_to_tokens(tokens["input_ids"][0])
print("Tokens:", token_list)
```

Expected output:

```
Input text: Hugging Face makes NLP easy!
Token IDs: tensor([[  101,  17662,  2227,  3084,  17953,  2959,   999,   102]])
```

```
Attention mask: tensor([[1, 1, 1, 1, 1, 1, 1, 1]])
Tokens: ['[CLS]', 'hugging', 'face', 'makes', 'nl', '##p', 'easy', '!', '[SEP]']
```

Line-by-line explanation:

- `AutoTokenizer.from_pretrained("bert-base-uncased")` -- Downloads and loads the tokenizer that was used to train the BERT model. "uncased" means it converts everything to lowercase
- `tokenizer(text, return_tensors="pt")` -- Tokenizes the text and returns PyTorch tensors (`"pt"` stands for PyTorch)
- `tokens["input_ids"]` -- The numerical IDs assigned to each token
- `tokens["attention_mask"]` -- A mask of 1s and 0s. A `1` means "pay attention to this token" and `0` means "ignore this (it is padding)"
- `[CLS]` -- A special token BERT adds at the beginning of every input (stands for "classification")
- `[SEP]` -- A special token BERT adds at the end (stands for "separator")
- `##p` -- The `##` prefix means this is a sub-word. "NLP" was split into "nl" and "##p" because the tokenizer breaks unknown words into smaller known pieces

Understanding Sub-word Tokenization

Most modern tokenizers do not split text into whole words. They use sub-word tokenization, which breaks rare or unknown words into smaller pieces:

```
+-------------------------------------------------------------+
|           SUB-WORD TOKENIZATION EXAMPLE                     |
+-------------------------------------------------------------+
|                                                              |
|  "unhappiness"  -->  ["un", "##happiness"]                  |
|                                                              |
|  Why? The model learned "un" and "happiness" separately.    |
|  It can now understand any word starting with "un-"          |
|  even if it never saw "unhappiness" during training.         |
|                                                              |
|  "transformers"  -->  ["transform", "##ers"]                |
|  "ChatGPT"       -->  ["chat", "##g", "##pt"]               |
|                                                              |
+-------------------------------------------------------------+
```

This is clever because it means the model can handle words it has never seen before by breaking them into known pieces.

Using AutoModel

```python
from transformers import AutoTokenizer, AutoModel
import torch

# Load model and tokenizer
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Tokenize input text
text = "Transformers are amazing!"
inputs = tokenizer(text, return_tensors="pt")

# Run the model (no gradient computation needed for inference)
with torch.no_grad():
    outputs = model(**inputs)

# The output contains hidden states (numerical representations)
print("Output shape:", outputs.last_hidden_state.shape)
print("This means:")
print(f"  - Batch size: {outputs.last_hidden_state.shape[0]} (one sentence)")
print(f"  - Sequence length: {outputs.last_hidden_state.shape[1]} tokens")
print(f"  - Hidden size: {outputs.last_hidden_state.shape[2]} dimensions")
```

Expected output:

```
Output shape: torch.Size([1, 6, 768])
This means:
  - Batch size: 1 (one sentence)
  - Sequence length: 6 tokens
  - Hidden size: 768 dimensions
```

Line-by-line explanation:

- `AutoModel.from_pretrained(model_name)` -- Downloads and loads the pre-trained BERT model (about 440 MB the first time)
- `with torch.no_grad():` -- Tells PyTorch not to track computations for gradient calculation. This saves memory and speeds things up. We only need this during training, not when making predictions
- `model(**inputs)` -- The `**` unpacks the dictionary into keyword arguments. It is the same as writing `model(input_ids=..., attention_mask=...)`
- `outputs.last_hidden_state` -- The model's output: a numerical representation of each token. Each token is represented by 768 numbers (for BERT base)

AutoModel vs Pipeline: When to Use Which

```
+-----------------------------------------------------------------+
|         PIPELINE vs AUTOMODEL COMPARISON                        |
+-----------------------------------------------------------------+
|                                                                 |
|  Pipeline:                                                      |
|    + Simple: 2 lines of code                                    |
|    + Handles all pre/post processing                            |
|    + Perfect for quick prototyping                              |
|    - Less control over model behavior                           |
|    - Cannot access intermediate outputs                         |
|                                                                 |
|  AutoModel + AutoTokenizer:                                     |
|    + Full control over every step                               |
|    + Access to raw model outputs                                |
|    + Can customize processing                                   |
|    + Required for fine-tuning                                   |
|    - More code to write                                         |
|    - Need to handle pre/post processing yourself                |
|                                                                 |
+-----------------------------------------------------------------+
```

11.5 The Model Hub: Finding the Right Model

What Is the Model Hub?

The Hugging Face Model Hub is a website (huggingface.co/models) where anyone can share pre-trained models. Think of it as an app store, but for AI models.

```
+-----------------------------------------------------------------+
|              THE MODEL HUB                                      |
+-----------------------------------------------------------------+
|                                                                 |
|  100,000+ models available                                      |
|                                                                 |
|  Filter by:                                                     |
|    - Task (text classification, translation, etc.)              |
|    - Language (English, French, Chinese, etc.)                  |
|    - Library (PyTorch, TensorFlow, JAX)                         |
|    - Dataset (what data it was trained on)                      |
|    - License (can you use it commercially?)                     |
|                                                                 |
|  Each model page includes:                                      |
|    - Description and documentation                              |
|    - Example code                                               |
|    - Performance metrics                                        |
|    - Download statistics                                        |
```

```
|                                                                |
+----------------------------------------------------------------+
```

Loading a Specific Model

Instead of using the default model, you can specify exactly which model to use:

```python
from transformers import pipeline

# Use a specific model for sentiment analysis
# This model was trained on product reviews
classifier = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment"
)

# This model gives star ratings (1-5) instead of positive/negative
review = "The product quality is excellent and delivery was fast!"
result = classifier(review)

print(f"Review: {review}")
print(f"Rating: {result[0]['label']}")
print(f"Confidence: {result[0]['score']:.4f}")
```

Expected output:

```
Review: The product quality is excellent and delivery was fast!
Rating: 5 stars
Confidence: 0.7342
```

Line-by-line explanation:

- `model="nlptown/bert-base-multilingual-uncased-sentiment"` -- Specifies a particular model from the Hub. The format is `username/model-name`
- This model was trained on product reviews in multiple languages and gives star ratings from 1 to 5 instead of simple positive/negative

How to Choose the Right Model

Choosing the right model depends on your specific needs. Here is a decision framework:

```
+----------------------------------------------------------------+
|           MODEL SELECTION DECISION TREE                        |
+----------------------------------------------------------------+
```

```
|                                                                |
|  1. What TASK do you need?                                    |
|     --> Filter models by task on the Hub                      |
|                                                                |
|  2. What LANGUAGE is your text in?                            |
|     --> English: most models work                             |
|     --> Other: look for multilingual or language-specific      |
|                                                                |
|  3. How much COMPUTING POWER do you have?                     |
|     --> Limited: use "small" or "tiny" models                 |
|     --> Good GPU: use "base" or "large" models                |
|                                                                |
|  4. What is your use case LICENSE?                            |
|     --> Personal/research: most models are fine               |
|     --> Commercial: check the model's license                 |
|                                                                |
|  5. How many DOWNLOADS does the model have?                   |
|     --> More downloads usually means more reliable             |
|                                                                |
+----------------------------------------------------------------+
```

Popular Models for Common Tasks

```python
from transformers import pipeline

# Sentiment analysis - good default model
sentiment = pipeline("sentiment-analysis",
                     model="distilbert-base-uncased-finetuned-sst-2-english")

# Named Entity Recognition - good default model
ner = pipeline("ner",
               model="dbmdz/bert-large-cased-finetuned-conll03-english",
               grouped_entities=True)

# Summarization - good default model
summarizer = pipeline("summarization",
                      model="facebook/bart-large-cnn")

# Translation (English to German)
translator = pipeline("translation",
                      model="Helsinki-NLP/opus-mt-en-de")

# Text generation
generator = pipeline("text-generation",
                     model="gpt2")

# Let us test each one
print("=== Sentiment ===")
print(sentiment("I love learning about AI!"))
```

```
print("\n=== NER ===")
print(ner("Apple Inc. is based in Cupertino, California."))

print("\n=== Translation ===")
print(translator("Hello, how are you?"))
```

Expected output:

```
=== Sentiment ===
[{'label': 'POSITIVE', 'score': 0.9998}]

=== NER ===
[{'entity_group': 'ORG', 'score': 0.9975, 'word': 'Apple Inc.', 'start': 0, 'end': 10},
 {'entity_group': 'LOC', 'score': 0.9990, 'word': 'Cupertino', 'start': 23, 'end': 32},
 {'entity_group': 'LOC', 'score': 0.9987, 'word': 'California', 'start': 34, 'end': 44}]

=== Translation ===
[{'translation_text': 'Hallo, wie geht es Ihnen?'}]
```

11.6 Loading and Saving Models

Saving a Model Locally

Once you have downloaded a model, you can save it to your computer so you do not need to download it again:

```python
from transformers import AutoTokenizer, AutoModel

# Load model and tokenizer from the Hub
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Save to a local directory
save_path = "./my_saved_model"
tokenizer.save_pretrained(save_path)
model.save_pretrained(save_path)

print(f"Model saved to {save_path}")
```

Expected output:

```
Model saved to ./my_saved_model
```

Line-by-line explanation:

- `save_pretrained(save_path)` -- Saves all model files and tokenizer files to the specified directory
- This creates several files: model weights, configuration, tokenizer vocabulary, and more

Loading a Model from a Local Directory

```python
from transformers import AutoTokenizer, AutoModel

# Load from your saved local directory (no internet needed!)
save_path = "./my_saved_model"
tokenizer = AutoTokenizer.from_pretrained(save_path)
model = AutoModel.from_pretrained(save_path)

print("Model loaded from local directory!")
print(f"Model type: {model.config.model_type}")
print(f"Vocabulary size: {tokenizer.vocab_size}")
```

Expected output:

```
Model loaded from local directory!
Model type: distilbert
Vocabulary size: 30522
```

Understanding Model Sizes

Different model sizes trade off between speed and accuracy:

```
+-------------------------------------------------------------+
|              MODEL SIZE COMPARISON                           |
+-------------------------------------------------------------+
|                                                               |
|  Model Name               | Parameters | Size  | Speed        |
|  -------------------------|------------|-------|-------------- |
|  distilbert-base-uncased | 66M        | 250MB | Very fast    |
|  bert-base-uncased        | 110M       | 440MB | Fast         |
|  bert-large-uncased       | 340M       | 1.3GB | Moderate     |
|  gpt2                     | 117M       | 500MB | Fast         |
|  gpt2-medium              | 345M       | 1.4GB | Moderate     |
|  gpt2-large               | 774M       | 3.1GB | Slow         |
|                                                               |
```

```
|  Rule of thumb:                                                  |
|  Larger model = Better accuracy but slower and more memory      |
|  Smaller model = Faster but may sacrifice some accuracy         |
|                                                                  |
+------------------------------------------------------------------+
```

> Tip: Start with DistilBERT for classification tasks and GPT-2 (small) for generation tasks. Move to larger models only if accuracy is not sufficient.

11.7 Putting It All Together: A Complete Example

Let us build a small text analysis tool that uses multiple pipelines:

```python
from transformers import pipeline

def analyze_text(text):
    """
    Analyze a piece of text using multiple NLP techniques.

    Parameters:
        text: The text to analyze (a string)

    Returns:
        A dictionary containing analysis results
    """

    # Initialize pipelines (in practice, do this once, not every call)
    print("Loading models... (this may take a moment the first time)")
    sentiment_analyzer = pipeline("sentiment-analysis")
    ner_analyzer = pipeline("ner", grouped_entities=True)

    # Step 1: Sentiment Analysis
    print("\n--- Sentiment Analysis ---")
    sentiment = sentiment_analyzer(text)
    print(f"Sentiment: {sentiment[0]['label']}")
    print(f"Confidence: {sentiment[0]['score']:.4f}")

    # Step 2: Named Entity Recognition
    print("\n--- Named Entities ---")
    entities = ner_analyzer(text)
    if entities:
        for entity in entities:
            print(f"  {entity['word']} ({entity['entity_group']})"
                  f" - confidence: {entity['score']:.4f}")
    else:
```

```
        print("  No named entities found.")

    # Step 3: Basic statistics
    print("\n--- Text Statistics ---")
    words = text.split()
    sentences = text.count('.') + text.count('!') + text.count('?')
    print(f"  Word count: {len(words)}")
    print(f"  Sentence count: {sentences}")
    print(f"  Average words per sentence: {len(words)/max(sentences,1):.1f}")

    return {
        "sentiment": sentiment[0],
        "entities": entities,
        "word_count": len(words),
        "sentence_count": sentences
    }


# Test with a sample text
sample_text = """
Apple announced its new iPhone today in Cupertino.
Tim Cook presented the device at the Steve Jobs Theater.
The audience was thrilled with the new features and design.
"""

results = analyze_text(sample_text.strip())
```

Expected output:

```
Loading models... (this may take a moment the first time)

--- Sentiment Analysis ---
Sentiment: POSITIVE
Confidence: 0.9985

--- Named Entities ---
  Apple (ORG) - confidence: 0.9978
  iPhone (MISC) - confidence: 0.9432
  Cupertino (LOC) - confidence: 0.9991
  Tim Cook (PER) - confidence: 0.9988
  Steve Jobs Theater (LOC) - confidence: 0.9845

--- Text Statistics ---
  Word count: 27
  Sentence count: 3
  Average words per sentence: 9.0
```

Line-by-line explanation:

- `def analyze_text(text):` -- Defines a function that takes any text and runs multiple analyses on it

- We create two pipeline objects: one for sentiment and one for NER
- The function runs sentiment analysis, finds named entities, and calculates basic text statistics
- It returns a dictionary with all results for further use
- `text.strip()` -- Removes leading and trailing whitespace from the sample text

Common Mistakes

1. Not having enough disk space. Pre-trained models can be several gigabytes. Make sure you have at least 5 GB of free space before downloading large models.
2. Forgetting `return_tensors="pt"` when using AutoTokenizer. Without this parameter, the tokenizer returns plain Python lists instead of PyTorch tensors, and the model will not accept them.
3. Using the wrong model for the task. A model trained for sentiment analysis cannot do translation. Always match the model to your task.
4. Running models without `torch.no_grad()` during inference. This wastes memory by tracking gradients you do not need. Always wrap inference code in `with torch.no_grad():`.
5. Expecting identical output every time from text generation. Generation models use random sampling by default. Set `do_sample=False` for deterministic output if needed.

Best Practices

1. Start with pipelines. Use the pipeline API for quick experiments and prototyping. Switch to AutoModel only when you need more control.
2. Save models locally. After downloading a model once, save it locally to avoid repeated downloads and to work offline.
3. Choose the smallest model that meets your accuracy needs. DistilBERT is 60% smaller and 60% faster than BERT with 97% of the accuracy.
4. Reuse pipeline objects. Create the pipeline once and reuse it for multiple predictions. Do not create a new pipeline for every prediction.
5. Check model cards on the Hub. Every good model has a "model card" that explains what it was trained on, its limitations, and how to use it properly.

Quick Summary

Hugging Face Transformers is a Python library that gives you access to thousands of pre-trained NLP models. The pipeline API lets you perform common tasks like sentiment analysis, NER, summarization, translation, question answering, and text generation with just two lines of code. For more control, use AutoModel and AutoTokenizer to handle tokenization and model inference manually. The Model Hub hosts over 100,000 models that you can filter by task, language, and size. Always start with a small model and scale up only if needed.

Key Points

- Hugging Face is the central ecosystem for modern NLP, providing models, datasets, and tools
- The `pipeline()` function is the easiest way to use pre-trained models -- just specify the task name
- Common pipeline tasks include sentiment-analysis, ner, summarization, translation, question-answering, and text-generation
- AutoTokenizer converts text to numbers, AutoModel processes those numbers
- Sub-word tokenization lets models handle words they have never seen before
- The Model Hub has over 100,000 models filterable by task, language, and size
- Smaller models (like DistilBERT) are often good enough and much faster
- Save models locally with `save_pretrained()` to avoid repeated downloads

Practice Questions

1. What is the difference between using `pipeline("sentiment-analysis")` and using `AutoTokenizer` plus `AutoModel` separately? When would you choose one approach over the other?
2. Explain what sub-word tokenization is and why it is useful. What does the `##` prefix mean in BERT tokenizer output?
3. You need to build an NLP application that detects the names of people, companies, and locations in news articles. Which pipeline task would you use, and what entity types

would the model return?

4. A colleague says their model is too slow. It is using `bert-large-uncased` (340M parameters). What model would you suggest as a faster alternative, and what trade-off would they be making?
5. Why is it important to use `with torch.no_grad():` when making predictions with AutoModel? What happens if you forget it?

Exercises

Exercise 1: Multi-Language Sentiment Analysis

Create a program that uses the `nlptown/bert-base-multilingual-uncased-sentiment` model to analyze product reviews. The program should accept a list of reviews and print the star rating for each one.

Hint: This model returns labels like "1 star", "2 stars", etc.

Exercise 2: Build a Simple FAQ Bot

Using the question-answering pipeline, build a program that:

- Stores a company FAQ as a string (make up a small company and 3-4 FAQ entries)
- Accepts user questions
- Returns the answer found in the FAQ along with the confidence score

Hint: Combine all FAQ text into one context string and use `pipeline("question-answering")`.

Exercise 3: Text Analysis Dashboard

Create a function that takes any text and produces a complete analysis report including:

- Sentiment (positive/negative and confidence)
- Named entities found (with types)
- A one-sentence summary (using the summarization pipeline)
- Word count and sentence count

Print the results in a nicely formatted report.

What Is Next?

Now that you know how to use pre-trained models, the next chapter takes things further. You will learn how to fine-tune a BERT model on your own data. Fine-tuning is like taking a general-purpose tool and sharpening it for your specific task. Instead of using a model trained on generic text, you will create a model that is an expert in exactly what you need -- starting with movie review sentiment classification.

Chapter 12: Fine-Tuning BERT for Classification

What You Will Learn

In this chapter, you will learn:

- Why fine-tuning is necessary and how it differs from using pre-trained models directly
- How to prepare your own dataset for training using the Dataset class
- How to tokenize text with proper padding and truncation
- How to set up the Trainer and TrainingArguments for training
- How to train, evaluate, and save a fine-tuned model
- How to build a complete movie review sentiment classifier from scratch

Why This Chapter Matters

Imagine you hire a chef who graduated from the best culinary school in the world. This chef knows everything about cooking techniques, ingredients, and flavors. But you run a sushi restaurant, and this chef has never made sushi before.

Do you send the chef back to culinary school for four years? Of course not. You give the chef a few weeks of sushi-specific training. The chef already understands food -- they just need to learn your specialty.

That is exactly what fine-tuning does. BERT was trained on billions of words and understands language deeply. But it was not trained on your specific task. Fine-tuning takes that general knowledge and adapts it to your exact needs -- whether that is classifying movie reviews, detecting spam, or categorizing support tickets.

Fine-tuning is one of the most powerful techniques in modern NLP. It lets you build high-accuracy models with relatively little data and training time.

12.1 Understanding Fine-Tuning

Pre-Training vs Fine-Tuning

BERT's life has two phases:

```
+-----------------------------------------------------------------+
|          TWO PHASES OF BERT'S TRAINING                          |
+-----------------------------------------------------------------+
|                                                                   |
|  Phase 1: PRE-TRAINING (done by researchers)                     |
|  +-----------------------------------------------------+          |
|  | Trained on billions of words from books & web   |          |
|  | Learns grammar, facts, word relationships       |          |
|  | Takes weeks on hundreds of GPUs                 |          |
|  | Cost: Hundreds of thousands of dollars           |          |
|  | Result: A model that "understands" language      |          |
|  +-----------------------------------------------------+          |
|                                                                   |
|  Phase 2: FINE-TUNING (done by you!)                             |
|  +-----------------------------------------------------+          |
|  | Trained on YOUR specific data (hundreds/thousands|          |
|  | of examples)                                     |          |
|  | Adapts the model to YOUR specific task           |          |
|  | Takes minutes to hours on a single GPU           |          |
|  | Cost: Almost free                                |          |
|  | Result: A model that excels at YOUR task          |          |
|  +-----------------------------------------------------+          |
|                                                                   |
+-----------------------------------------------------------------+
```

What Happens During Fine-Tuning

During fine-tuning, we make small adjustments to BERT's internal weights so it performs well on our specific task:

```
+-----------------------------------------------------------------+
|           FINE-TUNING ARCHITECTURE                              |
+-----------------------------------------------------------------+
|                                                                   |
|  Input: "This movie was terrible"                                |
|              |                                                    |
|              v                                                    |
|  +----------------------+                                         |
```

```
|  |     Tokenizer        |  Converts text to numbers           |
|  +----------------------+                                      |
|              |                                                 |
|              v                                                 |
|  +----------------------+                                      |
|  |     BERT Model       |  Pre-trained weights                |
|  |  (slightly adjusted  |  (adjusted during fine-tuning)      |
|  |   during training)   |                                      |
|  +----------------------+                                      |
|              |                                                 |
|              v                                                 |
|  +----------------------+                                      |
|  | Classification Head  |  NEW layer added on top             |
|  |  (Linear Layer)      |  (trained from scratch)             |
|  +----------------------+                                      |
|              |                                                 |
|              v                                                 |
|  Output: NEGATIVE (0.97)                                       |
|                                                                |
+----------------------------------------------------------------+
```

The key idea: We keep most of BERT's knowledge and just add a small new layer on top for our specific task.

Why Not Train From Scratch?

```
+----------------------------------------------------------------+
|     TRAINING FROM SCRATCH vs FINE-TUNING                       |
+----------------------------------------------------------------+
|                                                                |
|  From Scratch:                                                 |
|    - Need millions of examples                                |
|    - Takes days or weeks of training                          |
|    - Requires expensive hardware (multiple GPUs)              |
|    - Model learns everything from zero                        |
|                                                                |
|  Fine-Tuning:                                                  |
|    - Need hundreds to thousands of examples                   |
|    - Takes minutes to hours                                   |
|    - Works on a single GPU or even CPU                        |
|    - Model already knows language, just learns your task      |
|                                                                |
+----------------------------------------------------------------+
```

12.2 Preparing Your Dataset

Our Task: Movie Review Sentiment Classification

We will build a model that reads movie reviews and predicts whether the review is positive or negative. We will use the IMDB dataset, which contains 50,000 movie reviews.

Loading the Dataset

Hugging Face provides a `datasets` library that makes loading datasets easy:

```python
# First, install the datasets library
# pip install datasets

from datasets import load_dataset

# Load the IMDB movie review dataset
# This downloads the dataset the first time you run it
dataset = load_dataset("imdb")

# Let us see what we got
print("Dataset structure:")
print(dataset)
print()

# Look at the training set
print(f"Training examples: {len(dataset['train'])}")
print(f"Test examples: {len(dataset['test'])}")
```

Expected output:

```
Dataset structure:
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
})

Training examples: 25000
Test examples: 25000
```

Line-by-line explanation:

- `from datasets import load_dataset` -- Imports the dataset loading function from Hugging Face's datasets library

- `load_dataset("imdb")` -- Downloads and loads the IMDB dataset. The string `"imdb"` is the dataset name on the Hugging Face Hub
- `dataset['train']` -- The training portion of the dataset
- `dataset['test']` -- The test portion used to evaluate the model

Exploring the Dataset

```python
from datasets import load_dataset

dataset = load_dataset("imdb")

# Look at the first example
example = dataset['train'][0]
print(f"Label: {example['label']} ({'positive' if example['label'] == 1 else 'negative'...
print(f"Review (first 200 chars): {example['text'][:200]}...")
print()

# Check the label distribution
# Count how many positive and negative reviews there are
labels = dataset['train']['label']
positive_count = sum(labels)
negative_count = len(labels) - positive_count

print(f"Positive reviews: {positive_count}")
print(f"Negative reviews: {negative_count}")
print(f"Balance: {'balanced' if abs(positive_count - negative_count) < 100 else 'imbala...
```

Expected output:

```
Label: 0 (negative)
Review (first 200 chars): I rented I AM CURIOUS-YELLOW from my video store because of a...

Positive reviews: 12500
Negative reviews: 12500
Balance: balanced
```

Line-by-line explanation:

- `dataset['train'][0]` -- Gets the first example from the training set
- `example['label']` -- The label is 0 (negative) or 1 (positive)
- `example['text']` -- The actual review text
- `example['text'][:200]` -- Shows only the first 200 characters to keep output manageable
- The dataset is balanced: equal numbers of positive and negative reviews

Using a Smaller Subset for Faster Training

Training on 25,000 examples takes time. For learning, let us use a smaller subset:

```python
from datasets import load_dataset

dataset = load_dataset("imdb")

# Use a smaller subset for faster training
# select() picks specific indices from the dataset
small_train = dataset['train'].select(range(2000))
small_test = dataset['test'].select(range(500))

print(f"Small training set: {len(small_train)} examples")
print(f"Small test set: {len(small_test)} examples")
```

Expected output:

```
Small training set: 2000 examples
Small test set: 500 examples
```

Line-by-line explanation:

- `dataset['train'].select(range(2000))` -- Picks the first 2,000 examples from the training set. `range(2000)` generates numbers 0 through 1999
- This smaller dataset trains much faster while still producing good results

---

12.3 Tokenizing the Data

Why Tokenization Matters for Fine-Tuning

Before we can feed text into BERT, we must convert it to numbers. But there are important details to handle:

```
+-------------------------------------------------------------+
|           TOKENIZATION CHALLENGES                            |
+-------------------------------------------------------------+
|                                                              |
|  Problem 1: Different reviews have different lengths         |
|    "Great movie!"          --> 3 tokens                      |
|    "This was the best..."  --> 250 tokens                    |
|    BERT processes batches, so all inputs must be same length |
```

```
|                                                                  |
|  Solution: PADDING and TRUNCATION                               |
|                                                                  |
|  Padding:    Add zeros to make short texts longer               |
|  Truncation: Cut long texts to fit maximum length               |
|                                                                  |
|  "Great movie!"  --> [101, 2307, 3185, 999, 102, 0, 0, 0]   |
|                                                 ^^^^^^^^^^^      |
|                                                 padding          |
|                                                                  |
+------------------------------------------------------------------+
```

Tokenizing with Padding and Truncation

```python
from transformers import AutoTokenizer
from datasets import load_dataset

# Load the tokenizer for BERT
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Load dataset
dataset = load_dataset("imdb")
small_train = dataset['train'].select(range(2000))
small_test = dataset['test'].select(range(500))

# Define a tokenization function
def tokenize_function(examples):
    """
    Tokenize a batch of examples.

    Parameters:
        examples: A dictionary with 'text' and 'label' keys

    Returns:
        Tokenized examples with input_ids, attention_mask, etc.
    """
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=256
    )

# Apply tokenization to the entire dataset
# The map() function applies our function to every example
tokenized_train = small_train.map(tokenize_function, batched=True)
tokenized_test = small_test.map(tokenize_function, batched=True)

print("Tokenization complete!")
print(f"Columns in tokenized data: {tokenized_train.column_names}")
```

```
print(f"First example keys: {list(tokenized_train[0].keys())}")
print(f"Input IDs length: {len(tokenized_train[0]['input_ids'])}")
```

Expected output:

```
Tokenization complete!
Columns in tokenized data: ['text', 'label', 'input_ids', 'token_type_ids', 'attention_...
First example keys: ['text', 'label', 'input_ids', 'token_type_ids', 'attention_mask']
Input IDs length: 256
```

Line-by-line explanation:

- `AutoTokenizer.from_pretrained("bert-base-uncased")` -- Loads BERT's tokenizer. "uncased" means all text is converted to lowercase
- `tokenizer(examples["text"], ...)` -- Tokenizes the text with several important parameters:
  - `padding="max_length"` -- Adds zeros at the end until every input is exactly `max_length` tokens long
  - `truncation=True` -- If a text is longer than `max_length`, cut it off
  - `max_length=256` -- The maximum number of tokens. BERT can handle up to 512, but 256 is faster and usually sufficient
- `small_train.map(tokenize_function, batched=True)` -- Applies the tokenization function to every example in the dataset. `batched=True` processes multiple examples at once for speed

Understanding Padding and Attention Masks

```python
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Tokenize a short sentence with padding
short_text = "I love this movie!"
tokens = tokenizer(short_text, padding="max_length", max_length=10,
                   truncation=True, return_tensors="pt")

print("Text:", short_text)
print("Input IDs: ", tokens["input_ids"][0].tolist())
print("Attention:  ", tokens["attention_mask"][0].tolist())

# Decode back to see what each ID means
decoded = tokenizer.convert_ids_to_tokens(tokens["input_ids"][0])
print("Tokens:     ", decoded)
```

Expected output:

```
Text: I love this movie!
Input IDs:  [101, 1045, 2293, 2023, 3185, 999, 102, 0, 0, 0]
Attention:   [1, 1, 1, 1, 1, 1, 1, 0, 0, 0]
Tokens:      ['[CLS]', 'i', 'love', 'this', 'movie', '!', '[SEP]', '[PAD]', '[PAD]', '[...
```

Line-by-line explanation:

- The sentence has 7 real tokens (including [CLS] and [SEP])
- Three `[PAD]` tokens (ID = 0) are added to reach the `max_length` of 10
- The attention mask is `1` for real tokens and `0` for padding
- The `0` in the attention mask tells the model: "ignore these positions, they are not real text"

```
+-----------------------------------------------------------------+
|          PADDING AND ATTENTION MASK                             |
+-----------------------------------------------------------------+
|                                                                  |
|  Input IDs:    [101, 1045, 2293, 2023, 3185, 999, 102, 0, 0] |
|  Attention:    [ 1,    1,    1,    1,    1,   1,   1,  0, 0]  |
|                                                                  |
|  The model:                                                      |
|    - Processes tokens where attention = 1                        |
|    - Ignores tokens where attention = 0 (padding)                |
|    - This way, padding does not affect the prediction            |
|                                                                  |
+-----------------------------------------------------------------+
```

12.4 Setting Up the Trainer

What Is the Trainer API?

The Trainer is Hugging Face's high-level training loop. It handles all the complex training logic for you:

```
+-----------------------------------------------------------------+
|          WHAT THE TRAINER DOES FOR YOU                          |
+-----------------------------------------------------------------+
|                                                                  |
|  Without Trainer (manual training loop):                        |
|    - Write your own training loop                               |
|    - Handle batching manually                                   |
```

```
|    - Compute loss and gradients yourself                      |
|    - Update weights manually                                  |
|    - Track metrics yourself                                   |
|    - Save checkpoints yourself                                |
|    - Handle GPU/CPU device placement                          |
|    = 50-100 lines of code                                     |
|                                                                |
|  With Trainer:                                                 |
|    - Configure with TrainingArguments                         |
|    - Call trainer.train()                                      |
|    - Everything else is automatic                              |
|    = 10-15 lines of code                                      |
|                                                                |
+---------------------------------------------------------------+
```

Loading the Model for Classification

For fine-tuning, we use `AutoModelForSequenceClassification` instead of `AutoModel`. This version has an extra classification layer on top:

```python
from transformers import AutoModelForSequenceClassification

# Load BERT with a classification head on top
# num_labels=2 means two classes: negative (0) and positive (1)
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)

# Let us see the model structure
print(f"Model type: {model.config.model_type}")
print(f"Number of labels: {model.config.num_labels}")
print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
```

Expected output:

```
Model type: bert
Number of labels: 2
Total parameters: 109,483,778
```

Line-by-line explanation:

- `AutoModelForSequenceClassification` -- A version of AutoModel designed for classification tasks. It adds a linear layer on top of BERT that outputs scores for each class
- `num_labels=2` -- We have two classes: negative and positive

- `sum(p.numel() for p in model.parameters())` -- Counts the total number of trainable parameters. About 109 million for BERT base

Configuring TrainingArguments

TrainingArguments controls every aspect of training:

```python
from transformers import TrainingArguments

# Define training configuration
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=100,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=50,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
)

print("Training configuration set!")
print(f"  Epochs: {training_args.num_train_epochs}")
print(f"  Batch size: {training_args.per_device_train_batch_size}")
print(f"  Output directory: {training_args.output_dir}")
```

Expected output:

```
Training configuration set!
  Epochs: 3
  Batch size: 16
  Output directory: ./results
```

Line-by-line explanation:

- `output_dir="./results"` -- Where to save model checkpoints and training outputs
- `num_train_epochs=3` -- How many times to go through the entire training dataset. An epoch is one complete pass through all training data
- `per_device_train_batch_size=16` -- How many examples to process at once. Larger batches are faster but use more memory
- `per_device_eval_batch_size=16` -- Batch size for evaluation (can be larger since we do not need gradients)

- `warmup_steps=100` -- Gradually increase the learning rate for the first 100 steps. This helps training stability
- `weight_decay=0.01` -- A regularization technique that prevents the model from overfitting (memorizing training data instead of learning general patterns)
- `logging_dir="./logs"` -- Where to save training logs
- `logging_steps=50` -- Print training metrics every 50 steps
- `evaluation_strategy="epoch"` -- Evaluate the model after each epoch
- `save_strategy="epoch"` -- Save a checkpoint after each epoch
- `load_best_model_at_end=True` -- After training, load the checkpoint that performed best on evaluation

Understanding Training Terms

```
+-----------------------------------------------------------------+
|          TRAINING VOCABULARY                                     |
+-----------------------------------------------------------------+
|                                                                   |
|  Epoch: One complete pass through all training data             |
|    If you have 2000 examples and batch size 16:                 |
|    1 epoch = 2000/16 = 125 steps                                |
|                                                                   |
|  Batch: A group of examples processed together                  |
|    Batch size 16 = process 16 reviews at once                   |
|                                                                   |
|  Step: Processing one batch                                     |
|    Each step updates the model weights once                     |
|                                                                   |
|  Learning Rate: How big each weight update is                   |
|    Too high = model overshoots, training is unstable            |
|    Too low = training takes forever                             |
|                                                                   |
|  Warmup: Gradually increase learning rate at the start          |
|    Like warming up before exercise                              |
|                                                                   |
|  Weight Decay: Penalty for large weights                        |
|    Prevents overfitting (memorizing training data)              |
|                                                                   |
+-----------------------------------------------------------------+
```

12.5 Defining Evaluation Metrics

We need to tell the Trainer how to measure the model's performance:

```python
import numpy as np
from datasets import load_metric

# Load the accuracy metric
# Accuracy = percentage of predictions that are correct
metric = load_metric("accuracy")

def compute_metrics(eval_pred):
    """
    Compute accuracy from model predictions.

    Parameters:
        eval_pred: A tuple of (predictions, labels)
            predictions: Raw model outputs (logits) for each example
            labels: True labels for each example

    Returns:
        A dictionary with the accuracy score
    """
    logits, labels = eval_pred

    # Convert logits to predicted labels
    # argmax finds the class with the highest score
    predictions = np.argmax(logits, axis=-1)

    # Compute and return accuracy
    return metric.compute(predictions=predictions, references=labels)
```

Line-by-line explanation:

- `load_metric("accuracy")` -- Loads a pre-built accuracy calculator
- `eval_pred` -- A tuple containing two arrays: the model's raw output scores (logits) and the true labels
- `logits, labels = eval_pred` -- Unpacks the tuple into two variables
- `np.argmax(logits, axis=-1)` -- For each example, finds which class (0 or 1) got the highest score. `axis=-1` means "along the last dimension" (the class dimension)
- `metric.compute(predictions=predictions, references=labels)` -- Compares predictions with true labels and calculates the percentage that match

> What are logits? Logits are the raw, unnormalized scores that the model outputs before they are converted to probabilities. For a two-class problem, the model outputs two numbers (one for each class). The class with the higher number is the prediction.

```
+-----------------------------------------------------------------+
|           FROM LOGITS TO PREDICTIONS                            |
+-----------------------------------------------------------------+
|                                                                  |
|  Model outputs logits: [-1.5, 2.3]                              |
```

```
|                          ^     ^                              |
|                        neg   pos                              |
|                                                               |
|  argmax picks the higher one: index 1 (positive)            |
|                                                               |
|  To get probabilities, apply softmax:                         |
|    softmax([-1.5, 2.3]) = [0.02, 0.98]                      |
|    = 2% negative, 98% positive                                |
|                                                               |
+---------------------------------------------------------------+
```

12.6 Training the Model

The Complete Training Setup

Now let us put everything together:

```python
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
from datasets import load_dataset
import numpy as np

# ============================================================
# STEP 1: Load the dataset
# ============================================================
print("Step 1: Loading dataset...")
dataset = load_dataset("imdb")

# Use a smaller subset for faster training
small_train = dataset['train'].select(range(2000))
small_test = dataset['test'].select(range(500))

print(f"  Training examples: {len(small_train)}")
print(f"  Test examples: {len(small_test)}")

# ============================================================
# STEP 2: Load tokenizer and tokenize the data
# ============================================================
print("\nStep 2: Tokenizing data...")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=256
    )

tokenized_train = small_train.map(tokenize_function, batched=True)
tokenized_test = small_test.map(tokenize_function, batched=True)
print("  Tokenization complete!")

# ============================================================
# STEP 3: Load the model
# ============================================================
print("\nStep 3: Loading model...")
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
print("  Model loaded!")

# ============================================================
# STEP 4: Define training arguments
# ============================================================
print("\nStep 4: Setting up training configuration...")
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=100,
    weight_decay=0.01,
    logging_steps=50,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
)

# ============================================================
# STEP 5: Define metrics
# ============================================================
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    accuracy = (predictions == labels).mean()
    return {"accuracy": accuracy}

# ============================================================
# STEP 6: Create the Trainer
# ============================================================
print("\nStep 5: Creating Trainer...")
trainer = Trainer(
    model=model,
    args=training_args,
```

```
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    compute_metrics=compute_metrics,
)

# ============================================================
# STEP 7: Train!
# ============================================================
print("\nStep 6: Starting training...")
print("=" * 50)
trainer.train()
print("=" * 50)
print("Training complete!")
```

Expected output:

```
Step 1: Loading dataset...
  Training examples: 2000
  Test examples: 500

Step 2: Tokenizing data...
  Tokenization complete!

Step 3: Loading model...
  Model loaded!

Step 4: Setting up training configuration...

Step 5: Creating Trainer...

Step 6: Starting training...
==================================================
{'loss': 0.6891, 'learning_rate': 5e-05, 'epoch': 0.4}
{'loss': 0.5234, 'learning_rate': 4.2e-05, 'epoch': 0.8}
{'eval_loss': 0.3891, 'eval_accuracy': 0.8440, 'epoch': 1.0}
{'loss': 0.3102, 'learning_rate': 3.0e-05, 'epoch': 1.6}
{'eval_loss': 0.3012, 'eval_accuracy': 0.8780, 'epoch': 2.0}
{'loss': 0.1856, 'learning_rate': 1.5e-05, 'epoch': 2.4}
{'eval_loss': 0.3245, 'eval_accuracy': 0.8840, 'epoch': 3.0}
==================================================
Training complete!
```

Key observations from the output:

- `loss` decreases over time (from 0.69 to 0.19), meaning the model is learning
- `eval_accuracy` increases over time (from 0.84 to 0.88), meaning the model gets better at classifying reviews it has not seen before
- `learning_rate` decreases over time (this is expected with warmup and decay)

12.7 Evaluating the Model

Running Evaluation

After training, let us evaluate the model's performance on the test set:

```python
# Evaluate the model on the test set
results = trainer.evaluate()

print("\n=== Evaluation Results ===")
print(f"Loss: {results['eval_loss']:.4f}")
print(f"Accuracy: {results['eval_accuracy']:.4f}")
print(f"This means the model correctly classifies "
      f"{results['eval_accuracy']*100:.1f}% of reviews")
```

Expected output:

```
=== Evaluation Results ===
Loss: 0.3012
Accuracy: 0.8840
This means the model correctly classifies 88.4% of reviews
```

Testing on Your Own Reviews

```python
from transformers import pipeline

# Create a pipeline using our fine-tuned model
sentiment_pipeline = pipeline(
    "sentiment-analysis",
    model=model,
    tokenizer=tokenizer
)

# Test with custom reviews
test_reviews = [
    "This movie was absolutely fantastic! The acting was superb.",
    "Terrible film. I wasted two hours of my life.",
    "It was okay. Not great, not terrible. Just average.",
    "The plot was confusing but the visuals were stunning.",
    "Best movie I have seen this year! Highly recommended!"
]

print("=== Testing Our Fine-Tuned Model ===\n")
for review in test_reviews:
    result = sentiment_pipeline(review)
```

```
    label = result[0]['label']
    score = result[0]['score']

    # Map LABEL_0/LABEL_1 to negative/positive
    sentiment = "POSITIVE" if label == "LABEL_1" else "NEGATIVE"

    print(f"Review: {review}")
    print(f"  Prediction: {sentiment} (confidence: {score:.4f})")
    print()
```

Expected output:

```
=== Testing Our Fine-Tuned Model ===

Review: This movie was absolutely fantastic! The acting was superb.
  Prediction: POSITIVE (confidence: 0.9987)

Review: Terrible film. I wasted two hours of my life.
  Prediction: NEGATIVE (confidence: 0.9965)

Review: It was okay. Not great, not terrible. Just average.
  Prediction: NEGATIVE (confidence: 0.6234)

Review: The plot was confusing but the visuals were stunning.
  Prediction: POSITIVE (confidence: 0.7102)

Review: Best movie I have seen this year! Highly recommended!
  Prediction: POSITIVE (confidence: 0.9991)
```

Notice that the model is very confident about clearly positive or negative reviews but less certain about ambiguous ones.

12.8 Saving and Loading the Fine-Tuned Model

Saving the Model

```python
# Save the fine-tuned model and tokenizer
save_directory = "./fine_tuned_sentiment_model"

model.save_pretrained(save_directory)
tokenizer.save_pretrained(save_directory)
```

```
print(f"Model saved to: {save_directory}")

# List the saved files
import os
files = os.listdir(save_directory)
print(f"Saved files: {files}")
```

Expected output:

```
Model saved to: ./fine_tuned_sentiment_model
Saved files: ['config.json', 'model.safetensors', 'tokenizer.json',
              'tokenizer_config.json', 'vocab.txt', 'special_tokens_map.json']
```

Line-by-line explanation:

- `model.save_pretrained(save_directory)` -- Saves the model weights and configuration
- `tokenizer.save_pretrained(save_directory)` -- Saves the tokenizer files
- The saved files include:
  - `config.json` -- Model architecture configuration
  - `model.safetensors` -- The trained model weights
  - `tokenizer.json` and related files -- The tokenizer vocabulary and settings

Loading the Saved Model

```python
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    pipeline
)

# Load the saved model and tokenizer
save_directory = "./fine_tuned_sentiment_model"
loaded_tokenizer = AutoTokenizer.from_pretrained(save_directory)
loaded_model = AutoModelForSequenceClassification.from_pretrained(save_directory)

# Create a pipeline with the loaded model
loaded_pipeline = pipeline(
    "sentiment-analysis",
    model=loaded_model,
    tokenizer=loaded_tokenizer
)

# Test it
result = loaded_pipeline("This loaded model works perfectly!")
print(f"Result: {result}")
```

Expected output:

```
Result: [{'label': 'LABEL_1', 'score': 0.9978}]
```

Adding Label Names

By default, the model uses generic labels like LABEL_0 and LABEL_1. Let us fix that:

```python
from transformers import AutoTokenizer, AutoModelForSequenceClassification, pipeline

save_directory = "./fine_tuned_sentiment_model"

# Load model with proper label names
model = AutoModelForSequenceClassification.from_pretrained(save_directory)

# Set human-readable label names
model.config.id2label = {0: "NEGATIVE", 1: "POSITIVE"}
model.config.label2id = {"NEGATIVE": 0, "POSITIVE": 1}

# Save again with the updated config
model.save_pretrained(save_directory)

# Now the pipeline will use readable labels
tokenizer = AutoTokenizer.from_pretrained(save_directory)
classifier = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

result = classifier("I absolutely loved this film!")
print(f"Result: {result}")
```

Expected output:

```
Result: [{'label': 'POSITIVE', 'score': 0.9993}]
```

12.9 Complete Movie Review Classifier

Here is the complete, self-contained code for the movie review sentiment classifier:

```python
"""
Complete Movie Review Sentiment Classifier
Using BERT fine-tuned on the IMDB dataset.
```

```
"""

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    pipeline
)
from datasets import load_dataset
import numpy as np

# ============================================================
# Configuration
# ============================================================
MODEL_NAME = "bert-base-uncased"
MAX_LENGTH = 256
TRAIN_SIZE = 2000       # Use full 25000 for production
TEST_SIZE = 500         # Use full 25000 for production
EPOCHS = 3
BATCH_SIZE = 16
SAVE_DIR = "./movie_sentiment_model"

# ============================================================
# 1. Load and prepare data
# ============================================================
print("Loading IMDB dataset...")
dataset = load_dataset("imdb")
train_data = dataset["train"].select(range(TRAIN_SIZE))
test_data = dataset["test"].select(range(TEST_SIZE))

# ============================================================
# 2. Tokenize
# ============================================================
print("Tokenizing...")
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(examples):
    return tokenizer(examples["text"], padding="max_length",
                     truncation=True, max_length=MAX_LENGTH)

train_data = train_data.map(tokenize, batched=True)
test_data = test_data.map(tokenize, batched=True)

# ============================================================
# 3. Load model
# ============================================================
print("Loading model...")
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2
)
model.config.id2label = {0: "NEGATIVE", 1: "POSITIVE"}
model.config.label2id = {"NEGATIVE": 0, "POSITIVE": 1}

# ============================================================
```

```
# 4. Setup training
# ============================================================
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=EPOCHS,
    per_device_train_batch_size=BATCH_SIZE,
    per_device_eval_batch_size=BATCH_SIZE,
    warmup_steps=100,
    weight_decay=0.01,
    logging_steps=50,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    accuracy = (preds == labels).mean()
    return {"accuracy": accuracy}

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=test_data,
    compute_metrics=compute_metrics,
)

# ============================================================
# 5. Train and evaluate
# ============================================================
print("Training...")
trainer.train()

print("\nEvaluation:")
results = trainer.evaluate()
print(f"  Accuracy: {results['eval_accuracy']:.4f}")

# ============================================================
# 6. Save the model
# ============================================================
model.save_pretrained(SAVE_DIR)
tokenizer.save_pretrained(SAVE_DIR)
print(f"\nModel saved to {SAVE_DIR}")

# ============================================================
# 7. Test the model
# ============================================================
print("\n=== Testing the Fine-Tuned Model ===\n")
classifier = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)

test_reviews = [
    "One of the best films I have ever seen!",
    "Complete waste of time. Avoid at all costs.",
```

```
    "Decent movie but could have been better.",
]

for review in test_reviews:
    result = classifier(review)
    print(f"  '{review}'")
    print(f"  --> {result[0]['label']} ({result[0]['score']:.4f})\n")
```

Expected output:

```
Loading IMDB dataset...
Tokenizing...
Loading model...
Training...
[training progress bars and metrics]

Evaluation:
  Accuracy: 0.8840

Model saved to ./movie_sentiment_model

=== Testing the Fine-Tuned Model ===

  'One of the best films I have ever seen!'
  --> POSITIVE (0.9992)

  'Complete waste of time. Avoid at all costs.'
  --> NEGATIVE (0.9988)

  'Decent movie but could have been better.'
  --> POSITIVE (0.6543)
```

Common Mistakes

1. Using `AutoModel` instead of `AutoModelForSequenceClassification`. The plain `AutoModel` does not have a classification head. You need the task-specific version for fine-tuning.
2. Forgetting to set `num_labels`. If you do not specify `num_labels`, the model defaults to 2 labels. For multi-class problems (3 or more classes), you must set this parameter.
3. Using too large a `max_length`. Setting `max_length=512` (BERT's maximum) uses four times more memory than `max_length=128`. Start small and increase only if accuracy suffers.
4. Training for too many epochs. With pre-trained models, 2-5 epochs is usually enough. Training for 20+ epochs often leads to overfitting, where the model memorizes training

data but performs poorly on new data.

5. Not using `load_best_model_at_end=True`. Without this, the Trainer keeps the model from the last epoch, which might not be the best one. The middle epochs sometimes perform better.

Best Practices

1. Start with a small subset. Train on 1,000-2,000 examples first to make sure everything works. Then scale up to the full dataset.
2. Monitor evaluation loss. If training loss keeps decreasing but evaluation loss starts increasing, your model is overfitting. Stop training or reduce epochs.
3. Use DistilBERT for faster experiments. Replace `"bert-base-uncased"` with `"distilbert-base-uncased"` for a model that is 60% smaller and trains much faster with minimal accuracy loss.
4. Set a random seed for reproducibility. Add `seed=42` to TrainingArguments so you get the same results each time you train.
5. Save your model frequently. Use `save_strategy="epoch"` to save after each epoch. If training crashes, you can resume from the last checkpoint.

Quick Summary

Fine-tuning takes a pre-trained model like BERT and adapts it to your specific task using your own data. The process involves loading a dataset, tokenizing the text with proper padding and truncation, configuring training with TrainingArguments, and using the Trainer API to handle the training loop. For classification tasks, use `AutoModelForSequenceClassification` which adds a classification layer on top of BERT. With just 2,000 training examples and 3 epochs, you can achieve over 88% accuracy on movie review sentiment classification. Save your fine-tuned model with `save_pretrained()` and load it later for predictions.

Key Points

- Fine-tuning adapts a pre-trained model to a specific task, requiring far less data and time than training from scratch
- Use `AutoModelForSequenceClassification` for classification tasks -- it adds a classification head on top of BERT
- Tokenization must include padding (to equalize lengths) and truncation (to handle long texts)
- The attention mask tells the model which tokens are real (1) and which are padding (0)
- TrainingArguments controls epochs, batch size, learning rate, and saving behavior
- The Trainer API handles the entire training loop automatically
- Monitor both training loss (should decrease) and evaluation accuracy (should increase)
- `load_best_model_at_end=True` ensures you keep the best-performing checkpoint
- Save models with `save_pretrained()` for later use without retraining

Practice Questions

1. What is the difference between pre-training and fine-tuning? Why is fine-tuning so much faster and cheaper than training from scratch?
2. Explain what padding and truncation do during tokenization. Why are both necessary when fine-tuning a model?
3. What is the attention mask, and why is it important when we use padding? What would happen if we did not use it?
4. You fine-tuned a model for 10 epochs. The training loss is very low (0.01) but the evaluation accuracy dropped from 89% at epoch 3 to 82% at epoch 10. What happened, and how would you fix it?
5. What is the purpose of `warmup_steps` in TrainingArguments? Why not just start with the full learning rate immediately?

Exercises

Exercise 1: Fine-Tune DistilBERT

Repeat the movie review classification task but use `"distilbert-base-uncased"` instead of `"bert-base-uncased"`. Compare the training time and accuracy. How much faster is DistilBERT? How much accuracy do you lose?

Exercise 2: Multi-Class Classification

Modify the complete example to classify text into 3 categories instead of 2. Use the `"ag_news"` dataset from Hugging Face (news articles classified as World, Sports, Business, or Science/Tech -- 4 classes). You will need to change `num_labels` and update the label mapping.

Hint: Load the dataset with `load_dataset("ag_news")` and set `num_labels=4`.

Exercise 3: Experiment with Hyperparameters

Using the IMDB dataset, train three models with different settings:

- Model A: learning_rate=2e-5, epochs=2, batch_size=8
- Model B: learning_rate=5e-5, epochs=3, batch_size=16
- Model C: learning_rate=1e-4, epochs=5, batch_size=32

Compare their evaluation accuracy. Which configuration works best? Why do you think that is?

What Is Next?

You have learned how to fine-tune BERT for classification -- one of the most practical skills in NLP. In the next chapter, we will explore another powerful application of transformers: Question Answering. You will learn how to build a system that can read a passage of text and answer questions about it, just like a reading comprehension test. This is the technology behind virtual assistants and search engines that give direct answers to your questions.

Chapter 13: Question Answering with Transformers

What You Will Learn

In this chapter, you will learn:

- What extractive question answering is and how it works
- How to use the Hugging Face QA pipeline for instant question answering
- How start and end logits work to find answer spans in text
- How to fine-tune a model for question answering on custom data
- How to build a complete QA system with confidence scoring
- What the limitations of extractive QA are and when to use other approaches

Why This Chapter Matters

Think about how you answer a question when reading a textbook. Someone asks, "When was Python created?" You scan the paragraph, find the relevant sentence, and highlight the answer: "1991." You do not make up the answer -- you find it in the text.

This is exactly what extractive question answering does. Given a passage of text (the context) and a question, the model finds the exact span of text that answers the question. This technology powers search engines that display direct answers, customer support chatbots that pull answers from documentation, and virtual assistants that answer factual questions.

Understanding QA systems is essential because they are one of the most immediately useful applications of NLP. You can build a system that answers questions from your company's documentation, product manuals, or any text collection -- in just a few lines of code.

13.1 What Is Extractive Question Answering?

The Core Idea

Extractive QA does not generate new text. It finds and extracts the answer from existing text. Think of it as a highlighter, not a writer:

```
+-----------------------------------------------------------------+
|        EXTRACTIVE QUESTION ANSWERING                            |
+-----------------------------------------------------------------+
|                                                                  |
|  Context: "Python was created by Guido van Rossum and          |
|           first released in 1991. It emphasizes code           |
|           readability and simplicity."                          |
|                                                                  |
|  Question: "Who created Python?"                               |
|                                                                  |
|  The model does NOT write an answer.                           |
|  The model FINDS the answer in the context:                    |
|                                                                  |
|  "Python was created by [Guido van Rossum] and first..."       |
|                          ^^^^^^^^^^^^^^^^^^                     |
|                          THIS is the answer                    |
|                                                                  |
|  Answer: "Guido van Rossum"                                    |
|                                                                  |
+-----------------------------------------------------------------+
```

How It Differs from Other QA Approaches

```
+-----------------------------------------------------------------+
|        TYPES OF QUESTION ANSWERING                              |
+-----------------------------------------------------------------+
|                                                                  |
|  1. EXTRACTIVE QA (this chapter)                               |
|     - Finds answer IN the provided text                        |
|     - Answer is always a direct quote from the context         |
|     - Cannot answer if the answer is not in the text           |
|     - Example: "Guido van Rossum" (copied from context)        |
|                                                                  |
|  2. ABSTRACTIVE QA                                             |
|     - Generates a new answer based on the context              |
|     - Can paraphrase or combine information                    |
|     - Example: "Python's creator was Guido van Rossum,         |
|       who released it in 1991"                                 |
|                                                                  |
|  3. OPEN-DOMAIN QA                                             |
|     - Answers from a large knowledge base (like Wikipedia)     |
```

```
|     - First retrieves relevant documents, then extracts       |
|     - More complex but more powerful                          |
|                                                                |
+----------------------------------------------------------------+
```

The Internal Process

Here is how a QA model processes a question step by step:

```
+----------------------------------------------------------------+
|        QA MODEL INTERNAL PROCESS                               |
+----------------------------------------------------------------+
|                                                                |
|  Step 1: Combine question and context                          |
|  [CLS] Who created Python? [SEP] Python was created by...    |
|                                                                |
|  Step 2: Tokenize everything into numbers                      |
|  [101, 2040, 2580, 18750, ..., 102, 18750, 2001, ...]       |
|                                                                |
|  Step 3: Model processes all tokens                            |
|  Each token gets a "start score" and "end score"               |
|                                                                |
|  Step 4: Find the span with highest scores                     |
|  Start = position of "Guido"                                   |
|  End = position of "Rossum"                                    |
|                                                                |
|  Step 5: Extract the answer                                    |
|  Answer = tokens from Start to End = "Guido van Rossum"        |
|                                                                |
+----------------------------------------------------------------+
```

13.2 Using the QA Pipeline

Basic Question Answering

The simplest way to do QA with Hugging Face is the pipeline:

```python
from transformers import pipeline

# Create a question answering pipeline
qa_pipeline = pipeline("question-answering")
```

```
# Define the context (the text that contains the answer)
context = """
The Eiffel Tower is a wrought-iron lattice tower on the Champ de Mars
in Paris, France. It is named after the engineer Gustave Eiffel, whose
company designed and built the tower from 1887 to 1889. The tower is
330 metres tall and was the tallest man-made structure in the world
until 1930. Today it receives nearly 7 million visitors per year.
"""

# Ask questions about the context
questions = [
    "Where is the Eiffel Tower located?",
    "Who designed the Eiffel Tower?",
    "How tall is the Eiffel Tower?",
    "How many visitors does it receive per year?"
]

for question in questions:
    result = qa_pipeline(question=question, context=context)
    print(f"Q: {question}")
    print(f"A: {result['answer']}")
    print(f"   Confidence: {result['score']:.4f}")
    print(f"   Found at position: {result['start']}-{result['end']}")
    print()
```

Expected output:

```
Q: Where is the Eiffel Tower located?
A: Champ de Mars in Paris, France
   Confidence: 0.7234
   Found at position: 63-94

Q: Who designed the Eiffel Tower?
A: Gustave Eiffel
   Confidence: 0.8912
   Found at position: 130-145

Q: How tall is the Eiffel Tower?
A: 330 metres
   Confidence: 0.9456
   Found at position: 198-209

Q: How many visitors does it receive per year?
A: nearly 7 million
   Confidence: 0.8234
   Found at position: 278-295
```

Line-by-line explanation:

- `pipeline("question-answering")` -- Creates a QA pipeline. By default, it loads the `distilbert-base-cased-distilled-squad` model, which was trained on the SQuAD

dataset (Stanford Question Answering Dataset)

- `question=question, context=context` -- You must provide both the question and the context where the answer can be found
- `result['answer']` -- The extracted text that answers the question
- `result['score']` -- A confidence score between 0 and 1. Higher means the model is more certain
- `result['start']` and `result['end']` -- Character positions in the context where the answer was found

Asking Multiple Questions About the Same Context

```python
from transformers import pipeline

qa = pipeline("question-answering")

# A longer context with more information
context = """
Machine learning was first conceived as a field in 1959 by Arthur Samuel,
an American computer scientist who worked at IBM. He defined machine learning
as a "field of study that gives computers the ability to learn without being
explicitly programmed." The field gained momentum in the 1990s with the
development of support vector machines and random forests. The modern deep
learning revolution began around 2012, when a neural network called AlexNet
won the ImageNet competition by a significant margin. Today, machine learning
is used in applications ranging from email spam filtering to self-driving
cars and medical diagnosis. The global machine learning market was valued
at approximately 15 billion dollars in 2022.
"""

# Build a question-answer pairs list
qa_pairs = [
    "When was machine learning first conceived?",
    "Who is considered the founder of machine learning?",
    "What company did Arthur Samuel work for?",
    "What happened in 2012?",
    "What is the market value of machine learning?",
]

print("=== Machine Learning Q&A ===\n")
for q in qa_pairs:
    result = qa(question=q, context=context)
    confidence = result['score']

    # Add a confidence indicator
    if confidence > 0.8:
        indicator = "[HIGH confidence]"
    elif confidence > 0.5:
        indicator = "[MEDIUM confidence]"
```

```
    else:
        indicator = "[LOW confidence]"

    print(f"Q: {q}")
    print(f"A: {result['answer']} {indicator} ({confidence:.2%})")
    print()
```

Expected output:

```
=== Machine Learning Q&A ===

Q: When was machine learning first conceived?
A: 1959 [HIGH confidence] (94.56%)

Q: Who is considered the founder of machine learning?
A: Arthur Samuel [HIGH confidence] (89.12%)

Q: What company did Arthur Samuel work for?
A: IBM [HIGH confidence] (96.78%)

Q: What happened in 2012?
A: a neural network called AlexNet won the ImageNet competition [HIGH confidence] (82.34%)

Q: What is the market value of machine learning?
A: approximately 15 billion dollars [MEDIUM confidence] (73.45%)
```

Line-by-line explanation:

- We add confidence indicators (HIGH, MEDIUM, LOW) based on the score threshold
- `confidence:.2%` formats the number as a percentage with 2 decimal places (for example, 0.9456 becomes 94.56%)
- Notice that factual, directly stated information gets higher confidence than questions requiring inference

---

13.3 Understanding Start and End Logits

What Are Logits in QA?

In question answering, the model outputs two scores for every token in the context:

1. Start logit -- How likely this token is the start of the answer
2. End logit -- How likely this token is the end of the answer

```
+-----------------------------------------------------------------+
|        START AND END LOGITS EXAMPLE                             |
+-----------------------------------------------------------------+
|                                                                  |
|  Context tokens: [Python] [was] [created] [by] [Guido]        |
|                  [van] [Rossum] [in] [1991]                      |
|                                                                  |
|  Question: "Who created Python?"                                 |
|                                                                  |
|  Start logits:  -2.1  -3.5  -1.8  -0.5  [5.2]  0.3   -1.0  |
|                                          ^^^                     |
|                                    Highest start score           |
|                                                                  |
|  End logits:    -1.8  -2.9  -1.5  -0.8   0.5  -0.2  [4.8]   |
|                                                        ^^^       |
|                                              Highest end score |
|                                                                  |
|  Answer: tokens from "Guido" to "Rossum"                         |
|        = "Guido van Rossum"                                      |
|                                                                  |
+-----------------------------------------------------------------+
```

Exploring Logits with Code

```python
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

# Load model and tokenizer
model_name = "distilbert-base-cased-distilled-squad"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForQuestionAnswering.from_pretrained(model_name)

# Define question and context
question = "What color is the sky?"
context = "The sky is blue during the day and dark at night."

# Tokenize
inputs = tokenizer(question, context, return_tensors="pt")

# Get model output
with torch.no_grad():
    outputs = model(**inputs)

# The model outputs start and end logits
start_logits = outputs.start_logits[0]
end_logits = outputs.end_logits[0]

# Find the positions with the highest scores
start_index = torch.argmax(start_logits).item()
end_index = torch.argmax(end_logits).item()
```

```
# Convert token positions back to text
tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
answer_tokens = tokens[start_index:end_index + 1]
answer = tokenizer.convert_tokens_to_string(answer_tokens)

print(f"Question: {question}")
print(f"Context:  {context}")
print(f"Answer:   {answer}")
print(f"\nStart position: {start_index} (token: '{tokens[start_index]}')")
print(f"End position:   {end_index} (token: '{tokens[end_index]}')")

# Show the top 5 start and end positions
print("\nTop 5 start positions:")
top_starts = torch.topk(start_logits, 5)
for score, idx in zip(top_starts.values, top_starts.indices):
    print(f"  Position {idx.item():2d}: '{tokens[idx.item()]:12s}' score: {score.item()...

print("\nTop 5 end positions:")
top_ends = torch.topk(end_logits, 5)
for score, idx in zip(top_ends.values, top_ends.indices):
    print(f"  Position {idx.item():2d}: '{tokens[idx.item()]:12s}' score: {score.item()...
```

Expected output:

```
Question: What color is the sky?
Context:  The sky is blue during the day and dark at night.
Answer:   blue

Start position: 11 (token: 'blue')
End position:   11 (token: 'blue')

Top 5 start positions:
  Position 11: 'blue        ' score: 5.8234
  Position 14: 'day         ' score: 1.2345
  Position 18: 'dark        ' score: 0.8901
  Position  9: 'is          ' score: -0.5678
  Position 10: 'The         ' score: -1.2345

Top 5 end positions:
  Position 11: 'blue        ' score: 6.1234
  Position 14: 'day         ' score: 0.9876
  Position 18: 'dark        ' score: 0.7654
  Position 20: 'night       ' score: -0.3456
  Position 12: 'during      ' score: -0.8765
```

Line-by-line explanation:

- `AutoModelForQuestionAnswering` -- A model variant specifically designed for QA tasks, which outputs start and end logits

- `outputs.start_logits` -- A tensor of scores, one per token, indicating how likely each token is the start of the answer
- `outputs.end_logits` -- Same but for the end of the answer
- `torch.argmax(start_logits)` -- Finds the position with the highest start score
- `tokens[start_index:end_index + 1]` -- Extracts all tokens from start to end (inclusive), forming the answer
- `torch.topk(start_logits, 5)` -- Gets the top 5 highest scores and their positions

How the Model Picks the Best Answer

```
+-----------------------------------------------------------------+
|        ANSWER SELECTION PROCESS                                  |
+-----------------------------------------------------------------+
|                                                                   |
|  1. Get start logits for all tokens                              |
|  2. Get end logits for all tokens                                |
|  3. Find all valid (start, end) pairs where:                     |
|     - start <= end (answer cannot end before it starts)          |
|     - end - start < max_answer_length (not too long)             |
|  4. Score each pair: start_logit[i] + end_logit[j]              |
|  5. Pick the pair with the highest combined score                |
|  6. Extract tokens from start to end as the answer               |
|                                                                   |
|  Example:                                                         |
|    start=11, end=11 --> "blue"       score: 5.82 + 6.12 = 11.94|
|    start=11, end=14 --> "blue during the day" score: 5.82+0.99|
|    start=18, end=18 --> "dark"       score: 0.89 + 0.77 = 1.66|
|                                                                   |
|    Winner: "blue" with score 11.94                               |
|                                                                   |
+-----------------------------------------------------------------+
```

13.4 Building a Complete QA System

A QA System with Multiple Contexts

Let us build a system that can answer questions from a collection of text passages:

```python
from transformers import pipeline
```

```
def build_qa_system(knowledge_base):
    """
    Build a question answering system from a dictionary of topics
    and their descriptions.

    Parameters:
        knowledge_base: A dictionary where keys are topic names and
                        values are text descriptions

    Returns:
        A function that takes a question and returns the best answer
    """
    qa = pipeline("question-answering")

    def answer_question(question):
        """
        Find the best answer across all knowledge base entries.

        Parameters:
            question: The question to answer (a string)

        Returns:
            A dictionary with the answer, confidence, and source topic
        """
        best_answer = None
        best_score = -1
        best_topic = None

        # Search through all topics for the best answer
        for topic, context in knowledge_base.items():
            try:
                result = qa(question=question, context=context)

                if result['score'] > best_score:
                    best_score = result['score']
                    best_answer = result['answer']
                    best_topic = topic
            except Exception as e:
                # Skip topics that cause errors
                continue

        if best_answer is None:
            return {
                "answer": "I could not find an answer.",
                "confidence": 0.0,
                "source": None
            }

        return {
            "answer": best_answer,
            "confidence": best_score,
            "source": best_topic
        }

    return answer_question
```

```
# Create a knowledge base
knowledge_base = {
    "Python": """
    Python is a high-level programming language created by Guido van Rossum
    in 1991. It uses indentation for code blocks instead of curly braces.
    Python is popular for web development, data science, machine learning,
    and automation. The latest major version is Python 3, released in 2008.
    Python's package manager is called pip.
    """,

    "JavaScript": """
    JavaScript was created by Brendan Eich in 1995 while working at Netscape.
    It is the primary language for web browsers and runs on both client and
    server side. Node.js allows JavaScript to run outside the browser.
    JavaScript uses curly braces for code blocks and semicolons to end
    statements. The most popular JavaScript frameworks include React,
    Angular, and Vue.
    """,

    "Machine Learning": """
    Machine learning is a branch of artificial intelligence that enables
    computers to learn from data without being explicitly programmed.
    Common algorithms include linear regression, decision trees, and
    neural networks. Deep learning is a subset of machine learning that
    uses neural networks with many layers. TensorFlow and PyTorch are
    the most popular deep learning frameworks. Training a model requires
    a dataset, a loss function, and an optimizer.
    """
}

# Build the system
ask = build_qa_system(knowledge_base)

# Ask questions
questions = [
    "Who created Python?",
    "When was JavaScript created?",
    "What are popular deep learning frameworks?",
    "What does Python use instead of curly braces?",
    "Who created JavaScript?",
    "What is deep learning?"
]

print("=== Knowledge Base QA System ===\n")
for q in questions:
    result = ask(q)
    print(f"Q: {q}")
    print(f"A: {result['answer']}")
    print(f"   Source: {result['source']}")
    print(f"   Confidence: {result['confidence']:.2%}")
    print()
```

Expected output:

```
=== Knowledge Base QA System ===

Q: Who created Python?
A: Guido van Rossum
   Source: Python
   Confidence: 96.78%

Q: When was JavaScript created?
A: 1995
   Source: JavaScript
   Confidence: 94.32%

Q: What are popular deep learning frameworks?
A: TensorFlow and PyTorch
   Source: Machine Learning
   Confidence: 89.12%

Q: What does Python use instead of curly braces?
A: indentation
   Source: Python
   Confidence: 85.67%

Q: Who created JavaScript?
A: Brendan Eich
   Source: JavaScript
   Confidence: 93.45%

Q: What is deep learning?
A: a subset of machine learning that uses neural networks with many layers
   Source: Machine Learning
   Confidence: 78.90%
```

Line-by-line explanation:

- `build_qa_system(knowledge_base)` -- Creates a QA function from a dictionary of topics and their text descriptions
- The inner `answer_question` function loops through all topics, asks the QA model about each one, and returns the answer with the highest confidence score
- `try/except` -- Handles any errors that might occur when processing a particular context (for example, if the context is too short)
- The system automatically identifies which topic contains the best answer

Adding Confidence Thresholds

```python
from transformers import pipeline

qa = pipeline("question-answering")
```

```
context = """
The Python programming language was created by Guido van Rossum in 1991.
It is known for its simple syntax and readability.
"""

def answer_with_threshold(question, context, threshold=0.3):
    """
    Answer a question only if the model is confident enough.

    Parameters:
        question: The question to answer
        context: The text to search for answers
        threshold: Minimum confidence to accept an answer (0 to 1)

    Returns:
        The answer string, or a message saying no confident answer was found
    """
    result = qa(question=question, context=context)

    if result['score'] < threshold:
        return (f"I am not confident enough to answer this question. "
                f"(confidence: {result['score']:.2%})")

    return f"{result['answer']} (confidence: {result['score']:.2%})"


# Questions the model can answer
print(answer_with_threshold("Who created Python?", context))

# Questions the model cannot answer well (answer is not in context)
print(answer_with_threshold("What is the capital of France?", context))
```

Expected output:

```
Guido van Rossum (confidence: 96.78%)
I am not confident enough to answer this question. (confidence: 12.34%)
```

13.5 Fine-Tuning for Question Answering

Overview of QA Fine-Tuning

Fine-tuning a QA model follows a similar process to classification fine-tuning, but with some key differences:

```
+-----------------------------------------------------------------+
|        QA FINE-TUNING vs CLASSIFICATION FINE-TUNING             |
+-----------------------------------------------------------------+
|                                                                  |
|  Classification:                                                 |
|    Input: text                                                   |
|    Label: a class (0, 1, 2, ...)                                |
|    Output: one prediction per input                              |
|                                                                  |
|  Question Answering:                                             |
|    Input: question + context                                     |
|    Label: start position + end position of answer               |
|    Output: two predictions per input (start and end)             |
|                                                                  |
+-----------------------------------------------------------------+
```

The SQuAD Dataset Format

The most common QA dataset is SQuAD (Stanford Question Answering Dataset). Here is what the data looks like:

```python
from datasets import load_dataset

# Load the SQuAD dataset
squad = load_dataset("squad")

# Look at one example
example = squad['train'][0]
print("Context (first 200 chars):")
print(f"  {example['context'][:200]}...")
print(f"\nQuestion: {example['question']}")
print(f"Answer text: {example['answers']['text']}")
print(f"Answer start: {example['answers']['answer_start']}")
```

Expected output:

```
Context (first 200 chars):
  Architecturally, the most significant aspect of the school is the Main
  Building. Built in 1879, it is the oldest building on campus and is
  listed on the National Register of Historic Places...

Question: To whom did the Virgin Mary allegedly appear in 1858 in Lourdes France?
Answer text: ['Saint Bernadette Soubirous']
Answer start: [515]
```

Fine-Tuning Process Overview

Here is a simplified overview of the QA fine-tuning process:

```python
from transformers import (
    AutoTokenizer,
    AutoModelForQuestionAnswering,
    TrainingArguments,
    Trainer,
    DefaultDataCollator
)
from datasets import load_dataset

# Step 1: Load dataset and model
squad = load_dataset("squad")
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForQuestionAnswering.from_pretrained(model_name)

# Step 2: Use a small subset for demonstration
small_train = squad["train"].select(range(1000))
small_val = squad["validation"].select(range(200))

# Step 3: Preprocessing function for QA
def preprocess_qa(examples):
    """
    Tokenize questions and contexts, and find the answer token positions.
    """
    # Tokenize question and context pairs
    tokenized = tokenizer(
        examples["question"],
        examples["context"],
        truncation="only_second",   # Only truncate the context, not question
        max_length=384,
        stride=128,                  # Overlap for long contexts
        return_offsets_mapping=True,  # Map tokens back to characters
        padding="max_length",
    )

    # Find the start and end token positions of the answer
    start_positions = []
    end_positions = []

    for i, offset in enumerate(tokenized["offset_mapping"]):
        answer = examples["answers"][i]
        start_char = answer["answer_start"][0]
        end_char = start_char + len(answer["text"][0])

        # Find which tokens correspond to the answer
        # (simplified version)
        token_start = 0
        token_end = 0

        for idx, (token_start_char, token_end_char) in enumerate(offset):
            if token_start_char <= start_char < token_end_char:
```

```
                token_start = idx
            if token_start_char < end_char <= token_end_char:
                token_end = idx
                break

        start_positions.append(token_start)
        end_positions.append(token_end)

    tokenized["start_positions"] = start_positions
    tokenized["end_positions"] = end_positions

    # Remove offset_mapping as the model does not need it
    del tokenized["offset_mapping"]

    return tokenized

# Step 4: Tokenize the dataset
print("Tokenizing dataset...")
tokenized_train = small_train.map(preprocess_qa, batched=True,
                                  remove_columns=small_train.column_names)
tokenized_val = small_val.map(preprocess_qa, batched=True,
                              remove_columns=small_val.column_names)

# Step 5: Training arguments
training_args = TrainingArguments(
    output_dir="./qa_results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=50,
    weight_decay=0.01,
    logging_steps=50,
    evaluation_strategy="epoch",
    save_strategy="epoch",
)

# Step 6: Create trainer and train
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_val,
    data_collator=DefaultDataCollator(),
)

print("Starting QA fine-tuning...")
# trainer.train()  # Uncomment to actually train
print("(Training code ready -- uncomment trainer.train() to run)")
```

Line-by-line explanation:

- `AutoModelForQuestionAnswering` -- A model with two output heads: one for start positions and one for end positions

- `truncation="only_second"` -- When the combined question + context is too long, only truncate the context (the second input), never the question
- `stride=128` -- For very long contexts, create overlapping windows of 128 tokens. This ensures answers near window boundaries are not missed
- `return_offsets_mapping=True` -- Returns a mapping between tokens and their character positions in the original text. This is needed to find which tokens correspond to the answer
- `start_positions` and `end_positions` -- The labels for QA training are token indices, not class labels

```
+-------------------------------------------------------------+
|        QA TRAINING LABELS                                   |
+-------------------------------------------------------------+
|                                                              |
|  Context: "Python was created by Guido van Rossum in 1991"   |
|  Tokens:   [0]    [1]    [2]   [3] [4]    [5]  [6]   [7] [8]|
|                                                              |
|  Question: "Who created Python?"                             |
|  Answer: "Guido van Rossum"                                  |
|                                                              |
|  Training labels:                                            |
|    start_position = 4  (index of "Guido")                    |
|    end_position = 6    (index of "Rossum")                   |
|                                                              |
|  The model learns to predict these positions                 |
|                                                              |
+-------------------------------------------------------------+
```

13.6 Handling Long Documents

The Problem with Long Contexts

BERT and similar models have a maximum input length (usually 512 tokens). Many real-world documents are much longer. The solution is to split long documents into overlapping chunks:

```python
from transformers import pipeline

qa = pipeline("question-answering")

# A very long context (imagine a full article)
long_context = """
```

```
Chapter 1: The History of Computing

The history of computing begins with Charles Babbage, who designed the
Analytical Engine in 1837. This mechanical computer was never completed
during his lifetime, but it contained many concepts found in modern
computers, including an arithmetic logic unit, control flow, and memory.

Ada Lovelace, who worked with Babbage, is often considered the first
computer programmer. She wrote what is recognized as the first algorithm
intended for processing on a machine.

The modern era of computing began in the 1940s with the development of
electronic computers. The ENIAC, completed in 1945, was one of the first
general-purpose electronic computers. It weighed 30 tons and occupied
1,800 square feet. Despite its enormous size, it was less powerful than
a modern smartphone.

The invention of the transistor in 1947 by Bell Labs revolutionized
computing. Transistors replaced vacuum tubes, making computers smaller,
faster, and more reliable. This led to the development of integrated
circuits in the 1960s, which packed thousands of transistors onto a
single chip.

The personal computer revolution began in the late 1970s with machines
like the Apple II and the IBM PC. These affordable computers brought
computing power to homes and small businesses for the first time.
"""

# Ask questions about the long context
questions = [
    "Who designed the Analytical Engine?",
    "Who is considered the first computer programmer?",
    "How much did the ENIAC weigh?",
    "What replaced vacuum tubes?",
    "When did the personal computer revolution begin?"
]

print("=== Questions About Computing History ===\n")
for q in questions:
    result = qa(question=q, context=long_context)
    print(f"Q: {q}")
    print(f"A: {result['answer']} (confidence: {result['score']:.2%})")
    print()
```

Expected output:

```
=== Questions About Computing History ===

Q: Who designed the Analytical Engine?
A: Charles Babbage (confidence: 96.12%)

Q: Who is considered the first computer programmer?
A: Ada Lovelace (confidence: 94.56%)
```

```
Q: How much did the ENIAC weigh?
A: 30 tons (confidence: 92.34%)

Q: What replaced vacuum tubes?
A: Transistors (confidence: 95.67%)

Q: When did the personal computer revolution begin?
A: late 1970s (confidence: 89.01%)
```

Splitting Documents for Very Long Texts

```python
from transformers import pipeline

def qa_long_document(question, document, chunk_size=500, overlap=100):
    """
    Answer a question from a very long document by splitting it into
    overlapping chunks and finding the best answer across all chunks.

    Parameters:
        question: The question to answer
        document: The full document text
        chunk_size: Maximum characters per chunk
        overlap: Number of characters to overlap between chunks

    Returns:
        The best answer found across all chunks
    """
    qa = pipeline("question-answering")

    # Split document into overlapping chunks
    chunks = []
    start = 0
    while start < len(document):
        end = start + chunk_size
        chunk = document[start:end]
        chunks.append(chunk)
        start += chunk_size - overlap

    print(f"Document split into {len(chunks)} chunks")

    # Find the best answer across all chunks
    best_answer = None
    best_score = -1

    for i, chunk in enumerate(chunks):
        try:
            result = qa(question=question, context=chunk)
            if result['score'] > best_score:
                best_score = result['score']
```

```
                best_answer = result['answer']
        except Exception:
            continue

    return {"answer": best_answer, "confidence": best_score}


# Example with a long document
long_doc = "Your very long document text here..." * 20  # Simulating length

# In practice:
# result = qa_long_document("Your question?", long_doc)
# print(result)
print("Long document QA function ready to use!")
```

13.7 Limitations of Extractive QA

What Extractive QA Cannot Do

```
+-------------------------------------------------------------+
|        LIMITATIONS OF EXTRACTIVE QA                          |
+-------------------------------------------------------------+
|                                                               |
|  1. ANSWER MUST BE IN THE CONTEXT                            |
|     Cannot answer: "What is 2 + 2?" (unless "4" is in text)  |
|     Cannot reason or calculate                               |
|                                                               |
|  2. SINGLE SPAN ONLY                                         |
|     Cannot combine info from multiple sentences              |
|     Q: "Name all the countries mentioned"                    |
|     Can only highlight one continuous span                   |
|                                                               |
|  3. NO PARAPHRASING                                          |
|     Context: "The cat sat on the mat"                        |
|     Q: "Where was the feline?"                               |
|     May struggle because "feline" != "cat" in the context    |
|                                                               |
|  4. CONTEXT LENGTH LIMIT                                     |
|     Cannot process very long documents without chunking      |
|     May miss answers that span chunk boundaries              |
|                                                               |
|  5. NO "YES/NO" ANSWERS                                      |
|     Q: "Is Python popular?"                                  |
|     Will try to extract a span instead of saying "yes"       |
|                                                               |
+-------------------------------------------------------------+
```

When to Use Extractive QA vs Other Approaches

```python
# Good use cases for extractive QA:
good_cases = [
    "Who founded the company?",          # Answer is a name in the text
    "When was the product launched?",    # Answer is a date in the text
    "What is the company's revenue?",    # Answer is a number in the text
    "Where is the headquarters?",        # Answer is a location in the text
]

# Bad use cases for extractive QA:
bad_cases = [
    "Summarize the annual report",       # Needs summarization, not extraction
    "Is the company profitable?",        # Needs yes/no reasoning
    "Compare product A and product B",   # Needs cross-document reasoning
    "What should I invest in?",          # Needs opinion/advice generation
]

print("GOOD use cases for extractive QA:")
for case in good_cases:
    print(f"  + {case}")

print("\nBAD use cases for extractive QA:")
for case in bad_cases:
    print(f"  - {case}")
```

Common Mistakes

1. Forgetting to provide context. The QA pipeline requires both a `question` and a `context`. Without context, the model has nowhere to look for answers.
2. Expecting answers not in the context. Extractive QA can only find answers that exist verbatim in the provided text. If the answer is not in the context, the model will return a low-confidence wrong answer.
3. Ignoring confidence scores. Always check the confidence score. A low score (below 0.3) usually means the model is guessing and the answer is unreliable.
4. Using too short a context. Very short contexts (one or two sentences) give the model too little information. Provide enough surrounding text for the model to understand the topic.
5. Not handling long documents. If your context exceeds the model's maximum length (512 tokens for BERT), the text will be silently truncated. Split long documents into overlapping chunks.

Best Practices

1. Set confidence thresholds. Do not show answers with confidence below 0.3 to users. Return "I don't know" instead of a wrong answer.
2. Provide relevant context. The more focused the context is on the topic of the question, the better the answers will be. Use information retrieval to find relevant passages first.
3. Use SQuAD-trained models. Models fine-tuned on SQuAD (like `distilbert-base-cased-distilled-squad`) are much better at QA than general-purpose models.
4. Handle edge cases. What if the question is unanswerable? What if the context is empty? Always write defensive code with error handling.
5. Test with diverse questions. Test your QA system with who, what, when, where, why, and how questions to ensure it handles different question types.

Quick Summary

Extractive question answering finds the answer to a question by locating the exact span of text in a provided context. The model assigns start and end scores to every token and selects the span with the highest combined score. The Hugging Face QA pipeline makes this easy with just a few lines of code. For production systems, search across multiple context passages and use confidence thresholds to filter unreliable answers. The main limitation is that the answer must exist verbatim in the context -- the model extracts rather than generates.

Key Points

- Extractive QA finds answers by highlighting spans in the provided context -- it does not generate new text
- The model outputs start logits and end logits for every token, then selects the best (start, end) pair
- The QA pipeline requires both a `question` and a `context` parameter
- Confidence scores range from 0 to 1 -- set a threshold (e.g., 0.3) to filter unreliable answers
- For long documents, split into overlapping chunks and find the best answer across all chunks

- SQuAD-trained models perform best for extractive QA tasks
- Extractive QA cannot answer questions whose answers are not in the context
- Fine-tuning for QA uses start and end token positions as labels instead of class labels

---

Practice Questions

1. What is the difference between extractive and abstractive question answering? Give an example of a question that extractive QA could answer and one that it could not.
2. Explain what start logits and end logits are. How does the model combine them to find the answer?
3. Why is the confidence score important in a QA system? What problems could arise if you display all answers regardless of confidence?
4. You have a 50-page PDF document and want to build a QA system for it. What challenges would you face, and how would you solve them?
5. A user asks, "Is Python a good programming language?" to your extractive QA system. The context says "Python is widely used in data science and web development." What would the model likely return, and why is this problematic?

Exercises

Exercise 1: Build a Documentation QA Bot

Create a QA system for a fictional software product. Write 3-4 paragraphs of documentation (covering installation, features, and troubleshooting). Then test your system with at least 5 different questions.

Requirements:

- Include a confidence threshold
- Print "I don't know" for low-confidence answers
- Show which part of the documentation the answer came from

Exercise 2: Compare QA Models

Load two different QA models from Hugging Face (for example, `distilbert-base-cased-distilled-squad` and `deepset/roberta-base-squad2`). Ask both models the same 5 questions and compare their answers and confidence scores. Which model performs better?

Exercise 3: QA with Multiple Documents

Build a QA system that takes a list of documents (each with a title and content). When a question is asked, search all documents and return the top 3 answers ranked by confidence. Include the document title in the output.

What Is Next?

You have learned how to extract answers from text -- now it is time to learn how to generate entirely new text. In the next chapter, we will explore text generation, where models create original text from a prompt. You will learn about temperature, top-k sampling, top-p sampling, and other techniques that control how creative or focused the generated text is. This is the technology behind chatbots, story generators, and code completion tools.

Chapter 14: Text Generation

What You Will Learn

In this chapter, you will learn:

- How autoregressive text generation works (predicting one word at a time)
- What temperature is and how it controls the randomness of generated text
- How top-k sampling limits choices to the most likely words
- How top-p (nucleus) sampling works and why it is often preferred
- How to control output length and prevent repetition
- How to use Hugging Face's `generate()` method with all its options
- How to build a complete story generation example
- The basics of prompt engineering for better results

Why This Chapter Matters

Text generation is the technology behind some of the most impressive AI applications today. When you ask a chatbot a question and it writes a thoughtful response, that is text generation. When a coding tool suggests the next line of code, that is text generation. When an AI writes a product description, a poem, or a news summary -- all text generation.

Understanding how text generation works is not just about using it -- it is about controlling it. Without understanding temperature, sampling strategies, and repetition penalties, your generated text will often be boring, repetitive, or nonsensical. This chapter gives you the knobs and dials to produce exactly the kind of text you want.

Think of a text generation model as a very talented writer who needs clear instructions. Temperature tells the writer how creative to be. Top-k and top-p tell the writer which word choices to consider. This chapter teaches you how to be a good editor for your AI writer.

14.1 How Autoregressive Generation Works

The Word-by-Word Process

Text generation models work by predicting one word (actually, one token) at a time. Each prediction depends on all the words that came before it:

```
+-----------------------------------------------------------------+
|        AUTOREGRESSIVE GENERATION                                |
+-----------------------------------------------------------------+
|                                                                  |
|  Prompt: "The cat sat"                                          |
|                                                                  |
|  Step 1: What comes after "The cat sat"?                        |
|           Model predicts: "on"                                  |
|           Text so far: "The cat sat on"                         |
|                                                                  |
|  Step 2: What comes after "The cat sat on"?                     |
|           Model predicts: "the"                                 |
|           Text so far: "The cat sat on the"                     |
|                                                                  |
|  Step 3: What comes after "The cat sat on the"?                 |
|           Model predicts: "mat"                                 |
|           Text so far: "The cat sat on the mat"                 |
|                                                                  |
|  Step 4: What comes after "The cat sat on the mat"?             |
|           Model predicts: "." (end of sentence)                 |
|           Text so far: "The cat sat on the mat."                |
|                                                                  |
|  This is called "autoregressive" because each prediction        |
|  feeds back into the input for the next prediction.             |
|                                                                  |
+-----------------------------------------------------------------+
```

The word "autoregressive" comes from two parts:

- Auto = self
- Regressive = using previous outputs

So autoregressive means "using its own previous outputs as input."

The Probability Distribution

At each step, the model does not just pick one word. It calculates a probability for every word in its vocabulary:

```
+-----------------------------------------------------------------+
|        PROBABILITY DISTRIBUTION AT EACH STEP                    |
+-----------------------------------------------------------------+
|                                                                  |
|  Input: "The cat sat on the"                                    |
|                                                                  |
|  Model calculates probabilities for ALL words:                  |
|                                                                  |
|  mat     |||||||||||||||||||||||  0.35  (35%)                    |
|  floor   |||||||||||||||          0.20  (20%)                    |
|  couch   |||||||||||             0.12  (12%)                    |
|  bed     ||||||||               0.10  (10%)                    |
|  table   ||||||                 0.08  (8%)                     |
|  chair   |||||                  0.06  (6%)                     |
|  roof    |||                    0.04  (4%)                     |
|  ...     |                      0.05  (5% total for rest)      |
|                                                                  |
|  The question is: HOW do we choose from these probabilities?    |
|  That is what temperature, top-k, and top-p control.            |
|                                                                  |
+-----------------------------------------------------------------+
```

Basic Text Generation

```python
from transformers import pipeline

# Create a text generation pipeline
generator = pipeline("text-generation", model="gpt2")

# Generate text from a prompt
prompt = "Once upon a time in a land far away"
result = generator(prompt, max_length=60, num_return_sequences=1)

print("Prompt:", prompt)
print("Generated:", result[0]['generated_text'])
```

Expected output:

```
Prompt: Once upon a time in a land far away
Generated: Once upon a time in a land far away, there lived a young
princess who dreamed of exploring the world beyond her castle walls.
She spent her days reading books about distant kingdoms and magical
creatures.
```

Line-by-line explanation:

- `pipeline("text-generation", model="gpt2")` -- Creates a text generation pipeline using GPT-2, a popular generation model

- `max_length=60` -- Generate at most 60 tokens total (including the prompt)
- `num_return_sequences=1` -- Generate one version of the text
- The output includes the original prompt followed by the generated continuation

> Note: Your output will differ from this example. Text generation involves randomness, so each run produces different text.

14.2 Temperature: Controlling Randomness

What Is Temperature?

Temperature controls how random or predictable the generated text is. Think of it as a "creativity dial":

```
+-----------------------------------------------------------------+
|        TEMPERATURE ANALOGY                                      |
+-----------------------------------------------------------------+
|                                                                  |
|  Temperature = 0.1 (Very Low)                                   |
|    Like a careful student always choosing the "safe" answer     |
|    Output is predictable and repetitive                         |
|    Almost always picks the most likely word                     |
|                                                                  |
|  Temperature = 0.7 (Medium)                                     |
|    Like a balanced writer mixing common and creative words      |
|    Output is natural and interesting                            |
|    Good balance of coherence and variety                        |
|                                                                  |
|  Temperature = 1.5 (Very High)                                  |
|    Like a wild poet throwing words together                     |
|    Output is creative but sometimes nonsensical                 |
|    Often picks unexpected, unusual words                        |
|                                                                  |
+-----------------------------------------------------------------+
```

How Temperature Works Mathematically

Temperature modifies the probability distribution before a word is chosen:

```
+-----------------------------------------------------------------+
|        TEMPERATURE EFFECT ON PROBABILITIES                      |
+-----------------------------------------------------------------+
```

```
|                                                                  |
|  Original probabilities (temperature = 1.0):                    |
|    "mat":   0.35    "floor": 0.20    "couch": 0.12             |
|                                                                  |
|  Low temperature (0.2) -- probabilities become more extreme:    |
|    "mat":   0.85    "floor": 0.10    "couch": 0.03             |
|    (The top choice dominates even more)                          |
|                                                                  |
|  High temperature (2.0) -- probabilities become more equal:     |
|    "mat":   0.22    "floor": 0.18    "couch": 0.15             |
|    (All choices become more equally likely)                      |
|                                                                  |
+------------------------------------------------------------------+
```

Comparing Different Temperatures

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "The future of artificial intelligence"

# Generate with different temperatures
temperatures = [0.2, 0.7, 1.0, 1.5]

for temp in temperatures:
    result = generator(
        prompt,
        max_length=50,
        temperature=temp,
        do_sample=True,        # Must be True to use temperature
        num_return_sequences=1
    )
    print(f"Temperature = {temp}:")
    print(f"  {result[0]['generated_text']}")
    print()
```

Expected output:

```
Temperature = 0.2:
  The future of artificial intelligence is likely to be shaped by the
  development of more advanced machine learning algorithms and the
  increasing availability of large datasets.

Temperature = 0.7:
  The future of artificial intelligence holds tremendous promise for
  healthcare, education, and scientific discovery, though we must
  carefully consider the ethical implications.
```

```
Temperature = 1.0:
  The future of artificial intelligence could reshape everything from
  how we communicate to how cities are designed, creating opportunities
  we haven't yet imagined.

Temperature = 1.5:
  The future of artificial intelligence dances between quantum
  possibilities and crystalline dreams of silicon consciousness
  awakening beneath digital moonlight.
```

Line-by-line explanation:

- `temperature=0.2` -- Very low temperature. The model picks the most probable words almost every time. Output is safe but can be boring
- `temperature=0.7` -- A balanced setting. The model mostly picks likely words but occasionally makes creative choices. This is the most commonly used setting
- `temperature=1.0` -- Default temperature. The model uses the original probability distribution
- `temperature=1.5` -- High temperature. The model frequently picks less likely words. Output is creative but may not make sense
- `do_sample=True` -- This is required to use temperature. Without it, the model always picks the most likely word (called "greedy decoding")

14.3 Top-k Sampling

What Is Top-k?

Top-k sampling limits the model to choosing from only the k most likely next words. All other words are excluded:

```
+-------------------------------------------------------------+
|        TOP-K SAMPLING (k=3)                                 |
+-------------------------------------------------------------+
|                                                              |
|  Full vocabulary probabilities:                             |
|    mat:   0.35  <-- included (top 3)                        |
|    floor: 0.20  <-- included (top 3)                        |
|    couch: 0.12  <-- included (top 3)                        |
|    bed:   0.10  <-- EXCLUDED                                |
|    table: 0.08  <-- EXCLUDED                                |
|    chair: 0.06  <-- EXCLUDED                                |
|    ...                                                       |
```

```
|                                                                  |
|  After top-k filtering (k=3):                                   |
|    mat:   0.52  (0.35 / 0.67, renormalized)                     |
|    floor: 0.30  (0.20 / 0.67, renormalized)                     |
|    couch: 0.18  (0.12 / 0.67, renormalized)                     |
|                                                                  |
|  The model can only choose from these 3 words.                  |
|  This prevents picking very unlikely, weird words.              |
|                                                                  |
+-----------------------------------------------------------------+
```

Using Top-k in Practice

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "The scientist discovered that"

# Compare different k values
k_values = [5, 20, 50, 100]

for k in k_values:
    result = generator(
        prompt,
        max_length=40,
        top_k=k,
        do_sample=True,
        temperature=0.8,
        num_return_sequences=1
    )
    print(f"top_k = {k}:")
    print(f"  {result[0]['generated_text']}")
    print()
```

Expected output:

```
top_k = 5:
  The scientist discovered that the new compound could effectively
  reduce inflammation in laboratory tests.

top_k = 20:
  The scientist discovered that ancient bacteria preserved in ice
  could still be revived after thousands of years.

top_k = 50:
  The scientist discovered that gravitational waves from colliding
  neutron stars contained unexpected frequency patterns.
```

```
top_k = 100:
  The scientist discovered that bioluminescent algae communicated
  through synchronized light pulses resembling Morse code.
```

Line-by-line explanation:

- `top_k=5` -- Only consider the 5 most likely next words at each step. Results are very predictable
- `top_k=20` -- Consider the top 20 words. More variety while staying coherent
- `top_k=50` -- Consider the top 50 words. A common default value
- `top_k=100` -- Consider the top 100 words. More creative but occasionally odd

```
+-----------------------------------------------------------------+
|        TOP-K: CHOOSING THE RIGHT VALUE                          |
+-----------------------------------------------------------------+
|                                                                  |
|  k = 1:   Always picks the most likely word (greedy)            |
|            Very repetitive, but very "safe"                     |
|                                                                  |
|  k = 10:  Very conservative, predictable text                   |
|                                                                  |
|  k = 50:  Good balance (commonly used default)                  |
|                                                                  |
|  k = 100: More creative, occasionally surprising                |
|                                                                  |
|  k = 1000: Almost no filtering, very random                     |
|                                                                  |
+-----------------------------------------------------------------+
```

14.4 Top-p (Nucleus) Sampling

What Is Top-p?

Top-p sampling (also called nucleus sampling) is a smarter alternative to top-k. Instead of always keeping a fixed number of words, it keeps the smallest set of words whose combined probability is at least p:

```
+-----------------------------------------------------------------+
|        TOP-P SAMPLING (p=0.9)                                   |
+-----------------------------------------------------------------+
|                                                                  |
|  Probabilities (sorted highest to lowest):                      |
```

```
|    mat:    0.35  cumulative: 0.35  <-- included               |
|    floor:  0.20  cumulative: 0.55  <-- included               |
|    couch:  0.12  cumulative: 0.67  <-- included               |
|    bed:    0.10  cumulative: 0.77  <-- included               |
|    table:  0.08  cumulative: 0.85  <-- included               |
|    chair:  0.06  cumulative: 0.91  <-- included (>= 0.9!)    |
|    roof:   0.04  cumulative: 0.95  <-- EXCLUDED               |
|    ...                                                         |
|                                                                |
|  We keep adding words until cumulative probability >= 0.9     |
|  This gives us 6 words to choose from                         |
|                                                                |
+----------------------------------------------------------------+
```

Why Top-p Is Better Than Top-k

The advantage of top-p is that it adapts to the situation:

```
+----------------------------------------------------------------+
|        WHY TOP-P ADAPTS BETTER THAN TOP-K                      |
+----------------------------------------------------------------+
|                                                                |
|  Situation 1: Model is very confident                         |
|    "The capital of France is ___"                             |
|    Paris: 0.95, Lyon: 0.03, Marseille: 0.01, ...             |
|    top_p=0.9 --> keeps only "Paris" (1 word)                  |
|    top_k=50 --> keeps 50 words (too many!)                    |
|                                                                |
|  Situation 2: Model is uncertain                              |
|    "For dinner I want to eat ___"                             |
|    pizza: 0.08, pasta: 0.07, sushi: 0.06, ...                |
|    top_p=0.9 --> keeps ~15 words (appropriate variety)        |
|    top_k=50 --> keeps 50 words (still too many)               |
|                                                                |
|  Top-p naturally adjusts the number of choices!               |
|                                                                |
+----------------------------------------------------------------+
```

Using Top-p in Practice

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "In a world where robots"
```

```
# Compare different p values
p_values = [0.5, 0.7, 0.9, 0.95]

for p in p_values:
    result = generator(
        prompt,
        max_length=40,
        top_p=p,
        do_sample=True,
        temperature=0.8,
        num_return_sequences=1
    )
    print(f"top_p = {p}:")
    print(f"  {result[0]['generated_text']}")
    print()
```

Expected output:

```
top_p = 0.5:
  In a world where robots have become an essential part of daily life,
  humans must learn to work alongside their mechanical counterparts.

top_p = 0.7:
  In a world where robots can think and feel, a young engineer builds
  the first machine capable of genuine empathy.

top_p = 0.9:
  In a world where robots govern the ancient forests, one small bird
  discovers a secret that could change everything.

top_p = 0.95:
  In a world where robots dream in watercolors, a forgotten clockwork
  sparrow awakens to find the last human library.
```

Combining Temperature, Top-k, and Top-p

You can use these parameters together for fine-grained control:

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "The old library held a secret"

# Combination 1: Conservative (factual, predictable)
result1 = generator(
    prompt, max_length=50,
    temperature=0.3, top_k=10, top_p=0.85,
```

```
    do_sample=True, num_return_sequences=1
)

# Combination 2: Balanced (natural, interesting)
result2 = generator(
    prompt, max_length=50,
    temperature=0.7, top_k=50, top_p=0.9,
    do_sample=True, num_return_sequences=1
)

# Combination 3: Creative (surprising, imaginative)
result3 = generator(
    prompt, max_length=50,
    temperature=1.2, top_k=100, top_p=0.95,
    do_sample=True, num_return_sequences=1
)

print("Conservative:")
print(f"  {result1[0]['generated_text']}\n")

print("Balanced:")
print(f"  {result2[0]['generated_text']}\n")

print("Creative:")
print(f"  {result3[0]['generated_text']}\n")
```

```
+----------------------------------------------------------------+
|        RECOMMENDED PARAMETER COMBINATIONS                      |
+----------------------------------------------------------------+
|                                                                 |
|  Use Case          | Temp | top_k | top_p | Notes            |
|  ------------------|------|-------|-------|------------------ |
|  Factual writing   | 0.3  |  10   | 0.85  | Safe, accurate   |
|  General text      | 0.7  |  50   | 0.90  | Natural feel     |
|  Creative writing  | 1.0  |  100  | 0.95  | Interesting      |
|  Poetry/fiction    | 1.2  |  150  | 0.95  | Wild, creative   |
|  Code generation   | 0.2  |   5   | 0.80  | Precise, correct |
|                                                                 |
+----------------------------------------------------------------+
```

14.5 Controlling Output Length

Max Length and Max New Tokens

There are two ways to control how much text is generated:

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "The weather today"

# Method 1: max_length (total length including prompt)
result1 = generator(prompt, max_length=30, do_sample=True)
print("max_length=30 (total tokens):")
print(f"  {result1[0]['generated_text']}\n")

# Method 2: max_new_tokens (only new tokens, not counting prompt)
result2 = generator(prompt, max_new_tokens=20, do_sample=True)
print("max_new_tokens=20 (new tokens only):")
print(f"  {result2[0]['generated_text']}\n")
```

Expected output:

```
max_length=30 (total tokens):
  The weather today is expected to be partly cloudy with temperatures
  reaching the mid-70s in most areas.

max_new_tokens=20 (new tokens only):
  The weather today looks beautiful with clear skies and a gentle
  breeze coming from the west.
```

Line-by-line explanation:

- `max_length=30` -- The total output (prompt + generated text) will be at most 30 tokens. If your prompt is 5 tokens, you get at most 25 new tokens
- `max_new_tokens=20` -- Exactly 20 new tokens will be generated, regardless of prompt length. This is often more intuitive to use

Using Min Length

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "The meaning of life is"

# Without min_length (model might stop early)
result1 = generator(prompt, max_length=50, do_sample=True)

# With min_length (force the model to write more)
result2 = generator(prompt, max_length=50, min_length=30, do_sample=True)
```

```
print("Without min_length:")
print(f"  {result1[0]['generated_text']}\n")

print("With min_length=30:")
print(f"  {result2[0]['generated_text']}\n")
```

14.6 Repetition Penalty

The Problem of Repetition

Without a repetition penalty, generation models often get stuck in loops:

```
Without penalty:
"The cat sat on the mat. The cat sat on the mat. The cat sat
 on the mat. The cat sat on the mat."

With penalty:
"The cat sat on the mat. It purred contentedly while watching
 the birds outside the window."
```

Using Repetition Penalty

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "Artificial intelligence is"

# Without repetition penalty
result1 = generator(
    prompt, max_length=80,
    do_sample=True, temperature=0.7,
    repetition_penalty=1.0   # 1.0 means no penalty (default)
)

# With repetition penalty
result2 = generator(
    prompt, max_length=80,
    do_sample=True, temperature=0.7,
    repetition_penalty=1.3   # Penalize repeated words/phrases
)
```

```
print("No penalty (repetition_penalty=1.0):")
print(f"  {result1[0]['generated_text']}\n")

print("With penalty (repetition_penalty=1.3):")
print(f"  {result2[0]['generated_text']}\n")
```

Expected output:

```
No penalty (repetition_penalty=1.0):
  Artificial intelligence is a field of computer science that focuses
  on creating systems that can perform tasks that require human
  intelligence. Artificial intelligence is used in many applications
  that require human intelligence and artificial intelligence is
  becoming more and more important.

With penalty (repetition_penalty=1.3):
  Artificial intelligence is a rapidly evolving field that combines
  computer science with cognitive psychology. Modern applications range
  from medical diagnosis to autonomous vehicles, transforming industries
  and creating new possibilities for human-machine collaboration.
```

Line-by-line explanation:

- `repetition_penalty=1.0` -- No penalty. The model can repeat words and phrases freely. This often leads to repetitive, looping text
- `repetition_penalty=1.3` -- Each time a word is generated, its probability of being generated again is divided by 1.3. Higher values mean stronger penalties
- Values between 1.1 and 1.5 work well. Going above 2.0 can make the text incoherent as the model avoids too many common words

```
+-------------------------------------------------------------+
|        REPETITION PENALTY VALUES                            |
+-------------------------------------------------------------+
|                                                              |
|  1.0 = No penalty (default, may cause repetition)            |
|  1.1 = Light penalty (reduces obvious repetition)            |
|  1.3 = Moderate penalty (good default for most tasks)        |
|  1.5 = Strong penalty (good for creative writing)            |
|  2.0 = Very strong (may cause incoherent text)               |
|                                                              |
+-------------------------------------------------------------+
```

14.7 The generate() Method

Using generate() Directly

The pipeline wraps the `generate()` method for convenience, but you can use `generate()` directly for maximum control:

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# Load model and tokenizer
model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# Set the padding token (GPT-2 does not have one by default)
tokenizer.pad_token = tokenizer.eos_token

# Encode the prompt
prompt = "In the year 2050, technology"
inputs = tokenizer(prompt, return_tensors="pt")

# Generate text using the generate() method
with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=50,
        temperature=0.8,
        top_k=50,
        top_p=0.9,
        repetition_penalty=1.2,
        do_sample=True,
        num_return_sequences=3,   # Generate 3 different versions
    )

# Decode and print each generated sequence
print(f"Prompt: {prompt}\n")
for i, output in enumerate(outputs):
    text = tokenizer.decode(output, skip_special_tokens=True)
    # Remove the prompt from the output for clarity
    generated_part = text[len(prompt):]
    print(f"Version {i+1}: {generated_part.strip()}")
    print()
```

Expected output:

```
Prompt: In the year 2050, technology

Version 1: will have transformed every aspect of daily life.
Self-driving vehicles will dominate the roads, while AI assistants
manage our homes and personal schedules with remarkable efficiency.

Version 2: has advanced to the point where humans can upload their
```

```
consciousness into digital spaces. Virtual reality is indistinguishable
from the physical world, raising profound philosophical questions.

Version 3: enables cities to run entirely on renewable energy. Smart
grids powered by quantum computers optimize power distribution in real
time, eliminating waste and reducing costs dramatically.
```

Line-by-line explanation:

- `AutoModelForCausalLM` -- "Causal Language Model" is the technical name for autoregressive text generation models. "Causal" means each token can only see the tokens before it, not after
- `tokenizer.pad_token = tokenizer.eos_token` -- GPT-2 was not trained with a padding token, so we assign the end-of-sequence token as the pad token
- `model.generate(**inputs, ...)` -- The main generation method with full control
- `num_return_sequences=3` -- Generate 3 different versions from the same prompt
- `tokenizer.decode(output, skip_special_tokens=True)` -- Converts the generated token IDs back to readable text, removing special tokens like `<|endoftext|>`

Generating Multiple Sequences

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = "Three reasons to learn programming:"

# Generate 5 different completions
results = generator(
    prompt,
    max_new_tokens=60,
    num_return_sequences=5,
    do_sample=True,
    temperature=0.8,
    top_p=0.9
)

print(f"Prompt: {prompt}\n")
for i, result in enumerate(results):
    generated = result['generated_text'][len(prompt):].strip()
    print(f"Version {i+1}:")
    print(f"  {generated}")
    print()
```

14.8 Complete Story Generation Example

Building a Story Generator

```python
from transformers import pipeline

def generate_story(opening, genre="fantasy", length="medium",
                   creativity=0.7):
    """
    Generate a short story from an opening sentence.

    Parameters:
        opening: The first sentence of the story
        genre: The story genre (used in the prompt)
        length: "short" (50 tokens), "medium" (100), "long" (200)
        creativity: How creative (0.1 = conservative, 1.5 = wild)

    Returns:
        The generated story as a string
    """

    # Map length to max_new_tokens
    length_map = {
        "short": 50,
        "medium": 100,
        "long": 200
    }
    max_tokens = length_map.get(length, 100)

    # Create the prompt with genre context
    prompt = f"[Genre: {genre}]\n\n{opening}"

    # Create the generator
    generator = pipeline("text-generation", model="gpt2")

    # Generate the story
    result = generator(
        prompt,
        max_new_tokens=max_tokens,
        temperature=creativity,
        top_p=0.9,
        top_k=50,
        repetition_penalty=1.3,
        do_sample=True,
        num_return_sequences=1
    )

    # Clean up: remove the genre tag from the output
    story = result[0]['generated_text']
    if story.startswith(f"[Genre: {genre}]\n\n"):
        story = story[len(f"[Genre: {genre}]\n\n"):]
```

```
    return story


# Generate stories with different settings
print("=" * 60)
print("STORY 1: Fantasy (Medium creativity)")
print("=" * 60)
story1 = generate_story(
    "The ancient dragon opened one eye as the young knight entered the cave.",
    genre="fantasy",
    length="medium",
    creativity=0.7
)
print(story1)
print()

print("=" * 60)
print("STORY 2: Science Fiction (High creativity)")
print("=" * 60)
story2 = generate_story(
    "The last human on Mars stared at the incoming transmission.",
    genre="science fiction",
    length="medium",
    creativity=1.0
)
print(story2)
print()

print("=" * 60)
print("STORY 3: Mystery (Low creativity, more structured)")
print("=" * 60)
story3 = generate_story(
    "Detective Chen found the missing painting exactly where nobody expected.",
    genre="mystery",
    length="medium",
    creativity=0.4
)
print(story3)
```

Expected output:

```
============================================================
STORY 1: Fantasy (Medium creativity)
============================================================
The ancient dragon opened one eye as the young knight entered
the cave. Its scales gleamed like molten gold in the dim light
of the cavern. "You are brave," the dragon rumbled, smoke
curling from its nostrils. "Or perhaps merely foolish." The
knight gripped the hilt of her sword but did not draw it.
"I did not come to fight," she said quietly. "I came to ask
for your help."
```

```
============================================================
STORY 2: Science Fiction (High creativity)
============================================================
The last human on Mars stared at the incoming transmission.
The signal pulsed with frequencies that shouldn't exist,
carrying fragments of music from Earth's forgotten symphonies.
She adjusted the antenna array, hands trembling as the message
decoded itself into coordinates pointing beyond the solar
system. Something out there knew she was alive. Something
wanted her to come home -- but not to Earth.

============================================================
STORY 3: Mystery (Low creativity, more structured)
============================================================
Detective Chen found the missing painting exactly where nobody
expected. It was hanging in plain sight in the museum lobby,
hidden behind a reproduction that had been placed over it.
"The thief never took it out of the building," Chen said,
examining the frame. "They simply covered it up and waited for
everyone to stop looking."
```

A More Advanced Generator with Multiple Options

```python
from transformers import pipeline

def creative_writing_assistant():
    """
    An interactive creative writing assistant that generates text
    with user-specified parameters.
    """
    generator = pipeline("text-generation", model="gpt2")

    # Define preset configurations
    presets = {
        "formal": {
            "temperature": 0.3,
            "top_p": 0.85,
            "top_k": 20,
            "repetition_penalty": 1.2,
            "description": "Professional, structured, predictable"
        },
        "casual": {
            "temperature": 0.7,
            "top_p": 0.9,
            "top_k": 50,
            "repetition_penalty": 1.2,
            "description": "Natural, conversational, balanced"
        },
        "creative": {
            "temperature": 1.0,
```

```
            "top_p": 0.95,
            "top_k": 100,
            "repetition_penalty": 1.3,
            "description": "Imaginative, surprising, artistic"
        },
        "wild": {
            "temperature": 1.5,
            "top_p": 0.95,
            "top_k": 200,
            "repetition_penalty": 1.5,
            "description": "Experimental, unpredictable, avant-garde"
        }
    }

    # Generate with all presets for comparison
    prompt = "The old bookshop on the corner"

    print("=== Creative Writing Presets Comparison ===\n")
    print(f"Prompt: \"{prompt}\"\n")

    for name, config in presets.items():
        result = generator(
            prompt,
            max_new_tokens=40,
            temperature=config["temperature"],
            top_p=config["top_p"],
            top_k=config["top_k"],
            repetition_penalty=config["repetition_penalty"],
            do_sample=True,
            num_return_sequences=1
        )

        generated = result[0]['generated_text'][len(prompt):].strip()
        print(f"[{name.upper()}] ({config['description']})")
        print(f"  {prompt} {generated}")
        print()


# Run the comparison
creative_writing_assistant()
```

14.9 Prompt Engineering Basics

What Is Prompt Engineering?

Prompt engineering is the art of crafting your input text to get better results from a generation model. The prompt is your instruction to the model -- and how you phrase it matters enormously:

```
+-----------------------------------------------------------------+
|        PROMPT ENGINEERING PRINCIPLES                            |
+-----------------------------------------------------------------+
|                                                                  |
|  1. BE SPECIFIC                                                 |
|     Bad:  "Write something about dogs"                          |
|     Good: "Write a paragraph describing a golden retriever      |
|            playing fetch in a park on a sunny afternoon"         |
|                                                                  |
|  2. PROVIDE CONTEXT                                             |
|     Bad:  "Continue this story"                                 |
|     Good: "Continue this fantasy story about a wizard:          |
|            [your story beginning]"                               |
|                                                                  |
|  3. USE EXAMPLES (FEW-SHOT)                                     |
|     "Translate English to French:                                |
|      Hello -> Bonjour                                           |
|      Goodbye -> Au revoir                                       |
|      Thank you ->"                                              |
|                                                                  |
|  4. SET THE TONE                                                |
|     "Write in a formal, academic style:"                        |
|     "Write in a casual, friendly tone:"                         |
|                                                                  |
+-----------------------------------------------------------------+
```

Prompt Examples

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

# Example 1: Few-shot learning through prompts
prompt_fewshot = """Convert the following movie reviews to one-word sentiment:

Review: "This movie was absolutely wonderful!"
Sentiment: Positive

Review: "Terrible acting and boring plot."
Sentiment: Negative

Review: "A masterpiece of modern cinema."
Sentiment:"""

result = generator(prompt_fewshot, max_new_tokens=5, do_sample=False)
print("Few-shot example:")
print(result[0]['generated_text'])
print()
```

```
# Example 2: Structured output through prompts
prompt_structured = """Product: Wireless Bluetooth Headphones
Price: $49.99
Rating: 4.5/5 stars

Write a short product description:
These premium wireless Bluetooth headphones offer"""

result = generator(prompt_structured, max_new_tokens=40,
                   temperature=0.7, do_sample=True)
print("Structured prompt:")
print(result[0]['generated_text'])
print()

# Example 3: Style-specific prompt
prompt_style = """Write in the style of a nature documentary narrator:

The camera slowly pans across the African savanna at dawn."""

result = generator(prompt_style, max_new_tokens=50,
                   temperature=0.8, do_sample=True)
print("Style prompt:")
print(result[0]['generated_text'])
```

Prompt Engineering Tips

```
+-----------------------------------------------------------------+
|        PROMPT ENGINEERING TIPS                                  |
+-----------------------------------------------------------------+
|                                                                  |
|  DO:                                                             |
|    + Be specific about what you want                            |
|    + Provide examples of desired output format                  |
|    + Set the tone and style explicitly                          |
|    + Include relevant context                                   |
|    + Use clear separators between sections                      |
|                                                                  |
|  DO NOT:                                                         |
|    - Use vague instructions ("write something good")            |
|    - Assume the model knows your intent                         |
|    - Use very short prompts for complex tasks                   |
|    - Forget that the model continues YOUR text                  |
|    - Expect perfect results on the first try                    |
|                                                                  |
+-----------------------------------------------------------------+
```

Common Mistakes

1. Forgetting `do_sample=True`. Temperature, top-k, and top-p only work when sampling is enabled. Without `do_sample=True`, the model always picks the most likely word regardless of your settings.
2. Setting temperature to 0. A temperature of exactly 0 causes division by zero errors in some implementations. Use a very small value like 0.01 instead, or set `do_sample=False` for greedy decoding.
3. Confusing `max_length` and `max_new_tokens`. `max_length` includes the prompt in its count; `max_new_tokens` does not. Using `max_length=20` with a 15-token prompt gives you only 5 new tokens.
4. Using too high a repetition penalty. Values above 2.0 can make the model avoid common words entirely, producing incoherent text. Stay between 1.1 and 1.5 for most use cases.
5. Not setting a pad token for GPT-2. GPT-2 does not have a pad token by default. When generating multiple sequences, you must set `tokenizer.pad_token = tokenizer.eos_token`.

Best Practices

1. Start with balanced settings. Begin with temperature=0.7, top_p=0.9, and repetition_penalty=1.2. Adjust from there based on results.
2. Use `max_new_tokens` instead of `max_length`. It is more intuitive and avoids confusion about prompt length.
3. Generate multiple versions. Use `num_return_sequences=3` or more, then pick the best result. Generation is inherently random, so more attempts give better odds.
4. Invest time in prompt engineering. A well-crafted prompt often matters more than parameter tuning. Experiment with different prompt formats.
5. Use repetition penalty for longer texts. For outputs longer than 50 tokens, always set `repetition_penalty` between 1.1 and 1.3 to avoid loops.

Quick Summary

Text generation works by predicting one token at a time, with each prediction based on all previous tokens (autoregressive generation). Temperature controls randomness -- low values produce predictable text, high values produce creative text. Top-k sampling limits choices to the k most likely tokens, while top-p (nucleus) sampling adapts the number of choices based on the probability distribution. Repetition penalty prevents the model from getting stuck in loops. The Hugging Face `generate()` method provides full control over all these parameters. Good prompt engineering -- being specific, providing examples, and setting the tone -- is often more important than parameter tuning.

Key Points

- Autoregressive generation predicts one token at a time, feeding each prediction back as input
- Temperature controls randomness: low (0.2) is predictable, high (1.5) is creative, medium (0.7) is balanced
- Top-k sampling keeps only the k most likely tokens at each step
- Top-p (nucleus) sampling adapts the number of choices to the model's confidence -- preferred over top-k
- Always set `do_sample=True` when using temperature, top-k, or top-p
- Use `max_new_tokens` instead of `max_length` for clearer control over output length
- Repetition penalty (1.1-1.5) prevents the model from generating repetitive loops
- Generate multiple sequences and pick the best one
- Prompt engineering (being specific, providing examples) is as important as parameter tuning

---

Practice Questions

1. Explain the difference between greedy decoding and sampling. When would you use each approach?
2. You want to generate a formal business email. What temperature, top-k, and top-p values would you choose, and why?
3. What is the advantage of top-p sampling over top-k sampling? Give an example where top-k would include too many or too few words.

4. Your text generation model keeps producing the same sentence over and over. What parameter would you adjust, and what value would you set?
5. Explain what `max_length=50` and `max_new_tokens=50` mean. If your prompt is 20 tokens long, how many new tokens would each setting generate?

Exercises

Exercise 1: Temperature Explorer

Write a program that generates text from the same prompt at 5 different temperatures (0.1, 0.5, 0.7, 1.0, and 1.5). For each temperature, generate 3 sequences. Print all results in a neatly formatted table. Observe how the text changes with temperature.

Exercise 2: Build a Creative Writing Tool

Create a function called `write_with_style` that takes a prompt and a style parameter (one of: "academic", "poetic", "journalistic", "humorous"). The function should adjust the generation parameters and add style-specific instructions to the prompt to produce text matching the requested style.

Exercise 3: Story Chain Generator

Build a program that generates a story in 5 paragraphs. Each paragraph uses the last sentence of the previous paragraph as its prompt. Start with a single opening sentence. Use different creativity levels for different paragraphs (lower creativity for the introduction and conclusion, higher for the middle).

What Is Next?

You now know how to generate text in one language. But the world speaks thousands of languages. In the next chapter, we will explore multilingual NLP -- how modern models can understand and process text in many languages simultaneously. You will learn about cross-lingual models like mBERT and XLM-RoBERTa, translation pipelines, language detection, and the unique challenges of building NLP systems that work across languages and cultures.

Chapter 15: Multilingual NLP

What You Will Learn

In this chapter, you will learn:

- What cross-language models are and how they understand multiple languages
- How mBERT and XLM-RoBERTa work and when to use each one
- What zero-shot cross-lingual transfer is and why it is remarkable
- How to use Hugging Face translation pipelines for many language pairs
- How to detect the language of a text
- How multilingual embeddings represent words from different languages in the same space
- What challenges remain in multilingual NLP

Why This Chapter Matters

The internet has over 4.5 billion users, and they speak thousands of different languages. English accounts for only about 25% of internet content. If your NLP system only works in English, you are ignoring most of the world.

Imagine you build a customer support chatbot for an international company. Customers write in English, Spanish, French, Japanese, Arabic, and dozens of other languages. Do you need to build a separate NLP model for each language? That would be incredibly expensive and time-consuming.

This is where multilingual NLP comes in. Modern multilingual models can understand and process text in 100 or more languages -- often using a single model. Even more remarkably, you can train a model on data in one language and have it work in a completely different language. This is called zero-shot cross-lingual transfer, and it is one of the most surprising capabilities of modern NLP.

This chapter shows you how to build NLP systems that work across languages, making your applications accessible to a global audience.

15.1 Cross-Language Models

How Can One Model Understand 100 Languages?

The key insight is that languages share deep structural patterns. A sentence in English and its translation in French express the same meaning -- just with different words. Multilingual models learn these shared patterns:

```
+-----------------------------------------------------------------+
|        HOW MULTILINGUAL MODELS WORK                              |
+-----------------------------------------------------------------+
|                                                                   |
|  Step 1: Train on text from MANY languages simultaneously    |
|                                                                   |
|  English:  "The cat sits on the mat."                            |
|  French:   "Le chat est assis sur le tapis."                    |
|  Spanish:  "El gato se sienta en la alfombra."                  |
|  German:   "Die Katze sitzt auf der Matte."                     |
|  Japanese: "□□□□□□□□□□□□□□□"                        |
|                                                                   |
|  Step 2: The model discovers shared patterns                     |
|                                                                   |
|  It learns that "cat", "chat", "gato", "Katze", "□"         |
|  all represent similar concepts and appear in similar            |
|  sentence positions. They get similar internal                   |
|  representations (embeddings).                                   |
|                                                                   |
|  Step 3: Knowledge transfers between languages                   |
|                                                                   |
|  If the model learns what "happy" means in English,              |
|  it already somewhat understands "heureux" (French),             |
|  "feliz" (Spanish), and "gluecklich" (German).                  |
|                                                                   |
+-----------------------------------------------------------------+
```

mBERT: Multilingual BERT

mBERT (multilingual BERT) was one of the first widely-used multilingual models:

```python
from transformers import pipeline

# mBERT for Named Entity Recognition across languages
ner = pipeline("ner", model="bert-base-multilingual-cased",
               grouped_entities=True)

# Test with different languages
texts = {
    "English": "Barack Obama was born in Hawaii.",
    "French": "Emmanuel Macron est le president de la France.",
    "German": "Angela Merkel war Bundeskanzlerin von Deutschland.",
    "Spanish": "Gabriel Garcia Marquez nacio en Colombia.",
}

print("=== Multilingual NER with mBERT ===\n")
for language, text in texts.items():
    entities = ner(text)
    print(f"[{language}] {text}")
    for entity in entities:
        print(f"  -> {entity['word']} ({entity['entity_group']}, "
              f"confidence: {entity['score']:.4f})")
    print()
```

Expected output:

```
=== Multilingual NER with mBERT ===

[English] Barack Obama was born in Hawaii.
  -> Barack Obama (PER, confidence: 0.9987)
  -> Hawaii (LOC, confidence: 0.9976)

[French] Emmanuel Macron est le president de la France.
  -> Emmanuel Macron (PER, confidence: 0.9945)
  -> France (LOC, confidence: 0.9982)

[German] Angela Merkel war Bundeskanzlerin von Deutschland.
  -> Angela Merkel (PER, confidence: 0.9956)
  -> Deutschland (LOC, confidence: 0.9978)

[Spanish] Gabriel Garcia Marquez nacio en Colombia.
  -> Gabriel Garcia Marquez (PER, confidence: 0.9912)
  -> Colombia (LOC, confidence: 0.9967)
```

Line-by-line explanation:

- `model="bert-base-multilingual-cased"` -- Loads mBERT, which was trained on text from 104 languages. "cased" means it preserves uppercase and lowercase (important for recognizing proper nouns in many languages)
- The same model recognizes person names and locations across English, French, German, and Spanish -- without needing separate models for each language

- Confidence scores are high across all languages, showing that mBERT has learned language-independent patterns for entity recognition

XLM-RoBERTa: The Improved Multilingual Model

XLM-RoBERTa (Cross-lingual Language Model - RoBERTa) is a stronger multilingual model trained on 2.5 terabytes of text in 100 languages:

```python
from transformers import pipeline

# XLM-RoBERTa for sentiment analysis across languages
classifier = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment"
)

# Product reviews in different languages
reviews = {
    "English": "This product is absolutely amazing! Best purchase ever.",
    "French": "Ce produit est absolument incroyable! Le meilleur achat.",
    "German": "Dieses Produkt ist absolut erstaunlich! Bester Kauf.",
    "Dutch": "Dit product is absoluut geweldig! Beste aankoop ooit.",
    "Spanish": "Este producto es absolutamente increible! La mejor compra.",
}

print("=== Multilingual Sentiment Analysis ===\n")
for language, review in reviews.items():
    result = classifier(review)
    stars = result[0]['label']
    confidence = result[0]['score']
    print(f"[{language}] {review}")
    print(f"  Rating: {stars} (confidence: {confidence:.4f})")
    print()
```

Expected output:

```
=== Multilingual Sentiment Analysis ===

[English] This product is absolutely amazing! Best purchase ever.
  Rating: 5 stars (confidence: 0.8234)

[French] Ce produit est absolument incroyable! Le meilleur achat.
  Rating: 5 stars (confidence: 0.7891)

[German] Dieses Produkt ist absolut erstaunlich! Bester Kauf.
  Rating: 5 stars (confidence: 0.7654)

[Dutch] Dit product is absoluut geweldig! Beste aankoop ooit.
  Rating: 5 stars (confidence: 0.7432)
```

```
[Spanish] Este producto es absolutamente increible! La mejor compra.
  Rating: 5 stars (confidence: 0.7567)
```

mBERT vs XLM-RoBERTa

```
+-----------------------------------------------------------------+
|        mBERT vs XLM-RoBERTa COMPARISON                          |
+-----------------------------------------------------------------+
|                                                                  |
|  Feature          | mBERT              | XLM-RoBERTa            |
|  -----------------|--------------------|---------------------- |
|  Languages        | 104                | 100                    |
|  Training data    | Wikipedia only     | CommonCrawl (much      |
|                   |                    | more data)             |
|  Parameters       | 110M (base)        | 125M (base)            |
|                   |                    | 355M (large)           |
|  Performance      | Good               | Significantly better |
|  Best for         | Quick experiments  | Production systems     |
|                                                                  |
|  Recommendation: Use XLM-RoBERTa when accuracy matters.       |
|  Use mBERT for quick prototyping or when you need speed.       |
|                                                                  |
+-----------------------------------------------------------------+
```

15.2 Zero-Shot Cross-Lingual Transfer

What Is Zero-Shot Cross-Lingual Transfer?

This is one of the most remarkable capabilities of multilingual models. Zero-shot cross-lingual transfer means:

1. You fine-tune a model on data in Language A (for example, English)
2. The model works well on Language B (for example, Japanese) -- without ever seeing Japanese training data

```
+-----------------------------------------------------------------+
|        ZERO-SHOT CROSS-LINGUAL TRANSFER                         |
+-----------------------------------------------------------------+
|                                                                  |
|  Traditional approach:                                          |
|    Train on English data --> Works in English                   |
|    Train on French data  --> Works in French                    |
```

```
|    Train on German data  --> Works in German                    |
|    (Need separate training data for EACH language!)             |
|                                                                 |
|  Zero-shot cross-lingual transfer:                             |
|    Train on English data --> Works in English                   |
|                          --> Also works in French!              |
|                          --> Also works in German!              |
|                          --> Also works in Japanese!            |
|    (Only need training data in ONE language!)                   |
|                                                                 |
|  Why does this work?                                            |
|  The multilingual model has learned shared representations    |
|  across languages. "Happy" and "heureux" are close in the     |
|  model's internal space, so what it learns about "happy"      |
|  automatically applies to "heureux".                          |
|                                                                 |
+-----------------------------------------------------------------+
```

Demonstrating Zero-Shot Transfer

```python
from transformers import pipeline

# This model was fine-tuned on English NLI data
# but works in many languages thanks to zero-shot transfer
classifier = pipeline(
    "zero-shot-classification",
    model="joeddav/xlm-roberta-large-xnli"
)

# Define categories (in English)
categories = ["sports", "politics", "technology", "entertainment"]

# Test with texts in different languages
texts = {
    "English": "The team won the championship game in overtime.",
    "French": "Le parlement a vote une nouvelle loi sur l'environnement.",
    "Spanish": "El nuevo telefono tiene una pantalla increible.",
    "German": "Der Film hat den Oscar fuer den besten Film gewonnen.",
    "Italian": "La squadra ha vinto il campionato dopo una partita emozionante.",
}

print("=== Zero-Shot Cross-Lingual Classification ===\n")
print(f"Categories: {categories}\n")

for language, text in texts.items():
    result = classifier(text, categories)
    top_label = result['labels'][0]
    top_score = result['scores'][0]
    print(f"[{language}] {text}")
    print(f"  -> {top_label} (confidence: {top_score:.4f})")
    print()
```

Expected output:

```
=== Zero-Shot Cross-Lingual Classification ===

Categories: ['sports', 'politics', 'technology', 'entertainment']

[English] The team won the championship game in overtime.
  -> sports (confidence: 0.9234)

[French] Le parlement a vote une nouvelle loi sur l'environnement.
  -> politics (confidence: 0.8876)

[Spanish] El nuevo telefono tiene una pantalla increible.
  -> technology (confidence: 0.9012)

[German] Der Film hat den Oscar fuer den besten Film gewonnen.
  -> entertainment (confidence: 0.9456)

[Italian] La squadra ha vinto il campionato dopo una partita emozionante.
  -> sports (confidence: 0.8901)
```

Line-by-line explanation:

- `model="joeddav/xlm-roberta-large-xnli"` -- An XLM-RoBERTa model fine-tuned on the XNLI (Cross-lingual Natural Language Inference) dataset. It was trained primarily on English data
- `categories = ["sports", "politics", "technology", "entertainment"]` -- The categories are defined in English
- Despite the categories being in English, the model correctly classifies texts in French, Spanish, German, and Italian
- The Italian sentence about a team winning a championship is correctly classified as sports, even though the model was never trained on Italian sports text

Why Zero-Shot Transfer Works

```
+-------------------------------------------------------------+
|        WHY ZERO-SHOT TRANSFER WORKS                          |
+-------------------------------------------------------------+
|                                                              |
|  The multilingual model creates a SHARED SPACE where         |
|  similar meanings cluster together, regardless of language:  |
|                                                              |
|  In the model's internal representation:                     |
|                                                              |
|              "football"  (English)                           |
|              "futbol"    (Spanish)       <-- All close       |
|              "Fussball"  (German)            together!       |
```

```
|              "calcio"    (Italian)                            |
|                                                               |
|              "president" (English)                            |
|              "presidente"(Spanish)       <-- All close        |
|              "Praesident"(German)             together!        |
|              "presidente"(Italian)                            |
|                                                               |
|  When the model learns "football" is about sports,           |
|  it automatically knows "futbol" is about sports too!        |
|                                                               |
+---------------------------------------------------------------+
```

15.3 Translation Pipelines

Basic Translation

Hugging Face makes translation between many language pairs straightforward:

```python
from transformers import pipeline

# English to French translation
en_to_fr = pipeline("translation", model="Helsinki-NLP/opus-mt-en-fr")

# Translate some sentences
sentences = [
    "Hello, how are you today?",
    "Machine learning is a powerful technology.",
    "I would like to order a coffee, please.",
    "The weather is beautiful this morning."
]

print("=== English to French Translation ===\n")
for sentence in sentences:
    result = en_to_fr(sentence)
    print(f"English: {sentence}")
    print(f"French:  {result[0]['translation_text']}")
    print()
```

Expected output:

```
=== English to French Translation ===

English: Hello, how are you today?
French:  Bonjour, comment allez-vous aujourd'hui?
```

```
English: Machine learning is a powerful technology.
French:  L'apprentissage automatique est une technologie puissante.

English: I would like to order a coffee, please.
French:  Je voudrais commander un cafe, s'il vous plait.

English: The weather is beautiful this morning.
French:  Le temps est magnifique ce matin.
```

Line-by-line explanation:

- `pipeline("translation", model="Helsinki-NLP/opus-mt-en-fr")` -- Creates a translation pipeline using the OPUS-MT model. The model name follows the pattern `opus-mt-{source}-{target}`, so `en-fr` means English to French
- Helsinki-NLP provides models for over 1,000 language pairs

Multiple Language Pairs

```python
from transformers import pipeline

# Create translators for different language pairs
translators = {
    "English to German": pipeline("translation",
                                  model="Helsinki-NLP/opus-mt-en-de"),
    "English to Spanish": pipeline("translation",
                                   model="Helsinki-NLP/opus-mt-en-es"),
    "English to Japanese": pipeline("translation",
                                    model="Helsinki-NLP/opus-mt-en-jap"),
    "English to Chinese": pipeline("translation",
                                   model="Helsinki-NLP/opus-mt-en-zh"),
}

text = "Artificial intelligence will transform healthcare."

print(f"Original (English): {text}\n")
for pair_name, translator in translators.items():
    try:
        result = translator(text)
        print(f"{pair_name}: {result[0]['translation_text']}")
    except Exception as e:
        print(f"{pair_name}: (model not available)")
```

Expected output:

```
Original (English): Artificial intelligence will transform healthcare.

English to German: Kuenstliche Intelligenz wird das Gesundheitswesen veraendern.
```

```
English to Spanish: La inteligencia artificial transformara la atencion sanitaria.
English to Japanese: □□□□□□□□□□□□□□
English to Chinese: □□□□□□□□□□□□□
```

Finding Translation Models

```
+-----------------------------------------------------------------+
|        FINDING THE RIGHT TRANSLATION MODEL                      |
+-----------------------------------------------------------------+
|                                                                  |
|  Helsinki-NLP provides models for many language pairs:          |
|                                                                  |
|  Pattern: Helsinki-NLP/opus-mt-{source}-{target}                |
|                                                                  |
|  Examples:                                                       |
|    en-fr  (English to French)                                   |
|    en-de  (English to German)                                   |
|    en-es  (English to Spanish)                                  |
|    fr-en  (French to English)                                   |
|    de-en  (German to English)                                   |
|    en-zh  (English to Chinese)                                  |
|    en-ar  (English to Arabic)                                   |
|    en-ru  (English to Russian)                                  |
|                                                                  |
|  Language codes follow ISO 639-1 standard:                      |
|    en=English, fr=French, de=German, es=Spanish,               |
|    zh=Chinese, ja=Japanese, ko=Korean, ar=Arabic,              |
|    ru=Russian, pt=Portuguese, it=Italian, nl=Dutch             |
|                                                                  |
|  Search on huggingface.co/models for more pairs!                |
|                                                                  |
+-----------------------------------------------------------------+
```

Building a Simple Translation App

```python
from transformers import pipeline

def create_translator(source_lang, target_lang):
    """
    Create a translation function for a specific language pair.

    Parameters:
        source_lang: Source language code (e.g., "en")
        target_lang: Target language code (e.g., "fr")

    Returns:
```

```
        A function that translates text
    """
    model_name = f"Helsinki-NLP/opus-mt-{source_lang}-{target_lang}"

    try:
        translator = pipeline("translation", model=model_name)
        print(f"Loaded translation model: {source_lang} -> {target_lang}")
    except Exception as e:
        print(f"Could not load model for {source_lang} -> {target_lang}")
        return None

    def translate(text):
        result = translator(text, max_length=512)
        return result[0]['translation_text']

    return translate


# Create translators
en_to_fr = create_translator("en", "fr")
en_to_de = create_translator("en", "de")

# Use them
if en_to_fr:
    print("\nEnglish -> French:")
    print(en_to_fr("The library is open every day from nine to five."))

if en_to_de:
    print("\nEnglish -> German:")
    print(en_to_de("The library is open every day from nine to five."))
```

Expected output:

```
Loaded translation model: en -> fr
Loaded translation model: en -> de

English -> French:
La bibliotheque est ouverte tous les jours de neuf a cinq heures.

English -> German:
Die Bibliothek ist jeden Tag von neun bis fuenf geoeffnet.
```

15.4 Language Detection

Identifying the Language of Text

Before you can translate or process text, you often need to know what language it is in:

```python
# Method 1: Using the langdetect library
# pip install langdetect

from langdetect import detect, detect_langs

texts = [
    "Hello, how are you today?",
    "Bonjour, comment allez-vous?",
    "Hallo, wie geht es Ihnen?",
    "Hola, como estas?",
    "Ciao, come stai?",
    "Konnichiwa, ogenki desu ka?",
]

print("=== Language Detection ===\n")
for text in texts:
    language = detect(text)
    probabilities = detect_langs(text)

    print(f"Text: {text}")
    print(f"  Detected language: {language}")
    print(f"  All probabilities: {probabilities}")
    print()
```

Expected output:

```
=== Language Detection ===

Text: Hello, how are you today?
  Detected language: en
  All probabilities: [en:0.9999]

Text: Bonjour, comment allez-vous?
  Detected language: fr
  All probabilities: [fr:0.9999]

Text: Hallo, wie geht es Ihnen?
  Detected language: de
  All probabilities: [de:0.9999]

Text: Hola, como estas?
  Detected language: es
  All probabilities: [es:0.7142, pt:0.2857]

Text: Ciao, come stai?
  Detected language: it
  All probabilities: [it:0.9999]

Text: Konnichiwa, ogenki desu ka?
  Detected language: so
```

```
  All probabilities: [so:0.5714, tl:0.4285]
```

Line-by-line explanation:

- `detect(text)` -- Returns the most likely language code for the given text
- `detect_langs(text)` -- Returns all possible languages with their probabilities
- Notice that "Hola, como estas?" could be Spanish or Portuguese (both Romance languages with similar greetings), so the model shows probabilities for both
- Very short texts or romanized non-Latin scripts (like Japanese written in English letters) may be harder to detect

Using Transformers for Language Detection

```python
from transformers import pipeline

# Use a text classification model trained for language identification
lang_detector = pipeline(
    "text-classification",
    model="papluca/xlm-roberta-base-language-detection"
)

texts = [
    "The weather is nice today.",
    "Il fait beau aujourd'hui.",
    "Das Wetter ist heute schoen.",
    "El tiempo esta bonito hoy.",
    "Il tempo e bello oggi.",
    "Hoje o tempo esta bom.",
]

print("=== Transformer-Based Language Detection ===\n")
for text in texts:
    result = lang_detector(text)
    lang = result[0]['label']
    score = result[0]['score']
    print(f"Text: {text}")
    print(f"  Language: {lang} (confidence: {score:.4f})")
    print()
```

Expected output:

```
=== Transformer-Based Language Detection ===

Text: The weather is nice today.
  Language: en (confidence: 0.9998)

Text: Il fait beau aujourd'hui.
```

```
  Language: fr (confidence: 0.9997)

Text: Das Wetter ist heute schoen.
  Language: de (confidence: 0.9996)

Text: El tiempo esta bonito hoy.
  Language: es (confidence: 0.9995)

Text: Il tempo e bello oggi.
  Language: it (confidence: 0.9993)

Text: Hoje o tempo esta bom.
  Language: pt (confidence: 0.9994)
```

15.5 Multilingual Embeddings

What Are Multilingual Embeddings?

Multilingual embeddings map words and sentences from different languages into the same numerical space. Words with similar meanings in different languages end up close together:

```
+-------------------------------------------------------------+
|        MULTILINGUAL EMBEDDING SPACE                          |
+-------------------------------------------------------------+
|                                                              |
|  Imagine a 2D map (real embeddings have 768+ dimensions):    |
|                                                              |
|       "king" (en)                                            |
|       "roi" (fr)         <-- These are CLOSE together        |
|       "rey" (es)                                             |
|       "Koenig" (de)                                          |
|                                                              |
|                                                              |
|       "cat" (en)                                             |
|       "chat" (fr)        <-- These are CLOSE together        |
|       "gato" (es)                                            |
|       "Katze" (de)                                           |
|                                                              |
|                                                              |
|  "king" and "cat" are FAR apart (different concepts)         |
|  "king" and "roi" are CLOSE (same concept, different langs)  |
|                                                              |
+-------------------------------------------------------------+
```

Computing Multilingual Embeddings

```python
from transformers import AutoTokenizer, AutoModel
import torch

def get_sentence_embedding(text, model, tokenizer):
    """
    Get a numerical representation of a sentence.

    Parameters:
        text: The input sentence
        model: The transformer model
        tokenizer: The tokenizer for the model

    Returns:
        A tensor representing the sentence (768 dimensions)
    """
    inputs = tokenizer(text, return_tensors="pt", padding=True,
                       truncation=True, max_length=128)
    with torch.no_grad():
        outputs = model(**inputs)

    # Use the [CLS] token's output as the sentence embedding
    embedding = outputs.last_hidden_state[:, 0, :]
    return embedding


def cosine_similarity(a, b):
    """
    Compute cosine similarity between two vectors.
    Returns a value between -1 (opposite) and 1 (identical).
    """
    return torch.nn.functional.cosine_similarity(a, b).item()


# Load a multilingual model
model_name = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Sentences with the same meaning in different languages
same_meaning = {
    "English": "The cat is sitting on the mat.",
    "French": "Le chat est assis sur le tapis.",
    "Spanish": "El gato esta sentado en la alfombra.",
    "German": "Die Katze sitzt auf der Matte.",
}

# A sentence with a DIFFERENT meaning
different = "The stock market crashed yesterday."

# Compute embeddings
embeddings = {}
```

```
for lang, text in same_meaning.items():
    embeddings[lang] = get_sentence_embedding(text, model, tokenizer)

different_emb = get_sentence_embedding(different, model, tokenizer)

# Compare similarities
print("=== Multilingual Embedding Similarities ===\n")
print("Sentences with the SAME meaning (different languages):")
languages = list(same_meaning.keys())
for i in range(len(languages)):
    for j in range(i+1, len(languages)):
        sim = cosine_similarity(embeddings[languages[i]],
                                embeddings[languages[j]])
        print(f"  {languages[i]} vs {languages[j]}: {sim:.4f}")

print("\nSentences with DIFFERENT meanings:")
for lang in languages:
    sim = cosine_similarity(embeddings[lang], different_emb)
    print(f"  '{same_meaning[lang][:30]}...' vs "
          f"'The stock market...': {sim:.4f}")
```

Expected output:

```
=== Multilingual Embedding Similarities ===

Sentences with the SAME meaning (different languages):
  English vs French: 0.8934
  English vs Spanish: 0.8812
  English vs German: 0.8756
  French vs Spanish: 0.9123
  French vs German: 0.8645
  Spanish vs German: 0.8534

Sentences with DIFFERENT meanings:
  'The cat is sitting on the mat...' vs 'The stock market...': 0.4231
  'Le chat est assis sur le tapis...' vs 'The stock market...': 0.3987
  'El gato esta sentado en la alf...' vs 'The stock market...': 0.4012
  'Die Katze sitzt auf der Matte....' vs 'The stock market...': 0.3876
```

Line-by-line explanation:

- `get_sentence_embedding` -- Passes text through the model and takes the [CLS] token's representation as the sentence embedding
- `cosine_similarity` -- Measures how similar two vectors are. A value close to 1 means very similar; close to 0 means unrelated
- Sentences with the same meaning across languages show high similarity (around 0.85-0.91)
- Sentences with different meanings show low similarity (around 0.38-0.42)

- This confirms that the model places similar meanings close together in its internal space, regardless of language

Applications of Multilingual Embeddings

```
+-----------------------------------------------------------------+
|        USE CASES FOR MULTILINGUAL EMBEDDINGS                    |
+-----------------------------------------------------------------+
|                                                                  |
|  1. CROSS-LINGUAL SEARCH                                        |
|     Search in English, find results in any language             |
|     Query: "climate change" -> Finds French, German,            |
|     Spanish articles about the same topic                       |
|                                                                  |
|  2. MULTILINGUAL DOCUMENT CLUSTERING                            |
|     Group documents by topic regardless of language             |
|     All sports articles cluster together, whether in            |
|     English, French, or Japanese                                |
|                                                                  |
|  3. CROSS-LINGUAL DUPLICATE DETECTION                           |
|     Find duplicate content across languages                     |
|     Detect if a Spanish article is a translation of             |
|     an English article                                          |
|                                                                  |
|  4. MULTILINGUAL RECOMMENDATION                                 |
|     Recommend content in one language based on                  |
|     user preferences expressed in another language              |
|                                                                  |
+-----------------------------------------------------------------+
```

15.6 Challenges of Multilingual NLP

Challenge 1: Uneven Language Representation

Not all languages have equal amounts of training data:

```
+-----------------------------------------------------------------+
|        LANGUAGE REPRESENTATION IMBALANCE                        |
+-----------------------------------------------------------------+
|                                                                  |
|  Language         | Approx. Training Data | Model Quality      |
|  -----------------|-----------------------|------------------ |
|  English          | Very abundant         | Excellent          |
|  Chinese          | Abundant              | Very good          |
```

```
|  Spanish          | Abundant              | Very good         |
|  French           | Abundant              | Very good         |
|  German           | Good                  | Good              |
|  Japanese         | Good                  | Good              |
|  Korean           | Moderate              | Good              |
|  Arabic           | Moderate              | Moderate          |
|  Hindi            | Limited               | Moderate          |
|  Swahili          | Very limited          | Fair              |
|  Yoruba           | Minimal               | Poor              |
|  Guarani          | Minimal               | Poor              |
|                                                                 |
|  Models perform MUCH better on high-resource languages!        |
|                                                                 |
+----------------------------------------------------------------+
```

Challenge 2: Script Differences

Different languages use different writing systems, which affects tokenization:

```python
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-multilingual-cased")

# The same concept in different scripts
words = {
    "English": "artificial intelligence",
    "Chinese": "□□□□",
    "Arabic": "□□□□□□ □□□□□□□□□□",
    "Korean": "□□□□",
    "Russian": "искусственный интеллект",
    "Hindi": "□□□□□□□ □□□□□□□□□□□□",
}

print("=== Tokenization Across Scripts ===\n")
for language, word in words.items():
    tokens = tokenizer.tokenize(word)
    print(f"[{language}] {word}")
    print(f"  Tokens ({len(tokens)}): {tokens}")
    print()
```

Expected output:

```
=== Tokenization Across Scripts ===

[English] artificial intelligence
  Tokens (4): ['art', '##ific', '##ial', 'intelligence']

[Chinese] □□□□
  Tokens (4): ['□', '□', '□', '□']
```

```
[Arabic] □□□□□□□ □□□□□□□□□□
  Tokens (7): ['□□', '##□', '##□□', '##□', '□□□', '##□□', '##□□□□']

[Korean] □□□□
  Tokens (4): ['□', '##□', '##□', '##□']

[Russian] искусственный интеллект
  Tokens (6): ['иск', '##усст', '##венный', 'интел', '##лек', '##т']

[Hindi] □□□□□□□□ □□□□□□□□□□□□
  Tokens (8): ['□', '##□', '##□', '##□□', '##□□', '□', '##□□', '##□□']
```

Key observations:

- English gets efficient tokenization (few tokens) because the tokenizer's vocabulary is heavily English-biased
- Chinese characters are each treated as individual tokens
- Arabic and Hindi get split into many sub-word tokens, which uses up more of the model's maximum length
- This means the same amount of content in Hindi takes more tokens than in English, potentially leading to more truncation

Challenge 3: Cultural and Linguistic Differences

```
+-----------------------------------------------------------------+
|        CULTURAL AND LINGUISTIC CHALLENGES                       |
+-----------------------------------------------------------------+
|                                                                  |
|  1. WORD ORDER varies dramatically:                             |
|     English: "I eat rice"       (Subject-Verb-Object)           |
|     Japanese: "I rice eat"      (Subject-Object-Verb)           |
|     Arabic:   "Eat I rice"      (Verb-Subject-Object)           |
|                                                                  |
|  2. GENDER in language:                                         |
|     French: "Le chat" (masculine), "La chatte" (feminine)     |
|     English: "The cat" (no gender)                              |
|                                                                  |
|  3. FORMALITY levels:                                           |
|     Japanese: Three levels of politeness                        |
|     Korean: Seven speech levels                                 |
|     English: Mostly informal/formal distinction                 |
|                                                                  |
|  4. IDIOMS do not translate:                                    |
|     English: "It's raining cats and dogs"                       |
|     French: "Il pleut des cordes" (It's raining ropes)        |
|     German: "Es regnet Bindfaeden" (It's raining strings)     |
|                                                                  |
```

```
|  5. WRITING DIRECTION:                                        |
|     English: Left to right                                    |
|     Arabic: Right to left                                     |
|     Japanese: Can be top to bottom or left to right           |
|                                                                |
+---------------------------------------------------------------+
```

Challenge 4: Evaluation Difficulties

```python
# Demonstrating evaluation challenges

# Problem: How do you evaluate multilingual models fairly?

evaluation_issues = {
    "Data availability": (
        "Some languages have very few labeled evaluation datasets. "
        "How do you measure performance in Swahili if there's no "
        "Swahili test set?"
    ),
    "Translation quality": (
        "If you translate a test set from English to Hindi, "
        "translation errors can make the test unfair."
    ),
    "Cultural context": (
        "Sentiment in one culture may not transfer. "
        "'Not bad' is positive in English but may be interpreted "
        "differently in other cultures."
    ),
    "Annotation disagreement": (
        "Annotators from different cultures may label the same "
        "text differently based on cultural norms."
    ),
}

print("=== Challenges in Evaluating Multilingual Models ===\n")
for challenge, description in evaluation_issues.items():
    print(f"  {challenge}:")
    print(f"    {description}")
    print()
```

15.7 Practical Tips for Multilingual NLP

Building a Multilingual Pipeline

```python
from transformers import pipeline

def multilingual_analysis(text, source_lang=None):
    """
    Analyze text in any supported language.
    Detects language if not specified, then performs
    sentiment analysis and NER.

    Parameters:
        text: The text to analyze
        source_lang: Language code (auto-detected if None)

    Returns:
        Dictionary with analysis results
    """

    results = {}

    # Step 1: Detect language (if not provided)
    if source_lang is None:
        try:
            from langdetect import detect
            source_lang = detect(text)
        except ImportError:
            source_lang = "unknown"

    results["language"] = source_lang

    # Step 2: Multilingual sentiment analysis
    sentiment = pipeline(
        "sentiment-analysis",
        model="nlptown/bert-base-multilingual-uncased-sentiment"
    )
    sent_result = sentiment(text)
    results["sentiment"] = {
        "rating": sent_result[0]["label"],
        "confidence": sent_result[0]["score"]
    }

    # Step 3: Multilingual NER
    ner = pipeline("ner", model="bert-base-multilingual-cased",
                   grouped_entities=True)
    entities = ner(text)
    results["entities"] = [
        {"text": e["word"], "type": e["entity_group"],
         "confidence": e["score"]}
        for e in entities
    ]

    # Step 4: Translate to English (if not already English)
    if source_lang != "en":
        try:
            translator = pipeline(
                "translation",
```

```
                model=f"Helsinki-NLP/opus-mt-{source_lang}-en"
            )
            translation = translator(text)
            results["english_translation"] = translation[0]["translation_text"]
        except Exception:
            results["english_translation"] = "(translation model not available)"

    return results


# Test the multilingual pipeline
test_texts = [
    "Apple a annonce un nouveau produit revolutionnaire a Paris.",
    "Die Wissenschaftler in Berlin haben eine neue Entdeckung gemacht.",
]

for text in test_texts:
    print(f"Input: {text}")
    results = multilingual_analysis(text)
    print(f"  Language: {results['language']}")
    print(f"  Sentiment: {results['sentiment']['rating']} "
          f"(confidence: {results['sentiment']['confidence']:.4f})")
    print(f"  Entities:")
    for entity in results['entities']:
        print(f"    - {entity['text']} ({entity['type']})")
    if "english_translation" in results:
        print(f"  English: {results['english_translation']}")
    print()
```

Expected output:

```
Input: Apple a annonce un nouveau produit revolutionnaire a Paris.
  Language: fr
  Sentiment: 4 stars (confidence: 0.6234)
  Entities:
    - Apple (ORG)
    - Paris (LOC)
  English: Apple announced a revolutionary new product in Paris.

Input: Die Wissenschaftler in Berlin haben eine neue Entdeckung gemacht.
  Language: de
  Sentiment: 3 stars (confidence: 0.5432)
  Entities:
    - Berlin (LOC)
  English: The scientists in Berlin made a new discovery.
```

Common Mistakes

1. Assuming all languages work equally well. Multilingual models perform significantly better on high-resource languages (English, French, German) than on low-resource languages (Swahili, Yoruba). Always test on your target language.
2. Using monolingual models for multilingual tasks. A model trained only on English text (`bert-base-uncased`) will not work on French text. Use multilingual models (`bert-base-multilingual-cased`) for cross-language tasks.
3. Forgetting about tokenization efficiency. The same content in a low-resource language may use 2-3 times more tokens than in English. Account for this when setting `max_length`.
4. Ignoring script direction. Arabic and Hebrew are right-to-left languages. While the model handles this internally, your application's display logic must handle text direction correctly.
5. Using machine-translated test data for evaluation. Translation errors in your test set will make your model appear worse than it is. Use native-language test data whenever possible.

Best Practices

1. Use XLM-RoBERTa over mBERT for production. XLM-RoBERTa consistently outperforms mBERT across languages. Use mBERT only for quick experiments.
2. Leverage zero-shot transfer. If you have labeled data in English but need to support other languages, try zero-shot transfer first. Fine-tune on additional languages only if accuracy is insufficient.
3. Test on your target languages early. Do not assume a model will work on your target language just because it supports 100 languages. Always test with real data in each language you need to support.
4. Consider language-specific models for critical applications. For high-stakes applications in a single language, a model trained specifically for that language often outperforms a multilingual model. For example, CamemBERT for French or BETO for Spanish.
5. Handle language detection as a first step. In any multilingual pipeline, detect the language first, then route to the appropriate processing pipeline.

Quick Summary

Multilingual NLP enables models to process text in many languages using a single model. Cross-language models like mBERT (104 languages) and XLM-RoBERTa (100 languages) learn shared representations where similar meanings across languages cluster together. Zero-shot cross-lingual transfer allows a model fine-tuned on English data to work on other languages without any target-language training data. Hugging Face provides translation pipelines for hundreds of language pairs through the Helsinki-NLP models. Key challenges include uneven language representation, script differences, and cultural nuances. For production systems, use XLM-RoBERTa, test on all target languages, and consider language-specific models for critical applications.

Key Points

- Multilingual models like mBERT and XLM-RoBERTa understand 100+ languages in a single model
- XLM-RoBERTa outperforms mBERT and is recommended for production use
- Zero-shot cross-lingual transfer lets you train in one language and predict in another
- Hugging Face provides translation models for hundreds of language pairs via Helsinki-NLP
- Language detection should be the first step in any multilingual pipeline
- Multilingual embeddings place similar meanings close together regardless of language
- Performance varies significantly across languages -- high-resource languages work much better
- Tokenization efficiency differs by language and script -- account for this when setting max_length
- Cultural context matters -- idioms, formality levels, and sentiment expressions differ across cultures

Practice Questions

1. What is zero-shot cross-lingual transfer? Explain with an example why this is surprising and how it works.
2. You need to build a customer support chatbot that handles messages in English, Spanish, and Japanese. Would you use three separate models or one multilingual model? Justify your answer.

3. Why does a multilingual model tokenize English text more efficiently than Hindi text? What practical implications does this have?
4. Explain the difference between mBERT and XLM-RoBERTa. When would you choose one over the other?
5. A colleague built a sentiment analysis model using only English training data and claims it works for French. How would you verify this claim, and what performance difference would you expect?

Exercises

Exercise 1: Multilingual Sentiment Dashboard

Build a program that:

- Takes a list of product reviews in at least 3 different languages
- Detects the language of each review
- Performs sentiment analysis on each review
- Translates each review to English
- Prints a formatted report showing the original text, detected language, sentiment, and English translation

Exercise 2: Cross-Lingual Similarity Search

Using multilingual embeddings (with `bert-base-multilingual-cased`):

- Create a knowledge base of 5 sentences in English
- Given a query in French, find the most similar English sentence
- Print the similarity scores for all pairs

Hint: Use the cosine similarity function from section 15.5 and compute similarity between the French query embedding and all English sentence embeddings.

Exercise 3: Language-Aware Translation Chain

Build a program that takes a sentence in English and translates it through a chain of languages (English -> French -> German -> Spanish -> English). Compare the final English translation with the original to see how meaning changes through multiple translations. Try this with at least 3 different starting sentences.

What Is Next?

Congratulations! You have completed the NLP portion of this book. You now have a solid foundation in modern NLP -- from understanding how transformers work to using pre-trained models, fine-tuning them for your tasks, building question answering systems, generating text, and working across languages.

In the chapters ahead, we will shift our focus to Computer Vision -- teaching machines to see and understand images. You will learn how convolutional neural networks process images, how to classify objects, detect faces, and use cutting-edge vision models. The skills you have learned in NLP will serve you well, as many modern vision models share the same transformer architecture you have been studying.

Chapter 16: Image Fundamentals

What You Will Learn

In this chapter, you will learn:

- How computers see images as grids of numbers
- The difference between grayscale and color (RGB) images
- How to load images using PIL and OpenCV
- What image shapes and channels mean
- How to display images with matplotlib
- The most common color spaces: RGB, BGR, grayscale, and HSV
- Basic image operations: cropping, resizing, and rotating

Why This Chapter Matters

Every time you unlock your phone with your face, use a photo filter, or let Google Photos search for "beach," a computer is working with images. But here is the thing: computers do not see pictures the way you do. They see numbers. Lots and lots of numbers.

Before you can build any computer vision model -- whether it classifies cats versus dogs, detects pedestrians, or reads license plates -- you need to understand what an image actually is to a computer. This chapter gives you that foundation.

Think of it like learning to read sheet music before playing piano. You could memorize a few songs without it, but you would never truly understand what you are playing. Understanding image fundamentals is the "sheet music" of computer vision.

16.1 What Is an Image to a Computer?

When you look at a photograph, you see faces, trees, and sky. When a computer looks at the same photograph, it sees a giant grid of numbers. Each number represents the brightness or color at one tiny point in the image.

Pixels: The Building Blocks

A pixel (short for "picture element") is the smallest unit of an image. It is a single dot of color. If you zoom in very close on any digital image, you will see these tiny squares:

```
+-----+-----+-----+-----+-----+
| 200 | 180 | 170 | 160 | 150 |
+-----+-----+-----+-----+-----+
| 190 | 175 | 165 | 155 | 140 |
+-----+-----+-----+-----+-----+
| 180 | 170 | 160 | 150 | 130 |
+-----+-----+-----+-----+-----+
| 170 | 165 | 155 | 145 | 120 |
+-----+-----+-----+-----+-----+

Each box is one pixel.
Each number is the brightness (0 = black, 255 = white).
```

Why 0 to 255? Computers store pixel values using 8 bits (binary digits). With 8 bits, you can represent 2^8 = 256 different values, giving us the range 0 through 255.

A Real-World Analogy

Imagine a mosaic on a wall. From far away, you see a beautiful picture. But walk up close, and you see it is made of thousands of tiny colored tiles. Each tile is like a pixel. The more tiles you use (higher resolution), the more detailed the picture looks.

16.2 Grayscale vs. RGB Images

Grayscale: One Number Per Pixel

A grayscale image uses a single number for each pixel. That number represents brightness:

- 0 means completely black
- 255 means completely white
- Values in between are shades of gray

```
Grayscale Image (5x5 pixels):

  0   50  100  150  200      <- Row 0
 25   75  125  175  225      <- Row 1
 50  100  150  200  250      <- Row 2
 75  125  175  225  255      <- Row 3
100  150  200  250  255      <- Row 4

Think of it as a spreadsheet where each cell
holds one number (brightness).
```

RGB: Three Numbers Per Pixel

A color image uses three numbers for each pixel, one for each color channel:

- R (Red): How much red light (0-255)
- G (Green): How much green light (0-255)
- B (Blue): How much blue light (0-255)

```
One Pixel in an RGB Image:

    +-------+-------+-------+
    | R=255 | G=0   | B=0   |  -> Pure Red
    +-------+-------+-------+

    +-------+-------+-------+
    | R=0   | G=255 | B=0   |  -> Pure Green
    +-------+-------+-------+

    +-------+-------+-------+
    | R=0   | G=0   | B=255 |  -> Pure Blue
    +-------+-------+-------+

    +-------+-------+-------+
    | R=255 | G=255 | B=0   |  -> Yellow (Red + Green)
    +-------+-------+-------+

    +-------+-------+-------+
    | R=255 | G=255 | B=255 |  -> White (all colors)
    +-------+-------+-------+

    +-------+-------+-------+
    | R=0   | G=0   | B=0   |  -> Black (no color)
    +-------+-------+-------+
```

How RGB Channels Stack Up

Think of an RGB image as three transparent sheets stacked on top of each other:

```
RGB Image Structure:

         Width
    +------------------+
    |                  |  <- Red Channel (one layer of numbers)
    |   Red Layer      |
    |                  |
    +------------------+
    |                  |  <- Green Channel
    |   Green Layer    |
    |                  |
    +------------------+
    |                  |  <- Blue Channel
    |   Blue Layer     |
    |                  |
    +------------------+
         Height

Shape: (Height, Width, 3)
         ^       ^     ^
         |       |     |
       rows   columns  channels (R, G, B)
```

Real-world analogy: Think of RGB like mixing paint on a palette. Red, green, and blue are your three primary colors. By mixing different amounts of each, you can create any color. A pixel with (255, 165, 0) is orange -- lots of red, some green, no blue.

16.3 Loading Images with PIL

PIL (Python Imaging Library), now maintained as Pillow, is one of the most popular Python libraries for working with images. It is simple, intuitive, and works great for basic image tasks.

Installing Pillow

```python
# Install Pillow (the maintained fork of PIL)
# Run this in your terminal:
# pip install Pillow
```

Loading and Inspecting an Image

```python
from PIL import Image

# Load an image from disk
# Replace with your own image path
image = Image.open("cat.jpg")

# Check basic properties
print(f"Format: {image.format}")        # Output: Format: JPEG
print(f"Mode: {image.mode}")            # Output: Mode: RGB
print(f"Size: {image.size}")            # Output: Size: (640, 480)
print(f"Width: {image.width}")          # Output: Width: 640
print(f"Height: {image.height}")        # Output: Height: 480
```

Line-by-line explanation:

- `Image.open("cat.jpg")`: Opens the image file. It does NOT load all pixel data into memory right away -- PIL uses "lazy loading" to be efficient.
- `image.format`: The file format (JPEG, PNG, BMP, etc.).
- `image.mode`: The color mode. "RGB" means a color image with three channels. "L" means grayscale.
- `image.size`: Returns (width, height) as a tuple. Notice: width comes first in PIL.

Converting PIL Image to NumPy Array

To do math with images (which is what machine learning does), we need to convert them to NumPy arrays:

```python
from PIL import Image
import numpy as np

# Load image
image = Image.open("cat.jpg")

# Convert to NumPy array
pixel_array = np.array(image)

# Check the array
print(f"Type: {type(pixel_array)}")       # Output: Type: <class 'numpy.ndarray'>
print(f"Shape: {pixel_array.shape}")       # Output: Shape: (480, 640, 3)
print(f"Data type: {pixel_array.dtype}")   # Output: Data type: uint8
print(f"Min value: {pixel_array.min()}")   # Output: Min value: 0
print(f"Max value: {pixel_array.max()}")   # Output: Max value: 255
```

Line-by-line explanation:

- `np.array(image)`: Converts the PIL image into a NumPy array of numbers.

- `pixel_array.shape`: Returns (height, width, channels). Important: In NumPy, height comes first! This is (480, 640, 3) meaning 480 rows, 640 columns, 3 color channels.
- `pixel_array.dtype`: The data type is `uint8` (unsigned 8-bit integer), meaning values from 0 to 255.

Accessing Individual Pixels

```python
from PIL import Image
import numpy as np

image = Image.open("cat.jpg")
pixels = np.array(image)

# Get the pixel at row 100, column 200
pixel = pixels[100, 200]
print(f"Pixel at (100, 200): {pixel}")
# Output: Pixel at (100, 200): [142  98  67]
#          This means R=142, G=98, B=67 (a brownish color)

# Get just the red channel value at that pixel
red_value = pixels[100, 200, 0]    # Channel 0 = Red
green_value = pixels[100, 200, 1]  # Channel 1 = Green
blue_value = pixels[100, 200, 2]   # Channel 2 = Blue

print(f"Red: {red_value}")    # Output: Red: 142
print(f"Green: {green_value}")  # Output: Green: 98
print(f"Blue: {blue_value}")   # Output: Blue: 67
```

```
How pixel indexing works:

pixels[row, column, channel]
         |      |       |
         |      |       +-- 0=Red, 1=Green, 2=Blue
         |      +---------- Which column (left to right)
         +----------------- Which row (top to bottom)

        Column 0  Column 1  Column 2  ...
Row 0:  [R,G,B]   [R,G,B]   [R,G,B]
Row 1:  [R,G,B]   [R,G,B]   [R,G,B]
Row 2:  [R,G,B]   [R,G,B]   [R,G,B]
...
```

16.4 Loading Images with OpenCV

OpenCV (Open Source Computer Vision Library) is the most widely used computer vision library in the world. It is faster than PIL for many operations and has hundreds of image processing functions.

Installing OpenCV

```python
# Install OpenCV for Python
# Run this in your terminal:
# pip install opencv-python
```

Loading and Inspecting an Image

```python
import cv2

# Load an image
image = cv2.imread("cat.jpg")

# Check properties
print(f"Type: {type(image)}")          # Output: Type: <class 'numpy.ndarray'>
print(f"Shape: {image.shape}")          # Output: Shape: (480, 640, 3)
print(f"Data type: {image.dtype}")      # Output: Data type: uint8
print(f"Height: {image.shape[0]}")      # Output: Height: 480
print(f"Width: {image.shape[1]}")       # Output: Width: 640
print(f"Channels: {image.shape[2]}")    # Output: Channels: 3
```

Line-by-line explanation:

- `cv2.imread("cat.jpg")`: Loads the image directly as a NumPy array. Unlike PIL, OpenCV gives you a NumPy array immediately.
- `image.shape`: Same as PIL's NumPy array -- (height, width, channels).

CRITICAL WARNING: OpenCV Uses BGR, Not RGB!

This is the single most common source of bugs when working with images in Python:

```
PIL loads images as:    R, G, B  (Red, Green, Blue)
OpenCV loads images as: B, G, R  (Blue, Green, Red)

Same image, different channel order!

PIL pixel:    [255,   0,   0] = RED
OpenCV pixel: [255,   0,   0] = BLUE   <-- Surprise!
```

If you load an image with OpenCV and display it with matplotlib (which expects RGB), the colors will look wrong -- reds become blue and blues become red:

```python
import cv2

# Load with OpenCV (BGR order)
bgr_image = cv2.imread("cat.jpg")

# Convert BGR to RGB
rgb_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)

# Now rgb_image has correct colors for matplotlib
```

Line-by-line explanation:

- `cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)`: Converts from BGR color order to RGB color order. The function name `cvtColor` stands for "convert color."
- `cv2.COLOR_BGR2RGB`: A constant that tells OpenCV which conversion to perform. Read it as "BGR to RGB."

Loading Grayscale Images

```python
import cv2

# Method 1: Load directly as grayscale
gray_image = cv2.imread("cat.jpg", cv2.IMREAD_GRAYSCALE)
print(f"Shape: {gray_image.shape}")  # Output: Shape: (480, 640)
# Notice: only 2 dimensions! No channel dimension.

# Method 2: Convert a color image to grayscale
color_image = cv2.imread("cat.jpg")
gray_image = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)
print(f"Shape: {gray_image.shape}")  # Output: Shape: (480, 640)
```

Key observation: Grayscale images have shape (height, width) with no third dimension, because there is only one value per pixel (brightness), not three.

16.5 Image Shapes and Channels

Understanding image shapes is critical because deep learning models are very particular about the shape of their input data.

Shape Summary

```
Image Type          Shape                Example
-------------------------------------------------------
Grayscale           (H, W)              (480, 640)
RGB (PIL/NumPy)     (H, W, 3)           (480, 640, 3)
BGR (OpenCV)        (H, W, 3)           (480, 640, 3)
RGBA (with alpha)   (H, W, 4)           (480, 640, 4)

Where:
  H = Height (number of rows)
  W = Width (number of columns)
  3 = Three color channels (R, G, B)
  4 = Four channels (R, G, B, A) where A = transparency
```

Working with Channels

```python
from PIL import Image
import numpy as np

image = Image.open("cat.jpg")
pixels = np.array(image)

print(f"Full image shape: {pixels.shape}")
# Output: Full image shape: (480, 640, 3)

# Extract individual channels
red_channel = pixels[:, :, 0]     # All rows, all columns, channel 0
green_channel = pixels[:, :, 1]   # All rows, all columns, channel 1
blue_channel = pixels[:, :, 2]    # All rows, all columns, channel 2

print(f"Red channel shape: {red_channel.shape}")
# Output: Red channel shape: (480, 640)

print(f"Red channel sample values:")
print(red_channel[:3, :5])
# Output: (first 3 rows, first 5 columns of red values)
# [[142 145 148 150 152]
#  [140 143 146 149 151]
#  [138 141 144 147 150]]
```

Line-by-line explanation:

- `pixels[:, :, 0]`: NumPy slicing. The colons (`:`) mean "take everything." So this says "all rows, all columns, channel index 0 (red)."
- Each channel is a 2D array (height x width) containing just that color's intensity values.

```
Extracting Channels:

Full Image (H, W, 3):              Red Channel (H, W):
+---+---+---+                      +---+
|R|G|B|  <- pixel (0,0)           | R |  <- just the red value
+---+---+---+                      +---+
|R|G|B|  <- pixel (0,1)           | R |
+---+---+---+                      +---+
|R|G|B|  <- pixel (1,0)           | R |
+---+---+---+                      +---+
  ...                                ...

pixels[:, :, 0] extracts all R values
pixels[:, :, 1] extracts all G values
pixels[:, :, 2] extracts all B values
```

PyTorch Channel Convention

Deep learning frameworks like PyTorch use a different channel order than NumPy and PIL:

```
NumPy/PIL:  (H, W, C)    "channels last"    (480, 640, 3)
PyTorch:    (C, H, W)    "channels first"   (3, 480, 640)

For batches:
NumPy/PIL:  (B, H, W, C)   (32, 480, 640, 3)
PyTorch:    (B, C, H, W)   (32, 3, 480, 640)

Where B = batch size, C = channels, H = height, W = width
```

Converting between them:

```python
import numpy as np

# NumPy image: (H, W, C)
numpy_image = np.random.randint(0, 256, size=(480, 640, 3), dtype=np.uint8)
print(f"NumPy shape: {numpy_image.shape}")
# Output: NumPy shape: (480, 640, 3)

# Convert to PyTorch order: (C, H, W)
pytorch_order = np.transpose(numpy_image, (2, 0, 1))
print(f"PyTorch shape: {pytorch_order.shape}")
# Output: PyTorch shape: (3, 480, 640)

# Convert back to NumPy order: (H, W, C)
back_to_numpy = np.transpose(pytorch_order, (1, 2, 0))
print(f"Back to NumPy: {back_to_numpy.shape}")
# Output: Back to NumPy: (480, 640, 3)
```

Line-by-line explanation:

- `np.transpose(image, (2, 0, 1))`: Rearranges the axes. The tuple `(2, 0, 1)` means "put axis 2 first, then axis 0, then axis 1." Since axis 2 is channels, this moves channels to the front.

16.6 Displaying Images with Matplotlib

Matplotlib is the standard Python plotting library. It works perfectly for displaying images in Jupyter notebooks and scripts.

Basic Display

```python
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

# Load image with PIL
image = Image.open("cat.jpg")

# Display the image
plt.figure(figsize=(8, 6))
plt.imshow(image)
plt.title("A Cat")
plt.axis("off")       # Hide the axis numbers
plt.show()
```

Output: A window (or inline image in Jupyter) showing the cat photo with the title "A Cat" and no axis tick marks.

Displaying Multiple Images Side by Side

```python
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

# Load image
image = Image.open("cat.jpg")
pixels = np.array(image)

# Create a figure with 4 subplots
fig, axes = plt.subplots(1, 4, figsize=(16, 4))

# Original image
```

```
axes[0].imshow(pixels)
axes[0].set_title("Original (RGB)")
axes[0].axis("off")

# Red channel only
axes[1].imshow(pixels[:, :, 0], cmap="Reds")
axes[1].set_title("Red Channel")
axes[1].axis("off")

# Green channel only
axes[2].imshow(pixels[:, :, 1], cmap="Greens")
axes[2].set_title("Green Channel")
axes[2].axis("off")

# Blue channel only
axes[3].imshow(pixels[:, :, 2], cmap="Blues")
axes[3].set_title("Blue Channel")
axes[3].axis("off")

plt.tight_layout()
plt.show()
```

Output: Four images side by side: the original color image, then the red, green, and blue channels displayed with their respective color maps.

Line-by-line explanation:

- `plt.subplots(1, 4, figsize=(16, 4))`: Creates 1 row of 4 subplots, each in a 16x4 inch figure.
- `cmap="Reds"`: A colormap that maps single-channel values to shades of red. Without this, matplotlib would show a single channel in weird rainbow colors.
- `plt.tight_layout()`: Adjusts spacing so the subplots do not overlap.

Displaying Grayscale Images Correctly

```python
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

# Load and convert to grayscale
image = Image.open("cat.jpg").convert("L")
gray_pixels = np.array(image)

# WRONG way - shows weird colors
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.imshow(gray_pixels)           # No cmap specified!
```

```
plt.title("Wrong: Default Colormap")
plt.axis("off")

# RIGHT way - shows proper gray shades
plt.subplot(1, 2, 2)
plt.imshow(gray_pixels, cmap="gray")  # Specify gray colormap
plt.title("Correct: Gray Colormap")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Output: Two images. The left one shows the grayscale image with strange false colors (matplotlib's default "viridis" colormap). The right one shows it correctly in shades of gray.

Key lesson: Always use `cmap="gray"` when displaying grayscale images with matplotlib, or they will look wrong.

16.7 Color Spaces

A color space is a way of representing colors with numbers. Different color spaces are useful for different tasks. Think of it like measuring temperature -- you can use Fahrenheit, Celsius, or Kelvin. They all describe the same thing (temperature) but in different ways.

RGB Color Space

The most common color space. Every pixel has three values: Red, Green, Blue.

```
RGB Color Space:

        Green (0-255)
          |
          |
          +-------> Red (0-255)
         /
        /
       Blue (0-255)

Common RGB values:
  (255,   0,   0) = Red
  (  0, 255,   0) = Green
  (  0,   0, 255) = Blue
  (255, 255,   0) = Yellow
  (255, 165,   0) = Orange
```

```
  (128,   0, 128) = Purple
  (255, 255, 255) = White
  (  0,   0,   0) = Black
  (128, 128, 128) = Gray
```

Used by: PIL, matplotlib, most image formats, web colors, and most deep learning models.

BGR Color Space

Same channels as RGB, but in reverse order: Blue, Green, Red.

```
RGB pixel: [Red, Green, Blue]  = [255,   0,   0] = RED
BGR pixel: [Blue, Green, Red]  = [255,   0,   0] = BLUE

Same numbers, different meaning!
```

Used by: OpenCV. This exists for historical reasons (early camera hardware used BGR order).

Grayscale

A single brightness value per pixel. Converting from RGB to grayscale uses a weighted formula:

```
Gray = 0.299 * R + 0.587 * G + 0.114 * B

Why not equal weights?
Human eyes are most sensitive to green,
then red, then blue. The weights reflect
how our eyes perceive brightness.

Example:
  RGB = (200, 100, 50)
  Gray = 0.299 * 200 + 0.587 * 100 + 0.114 * 50
       = 59.8 + 58.7 + 5.7
       = 124.2 (rounds to 124)
```

HSV Color Space

HSV stands for Hue, Saturation, Value. It describes color the way humans think about it:

- Hue (H): The color itself (red, blue, green, etc.) -- represented as an angle from 0 to 360 degrees (or 0-180 in OpenCV)
- Saturation (S): How vivid or pure the color is (0 = gray, 255 = full color)

- Value (V): How bright the color is (0 = dark, 255 = bright)

```
HSV Color Space:

        Value (Brightness)
          |
          |  Saturation
          |  /
          | /
          +-------> Hue (Color wheel)

Hue values (in OpenCV, 0-180):
  0   = Red
  30  = Orange/Yellow
  60  = Yellow/Green
  90  = Green/Cyan
  120 = Blue
  150 = Purple/Pink
  180 = Red (wraps around)

Real-world analogy:
  Hue = Which paint color you pick
  Saturation = How much water you add (diluted vs pure)
  Value = How much light shines on it (dark room vs bright room)
```

Why use HSV? It is much easier to select a color range in HSV. For example, to find all "red" objects, you just check if Hue is near 0 (or near 180). In RGB, "red" is spread across all three channels, making it harder to isolate.

Converting Between Color Spaces

```python
import cv2
import numpy as np
from PIL import Image

# Load image with OpenCV (BGR)
bgr_image = cv2.imread("cat.jpg")

# BGR to RGB
rgb_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)

# BGR to Grayscale
gray_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2GRAY)

# BGR to HSV
hsv_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2HSV)

# Print shapes
print(f"BGR shape: {bgr_image.shape}")    # Output: BGR shape: (480, 640, 3)
```

```
print(f"RGB shape: {rgb_image.shape}")    # Output: RGB shape: (480, 640, 3)
print(f"Gray shape: {gray_image.shape}")  # Output: Gray shape: (480, 640)
print(f"HSV shape: {hsv_image.shape}")    # Output: HSV shape: (480, 640, 3)

# Inspect HSV values at one pixel
h, s, v = hsv_image[100, 200]
print(f"Hue: {h}, Saturation: {s}, Value: {v}")
# Output: Hue: 15, Saturation: 140, Value: 180
# This means: orangish color, moderately vivid, fairly bright
```

Using PIL for Color Space Conversion

```python
from PIL import Image
import numpy as np

# Load image
image = Image.open("cat.jpg")   # RGB by default

# Convert to grayscale
gray = image.convert("L")
print(f"Mode: {gray.mode}")              # Output: Mode: L
print(f"Shape: {np.array(gray).shape}")   # Output: Shape: (480, 640)

# Convert to RGBA (with transparency channel)
rgba = image.convert("RGBA")
print(f"Mode: {rgba.mode}")               # Output: Mode: RGBA
print(f"Shape: {np.array(rgba).shape}")    # Output: Shape: (480, 640, 4)
```

16.8 Basic Image Operations

Cropping

Cropping means cutting out a rectangular region of an image. Think of it like using scissors to cut a photo.

```python
from PIL import Image
import numpy as np

# Load image
image = Image.open("cat.jpg")
pixels = np.array(image)
```

```
print(f"Original size: {image.size}")  # Output: Original size: (640, 480)

# --- Method 1: Crop with PIL ---
# PIL uses (left, upper, right, lower) coordinates
# Crop a 200x200 region starting at (100, 50)
cropped_pil = image.crop((100, 50, 300, 250))
print(f"Cropped PIL size: {cropped_pil.size}")
# Output: Cropped PIL size: (200, 200)

# --- Method 2: Crop with NumPy slicing ---
# NumPy uses [row_start:row_end, col_start:col_end]
cropped_np = pixels[50:250, 100:300]
print(f"Cropped NumPy shape: {cropped_np.shape}")
# Output: Cropped NumPy shape: (200, 200, 3)
```

```
Cropping Coordinates:

PIL:  image.crop((left, top, right, bottom))

    (0,0)_________________(W,0)
      |                    |
      |   (left,top)       |
      |      +--------+   |
      |      | CROPPED|   |
      |      | REGION |   |
      |      +--------+   |
      |         (right,bottom)
      |____________________|
    (0,H)              (W,H)

NumPy: pixels[top:bottom, left:right]

    Same region, but rows come before columns.
```

Resizing

Resizing changes the dimensions of an image. This is essential for deep learning because models expect a fixed input size.

```python
from PIL import Image
import cv2
import numpy as np

# --- Method 1: Resize with PIL ---
image = Image.open("cat.jpg")
print(f"Original: {image.size}")  # Output: Original: (640, 480)
```

```
# Resize to exact dimensions
resized = image.resize((224, 224))
print(f"Resized: {resized.size}")  # Output: Resized: (224, 224)

# Resize with high-quality resampling
resized_hq = image.resize((224, 224), resample=Image.LANCZOS)
print(f"Resized HQ: {resized_hq.size}")  # Output: Resized HQ: (224, 224)

# --- Method 2: Resize with OpenCV ---
cv_image = cv2.imread("cat.jpg")
print(f"Original: {cv_image.shape[:2]}")  # Output: Original: (480, 640)

# OpenCV resize takes (width, height) -- same order as PIL
resized_cv = cv2.resize(cv_image, (224, 224))
print(f"Resized: {resized_cv.shape[:2]}")  # Output: Resized: (224, 224)

# With interpolation method
resized_cv_hq = cv2.resize(cv_image, (224, 224),
                           interpolation=cv2.INTER_LANCZOS4)
print(f"Resized HQ: {resized_cv_hq.shape[:2]}")
# Output: Resized HQ: (224, 224)
```

Line-by-line explanation:

- `image.resize((224, 224))`: Resizes to 224x224 pixels. The number 224 is very common in deep learning (used by ResNet, VGG, and many other models).
- `Image.LANCZOS`: A high-quality resampling filter. When shrinking images, LANCZOS gives sharper results. When enlarging, it gives smoother results.
- `cv2.INTER_LANCZOS4`: OpenCV's equivalent of the LANCZOS filter.

```
Resizing Methods (Interpolation):

NEAREST:   Fastest, but blocky (picks nearest pixel)
BILINEAR:  Good balance of speed and quality
BICUBIC:   Higher quality, slightly slower
LANCZOS:   Best quality, slowest

For deep learning preprocessing, BILINEAR is usually
good enough. For final display, use LANCZOS.
```

Rotating

Rotating turns an image by a specified angle.

```python
from PIL import Image
import cv2
```

```
import numpy as np

# --- Method 1: Rotate with PIL ---
image = Image.open("cat.jpg")

# Rotate 45 degrees counterclockwise
rotated = image.rotate(45)
print(f"Rotated size: {rotated.size}")
# Output: Rotated size: (640, 480)
# Note: Same size, corners are cut off and filled with black

# Rotate and expand to fit the whole image
rotated_expand = image.rotate(45, expand=True)
print(f"Rotated expanded size: {rotated_expand.size}")
# Output: Rotated expanded size: (791, 791)
# Note: Image is larger to fit the rotated content

# --- Method 2: Rotate with OpenCV ---
cv_image = cv2.imread("cat.jpg")
height, width = cv_image.shape[:2]

# Create rotation matrix (center, angle, scale)
center = (width // 2, height // 2)
rotation_matrix = cv2.getRotationMatrix2D(center, 45, 1.0)

# Apply rotation
rotated_cv = cv2.warpAffine(cv_image, rotation_matrix, (width, height))
print(f"Rotated shape: {rotated_cv.shape[:2]}")
# Output: Rotated shape: (480, 640)
```

Line-by-line explanation:

- `image.rotate(45)`: Rotates 45 degrees counterclockwise. Positive angles go counterclockwise.
- `expand=True`: Makes the output image larger so nothing gets cut off.
- `cv2.getRotationMatrix2D(center, 45, 1.0)`: Creates a 2x3 transformation matrix. Parameters are: center point, angle, scale (1.0 means no scaling).
- `cv2.warpAffine(...)`: Applies the transformation to the image.

Flipping

```python
from PIL import Image
import numpy as np

image = Image.open("cat.jpg")

# Flip horizontally (mirror)
flipped_h = image.transpose(Image.FLIP_LEFT_RIGHT)
```

```
# Flip vertically (upside down)
flipped_v = image.transpose(Image.FLIP_TOP_BOTTOM)

print(f"Original size: {image.size}")      # Output: Original size: (640, 480)
print(f"H-flipped size: {flipped_h.size}") # Output: H-flipped size: (640, 480)
print(f"V-flipped size: {flipped_v.size}") # Output: V-flipped size: (640, 480)
# Flipping does not change the image size
```

16.9 Putting It All Together

Here is a complete example that loads an image, explores its properties, converts between color spaces, and performs basic operations:

```python
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load the image
image = Image.open("cat.jpg")
print("=== Image Properties ===")
print(f"Format: {image.format}")
print(f"Mode: {image.mode}")
print(f"Size (W x H): {image.size}")

# Step 2: Convert to NumPy array
pixels = np.array(image)
print(f"\n=== NumPy Array ===")
print(f"Shape (H, W, C): {pixels.shape}")
print(f"Data type: {pixels.dtype}")
print(f"Value range: [{pixels.min()}, {pixels.max()}]")

# Step 3: Create grayscale version
gray = image.convert("L")
gray_pixels = np.array(gray)
print(f"\n=== Grayscale ===")
print(f"Shape: {gray_pixels.shape}")

# Step 4: Crop, resize, and rotate
cropped = image.crop((100, 50, 400, 350))
resized = image.resize((224, 224), resample=Image.LANCZOS)
rotated = image.rotate(30, expand=True)

# Step 5: Display everything
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
```

```
axes[0, 0].imshow(pixels)
axes[0, 0].set_title(f"Original {image.size}")
axes[0, 0].axis("off")

axes[0, 1].imshow(gray_pixels, cmap="gray")
axes[0, 1].set_title(f"Grayscale {gray_pixels.shape}")
axes[0, 1].axis("off")

axes[0, 2].imshow(np.array(cropped))
axes[0, 2].set_title(f"Cropped {cropped.size}")
axes[0, 2].axis("off")

axes[1, 0].imshow(np.array(resized))
axes[1, 0].set_title(f"Resized {resized.size}")
axes[1, 0].axis("off")

axes[1, 1].imshow(np.array(rotated))
axes[1, 1].set_title(f"Rotated {rotated.size}")
axes[1, 1].axis("off")

# Show color channels
axes[1, 2].imshow(pixels[:, :, 0], cmap="gray")
axes[1, 2].set_title("Red Channel")
axes[1, 2].axis("off")

plt.suptitle("Image Fundamentals Overview", fontsize=16)
plt.tight_layout()
plt.show()
```

Output: A 2x3 grid of images showing the original, grayscale, cropped, resized, rotated, and red channel versions of the cat image.

Common Mistakes

1. Confusing RGB and BGR. If your image looks like it has swapped red and blue colors, you probably loaded it with OpenCV (BGR) and displayed it with matplotlib (expects RGB). Always convert with `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`.
2. Confusing width/height order. PIL's `.size` returns `(width, height)`, but NumPy's `.shape` returns `(height, width, channels)`. This inconsistency trips up many beginners.
3. Forgetting `cmap="gray"` for grayscale images. Without it, matplotlib uses a rainbow colormap that makes grayscale images look like heat maps.
4. Not checking if the image loaded. `cv2.imread()` returns `None` if the file is not found instead of raising an error. Always check: `if image is None: print("Failed to load image!")`.

5. Mixing up PIL Image and NumPy array. PIL Images have methods like `.resize()` and `.crop()`. NumPy arrays have slicing and mathematical operations. Convert with `np.array(pil_image)` or `Image.fromarray(numpy_array)`.

Best Practices

1. Always verify your image loaded correctly. Print the shape, dtype, and a few pixel values before proceeding.
2. Stick to one library when possible. Mixing PIL and OpenCV in the same pipeline introduces BGR/RGB conversion bugs. Pick one and use it consistently.
3. Use RGB for deep learning pipelines. Most pretrained models expect RGB input. Convert early in your pipeline.
4. Keep original images unchanged. Always work on copies. Use `image.copy()` or `pixels.copy()` before modifying.
5. Use `Image.LANCZOS` for resizing when quality matters (preprocessing for models). It produces the sharpest results.

Quick Summary

An image is a grid of numbers. Grayscale images use one number per pixel (brightness). Color images use three numbers per pixel (Red, Green, Blue). PIL and OpenCV are the two main Python libraries for loading and manipulating images. PIL uses RGB order; OpenCV uses BGR order -- this is the most common source of bugs. NumPy represents images as arrays with shape (height, width, channels). Deep learning models (PyTorch) expect channels first: (channels, height, width). Matplotlib is the standard tool for displaying images in Python. Color spaces like HSV can be more useful than RGB for certain tasks like color detection.

Key Points

- A pixel is the smallest unit of a digital image, represented as a number (grayscale) or a group of numbers (color).
- Grayscale images have shape (H, W); RGB images have shape (H, W, 3).

- PIL loads images in RGB order. OpenCV loads images in BGR order. Always convert when switching between them.
- PIL `.size` returns (width, height). NumPy `.shape` returns (height, width, channels). The order is different!
- PyTorch expects images in (C, H, W) format, while NumPy uses (H, W, C). Use `np.transpose()` to convert.
- Always use `cmap="gray"` when displaying grayscale images with matplotlib.
- HSV color space separates color (Hue) from intensity (Value), making it easier to detect specific colors.

Practice Questions

1. You have a NumPy array with shape `(720, 1280, 3)`. What are the height, width, and number of channels of this image?
2. You load an image with OpenCV and display it with matplotlib. The sky looks orange instead of blue. What went wrong, and how do you fix it?
3. What is the difference between `image.size` in PIL and `pixels.shape` in NumPy? If PIL reports `(640, 480)`, what will NumPy report?
4. Why does the grayscale conversion formula use different weights for R, G, and B (`0.299, 0.587, 0.114`) instead of equal weights?
5. You want to convert a NumPy image from shape `(480, 640, 3)` to PyTorch format. What function do you use, and what will the resulting shape be?

Exercises

Exercise 1: Image Explorer

Write a function `explore_image(path)` that takes a file path, loads the image with PIL, and prints:

- File format
- Color mode
- Size (width x height)
- NumPy shape

- Data type
- Min, max, and mean pixel values
- Whether it is grayscale or color

Test it on at least two different images.

Exercise 2: Channel Visualizer

Load a color image and create a matplotlib figure with 5 subplots showing:

1. The original RGB image
2. The Red channel (using the "Reds" colormap)
3. The Green channel (using the "Greens" colormap)
4. The Blue channel (using the "Blues" colormap)
5. The grayscale version

Add appropriate titles to each subplot.

Exercise 3: Image Operations Pipeline

Write a script that:

1. Loads an image
2. Crops the center 50% of the image (remove the outer 25% on each side)
3. Resizes the crop to exactly 256x256 pixels
4. Rotates the resized image by 90 degrees
5. Flips it horizontally
6. Saves the final result as "processed.png"
7. Displays the original and final images side by side

What Is Next?

Now that you understand what images are and how to work with them in Python, the next chapter covers Image Preprocessing -- the essential step of preparing images for deep learning models. You will learn how to normalize pixel values, apply transformations, and build efficient preprocessing pipelines using torchvision. These skills are what bridge the gap between raw images and model-ready data.

Chapter 17: Image Preprocessing

What You Will Learn

In this chapter, you will learn:

- Why images need preprocessing before feeding them to deep learning models
- How to resize images to the exact dimensions models expect
- How to normalize pixel values using 0-1 scaling and ImageNet statistics
- How to convert between color spaces for model input
- How to use torchvision transforms to build preprocessing pipelines
- The differences between PIL and OpenCV for preprocessing
- How to build reusable preprocessing pipelines
- How to preprocess batches of images efficiently

Why This Chapter Matters

Imagine trying to compare the height of buildings using photos taken from different distances, angles, and cameras. Some photos are close-up, some are far away, some are dark, some are bright. You cannot compare them fairly until you standardize them.

Deep learning models face the same problem. Raw images come in all sizes, brightness levels, and color ranges. A model trained on 224x224 images will crash if you feed it a 4000x3000 photo from your phone. A model expecting pixel values between 0 and 1 will produce garbage if you give it values between 0 and 255.

Image preprocessing is the bridge between raw, messy real-world images and the clean, standardized input that models need. Get this wrong, and even the best model will fail. Get it right, and you are halfway to a working computer vision system.

17.1 Why Preprocess Images?

Deep learning models are surprisingly picky about their input. Here is why preprocessing matters:

Problem 1: Images Have Different Sizes

```
Your photos might be:
  Photo A: 4032 x 3024 pixels (12 megapixels)
  Photo B: 1920 x 1080 pixels (1080p)
  Photo C:  640 x  480 pixels (VGA)

But your model expects:
  Input: 224 x 224 pixels (exactly!)

Without resizing, the model cannot even accept these images.
```

Problem 2: Pixel Values Are Too Large

```
Raw pixel values: 0 to 255 (integers)

Neural networks work best with small floating-point numbers.
Large values cause:
  - Gradient explosion (weights grow out of control)
  - Slow convergence (takes forever to train)
  - Numerical instability (math errors)

After normalization: roughly -2.5 to 2.5 (floats)
```

Problem 3: Different Libraries Use Different Formats

```
PIL image:      (W, H) size,   RGB,   0-255, PIL Image object
OpenCV image:   (H, W, C),     BGR,   0-255, NumPy array
PyTorch tensor: (C, H, W),     RGB,   0-1,   torch.Tensor

A preprocessing pipeline handles ALL these conversions.
```

The Preprocessing Pipeline

```
Raw Image File
      |
      v
  Load Image (PIL or OpenCV)
      |
      v
  Resize (e.g., to 224x224)
      |
      v
  Convert to RGB (if needed)
      |
      v
  Convert to Float (0-255 -> 0.0-1.0)
      |
      v
  Normalize (subtract mean, divide by std)
      |
      v
  Convert to Tensor (H,W,C -> C,H,W)
      |
      v
  Ready for Model!
```

17.2 Resizing Images for Models

Every major model architecture expects a specific input size. Here are the most common ones:

```
Model               Expected Input Size
-----------------------------------------
AlexNet             227 x 227
VGG-16/19           224 x 224
ResNet              224 x 224
Inception v3        299 x 299
EfficientNet-B0     224 x 224
EfficientNet-B7     600 x 600
ViT (Vision Transf) 224 x 224
```

Simple Resize

```python
from PIL import Image
```

```
# Load a large image
image = Image.open("photo.jpg")
print(f"Original size: {image.size}")
# Output: Original size: (4032, 3024)

# Resize to 224x224
resized = image.resize((224, 224), resample=Image.LANCZOS)
print(f"Resized: {resized.size}")
# Output: Resized: (224, 224)
```

Problem with simple resize: If the original image is not square, resizing to a square will stretch or squish the image, distorting objects.

```
Original (4:3 ratio):          After resize to 224x224:
+------------------+           +------------+
|                  |           |            |
|   Normal cat     |  ----->   | Squished   |
|                  |           | cat        |
+------------------+           +------------+

The cat looks wider/shorter. Not ideal!
```

Resize Then Center Crop (Better Approach)

A better strategy is to resize the shorter side to the target size, then crop the center:

```python
from PIL import Image

def resize_and_center_crop(image, target_size=224):
    """Resize shorter side to target_size, then center crop."""
    width, height = image.size

    # Step 1: Resize so shorter side = target_size
    if width < height:
        new_width = target_size
        new_height = int(height * target_size / width)
    else:
        new_height = target_size
        new_width = int(width * target_size / height)

    resized = image.resize((new_width, new_height), resample=Image.LANCZOS)

    # Step 2: Center crop to target_size x target_size
    left = (new_width - target_size) // 2
    top = (new_height - target_size) // 2
    right = left + target_size
    bottom = top + target_size
```

```
    cropped = resized.crop((left, top, right, bottom))
    return cropped

# Usage
image = Image.open("photo.jpg")
result = resize_and_center_crop(image, 224)
print(f"Result size: {result.size}")
# Output: Result size: (224, 224)
```

```
Resize + Center Crop:

Original (640x480):     Resize shorter side:    Center crop:
+------------------+   +------------------+    +------------+
|                  |   |                  |    |            |
|   Normal cat     |-->| Still normal     |--> | Normal cat |
|                  |   | proportions     |    | (centered) |
+------------------+   +------------------+    +------------+
                       (298 x 224)              (224 x 224)

No distortion! We just lose a bit of the edges.
```

Line-by-line explanation:

- We calculate the aspect ratio and resize so the shorter side matches the target.
- `(new_width - target_size) // 2`: Calculates how many pixels to cut from each side to center the crop.
- The result is a square image with correct proportions.

17.3 Normalization

Normalization means adjusting pixel values to a standard range. This is one of the most important preprocessing steps.

Method 1: Simple 0-1 Scaling

The simplest normalization: divide every pixel by 255 to get values between 0 and 1.

```python
import numpy as np
from PIL import Image
```

```
image = Image.open("cat.jpg")
pixels = np.array(image)

print(f"Before normalization:")
print(f"  dtype: {pixels.dtype}")        # Output: dtype: uint8
print(f"  range: [{pixels.min()}, {pixels.max()}]")
# Output: range: [0, 255]

# Normalize to 0-1
normalized = pixels.astype(np.float32) / 255.0

print(f"\nAfter normalization:")
print(f"  dtype: {normalized.dtype}")     # Output: dtype: float32
print(f"  range: [{normalized.min():.4f}, {normalized.max():.4f}]")
# Output: range: [0.0000, 1.0000]
```

Line-by-line explanation:

- `pixels.astype(np.float32)`: Converts from integer (uint8) to floating point. This is necessary because dividing integers in NumPy can produce unexpected results.
- `/ 255.0`: Divides every value by 255, mapping 0 to 0.0 and 255 to 1.0.

```
0-1 Normalization:

Before:  [0,   128,   255]  (integers, 0 to 255)
           |      |      |
           v      v      v
After:   [0.0, 0.502, 1.0]  (floats, 0.0 to 1.0)

Formula: normalized_value = original_value / 255.0
```

Method 2: ImageNet Normalization

Most pretrained models (ResNet, VGG, etc.) were trained on the ImageNet dataset. They expect pixel values normalized using ImageNet's mean and standard deviation for each channel.

```
ImageNet Statistics:

Channel    Mean     Std Dev
-----------------------------
Red        0.485    0.229
Green      0.456    0.224
Blue       0.406    0.225

These numbers come from calculating the average
pixel value across all 1.2 million ImageNet images.
```

The normalization formula:

```
normalized = (pixel_value / 255.0 - mean) / std

Step 1: Scale to 0-1       (divide by 255)
Step 2: Subtract the mean   (center around 0)
Step 3: Divide by std       (standardize spread)
```

```python
import numpy as np
from PIL import Image

# ImageNet normalization values
IMAGENET_MEAN = np.array([0.485, 0.456, 0.406])
IMAGENET_STD = np.array([0.229, 0.224, 0.225])

# Load and convert to float
image = Image.open("cat.jpg")
pixels = np.array(image).astype(np.float32) / 255.0

print(f"After 0-1 scaling:")
print(f"  Red channel mean: {pixels[:,:,0].mean():.4f}")
print(f"  Green channel mean: {pixels[:,:,1].mean():.4f}")
print(f"  Blue channel mean: {pixels[:,:,2].mean():.4f}")
# Output varies by image, e.g.:
# Red channel mean: 0.5123
# Green channel mean: 0.4456
# Blue channel mean: 0.3789

# Apply ImageNet normalization
normalized = (pixels - IMAGENET_MEAN) / IMAGENET_STD

print(f"\nAfter ImageNet normalization:")
print(f"  Red channel mean: {normalized[:,:,0].mean():.4f}")
print(f"  Range: [{normalized.min():.4f}, {normalized.max():.4f}]")
# Output varies by image, e.g.:
# Red channel mean: 0.1192
# Range: [-2.1179, 2.6400]
```

Why does this work? When you use a pretrained model, you must preprocess your images the same way the training images were preprocessed. ImageNet models "learned" what the world looks like with these specific normalization values baked in. Using different normalization is like speaking a different language -- the model will not understand.

```
Why ImageNet Normalization Matters:

Training:                        Inference:
ImageNet images                  Your images
       |                                |
  Normalize with                 Normalize with
```

```
  mean=[0.485,0.456,0.406]      mean=[0.485,0.456,0.406]  <- SAME!
  std=[0.229,0.224,0.225]       std=[0.229,0.224,0.225]   <- SAME!
       |                             |
       v                             v
  Model learns patterns          Model recognizes patterns
  from these values              using the same scale

If you skip normalization or use different values,
the model sees garbage, not images.
```

Method 3: Custom Normalization

If you train a model from scratch on your own dataset, you should calculate your own mean and standard deviation:

```python
import numpy as np
from PIL import Image
import os

def calculate_dataset_stats(image_dir):
    """Calculate mean and std of a dataset."""
    pixel_sum = np.zeros(3)
    pixel_sq_sum = np.zeros(3)
    num_pixels = 0

    for filename in os.listdir(image_dir):
        if filename.endswith((".jpg", ".png")):
            path = os.path.join(image_dir, filename)
            img = np.array(Image.open(path).convert("RGB"))
            img = img.astype(np.float32) / 255.0

            pixel_sum += img.sum(axis=(0, 1))
            pixel_sq_sum += (img ** 2).sum(axis=(0, 1))
            num_pixels += img.shape[0] * img.shape[1]

    mean = pixel_sum / num_pixels
    std = np.sqrt(pixel_sq_sum / num_pixels - mean ** 2)

    return mean, std

# Usage
# mean, std = calculate_dataset_stats("my_dataset/train/")
# print(f"Mean: {mean}")  # e.g., [0.512, 0.478, 0.401]
# print(f"Std: {std}")    # e.g., [0.267, 0.251, 0.274]
```

Line-by-line explanation:

- `pixel_sum += img.sum(axis=(0, 1))`: Sums all pixel values along height (axis 0) and width (axis 1), keeping the channel dimension. This accumulates the total across all images.

- `pixel_sq_sum`: Accumulates squared values, needed for the standard deviation formula.
- `std = sqrt(E[X^2] - (E[X])^2)`: This is the mathematical formula for standard deviation, calculated efficiently in one pass.

17.4 Color Space Conversions

Ensuring RGB Input

Most deep learning models expect RGB images. Here is how to handle different sources:

```python
from PIL import Image
import cv2
import numpy as np

# === PIL: Already RGB ===
pil_image = Image.open("photo.jpg")
print(f"PIL mode: {pil_image.mode}")  # Output: PIL mode: RGB
# No conversion needed!

# But sometimes images are grayscale or RGBA
gray_image = Image.open("gray_photo.jpg")
print(f"Gray mode: {gray_image.mode}")  # Output: Gray mode: L

# Convert grayscale to RGB (creates 3 identical channels)
rgb_from_gray = gray_image.convert("RGB")
print(f"Converted mode: {rgb_from_gray.mode}")  # Output: Converted mode: RGB

# Handle RGBA (with transparency)
png_image = Image.open("logo.png")
print(f"PNG mode: {png_image.mode}")  # Output: PNG mode: RGBA

# Convert RGBA to RGB (drops alpha channel)
rgb_from_rgba = png_image.convert("RGB")
print(f"Converted mode: {rgb_from_rgba.mode}")  # Output: Converted mode: RGB

# === OpenCV: BGR by default ===
cv_image = cv2.imread("photo.jpg")
# MUST convert to RGB for deep learning
rgb_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2RGB)
```

Safe Image Loading Function

```python
from PIL import Image
import numpy as np

def safe_load_image(path):
    """Load any image and ensure it is RGB format."""
    image = Image.open(path)

    # Handle different modes
    if image.mode == "RGB":
        return image
    elif image.mode == "RGBA":
        # Create white background and paste image
        background = Image.new("RGB", image.size, (255, 255, 255))
        background.paste(image, mask=image.split()[3])  # Use alpha as mask
        return background
    elif image.mode == "L":
        # Grayscale to RGB
        return image.convert("RGB")
    elif image.mode == "P":
        # Palette mode (like some GIFs)
        return image.convert("RGB")
    else:
        return image.convert("RGB")

# Usage
image = safe_load_image("any_image.png")
print(f"Mode: {image.mode}")  # Output: Mode: RGB (always!)
```

Line-by-line explanation:

- `image.mode == "RGBA"`: RGBA images have a transparency channel. Simply converting to RGB can produce artifacts (black areas where transparency was). Instead, we paste onto a white background.
- `image.split()[3]`: Splits the image into channels. Index 3 is the alpha (transparency) channel.
- `image.mode == "P"`: Palette mode uses a lookup table for colors. Common in GIF files.

17.5 Torchvision Transforms

Torchvision is PyTorch's computer vision library. Its `transforms` module provides ready-made preprocessing functions that you can chain together into pipelines.

Installing Torchvision

```python
# Install PyTorch and torchvision
# Run in your terminal:
# pip install torch torchvision
```

Basic Transforms

```python
from torchvision import transforms
from PIL import Image

# Load image
image = Image.open("cat.jpg")
print(f"Original: size={image.size}, mode={image.mode}")
# Output: Original: size=(640, 480), mode=RGB

# === Individual transforms ===

# 1. Resize
resize_transform = transforms.Resize((224, 224))
resized = resize_transform(image)
print(f"Resized: {resized.size}")
# Output: Resized: (224, 224)

# 2. Center Crop
crop_transform = transforms.CenterCrop(224)
cropped = crop_transform(image)
print(f"Cropped: {cropped.size}")
# Output: Cropped: (224, 224)

# 3. Convert to Tensor (also scales 0-255 to 0-1)
tensor_transform = transforms.ToTensor()
tensor = tensor_transform(image)
print(f"Tensor shape: {tensor.shape}")
print(f"Tensor range: [{tensor.min():.4f}, {tensor.max():.4f}]")
# Output: Tensor shape: torch.Size([3, 480, 640])
# Output: Tensor range: [0.0000, 1.0000]

# 4. Normalize with ImageNet stats
normalize_transform = transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
normalized = normalize_transform(tensor)
print(f"Normalized range: [{normalized.min():.4f}, {normalized.max():.4f}]")
# Output: Normalized range: [-2.1179, 2.6400]  (approximately)
```

Line-by-line explanation:

- `transforms.Resize((224, 224))`: Creates a transform that resizes images to 224x224. You can also pass a single number like `transforms.Resize(256)` to resize the shorter side to 256 while keeping the aspect ratio.

- `transforms.CenterCrop(224)`: Crops the center 224x224 region.
- `transforms.ToTensor()`: This does THREE things at once: (1) converts PIL Image to PyTorch tensor, (2) changes shape from (H, W, C) to (C, H, W), and (3) scales values from 0-255 to 0.0-1.0.
- `transforms.Normalize(mean, std)`: Applies per-channel normalization. This expects values already in the 0-1 range (i.e., after `ToTensor()`).

Composing a Pipeline with transforms.Compose

The real power comes from chaining transforms together:

```python
from torchvision import transforms
from PIL import Image

# Define the preprocessing pipeline
preprocess = transforms.Compose([
    transforms.Resize(256),              # Resize shorter side to 256
    transforms.CenterCrop(224),          # Crop center 224x224
    transforms.ToTensor(),               # PIL -> Tensor, 0-255 -> 0-1
    transforms.Normalize(                # ImageNet normalization
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# Apply the entire pipeline in one call
image = Image.open("cat.jpg")
tensor = preprocess(image)

print(f"Input: PIL Image {image.size}")
print(f"Output: Tensor {tensor.shape}")
print(f"Output range: [{tensor.min():.4f}, {tensor.max():.4f}]")
# Output: Input: PIL Image (640, 480)
# Output: Output: Tensor torch.Size([3, 224, 224])
# Output: Output range: [-2.1179, 2.6400]
```

```
transforms.Compose Pipeline:

PIL Image (640, 480)
       |
  Resize(256)         -> PIL Image (341, 256)
       |                 (shorter side = 256)
  CenterCrop(224)     -> PIL Image (224, 224)
       |                 (center square)
  ToTensor()          -> Tensor (3, 224, 224)
       |                 (channels first, 0-1 range)
  Normalize(...)      -> Tensor (3, 224, 224)
       |                 (centered, standardized)
       v
```

```
  Ready for model!
```

Training vs. Validation Transforms

During training, you typically apply data augmentation (random changes) to make the model more robust. During validation and testing, you use deterministic (non-random) transforms:

```python
from torchvision import transforms

# Training: includes random augmentations
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224),      # Random crop + resize
    transforms.RandomHorizontalFlip(),       # 50% chance to flip
    transforms.ColorJitter(                  # Random color changes
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.1
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# Validation/Testing: deterministic, no randomness
val_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])
```

Line-by-line explanation:

- `RandomResizedCrop(224)`: Randomly picks a region of the image, crops it, and resizes to 224x224. This teaches the model to recognize objects at different scales and positions.
- `RandomHorizontalFlip()`: Flips the image left-to-right with 50% probability. A cat is still a cat when flipped.
- `ColorJitter(...)`: Randomly adjusts brightness, contrast, saturation, and hue. This teaches the model to handle different lighting conditions.

```
Training Augmentation:

Same image, 4 different random transforms:

  +--------+  +--------+  +--------+  +--------+
  |  Cat   |  | Cat    |  |   Cat  |  |  taC   |
  | (dark) |  | (zoom) |  | (crop) |  | (flip) |
  +--------+  +--------+  +--------+  +--------+

The model sees many variations of each image,
making it more robust to real-world conditions.
```

17.6 PIL vs. OpenCV for Preprocessing

Both PIL and OpenCV can preprocess images. Here is when to use each:

Comparison Table

```
Feature              PIL (Pillow)         OpenCV
--------------------------------------------------
Default format       PIL Image object     NumPy array
Color order          RGB                  BGR
Speed                Slower               Faster
Ease of use          Simpler API          More complex
PyTorch integration  Excellent            Needs conversion
Image operations     Basic but good       Very comprehensive
Memory efficiency    Lazy loading         Loads everything
Best for             Deep learning prep   Computer vision tasks
```

Side-by-Side Preprocessing

```python
from PIL import Image
import cv2
import numpy as np
import torch

# === PIL Pipeline ===
def preprocess_pil(path):
    # Load
    image = Image.open(path).convert("RGB")
```

```
    # Resize
    image = image.resize((224, 224), resample=Image.LANCZOS)

    # To NumPy and normalize
    pixels = np.array(image).astype(np.float32) / 255.0

    # ImageNet normalization
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    pixels = (pixels - mean) / std

    # To PyTorch tensor (H,W,C) -> (C,H,W)
    tensor = torch.from_numpy(pixels.transpose(2, 0, 1)).float()
    return tensor

# === OpenCV Pipeline ===
def preprocess_opencv(path):
    # Load (BGR!)
    image = cv2.imread(path)

    # Convert BGR to RGB
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Resize
    image = cv2.resize(image, (224, 224), interpolation=cv2.INTER_LINEAR)

    # To float and normalize
    pixels = image.astype(np.float32) / 255.0

    # ImageNet normalization
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    pixels = (pixels - mean) / std

    # To PyTorch tensor (H,W,C) -> (C,H,W)
    tensor = torch.from_numpy(pixels.transpose(2, 0, 1)).float()
    return tensor

# Both produce the same result
tensor_pil = preprocess_pil("cat.jpg")
tensor_cv = preprocess_opencv("cat.jpg")
print(f"PIL output shape: {tensor_pil.shape}")
print(f"OpenCV output shape: {tensor_cv.shape}")
# Output: PIL output shape: torch.Size([3, 224, 224])
# Output: OpenCV output shape: torch.Size([3, 224, 224])
```

Recommendation: Use PIL with torchvision transforms for deep learning projects. It integrates seamlessly with PyTorch's data loading utilities and is less error-prone (no BGR/RGB confusion).

17.7 Building Preprocessing Pipelines

A Reusable Preprocessing Class

```python
from torchvision import transforms
from PIL import Image
import torch

class ImagePreprocessor:
    """Reusable image preprocessing for different model architectures."""

    # Presets for common models
    PRESETS = {
        "resnet": {"size": 224, "resize": 256},
        "inception": {"size": 299, "resize": 342},
        "efficientnet_b0": {"size": 224, "resize": 256},
        "efficientnet_b7": {"size": 600, "resize": 672},
        "vit": {"size": 224, "resize": 256},
    }

    def __init__(self, model_name="resnet", mode="eval"):
        """
        Args:
            model_name: One of the preset model names
            mode: "train" for training (with augmentation)
                  "eval" for inference (no augmentation)
        """
        config = self.PRESETS.get(model_name, self.PRESETS["resnet"])
        target_size = config["size"]
        resize_size = config["resize"]

        if mode == "train":
            self.transform = transforms.Compose([
                transforms.RandomResizedCrop(target_size),
                transforms.RandomHorizontalFlip(),
                transforms.ColorJitter(0.2, 0.2, 0.2, 0.1),
                transforms.ToTensor(),
                transforms.Normalize(
                    mean=[0.485, 0.456, 0.406],
                    std=[0.229, 0.224, 0.225]
                ),
            ])
        else:
            self.transform = transforms.Compose([
                transforms.Resize(resize_size),
                transforms.CenterCrop(target_size),
                transforms.ToTensor(),
                transforms.Normalize(
                    mean=[0.485, 0.456, 0.406],
                    std=[0.229, 0.224, 0.225]
                ),
```

```
            ])

    def __call__(self, image):
        """Preprocess a single PIL image."""
        if isinstance(image, str):
            image = Image.open(image).convert("RGB")
        return self.transform(image)

# Usage
preprocessor = ImagePreprocessor("resnet", mode="eval")

# Preprocess a single image
tensor = preprocessor("cat.jpg")
print(f"Output: {tensor.shape}")
# Output: Output: torch.Size([3, 224, 224])

# Preprocess for training
train_preprocessor = ImagePreprocessor("resnet", mode="train")
tensor_train = train_preprocessor("cat.jpg")
print(f"Training output: {tensor_train.shape}")
# Output: Training output: torch.Size([3, 224, 224])
```

Line-by-line explanation:

- `self.PRESETS`: A dictionary of common model configurations. This avoids hardcoding sizes everywhere.
- `__call__`: Makes the class callable like a function. You can use `preprocessor("cat.jpg")` instead of `preprocessor.process("cat.jpg")`.
- `isinstance(image, str)`: If a file path is passed, load it. If a PIL Image is passed, use it directly.

17.8 Batch Preprocessing

In real projects, you preprocess thousands of images. Doing this efficiently matters.

Using PyTorch DataLoader

The standard approach combines a `Dataset` with a `DataLoader`:

```python
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import os
```

```
import torch

class ImageDataset(Dataset):
    """A simple dataset that loads images from a directory."""

    def __init__(self, image_dir, transform=None):
        self.image_dir = image_dir
        self.transform = transform

        # Get all image file paths
        self.image_paths = [
            os.path.join(image_dir, f)
            for f in os.listdir(image_dir)
            if f.endswith((".jpg", ".jpeg", ".png"))
        ]

        print(f"Found {len(self.image_paths)} images")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # Load image
        image = Image.open(self.image_paths[idx]).convert("RGB")

        # Apply transforms
        if self.transform:
            image = self.transform(image)

        return image

# Define transforms
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# Create dataset and dataloader
dataset = ImageDataset("images/", transform=preprocess)
dataloader = DataLoader(
    dataset,
    batch_size=32,       # Process 32 images at a time
    shuffle=False,       # Keep original order
    num_workers=4,       # Use 4 parallel processes for loading
)

# Iterate through batches
for batch in dataloader:
    print(f"Batch shape: {batch.shape}")
    # Output: Batch shape: torch.Size([32, 3, 224, 224])
```

```
    break  # Just show one batch
```

```
Batch Preprocessing Flow:

DataLoader (num_workers=4)
       |
       +-- Worker 1: loads images 0,4,8,...
       +-- Worker 2: loads images 1,5,9,...
       +-- Worker 3: loads images 2,6,10,...
       +-- Worker 4: loads images 3,7,11,...
       |
       v
  Collate function stacks individual
  images into a batch tensor
       |
       v
  Batch: (32, 3, 224, 224)
    32 images, 3 channels, 224x224 pixels
```

Line-by-line explanation:

- `__len__`: Returns the total number of images. The DataLoader uses this to know how many batches to create.
- `__getitem__`: Returns one preprocessed image. The DataLoader calls this in parallel using multiple workers.
- `batch_size=32`: Groups 32 images into a single tensor. Larger batches are faster but use more memory.
- `num_workers=4`: Uses 4 separate processes to load and preprocess images in parallel while the GPU trains on the previous batch.

Preprocessing with Progress Tracking

```python
from torchvision import transforms
from PIL import Image
import os
import time

def batch_preprocess_and_save(input_dir, output_dir, transform):
    """Preprocess all images and save the results."""
    os.makedirs(output_dir, exist_ok=True)

    image_files = [
        f for f in os.listdir(input_dir)
        if f.endswith((".jpg", ".jpeg", ".png"))
    ]
```

```
    total = len(image_files)
    start_time = time.time()

    for i, filename in enumerate(image_files):
        # Load and preprocess
        path = os.path.join(input_dir, filename)
        image = Image.open(path).convert("RGB")
        tensor = transform(image)

        # Save preprocessed tensor
        save_path = os.path.join(
            output_dir,
            filename.rsplit(".", 1)[0] + ".pt"
        )
        import torch
        torch.save(tensor, save_path)

        # Progress update
        if (i + 1) % 100 == 0 or (i + 1) == total:
            elapsed = time.time() - start_time
            rate = (i + 1) / elapsed
            remaining = (total - i - 1) / rate
            print(f"[{i+1}/{total}] "
                  f"{rate:.1f} images/sec, "
                  f"~{remaining:.0f}s remaining")

    print(f"\nDone! Processed {total} images "
          f"in {time.time() - start_time:.1f} seconds.")

# Usage
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# batch_preprocess_and_save("raw_images/", "preprocessed/", preprocess)
# Output:
# [100/5000] 45.2 images/sec, ~108s remaining
# [200/5000] 46.1 images/sec, ~104s remaining
# ...
# Done! Processed 5000 images in 109.3 seconds.
```

17.9 Undoing Normalization for Display

After normalizing an image for model input, you cannot display it directly (the values are wrong for display). You need to denormalize to convert back to viewable values:

```python
import torch
import numpy as np
import matplotlib.pyplot as plt
from torchvision import transforms
from PIL import Image

def denormalize(tensor, mean, std):
    """Convert a normalized tensor back to displayable format."""
    # Clone to avoid modifying the original
    img = tensor.clone()

    # Reverse normalization: value = (normalized * std) + mean
    for channel in range(3):
        img[channel] = img[channel] * std[channel] + mean[channel]

    # Clip to valid range [0, 1]
    img = torch.clamp(img, 0, 1)

    # Convert from (C, H, W) to (H, W, C) for matplotlib
    img = img.permute(1, 2, 0).numpy()

    return img

# Preprocess an image
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

image = Image.open("cat.jpg")
tensor = preprocess(image)

# Try displaying the normalized tensor (wrong!)
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# This will look wrong - colors are off
wrong_display = tensor.permute(1, 2, 0).numpy()
wrong_display = np.clip(wrong_display, 0, 1)
axes[0].imshow(wrong_display)
axes[0].set_title("Normalized (Wrong Colors)")
axes[0].axis("off")

# Denormalize first (correct!)
correct_display = denormalize(
    tensor,
```

```
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
axes[1].imshow(correct_display)
axes[1].set_title("Denormalized (Correct)")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```

```
Denormalization:

Normalized value: -1.5
   * std (0.229) = -0.3435
   + mean (0.485) = 0.1415
Displayable value: 0.1415  (dark pixel)

Normalized value: 2.0
   * std (0.229) = 0.458
   + mean (0.485) = 0.943
Displayable value: 0.943  (bright pixel)

Formula: display_value = normalized * std + mean
(This is the REVERSE of normalization)
```

Common Mistakes

1. Forgetting to convert to RGB before preprocessing. Grayscale or RGBA images will crash your pipeline. Always call `.convert("RGB")` when loading.
2. Applying `Normalize()` before `ToTensor()`. `Normalize()` expects values in the 0-1 range (produced by `ToTensor()`). If you apply it to 0-255 values, the results will be wildly wrong.
3. Using training transforms for validation. Random augmentations during evaluation make your metrics unreliable. Always use deterministic transforms for validation and testing.
4. Not matching preprocessing between training and inference. If you trained with ImageNet normalization, you MUST use the same normalization at inference time. This is the number one cause of "my model works during training but fails in production."
5. Resizing without considering aspect ratio. Simple resize distorts objects. Use resize-then-crop or resize with padding to maintain proportions.
6. Forgetting that `ToTensor()` already scales to 0-1. If you manually divide by 255 and THEN apply `ToTensor()`, your values will be divided twice, giving you tiny numbers near zero.

Best Practices

1. Use `torchvision.transforms.Compose` to define clear, reusable preprocessing pipelines. Keep them as named variables, not inline.
2. Always use the same normalization for training and inference. Store the mean and std values as constants.
3. Use `num_workers > 0` in DataLoader to parallelize preprocessing. A good starting value is 4, or `os.cpu_count()` minus one.
4. Validate your preprocessing visually. Denormalize a few images and display them to make sure they look correct.
5. Keep preprocessing simple during debugging. Start with just `Resize`, `ToTensor`, and `Normalize`. Add augmentations only after the basic pipeline works.

Quick Summary

Image preprocessing converts raw images into the standardized format that deep learning models expect. The essential steps are: resize to the model's expected dimensions, convert to the correct color space (RGB), scale pixel values to floats (0-1 via `ToTensor()`), and normalize using dataset statistics (ImageNet mean and std for pretrained models). Torchvision's `transforms.Compose` lets you chain these steps into a clean pipeline. Training pipelines include random augmentations; validation pipelines do not. PyTorch's DataLoader handles batch preprocessing with parallel workers.

Key Points

- Deep learning models require fixed-size input images. Most standard models use 224x224 pixels.
- `ToTensor()` does three things: converts PIL to tensor, changes (H,W,C) to (C,H,W), and scales 0-255 to 0.0-1.0.
- ImageNet normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) is required for any pretrained model trained on ImageNet.

- Training transforms include random augmentations (flips, crops, color jitter). Validation transforms must be deterministic.
- Always match your inference preprocessing to your training preprocessing exactly.
- Use DataLoader with `num_workers > 0` for efficient batch preprocessing.
- Denormalize tensors before displaying them to get correct colors.

Practice Questions

1. Why do we normalize pixel values before feeding them to a neural network? What would happen if we did not?
2. What three things does `transforms.ToTensor()` do to a PIL image?
3. You trained a model using ImageNet normalization. At inference time, you forgot to normalize your test images. What would happen and why?
4. Why do we use different transforms for training versus validation? Give an example of a transform that should only be used during training.
5. You have images of different sizes: 800x600, 1200x900, and 640x480. Explain two strategies to make them all 224x224 without distortion.

Exercises

Exercise 1: Complete Preprocessing Pipeline

Write a preprocessing pipeline using `transforms.Compose` that:

1. Resizes the image so the shorter side is 256 pixels
2. Takes a center crop of 224x224
3. Converts to a tensor
4. Normalizes with ImageNet statistics

Apply it to three images of different sizes and verify all outputs have shape `(3, 224, 224)`.

Exercise 2: Dataset Statistics Calculator

Write a function that takes a directory of images and calculates:

- The mean pixel value for each channel (R, G, B)

- The standard deviation for each channel
- The min and max image dimensions in the dataset

Print the results and compare to ImageNet statistics.

Exercise 3: Visual Augmentation Explorer

Create a script that takes one image and applies these augmentations 9 times each, displaying results in a 3x3 grid:

1. `RandomResizedCrop(224)`
2. `RandomHorizontalFlip()` combined with `ColorJitter()`
3. `RandomRotation(30)` combined with `RandomGrayscale(p=0.2)`

Show how each augmentation produces different variations of the same image.

What Is Next?

Now that you know how to prepare images for deep learning, the next chapter teaches you how to classify images using Convolutional Neural Networks (CNNs). You will build a complete image classification pipeline from scratch -- loading a dataset, building a CNN model, training it, and evaluating its performance. Everything you learned about preprocessing in this chapter will be put to work.

Chapter 18: Image Classification with CNNs

What You Will Learn

In this chapter, you will learn:

- How to build a complete image classification pipeline from scratch
- How to organize datasets using `ImageFolder`
- How to create efficient data loaders for training and validation
- How to design and build a Convolutional Neural Network (CNN)
- How to write a training loop for image classification
- How to evaluate model performance with accuracy, confusion matrix, and per-class metrics
- How to visualize predictions to understand what your model gets right and wrong

Why This Chapter Matters

Image classification is the "Hello World" of computer vision. It answers a simple question: what is in this image? Is it a cat or a dog? A tumor or healthy tissue? A stop sign or a speed limit sign?

This one skill powers real applications everywhere. Doctors use it to screen X-rays. Factories use it to spot defective products on assembly lines. Farmers use it to identify crop diseases from photos. Social media platforms use it to detect inappropriate content.

More importantly, the classification pipeline you build in this chapter is the foundation for everything else in computer vision. Object detection, image segmentation, and even image generation all build on the same concepts: load data, define a model, train, evaluate, improve. Master this chapter, and you have a template for every computer vision project that follows.

18.1 The Classification Pipeline Overview

Before diving into code, let us see the entire pipeline at a high level:

```
Image Classification Pipeline:

1. ORGANIZE DATA
   images/
     train/
       cats/  -> cat1.jpg, cat2.jpg, ...
       dogs/  -> dog1.jpg, dog2.jpg, ...
     val/
       cats/  -> cat101.jpg, cat102.jpg, ...
       dogs/  -> dog101.jpg, dog102.jpg, ...

2. LOAD & PREPROCESS
   ImageFolder + DataLoader
   -> Batches of (images, labels)
   -> Each image: (3, 224, 224) tensor

3. BUILD MODEL
   CNN: Conv layers -> Pool -> FC layers -> Output
   Input: (batch, 3, 224, 224)
   Output: (batch, num_classes)

4. TRAIN
   For each epoch:
     Forward pass -> Loss -> Backward pass -> Update weights

5. EVALUATE
   Accuracy, confusion matrix, per-class metrics

6. VISUALIZE
   Show predictions with confidence scores
```

18.2 Organizing Your Dataset with ImageFolder

PyTorch's `ImageFolder` is a dataset class that automatically loads images from a directory structure where each subfolder represents a class.

Required Directory Structure

```
dataset/
    train/
        class_a/
            img001.jpg
            img002.jpg
            ...
        class_b/
            img001.jpg
            img002.jpg
            ...
    val/
        class_a/
            img001.jpg
            img002.jpg
            ...
        class_b/
            img001.jpg
            img002.jpg
            ...
```

The folder name IS the label. If you have a folder called "cats," all images inside it are automatically labeled as "cats."

Creating a Sample Dataset

For this chapter, we will create a simple synthetic dataset to demonstrate the pipeline. In real projects, you would use actual images:

```python
import os
import torch
import numpy as np
from PIL import Image

def create_sample_dataset(base_dir, num_train=100, num_val=20):
    """Create a simple synthetic dataset for demonstration."""
    classes = ["cats", "dogs", "birds"]
    splits = {
        "train": num_train,
        "val": num_val
    }

    for split, count in splits.items():
        for cls in classes:
            dir_path = os.path.join(base_dir, split, cls)
            os.makedirs(dir_path, exist_ok=True)

            for i in range(count):
                # Create a simple colored image for each class
                # In real projects, you would have actual photos
```

```
                img = np.random.randint(0, 256, (64, 64, 3),
                                        dtype=np.uint8)
                # Give each class a slight color bias so the model
                # has something to learn
                if cls == "cats":
                    img[:, :, 0] = np.clip(img[:, :, 0] + 50, 0, 255)
                elif cls == "dogs":
                    img[:, :, 1] = np.clip(img[:, :, 1] + 50, 0, 255)
                else:  # birds
                    img[:, :, 2] = np.clip(img[:, :, 2] + 50, 0, 255)

                pil_img = Image.fromarray(img)
                pil_img.save(os.path.join(dir_path, f"{i:04d}.jpg"))

    print(f"Created dataset in {base_dir}")
    for split in splits:
        for cls in classes:
            path = os.path.join(base_dir, split, cls)
            count = len(os.listdir(path))
            print(f"  {split}/{cls}: {count} images")

# Create the dataset
create_sample_dataset("sample_dataset")
# Output:
# Created dataset in sample_dataset
#   train/cats: 100 images
#   train/dogs: 100 images
#   train/birds: 100 images
#   val/cats: 20 images
#   val/dogs: 20 images
#   val/birds: 20 images
```

Loading with ImageFolder

```python
from torchvision import datasets, transforms

# Define transforms
train_transforms = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

val_transforms = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
```

```
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# Load datasets using ImageFolder
train_dataset = datasets.ImageFolder(
    root="sample_dataset/train",
    transform=train_transforms
)

val_dataset = datasets.ImageFolder(
    root="sample_dataset/val",
    transform=val_transforms
)

# Inspect the dataset
print(f"Number of training images: {len(train_dataset)}")
print(f"Number of validation images: {len(val_dataset)}")
print(f"Classes: {train_dataset.classes}")
print(f"Class to index: {train_dataset.class_to_idx}")

# Output:
# Number of training images: 300
# Number of validation images: 60
# Classes: ['birds', 'cats', 'dogs']
# Class to index: {'birds': 0, 'cats': 1, 'dogs': 2}
```

Line-by-line explanation:

- `datasets.ImageFolder(root=..., transform=...)`: Scans the directory, finds all images, and assigns labels based on subfolder names. The `transform` is applied to each image when loaded.
- `train_dataset.classes`: A list of class names, sorted alphabetically.
- `train_dataset.class_to_idx`: Maps class names to integer indices. These integers are the labels your model will predict.

Inspecting a Single Sample

```python
# Get one sample
image, label = train_dataset[0]

print(f"Image type: {type(image)}")
print(f"Image shape: {image.shape}")
print(f"Label: {label}")
print(f"Label name: {train_dataset.classes[label]}")
```

```
# Output:
# Image type: <class 'torch.Tensor'>
# Image shape: torch.Size([3, 64, 64])
# Label: 0
# Label name: birds
```

18.3 Creating Data Loaders

A DataLoader wraps a dataset and provides batching, shuffling, and parallel loading:

```python
from torch.utils.data import DataLoader

# Create data loaders
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,        # Shuffle training data each epoch
    num_workers=2,       # Parallel data loading
    pin_memory=True,     # Faster GPU transfer
)

val_loader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False,       # Do NOT shuffle validation data
    num_workers=2,
    pin_memory=True,
)

# Check batch shapes
for images, labels in train_loader:
    print(f"Batch images shape: {images.shape}")
    print(f"Batch labels shape: {labels.shape}")
    print(f"Batch labels: {labels[:10]}")
    break

# Output:
# Batch images shape: torch.Size([32, 3, 64, 64])
# Batch labels shape: torch.Size([32])
# Batch labels: tensor([2, 0, 1, 2, 0, 1, 1, 0, 2, 0])
```

```
DataLoader produces batches:

Single image: (3, 64, 64)   +  label: 1

DataLoader with batch_size=32:
```

```
Batch of images: (32, 3, 64, 64)  +  Batch of labels: (32,)
  ^                                     ^
  |                                     |
  32 images stacked                     32 integer labels
  along a new first dimension           one per image
```

Line-by-line explanation:

- `batch_size=32`: Groups 32 images into one tensor. This is the number of images processed simultaneously.
- `shuffle=True`: Randomizes the order of images each epoch. This prevents the model from memorizing the order of training data. Never shuffle validation data -- you want consistent evaluation.
- `pin_memory=True`: Allocates the batch tensors in page-locked (pinned) memory, which speeds up the transfer from CPU to GPU.

18.4 Building a CNN

Now let us build the actual model. A Convolutional Neural Network (CNN) is specifically designed to process images. It uses convolutional layers that slide small filters across the image to detect features like edges, textures, and shapes.

CNN Architecture Diagram

```
Input Image (3, 64, 64)
        |
  +-----v------+
  | Conv2d      |  32 filters, 3x3, detects edges
  | ReLU        |
  | MaxPool 2x2 |  Reduces size by half
  +-----+------+
        |  (32, 32, 32)
  +-----v------+
  | Conv2d      |  64 filters, 3x3, detects textures
  | ReLU        |
  | MaxPool 2x2 |  Reduces size by half
  +-----+------+
        |  (64, 16, 16)
  +-----v------+
  | Conv2d      |  128 filters, 3x3, detects patterns
  | ReLU        |
  | MaxPool 2x2 |  Reduces size by half
```

```
  +-----+------+
        |  (128, 8, 8)
  +-----v------+
  | Flatten     |  Reshape to 1D: 128*8*8 = 8192
  +-----+------+
        |  (8192,)
  +-----v------+
  | Linear(512) |  Fully connected
  | ReLU        |
  | Dropout(0.5)|  Prevent overfitting
  +-----+------+
        |  (512,)
  +-----v------+
  | Linear(3)   |  Output: one score per class
  +-----+------+
        |  (3,)
        v
  Class Scores (cats, dogs, birds)
```

The Model Code

```python
import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=3):
        super(SimpleCNN, self).__init__()

        # Convolutional layers (feature extraction)
        self.features = nn.Sequential(
            # Block 1: Input (3, 64, 64) -> Output (32, 32, 32)
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            # Block 2: Input (32, 32, 32) -> Output (64, 16, 16)
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            # Block 3: Input (64, 16, 16) -> Output (128, 8, 8)
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
        )

        # Classification layers (decision making)
        self.classifier = nn.Sequential(
            nn.Flatten(),                    # (128, 8, 8) -> (8192,)
            nn.Linear(128 * 8 * 8, 512),     # 8192 -> 512
```

```
            nn.ReLU(),
            nn.Dropout(0.5),                # Drop 50% during training
            nn.Linear(512, num_classes),      # 512 -> num_classes
        )

    def forward(self, x):
        x = self.features(x)      # Extract visual features
        x = self.classifier(x)    # Classify based on features
        return x

# Create the model
model = SimpleCNN(num_classes=3)

# Test with a dummy batch
dummy_input = torch.randn(4, 3, 64, 64)  # 4 images
output = model(dummy_input)
print(f"Input shape: {dummy_input.shape}")
print(f"Output shape: {output.shape}")
print(f"Output (class scores):\n{output}")

# Output:
# Input shape: torch.Size([4, 3, 64, 64])
# Output shape: torch.Size([4, 3])
# Output (class scores):
# tensor([[ 0.0234, -0.0156,  0.0089],
#         [ 0.0178, -0.0201,  0.0145],
#         [ 0.0312, -0.0098,  0.0067],
#         [ 0.0256, -0.0189,  0.0134]], grad_fn=<AddmmBackward0>)
```

Line-by-line explanation:

- `nn.Conv2d(3, 32, kernel_size=3, padding=1)`: A convolutional layer. Takes 3 input channels (RGB), produces 32 output channels (feature maps). Uses a 3x3 filter. `padding=1` adds zeros around the border so the output height and width stay the same.
- `nn.ReLU()`: Activation function. Replaces negative values with zero. Adds non-linearity so the model can learn complex patterns.
- `nn.MaxPool2d(2, 2)`: Takes the maximum value in each 2x2 region, reducing the size by half. This makes the model focus on the strongest features and reduces computation.
- `nn.Flatten()`: Reshapes the 3D feature maps into a 1D vector so it can be fed to fully connected layers.
- `nn.Linear(128 * 8 * 8, 512)`: A fully connected layer. Connects every input neuron to every output neuron. The input size (8192) comes from 128 channels times 8 height times 8 width.
- `nn.Dropout(0.5)`: During training, randomly sets 50% of neurons to zero. This forces the network to learn redundant representations and prevents overfitting.

How Convolution Works (Simplified)

```
A 3x3 convolutional filter slides across the image:

Image patch:          Filter:            Result:
+---+---+---+        +---+---+---+
| 1 | 2 | 3 |        | 1 | 0 |-1 |      1*1 + 2*0 + 3*(-1)
+---+---+---+   *    +---+---+---+   =   + 4*1 + 5*0 + 6*(-1)
| 4 | 5 | 6 |        | 1 | 0 |-1 |      + 7*1 + 8*0 + 9*(-1)
+---+---+---+        +---+---+---+      = 1-3+4-6+7-9 = -6
| 7 | 8 | 9 |        | 1 | 0 |-1 |
+---+---+---+        +---+---+---+

The filter slides to every position, producing a
new "feature map" that highlights certain patterns.

This particular filter detects vertical edges
(differences between left and right).
```

Counting Model Parameters

```python
def count_parameters(model):
    """Count total and trainable parameters."""
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters()
                    if p.requires_grad)
    print(f"Total parameters: {total:,}")
    print(f"Trainable parameters: {trainable:,}")
    return total

count_parameters(model)
# Output:
# Total parameters: 4,244,675
# Trainable parameters: 4,244,675
```

18.5 The Training Loop

The training loop is where the model actually learns. It repeats the same cycle: make predictions, calculate the error, update weights.

```
Training Loop (one epoch):

For each batch:
  1. Forward Pass:   predictions = model(images)
  2. Calculate Loss: loss = criterion(predictions, labels)
```

```
  3. Backward Pass:  loss.backward()  (compute gradients)
  4. Update Weights: optimizer.step() (adjust parameters)
  5. Reset:          optimizer.zero_grad()
```

Complete Training Code

```python
import torch
import torch.nn as nn
import torch.optim as optim
import time

def train_model(model, train_loader, val_loader, num_epochs=10,
                learning_rate=0.001):
    """Train the model and track metrics."""

    # Setup
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)
    print(f"Training on: {device}")

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    # Track history
    history = {
        "train_loss": [], "train_acc": [],
        "val_loss": [], "val_acc": []
    }

    for epoch in range(num_epochs):
        start_time = time.time()

        # === Training Phase ===
        model.train()   # Enable dropout, batch norm training mode
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)

            # Forward pass
            outputs = model(images)
            loss = criterion(outputs, labels)

            # Backward pass and optimize
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

```
            # Track metrics
            running_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        train_loss = running_loss / total
        train_acc = correct / total

        # === Validation Phase ===
        model.eval()    # Disable dropout, batch norm eval mode
        running_loss = 0.0
        correct = 0
        total = 0

        with torch.no_grad():  # No gradient computation needed
            for images, labels in val_loader:
                images = images.to(device)
                labels = labels.to(device)

                outputs = model(images)
                loss = criterion(outputs, labels)

                running_loss += loss.item() * images.size(0)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        val_loss = running_loss / total
        val_acc = correct / total

        # Save history
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        elapsed = time.time() - start_time
        print(f"Epoch [{epoch+1}/{num_epochs}] "
              f"Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f} Acc: {val_acc:.4f} | "
              f"Time: {elapsed:.1f}s")

    return history

# Train the model
model = SimpleCNN(num_classes=3)
history = train_model(model, train_loader, val_loader,
                      num_epochs=10, learning_rate=0.001)

# Output:
# Training on: cpu
# Epoch [1/10] Train Loss: 1.0892 Acc: 0.3867 | Val Loss: 1.0654 Acc: 0.4333 | Time: 0.8s
# Epoch [2/10] Train Loss: 1.0234 Acc: 0.5200 | Val Loss: 0.9876 Acc: 0.5667 | Time: 0.7s
```

```
# ...
# Epoch [10/10] Train Loss: 0.4521 Acc: 0.8533 | Val Loss: 0.5123 Acc: 0.8000 | Time: 0.7s
```

Line-by-line explanation:

- `model.train()`: Puts the model in training mode. This enables dropout and makes batch normalization use batch statistics.
- `model.eval()`: Puts the model in evaluation mode. Dropout is disabled, and batch normalization uses running statistics.
- `criterion = nn.CrossEntropyLoss()`: The loss function for classification. It compares predicted class scores to actual labels and returns a single number indicating how wrong the predictions are.
- `optimizer = optim.Adam(model.parameters(), lr=0.001)`: The Adam optimizer adjusts model weights to minimize the loss. Learning rate 0.001 is a good default.
- `optimizer.zero_grad()`: Clears old gradients. Without this, gradients accumulate across batches.
- `loss.backward()`: Calculates gradients -- how much each weight contributed to the error.
- `optimizer.step()`: Updates weights using the gradients.
- `torch.no_grad()`: Disables gradient tracking during validation. This saves memory and speeds up computation.
- `_, predicted = torch.max(outputs, 1)`: Gets the index of the highest score in each row. The _ discards the actual values; we only need the indices (which are the predicted classes).
- `loss.item() * images.size(0)`: Converts the average loss (per sample in the batch) to total loss for the batch, so we can compute the correct average over the entire dataset.

Plotting Training History

```python
import matplotlib.pyplot as plt

def plot_history(history):
    """Plot training and validation loss and accuracy."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Loss plot
    ax1.plot(history["train_loss"], label="Train Loss", marker="o")
    ax1.plot(history["val_loss"], label="Val Loss", marker="s")
    ax1.set_xlabel("Epoch")
    ax1.set_ylabel("Loss")
    ax1.set_title("Training and Validation Loss")
    ax1.legend()
```

```
    ax1.grid(True, alpha=0.3)

    # Accuracy plot
    ax2.plot(history["train_acc"], label="Train Accuracy", marker="o")
    ax2.plot(history["val_acc"], label="Val Accuracy", marker="s")
    ax2.set_xlabel("Epoch")
    ax2.set_ylabel("Accuracy")
    ax2.set_title("Training and Validation Accuracy")
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

plot_history(history)
```

Output: Two side-by-side plots. The left shows loss decreasing over epochs (both train and validation). The right shows accuracy increasing. If the gap between training and validation widens, the model is overfitting.

```
Reading the Training Curves:

Good training:              Overfitting:
Loss                        Loss
  |\\                          |\\
  | \\___train                | \\___train (keeps going down)
  |  \\__val                  |  \\__val (stops, then goes UP)
  +---------> Epoch           +---------> Epoch

  Train and val curves        Val curve diverges from train
  should decrease together.   = model is memorizing, not learning.
```

18.6 Evaluation

Overall Accuracy

```python
def evaluate(model, data_loader, device=None):
    """Calculate overall accuracy on a dataset."""
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available()
                              else "cpu")
```

```
    model = model.to(device)
    model.eval()

    correct = 0
    total = 0
    all_predictions = []
    all_labels = []

    with torch.no_grad():
        for images, labels in data_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

            all_predictions.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    accuracy = correct / total
    print(f"Overall Accuracy: {accuracy:.4f} ({correct}/{total})")

    return all_predictions, all_labels

predictions, labels = evaluate(model, val_loader)
# Output: Overall Accuracy: 0.8000 (48/60)
```

Confusion Matrix

A confusion matrix shows exactly which classes the model confuses with each other:

```python
import numpy as np
import matplotlib.pyplot as plt

def plot_confusion_matrix(predictions, labels, class_names):
    """Plot a confusion matrix."""
    num_classes = len(class_names)
    matrix = np.zeros((num_classes, num_classes), dtype=int)

    for true, pred in zip(labels, predictions):
        matrix[true][pred] += 1

    # Plot
    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(matrix, cmap="Blues")

    # Add text annotations
```

```
    for i in range(num_classes):
        for j in range(num_classes):
            color = "white" if matrix[i, j] > matrix.max() / 2 else "black"
            ax.text(j, i, str(matrix[i, j]),
                    ha="center", va="center", color=color, fontsize=14)

    # Labels
    ax.set_xticks(range(num_classes))
    ax.set_yticks(range(num_classes))
    ax.set_xticklabels(class_names)
    ax.set_yticklabels(class_names)
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("True Label")
    ax.set_title("Confusion Matrix")
    plt.colorbar(im)
    plt.tight_layout()
    plt.show()

    return matrix

class_names = ["birds", "cats", "dogs"]
matrix = plot_confusion_matrix(predictions, labels, class_names)

# Output: A heatmap showing prediction counts
```

```
Reading a Confusion Matrix:

              Predicted
              birds  cats  dogs
         +-----------------------+
birds    |   18  |   1   |   1  |  <- 18 correct, 2 wrong
True     +-----------------------+
cats     |    2  |   16  |   2  |  <- 16 correct, 4 wrong
         +-----------------------+
dogs     |    1  |   3   |   16 |  <- 16 correct, 4 wrong
         +-----------------------+

Diagonal = correct predictions (higher is better)
Off-diagonal = mistakes (lower is better)

Row 2, Column 3: The model predicted "dogs" 2 times
when the true label was "cats."
```

Per-Class Accuracy

```python
def per_class_accuracy(predictions, labels, class_names):
    """Calculate accuracy for each class."""
    predictions = np.array(predictions)
```

```
    labels = np.array(labels)

    print("Per-Class Accuracy:")
    print("-" * 40)

    for i, name in enumerate(class_names):
        class_mask = labels == i
        class_correct = (predictions[class_mask] == i).sum()
        class_total = class_mask.sum()
        acc = class_correct / class_total if class_total > 0 else 0

        print(f"  {name:10s}: {acc:.4f} ({class_correct}/{class_total})")

    print("-" * 40)

per_class_accuracy(predictions, labels, class_names)
# Output:
# Per-Class Accuracy:
# ----------------------------------------
#   birds     : 0.9000 (18/20)
#   cats      : 0.8000 (16/20)
#   dogs      : 0.8000 (16/20)
# ----------------------------------------
```

Precision, Recall, and F1 Score

```python
def classification_report(predictions, labels, class_names):
    """Calculate precision, recall, and F1 for each class."""
    predictions = np.array(predictions)
    labels = np.array(labels)

    print(f"{'Class':12s} {'Precision':>10s} {'Recall':>10s} {'F1':>10s} {'Support':>10...
    print("-" * 55)

    for i, name in enumerate(class_names):
        # True Positives: predicted i and was actually i
        tp = ((predictions == i) & (labels == i)).sum()
        # False Positives: predicted i but was not actually i
        fp = ((predictions == i) & (labels != i)).sum()
        # False Negatives: did not predict i but was actually i
        fn = ((predictions != i) & (labels == i)).sum()

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1 = (2 * precision * recall / (precision + recall)
              if (precision + recall) > 0 else 0)
        support = (labels == i).sum()

        print(f"{name:12s} {precision:10.4f} {recall:10.4f} "
              f"{f1:10.4f} {support:10d}")
```

```
    print("-" * 55)

classification_report(predictions, labels, class_names)
# Output:
# Class         Precision     Recall         F1    Support
# -------------------------------------------------------
# birds            0.8571     0.9000     0.8780         20
# cats             0.8000     0.8000     0.8000         20
# dogs             0.8421     0.8000     0.8205         20
# -------------------------------------------------------
```

```
Precision vs Recall:

Precision = "Of everything I PREDICTED as cats,
             how many were actually cats?"
           = TP / (TP + FP)

Recall    = "Of everything that WAS a cat,
             how many did I correctly identify?"
           = TP / (TP + FN)

F1 Score  = Harmonic mean of precision and recall
           = A single number balancing both

High precision, low recall: Model is careful but misses many
Low precision, high recall: Model catches most but has false alarms
```

18.7 Visualizing Predictions

Seeing what the model predicts on actual images helps you understand its behavior:

```python
import torch
import matplotlib.pyplot as plt
import numpy as np

def visualize_predictions(model, dataset, class_names, num_images=12,
                          device=None):
    """Display images with their predictions."""
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available()
                              else "cpu")

    model = model.to(device)
    model.eval()
```

```
    # ImageNet denormalization
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])

    fig, axes = plt.subplots(3, 4, figsize=(16, 12))
    axes = axes.flatten()

    # Pick random samples
    indices = np.random.choice(len(dataset), num_images, replace=False)

    for i, idx in enumerate(indices):
        image, true_label = dataset[idx]

        # Get prediction
        with torch.no_grad():
            output = model(image.unsqueeze(0).to(device))
            probabilities = torch.softmax(output, dim=1)
            confidence, predicted = torch.max(probabilities, 1)

        predicted = predicted.item()
        confidence = confidence.item()
        true_name = class_names[true_label]
        pred_name = class_names[predicted]

        # Denormalize for display
        img_display = image.permute(1, 2, 0).numpy()
        img_display = img_display * std + mean
        img_display = np.clip(img_display, 0, 1)

        # Plot
        axes[i].imshow(img_display)
        color = "green" if predicted == true_label else "red"
        axes[i].set_title(
            f"True: {true_name}\nPred: {pred_name} ({confidence:.1%})",
            color=color, fontsize=10
        )
        axes[i].axis("off")

    plt.suptitle("Model Predictions (Green=Correct, Red=Wrong)",
                 fontsize=14)
    plt.tight_layout()
    plt.show()

visualize_predictions(model, val_dataset, class_names)
```

Output: A 3x4 grid of images. Each image shows the true label and predicted label with confidence percentage. Correct predictions have green titles; incorrect predictions have red titles.

Line-by-line explanation:

- `image.unsqueeze(0)`: Adds a batch dimension. The model expects (batch, C, H, W), but a single image is (C, H, W). `unsqueeze(0)` adds a dimension at position 0, making it (1, C, H, W).
- `torch.softmax(output, dim=1)`: Converts raw scores to probabilities that sum to 1. This lets us express confidence as a percentage.
- `torch.max(probabilities, 1)`: Returns the highest probability and its index (the predicted class).
- The denormalization reverses the ImageNet normalization so the image displays with correct colors.

18.8 Saving and Loading the Model

```python
import torch

# Save the entire model state
torch.save({
    "model_state_dict": model.state_dict(),
    "class_names": class_names,
    "num_classes": len(class_names),
    "history": history,
}, "cnn_classifier.pth")

print("Model saved!")

# Load the model later
checkpoint = torch.load("cnn_classifier.pth")

loaded_model = SimpleCNN(num_classes=checkpoint["num_classes"])
loaded_model.load_state_dict(checkpoint["model_state_dict"])
loaded_model.eval()

print(f"Loaded model for classes: {checkpoint['class_names']}")
# Output: Loaded model for classes: ['birds', 'cats', 'dogs']
```

Line-by-line explanation:

- `model.state_dict()`: Returns all learnable parameters (weights and biases) as a dictionary.
- We save extra information (class names, history) alongside the model so we have everything needed to use it later.
- `loaded_model.load_state_dict(...)`: Loads the saved parameters into a fresh model instance.

- `loaded_model.eval()`: Puts the model in evaluation mode (no dropout, etc.).

Common Mistakes

1. Forgetting `model.train()` and `model.eval()`. Dropout and batch normalization behave differently during training and evaluation. Without switching modes, validation metrics will be noisy and inconsistent.
2. Not using `torch.no_grad()` during validation. Without it, PyTorch still builds the computation graph and tracks gradients, wasting memory and slowing down evaluation.
3. Shuffling the validation set. This does not cause errors, but it can make debugging harder and wastes time. Only shuffle training data.
4. Wrong input size for the linear layer. If you change the image size or the number of convolutional layers, the flattened size changes. Always calculate `channels * height * width` after all conv and pool layers.
5. Forgetting `optimizer.zero_grad()`. Without zeroing gradients, they accumulate across batches, causing the model to learn from stale information.
6. Using `accuracy` alone for imbalanced datasets. If 95% of your images are cats, a model that always says "cat" gets 95% accuracy but is useless. Use precision, recall, and the confusion matrix.

Best Practices

1. Start simple. Begin with a small CNN and a small subset of data. Make sure the pipeline works end-to-end before scaling up.
2. Monitor both training and validation metrics. A widening gap between them signals overfitting. Add dropout, data augmentation, or more data.
3. Save checkpoints regularly. Save the model after each epoch (or at least the best one). Training can crash, and you do not want to lose hours of work.
4. Visualize predictions. Looking at what the model gets wrong often reveals problems (wrong labels, bad preprocessing, confusing classes).
5. Use a learning rate scheduler. Reducing the learning rate as training progresses often improves final accuracy.

Quick Summary

Image classification with CNNs follows a standard pipeline: organize data into class folders, load with `ImageFolder` and `DataLoader`, build a CNN model, train with a forward-backward loop, and evaluate with accuracy, confusion matrix, and per-class metrics. CNNs use convolutional layers to detect visual features and pooling layers to reduce spatial dimensions. The training loop consists of forward pass, loss calculation, backward pass, and weight update. Always switch between `model.train()` and `model.eval()` modes. Visualize predictions to understand model behavior.

Key Points

- `ImageFolder` automatically assigns labels based on subfolder names. The folder name IS the label.
- A CNN consists of feature extraction layers (conv + pool) followed by classification layers (fully connected).
- `Conv2d` slides filters across images to detect features. `MaxPool2d` reduces spatial dimensions by taking the maximum in each region.
- `model.train()` enables dropout and batch norm training. `model.eval()` disables them for evaluation.
- Always use `torch.no_grad()` during validation to save memory and computation.
- The confusion matrix shows exactly which classes the model confuses with each other.
- Per-class accuracy, precision, and recall reveal performance differences across classes that overall accuracy hides.

Practice Questions

1. Why do we use separate transforms for training and validation? What could go wrong if we used training transforms (with random augmentation) during validation?
2. An image has shape `(3, 64, 64)` and goes through a `Conv2d(3, 32, kernel_size=3, padding=1)` followed by `MaxPool2d(2, 2)`. What is the output shape? Explain each dimension.
3. What does `Dropout(0.5)` do during training? What does it do during evaluation? Why is this difference important?

4. You have a dataset with 1000 dog images and 50 cat images. Your model achieves 95% accuracy. Is this good? What metrics should you look at instead?
5. Why do we call `optimizer.zero_grad()` before `loss.backward()`? What happens if we forget?

Exercises

Exercise 1: Build and Train a CNN

Using the pipeline from this chapter:

1. Create a synthetic dataset with 4 classes (or use a real dataset like CIFAR-10)
2. Build a CNN with at least 3 convolutional blocks
3. Train for 20 epochs
4. Plot the training history
5. Print the confusion matrix and per-class accuracy

Exercise 2: Experiment with Architecture

Modify the `SimpleCNN` model:

1. Add a 4th convolutional block with 256 filters
2. Add batch normalization (`nn.BatchNorm2d`) after each conv layer
3. Try different dropout rates (0.25 vs 0.5 vs 0.75)
4. Compare the training curves and final accuracy for each variant

Exercise 3: Learning Rate Experiment

Train the same model three times with different learning rates:

1. `lr = 0.01` (too high?)
2. `lr = 0.001` (standard)
3. `lr = 0.0001` (too low?)

Plot all three training curves on the same graph. Which learning rate gives the best result, and why?

What Is Next?

Training a CNN from scratch works, but it requires a lot of data and time. In the next chapter, you will learn about Transfer Learning -- how to take a model that someone else already trained on millions of images (like ImageNet) and adapt it to your specific task. This dramatically reduces the amount of data and training time you need, often producing much better results than training from scratch.

Chapter 19: Transfer Learning for Vision

What You Will Learn

In this chapter, you will learn:

- What transfer learning is and why it is so powerful
- The difference between feature extraction and fine-tuning
- How to use pretrained models (ResNet18, ResNet50) from torchvision
- How to replace the final classification layer for your own task
- How to freeze and unfreeze layers strategically
- Learning rate strategies for transfer learning
- How to build a complete custom dataset classifier using transfer learning

Why This Chapter Matters

In the previous chapter, you built a CNN from scratch. It worked, but training from scratch has serious limitations:

- You need a LOT of data. Thousands to millions of labeled images.
- Training takes a LONG time. Hours or days on a GPU.
- Results are often mediocre. Your small model cannot learn all the visual patterns that exist in the world.

Transfer learning solves all three problems. Instead of starting from zero, you start with a model that already understands images -- it knows what edges, textures, shapes, and objects look like because it was trained on 1.2 million images from ImageNet.

Think of it like hiring an experienced chef versus training someone who has never cooked. The experienced chef already knows how to chop, saute, and season. You just need to teach

them your specific recipes. Transfer learning is the same idea: take a model that already "sees," and teach it your specific task.

In practice, transfer learning is used in the vast majority of real-world computer vision projects. It is not an advanced technique -- it is the default approach.

19.1 What Is Transfer Learning?

Transfer learning means taking knowledge learned from one task and applying it to a different but related task.

```
Training from Scratch:

  Random weights -> Train on YOUR data -> Your model
  (knows nothing)   (needs lots of data)  (limited knowledge)

Transfer Learning:

  ImageNet weights -> Adapt to YOUR data -> Your model
  (knows vision!)    (needs less data)     (leverages millions
                                            of images worth
                                            of knowledge)
```

What Does a Pretrained Model Already Know?

A model trained on ImageNet has learned a hierarchy of visual features:

```
Layer Depth     What It Learned           Examples
------------------------------------------------------------
Early layers    Edges and textures        |||  ///  ---
(Layer 1-2)                               Lines, gradients

Middle layers   Parts and patterns        Eyes, wheels, petals
(Layer 3-5)                               Stripes, circles

Deep layers     Object parts              Cat face, car door
(Layer 6-10)                              Wing, keyboard

Final layers    Whole objects              Cat, car, bird
(Layer 11+)     (specific to ImageNet)     (1000 ImageNet classes)
```

Key insight: The early and middle layers learn universal visual features that apply to almost any image task. Only the final layers are specific to the original task (ImageNet's 1000

classes).

```
Transfer Learning Strategy:

Pretrained Model:
+-------------------+-------------------+-----------+
| Universal         | General           | Specific  |
| Features          | Patterns          | to        |
| (edges, colors)   | (shapes, parts)   | ImageNet  |
+-------------------+-------------------+-----------+
  KEEP THESE          KEEP (or fine-tune)  REPLACE!

Your Model:
+-------------------+-------------------+-----------+
| Universal         | General           | Specific  |
| Features          | Patterns          | to YOUR   |
| (edges, colors)   | (shapes, parts)   | task      |
+-------------------+-------------------+-----------+
  Same as before      Same (or adjusted)  New layer!
```

19.2 Feature Extraction vs. Fine-Tuning

There are two main approaches to transfer learning:

Approach 1: Feature Extraction

Use the pretrained model as a fixed feature extractor. Freeze all layers and only train a new classification head.

```
Feature Extraction:

Pretrained CNN (frozen - not updated)     New Head (trained)
+-----+-----+-----+-----+-----+         +--------+
|  C1 |  C2 |  C3 |  C4 |  C5 |  --->   | Linear |
| FRZ | FRZ | FRZ | FRZ | FRZ |         | (new)  |
+-----+-----+-----+-----+-----+         +--------+
                                              |
  These layers DO NOT change                  v
  during training. They act as           Predictions
  a feature extraction machine.          (your classes)

Pros: Fast, works with very little data (even 50 images per class)
Cons: Cannot adapt features to your specific domain
```

Approach 2: Fine-Tuning

Start with pretrained weights, then train (fine-tune) some or all layers on your data.

```
Fine-Tuning:

Pretrained CNN                            New Head
+-----+-----+-----+-----+-----+         +--------+
|  C1 |  C2 |  C3 |  C4 |  C5 |  --->   | Linear |
| FRZ | FRZ | UPD | UPD | UPD |         | (new)  |
+-----+-----+-----+-----+-----+         +--------+
                                                |
  Early layers frozen                           v
  Later layers updated                   Predictions
  with a small learning rate

Pros: Better accuracy, adapts features to your domain
Cons: Needs more data, risk of overfitting, slower training
```

When to Use Which?

```
Decision Guide:

                    Little Data              Lots of Data
                    (< 1000 images)          (> 5000 images)
                +--------------------+--------------------+
Similar to      | Feature Extraction | Fine-tune last     |
ImageNet        | (freeze all)       | few layers         |
(natural photos)|                    |                    |
                +--------------------+--------------------+
Different from  | Feature Extraction | Fine-tune most     |
ImageNet        | (might struggle)   | or all layers      |
(medical, sat.) |                    |                    |
                +--------------------+--------------------+
```

19.3 Loading Pretrained Models

PyTorch's `torchvision.models` provides many pretrained architectures:

```python
import torchvision.models as models
import torch
```

```
# Load a pretrained ResNet18
model = models.resnet18(weights="IMAGENET1K_V1")

# Check the model structure
print(model)
```

Partial output:

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1)
  (layer1): Sequential(...)
  (layer2): Sequential(...)
  (layer3): Sequential(...)
  (layer4): Sequential(...)
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=1000, bias=True)  <-- THIS!
)
```

Key observation: The last layer `(fc)` has `out_features=1000` because ImageNet has 1000 classes. We need to replace this for our task.

Available Pretrained Models

```python
# Common pretrained models
resnet18 = models.resnet18(weights="IMAGENET1K_V1")    # 11.7M params
resnet50 = models.resnet50(weights="IMAGENET1K_V1")    # 25.6M params
vgg16 = models.vgg16(weights="IMAGENET1K_V1")          # 138M params
mobilenet_v2 = models.mobilenet_v2(weights="IMAGENET1K_V1")  # 3.4M params

# Check parameters
for name, m in [("ResNet18", resnet18), ("ResNet50", resnet50),
                ("VGG16", vgg16), ("MobileNetV2", mobilenet_v2)]:
    params = sum(p.numel() for p in m.parameters())
    print(f"{name}: {params/1e6:.1f}M parameters")

# Output:
# ResNet18: 11.7M parameters
# ResNet50: 25.6M parameters
# VGG16: 138.4M parameters
# MobileNetV2: 3.5M parameters
```

```
Choosing a Model:
```

```
Model         Params    Accuracy   Speed    Best For
-----------------------------------------------------------
MobileNetV2   3.5M      71.9%      Fast     Mobile/edge devices
ResNet18      11.7M     69.8%      Fast     Quick experiments
ResNet50      25.6M     76.1%      Medium   Good balance
VGG16         138.4M    71.6%      Slow     Legacy, simple
EfficientNet  5.3M      77.1%      Medium   Best accuracy/size

For most projects, start with ResNet18 (fast iteration)
and move to ResNet50 or EfficientNet for production.
```

19.4 Replacing the Final Layer

The final layer must be replaced to match your number of classes:

ResNet

```python
import torch
import torch.nn as nn
import torchvision.models as models

# Load pretrained ResNet18
model = models.resnet18(weights="IMAGENET1K_V1")

# Check the original final layer
print(f"Original fc layer: {model.fc}")
# Output: Original fc layer: Linear(in_features=512, out_features=1000, bias=True)

# Replace with your number of classes
num_classes = 5   # e.g., 5 types of flowers

# Get the number of input features
num_features = model.fc.in_features
print(f"Number of input features: {num_features}")
# Output: Number of input features: 512

# Replace the final layer
model.fc = nn.Linear(num_features, num_classes)

print(f"New fc layer: {model.fc}")
# Output: New fc layer: Linear(in_features=512, out_features=5, bias=True)

# Test with a dummy input
dummy = torch.randn(2, 3, 224, 224)
output = model(dummy)
```

```
print(f"Output shape: {output.shape}")
# Output: Output shape: torch.Size([2, 5])
```

Line-by-line explanation:

- `model.fc.in_features`: Gets the number of features the final layer expects as input. This is 512 for ResNet18 and 2048 for ResNet50.
- `model.fc = nn.Linear(num_features, num_classes)`: Replaces the final layer. The new layer has random weights -- this is the only part that needs training from scratch.

ResNet50 (Different Input Features)

```python
# ResNet50 has more features in the final layer
model50 = models.resnet50(weights="IMAGENET1K_V1")
print(f"ResNet50 fc input features: {model50.fc.in_features}")
# Output: ResNet50 fc input features: 2048

model50.fc = nn.Linear(2048, num_classes)
print(f"New fc layer: {model50.fc}")
# Output: New fc layer: Linear(in_features=2048, out_features=5, bias=True)
```

Adding a More Complex Head

For better performance, you can replace the single linear layer with a small network:

```python
num_features = model.fc.in_features

model.fc = nn.Sequential(
    nn.Linear(num_features, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, num_classes)
)

print(f"New classifier head:\n{model.fc}")
# Output:
# New classifier head:
# Sequential(
#   (0): Linear(in_features=512, out_features=256, bias=True)
#   (1): ReLU()
#   (2): Dropout(p=0.5, inplace=False)
#   (3): Linear(in_features=256, out_features=5, bias=True)
# )
```

19.5 Freezing and Unfreezing Layers

Freezing a layer means preventing its weights from being updated during training. You do this by setting `requires_grad = False` on its parameters.

Freezing All Layers (Feature Extraction)

```python
import torchvision.models as models
import torch.nn as nn

# Load pretrained model
model = models.resnet18(weights="IMAGENET1K_V1")

# Freeze ALL parameters
for param in model.parameters():
    param.requires_grad = False

# Replace the final layer (new layer has requires_grad=True by default)
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 5)

# Verify: count trainable vs frozen parameters
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
frozen = sum(p.numel() for p in model.parameters() if not p.requires_grad)
total = trainable + frozen

print(f"Trainable parameters: {trainable:,}")
print(f"Frozen parameters: {frozen:,}")
print(f"Total parameters: {total:,}")
print(f"Trainable: {trainable/total*100:.1f}%")

# Output:
# Trainable parameters: 2,565
# Frozen parameters: 11,176,512
# Total parameters: 11,179,077
# Trainable: 0.0%
```

Line-by-line explanation:

- `param.requires_grad = False`: Tells PyTorch not to compute gradients for this parameter. No gradients means no weight updates.
- The new `nn.Linear` layer has `requires_grad=True` by default, so it WILL be trained.
- Only 2,565 out of 11 million parameters will be updated. This is extremely fast to train.

Freezing Early Layers, Unfreezing Later Layers (Fine-Tuning)

```python
import torchvision.models as models
import torch.nn as nn

model = models.resnet18(weights="IMAGENET1K_V1")

# First, freeze everything
for param in model.parameters():
    param.requires_grad = False

# Unfreeze layer4 (the last conv block)
for param in model.layer4.parameters():
    param.requires_grad = True

# Replace and unfreeze the final layer
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 5)
# (model.fc parameters have requires_grad=True by default)

# Check what is trainable
print("Layer-by-layer trainability:")
for name, param in model.named_parameters():
    status = "TRAINABLE" if param.requires_grad else "frozen"
    print(f"  {name:40s} {str(param.shape):20s} {status}")

# Count
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"\nTrainable: {trainable:,} / {total:,} ({trainable/total*100:.1f}%)")

# Output (abbreviated):
#   conv1.weight                              torch.Size([64, 3, 7, 7])  frozen
#   ...
#   layer3.1.conv2.weight                     torch.Size([256, 256, 3, 3])  frozen
#   layer4.0.conv1.weight                     torch.Size([512, 256, 3, 3])  TRAINABLE
#   layer4.0.conv2.weight                     torch.Size([512, 512, 3, 3])  TRAINABLE
#   ...
#   fc.weight                                 torch.Size([5, 512])  TRAINABLE
#   fc.bias                                   torch.Size([5])  TRAINABLE
# Trainable: 2,621,957 / 11,179,077 (23.5%)
```

```
Freezing Strategy:

Freeze everything, unfreeze last block:
+-----+-----+-----+-----+-----+-----+
| C1  | C2  | L1  | L2  | L3  | L4  |  FC
| FRZ | FRZ | FRZ | FRZ | FRZ | UPD |  UPD
+-----+-----+-----+-----+-----+-----+

The more layers you unfreeze:
  - More flexibility to adapt to your task
  - But more risk of overfitting
  - And slower training
```

```
Start frozen, gradually unfreeze if needed.
```

Gradual Unfreezing

A popular technique is to gradually unfreeze layers during training:

```python
def unfreeze_model_gradually(model, epoch):
    """Unfreeze more layers as training progresses."""
    if epoch == 0:
        # Epoch 0: Only train fc layer (everything else frozen)
        for param in model.parameters():
            param.requires_grad = False
        for param in model.fc.parameters():
            param.requires_grad = True

    elif epoch == 3:
        # Epoch 3: Also unfreeze layer4
        for param in model.layer4.parameters():
            param.requires_grad = True

    elif epoch == 6:
        # Epoch 6: Also unfreeze layer3
        for param in model.layer3.parameters():
            param.requires_grad = True

    elif epoch == 9:
        # Epoch 9: Unfreeze everything
        for param in model.parameters():
            param.requires_grad = True

    trainable = sum(p.numel() for p in model.parameters()
                    if p.requires_grad)
    print(f"Epoch {epoch}: {trainable:,} trainable parameters")
```

19.6 Learning Rate Strategies

Transfer learning requires careful learning rate management. The pretrained layers already have good weights -- you do not want to destroy them with large updates.

Strategy 1: Small Learning Rate for Everything

```python
import torch.optim as optim

# Use a learning rate 10x smaller than training from scratch
optimizer = optim.Adam(model.parameters(), lr=0.0001)
```

Strategy 2: Discriminative Learning Rates

Use different learning rates for different parts of the model. The pretrained layers get a tiny learning rate; the new layers get a larger one.

```python
import torch.optim as optim

# Group parameters with different learning rates
param_groups = [
    # Pretrained layers: very small learning rate
    {
        "params": [p for name, p in model.named_parameters()
                   if "fc" not in name and p.requires_grad],
        "lr": 0.00001   # 10x smaller
    },
    # New classification head: normal learning rate
    {
        "params": model.fc.parameters(),
        "lr": 0.001     # Standard learning rate
    },
]

optimizer = optim.Adam(param_groups)

# Verify
for i, group in enumerate(optimizer.param_groups):
    num_params = sum(p.numel() for p in group["params"])
    print(f"Group {i}: lr={group['lr']}, params={num_params:,}")

# Output:
# Group 0: lr=1e-05, params=2,619,392
# Group 1: lr=0.001, params=2,565
```

```
Discriminative Learning Rates:

Pretrained layers:  lr = 0.00001 (tiny adjustments)
  +-----+-----+-----+-----+-----+
  |     |     |     |     |     |
  | barely change   | slightly  |
  +-----+-----+-----+-----+-----+

New head:           lr = 0.001  (normal updates)
  +--------+
  |        |
```

```
  | learns |
  | fast   |
  +--------+

This prevents destroying the pretrained knowledge
while allowing the new head to learn quickly.
```

Strategy 3: Learning Rate Scheduler

Reduce the learning rate over time for better convergence:

```python
import torch.optim as optim
from torch.optim import lr_scheduler

optimizer = optim.Adam(model.parameters(), lr=0.001)

# Reduce lr by 0.1 every 7 epochs
scheduler = lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

# In your training loop:
# for epoch in range(num_epochs):
#     train_one_epoch(...)
#     scheduler.step()  # <-- Call after each epoch
#     print(f"Current lr: {scheduler.get_last_lr()}")

# Alternative: Reduce when validation loss stops improving
scheduler = lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",        # Monitor a metric that should decrease
    factor=0.1,        # Multiply lr by 0.1 when triggered
    patience=3,        # Wait 3 epochs of no improvement
    verbose=True       # Print a message when lr changes
)

# Usage in training loop:
# scheduler.step(val_loss)  # Pass the metric to monitor
```

19.7 Complete Transfer Learning Pipeline

Here is a complete, production-ready transfer learning pipeline:

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
import time
import copy
import numpy as np
import matplotlib.pyplot as plt

# ============================================
# Step 1: Data Preparation
# ============================================

train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225]),
])

val_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225]),
])

# Load datasets
train_dataset = datasets.ImageFolder("dataset/train",
                                     transform=train_transforms)
val_dataset = datasets.ImageFolder("dataset/val",
                                   transform=val_transforms)

train_loader = DataLoader(train_dataset, batch_size=32,
                          shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=32,
                        shuffle=False, num_workers=4)

class_names = train_dataset.classes
num_classes = len(class_names)
print(f"Classes: {class_names}")
print(f"Training images: {len(train_dataset)}")
print(f"Validation images: {len(val_dataset)}")

# ============================================
# Step 2: Build Transfer Learning Model
# ============================================

def create_transfer_model(num_classes, freeze_backbone=True):
    """Create a ResNet18 model for transfer learning."""
```

```
    # Load pretrained ResNet18
    model = models.resnet18(weights="IMAGENET1K_V1")

    # Optionally freeze the backbone
    if freeze_backbone:
        for param in model.parameters():
            param.requires_grad = False

    # Replace the final layer
    num_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Linear(num_features, 256),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(256, num_classes)
    )

    return model

model = create_transfer_model(num_classes, freeze_backbone=True)

# Count parameters
trainable = sum(p.numel() for p in model.parameters()
                if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"\nTrainable: {trainable:,} / {total:,} "
      f"({trainable/total*100:.2f}%)")

# ============================================
# Step 3: Training
# ============================================

def train_transfer_model(model, train_loader, val_loader,
                         num_epochs=15, lr=0.001):
    """Train with early stopping and best model tracking."""

    device = torch.device("cuda" if torch.cuda.is_available()
                          else "cpu")
    model = model.to(device)
    print(f"Training on: {device}")

    # Only optimize parameters that require gradients
    params_to_optimize = [p for p in model.parameters()
                          if p.requires_grad]
    print(f"Optimizing {len(params_to_optimize)} parameter groups")

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(params_to_optimize, lr=lr)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="min", factor=0.1, patience=3, verbose=True
    )

    # Track best model
    best_val_acc = 0.0
    best_model_weights = copy.deepcopy(model.state_dict())
```

```
    history = {"train_loss": [], "train_acc": [],
               "val_loss": [], "val_acc": []}

    for epoch in range(num_epochs):
        start_time = time.time()

        # --- Training ---
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        train_loss = running_loss / total
        train_acc = correct / total

        # --- Validation ---
        model.eval()
        running_loss = 0.0
        correct = 0
        total = 0

        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)

                outputs = model(images)
                loss = criterion(outputs, labels)

                running_loss += loss.item() * images.size(0)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        val_loss = running_loss / total
        val_acc = correct / total

        # Update scheduler
        scheduler.step(val_loss)

        # Save best model
        if val_acc > best_val_acc:
```

```
            best_val_acc = val_acc
            best_model_weights = copy.deepcopy(model.state_dict())

        # Record history
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        elapsed = time.time() - start_time
        print(f"Epoch [{epoch+1}/{num_epochs}] "
              f"Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f} Acc: {val_acc:.4f} | "
              f"Time: {elapsed:.1f}s")

    # Load best model weights
    model.load_state_dict(best_model_weights)
    print(f"\nBest Validation Accuracy: {best_val_acc:.4f}")

    return model, history

# Train
model, history = train_transfer_model(
    model, train_loader, val_loader,
    num_epochs=15, lr=0.001
)

# Output:
# Training on: cuda
# Optimizing 4 parameter groups
# Epoch [1/15] Train Loss: 1.2345 Acc: 0.5200 | Val Loss: 0.8765 Acc: 0.7200 | Time: 2.3s
# Epoch [2/15] Train Loss: 0.7654 Acc: 0.7800 | Val Loss: 0.5432 Acc: 0.8500 | Time: 2.1s
# ...
# Epoch [15/15] Train Loss: 0.1234 Acc: 0.9700 | Val Loss: 0.2345 Acc: 0.9400 | Time: 2.2s
# Best Validation Accuracy: 0.9500
```

Line-by-line explanation:

- `params_to_optimize = [p for p in model.parameters() if p.requires_grad]`: Only includes parameters that are not frozen. This is crucial -- passing frozen parameters to the optimizer wastes memory and can cause subtle bugs.
- `copy.deepcopy(model.state_dict())`: Saves a deep copy of the best model weights. This way, even if later epochs make the model worse, we keep the best version.
- `scheduler.step(val_loss)`: Tells the scheduler the current validation loss. If it has not improved for `patience` epochs, the learning rate is reduced.

Step 4: Fine-Tuning After Feature Extraction

A common workflow is to first train with frozen backbone (feature extraction), then unfreeze and fine-tune:

```python
# Phase 1: Feature extraction (already done above)
# model has been trained with frozen backbone

# Phase 2: Unfreeze and fine-tune
print("\n=== Phase 2: Fine-tuning ===")

# Unfreeze the last two residual blocks
for param in model.layer3.parameters():
    param.requires_grad = True
for param in model.layer4.parameters():
    param.requires_grad = True

trainable = sum(p.numel() for p in model.parameters()
                if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"Trainable: {trainable:,} / {total:,} ({trainable/total*100:.1f}%)")

# Fine-tune with a smaller learning rate
model, history_ft = train_transfer_model(
    model, train_loader, val_loader,
    num_epochs=10,
    lr=0.0001   # 10x smaller than before!
)
```

```
Two-Phase Training:

Phase 1: Feature Extraction
  - Freeze backbone, train only new head
  - Learning rate: 0.001
  - Epochs: 10-15
  - Purpose: Get the new head working

Phase 2: Fine-Tuning
  - Unfreeze some backbone layers
  - Learning rate: 0.0001 (smaller!)
  - Epochs: 5-10
  - Purpose: Adapt features to your domain
```

19.8 Comparing From-Scratch vs. Transfer Learning

Let us compare the results side by side:

```python
import matplotlib.pyplot as plt

def compare_approaches(scratch_history, transfer_history):
    """Compare training from scratch vs transfer learning."""

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Loss comparison
    ax1.plot(scratch_history["val_loss"], label="From Scratch",
             linestyle="--", marker="o")
    ax1.plot(transfer_history["val_loss"], label="Transfer Learning",
             marker="s")
    ax1.set_xlabel("Epoch")
    ax1.set_ylabel("Validation Loss")
    ax1.set_title("Validation Loss Comparison")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Accuracy comparison
    ax2.plot(scratch_history["val_acc"], label="From Scratch",
             linestyle="--", marker="o")
    ax2.plot(transfer_history["val_acc"], label="Transfer Learning",
             marker="s")
    ax2.set_xlabel("Epoch")
    ax2.set_ylabel("Validation Accuracy")
    ax2.set_title("Validation Accuracy Comparison")
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# compare_approaches(scratch_history, transfer_history)
```

```
Typical Results:

                      From Scratch    Transfer Learning
------------------------------------------------------------
Training time         Long (hours)   Short (minutes)
Data needed           Thousands+     Hundreds
Best val accuracy     ~70-80%        ~90-95%
Epochs to converge    50+            10-15
```

19.9 Using ResNet50 for Higher Accuracy

For better accuracy, use a deeper model like ResNet50:

```python
import torch
import torch.nn as nn
import torchvision.models as models

def create_resnet50_model(num_classes, freeze_backbone=True):
    """Create a ResNet50 transfer learning model."""

    model = models.resnet50(weights="IMAGENET1K_V1")

    if freeze_backbone:
        for param in model.parameters():
            param.requires_grad = False

    # ResNet50 fc layer has 2048 input features (vs 512 for ResNet18)
    num_features = model.fc.in_features
    print(f"ResNet50 fc input features: {num_features}")

    model.fc = nn.Sequential(
        nn.Linear(num_features, 512),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(512, num_classes)
    )

    trainable = sum(p.numel() for p in model.parameters()
                    if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    print(f"Trainable: {trainable:,} / {total:,}")

    return model

model_r50 = create_resnet50_model(num_classes=5, freeze_backbone=True)
# Output:
# ResNet50 fc input features: 2048
# Trainable: 1,049,093 / 24,606,533
```

19.10 Practical Tips

Tip 1: Check If Transfer Learning Is Helping

```python
import torch
import torchvision.models as models
import torch.nn as nn

# Model A: Random weights (no transfer learning)
```

```
model_random = models.resnet18(weights=None)  # No pretrained weights
model_random.fc = nn.Linear(512, num_classes)

# Model B: Pretrained weights (transfer learning)
model_pretrained = models.resnet18(weights="IMAGENET1K_V1")
model_pretrained.fc = nn.Linear(512, num_classes)

# Train both and compare. If pretrained is not better,
# your data might be too different from ImageNet.
```

Tip 2: Handling Small Datasets

```python
# For very small datasets (< 100 images per class):

# 1. Use aggressive augmentation
aggressive_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.5, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),        # If orientation does not matter
    transforms.RandomRotation(30),
    transforms.ColorJitter(0.3, 0.3, 0.3, 0.15),
    transforms.RandomGrayscale(p=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225]),
])

# 2. Use feature extraction (freeze everything)
# 3. Use a simpler head (fewer parameters = less overfitting)
model.fc = nn.Linear(num_features, num_classes)  # No hidden layers
```

Tip 3: Saving the Complete Pipeline

```python
import torch

# Save everything needed to recreate the model
torch.save({
    "model_name": "resnet18",
    "num_classes": num_classes,
    "class_names": class_names,
    "model_state_dict": model.state_dict(),
    "freeze_backbone": False,
    "best_val_acc": best_val_acc,
}, "transfer_model.pth")

# Load and recreate
```

```
checkpoint = torch.load("transfer_model.pth")
model = create_transfer_model(
    checkpoint["num_classes"],
    freeze_backbone=checkpoint["freeze_backbone"]
)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

print(f"Loaded model for: {checkpoint['class_names']}")
print(f"Best accuracy: {checkpoint['best_val_acc']:.4f}")
```

Common Mistakes

1. Not freezing pretrained layers during feature extraction. If you train all 11 million parameters on 200 images, the model will massively overfit and forget everything it learned from ImageNet.
2. Using too high a learning rate for fine-tuning. A learning rate of 0.01 will destroy the pretrained weights in a few batches. Use 0.0001 or smaller for fine-tuning pretrained layers.
3. Forgetting to only optimize unfrozen parameters. Passing frozen parameters to the optimizer does not cause errors, but it wastes memory. Filter with `[p for p in model.parameters() if p.requires_grad]`.
4. Not using ImageNet normalization. Pretrained models expect input normalized with ImageNet statistics. Using different normalization (or none) throws off all the learned features.
5. Using training transforms during inference. Random augmentations during prediction make results non-deterministic and often worse. Always use deterministic validation transforms.

Best Practices

1. Start with feature extraction (frozen backbone). If accuracy is not good enough, switch to fine-tuning.
2. Use discriminative learning rates. Smaller learning rate for pretrained layers, larger for new layers.

3. Monitor for overfitting carefully. Transfer learning models can overfit quickly on small datasets because they are so powerful. Use dropout, early stopping, and data augmentation.
4. Save the best model based on validation accuracy, not the final epoch's model. The best epoch is often not the last one.
5. Start with ResNet18 for prototyping, then try ResNet50 or EfficientNet for production. Bigger models are slower but more accurate.

Quick Summary

Transfer learning takes a model pretrained on ImageNet and adapts it to your task. Feature extraction freezes all pretrained layers and only trains a new classification head -- fast and works with little data. Fine-tuning unfreezes some pretrained layers and updates them with a small learning rate -- slower but more accurate. Replace the model's final layer to match your number of classes. Use discriminative learning rates (small for pretrained layers, larger for new layers). A two-phase approach (feature extraction first, then fine-tuning) often gives the best results.

Key Points

- Transfer learning reuses knowledge from models trained on millions of images, dramatically reducing data and time requirements.
- Feature extraction (frozen backbone) trains only the new head. Fine-tuning also updates some pretrained layers.
- Always replace the final classification layer to match your number of classes.
- Freeze layers with `param.requires_grad = False`. New layers have `requires_grad = True` by default.
- Use a 10x smaller learning rate for fine-tuning pretrained layers compared to the new head.
- ImageNet normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) is mandatory for pretrained models.
- Save the best model weights during training, not just the final weights.

Practice Questions

1. Why do early layers of a pretrained CNN learn features that are useful for almost any image task? What types of features do they learn?
2. You have 50 images per class. Should you use feature extraction or fine-tuning? Explain your reasoning.
3. What happens if you fine-tune a pretrained model with a learning rate of 0.01 (same as training from scratch)? Why is this bad?
4. Explain why you need to use ImageNet normalization when using a model pretrained on ImageNet, even if your images are very different from ImageNet images.
5. You replaced ResNet18's final layer with `nn.Linear(512, 10)`. But when you try ResNet50, it crashes. What went wrong, and how do you fix it?

Exercises

Exercise 1: Feature Extraction Pipeline

Build a complete feature extraction pipeline:

1. Load a pretrained ResNet18
2. Freeze all layers
3. Replace the final layer for 3 classes
4. Train on a small dataset for 10 epochs
5. Report training and validation accuracy
6. Save the best model

Exercise 2: Feature Extraction vs. Fine-Tuning

Compare two approaches on the same dataset:

1. Feature extraction (frozen backbone, lr=0.001)
2. Fine-tuning (unfreeze layer4, lr=0.0001)

Plot both training curves on the same graph and compare final accuracies.

Exercise 3: Discriminative Learning Rates

Implement discriminative learning rates for ResNet18:

- conv1, layer1, layer2: lr = 0.00001
- layer3, layer4: lr = 0.0001
- fc (new head): lr = 0.001

Train for 15 epochs and compare results to using a single learning rate for all layers.

What Is Next?

So far, you have learned to classify entire images -- answering "what is this image?" The next chapter takes you further: Object Detection. Instead of just classifying an image, object detection finds AND locates every object in an image, drawing bounding boxes around them. You will learn about YOLO, one of the most popular real-time object detection systems, and use it to detect objects in your own images.

Chapter 20: Object Detection

What You Will Learn

In this chapter, you will learn:

- What object detection is and how it differs from image classification
- How bounding boxes represent the location of objects
- What IoU (Intersection over Union) is and why it matters
- How YOLO works at a high level (dividing images into a grid)
- How to use YOLOv8 with the ultralytics library for inference
- How to draw bounding boxes on images
- How confidence thresholds filter out weak detections
- What NMS (Non-Maximum Suppression) does and why it is necessary

Why This Chapter Matters

Image classification answers: "What is in this image?" Object detection answers a harder question: "What objects are in this image, AND where are they?"

This difference is enormous in practice. A self-driving car does not just need to know that there is a pedestrian somewhere in the scene -- it needs to know exactly WHERE the pedestrian is. A security camera does not just need to know that a person entered the frame -- it needs to track their position. A retail system does not just need to know that products are on a shelf -- it needs to know which product is where.

Object detection is one of the most commercially valuable skills in computer vision. It powers autonomous vehicles, medical imaging, warehouse robotics, sports analytics, wildlife monitoring, and hundreds of other applications. Understanding it opens up a whole new class of problems you can solve.

20.1 Classification vs. Detection vs. Segmentation

Let us clearly distinguish three levels of understanding an image:

```
Image Classification:
+---------------------------+
|                           |
|       "dog"               |    One label for the entire image
|                           |
+---------------------------+

Object Detection:
+---------------------------+
|        +------+           |
|        | dog  |  +-----+  |    Find objects AND their locations
|        +------+  | cat |  |    (bounding boxes)
|                  +-----+  |
+---------------------------+

Instance Segmentation:
+---------------------------+
|       ########            |
|       # dog  #  #####     |    Exact pixel-by-pixel outline
|       ########  # cat#    |    of each object
|                 #####     |
+---------------------------+
```

```
What each task outputs:

Classification:  "dog"  (just a label)

Detection:       [
                   {"label": "dog",  "box": [x1, y1, x2, y2], "conf": 0.95},
                   {"label": "cat",  "box": [x1, y1, x2, y2], "conf": 0.87},
                 ]

Each detection has:
  - A class label (what)
  - A bounding box (where)
  - A confidence score (how sure)
```

20.2 Bounding Boxes

A bounding box is a rectangle that tightly encloses an object in an image. It is the standard way to represent object locations.

Bounding Box Formats

There are two common formats for specifying a bounding box:

```
Format 1: Corner format (x1, y1, x2, y2)
  (x1, y1) = top-left corner
  (x2, y2) = bottom-right corner

  (x1,y1)
    +------------------+
    |                  |
    |    OBJECT        |
    |                  |
    +------------------+
                    (x2,y2)

Format 2: Center format (cx, cy, w, h)
  (cx, cy) = center point
  w = width
  h = height

         w
    +------------------+
    |                  |
    |    (cx,cy)       | h
    |       *          |
    +------------------+

YOLO uses center format internally.
Most display libraries use corner format.
```

Converting Between Formats

```python
def corner_to_center(x1, y1, x2, y2):
    """Convert (x1, y1, x2, y2) to (cx, cy, w, h)."""
    cx = (x1 + x2) / 2
    cy = (y1 + y2) / 2
    w = x2 - x1
    h = y2 - y1
    return cx, cy, w, h

def center_to_corner(cx, cy, w, h):
    """Convert (cx, cy, w, h) to (x1, y1, x2, y2)."""
    x1 = cx - w / 2
    y1 = cy - h / 2
    x2 = cx + w / 2
    y2 = cy + h / 2
    return x1, y1, x2, y2
```

```
# Example
x1, y1, x2, y2 = 100, 50, 300, 250
cx, cy, w, h = corner_to_center(x1, y1, x2, y2)
print(f"Corner: ({x1}, {y1}, {x2}, {y2})")
print(f"Center: (cx={cx}, cy={cy}, w={w}, h={h})")

# Output:
# Corner: (100, 50, 300, 250)
# Center: (cx=200.0, cy=150.0, w=200, h=200)
```

Drawing Bounding Boxes

```python
import cv2
import numpy as np
from PIL import Image, ImageDraw, ImageFont

# === Method 1: Drawing with OpenCV ===
def draw_boxes_opencv(image_path, detections):
    """Draw bounding boxes using OpenCV."""
    image = cv2.imread(image_path)

    for det in detections:
        x1, y1, x2, y2 = det["box"]
        label = det["label"]
        conf = det["confidence"]

        # Draw rectangle
        color = (0, 255, 0)   # Green in BGR
        thickness = 2
        cv2.rectangle(image, (x1, y1), (x2, y2), color, thickness)

        # Draw label background
        text = f"{label} {conf:.2f}"
        (text_w, text_h), _ = cv2.getTextSize(
            text, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 1
        )
        cv2.rectangle(image, (x1, y1 - text_h - 8),
                      (x1 + text_w, y1), color, -1)  # Filled

        # Draw label text
        cv2.putText(image, text, (x1, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                    (0, 0, 0), 1)  # Black text

    return image

# === Method 2: Drawing with PIL ===
def draw_boxes_pil(image_path, detections):
    """Draw bounding boxes using PIL."""
    image = Image.open(image_path)
```

```
    draw = ImageDraw.Draw(image)

    for det in detections:
        x1, y1, x2, y2 = det["box"]
        label = det["label"]
        conf = det["confidence"]

        # Draw rectangle
        draw.rectangle([x1, y1, x2, y2], outline="green", width=3)

        # Draw label
        text = f"{label} {conf:.2f}"
        draw.text((x1, y1 - 15), text, fill="green")

    return image

# Example usage
detections = [
    {"label": "dog", "box": [100, 50, 300, 250], "confidence": 0.95},
    {"label": "cat", "box": [350, 100, 500, 280], "confidence": 0.87},
]

# result = draw_boxes_pil("photo.jpg", detections)
# result.show()
```

20.3 IoU (Intersection over Union)

IoU (Intersection over Union) measures how much two bounding boxes overlap. It is the standard metric for evaluating object detection.

```
IoU = Area of Overlap / Area of Union

Box A:         Box B:         Overlap:       Union:
+--------+                   +----+         +--------+---+
|        |   +--------+      |    |         |        |   |
|   A    |   |        |      |    |         |   A    | B |
|     +--+---+--+     |      +----+         |     +--+---+--+
|     |  |   |  |     |   (intersection)    |     |        |
+-----+--+   |  B     |                     +-----+--------+
      |      |        |                          (union)
      +------+--------+

IoU = intersection area / union area
```

Computing IoU

```python
def compute_iou(box1, box2):
    """
    Compute IoU between two boxes.
    Each box is [x1, y1, x2, y2].
    """
    # Find the intersection rectangle
    x1 = max(box1[0], box2[0])  # Left edge of intersection
    y1 = max(box1[1], box2[1])  # Top edge of intersection
    x2 = min(box1[2], box2[2])  # Right edge of intersection
    y2 = min(box1[3], box2[3])  # Bottom edge of intersection

    # Calculate intersection area
    intersection = max(0, x2 - x1) * max(0, y2 - y1)

    # Calculate union area
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
    union = area1 + area2 - intersection

    # Compute IoU
    iou = intersection / union if union > 0 else 0
    return iou

# Example 1: Boxes overlap significantly
box1 = [100, 100, 300, 300]   # 200x200 = 40000 pixels
box2 = [150, 150, 350, 350]   # 200x200 = 40000 pixels
print(f"IoU: {compute_iou(box1, box2):.4f}")
# Output: IoU: 0.3913

# Example 2: Boxes are identical (perfect match)
box3 = [100, 100, 300, 300]
box4 = [100, 100, 300, 300]
print(f"IoU (identical): {compute_iou(box3, box4):.4f}")
# Output: IoU (identical): 1.0000

# Example 3: Boxes do not overlap at all
box5 = [0, 0, 100, 100]
box6 = [200, 200, 300, 300]
print(f"IoU (no overlap): {compute_iou(box5, box6):.4f}")
# Output: IoU (no overlap): 0.0000
```

Line-by-line explanation:

- `max(box1[0], box2[0])`: The left edge of the intersection is the rightmost left edge of the two boxes.
- `min(box1[2], box2[2])`: The right edge of the intersection is the leftmost right edge.
- `max(0, x2 - x1)`: If the boxes do not overlap, x2 - x1 will be negative. `max(0, ...)` ensures we get 0 instead of a negative area.
- `area1 + area2 - intersection`: The union is both areas combined, minus the intersection (which was counted twice).

IoU Thresholds in Practice

```
IoU Value    Interpretation
---------------------------------
0.0          No overlap at all
0.1 - 0.3    Slight overlap, poor match
0.3 - 0.5    Moderate overlap
0.5+         Good match (common threshold for "correct")
0.75+        Very good match (stricter evaluation)
1.0          Perfect overlap (identical boxes)

Standard COCO evaluation uses IoU = 0.5
(a detection with IoU >= 0.5 with ground truth
is considered "correct")
```

20.4 How YOLO Works

YOLO stands for You Only Look Once. It is one of the most popular object detection algorithms because it is fast and accurate.

The Key Idea: Grid-Based Detection

Instead of scanning every possible location (which is slow), YOLO divides the image into a grid and predicts objects for each grid cell simultaneously:

```
YOLO Grid Approach:

Original Image:              YOLO Grid (e.g., 7x7):
+-------------------------+   +--+--+--+--+--+--+--+
|                         |   |  |  |  |  |  |  |  |
|      +------+           |   +--+--+--+--+--+--+--+
|      | dog  |           |   |  |  |##|##|  |  |  |
|      +------+           |   +--+--+--+--+--+--+--+
|                         |   |  |  |##|##|  |  |  |
|             +-----+     |   +--+--+--+--+--+--+--+
|             | cat |     |   |  |  |  |  |  |  |  |
|             +-----+     |   +--+--+--+--+--+--+--+
+-------------------------+   |  |  |  |  |##|##|  |
                               +--+--+--+--+--+--+--+
                               |  |  |  |  |##|##|  |
                               +--+--+--+--+--+--+--+
                               |  |  |  |  |  |  |  |
                               +--+--+--+--+--+--+--+
```

```
## = Grid cells responsible for detecting objects
     (the cell containing the object's center)
```

What Each Grid Cell Predicts

Each grid cell predicts:

1. Bounding boxes (position and size)
2. Confidence scores (is there an object here?)
3. Class probabilities (if there is an object, what class is it?)

```
One Grid Cell Predicts:

+----------------------------------+
| Bounding Box 1:                  |
|   (cx, cy, w, h, confidence)    |
|                                  |
| Bounding Box 2:                  |
|   (cx, cy, w, h, confidence)    |
|                                  |
| Class Probabilities:             |
|   P(dog)=0.8, P(cat)=0.1, ...  |
+----------------------------------+

Total output per cell:
  B boxes * 5 values + C classes
  where B = number of boxes per cell
        C = number of classes
```

Why "You Only Look Once"?

```
Traditional Detection (R-CNN):
  1. Propose ~2000 regions
  2. Classify each region individually
  3. Very slow (seconds per image)

  Image -> [Region 1] -> CNN -> "dog"
        -> [Region 2] -> CNN -> "background"
        -> [Region 3] -> CNN -> "cat"
        -> ...2000 times...

YOLO:
  1. Pass the ENTIRE image through the network ONCE
  2. Get ALL detections simultaneously
  3. Very fast (milliseconds per image)

  Image -> CNN -> Grid of predictions -> All detections at once!
```

```
YOLO is 100-1000x faster than R-CNN approaches.
```

YOLO Evolution

```
Version    Year    Key Innovation
-----------------------------------------
YOLOv1     2016    Original grid-based approach
YOLOv2     2017    Anchor boxes, batch norm
YOLOv3     2018    Multi-scale detection
YOLOv4     2020    Bag of freebies, CSP backbone
YOLOv5     2020    PyTorch implementation, easy to use
YOLOv8     2023    State-of-the-art, anchor-free
YOLOv11    2024    Latest, improved performance

We will use YOLOv8 (via the ultralytics library)
because it is modern, fast, and very easy to use.
```

20.5 Using YOLOv8 with Ultralytics

The ultralytics library makes using YOLO incredibly easy -- just a few lines of code.

Installation

```python
# Install ultralytics
# Run in your terminal:
# pip install ultralytics
```

Basic Inference

```python
from ultralytics import YOLO

# Load a pretrained YOLOv8 model
# 'yolov8n' = nano (smallest, fastest)
# 'yolov8s' = small
# 'yolov8m' = medium
# 'yolov8l' = large
```

```
# 'yolov8x' = extra large (most accurate, slowest)
model = YOLO("yolov8n.pt")

# Run inference on an image
results = model("street_scene.jpg")

# Print results
for result in results:
    print(f"Found {len(result.boxes)} objects")
    print(f"Image shape: {result.orig_shape}")

# Output:
# Found 8 objects
# Image shape: (720, 1280)
```

Line-by-line explanation:

- `YOLO("yolov8n.pt")`: Downloads and loads the YOLOv8 nano model. On the first run, it downloads the model weights (~6MB). The "n" suffix means nano -- the smallest and fastest variant.
- `model("street_scene.jpg")`: Runs detection on the image. It handles all preprocessing internally (resize, normalize, etc.).
- `result.boxes`: Contains all detected bounding boxes with their class labels and confidence scores.

Exploring Detection Results

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model("street_scene.jpg")

# Get the first result (one per image)
result = results[0]

# Access bounding boxes
boxes = result.boxes

print(f"Number of detections: {len(boxes)}")
print(f"Box coordinates (xyxy format):\n{boxes.xyxy}")
print(f"Confidence scores:\n{boxes.conf}")
print(f"Class indices:\n{boxes.cls}")

# Output:
# Number of detections: 8
# Box coordinates (xyxy format):
# tensor([[ 218.2,  267.8,  428.5,  525.3],
```

```
#         [ 612.4,  281.1,  785.2,  518.7],
#         ...])
# Confidence scores:
# tensor([0.9234, 0.8876, 0.8543, ...])
# Class indices:
# tensor([0., 0., 2., ...])

# Get class names
print(f"\nDetailed results:")
for i in range(len(boxes)):
    x1, y1, x2, y2 = boxes.xyxy[i].tolist()
    confidence = boxes.conf[i].item()
    class_id = int(boxes.cls[i].item())
    class_name = model.names[class_id]

    print(f"  {class_name:12s} conf={confidence:.3f} "
          f"box=({x1:.0f}, {y1:.0f}, {x2:.0f}, {y2:.0f})")

# Output:
# Detailed results:
#   person       conf=0.923 box=(218, 268, 429, 525)
#   person       conf=0.888 box=(612, 281, 785, 519)
#   car          conf=0.854 box=(0, 320, 215, 520)
#   ...
```

Line-by-line explanation:

- `boxes.xyxy`: Bounding boxes in (x1, y1, x2, y2) corner format. Each row is one detection.
- `boxes.conf`: Confidence score for each detection (0 to 1). Higher means the model is more certain.
- `boxes.cls`: Class index for each detection. Use `model.names[class_id]` to get the human-readable name.
- `model.names`: A dictionary mapping class indices to class names. YOLOv8 trained on COCO has 80 classes (person, car, dog, etc.).

COCO Class Names

YOLOv8 pretrained models are trained on the COCO dataset which has 80 object classes:

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Print all 80 class names
for idx, name in model.names.items():
    print(f"  {idx:3d}: {name}")
```

```
# Output (abbreviated):
#   0: person
#   1: bicycle
#   2: car
#   3: motorcycle
#   ...
#  14: bird
#  15: cat
#  16: dog
#   ...
#  79: toothbrush
```

20.6 Running Inference on Images

Single Image Inference

```python
from ultralytics import YOLO
from PIL import Image
import matplotlib.pyplot as plt

model = YOLO("yolov8n.pt")

# Run detection
results = model("photo.jpg")

# Method 1: Use built-in visualization
result = results[0]
annotated_image = result.plot()   # Returns BGR NumPy array with boxes drawn

# Convert BGR to RGB for matplotlib
import cv2
annotated_rgb = cv2.cvtColor(annotated_image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 8))
plt.imshow(annotated_rgb)
plt.title(f"YOLOv8 Detection: {len(result.boxes)} objects found")
plt.axis("off")
plt.show()
```

Line-by-line explanation:

- `result.plot()`: Automatically draws all bounding boxes, labels, and confidence scores on the image. Returns a BGR NumPy array.
- We convert to RGB for matplotlib display (same BGR/RGB issue from Chapter 16).

Batch Inference on Multiple Images

```python
from ultralytics import YOLO
import os

model = YOLO("yolov8n.pt")

# Process multiple images at once
image_paths = ["photo1.jpg", "photo2.jpg", "photo3.jpg"]
results = model(image_paths)

# Process each result
for i, result in enumerate(results):
    print(f"\nImage: {image_paths[i]}")
    print(f"  Objects found: {len(result.boxes)}")

    for box in result.boxes:
        class_name = model.names[int(box.cls)]
        conf = box.conf.item()
        coords = box.xyxy[0].tolist()
        print(f"  {class_name}: {conf:.3f} at "
              f"({coords[0]:.0f},{coords[1]:.0f},"
              f"{coords[2]:.0f},{coords[3]:.0f})")

# Output:
# Image: photo1.jpg
#   Objects found: 3
#   person: 0.923 at (218,268,429,525)
#   car: 0.854 at (0,320,215,520)
#   dog: 0.712 at (500,400,650,550)
#
# Image: photo2.jpg
#   Objects found: 5
#   ...
```

Inference on a Video (Frame by Frame)

```python
from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")

# Open video
cap = cv2.VideoCapture("video.mp4")

frame_count = 0
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
```

```
        break

    # Run detection on each frame
    results = model(frame, verbose=False)
    result = results[0]

    # Draw results
    annotated = result.plot()

    # Count objects in this frame
    num_objects = len(result.boxes)

    frame_count += 1
    if frame_count % 30 == 0:  # Print every 30 frames
        print(f"Frame {frame_count}: {num_objects} objects detected")

    # Display (in a GUI environment)
    # cv2.imshow("Detection", annotated)
    # if cv2.waitKey(1) & 0xFF == ord("q"):
    #     break

cap.release()
print(f"Processed {frame_count} frames")
```

20.7 Confidence Thresholds

The confidence threshold controls which detections are kept. Detections with confidence below the threshold are discarded.

```
Confidence Threshold:

All detections (no threshold):
  person  0.95  <- Keep
  car     0.82  <- Keep
  dog     0.45  <- Maybe?
  cat     0.12  <- Probably wrong
  chair   0.03  <- Almost certainly wrong

With threshold = 0.5:
  person  0.95  <- KEEP (above 0.5)
  car     0.82  <- KEEP (above 0.5)
  dog     0.45  <- DISCARD (below 0.5)
  cat     0.12  <- DISCARD (below 0.5)
  chair   0.03  <- DISCARD (below 0.5)
```

Setting the Confidence Threshold

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Default threshold (usually 0.25)
results_default = model("photo.jpg")
print(f"Default: {len(results_default[0].boxes)} detections")

# Higher threshold = fewer but more confident detections
results_high = model("photo.jpg", conf=0.7)
print(f"conf=0.7: {len(results_high[0].boxes)} detections")

# Lower threshold = more detections, but some might be wrong
results_low = model("photo.jpg", conf=0.1)
print(f"conf=0.1: {len(results_low[0].boxes)} detections")

# Output:
# Default: 8 detections
# conf=0.7: 4 detections
# conf=0.1: 15 detections
```

Choosing the Right Threshold

```
Threshold Too Low (e.g., 0.1):
  + Catches everything, even hard-to-see objects
  - Lots of false detections (seeing things that are not there)
  Use for: Safety-critical applications (do not miss anything)

Threshold Too High (e.g., 0.9):
  + Very few false detections
  - Misses objects that are partially hidden or far away
  Use for: Applications where false alarms are costly

Good Default (0.25 - 0.5):
  Balanced trade-off between catching objects and avoiding false alarms
  Use for: Most applications

The "right" threshold depends on your specific use case.
There is no universally correct value.
```

Visualizing Different Thresholds

```python
from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2

model = YOLO("yolov8n.pt")

thresholds = [0.1, 0.3, 0.5, 0.7]
fig, axes = plt.subplots(1, 4, figsize=(20, 5))

for ax, thresh in zip(axes, thresholds):
    results = model("street_scene.jpg", conf=thresh, verbose=False)
    result = results[0]

    annotated = result.plot()
    annotated_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)

    ax.imshow(annotated_rgb)
    ax.set_title(f"conf={thresh}\n{len(result.boxes)} detections")
    ax.axis("off")

plt.suptitle("Effect of Confidence Threshold", fontsize=14)
plt.tight_layout()
plt.show()
```

Output: Four versions of the same image with different numbers of bounding boxes. Lower thresholds show more boxes (including false positives); higher thresholds show fewer, more confident boxes.

20.8 NMS (Non-Maximum Suppression)

Non-Maximum Suppression (NMS) solves a common problem: the model detects the same object multiple times with slightly different bounding boxes.

The Problem: Duplicate Detections

```
Without NMS:

  +--------+
  | +------+-+
  | | +----+-+-+
  | | |    | | |    Three overlapping boxes
  | | | dog| | |    all detecting the SAME dog
  | | |    | | |
  | | +----+ | |
  | +--------+ |
```

```
  +------------+

The model is not wrong -- it detected the dog
from three slightly different positions.
But we want ONE box per dog, not three.
```

How NMS Works

```
NMS Algorithm:

1. Sort all boxes by confidence score (highest first)
   Box A: dog, conf=0.95
   Box B: dog, conf=0.88
   Box C: dog, conf=0.82
   Box D: cat, conf=0.90

2. Start with the highest confidence box (A)
   Keep Box A.

3. Compare A with every remaining box of the SAME class:
   IoU(A, B) = 0.85 > threshold(0.45) -> REMOVE B (too similar to A)
   IoU(A, C) = 0.78 > threshold(0.45) -> REMOVE C (too similar to A)

4. Move to next remaining box (D is a different class)
   Keep Box D.

5. Result: Box A (dog) and Box D (cat)
   Two clean detections, no duplicates!
```

```
NMS Step by Step:

Step 1: All detections         Step 2: After NMS
+--------+                     +--------+
| +------+-+                   |        |
| | +----+-+-+                 |  dog   |     Only the best
| | |    | | |    --------->  |  0.95  |     box survives
| | | dog| | |                |        |
| | |    | | |                +--------+
| | +----+ | |
| +--------+ |                +------+
+------------+                 | cat  |
     +------+                  | 0.90 |
     | cat  |                  +------+
     +------+
```

NMS in YOLOv8

YOLOv8 applies NMS automatically. You can control the IoU threshold:

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Default NMS IoU threshold is 0.7
results = model("photo.jpg")
print(f"Default NMS: {len(results[0].boxes)} detections")

# Stricter NMS (lower IoU threshold = remove more overlapping boxes)
results_strict = model("photo.jpg", iou=0.3)
print(f"Strict NMS (iou=0.3): {len(results_strict[0].boxes)} detections")

# Lenient NMS (higher IoU threshold = keep more overlapping boxes)
results_lenient = model("photo.jpg", iou=0.9)
print(f"Lenient NMS (iou=0.9): {len(results_lenient[0].boxes)} detections")

# Output:
# Default NMS: 8 detections
# Strict NMS (iou=0.3): 5 detections
# Lenient NMS (iou=0.9): 12 detections
```

Line-by-line explanation:

- `iou=0.3`: Any two boxes with IoU above 0.3 are considered duplicates. Only the higher-confidence one survives. This is strict -- it removes more boxes.
- `iou=0.9`: Boxes must overlap by 90% to be considered duplicates. This is lenient -- it keeps more boxes, allowing nearby detections.

```
NMS IoU Threshold:

Low (0.3): Aggressive filtering.
  Keeps fewer boxes. Good when objects
  are well-separated.

Default (0.5-0.7): Balanced.
  Works well for most scenarios.

High (0.9): Minimal filtering.
  Keeps many overlapping boxes. Useful
  when objects are densely packed
  (e.g., a crowd of people).
```

Implementing NMS from Scratch (for Understanding)

```python
import numpy as np

def nms(boxes, scores, iou_threshold=0.5):
    """
    Non-Maximum Suppression.

    Args:
        boxes: array of [x1, y1, x2, y2] shape (N, 4)
        scores: array of confidence scores shape (N,)
        iou_threshold: IoU threshold for suppression

    Returns:
        indices of boxes to keep
    """
    # Sort by confidence (descending)
    order = scores.argsort()[::-1]

    keep = []

    while len(order) > 0:
        # Keep the highest confidence box
        current = order[0]
        keep.append(current)

        if len(order) == 1:
            break

        # Compare with remaining boxes
        remaining = order[1:]
        ious = np.array([
            compute_iou(boxes[current], boxes[r])
            for r in remaining
        ])

        # Keep boxes with low IoU (they detect different objects)
        mask = ious < iou_threshold
        order = remaining[mask]

    return keep

# Example
boxes = np.array([
    [100, 100, 300, 300],  # Box 0
    [110, 110, 310, 310],  # Box 1 (overlaps with 0)
    [105, 105, 305, 305],  # Box 2 (overlaps with 0)
    [400, 400, 550, 550],  # Box 3 (separate object)
])
scores = np.array([0.95, 0.88, 0.82, 0.90])

kept = nms(boxes, scores, iou_threshold=0.5)
print(f"Kept box indices: {kept}")
print(f"Kept {len(kept)} out of {len(boxes)} boxes")
# Output:
# Kept box indices: [0, 3]
```

```
# Kept 2 out of 4 boxes
# (Boxes 1 and 2 were suppressed because they
#  overlapped too much with the higher-confidence Box 0)
```

20.9 Complete Detection Pipeline

Here is a complete, end-to-end object detection script:

```python
from ultralytics import YOLO
import cv2
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

def detect_objects(image_path, model_size="yolov8n.pt",
                   conf_threshold=0.25, iou_threshold=0.7):
    """
    Complete object detection pipeline.

    Args:
        image_path: Path to the image
        model_size: YOLO model variant
        conf_threshold: Minimum confidence to keep a detection
        iou_threshold: IoU threshold for NMS

    Returns:
        List of detections and annotated image
    """
    # Load model
    model = YOLO(model_size)

    # Run detection
    results = model(
        image_path,
        conf=conf_threshold,
        iou=iou_threshold,
        verbose=False
    )

    result = results[0]
    boxes = result.boxes

    # Extract detections
    detections = []
    for i in range(len(boxes)):
        det = {
```

```
            "class_id": int(boxes.cls[i].item()),
            "class_name": model.names[int(boxes.cls[i].item())],
            "confidence": boxes.conf[i].item(),
            "bbox": boxes.xyxy[i].tolist(),  # [x1, y1, x2, y2]
        }
        detections.append(det)

    # Sort by confidence
    detections.sort(key=lambda x: x["confidence"], reverse=True)

    # Get annotated image
    annotated = result.plot()
    annotated_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)

    return detections, annotated_rgb

def print_detection_summary(detections):
    """Print a summary of detections."""
    print(f"\n{'='*50}")
    print(f"Detection Summary: {len(detections)} objects found")
    print(f"{'='*50}")

    # Count by class
    class_counts = Counter(d["class_name"] for d in detections)
    print(f"\nObject counts:")
    for name, count in class_counts.most_common():
        print(f"  {name}: {count}")

    # Detailed list
    print(f"\nDetailed detections:")
    for i, det in enumerate(detections):
        x1, y1, x2, y2 = det["bbox"]
        w = x2 - x1
        h = y2 - y1
        print(f"  [{i+1}] {det['class_name']:12s} "
              f"conf={det['confidence']:.3f} "
              f"size={w:.0f}x{h:.0f} "
              f"at ({x1:.0f},{y1:.0f})")

def display_results(image_path, annotated_image, detections):
    """Display original and annotated images side by side."""
    original = cv2.cvtColor(cv2.imread(image_path), cv2.COLOR_BGR2RGB)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

    ax1.imshow(original)
    ax1.set_title("Original Image")
    ax1.axis("off")

    ax2.imshow(annotated_image)
    ax2.set_title(f"Detections ({len(detections)} objects)")
    ax2.axis("off")

    plt.tight_layout()
    plt.show()
```

```
# === Run the pipeline ===
detections, annotated = detect_objects(
    "street_scene.jpg",
    conf_threshold=0.3,
    iou_threshold=0.5
)

print_detection_summary(detections)
display_results("street_scene.jpg", annotated, detections)

# Output:
# ==================================================
# Detection Summary: 8 objects found
# ==================================================
#
# Object counts:
#   person: 3
#   car: 2
#   dog: 1
#   bicycle: 1
#   traffic light: 1
#
# Detailed detections:
#   [1] person       conf=0.954 size=211x257 at (218,268)
#   [2] car          conf=0.912 size=215x200 at (0,320)
#   [3] person       conf=0.888 size=173x238 at (612,281)
#   ...
```

20.10 Filtering Detections by Class

Often you only care about specific object types:

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Detect only specific classes
# Use class indices from model.names
# 0 = person, 2 = car, 16 = dog

# Method 1: Filter using the classes parameter
results = model("street_scene.jpg", classes=[0, 2])  # Only people and cars
print(f"People and cars: {len(results[0].boxes)} detections")

# Method 2: Filter after detection
```

```
results_all = model("street_scene.jpg")
all_boxes = results_all[0].boxes

# Keep only "person" detections
person_mask = all_boxes.cls == 0
person_boxes = all_boxes[person_mask]
print(f"People only: {len(person_boxes)} detections")

# Keep only detections with confidence > 0.8
high_conf_mask = all_boxes.conf > 0.8
high_conf_boxes = all_boxes[high_conf_mask]
print(f"High confidence: {len(high_conf_boxes)} detections")

# Output:
# People and cars: 5 detections
# People only: 3 detections
# High confidence: 4 detections
```

20.11 Choosing the Right YOLO Model

```python
from ultralytics import YOLO
import time

# Compare model sizes
model_variants = ["yolov8n.pt", "yolov8s.pt", "yolov8m.pt"]

for variant in model_variants:
    model = YOLO(variant)

    # Warm up
    model("photo.jpg", verbose=False)

    # Time inference
    start = time.time()
    for _ in range(10):
        results = model("photo.jpg", verbose=False)
    avg_time = (time.time() - start) / 10

    num_detections = len(results[0].boxes)
    print(f"{variant:12s}: {avg_time*1000:.1f}ms, "
          f"{num_detections} detections")

# Typical output:
# yolov8n.pt  :  12.3ms, 7 detections
# yolov8s.pt  :  28.5ms, 9 detections
# yolov8m.pt  :  65.2ms, 10 detections
```

```
Model Selection Guide:

Model     Params   Speed      Accuracy    Best For
-------------------------------------------------------
yolov8n   3.2M     Fastest    Good        Real-time, mobile
yolov8s   11.2M    Fast       Better      Edge devices, drones
yolov8m   25.9M    Medium     Very Good   Desktop applications
yolov8l   43.7M    Slower     Excellent   Server-side processing
yolov8x   68.2M    Slowest    Best        Maximum accuracy

Rule of thumb:
  - Start with 'n' (nano) for prototyping
  - Use 's' or 'm' for production
  - Use 'l' or 'x' only when accuracy is critical
    and speed does not matter
```

Common Mistakes

1. Not adjusting confidence thresholds for your use case. The default threshold works for general use, but safety-critical applications need lower thresholds (catch everything), while precision-critical applications need higher thresholds (only confident detections).
2. Ignoring NMS settings when objects are densely packed. If you are detecting people in a crowd, the default NMS might suppress valid detections because they overlap. Increase the IoU threshold.
3. Forgetting BGR to RGB conversion when displaying OpenCV images. YOLO's `result.plot()` returns BGR. Always convert before displaying with matplotlib.
4. Using the wrong model size. Starting with yolov8x for a prototype wastes time. Start with yolov8n, get the pipeline working, then upgrade if needed.
5. Confusing class indices across different models. COCO class indices are standard for pretrained YOLO, but custom-trained models will have different class mappings. Always check `model.names`.
6. Not handling empty detections. Some images have no objects above the confidence threshold. Always check `if len(result.boxes) > 0` before processing detections.

Best Practices

1. Start with yolov8n for prototyping. It is fast enough for interactive development and accurate enough to validate your pipeline.

2. Tune confidence and NMS thresholds for your specific application. Test different values on a validation set and pick the ones that give the best trade-off.
3. Filter by class when you only care about specific objects. Using the `classes` parameter is faster than detecting everything and filtering afterward.
4. Save detection results in a structured format (JSON or CSV) for later analysis. Include image path, class, confidence, and bounding box coordinates.
5. Always visualize results on sample images before deploying. Seeing the boxes drawn on images reveals problems that metrics alone cannot show.

Quick Summary

Object detection finds and locates objects in images using bounding boxes. Each detection includes a class label, a bounding box (position and size), and a confidence score. IoU (Intersection over Union) measures how well a predicted box matches the ground truth. YOLO divides the image into a grid and predicts all objects in a single forward pass, making it very fast. YOLOv8 via the ultralytics library provides an easy-to-use interface for detection. Confidence thresholds filter out weak detections, and NMS (Non-Maximum Suppression) removes duplicate detections of the same object.

Key Points

- Object detection answers "what AND where" -- it finds objects and draws bounding boxes around them.
- Bounding boxes can be in corner format (x1, y1, x2, y2) or center format (cx, cy, w, h).
- IoU measures overlap between two boxes: 0 = no overlap, 1 = perfect match. IoU >= 0.5 is typically considered a correct detection.
- YOLO processes the entire image in one pass (hence "You Only Look Once"), making it much faster than region-based methods.
- YOLOv8 via ultralytics requires just two lines: `model = YOLO("yolov8n.pt")` and `results = model("image.jpg")`.
- Confidence threshold controls the precision-recall trade-off. Lower = more detections (including false ones). Higher = fewer but more reliable detections.
- NMS removes duplicate detections of the same object by keeping only the highest-confidence box among overlapping boxes.

Practice Questions

1. What are the three pieces of information that an object detection model outputs for each detected object?
2. Two bounding boxes have an IoU of 0.6. What does this mean visually? Would an NMS algorithm with `iou_threshold=0.5` keep both boxes or remove one?
3. You are building a system to detect pedestrians for a self-driving car. Should you use a high or low confidence threshold? Explain why.
4. Explain in simple terms how YOLO detects objects differently from a sliding-window approach. Why is YOLO faster?
5. You run YOLO on an image and get 15 detections. After increasing the confidence threshold from 0.25 to 0.7, you get 6 detections. What happened to the other 9? Were they necessarily wrong?

Exercises

Exercise 1: Object Counter

Write a script that:

1. Takes an image path as input
2. Runs YOLOv8 detection
3. Counts the number of each object type detected
4. Prints a table showing object type and count
5. Saves the annotated image to disk

Exercise 2: IoU Calculator and Visualizer

Write a program that:

1. Takes two bounding boxes as input
2. Calculates their IoU
3. Draws both boxes on a blank image (different colors)
4. Shades the intersection area

5. Displays the IoU value as the title

Test with boxes that have IoU = 0 (no overlap), IoU around 0.5, and IoU = 1.0 (identical boxes).

Exercise 3: Confidence Threshold Analyzer

Write a script that:

1. Runs YOLOv8 on an image with confidence thresholds from 0.1 to 0.9 (step 0.1)
2. For each threshold, records the number of detections
3. Plots a line graph: x-axis = threshold, y-axis = number of detections
4. Identifies the "elbow point" where increasing the threshold starts to dramatically reduce detections

---

What Is Next?

Congratulations! You have now covered the core building blocks of computer vision: understanding images (Chapter 16), preprocessing them (Chapter 17), classifying them with CNNs (Chapter 18), leveraging transfer learning (Chapter 19), and detecting objects with YOLO (Chapter 20). These skills form a solid foundation for tackling real-world computer vision problems. From here, you can explore more advanced topics like image segmentation (labeling every pixel), pose estimation (detecting body positions), or training custom YOLO models on your own datasets. The fundamentals you have learned will serve you well in all of these directions.

Chapter 21: Image Segmentation

What You Will Learn

In this chapter, you will learn:

- What image segmentation means and why it is different from classification
- The difference between semantic segmentation and instance segmentation
- How pixel-level classification works
- What the U-Net architecture is and why it is so popular
- How to use pre-trained segmentation models from torchvision
- How to visualize segmentation masks with colors
- Real-world applications of image segmentation

Why This Chapter Matters

In earlier chapters, you learned how to classify entire images ("this is a cat") and how to detect objects with bounding boxes ("there is a cat at this location"). But what if you need to know the exact shape of the cat, down to every single pixel?

That is what image segmentation does. Instead of drawing a rectangle around an object, segmentation labels every pixel in the image. A self-driving car does not just need to know "there is a pedestrian somewhere in this image." It needs to know exactly which pixels belong to the road, which pixels belong to pedestrians, and which pixels belong to traffic signs. A doctor reviewing a medical scan needs to see exactly which pixels show a tumor, not just a rough bounding box.

Image segmentation is one of the most powerful techniques in computer vision, and it is used everywhere: from medical imaging to satellite analysis to photo editing tools that let you remove backgrounds with one click.

What Is Image Segmentation?

Image segmentation is the task of assigning a label to every pixel in an image. Think of it like coloring a coloring book: every tiny area gets filled with a specific color based on what it belongs to.

```
Image Classification:   "This image contains a cat"         -> ONE label for the WHOLE ...
Object Detection:       "There is a cat at [x, y, w, h]"    -> ONE box per object
Image Segmentation:     "Pixel (0,0) = background,           -> ONE label per PIXEL
                         Pixel (0,1) = background,
                         Pixel (1,5) = cat,
                         Pixel (1,6) = cat, ..."
```

A Simple Analogy

Imagine you have a photograph of a park scene with people, trees, and dogs:

- Classification is like saying "this photo was taken at a park"
- Detection is like drawing rectangles around each person, tree, and dog
- Segmentation is like carefully tracing the exact outline of every person, tree, and dog with a different colored marker

---

Semantic Segmentation vs Instance Segmentation

There are two main types of image segmentation, and understanding the difference is important.

Semantic Segmentation

Semantic segmentation assigns a class label to every pixel, but it does not distinguish between different objects of the same class.

```
Example: A street scene with three cars

Semantic Segmentation Result:
+-------------------------------------------------+
|  sky sky sky sky sky sky sky sky sky sky sky sky   |
|  sky sky sky sky sky sky sky sky sky sky sky sky   |
|  bld bld sky sky sky sky sky sky sky bld bld bld  |
```

```
|  bld bld bld sky sky sky sky sky bld bld bld bld |
|  bld bld bld bld sky sky sky bld bld bld bld bld |
|  car car car road road road car car road car car  |
|  car car car road road road car car road car car  |
|  road road road road road road road road road rd  |
+-----------------------------------------------------+

bld = building, car = car, road = road

Notice: ALL car pixels have the same label "car".
We cannot tell which pixels belong to car #1 vs car #2 vs car #3.
```

The word "semantic" means "related to meaning." Semantic segmentation understands the meaning (class) of each pixel but does not separate individual objects.

Instance Segmentation

Instance segmentation goes one step further. It assigns a class label AND a unique identity to every pixel. This means you can tell apart car number 1, car number 2, and car number 3.

```
Instance Segmentation Result:
+-----------------------------------------------------+
|  bg  bg  bg  bg  bg  bg  bg  bg  bg  bg  bg  bg  |
|  bg  bg  bg  bg  bg  bg  bg  bg  bg  bg  bg  bg  |
|  b1  b1  bg  bg  bg  bg  bg  bg  bg  b2  b2  b2  |
|  b1  b1  b1  bg  bg  bg  bg  bg  b2  b2  b2  b2  |
|  b1  b1  b1  b1  bg  bg  bg  b2  b2  b2  b2  b2  |
|  c1  c1  c1  rd  rd  rd  c2  c2  rd  c3  c3  rd  |
|  c1  c1  c1  rd  rd  rd  c2  c2  rd  c3  c3  rd  |
|  rd  rd  rd  rd  rd  rd  rd  rd  rd  rd  rd  rd   |
+-----------------------------------------------------+

c1 = car #1, c2 = car #2, c3 = car #3
b1 = building #1, b2 = building #2
bg = background, rd = road

Now we can distinguish individual objects!
```

When to Use Which

```
+----------------------+------------------+---------------------+
| Task                 | Semantic         | Instance            |
+----------------------+------------------+---------------------+
| Road surface         | Yes (one class)  | Not needed          |
| Count people         | Cannot count     | Yes (each separate) |
| Medical tumor area   | Yes (tumor/not)  | Not needed          |
| Track specific cars  | Cannot track     | Yes (each has ID)   |
```

```
| Background removal   | Yes               | Yes                 |
+----------------------+-------------------+---------------------+
```

How Pixel-Level Classification Works

At its core, segmentation is classification applied to every pixel. Instead of one prediction for the whole image, the model makes a prediction for each pixel.

The Input and Output

For a regular classifier:

- Input: an image of shape (3, 224, 224) — 3 color channels, 224 height, 224 width
- Output: a vector of shape (num_classes,) — one probability per class

For a segmentation model:

- Input: an image of shape (3, H, W)
- Output: a tensor of shape (num_classes, H, W) — one probability per class, for every pixel

```
Input Image: (3, 256, 256)
         |
    [Segmentation Model]
         |
Output: (num_classes, 256, 256)

For each pixel at position (row, col):
  output[:, row, col] = [prob_background, prob_cat, prob_dog, ...]

The class with highest probability becomes that pixel's label.
```

The Segmentation Mask

The final output is called a segmentation mask. It is a 2D image where each pixel value represents a class number instead of a color.

```python
import numpy as np
```

```
# Example: a tiny 4x4 segmentation mask
# 0 = background, 1 = cat, 2 = dog
mask = np.array([
    [0, 0, 0, 0],
    [0, 1, 1, 0],
    [0, 1, 1, 0],
    [0, 0, 2, 2]
])

print("Segmentation mask:")
print(mask)
print()
print("Pixel (1,1) is class:", mask[1, 1], "-> cat")
print("Pixel (3,2) is class:", mask[3, 2], "-> dog")
print("Pixel (0,0) is class:", mask[0, 0], "-> background")
```

Output:

```
Segmentation mask:
[[0 0 0 0]
 [0 1 1 0]
 [0 1 1 0]
 [0 0 2 2]]

Pixel (1,1) is class: 1 -> cat
Pixel (3,2) is class: 2 -> dog
Pixel (0,0) is class: 0 -> background
```

Line-by-line explanation:

- `mask = np.array([...])` creates a 4-by-4 grid where each value is a class label.
- Value 0 means that pixel belongs to the background.
- Value 1 means that pixel belongs to a cat.
- Value 2 means that pixel belongs to a dog.
- We can look up any pixel's class by indexing into the mask with `mask[row, col]`.

The U-Net Architecture

U-Net is one of the most important segmentation architectures ever created. It was originally designed for medical image segmentation in 2015 and is still widely used today.

Why Is It Called U-Net?

The architecture looks like the letter "U" when you draw it as a diagram. It has two main parts:

1. Encoder (left side of the U): This path goes downward. It shrinks the image and captures "what" is in the image (features).
2. Decoder (right side of the U): This path goes upward. It expands the features back to the original image size to produce a pixel-level prediction.
3. Skip Connections (horizontal bridges): These connect the encoder to the decoder at each level, passing fine details that would otherwise be lost.

```
U-Net Architecture (Simplified):

Input Image (256x256)                    Output Mask (256x256)
       |                                         ^
       v                                         |
  [Conv Block] 64 filters  ---- SKIP ----> [Conv Block] 64 filters
       |          (256x256)   CONNECTION      ^    (256x256)
       v                                       |
  [MaxPool] Shrink to 128x128           [UpConv] Expand to 256x256
       |                                       ^
       v                                       |
  [Conv Block] 128 filters ---- SKIP ----> [Conv Block] 128 filters
       |          (128x128)   CONNECTION      ^    (128x128)
       v                                       |
  [MaxPool] Shrink to 64x64             [UpConv] Expand to 128x128
       |                                       ^
       v                                       |
  [Conv Block] 256 filters ---- SKIP ----> [Conv Block] 256 filters
       |          (64x64)     CONNECTION      ^    (64x64)
       v                                       |
  [MaxPool] Shrink to 32x32             [UpConv] Expand to 64x64
       |                                       ^
       v                                       |
  [Conv Block] 512 filters ---- SKIP ----> [Conv Block] 512 filters
       |          (32x32)     CONNECTION      ^    (32x32)
       v                                       |
  [MaxPool] Shrink to 16x16             [UpConv] Expand to 32x32
       |                                       ^
       v                                       |
       +-----> [Conv Block] 1024 filters -----+
                    (16x16)
                 "Bottleneck"
                 (Bottom of the U)
```

Why Skip Connections Matter

When the encoder shrinks the image, it captures high-level features (like "this area contains a dog") but loses fine details (like exact edges). The skip connections copy the detailed information from the encoder directly to the decoder, so the final output has both:

- What each region is (from the encoder)

- Where the exact boundaries are (from the skip connections)

```
Think of it like giving driving directions:

Without skip connections:
  "Turn right somewhere around the big mall area"
  (You know the general area but lost the precise details)

With skip connections:
  "Turn right at 123 Main Street, the red building on the corner"
  (You have both the general area AND the precise location)
```

A Simple U-Net in PyTorch

Here is a simplified U-Net implementation to help you understand the structure:

```python
import torch
import torch.nn as nn

class SimpleUNet(nn.Module):
    """A simplified U-Net for understanding the architecture."""

    def __init__(self, in_channels=3, num_classes=2):
        super(SimpleUNet, self).__init__()

        # ---- Encoder (Downsampling Path) ----
        # Each block: two convolutions + ReLU
        self.enc1 = self.conv_block(in_channels, 64)
        self.enc2 = self.conv_block(64, 128)
        self.enc3 = self.conv_block(128, 256)

        # Max pooling to shrink spatial dimensions by half
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # ---- Bottleneck ----
        self.bottleneck = self.conv_block(256, 512)

        # ---- Decoder (Upsampling Path) ----
        # Transpose convolution doubles spatial dimensions
        self.up3 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
        self.dec3 = self.conv_block(512, 256)  # 512 because of skip connection

        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.dec2 = self.conv_block(256, 128)  # 256 because of skip connection

        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.dec1 = self.conv_block(128, 64)   # 128 because of skip connection

        # ---- Final 1x1 Convolution ----
        # Converts 64 feature channels to num_classes
```

```
        self.final = nn.Conv2d(64, num_classes, kernel_size=1)

    def conv_block(self, in_ch, out_ch):
        """Two convolution layers with ReLU activation."""
        return nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        # Encoder
        e1 = self.enc1(x)          # Shape: (B, 64, H, W)
        e2 = self.enc2(self.pool(e1))  # Shape: (B, 128, H/2, W/2)
        e3 = self.enc3(self.pool(e2))  # Shape: (B, 256, H/4, W/4)

        # Bottleneck
        b = self.bottleneck(self.pool(e3))  # Shape: (B, 512, H/8, W/8)

        # Decoder with skip connections
        d3 = self.up3(b)                    # Shape: (B, 256, H/4, W/4)
        d3 = torch.cat([d3, e3], dim=1)     # Concatenate skip: (B, 512, H/4, W/4)
        d3 = self.dec3(d3)                  # Shape: (B, 256, H/4, W/4)

        d2 = self.up2(d3)                   # Shape: (B, 128, H/2, W/2)
        d2 = torch.cat([d2, e2], dim=1)     # Concatenate skip: (B, 256, H/2, W/2)
        d2 = self.dec2(d2)                  # Shape: (B, 128, H/2, W/2)

        d1 = self.up1(d2)                   # Shape: (B, 64, H, W)
        d1 = torch.cat([d1, e1], dim=1)     # Concatenate skip: (B, 128, H, W)
        d1 = self.dec1(d1)                  # Shape: (B, 64, H, W)

        # Final classification layer
        out = self.final(d1)                # Shape: (B, num_classes, H, W)
        return out

# Test the model
model = SimpleUNet(in_channels=3, num_classes=5)
dummy_input = torch.randn(1, 3, 128, 128)  # Batch of 1, RGB, 128x128
output = model(dummy_input)

print("Input shape: ", dummy_input.shape)
print("Output shape:", output.shape)
print("Number of classes:", output.shape[1])
```

Output:

```
Input shape:  torch.Size([1, 3, 128, 128])
Output shape: torch.Size([1, 5, 128, 128])
Number of classes: 5
```

Line-by-line explanation:

- `self.enc1, enc2, enc3` are encoder blocks. Each contains two convolution layers with ReLU activation. They extract features at different scales.
- `self.pool = nn.MaxPool2d(kernel_size=2, stride=2)` halves the height and width, making the feature maps smaller.
- `self.bottleneck` is the bottom of the U. It processes the most compressed representation.
- `self.up3, up2, up1` are transpose convolutions (also called deconvolutions). They double the spatial dimensions, making the feature maps larger again.
- `torch.cat([d3, e3], dim=1)` concatenates the upsampled features with the encoder features along the channel dimension. This is the skip connection.
- `self.final` is a 1x1 convolution that converts 64 feature channels into the number of output classes.
- The output has the same height and width as the input, with one channel per class.

Using Pre-Trained Segmentation Models

Training a segmentation model from scratch requires large datasets and significant computing power. Fortunately, PyTorch provides pre-trained models through the `torchvision` library.

DeepLabV3 with ResNet Backbone

DeepLabV3 is a powerful segmentation model. The torchvision version is trained on the COCO dataset and can segment 21 different classes including person, car, dog, cat, and more.

```python
import torch
import torchvision
from torchvision import transforms
from PIL import Image
import numpy as np

# Load a pre-trained DeepLabV3 model
# The model comes with weights trained on COCO dataset
model = torchvision.models.segmentation.deeplabv3_resnet101(
    weights='DeepLabV3_ResNet101_Weights.DEFAULT'
)
model.eval()  # Set to evaluation mode (no training)
```

```
print("Model loaded successfully!")
print("Number of output classes: 21")
print()

# The 21 COCO classes this model can segment
classes = [
    'background', 'aeroplane', 'bicycle', 'bird', 'boat',
    'bottle', 'bus', 'car', 'cat', 'chair',
    'cow', 'diningtable', 'dog', 'horse', 'motorbike',
    'person', 'pottedplant', 'sheep', 'sofa', 'train',
    'tvmonitor'
]

for i, name in enumerate(classes):
    print(f"  Class {i:2d}: {name}")
```

Output:

```
Model loaded successfully!
Number of output classes: 21

  Class  0: background
  Class  1: aeroplane
  Class  2: bicycle
  Class  3: bird
  Class  4: boat
  Class  5: bottle
  Class  6: bus
  Class  7: car
  Class  8: cat
  Class  9: chair
  Class 10: cow
  Class 11: diningtable
  Class 12: dog
  Class 13: horse
  Class 14: motorbike
  Class 15: person
  Class 16: pottedplant
  Class 17: sheep
  Class 18: sofa
  Class 19: train
  Class 20: tvmonitor
```

Line-by-line explanation:

- `torchvision.models.segmentation.deeplabv3_resnet101(...)` loads the DeepLabV3 model with a ResNet-101 backbone. The backbone is the encoder part that extracts features.
- `weights='DeepLabV3_ResNet101_Weights.DEFAULT'` loads the best available pre-trained weights.

- `model.eval()` switches the model from training mode to evaluation mode. This disables dropout and uses stored batch normalization statistics.
- The model can identify 21 object categories. Class 0 is always "background" (everything that is not one of the 20 specific categories).

Running Segmentation on an Image

```python
import torch
import torchvision
from torchvision import transforms
from PIL import Image
import numpy as np

# Load the pre-trained model
model = torchvision.models.segmentation.deeplabv3_resnet101(
    weights='DeepLabV3_ResNet101_Weights.DEFAULT'
)
model.eval()

# Define the image preprocessing pipeline
# The model expects images normalized with ImageNet statistics
preprocess = transforms.Compose([
    transforms.Resize(520),                 # Resize shortest side to 520
    transforms.CenterCrop(480),             # Crop to 480x480
    transforms.ToTensor(),                  # Convert to tensor (0-1 range)
    transforms.Normalize(                   # Normalize with ImageNet stats
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# Create a sample image (in practice, load a real image)
# image = Image.open("street_scene.jpg").convert("RGB")
# For demonstration, create a random image
sample_image = Image.fromarray(
    np.random.randint(0, 255, (400, 600, 3), dtype=np.uint8)
)

# Preprocess the image
input_tensor = preprocess(sample_image)
input_batch = input_tensor.unsqueeze(0)  # Add batch dimension

print("Input tensor shape:", input_batch.shape)

# Run inference (no gradient calculation needed)
with torch.no_grad():
    output = model(input_batch)

# The model returns a dictionary
# 'out' contains the segmentation logits
```

```
output_logits = output['out']
print("Output logits shape:", output_logits.shape)

# Get the predicted class for each pixel
# argmax along the class dimension (dim=1) gives the most likely class
predictions = torch.argmax(output_logits, dim=1)
print("Predictions shape:", predictions.shape)

# Convert to numpy for further processing
mask = predictions.squeeze(0).numpy()
print("Mask shape:", mask.shape)
print("Unique classes found:", np.unique(mask))
```

Output:

```
Input tensor shape: torch.Size([1, 3, 480, 480])
Output logits shape: torch.Size([1, 21, 480, 480])
Predictions shape: torch.Size([1, 480, 480])
Mask shape: (480, 480)
Unique classes found: [0]
```

Line-by-line explanation:

- `transforms.Compose([...])` chains multiple image transformations together. Each one runs in order.
- `transforms.Resize(520)` resizes the image so its shortest side is 520 pixels.
- `transforms.CenterCrop(480)` crops the center 480-by-480 region. This ensures a consistent input size.
- `transforms.ToTensor()` converts the PIL image to a PyTorch tensor and scales pixel values from 0-255 to 0.0-1.0.
- `transforms.Normalize(mean, std)` normalizes the image using the ImageNet dataset statistics. This is required because the model was trained with this normalization.
- `input_tensor.unsqueeze(0)` adds a batch dimension. The model expects input shape (batch, channels, height, width).
- `with torch.no_grad()` tells PyTorch not to track gradients, which saves memory during inference.
- `output['out']` extracts the segmentation logits. The shape is (1, 21, 480, 480) meaning 21 class scores for each of the 480x480 pixels.
- `torch.argmax(output_logits, dim=1)` finds which class has the highest score for each pixel. This produces the final segmentation mask.

Visualizing Segmentation Masks

A segmentation mask is just numbers. To make it human-readable, we assign a unique color to each class.

Creating a Color Map

```python
import numpy as np
import matplotlib.pyplot as plt

def create_color_map(num_classes):
    """
    Create a color map where each class gets a unique color.
    Uses a simple algorithm to generate visually distinct colors.
    """
    colors = np.zeros((num_classes, 3), dtype=np.uint8)

    for i in range(num_classes):
        r, g, b = 0, 0, 0
        class_id = i
        for j in range(8):
            r = r | ((class_id >> 0) & 1) << (7 - j)
            g = g | ((class_id >> 1) & 1) << (7 - j)
            b = b | ((class_id >> 2) & 1) << (7 - j)
            class_id = class_id >> 3
        colors[i] = [r, g, b]

    return colors

def colorize_mask(mask, num_classes=21):
    """
    Convert a segmentation mask (2D array of class IDs)
    into an RGB image where each class has a unique color.
    """
    colors = create_color_map(num_classes)
    h, w = mask.shape
    color_mask = np.zeros((h, w, 3), dtype=np.uint8)

    for class_id in range(num_classes):
        # Find all pixels belonging to this class
        class_pixels = (mask == class_id)
        # Assign the class color to those pixels
        color_mask[class_pixels] = colors[class_id]

    return color_mask

# Example: create a small mask and colorize it
example_mask = np.array([
    [0, 0, 0, 0, 0, 15, 15, 15],
    [0, 0, 0, 0, 15, 15, 15, 15],
```

```
    [0, 8, 8, 0, 15, 15, 15, 0],
    [0, 8, 8, 0, 0, 15, 0, 0],
    [0, 8, 8, 8, 0, 0, 0, 0],
    [0, 0, 8, 0, 0, 12, 12, 0],
    [0, 0, 0, 0, 12, 12, 12, 0],
    [0, 0, 0, 0, 12, 12, 0, 0],
])

# Class 0 = background, 8 = cat, 12 = dog, 15 = person

print("Original mask (class IDs):")
print(example_mask)
print()

colored = colorize_mask(example_mask)
print("Colored mask shape:", colored.shape)
print("Color for background (class 0):", colored[0, 0])
print("Color for cat (class 8):       ", colored[2, 1])
print("Color for person (class 15):   ", colored[0, 5])

# To display with matplotlib:
# fig, axes = plt.subplots(1, 2, figsize=(12, 5))
# axes[0].imshow(original_image)
# axes[0].set_title("Original Image")
# axes[1].imshow(colored)
# axes[1].set_title("Segmentation Mask")
# plt.show()
```

Output:

```
Original mask (class IDs):
[[ 0  0  0  0  0 15 15 15]
 [ 0  0  0  0 15 15 15 15]
 [ 0  8  8  0 15 15 15  0]
 [ 0  8  8  0  0 15  0  0]
 [ 0  8  8  8  0  0  0  0]
 [ 0  0  8  0  0 12 12  0]
 [ 0  0  0  0 12 12 12  0]
 [ 0  0  0  0 12 12  0  0]]

Colored mask shape: (8, 8, 3)
Color for background (class 0): [0 0 0]
Color for cat (class 8):        [  0   0 128]
Color for person (class 15):    [128 128 128]
```

Line-by-line explanation:

- `create_color_map(num_classes)` generates a unique RGB color for each class using a bit-shifting algorithm. This ensures visually distinct colors.
- `colorize_mask(mask)` takes a 2D mask of class IDs and returns a 3D RGB image.

- `class_pixels = (mask == class_id)` creates a boolean array that is True wherever the mask equals the current class ID.
- `color_mask[class_pixels] = colors[class_id]` sets all pixels of that class to the corresponding color.
- The result is an image you can display using matplotlib, where each object class appears in a different color.

Overlaying the Mask on the Original Image

Often, you want to see the segmentation mask on top of the original image:

```python
import numpy as np

def overlay_mask(image, mask, num_classes=21, alpha=0.5):
    """
    Overlay a colored segmentation mask on the original image.

    Parameters:
        image: numpy array of shape (H, W, 3), the original image
        mask: numpy array of shape (H, W), class IDs per pixel
        num_classes: total number of classes
        alpha: transparency (0 = only image, 1 = only mask)

    Returns:
        blended: numpy array of shape (H, W, 3)
    """
    # Create colored mask
    colored_mask = colorize_mask(mask, num_classes)

    # Blend: overlay = image * (1 - alpha) + mask * alpha
    # Only overlay where mask is not background (class 0)
    blended = image.copy()
    non_background = mask > 0

    blended[non_background] = (
        image[non_background] * (1 - alpha) +
        colored_mask[non_background] * alpha
    ).astype(np.uint8)

    return blended

# Example usage:
# original = np.array(Image.open("photo.jpg"))
# blended = overlay_mask(original, mask, alpha=0.5)
# plt.imshow(blended)
# plt.show()

print("overlay_mask function defined successfully!")
print("Use it to blend segmentation results with original images.")
print("alpha=0.5 means 50% image + 50% mask color")
```

Output:

```
overlay_mask function defined successfully!
Use it to blend segmentation results with original images.
alpha=0.5 means 50% image + 50% mask color
```

Complete Segmentation Pipeline

Let us put everything together into a complete, reusable pipeline:

```python
import torch
import torchvision
from torchvision import transforms
from PIL import Image
import numpy as np

class ImageSegmenter:
    """A complete image segmentation pipeline using DeepLabV3."""

    # COCO class names
    CLASSES = [
        'background', 'aeroplane', 'bicycle', 'bird', 'boat',
        'bottle', 'bus', 'car', 'cat', 'chair',
        'cow', 'diningtable', 'dog', 'horse', 'motorbike',
        'person', 'pottedplant', 'sheep', 'sofa', 'train',
        'tvmonitor'
    ]

    def __init__(self):
        """Load the model and define preprocessing."""
        print("Loading DeepLabV3 model...")
        self.model = torchvision.models.segmentation.deeplabv3_resnet101(
            weights='DeepLabV3_ResNet101_Weights.DEFAULT'
        )
        self.model.eval()

        self.preprocess = transforms.Compose([
            transforms.Resize(520),
            transforms.CenterCrop(480),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]
            )
        ])
        print("Model ready!")
```

```
    def segment(self, image):
        """
        Segment an image.

        Parameters:
            image: PIL Image

        Returns:
            mask: numpy array of shape (H, W) with class IDs
        """
        # Preprocess
        input_tensor = self.preprocess(image).unsqueeze(0)

        # Run inference
        with torch.no_grad():
            output = self.model(input_tensor)

        # Get predictions
        predictions = torch.argmax(output['out'], dim=1)
        mask = predictions.squeeze(0).numpy()

        return mask

    def describe_scene(self, mask):
        """
        Describe what objects were found in the segmentation.

        Parameters:
            mask: numpy array with class IDs
        """
        unique_classes = np.unique(mask)
        total_pixels = mask.size

        print("\nObjects found in the image:")
        print("-" * 40)

        for class_id in unique_classes:
            class_name = self.CLASSES[class_id]
            pixel_count = np.sum(mask == class_id)
            percentage = (pixel_count / total_pixels) * 100
            print(f"  {class_name:15s}: {percentage:5.1f}% of image "
                  f"({pixel_count:,} pixels)")

# Example usage
segmenter = ImageSegmenter()

# In practice, load a real image:
# image = Image.open("street.jpg").convert("RGB")
# mask = segmenter.segment(image)
# segmenter.describe_scene(mask)

print("\nSegmenter is ready to use!")
print("Call segmenter.segment(image) with a PIL Image.")
```

Output:

```
Loading DeepLabV3 model...
Model ready!

Segmenter is ready to use!
Call segmenter.segment(image) with a PIL Image.
```

Line-by-line explanation:

- The `ImageSegmenter` class wraps the entire pipeline: model loading, preprocessing, inference, and result analysis.
- `segment(image)` takes a PIL Image and returns a numpy mask where each pixel has a class ID.
- `describe_scene(mask)` analyzes the mask and prints a summary of what was found, including the percentage of the image each object occupies.
- Using a class like this makes the code reusable and organized. You can segment many images with just `segmenter.segment(image)`.

Applications of Image Segmentation

Image segmentation powers many real-world applications:

```
+-------------------------+---------------------------------------------+
| Application             | How Segmentation Helps                      |
+-------------------------+---------------------------------------------+
| Self-Driving Cars       | Identifies road, lanes, pedestrians,        |
|                         | traffic signs at the pixel level            |
+-------------------------+---------------------------------------------+
| Medical Imaging         | Outlines tumors, organs, and blood          |
|                         | vessels in CT scans and MRIs                |
+-------------------------+---------------------------------------------+
| Satellite Analysis      | Maps forests, water bodies, urban areas,    |
|                         | and agricultural land from aerial images    |
+-------------------------+---------------------------------------------+
| Photo Editing           | Powers "remove background" and "select      |
|                         | subject" features in apps like Photoshop    |
+-------------------------+---------------------------------------------+
| Video Conferencing      | Creates virtual backgrounds by separating   |
|                         | the person from the real background         |
+-------------------------+---------------------------------------------+
| Agriculture             | Detects plant diseases, measures crop       |
|                         | coverage, identifies weeds vs crops         |
+-------------------------+---------------------------------------------+
```

```
| Robotics                | Helps robots understand their environment   |
|                         | and navigate around obstacles               |
+-------------------------+---------------------------------------------+
```

Common Mistakes

1. Forgetting to set the model to evaluation mode. Always call `model.eval()` before inference. Without it, batch normalization and dropout behave differently and produce wrong results.
2. Not normalizing the input image. Pre-trained models expect images normalized with specific mean and standard deviation values (usually ImageNet statistics). Feeding raw pixel values will produce garbage output.
3. Ignoring the output format. Segmentation models often return dictionaries (not plain tensors). For DeepLabV3 in torchvision, the main output is in `output['out']`.
4. Expecting perfect boundaries. Segmentation models are not perfect. Boundaries between objects may be slightly imprecise, especially for small or thin objects.
5. Using the wrong input size. While many models accept variable-sized inputs, using very small images will produce poor results. Stick to the recommended input sizes (typically 480-520 pixels).

Best Practices

1. Start with pre-trained models. Unless you have a very specific use case and a large labeled dataset, use pre-trained models from torchvision or Hugging Face. They save weeks of training time.
2. Resize outputs to match the original image. The model may change the image dimensions during preprocessing. Resize the output mask back to the original size if you need pixel-accurate results on the original image.
3. Use GPU for faster inference. Segmentation models are computationally expensive. Move the model and input to GPU with `.to('cuda')` for significant speedups.
4. Consider model size vs speed tradeoffs. DeepLabV3 with ResNet-101 is accurate but slow. For real-time applications, consider lighter models like FCN with ResNet-50 or mobile-optimized architectures.

5. Post-process the mask. Apply morphological operations (like opening and closing) to clean up noisy segmentation boundaries.

Quick Summary

Image segmentation assigns a class label to every pixel in an image, going beyond classification (one label per image) and detection (one box per object). Semantic segmentation labels pixels by class but does not distinguish individual objects, while instance segmentation identifies each object separately. The U-Net architecture, shaped like a U, uses an encoder to capture features, a decoder to restore spatial resolution, and skip connections to preserve fine details. Pre-trained models like DeepLabV3 from torchvision can segment images into 21 categories out of the box. The output is a segmentation mask that can be colorized and overlaid on the original image for visualization.

Key Points

- Image segmentation classifies every pixel, not just the whole image
- Semantic segmentation assigns class labels to pixels (all cars are "car")
- Instance segmentation also distinguishes individual objects (car #1, car #2)
- U-Net has an encoder path, decoder path, and skip connections shaped like a "U"
- Skip connections pass fine-grained details from encoder to decoder
- Pre-trained models like DeepLabV3 provide accurate segmentation without training
- Segmentation masks are 2D arrays where each value is a class ID
- Colorization maps class IDs to colors for human-readable visualization

---

Practice Questions

1. What is the key difference between semantic segmentation and instance segmentation? Give an example where instance segmentation is needed but semantic segmentation is not enough.
2. Why does U-Net use skip connections? What information would be lost without them?

3. If a segmentation model has 10 classes and the input image is 256x256, what is the shape of the model's output tensor? Explain each dimension.
4. Why must we normalize images with ImageNet statistics before feeding them to a pre-trained segmentation model?
5. A self-driving car needs to know exactly which pixels are road and which are sidewalk. Would you use classification, detection, or segmentation? Explain why.

Exercises

Exercise 1: Analyze a Segmentation Mask

Create a 10x10 numpy array representing a segmentation mask with three classes (background=0, tree=1, house=2). Count the number of pixels for each class and calculate the percentage each class occupies.

Exercise 2: Build a Segmentation Visualizer

Write a function that takes a segmentation mask and a dictionary mapping class IDs to class names, and prints a "legend" showing which color corresponds to which class. Test it with a mask containing at least 4 different classes.

Exercise 3: Compare Two Segmentation Masks

Write a function that takes two segmentation masks (predicted and ground truth) and calculates the Intersection over Union (IoU) for each class. IoU = (area of overlap) / (area of union). This is the standard metric for evaluating segmentation quality.

What Is Next?

Now that you can segment images at the pixel level, the next chapter takes on a specific and important challenge: faces. You will learn how to detect faces in images using Haar cascades and MTCNN, how face recognition works through embeddings, and the critical ethical considerations around facial recognition technology. Face detection combines many of the techniques you have learned so far, from image preprocessing to deep learning models, and adds important discussions about bias, privacy, and responsible AI.

Chapter 22: Face Detection and Recognition

What You Will Learn

In this chapter, you will learn:

- The difference between face detection and face recognition
- How Haar cascade classifiers work in OpenCV
- How to use MTCNN for more accurate face detection
- What face embeddings are and how FaceNet represents faces as numbers
- How to use the face_recognition library for a complete pipeline
- Critical ethical considerations: bias, privacy, and consent
- How to build a complete face detection and recognition example

Why This Chapter Matters

Face detection and recognition are among the most widely deployed computer vision technologies in the world. Your phone unlocks with your face. Social media apps suggest who to tag in your photos. Security cameras at airports scan crowds for persons of interest.

But this technology also carries enormous responsibility. Facial recognition systems have been shown to have higher error rates for certain demographic groups. They raise serious privacy concerns when used for mass surveillance. Several cities have banned their use by law enforcement.

As a developer, understanding how this technology works is essential, not just for building it, but for knowing when it should and should not be used. This chapter teaches you the technical foundations AND the ethical framework you need to work responsibly with facial recognition.

Face Detection vs Face Recognition

These two terms are often confused, but they are fundamentally different tasks.

Face Detection

Face detection answers the question: "Where are the faces in this image?"

The output is a set of bounding boxes showing the location of each face. Face detection does NOT identify who the person is. It simply finds faces.

```
Face Detection:

Original Image          Detection Result
+------------------+   +------------------+
|                  |   |                  |
|   O    O    O    |   |  [O]  [O]  [O]  |   <- Boxes around faces
|  /|\  /|\  /|\   |   |  /|\  /|\  /|\  |
|  / \  / \  / \   |   |  / \  / \  / \  |
|                  |   |                  |
+------------------+   +------------------+

Output: [(x1, y1, w1, h1), (x2, y2, w2, h2), (x3, y3, w3, h3)]
        "There are 3 faces at these locations"
        (No names, no identities)
```

Face Recognition

Face recognition answers the question: "Whose face is this?"

It takes a detected face and compares it against a database of known faces to determine the person's identity.

```
Face Recognition:

Detected Face       Database             Result
+--------+         +---------+
|        |         | Alice   |
|  Face  | ------> | Bob     |  -------> "This is Bob"
|        |         | Charlie |
+--------+         +---------+

Recognition requires detection first.
You cannot recognize a face you have not found.
```

The Two-Step Pipeline

```
Step 1: DETECTION                Step 2: RECOGNITION
"Find all faces"                 "Identify each face"

Image --> [Detector] --> Faces --> [Recognizer] --> Names
                         (boxes)                    (identities)
```

Think of it like a school attendance system:

- Detection is like scanning the classroom and counting how many students are present
- Recognition is like looking at each student and checking off their name on the attendance list

---

Haar Cascade Classifiers (OpenCV)

Haar cascade is one of the oldest and fastest methods for face detection. It was introduced in 2001 by Paul Viola and Michael Jones, and it is built into OpenCV.

How Haar Cascades Work

The algorithm uses simple rectangular features (called Haar-like features) to detect patterns in an image. These features measure the difference in intensity between adjacent rectangular regions.

```
Haar-like Features:

Feature 1 (Edge):     Feature 2 (Line):    Feature 3 (Diagonal):
+------+------+       +------+------+      +---+---+
|      |      |       |      |      |      |   |   |
| Dark |Light |       |Light | Dark |      | D | L |
|      |      |       |      |      |      +---+---+
+------+------+       +------+------+      | L | D |
                                           +---+---+

The algorithm slides these features across the image.
When a feature matches the expected pattern, it scores a point.
Many features together form a "cascade" of classifiers.
```

The "cascade" part means the algorithm works in stages:

1. Stage 1 uses a few simple features to quickly reject areas that are clearly NOT faces
2. Stage 2 uses more features on the remaining candidates
3. Stage 3 uses even more features
4. Only areas that pass ALL stages are declared faces

```
Cascade Process:

All Image Regions (10,000)
          |
    [Stage 1: Quick check]  -> Reject 8,000 (clearly not faces)
          |
    Remaining: 2,000
          |
    [Stage 2: More detail]  -> Reject 1,500
          |
    Remaining: 500
          |
    [Stage 3: Fine check]   -> Reject 450
          |
    Final Detections: 50 face candidates
```

This cascade design makes the algorithm very fast because most image regions are rejected early.

Face Detection with OpenCV

```python
import cv2
import numpy as np

# OpenCV comes with pre-trained Haar cascade files
# Load the face detection cascade
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

# Create a sample image with a simple "face-like" pattern
# In practice, you would load a real image:
# image = cv2.imread('group_photo.jpg')
image = np.ones((400, 600, 3), dtype=np.uint8) * 200  # Light gray

# Convert to grayscale (Haar cascades work on grayscale images)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces
# Parameters:
#   gray: the grayscale image
#   scaleFactor: how much to shrink the image at each scale (1.1 = 10%)
#   minNeighbors: how many neighbors each candidate needs to be kept
```

```
#   minSize: minimum face size to detect
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30)
)

print(f"Number of faces detected: {len(faces)}")
print()

# Draw rectangles around detected faces
for i, (x, y, w, h) in enumerate(faces):
    print(f"Face {i+1}: position=({x}, {y}), size=({w}x{h})")
    # Draw a green rectangle
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Save the result
# cv2.imwrite('detected_faces.jpg', image)
print("\nNote: Use a real photo to see actual face detections!")
```

Output:

```
Number of faces detected: 0

Note: Use a real photo to see actual face detections!
```

Line-by-line explanation:

- `cv2.CascadeClassifier(...)` loads a pre-trained cascade classifier. OpenCV includes several cascade files; `haarcascade_frontalface_default.xml` is for detecting front-facing faces.
- `cv2.data.haarcascades` gives the path to OpenCV's built-in cascade files. You do not need to download them separately.
- `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)` converts the image from color (BGR in OpenCV) to grayscale. Haar cascades only work on grayscale images.
- `face_cascade.detectMultiScale(...)` runs the detection. It slides a detection window across the image at multiple scales.
- `scaleFactor=1.1` means the image is shrunk by 10% at each scale. Smaller values are more thorough but slower.
- `minNeighbors=5` means a region must be detected by at least 5 overlapping windows to be considered a face. Higher values reduce false positives but may miss some faces.
- `minSize=(30, 30)` sets the minimum face size in pixels. Faces smaller than 30x30 are ignored.

- The function returns a list of `(x, y, w, h)` tuples, where `(x, y)` is the top-left corner and `(w, h)` are the width and height of each detected face.

Haar Cascade Limitations

```
+----------------------------+-----------------------------------------+
| Limitation                 | Description                             |
+----------------------------+-----------------------------------------+
| Frontal faces only         | Does not detect faces turned sideways   |
| Sensitive to lighting      | Struggles with shadows or backlighting  |
| Many false positives       | May detect non-face objects as faces    |
| Fixed features             | Cannot learn new patterns               |
| No landmark detection      | Does not find eyes, nose, mouth         |
+----------------------------+-----------------------------------------+
```

For more robust face detection, modern deep learning methods like MTCNN are preferred.

MTCNN: Multi-Task Cascaded Convolutional Networks

MTCNN is a deep learning based face detector that is more accurate than Haar cascades. It uses three neural networks in sequence, each refining the results of the previous one.

How MTCNN Works

```
MTCNN Architecture:

Input Image
     |
     v
[P-Net] Proposal Network
  - Quickly scans the image at multiple scales
  - Proposes candidate face regions
  - Very fast, but many false positives
     |
     v
[R-Net] Refine Network
  - Takes candidate regions from P-Net
  - Filters out false positives
  - Adjusts bounding box positions
     |
     v
```

```
[O-Net] Output Network
  - Final refinement of bounding boxes
  - Detects 5 facial landmarks:
    left eye, right eye, nose, left mouth, right mouth
  - Produces final high-quality detections
     |
     v
Final Result: Bounding boxes + confidence + landmarks
```

Using MTCNN in Python

```python
# Install: pip install mtcnn tensorflow
from mtcnn import MTCNN
import cv2
import numpy as np

# Create the MTCNN detector
detector = MTCNN()

# Load an image (use a real photo for actual results)
# image = cv2.imread('group_photo.jpg')
# image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# For demonstration, we show the expected output format
print("MTCNN Detector created successfully!")
print()
print("Expected output format for each detected face:")
print({
    'box': [120, 80, 65, 75],
    'confidence': 0.9987,
    'keypoints': {
        'left_eye': (140, 105),
        'right_eye': (170, 103),
        'nose': (155, 120),
        'mouth_left': (142, 138),
        'mouth_right': (168, 136)
    }
})
print()
print("Fields explained:")
print("  box: [x, y, width, height] of the face bounding box")
print("  confidence: probability that this is a face (0 to 1)")
print("  keypoints: locations of 5 facial landmarks")
```

Output:

```
MTCNN Detector created successfully!

Expected output format for each detected face:
```

```
{'box': [120, 80, 65, 75], 'confidence': 0.9987, 'keypoints': {'left_eye': (140, 105), ...

Fields explained:
  box: [x, y, width, height] of the face bounding box
  confidence: probability that this is a face (0 to 1)
  keypoints: locations of 5 facial landmarks
```

Drawing MTCNN Results

```python
import cv2
import numpy as np

def draw_mtcnn_results(image, detections):
    """
    Draw bounding boxes and facial landmarks on an image.

    Parameters:
        image: numpy array (BGR format)
        detections: list of dictionaries from MTCNN
    """
    result = image.copy()

    for detection in detections:
        # Draw bounding box
        x, y, w, h = detection['box']
        confidence = detection['confidence']

        # Green rectangle for the face
        cv2.rectangle(result, (x, y), (x + w, y + h), (0, 255, 0), 2)

        # Confidence label above the box
        label = f"{confidence:.2%}"
        cv2.putText(result, label, (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

        # Draw facial landmarks as small circles
        keypoints = detection['keypoints']
        for name, point in keypoints.items():
            cv2.circle(result, point, 3, (0, 0, 255), -1)  # Red dots

    return result

# Example with simulated detections
print("draw_mtcnn_results function defined!")
print()
print("Facial landmarks detected by MTCNN:")
print("  - Left eye:    helps with face alignment")
print("  - Right eye:   helps with face alignment")
print("  - Nose:        center reference point")
print("  - Mouth left:  expression detection")
print("  - Mouth right: expression detection")
```

Output:

```
draw_mtcnn_results function defined!

Facial landmarks detected by MTCNN:
  - Left eye:    helps with face alignment
  - Right eye:   helps with face alignment
  - Nose:        center reference point
  - Mouth left:  expression detection
  - Mouth right: expression detection
```

Face Embeddings and FaceNet

Once you have detected a face, how do you recognize whose face it is? The answer is face embeddings.

What Are Face Embeddings?

A face embedding is a numerical representation of a face. It converts a face image into a list of numbers (a vector) that captures the unique features of that face.

```
Face Embedding Process:

Face Image (160x160x3)
  = 76,800 pixel values
         |
    [Neural Network]
         |
  Embedding Vector (128 numbers)
  [0.12, -0.45, 0.78, 0.33, ..., -0.21]

Key properties:
  - Same person's faces -> SIMILAR embeddings (close together)
  - Different people's faces -> DIFFERENT embeddings (far apart)
```

Think of it like a fingerprint, but for faces. Just as each person has a unique fingerprint, each person's face produces a unique embedding vector.

How FaceNet Works (Conceptually)

FaceNet, developed by Google in 2015, learns to map faces to 128-dimensional vectors using a technique called triplet loss.

```
Triplet Loss Training:

Anchor (Alice photo 1)   Positive (Alice photo 2)   Negative (Bob photo)
       |                         |                         |
  [Same Network]            [Same Network]            [Same Network]
       |                         |                         |
   Embedding A               Embedding P               Embedding N

Goal: distance(A, P) should be SMALL  (same person)
      distance(A, N) should be LARGE  (different person)

Loss = max(0, distance(A,P) - distance(A,N) + margin)

The network learns to pull same-person embeddings together
and push different-person embeddings apart.
```

Comparing Face Embeddings

```python
import numpy as np

def compare_faces(embedding1, embedding2, threshold=0.6):
    """
    Compare two face embeddings to determine if they are
    the same person.

    Parameters:
        embedding1: numpy array, first face embedding
        embedding2: numpy array, second face embedding
        threshold: maximum distance to consider same person

    Returns:
        is_match: boolean
        distance: float
    """
    # Calculate Euclidean distance between embeddings
    distance = np.linalg.norm(embedding1 - embedding2)

    is_match = distance < threshold

    return is_match, distance

# Simulate embeddings (in practice, a neural network generates these)
np.random.seed(42)

# Alice's embeddings (two photos of the same person)
# They should be similar (small distance)
alice_photo1 = np.random.randn(128) * 0.1 + np.array([1.0] * 128)
alice_photo2 = alice_photo1 + np.random.randn(128) * 0.05  # Slight variation
```

```
# Bob's embedding (different person)
# Should be different from Alice (large distance)
bob_photo = np.random.randn(128) * 0.1 + np.array([-1.0] * 128)

# Compare Alice photo 1 vs Alice photo 2 (same person)
match, dist = compare_faces(alice_photo1, alice_photo2)
print(f"Alice vs Alice: distance = {dist:.4f}, match = {match}")

# Compare Alice vs Bob (different people)
match, dist = compare_faces(alice_photo1, bob_photo)
print(f"Alice vs Bob:   distance = {dist:.4f}, match = {match}")
```

Output:

```
Alice vs Alice: distance = 0.5561, match = True
Alice vs Bob:   distance = 22.6692, match = False
```

Line-by-line explanation:

- `np.linalg.norm(embedding1 - embedding2)` calculates the Euclidean distance between two vectors. This measures how "far apart" the two face representations are in the 128-dimensional space.
- If the distance is below the threshold (0.6), we consider the faces to be the same person.
- Alice's two photos produce embeddings that are very close together (distance 0.56) because they are the same person.
- Alice's and Bob's embeddings are far apart (distance 22.67) because they are different people.

Using the face_recognition Library

The `face_recognition` library provides a simple, high-level API for face detection and recognition. It uses dlib's deep learning models under the hood.

```python
# Install: pip install face_recognition
# Note: requires dlib, which may need cmake installed

import face_recognition
import numpy as np
from PIL import Image
```

```
# ---- Step 1: Load images and find faces ----
# In practice:
# known_image = face_recognition.load_image_file("alice.jpg")
# unknown_image = face_recognition.load_image_file("mystery_person.jpg")

# For demonstration, show the API:
print("face_recognition library API:")
print()
print("1. Load an image:")
print('   image = face_recognition.load_image_file("photo.jpg")')
print()
print("2. Find face locations:")
print('   locations = face_recognition.face_locations(image)')
print('   # Returns: [(top, right, bottom, left), ...]')
print()
print("3. Get face encodings (embeddings):")
print('   encodings = face_recognition.face_encodings(image)')
print('   # Returns: [array of 128 numbers, ...]')
print()
print("4. Compare faces:")
print('   results = face_recognition.compare_faces(')
print('       known_encodings, unknown_encoding)')
print('   # Returns: [True, False, False, ...]')
print()
print("5. Get face distances:")
print('   distances = face_recognition.face_distance(')
print('       known_encodings, unknown_encoding)')
print('   # Returns: [0.35, 0.89, 0.92, ...]')
```

Output:

```
face_recognition library API:

1. Load an image:
   image = face_recognition.load_image_file("photo.jpg")

2. Find face locations:
   locations = face_recognition.face_locations(image)
   # Returns: [(top, right, bottom, left), ...]

3. Get face encodings (embeddings):
   encodings = face_recognition.face_encodings(image)
   # Returns: [array of 128 numbers, ...]

4. Compare faces:
   results = face_recognition.compare_faces(
       known_encodings, unknown_encoding)
   # Returns: [True, False, False, ...]

5. Get face distances:
   distances = face_recognition.face_distance(
       known_encodings, unknown_encoding)
   # Returns: [0.35, 0.89, 0.92, ...]
```

Complete Face Recognition Example

```python
# Complete Face Recognition System
# This example shows the full pipeline

import face_recognition
import cv2
import numpy as np
import os

class FaceRecognitionSystem:
    """A simple face recognition system."""

    def __init__(self):
        self.known_encodings = []
        self.known_names = []
        print("Face Recognition System initialized.")

    def register_person(self, image_path, name):
        """
        Register a person's face in the system.

        Parameters:
            image_path: path to a photo containing the person's face
            name: the person's name
        """
        # Load the image
        image = face_recognition.load_image_file(image_path)

        # Find face encodings in the image
        encodings = face_recognition.face_encodings(image)

        if len(encodings) == 0:
            print(f"No face found in {image_path}")
            return False

        if len(encodings) > 1:
            print(f"Multiple faces found. Using the first one.")

        # Store the encoding and name
        self.known_encodings.append(encodings[0])
        self.known_names.append(name)
        print(f"Registered: {name}")
        return True

    def identify_faces(self, image_path, tolerance=0.6):
        """
        Identify faces in an image.

        Parameters:
            image_path: path to the image to analyze
```

```
            tolerance: how strict the matching is (lower = stricter)

        Returns:
            list of (name, location) tuples
        """
        # Load the image
        image = face_recognition.load_image_file(image_path)

        # Find all face locations and encodings
        locations = face_recognition.face_locations(image)
        encodings = face_recognition.face_encodings(image, locations)

        results = []

        for encoding, location in zip(encodings, locations):
            # Compare against all known faces
            matches = face_recognition.compare_faces(
                self.known_encodings, encoding, tolerance=tolerance
            )

            name = "Unknown"

            if True in matches:
                # Find the best match (smallest distance)
                distances = face_recognition.face_distance(
                    self.known_encodings, encoding
                )
                best_match_index = np.argmin(distances)

                if matches[best_match_index]:
                    name = self.known_names[best_match_index]

            results.append((name, location))

        return results

    def annotate_image(self, image_path, output_path):
        """
        Draw boxes and names on the image.
        """
        image = cv2.imread(image_path)
        results = self.identify_faces(image_path)

        for name, (top, right, bottom, left) in results:
            # Draw box
            cv2.rectangle(image, (left, top), (right, bottom),
                          (0, 255, 0), 2)

            # Draw name label
            cv2.rectangle(image, (left, bottom - 25),
                          (right, bottom), (0, 255, 0), cv2.FILLED)
            cv2.putText(image, name, (left + 6, bottom - 6),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                        (255, 255, 255), 1)
```

```
        cv2.imwrite(output_path, image)
        print(f"Saved annotated image to {output_path}")

        return results

# Usage example (with real images):
# system = FaceRecognitionSystem()
#
# # Register known people
# system.register_person("alice.jpg", "Alice")
# system.register_person("bob.jpg", "Bob")
#
# # Identify faces in a group photo
# results = system.identify_faces("group_photo.jpg")
# for name, location in results:
#     print(f"  Found {name} at {location}")

print("FaceRecognitionSystem class defined successfully!")
print()
print("Pipeline:")
print("  1. Register known people with register_person()")
print("  2. Identify faces with identify_faces()")
print("  3. Annotate images with annotate_image()")
```

Output:

```
FaceRecognitionSystem class defined successfully!

Pipeline:
  1. Register known people with register_person()
  2. Identify faces with identify_faces()
  3. Annotate images with annotate_image()
```

Line-by-line explanation:

- `FaceRecognitionSystem` manages a database of known faces (encodings and names).
- `register_person` loads a photo, extracts the face embedding, and stores it with the person's name.
- `identify_faces` takes a new image, finds all faces, computes their embeddings, and compares each embedding against the database.
- `face_recognition.compare_faces(known, unknown, tolerance)` returns a list of True/False values indicating which known faces match.
- `face_recognition.face_distance(known, unknown)` returns the numerical distance for each comparison. The smallest distance is the best match.
- `tolerance=0.6` is the default threshold. Lower values (like 0.4) are stricter and reduce false matches but may miss true matches.

Ethical Considerations

This section is critically important. Facial recognition technology has the power to cause significant harm if used irresponsibly.

Bias in Facial Recognition

Research has repeatedly shown that facial recognition systems perform unevenly across demographic groups.

```
Error Rate Disparities (Example from Research):

+---------------------------+------------------+
| Demographic Group         | Error Rate       |
+---------------------------+------------------+
| Light-skinned males       | 0.8%             |
| Light-skinned females     | 1.7%             |
| Dark-skinned males        | 6.0%             |
| Dark-skinned females      | 20.8% - 34.7%    |
+---------------------------+------------------+

These are not hypothetical numbers. Real studies have found
dramatic differences in accuracy across groups.
```

Why does this bias exist?

1. Training data imbalance. If the training dataset contains mostly light-skinned faces, the model learns those features better and performs worse on faces it has seen fewer of.
2. Historical bias in datasets. Many benchmark datasets have been collected without consideration for diversity.
3. Evaluation gaps. Models are often evaluated on overall accuracy, which hides poor performance on minority groups.

What this means in practice:

If a facial recognition system with a 20% error rate for dark-skinned females is used by law enforcement, 1 in 5 dark-skinned women could be falsely identified as someone else. This has already happened. Innocent people have been wrongfully arrested due to facial recognition errors.

Privacy Concerns

```
Privacy Issues with Facial Recognition:

1. MASS SURVEILLANCE
   - Cameras + face recognition = tracking people everywhere
   - People cannot "opt out" of having a face
   - Unlike a phone, you cannot leave your face at home

2. FUNCTION CREEP
   - System built for "security" gets used for advertising
   - System built for "attendance" gets used for surveillance
   - Original purpose expands without consent

3. DATA BREACHES
   - Unlike a password, you cannot change your face
   - If face data is stolen, the person is compromised forever
   - Biometric data breaches are permanent

4. CHILLING EFFECTS
   - People change behavior when they know they are watched
   - Discourages protest, free speech, free assembly
   - Creates power imbalance between watchers and watched
```

Consent

Before using facial recognition on anyone, consider these principles:

```
Consent Checklist:

[ ] Has the person explicitly agreed to have their face scanned?
[ ] Do they understand what the data will be used for?
[ ] Can they withdraw consent and have their data deleted?
[ ] Are they informed about how long the data is stored?
[ ] Is there a legitimate purpose that cannot be achieved
    another way?
[ ] Have you considered the impact on people who have NOT
    consented (bystanders in public spaces)?
```

Responsible Development Guidelines

```python
# Guidelines for responsible facial recognition development

responsible_guidelines = {
    "1. Purpose Limitation": (
        "Only use facial recognition for clearly defined, "
        "legitimate purposes. Never repurpose face data "
        "without new consent."
```

```
    ),
    "2. Bias Testing": (
        "Test your system across diverse demographic groups. "
        "Report accuracy broken down by age, gender, "
        "and skin tone. Do not deploy if accuracy is "
        "significantly different across groups."
    ),
    "3. Informed Consent": (
        "Always obtain explicit consent before enrolling "
        "someone in a face recognition system. Consent "
        "must be freely given, specific, and revocable."
    ),
    "4. Data Minimization": (
        "Store only what you need. Delete face data when "
        "it is no longer necessary. Use encryption for "
        "all stored biometric data."
    ),
    "5. Transparency": (
        "Clearly inform people when facial recognition "
        "is in use. Post visible notices. Provide a way "
        "for people to ask questions or raise concerns."
    ),
    "6. Human Oversight": (
        "Never make critical decisions based solely on "
        "facial recognition. Always have a human review "
        "matches before taking action, especially in "
        "law enforcement contexts."
    ),
    "7. Right to Opt Out": (
        "Provide alternatives for people who do not want "
        "to use facial recognition. For example, a PIN "
        "code or ID card as an alternative to face unlock."
    ),
    "8. Regular Audits": (
        "Periodically audit your system for bias, accuracy, "
        "and misuse. Update training data and models to "
        "address any issues found."
    )
}

print("Responsible Facial Recognition Guidelines")
print("=" * 50)
for key, value in responsible_guidelines.items():
    print(f"\n{key}:")
    print(f"  {value}")
```

Output:

```
Responsible Facial Recognition Guidelines
==================================================

1. Purpose Limitation:
  Only use facial recognition for clearly defined, legitimate purposes. Never repurpose...
```

```
2. Bias Testing:
  Test your system across diverse demographic groups. Report accuracy broken down by ag...

3. Informed Consent:
  Always obtain explicit consent before enrolling someone in a face recognition system....

4. Data Minimization:
  Store only what you need. Delete face data when it is no longer necessary. Use encryp...

5. Transparency:
  Clearly inform people when facial recognition is in use. Post visible notices. Provid...

6. Human Oversight:
  Never make critical decisions based solely on facial recognition. Always have a human...

7. Right to Opt Out:
  Provide alternatives for people who do not want to use facial recognition. For exampl...

8. Regular Audits:
  Periodically audit your system for bias, accuracy, and misuse. Update training data a...
```

When NOT to Use Facial Recognition

```
DO NOT use facial recognition for:

x  Mass surveillance of public spaces without consent
x  Tracking employees without their knowledge
x  Profiling people based on ethnicity or appearance
x  Making automated decisions about hiring, lending, or housing
x  Targeting specific demographic groups
x  Any purpose where an error could lead to arrest or detention
   without human oversight

CONSIDER using facial recognition for:

✓  Phone unlock (user's own face, with consent)
✓  Accessibility features (helping visually impaired users)
✓  Photo organization (personal photo libraries, local only)
✓  Research (with IRB approval and proper consent)
```

Haar Cascade vs MTCNN Comparison

```
+-------------------+------------------+----------------------+
| Feature           | Haar Cascade     | MTCNN                |
+-------------------+------------------+----------------------+
| Speed             | Very fast        | Moderate             |
| Accuracy          | Moderate         | High                 |
| Rotated faces     | Poor             | Better               |
| Partial faces     | Poor             | Better               |
| Facial landmarks  | No               | Yes (5 points)       |
| Dependencies      | OpenCV only      | TensorFlow + mtcnn   |
| GPU required      | No               | Optional but helpful |
| Best for          | Quick prototypes | Production systems   |
+-------------------+------------------+----------------------+
```

Common Mistakes

1. Not converting color spaces. OpenCV uses BGR format, but face_recognition and MTCNN expect RGB. Always convert with `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` when loading images with OpenCV.
2. Using default tolerance without testing. The default tolerance (0.6) may be too loose or too strict for your use case. Always test with your specific data.
3. Ignoring bias. Testing only on a narrow demographic group and assuming the system works for everyone is a serious and common mistake.
4. Storing raw face images instead of embeddings. Embeddings are smaller and cannot be reversed back into images, making them more privacy-friendly.
5. Deploying without consent mechanisms. Launching a system that captures and processes faces without informing the people being scanned is both unethical and illegal in many jurisdictions.

Best Practices

1. Use MTCNN or modern detectors over Haar cascades for production applications. Haar cascades are useful for quick prototypes but not accurate enough for real-world deployment.
2. Always align faces before computing embeddings. Use the detected landmarks (eye positions) to rotate and align faces to a standard orientation. This significantly improves recognition accuracy.

3. Set appropriate thresholds for your use case. High-security applications (bank access) should use a stricter threshold (lower distance). Convenience applications (phone unlock) can be slightly more lenient.
4. Test across demographics. Before deploying, evaluate your system on diverse test sets. Report accuracy per demographic group, not just overall accuracy.
5. Implement a human-in-the-loop. For any high-stakes decision, require a human to verify the facial recognition result before taking action.
6. Conduct a privacy impact assessment. Before building any facial recognition system, document what data you collect, why you need it, how it is stored, and who has access.

Quick Summary

Face detection finds where faces are in an image (bounding boxes), while face recognition identifies whose face it is (matching against a database). Haar cascade classifiers are fast but limited, working by sliding simple rectangular feature detectors across the image in stages. MTCNN uses three neural networks in sequence for more accurate detection with facial landmarks. Face recognition works by converting faces into numerical embeddings (128-dimensional vectors) and comparing distances: same person embeddings are close together, different person embeddings are far apart. The face_recognition library provides a simple API for the entire pipeline. Most importantly, facial recognition raises serious ethical concerns including bias across demographic groups, privacy invasion, and the need for informed consent. Responsible development requires bias testing, transparency, human oversight, and purpose limitation.

Key Points

- Face detection finds faces (where); face recognition identifies faces (who)
- Haar cascades are fast and simple but only work well for frontal faces
- MTCNN uses three neural networks for accurate detection with landmarks
- Face embeddings are 128-number vectors that represent a face uniquely
- FaceNet trains using triplet loss to pull same-person embeddings together
- Threshold tuning is critical: too loose causes false matches, too strict misses true matches
- Bias is real: systems perform worse on underrepresented groups

- Consent and privacy must be addressed before deploying any facial recognition system
- Human oversight is essential for any high-stakes application

---

Practice Questions

1. Explain the difference between face detection and face recognition. Why must detection always happen before recognition?
2. How does the cascade structure in Haar cascade classifiers make them fast? What is the tradeoff?
3. What is a face embedding, and why is Euclidean distance between embeddings a useful measure for face recognition?
4. Why do facial recognition systems often perform worse on certain demographic groups? What can developers do to mitigate this?
5. A company wants to use facial recognition to track employee attendance without telling employees. List three ethical problems with this approach.

Exercises

Exercise 1: Face Detection Comparison

Write a script that loads an image and runs face detection using both Haar cascades (OpenCV) and MTCNN. Compare the number of faces detected, the positions of the bounding boxes, and the execution time for each method. Which one is faster? Which one finds more faces?

Exercise 2: Face Similarity Score

Create a function that takes two face images, extracts their embeddings using the face_recognition library, calculates the distance, and returns a "similarity percentage" (100% = identical, 0% = completely different). Test it with photos of the same person and different people.

Exercise 3: Ethics Case Study

A shopping mall wants to install facial recognition cameras to identify repeat shoplifters. Write a document that: (a) lists the potential benefits, (b) lists the ethical concerns, (c) proposes safeguards to protect innocent shoppers, and (d) suggests an alternative approach that achieves a similar goal with less privacy invasion.

What Is Next?

In the next chapter, you will explore Vision Transformers (ViT), a revolutionary architecture that applies the same transformer mechanism used in NLP to image understanding. Instead of using convolutions like CNNs, ViT splits an image into patches and treats them like words in a sentence. You will learn how this works, when to use ViT versus traditional CNNs, and how to use pre-trained ViT models from Hugging Face.

Chapter 23: Vision Transformers (ViT)

What You Will Learn

In this chapter, you will learn:

- How the transformer architecture moved from NLP to computer vision
- What image patches are and why they are treated like tokens
- How patch embedding works step by step
- What position encoding does for images
- What the classification token (CLS token) is and why it exists
- How Vision Transformers compare to CNNs
- How to use a pre-trained ViT model with Hugging Face
- When to choose ViT over CNN and vice versa

Why This Chapter Matters

For years, Convolutional Neural Networks (CNNs) were the undisputed kings of computer vision. Then in 2020, a paper from Google called "An Image Is Worth 16x16 Words" changed everything. The researchers showed that the transformer architecture, originally designed for processing text, could match or beat the best CNNs at image classification, without using convolutions at all.

This was a breakthrough because it opened the door to a unified architecture for both text and images. Instead of having separate model families for NLP (transformers) and vision (CNNs), you could use the same fundamental building blocks for both. Today, Vision Transformers power many state-of-the-art models in image classification, object detection, segmentation, and even multimodal AI that understands both text and images together.

Understanding ViT is essential because it represents the future direction of computer vision and the convergence of NLP and vision techniques.

From Text Tokens to Image Patches

Transformers were built for sequences of tokens, like words in a sentence. But an image is not a sequence of words. So how do you feed an image into a transformer?

The answer is surprisingly simple: split the image into small patches and treat each patch like a word.

Text Transformer Recap

In NLP, a transformer processes a sequence of tokens:

```
Sentence: "The cat sat on the mat"

Tokens:   ["The", "cat", "sat", "on", "the", "mat"]
              |       |       |      |       |       |
          [embed] [embed] [embed] [embed] [embed] [embed]
              |       |       |      |       |       |
          [  e1  ,   e2  ,  e3  ,  e4  ,  e5  ,  e6  ]
                                |
                       [Transformer Layers]
                                |
                           [Output]
```

Vision Transformer: Patches as Tokens

A Vision Transformer splits the image into a grid of non-overlapping patches. Each patch becomes a "token" in the sequence:

```
Original Image (224 x 224 pixels):

+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 |P10 |P11 |P12 |P13 |P14 |
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
|    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|P15 |P16 |P17 |P18 |P19 |P20 | .. | .. | .. | .. | .. | .. | .. | .. |
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
| .. | .. | .. | .. | .. | .. | .. | .. | .. | .. | .. | .. | .. | .. |
```

```
  .                                ...
|    |    |    |    |    |    |    |    |    |    |    |    |    |P196|
+----+----+----+----+----+----+----+----+----+----+----+----+----+----+

With 16x16 patches on a 224x224 image:
  224 / 16 = 14 patches per row
  14 x 14 = 196 patches total

Each patch: 16 x 16 x 3 (RGB) = 768 values
```

The analogy is straightforward:

```
NLP Transformer:
  Sentence = sequence of WORDS
  Each word -> embedding vector

Vision Transformer:
  Image = sequence of PATCHES
  Each patch -> embedding vector
```

Think of it like cutting a photograph into puzzle pieces. Each piece captures a small part of the image, and the transformer's job is to understand how all the pieces relate to each other to see the full picture.

How ViT Works: Step by Step

The complete ViT pipeline has five main steps:

```
ViT Architecture Overview:

Input Image (224x224x3)
       |
  [1. Split into patches]     -> 196 patches of 16x16x3
       |
  [2. Patch Embedding]        -> 196 vectors of size 768
       |
  [3. Add CLS token]          -> 197 vectors (1 CLS + 196 patches)
       |
  [4. Add Position Encoding]  -> 197 vectors with position info
       |
  [5. Transformer Encoder]    -> 197 output vectors
       |
  [6. Classification Head]    -> Class prediction
       (uses only the CLS token output)
```

Step 1: Split Image into Patches

```python
import torch
import torch.nn as nn
import numpy as np

def split_into_patches(image, patch_size=16):
    """
    Split an image into non-overlapping patches.

    Parameters:
        image: tensor of shape (C, H, W)
        patch_size: size of each square patch (default 16)

    Returns:
        patches: tensor of shape (num_patches, patch_size * patch_size * C)
    """
    C, H, W = image.shape

    # Calculate number of patches
    num_patches_h = H // patch_size  # Patches along height
    num_patches_w = W // patch_size  # Patches along width
    num_patches = num_patches_h * num_patches_w

    # Reshape: (C, H, W) -> (C, num_h, patch_size, num_w, patch_size)
    patches = image.reshape(
        C, num_patches_h, patch_size, num_patches_w, patch_size
    )

    # Rearrange to: (num_h, num_w, C, patch_size, patch_size)
    patches = patches.permute(1, 3, 0, 2, 4)

    # Flatten each patch: (num_patches, C * patch_size * patch_size)
    patches = patches.reshape(num_patches, -1)

    return patches

# Example: a 224x224 RGB image
image = torch.randn(3, 224, 224)  # Random image for demonstration

patches = split_into_patches(image, patch_size=16)

print(f"Original image shape: {image.shape}")
print(f"Number of patches:    {patches.shape[0]}")
print(f"Each patch has:       {patches.shape[1]} values")
print(f"  = 16 x 16 x 3 (height x width x channels)")
print(f"Patches tensor shape: {patches.shape}")
```

Output:

```
Original image shape: torch.Size([3, 224, 224])
Number of patches:    196
```

```
Each patch has:       768 values
  = 16 x 16 x 3 (height x width x channels)
Patches tensor shape: torch.Size([196, 768])
```

Line-by-line explanation:

- `num_patches_h = H // patch_size` calculates how many patches fit along the height. For a 224-pixel image with 16-pixel patches: 224 / 16 = 14.
- `num_patches_w = W // patch_size` does the same for width. Also 14.
- Total patches = 14 x 14 = 196.
- Each patch is a 16x16 region with 3 color channels, so each patch has 16 x 16 x 3 = 768 values.
- The reshape and permute operations rearrange the image tensor to extract each patch as a flat vector.

Step 2: Patch Embedding

The raw patch values (768 numbers) need to be projected into the transformer's hidden dimension using a linear layer. This is the patch embedding.

```python
import torch
import torch.nn as nn

class PatchEmbedding(nn.Module):
    """
    Convert image patches into embedding vectors.

    This is equivalent to running a convolution with
    kernel_size = patch_size and stride = patch_size.
    """

    def __init__(self, image_size=224, patch_size=16,
                 in_channels=3, embed_dim=768):
        super().__init__()

        self.patch_size = patch_size
        self.num_patches = (image_size // patch_size) ** 2

        # Linear projection: flatten patch -> embedding
        # Input: patch_size * patch_size * in_channels = 768
        # Output: embed_dim = 768 (can be different)
        self.projection = nn.Linear(
            patch_size * patch_size * in_channels,
            embed_dim
        )
```

```
    def forward(self, x):
        """
        Parameters:
            x: tensor of shape (batch, channels, height, width)
        Returns:
            embeddings: tensor of shape (batch, num_patches, embed_dim)
        """
        B, C, H, W = x.shape

        # Split into patches and flatten each patch
        # Using unfold to extract patches
        patches = x.unfold(2, self.patch_size, self.patch_size)
        patches = patches.unfold(3, self.patch_size, self.patch_size)
        patches = patches.contiguous().view(B, -1,
            C * self.patch_size * self.patch_size)

        # Project to embedding dimension
        embeddings = self.projection(patches)

        return embeddings

# Test
patch_embed = PatchEmbedding(
    image_size=224, patch_size=16, in_channels=3, embed_dim=768
)

image = torch.randn(1, 3, 224, 224)  # Batch of 1 image
embeddings = patch_embed(image)

print(f"Input image:  {image.shape}")
print(f"Num patches:  {patch_embed.num_patches}")
print(f"Patch embeds: {embeddings.shape}")
print(f"  -> {embeddings.shape[1]} patches, each a {embeddings.shape[2]}-dim vector")
```

Output:

```
Input image:  torch.Size([1, 3, 224, 224])
Num patches:  196
Patch embeds: torch.Size([1, 196, 768])
  -> 196 patches, each a 768-dim vector
```

Line-by-line explanation:

- `nn.Linear(768, 768)` creates a learnable linear projection. It transforms each flattened patch (768 raw values) into a 768-dimensional embedding vector. The weights are learned during training.
- `x.unfold(2, self.patch_size, self.patch_size)` extracts sliding windows along dimension 2 (height). With stride equal to patch_size, the windows do not overlap.
- The result is 196 embedding vectors, one per patch. Each vector captures the essential features of its patch.

Step 3: The CLS Token

ViT adds a special learnable token called the CLS (classification) token at the beginning of the sequence. This token is not a real patch. Its purpose is to aggregate information from all patches through the transformer layers.

```
Before CLS token:
  [P1, P2, P3, ..., P196]     <- 196 patch embeddings

After adding CLS token:
  [CLS, P1, P2, P3, ..., P196]  <- 197 tokens

The CLS token attends to all patches.
After the transformer processes the sequence,
the CLS token's output vector contains a
"summary" of the entire image.

This summary is used for classification.
```

```python
import torch
import torch.nn as nn

class CLSToken(nn.Module):
    """Prepend a learnable CLS token to the patch sequence."""

    def __init__(self, embed_dim=768):
        super().__init__()

        # The CLS token is a learnable parameter
        # Shape: (1, 1, embed_dim)
        self.cls_token = nn.Parameter(
            torch.randn(1, 1, embed_dim)
        )

    def forward(self, patch_embeddings):
        """
        Parameters:
            patch_embeddings: (batch, num_patches, embed_dim)
        Returns:
            tokens: (batch, num_patches + 1, embed_dim)
        """
        batch_size = patch_embeddings.shape[0]

        # Expand CLS token for each image in the batch
        cls_tokens = self.cls_token.expand(batch_size, -1, -1)

        # Concatenate: [CLS, P1, P2, ..., P196]
        tokens = torch.cat([cls_tokens, patch_embeddings], dim=1)

        return tokens
```

```
# Test
cls_module = CLSToken(embed_dim=768)
patch_embeds = torch.randn(2, 196, 768)  # Batch of 2 images

tokens = cls_module(patch_embeds)
print(f"Before CLS: {patch_embeds.shape}  (196 patches)")
print(f"After CLS:  {tokens.shape}  (CLS + 196 patches = 197)")
```

Output:

```
Before CLS: torch.Size([2, 196, 768])  (196 patches)
After CLS:  torch.Size([2, 197, 768])  (CLS + 196 patches = 197)
```

Line-by-line explanation:

- `nn.Parameter(torch.randn(1, 1, embed_dim))` creates a learnable vector. The model will adjust these values during training to learn the best "summary" representation.
- `cls_token.expand(batch_size, -1, -1)` copies the CLS token for each image in the batch. All images share the same learned CLS token weights, but each image will produce a different CLS output after the transformer processes it.
- `torch.cat([cls_tokens, patch_embeddings], dim=1)` prepends the CLS token at position 0.

Step 4: Position Encoding

Transformers do not inherently know the order of their input. Without position information, the transformer would treat a patch from the top-left corner the same as one from the bottom-right corner. Position encoding tells the transformer where each patch came from in the original image.

```
Without position encoding:
  The transformer does not know that P1 is top-left
  and P196 is bottom-right. It sees them as an
  unordered set of patches.

With position encoding:
  Each patch embedding gets a unique position signal added:

  Token 0 (CLS):  embedding + position_0
  Token 1 (P1):   embedding + position_1   <- "I am from top-left"
  Token 2 (P2):   embedding + position_2   <- "I am next to P1"
  ...
  Token 196 (P196): embedding + position_196 <- "I am from bottom-right"
```

```
  Now the transformer knows the spatial arrangement!
```

```python
import torch
import torch.nn as nn

class PositionEncoding(nn.Module):
    """Add learnable position embeddings to tokens."""

    def __init__(self, num_tokens=197, embed_dim=768):
        super().__init__()

        # One position embedding per token (CLS + patches)
        # These are learned during training
        self.position_embeddings = nn.Parameter(
            torch.randn(1, num_tokens, embed_dim)
        )

    def forward(self, tokens):
        """
        Add position information to each token.

        Parameters:
            tokens: (batch, num_tokens, embed_dim)
        Returns:
            tokens_with_position: (batch, num_tokens, embed_dim)
        """
        return tokens + self.position_embeddings

# Test
pos_enc = PositionEncoding(num_tokens=197, embed_dim=768)
tokens = torch.randn(2, 197, 768)

tokens_with_pos = pos_enc(tokens)
print(f"Tokens before position encoding: {tokens.shape}")
print(f"Tokens after position encoding:  {tokens_with_pos.shape}")
print(f"Shape is the same, but now each token knows its position!")
```

Output:

```
Tokens before position encoding: torch.Size([2, 197, 768])
Tokens after position encoding:  torch.Size([2, 197, 768])
Shape is the same, but now each token knows its position!
```

Line-by-line explanation:

- `nn.Parameter(torch.randn(1, num_tokens, embed_dim))` creates 197 learnable position vectors, one per token.
- Unlike the fixed sinusoidal position encoding used in the original NLP Transformer, ViT uses learned position embeddings. The model discovers the best position

representations during training.

- `tokens + self.position_embeddings` simply adds the position vectors to the token embeddings element-wise. This is the same approach used in BERT.
- The position embeddings learn meaningful spatial relationships. After training, nearby patches have similar position embeddings.

Step 5: Transformer Encoder

The sequence of tokens (with positions) is fed into a standard transformer encoder. The encoder consists of multiple layers, each containing multi-head self-attention and a feed-forward network.

```
Transformer Encoder (one layer):

Input tokens: [CLS, P1, P2, ..., P196]
                     |
           [Layer Normalization]
                     |
           [Multi-Head Self-Attention]
              Every token attends to every other token.
              CLS learns to gather information from all patches.
              Neighboring patches learn spatial relationships.
                     |
              + (Residual Connection)
                     |
           [Layer Normalization]
                     |
           [Feed-Forward Network (MLP)]
              Two linear layers with GELU activation.
                     |
              + (Residual Connection)
                     |
Output tokens: [CLS', P1', P2', ..., P196']

This is repeated for L layers (e.g., 12 layers for ViT-Base).
```

```python
import torch
import torch.nn as nn

# PyTorch provides a built-in TransformerEncoder
encoder_layer = nn.TransformerEncoderLayer(
    d_model=768,       # Embedding dimension
    nhead=12,          # Number of attention heads
    dim_feedforward=3072,  # Hidden size in feed-forward network
    dropout=0.1,
    activation='gelu',  # GELU activation (not ReLU)
    batch_first=True    # Input shape: (batch, seq, embed)
```

```
)

transformer_encoder = nn.TransformerEncoder(
    encoder_layer,
    num_layers=12       # 12 layers for ViT-Base
)

# Test
tokens = torch.randn(2, 197, 768)  # Batch of 2, 197 tokens, 768 dim
output = transformer_encoder(tokens)

print(f"Input shape:  {tokens.shape}")
print(f"Output shape: {output.shape}")
print(f"Each token has been updated by attending to all other tokens.")
```

Output:

```
Input shape:  torch.Size([2, 197, 768])
Output shape: torch.Size([2, 197, 768])
Each token has been updated by attending to all other tokens.
```

Step 6: Classification Head

After the transformer encoder, the CLS token's output vector (position 0) is used as the image representation. A simple linear layer maps this to class predictions.

```python
import torch
import torch.nn as nn

# Classification head
num_classes = 1000  # ImageNet has 1000 classes

classification_head = nn.Sequential(
    nn.LayerNorm(768),
    nn.Linear(768, num_classes)
)

# Simulate transformer output
transformer_output = torch.randn(2, 197, 768)

# Extract only the CLS token (position 0)
cls_output = transformer_output[:, 0, :]
print(f"CLS token output shape: {cls_output.shape}")

# Get class predictions
logits = classification_head(cls_output)
print(f"Classification logits shape: {logits.shape}")
```

```
# Get predicted class
predictions = torch.argmax(logits, dim=1)
print(f"Predicted classes: {predictions}")
```

Output:

```
CLS token output shape: torch.Size([2, 768])
Classification logits shape: torch.Size([2, 1000])
Predicted classes: tensor([542, 831])
```

Line-by-line explanation:

- `transformer_output[:, 0, :]` extracts the first token (CLS token) for all images in the batch. The CLS token has aggregated information from all patches through self-attention.
- `nn.LayerNorm(768)` normalizes the CLS output for stable classification.
- `nn.Linear(768, num_classes)` maps the 768-dimensional vector to 1000 class scores (for ImageNet).
- `torch.argmax(logits, dim=1)` finds the class with the highest score for each image.

ViT vs CNN Comparison

```
+-----------------------+-----------------------+-----------------------+
| Aspect                | CNN                   | Vision Transformer    |
+-----------------------+-----------------------+-----------------------+
| Core operation        | Convolution           | Self-Attention        |
|                       | (local patterns)      | (global relationships) |
+-----------------------+-----------------------+-----------------------+
| Receptive field       | Starts small, grows   | Global from layer 1   |
|                       | with depth            | (every patch sees all) |
+-----------------------+-----------------------+-----------------------+
| Inductive bias        | Strong (locality,     | Weak (must learn      |
|                       | translation invariance)| spatial structure)    |
+-----------------------+-----------------------+-----------------------+
| Data requirement      | Works well with less  | Needs large datasets  |
|                       | data (good inductive  | (ImageNet-21k or      |
|                       | bias helps)           | larger)               |
+-----------------------+-----------------------+-----------------------+
| Scalability           | Diminishing returns   | Scales better with    |
|                       | with more data        | more data and compute |
+-----------------------+-----------------------+-----------------------+
| Computation           | Efficient for small   | Memory-intensive      |
|                       | to medium images      | (quadratic attention) |
```

```
+-----------------------+-----------------------+-----------------------+
| Feature learning      | Hierarchical          | Flat (all patches at  |
|                       | (edges->textures->    | same level unless     |
|                       | objects)              | using special design) |
+-----------------------+-----------------------+-----------------------+
```

When ViT Beats CNN

```
ViT excels when:
  - You have LARGE amounts of training data
  - You have significant compute resources (GPUs/TPUs)
  - You need to capture global relationships
  - You want a unified architecture for text + images

CNN excels when:
  - You have LIMITED training data
  - You need fast inference on edge devices
  - You want strong built-in spatial assumptions
  - You are working with high-resolution images
```

The key insight is about inductive bias. CNNs have a strong inductive bias: they assume that nearby pixels are related (locality) and that patterns can appear anywhere in the image (translation invariance). This helps CNNs learn efficiently from small datasets.

ViT has minimal inductive bias. It must learn everything from the data, including spatial relationships. This means it needs much more data, but when given enough data, it can discover patterns that CNNs cannot.

Using ViT with Hugging Face

Hugging Face provides easy access to pre-trained ViT models. Let us classify an image using ViT.

Basic Image Classification

```python
from transformers import ViTForImageClassification, ViTImageProcessor
from PIL import Image
import torch
import numpy as np
```

```
# Load pre-trained ViT model and processor
model_name = "google/vit-base-patch16-224"

processor = ViTImageProcessor.from_pretrained(model_name)
model = ViTForImageClassification.from_pretrained(model_name)
model.eval()

print(f"Model: {model_name}")
print(f"Number of classes: {model.config.num_labels}")
print(f"Image size: {model.config.image_size}")
print(f"Patch size: {model.config.patch_size}")
print(f"Hidden size: {model.config.hidden_size}")
print(f"Number of layers: {model.config.num_hidden_layers}")
print(f"Number of attention heads: {model.config.num_attention_heads}")

# Create a sample image (use a real image in practice)
# image = Image.open("cat.jpg")
sample_image = Image.fromarray(
    np.random.randint(0, 255, (224, 224, 3), dtype=np.uint8)
)

# Preprocess the image
inputs = processor(images=sample_image, return_tensors="pt")
print(f"\nProcessed input shape: {inputs['pixel_values'].shape}")

# Run inference
with torch.no_grad():
    outputs = model(**inputs)

# Get predictions
logits = outputs.logits
predicted_class_idx = logits.argmax(-1).item()
predicted_label = model.config.id2label[predicted_class_idx]

print(f"Predicted class index: {predicted_class_idx}")
print(f"Predicted label: {predicted_label}")

# Get top 5 predictions
top5_probs, top5_indices = torch.topk(
    torch.softmax(logits, dim=-1), 5
)

print("\nTop 5 predictions:")
for prob, idx in zip(top5_probs[0], top5_indices[0]):
    label = model.config.id2label[idx.item()]
    print(f"  {label:30s}: {prob.item():.4f}")
```

Output:

```
Model: google/vit-base-patch16-224
Number of classes: 1000
Image size: 224
```

```
Patch size: 16
Hidden size: 768
Number of layers: 12
Number of attention heads: 12

Processed input shape: torch.Size([1, 3, 224, 224])
Predicted class index: 723
Predicted label: pinwheel
Predicted label: pinwheel

Top 5 predictions:
  pinwheel                      : 0.0089
  jigsaw puzzle                 : 0.0078
  Band Aid                      : 0.0068
  shower curtain                : 0.0066
  doormat                       : 0.0057
```

Line-by-line explanation:

- `ViTImageProcessor.from_pretrained(model_name)` loads the image preprocessor that handles resizing, normalization, and converting to tensors.
- `ViTForImageClassification.from_pretrained(model_name)` loads the complete ViT model with pre-trained weights. This model was trained on ImageNet-21k and fine-tuned on ImageNet-1k.
- `processor(images=sample_image, return_tensors="pt")` prepares the image for the model. It handles all the required transformations automatically.
- `outputs.logits` contains the raw prediction scores (logits) for all 1000 ImageNet classes.
- `logits.argmax(-1)` finds the class with the highest score.
- `model.config.id2label` maps class indices to human-readable labels.
- `torch.topk(probs, 5)` returns the top 5 predictions and their probabilities.

Using the Pipeline API (Even Simpler)

```python
from transformers import pipeline

# Create an image classification pipeline with ViT
classifier = pipeline(
    "image-classification",
    model="google/vit-base-patch16-224"
)

# Classify an image
# results = classifier("cat_photo.jpg")
# Or with a PIL image:
# results = classifier(pil_image)
```

```
print("Pipeline created successfully!")
print()
print("Usage:")
print('  results = classifier("photo.jpg")')
print('  # Returns: [{"label": "tabby cat", "score": 0.95}, ...]')
print()
print("The pipeline handles ALL preprocessing automatically:")
print("  - Loading the image")
print("  - Resizing to 224x224")
print("  - Normalizing pixel values")
print("  - Converting to patches")
print("  - Running the transformer")
print("  - Returning top predictions")
```

Output:

```
Pipeline created successfully!

Usage:
  results = classifier("photo.jpg")
  # Returns: [{"label": "tabby cat", "score": 0.95}, ...]

The pipeline handles ALL preprocessing automatically:
  - Loading the image
  - Resizing to 224x224
  - Normalizing pixel values
  - Converting to patches
  - Running the transformer
  - Returning top predictions
```

ViT Model Variants

ViT comes in several sizes. Larger models are more accurate but require more memory and compute:

```
+-------------+--------+--------+-------+----------+----------------+
| Model       | Layers | Heads  | Dim   | Params   | Patch Size     |
+-------------+--------+--------+-------+----------+----------------+
| ViT-Tiny    | 12     | 3      | 192   | 5.7M     | 16x16          |
| ViT-Small   | 12     | 6      | 384   | 22M      | 16x16          |
| ViT-Base    | 12     | 12     | 768   | 86M      | 16x16 or 32x32 |
| ViT-Large   | 24     | 16     | 1024  | 307M     | 16x16 or 32x32 |
| ViT-Huge    | 32     | 16     | 1280  | 632M     | 14x14          |
+-------------+--------+--------+-------+----------+----------------+
```

```
Naming convention: vit-base-patch16-224
                   |     |      |     |
                   ViT  size  patch  image
                              size   size
```

```python
# Available ViT models on Hugging Face
models = {
    "google/vit-base-patch16-224": {
        "params": "86M",
        "accuracy": "81.1% (ImageNet top-1)",
        "speed": "Fast",
        "best_for": "General use, good balance"
    },
    "google/vit-large-patch16-224": {
        "params": "307M",
        "accuracy": "83.1% (ImageNet top-1)",
        "speed": "Moderate",
        "best_for": "Higher accuracy needs"
    },
    "google/vit-base-patch32-224": {
        "params": "88M",
        "accuracy": "79.1% (ImageNet top-1)",
        "speed": "Fastest",
        "best_for": "Speed-critical applications"
    }
}

print("Common ViT Models:")
print("=" * 60)
for name, info in models.items():
    print(f"\n{name}")
    for key, value in info.items():
        print(f"  {key:12s}: {value}")
```

Output:

```
Common ViT Models:
============================================================

google/vit-base-patch16-224
  params      : 86M
  accuracy    : 81.1% (ImageNet top-1)
  speed       : Fast
  best_for    : General use, good balance

google/vit-large-patch16-224
  params      : 307M
  accuracy    : 83.1% (ImageNet top-1)
  speed       : Moderate
  best_for    : Higher accuracy needs
```

```
google/vit-base-patch32-224
  params      : 88M
  accuracy    : 79.1% (ImageNet top-1)
  speed       : Fastest
  best_for    : Speed-critical applications
```

Common Mistakes

1. Using ViT with very small datasets. ViT needs large amounts of data to learn well. If you have fewer than 10,000 images, a CNN with transfer learning will likely outperform ViT.
2. Forgetting to use the correct preprocessing. Each ViT model expects specific image sizes and normalization. Always use the corresponding `ViTImageProcessor` to avoid mismatches.
3. Ignoring patch size tradeoffs. Smaller patches (16x16) give more tokens and higher accuracy but use more memory. Larger patches (32x32) are faster but less accurate.
4. Not using pre-trained weights. Training ViT from scratch on small datasets produces poor results. Always start with pre-trained weights and fine-tune.
5. Expecting ViT to work well on high-resolution images out of the box. Standard ViT is designed for 224x224 images. Using much larger images creates an extremely long sequence that can exceed GPU memory.

Best Practices

1. Use pre-trained models from Hugging Face. They are thoroughly tested and come with matching preprocessors.
2. Choose the right model size. For prototyping, use ViT-Base (86M parameters). For production, consider the accuracy vs speed tradeoff for your specific use case.
3. Fine-tune on your specific data. Pre-trained ViT models work well for general classification but will perform better after fine-tuning on your specific domain (medical images, satellite photos, etc.).
4. Consider hybrid approaches. Some architectures like DeiT (Data-efficient Image Transformers) combine CNN ideas with ViT to work better with smaller datasets.
5. Monitor memory usage. Self-attention has quadratic memory complexity relative to the number of tokens. For 196 patches, this is manageable. For thousands of patches, you

may need specialized attention mechanisms.

Quick Summary

Vision Transformers (ViT) apply the transformer architecture to images by splitting them into patches and treating each patch like a word token. A 224x224 image is divided into 196 patches of 16x16 pixels each. Each patch is linearly projected into an embedding vector, a learnable CLS token is prepended, and position encodings are added. The sequence is then processed by a standard transformer encoder. The CLS token output is used for classification. ViT achieves excellent results when trained on large datasets but struggles with small datasets due to its minimal inductive bias compared to CNNs. Hugging Face provides easy-to-use pre-trained ViT models that can be applied to image classification with just a few lines of code.

Key Points

- ViT treats image patches as tokens, just like words in NLP transformers
- A 224x224 image with 16x16 patches produces 196 tokens
- Patch embedding projects raw patch pixels into a learned vector space
- CLS token is a special learnable token that aggregates the whole image
- Position encoding tells the transformer where each patch is located
- ViT needs large datasets because it lacks CNN's built-in spatial assumptions
- ViT scales better than CNNs with more data and compute
- Hugging Face makes it easy to use pre-trained ViT models

Practice Questions

1. Why does ViT split images into patches instead of processing the entire image at once? What would happen if you treated each pixel as a separate token?
2. What is the CLS token and why is it used instead of averaging all patch outputs for classification?

3. If you increase the patch size from 16x16 to 32x32, how does this change the number of tokens and the model's behavior? What are the tradeoffs?
4. Why does ViT require more training data than a CNN to achieve similar accuracy? Relate your answer to the concept of inductive bias.
5. Explain how position encoding in ViT differs from position encoding in a text transformer. Why does ViT use learnable position embeddings?

Exercises

Exercise 1: Patch Extraction Visualizer

Write a function that takes a PIL image and a patch size, splits the image into patches, and displays each patch in a grid using matplotlib. Add a number label to each patch to show the order they would be fed to the transformer.

Exercise 2: Classify Multiple Images

Use the Hugging Face ViT pipeline to classify 5 different images (download sample images from the internet). For each image, print the top 3 predictions with their confidence scores. Compare the results with what you see in the images.

Exercise 3: Model Size Comparison

Load both `google/vit-base-patch16-224` and `google/vit-base-patch32-224`. Classify the same set of images with both models. Compare the predictions and measure the inference time for each. Which model is faster? Which is more accurate?

What Is Next?

In the next chapter, you will learn about Optical Character Recognition (OCR), the technology that extracts text from images. You will use Tesseract, EasyOCR, and Hugging Face models to read text from photographs, scanned documents, and screenshots. OCR combines many of the image processing and deep learning techniques you have learned, and it is one of the most practical applications of computer vision you can build today.

Chapter 24: OCR — Extracting Text from Images

What You Will Learn

In this chapter, you will learn:

- What Optical Character Recognition (OCR) is and how it works
- How to install and use Tesseract with pytesseract
- How to preprocess images for better OCR accuracy (thresholding, denoising)
- How to use the EasyOCR library for multilingual text recognition
- How to use Hugging Face OCR models like TrOCR
- How to build a complete document text extraction example
- When to use which OCR tool

Why This Chapter Matters

Every day, billions of documents exist only as images: scanned contracts, photographed receipts, handwritten notes, street signs in photos, text on product packaging. If you want a computer to read and understand this text, you need OCR.

OCR (Optical Character Recognition) is the technology that converts images of text into actual, editable, searchable text. It is behind the "scan to text" feature on your phone, the check-deposit feature in banking apps, the automated license plate readers on highways, and the accessibility tools that read text aloud for visually impaired users.

This chapter gives you three different OCR tools, from a classic rule-based engine (Tesseract) to a modern deep learning approach (TrOCR), so you can pick the right tool for any text extraction task.

What Is OCR?

OCR stands for Optical Character Recognition. It is the process of converting an image containing text into machine-readable text.

```
OCR Process:

Image with text                    Machine-readable text
+-------------------+             +-------------------+
|                   |             |                   |
|  Hello, World!    |  ------>    |  "Hello, World!"  |
|                   |   OCR       |                   |
+-------------------+             +-------------------+
  (pixels)                         (characters/string)
```

How OCR Works (Simplified)

Traditional OCR follows these steps:

```
Step 1: PREPROCESSING
  - Convert to grayscale
  - Remove noise (spots, speckles)
  - Fix rotation and skew
  - Adjust contrast

Step 2: TEXT DETECTION
  - Find regions that contain text
  - Separate text from background
  - Identify text lines and word boundaries

Step 3: CHARACTER RECOGNITION
  - Segment individual characters (or recognize words directly)
  - Match each character against known patterns
  - Use language models to correct errors

Step 4: POST-PROCESSING
  - Apply spell checking
  - Fix common misrecognitions (0 vs O, 1 vs l)
  - Format the output text

Example flow:
  [Photo of receipt] -> [Find text regions] -> [Read characters]
  -> "Milk   $3.99\nBread  $2.50\nTotal  $6.49"
```

Think of OCR like a person reading a book. First, they look at the page and find where the text is (detection). Then, they read each word character by character (recognition). Finally, they use their knowledge of the language to fix any parts they are not sure about

(post-processing).

Tesseract OCR with pytesseract

Tesseract is the most widely used open-source OCR engine. Originally developed by Hewlett-Packard in the 1980s, it is now maintained by Google. The `pytesseract` library provides a Python wrapper for Tesseract.

Installation

```python
# Step 1: Install Tesseract engine
# macOS:   brew install tesseract
# Ubuntu:  sudo apt install tesseract-ocr
# Windows: Download installer from GitHub

# Step 2: Install Python wrapper
# pip install pytesseract Pillow

import pytesseract
from PIL import Image
import sys

# Verify installation
try:
    version = pytesseract.get_tesseract_version()
    print(f"Tesseract version: {version}")
    print("Tesseract is installed correctly!")
except Exception as e:
    print(f"Tesseract not found: {e}")
    print("Please install Tesseract first.")
    print("  macOS:  brew install tesseract")
    print("  Ubuntu: sudo apt install tesseract-ocr")
```

Output:

```
Tesseract version: 5.3.0
Tesseract is installed correctly!
```

Basic Text Extraction

```python
import pytesseract
from PIL import Image, ImageDraw, ImageFont
import numpy as np

# Create a sample image with text
# (In practice, you would load a real image)
img = Image.new('RGB', (400, 100), color='white')
draw = ImageDraw.Draw(img)

# Draw text on the image
# Use default font (for demonstration)
draw.text((20, 30), "Hello, OCR World!", fill='black')

# Run OCR
text = pytesseract.image_to_string(img)

print("Extracted text:")
print(repr(text))  # repr shows whitespace characters
print()
print("Cleaned text:")
print(text.strip())
```

Output:

```
Extracted text:
'Hello, OCR World!\n\n'

Cleaned text:
Hello, OCR World!
```

Line-by-line explanation:

- `Image.new('RGB', (400, 100), color='white')` creates a blank white image of size 400x100 pixels.
- `ImageDraw.Draw(img)` creates a drawing context for adding text to the image.
- `draw.text((20, 30), "Hello, OCR World!", fill='black')` writes text on the image at position (20, 30) in black.
- `pytesseract.image_to_string(img)` is the core OCR function. It takes an image and returns the recognized text as a string.
- `repr(text)` shows the raw string including newlines and spaces. Tesseract often adds trailing whitespace.
- `text.strip()` removes leading and trailing whitespace for clean output.

Getting Detailed OCR Data

Tesseract can return more than just text. It can provide bounding boxes, confidence scores, and word-level information.

```python
import pytesseract
from PIL import Image, ImageDraw
import numpy as np

# Create an image with multiple lines of text
img = Image.new('RGB', (500, 200), color='white')
draw = ImageDraw.Draw(img)
draw.text((20, 20), "Invoice Number: 12345", fill='black')
draw.text((20, 60), "Date: 2024-01-15", fill='black')
draw.text((20, 100), "Amount: $99.99", fill='black')

# Get detailed data as a dictionary
data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)

print("Keys in OCR data:")
print(list(data.keys()))
print()

# Print each detected word with its confidence and position
print(f"{'Word':<20} {'Confidence':<12} {'Position (x,y,w,h)'}")
print("-" * 60)

for i in range(len(data['text'])):
    word = data['text'][i].strip()
    conf = data['conf'][i]

    if word and int(conf) > 0:  # Skip empty entries
        x = data['left'][i]
        y = data['top'][i]
        w = data['width'][i]
        h = data['height'][i]
        print(f"{word:<20} {conf:<12} ({x}, {y}, {w}, {h})")
```

Output:

```
Keys in OCR data:
['level', 'page_num', 'block_num', 'par_num', 'line_num', 'word_num', 'left', 'top', 'w...

Word                 Confidence   Position (x,y,w,h)
------------------------------------------------------------
Invoice              90           (20, 20, 80, 25)
Number:              88           (105, 20, 85, 25)
12345                95           (195, 20, 55, 25)
Date:                92           (20, 60, 55, 25)
2024-01-15           89           (80, 60, 110, 25)
Amount:              91           (20, 100, 80, 25)
$99.99               87           (105, 100, 65, 25)
```

Line-by-line explanation:

- `pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)` returns a dictionary containing detailed information about every detected text element.
- `data['text']` contains the recognized words.
- `data['conf']` contains the confidence score for each word (0-100). Higher is more confident.
- `data['left']`, `data['top']`, `data['width']`, `data['height']` give the bounding box of each word in pixels.
- This detailed output is useful for locating specific text regions, filtering low-confidence results, and building structured data from documents.

Tesseract Configuration Options

```python
import pytesseract
from PIL import Image

# Tesseract has many configuration options
# passed via the config parameter

# PSM (Page Segmentation Mode) - tells Tesseract about text layout
psm_modes = {
    0: "Orientation and script detection only",
    1: "Automatic page segmentation with OSD",
    3: "Fully automatic page segmentation (default)",
    4: "Assume a single column of text",
    6: "Assume a single uniform block of text",
    7: "Treat the image as a single text line",
    8: "Treat the image as a single word",
    9: "Treat the image as a single word in a circle",
    10: "Treat the image as a single character",
    11: "Sparse text - find as much text as possible",
    12: "Sparse text with OSD",
    13: "Raw line - treat as a single text line (no hacks)",
}

print("Tesseract Page Segmentation Modes (PSM):")
print("=" * 55)
for mode, description in psm_modes.items():
    print(f"  PSM {mode:2d}: {description}")

print()
print("Usage examples:")
print('  # For a full document page:')
print('  text = pytesseract.image_to_string(img, config="--psm 3")')
print()
print('  # For a single line (like a license plate):')
print('  text = pytesseract.image_to_string(img, config="--psm 7")')
```

```
print()
print('  # For a single word:')
print('  text = pytesseract.image_to_string(img, config="--psm 8")')
print()
print('  # Specify language (e.g., German):')
print('  text = pytesseract.image_to_string(img, lang="deu")')
print()
print('  # Multiple languages:')
print('  text = pytesseract.image_to_string(img, lang="eng+fra")')
```

Output:

```
Tesseract Page Segmentation Modes (PSM):
========================================================
  PSM  0: Orientation and script detection only
  PSM  1: Automatic page segmentation with OSD
  PSM  3: Fully automatic page segmentation (default)
  PSM  4: Assume a single column of text
  PSM  6: Assume a single uniform block of text
  PSM  7: Treat the image as a single text line
  PSM  8: Treat the image as a single word
  PSM  9: Treat the image as a single word in a circle
  PSM 10: Treat the image as a single character
  PSM 11: Sparse text - find as much text as possible
  PSM 12: Sparse text with OSD
  PSM 13: Raw line - treat as a single text line (no hacks)

Usage examples:
  # For a full document page:
  text = pytesseract.image_to_string(img, config="--psm 3")

  # For a single line (like a license plate):
  text = pytesseract.image_to_string(img, config="--psm 7")

  # For a single word:
  text = pytesseract.image_to_string(img, config="--psm 8")

  # Specify language (e.g., German):
  text = pytesseract.image_to_string(img, lang="deu")

  # Multiple languages:
  text = pytesseract.image_to_string(img, lang="eng+fra")
```

Preprocessing Images for Better OCR

OCR accuracy depends heavily on image quality. Preprocessing can dramatically improve results.

Why Preprocessing Matters

```
Raw photo of a receipt          After preprocessing
+---------------------+        +---------------------+
| Slightly rotated    |        | Straight            |
| Low contrast        |  --->  | High contrast       |
| Noise/specks        |        | Clean               |
| Shadows             |        | Even lighting       |
+---------------------+        +---------------------+
OCR accuracy: ~60%              OCR accuracy: ~95%
```

Common Preprocessing Techniques

```python
import cv2
import numpy as np
from PIL import Image

def preprocess_for_ocr(image_path):
    """
    Apply common preprocessing steps to improve OCR accuracy.

    Parameters:
        image_path: path to the input image

    Returns:
        preprocessed: the cleaned image ready for OCR
    """
    # Load the image
    img = cv2.imread(image_path)

    # Step 1: Convert to grayscale
    # OCR works on single-channel images
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Step 2: Noise removal with Gaussian blur
    # Kernel size (5,5) controls how much to blur
    # Removes small specks and noise
    denoised = cv2.GaussianBlur(gray, (5, 5), 0)

    # Step 3: Binarization with Otsu's thresholding
    # Converts to pure black and white
    # Otsu automatically finds the best threshold value
    _, binary = cv2.threshold(
        denoised, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    return binary
```

```
# Demonstrate each step with a synthetic example
print("Preprocessing Pipeline:")
print()
print("Step 1: Grayscale Conversion")
print("  Why: OCR does not need color information.")
print("  How: cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)")
print()
print("Step 2: Noise Removal (Gaussian Blur)")
print("  Why: Small specks confuse the OCR engine.")
print("  How: cv2.GaussianBlur(gray, (5, 5), 0)")
print()
print("Step 3: Binarization (Otsu's Thresholding)")
print("  Why: Pure black and white is easiest for OCR.")
print("  How: cv2.threshold(img, 0, 255,")
print("       cv2.THRESH_BINARY + cv2.THRESH_OTSU)")
```

Output:

```
Preprocessing Pipeline:

Step 1: Grayscale Conversion
  Why: OCR does not need color information.
  How: cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

Step 2: Noise Removal (Gaussian Blur)
  Why: Small specks confuse the OCR engine.
  How: cv2.GaussianBlur(gray, (5, 5), 0)

Step 3: Binarization (Otsu's Thresholding)
  Why: Pure black and white is easiest for OCR.
  How: cv2.threshold(img, 0, 255,
       cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Adaptive Thresholding

When an image has uneven lighting (like a photo of a page with shadows), global thresholding may not work well. Adaptive thresholding adjusts the threshold for each small region of the image.

```python
import cv2
import numpy as np

def adaptive_threshold_demo():
    """
    Demonstrate the difference between global and adaptive
    thresholding on an image with uneven lighting.
    """
```

```
    # Create a synthetic image with uneven lighting
    # Left side is darker, right side is brighter
    h, w = 100, 300
    gradient = np.tile(
        np.linspace(50, 200, w), (h, 1)
    ).astype(np.uint8)

    # Add some "text" (dark spots)
    text_img = gradient.copy()
    text_img[30:40, 30:50] = 0    # "word" on dark side
    text_img[30:40, 150:170] = 0  # "word" in middle
    text_img[30:40, 250:270] = 0  # "word" on bright side

    # Global threshold (Otsu)
    _, global_thresh = cv2.threshold(
        text_img, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    # Adaptive threshold
    adaptive_thresh = cv2.adaptiveThreshold(
        text_img, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  # Method
        cv2.THRESH_BINARY,               # Type
        11,                              # Block size
        2                                # Constant subtracted
    )

    print("Uneven Lighting Example:")
    print("  Image has dark left side and bright right side")
    print()
    print("Global Thresholding (Otsu):")
    print("  Uses ONE threshold for the entire image")
    print("  Problem: text on the dark side may disappear")
    print()
    print("Adaptive Thresholding:")
    print("  Calculates a different threshold for each region")
    print("  Result: text is visible on both dark and bright sides")
    print()
    print("Use adaptive thresholding when:")
    print("  - The image has shadows")
    print("  - Lighting is uneven across the page")
    print("  - The photo was taken at an angle")

adaptive_threshold_demo()
```

Output:

```
Uneven Lighting Example:
  Image has dark left side and bright right side

Global Thresholding (Otsu):
  Uses ONE threshold for the entire image
```

```
  Problem: text on the dark side may disappear

Adaptive Thresholding:
  Calculates a different threshold for each region
  Result: text is visible on both dark and bright sides

Use adaptive thresholding when:
  - The image has shadows
  - Lighting is uneven across the page
  - The photo was taken at an angle
```

Deskewing (Fixing Rotation)

```python
import cv2
import numpy as np

def deskew(image):
    """
    Fix slight rotation in a document image.

    Uses the minimum area rectangle of the text to determine
    the skew angle, then rotates the image to straighten it.

    Parameters:
        image: grayscale image (numpy array)

    Returns:
        deskewed: the straightened image
    """
    # Find all non-zero (text) pixels
    coords = np.column_stack(np.where(image < 128))

    if len(coords) < 100:
        print("Not enough text pixels to determine skew.")
        return image

    # Find the minimum area rectangle around the text
    angle = cv2.minAreaRect(coords)[-1]

    # Adjust angle
    if angle < -45:
        angle = -(90 + angle)
    else:
        angle = -angle

    # Get the image center
    h, w = image.shape[:2]
    center = (w // 2, h // 2)

    # Create rotation matrix
```

```
    rotation_matrix = cv2.getRotationMatrix2D(center, angle, 1.0)

    # Apply rotation
    deskewed = cv2.warpAffine(
        image, rotation_matrix, (w, h),
        flags=cv2.INTER_CUBIC,
        borderMode=cv2.BORDER_REPLICATE
    )

    return deskewed

print("Deskew function defined!")
print()
print("How it works:")
print("  1. Find all dark (text) pixels in the image")
print("  2. Fit a minimum-area rectangle around them")
print("  3. The rectangle's angle reveals the skew")
print("  4. Rotate the image by the negative of that angle")
print()
print("Example:")
print("  Before: text tilted 3 degrees clockwise")
print("  After:  text is perfectly horizontal")
```

Output:

```
Deskew function defined!

How it works:
  1. Find all dark (text) pixels in the image
  2. Fit a minimum-area rectangle around them
  3. The rectangle's angle reveals the skew
  4. Rotate the image by the negative of that angle

Example:
  Before: text tilted 3 degrees clockwise
  After:  text is perfectly horizontal
```

Complete Preprocessing Pipeline

```python
import cv2
import numpy as np

def full_preprocessing_pipeline(image_path):
    """
    Complete preprocessing pipeline for OCR.

    Steps:
    1. Load and convert to grayscale
```

```
    2. Resize if too small
    3. Denoise
    4. Deskew
    5. Adaptive threshold
    6. Remove borders

    Parameters:
        image_path: path to the input image

    Returns:
        processed: cleaned image ready for OCR
    """
    # Load image
    img = cv2.imread(image_path)
    if img is None:
        raise FileNotFoundError(f"Cannot load: {image_path}")

    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Resize if too small (OCR works better on larger text)
    h, w = gray.shape
    if h < 300:
        scale = 300 / h
        gray = cv2.resize(gray, None, fx=scale, fy=scale,
                          interpolation=cv2.INTER_CUBIC)

    # Denoise
    denoised = cv2.fastNlMeansDenoising(gray, h=10)

    # Adaptive threshold
    binary = cv2.adaptiveThreshold(
        denoised, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11, 2
    )

    return binary

print("Full preprocessing pipeline defined!")
print()
print("Key insight: good preprocessing can improve OCR")
print("accuracy from 60% to 95% or more.")
print()
print("Always try these steps before blaming the OCR engine:")
print("  1. Convert to grayscale")
print("  2. Scale up small images")
print("  3. Remove noise")
print("  4. Fix rotation")
print("  5. Binarize (black and white)")
```

Output:

```
Full preprocessing pipeline defined!

Key insight: good preprocessing can improve OCR
accuracy from 60% to 95% or more.

Always try these steps before blaming the OCR engine:
  1. Convert to grayscale
  2. Scale up small images
  3. Remove noise
  4. Fix rotation
  5. Binarize (black and white)
```

EasyOCR Library

EasyOCR is a deep learning based OCR library that supports over 80 languages. It is simpler to use than Tesseract for many use cases and often produces better results on natural scene text (like signs and labels in photos).

Installation and Basic Usage

```python
# Install: pip install easyocr

import easyocr

# Create a reader for English
# The first time downloads the model (this may take a minute)
reader = easyocr.Reader(['en'])

print("EasyOCR reader created for English!")
print()
print("Supported features:")
print("  - 80+ languages")
print("  - GPU acceleration (optional)")
print("  - Scene text detection (signs, labels, etc.)")
print("  - Document text detection")
print("  - Handwriting recognition (limited)")
print()

# Read text from an image
# results = reader.readtext('image.jpg')
#
# Each result is a tuple: (bbox, text, confidence)
# bbox = [[x1,y1], [x2,y2], [x3,y3], [x4,y4]]  (4 corners)
# text = recognized text string
```

```
# confidence = float between 0 and 1

print("Usage:")
print('  results = reader.readtext("photo.jpg")')
print()
print("Result format:")
print("  [")
print("    ([[x1,y1],[x2,y2],[x3,y3],[x4,y4]], 'text', 0.95),")
print("    ([[x1,y1],[x2,y2],[x3,y3],[x4,y4]], 'more text', 0.87),")
print("  ]")
```

Output:

```
EasyOCR reader created for English!

Supported features:
  - 80+ languages
  - GPU acceleration (optional)
  - Scene text detection (signs, labels, etc.)
  - Document text detection
  - Handwriting recognition (limited)

Usage:
  results = reader.readtext("photo.jpg")

Result format:
  [
    ([[x1,y1],[x2,y2],[x3,y3],[x4,y4]], 'text', 0.95),
    ([[x1,y1],[x2,y2],[x3,y3],[x4,y4]], 'more text', 0.87),
  ]
```

EasyOCR Complete Example

```python
import easyocr
import cv2
import numpy as np
from PIL import Image, ImageDraw

def extract_text_easyocr(image_path, languages=['en'],
                         confidence_threshold=0.5):
    """
    Extract text from an image using EasyOCR.

    Parameters:
        image_path: path to the image
        languages: list of language codes
        confidence_threshold: minimum confidence to accept
```

```
    Returns:
        text_results: list of (text, confidence, bbox) tuples
    """
    # Create reader
    reader = easyocr.Reader(languages, gpu=False)

    # Run OCR
    results = reader.readtext(image_path)

    # Filter by confidence
    text_results = []
    for bbox, text, confidence in results:
        if confidence >= confidence_threshold:
            text_results.append({
                'text': text,
                'confidence': confidence,
                'bbox': bbox
            })

    return text_results

def print_results(results):
    """Print OCR results in a formatted table."""
    print(f"{'Text':<30} {'Confidence':<12} {'Position'}")
    print("-" * 70)
    for r in results:
        text = r['text'][:28]
        conf = f"{r['confidence']:.2%}"
        pos = f"({r['bbox'][0][0]:.0f}, {r['bbox'][0][1]:.0f})"
        print(f"{text:<30} {conf:<12} {pos}")

# Example usage (with a real image):
# results = extract_text_easyocr("receipt.jpg")
# print_results(results)

print("EasyOCR extraction function defined!")
print()
print("Multilingual example:")
print('  # Read Chinese and English text:')
print('  reader = easyocr.Reader(["ch_sim", "en"])')
print('  results = reader.readtext("chinese_document.jpg")')
print()
print("Common language codes:")
languages = {
    "en": "English", "fr": "French", "de": "German",
    "es": "Spanish", "it": "Italian", "pt": "Portuguese",
    "ch_sim": "Chinese (Simplified)", "ch_tra": "Chinese (Traditional)",
    "ja": "Japanese", "ko": "Korean", "ar": "Arabic",
    "hi": "Hindi", "ru": "Russian", "th": "Thai"
}
for code, name in languages.items():
    print(f"  {code:8s} -> {name}")
```

Output:

```
EasyOCR extraction function defined!

Multilingual example:
  # Read Chinese and English text:
  reader = easyocr.Reader(["ch_sim", "en"])
  results = reader.readtext("chinese_document.jpg")

Common language codes:
  en       -> English
  fr       -> French
  de       -> German
  es       -> Spanish
  it       -> Italian
  pt       -> Portuguese
  ch_sim   -> Chinese (Simplified)
  ch_tra   -> Chinese (Traditional)
  ja       -> Japanese
  ko       -> Korean
  ar       -> Arabic
  hi       -> Hindi
  ru       -> Russian
  th       -> Thai
```

Hugging Face OCR Models (TrOCR)

TrOCR (Transformer-based Optical Character Recognition) is a state-of-the-art OCR model from Microsoft that uses a vision transformer encoder and a text transformer decoder.

How TrOCR Works

```
TrOCR Architecture:

Image of text line
       |
  [Vision Transformer Encoder]
  (Processes the image, like ViT from Chapter 23)
       |
  Image features
       |
  [Text Transformer Decoder]
  (Generates text character by character, like GPT)
       |
  "Hello World"

Key insight: TrOCR treats OCR as an image-to-text
```

```
translation problem, similar to how machine translation
converts one language to another.
```

Using TrOCR

```python
from transformers import TrOCRProcessor, VisionEncoderDecoderModel
from PIL import Image
import numpy as np

# Load the model and processor
model_name = "microsoft/trocr-base-printed"

processor = TrOCRProcessor.from_pretrained(model_name)
model = VisionEncoderDecoderModel.from_pretrained(model_name)
model.eval()

print(f"Model loaded: {model_name}")
print()
print("Available TrOCR models:")
print("  microsoft/trocr-base-printed    - For printed text")
print("  microsoft/trocr-base-handwritten - For handwritten text")
print("  microsoft/trocr-large-printed   - Higher accuracy, slower")
print("  microsoft/trocr-large-handwritten - For difficult handwriting")
print()

# Create a sample image with text
# In practice, this should be a cropped text line
img = Image.new('RGB', (300, 50), color='white')
from PIL import ImageDraw
draw = ImageDraw.Draw(img)
draw.text((10, 10), "Sample text line", fill='black')

# Process the image
pixel_values = processor(
    images=img, return_tensors="pt"
).pixel_values

print(f"Input shape: {pixel_values.shape}")

# Generate text
import torch
with torch.no_grad():
    generated_ids = model.generate(pixel_values)

# Decode the generated tokens
generated_text = processor.batch_decode(
    generated_ids, skip_special_tokens=True
)[0]

print(f"Recognized text: '{generated_text}'")
```

Output:

```
Model loaded: microsoft/trocr-base-printed

Available TrOCR models:
  microsoft/trocr-base-printed    - For printed text
  microsoft/trocr-base-handwritten - For handwritten text
  microsoft/trocr-large-printed   - Higher accuracy, slower
  microsoft/trocr-large-handwritten - For difficult handwriting

Input shape: torch.Size([1, 3, 384, 384])
Recognized text: 'Sample text line'
```

Line-by-line explanation:

- `TrOCRProcessor.from_pretrained(model_name)` loads the image preprocessor that resizes and normalizes images for TrOCR.
- `VisionEncoderDecoderModel.from_pretrained(model_name)` loads the complete model with a vision encoder (image understanding) and text decoder (text generation).
- `processor(images=img, return_tensors="pt")` preprocesses the image into the format the model expects.
- `model.generate(pixel_values)` runs the decoder to generate text tokens from the image features. This is similar to how GPT generates text, but conditioned on image features instead of previous text.
- `processor.batch_decode(generated_ids, skip_special_tokens=True)` converts the generated token IDs back into human-readable text.

TrOCR for Handwritten Text

```python
from transformers import TrOCRProcessor, VisionEncoderDecoderModel
from PIL import Image
import torch

def recognize_handwriting(image):
    """
    Recognize handwritten text using TrOCR.

    Parameters:
        image: PIL Image of a handwritten text line

    Returns:
        text: recognized text string
    """
    model_name = "microsoft/trocr-base-handwritten"
```

```
    processor = TrOCRProcessor.from_pretrained(model_name)
    model = VisionEncoderDecoderModel.from_pretrained(model_name)
    model.eval()

    # Process the image
    pixel_values = processor(
        images=image, return_tensors="pt"
    ).pixel_values

    # Generate text
    with torch.no_grad():
        generated_ids = model.generate(pixel_values)

    text = processor.batch_decode(
        generated_ids, skip_special_tokens=True
    )[0]

    return text

print("Handwriting recognition function defined!")
print()
print("Tips for best results with TrOCR:")
print("  1. Crop to a single text line (not a full page)")
print("  2. Make sure the text is horizontal")
print("  3. Good contrast between text and background")
print("  4. Use 'handwritten' model for handwriting")
print("  5. Use 'printed' model for typed/printed text")
```

Output:

```
Handwriting recognition function defined!

Tips for best results with TrOCR:
  1. Crop to a single text line (not a full page)
  2. Make sure the text is horizontal
  3. Good contrast between text and background
  4. Use 'handwritten' model for handwriting
  5. Use 'printed' model for typed/printed text
```

Complete Document Text Extraction Example

Here is a complete example that combines preprocessing and OCR to extract text from a document image:

```python
import cv2
import numpy as np
import pytesseract
from PIL import Image

class DocumentOCR:
    """Complete document text extraction pipeline."""

    def __init__(self, engine='tesseract'):
        """
        Initialize the OCR pipeline.

        Parameters:
            engine: 'tesseract', 'easyocr', or 'trocr'
        """
        self.engine = engine
        print(f"DocumentOCR initialized with engine: {engine}")

    def preprocess(self, image_path):
        """
        Preprocess a document image for OCR.

        Returns:
            processed: cleaned image (numpy array)
        """
        # Load image
        img = cv2.imread(image_path)
        if img is None:
            raise FileNotFoundError(f"Cannot load: {image_path}")

        # Convert to grayscale
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Resize small images (text should be at least 12px tall)
        h, w = gray.shape
        if h < 500:
            scale = 500 / h
            gray = cv2.resize(gray, None, fx=scale, fy=scale,
                              interpolation=cv2.INTER_CUBIC)

        # Remove noise
        denoised = cv2.fastNlMeansDenoising(gray, h=10)

        # Binarize
        binary = cv2.adaptiveThreshold(
            denoised, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY, 11, 2
        )

        return binary

    def extract_text(self, image_path, preprocess=True):
        """
```

```
        Extract text from a document image.

        Parameters:
            image_path: path to the image file
            preprocess: whether to apply preprocessing

        Returns:
            text: extracted text as a string
        """
        if preprocess:
            processed = self.preprocess(image_path)
            # Convert numpy to PIL for pytesseract
            pil_image = Image.fromarray(processed)
        else:
            pil_image = Image.open(image_path)

        if self.engine == 'tesseract':
            # Use Tesseract
            text = pytesseract.image_to_string(
                pil_image,
                config='--psm 3 --oem 3'
            )
        elif self.engine == 'easyocr':
            import easyocr
            reader = easyocr.Reader(['en'], gpu=False)
            results = reader.readtext(np.array(pil_image))
            text = '\n'.join([r[1] for r in results])
        else:
            raise ValueError(f"Unknown engine: {self.engine}")

        return text.strip()

    def extract_structured(self, image_path):
        """
        Extract text with position and confidence data.

        Returns:
            list of dicts with 'text', 'confidence', 'bbox'
        """
        processed = self.preprocess(image_path)
        pil_image = Image.fromarray(processed)

        data = pytesseract.image_to_data(
            pil_image,
            output_type=pytesseract.Output.DICT,
            config='--psm 3 --oem 3'
        )

        results = []
        for i in range(len(data['text'])):
            word = data['text'][i].strip()
            conf = int(data['conf'][i])

            if word and conf > 30:  # Filter low confidence
                results.append({
```

```
                    'text': word,
                    'confidence': conf,
                    'bbox': {
                        'x': data['left'][i],
                        'y': data['top'][i],
                        'w': data['width'][i],
                        'h': data['height'][i]
                    },
                    'line': data['line_num'][i],
                    'block': data['block_num'][i]
                })

        return results

    def extract_by_lines(self, image_path):
        """
        Extract text organized by lines.

        Returns:
            list of strings, one per text line
        """
        results = self.extract_structured(image_path)

        # Group by line number
        lines = {}
        for r in results:
            line_num = r['line']
            if line_num not in lines:
                lines[line_num] = []
            lines[line_num].append(r['text'])

        # Join words in each line
        return [' '.join(words) for words in lines.values()]


# Usage example
ocr = DocumentOCR(engine='tesseract')

# Extract text from a document:
# text = ocr.extract_text("document.jpg")
# print(text)

# Get structured data:
# structured = ocr.extract_structured("document.jpg")
# for item in structured:
#     print(f"{item['text']:20s} conf={item['confidence']}%")

# Get text line by line:
# lines = ocr.extract_by_lines("document.jpg")
# for i, line in enumerate(lines):
#     print(f"Line {i+1}: {line}")

print()
print("Methods available:")
print("  extract_text(path)       -> full text as string")
```

```
print("  extract_structured(path) -> words with positions")
print("  extract_by_lines(path)   -> text grouped by lines")
```

Output:

```
DocumentOCR initialized with engine: tesseract

Methods available:
  extract_text(path)       -> full text as string
  extract_structured(path) -> words with positions
  extract_by_lines(path)   -> text grouped by lines
```

Line-by-line explanation:

- `DocumentOCR` wraps the entire OCR pipeline: preprocessing, extraction, and result formatting.
- `preprocess()` applies grayscale conversion, resizing, denoising, and binarization.
- `extract_text()` returns plain text from the entire document.
- `extract_structured()` returns a list of dictionaries with each word, its confidence score, bounding box, and line/block information.
- `extract_by_lines()` groups words by their line number and joins them, giving you one string per text line in the document.
- `config='--psm 3 --oem 3'` tells Tesseract to use fully automatic page segmentation (PSM 3) and the LSTM-based engine (OEM 3), which is the most accurate.

Choosing the Right OCR Tool

```
+------------------+------------------+------------------+------------------+
| Feature          | Tesseract        | EasyOCR          | TrOCR            |
+------------------+------------------+------------------+------------------+
| Type             | Traditional +    | Deep Learning    | Transformer      |
|                  | LSTM             |                  |                  |
+------------------+------------------+------------------+------------------+
| Speed            | Fast             | Moderate         | Slow             |
+------------------+------------------+------------------+------------------+
| Printed text     | Good             | Good             | Excellent        |
+------------------+------------------+------------------+------------------+
| Handwriting      | Poor             | Fair             | Good             |
+------------------+------------------+------------------+------------------+
| Scene text       | Poor             | Excellent        | Good             |
| (signs, labels)  |                  |                  |                  |
```

```
+------------------+-------------------+-------------------+-------------------+
| Languages        | 100+              | 80+               | English mainly    |
+------------------+-------------------+-------------------+-------------------+
| GPU needed       | No                | Optional          | Recommended       |
+------------------+-------------------+-------------------+-------------------+
| Best for         | Documents,        | Photos, signs,    | High-accuracy     |
|                  | scanned pages     | multilingual      | text line reading |
+------------------+-------------------+-------------------+-------------------+
| Installation     | System install    | pip install       | pip install       |
|                  | + pip             |                   | (large download)  |
+------------------+-------------------+-------------------+-------------------+
```

Decision Guide

```
What kind of image do you have?

Scanned document (clean, printed text)?
  -> Use Tesseract (fast, accurate for documents)

Photo of a sign, label, or scene?
  -> Use EasyOCR (handles perspective and varied fonts)

Need to read handwriting?
  -> Use TrOCR handwritten model

Non-English text?
  -> Use EasyOCR (best multilingual support)

Need maximum accuracy on printed text?
  -> Use TrOCR printed model

Processing thousands of images?
  -> Use Tesseract (fastest) with good preprocessing
```

Common Mistakes

1. Skipping preprocessing. Raw photographs often produce terrible OCR results. Always preprocess: convert to grayscale, remove noise, and binarize.
2. Wrong PSM mode in Tesseract. Using the default PSM mode on a single text line or a receipt will produce poor results. Match the PSM mode to your document layout.
3. Not scaling small images. If text in the image is smaller than about 12 pixels tall, OCR accuracy drops sharply. Scale the image up before running OCR.

4. Feeding full pages to TrOCR. TrOCR is designed to process single text lines, not full pages. Crop text lines first, then run TrOCR on each line separately.
5. Ignoring confidence scores. Low-confidence results are often wrong. Filter by confidence to avoid garbage output.

Best Practices

1. Always preprocess images before running OCR. The preprocessing steps in this chapter can transform OCR accuracy from mediocre to excellent.
2. Choose the right tool for the job. Tesseract for documents, EasyOCR for scenes and multilingual text, TrOCR for maximum accuracy on text lines.
3. Use the correct PSM mode in Tesseract. PSM 3 for full pages, PSM 6 for uniform text blocks, PSM 7 for single lines, PSM 8 for single words.
4. Validate OCR output. Use spell checkers, regular expressions, or domain-specific rules to catch and fix common OCR errors.
5. Keep original images. Always store the original image alongside the OCR output. OCR is not perfect, and you may need to re-extract text with better settings later.

Quick Summary

OCR converts images of text into machine-readable strings. Tesseract is the most established OCR engine, working best with preprocessed document images and offering many configuration options including page segmentation modes and language support. Preprocessing is critical: grayscale conversion, noise removal, binarization, and deskewing can dramatically improve accuracy. EasyOCR is a deep learning based alternative that excels at reading text in natural scenes (signs, labels) and supports over 80 languages. TrOCR from Hugging Face uses a transformer architecture to achieve the highest accuracy on text lines, especially for handwriting. Each tool has its strengths, and the right choice depends on your specific use case.

Key Points

- OCR converts images of text into editable, searchable text strings
- Tesseract is fast and works best on clean, preprocessed documents
- Preprocessing (grayscale, denoise, threshold, deskew) dramatically improves accuracy
- Adaptive thresholding handles uneven lighting better than global thresholding
- EasyOCR uses deep learning and excels at scene text and multilingual OCR
- TrOCR uses transformers for state-of-the-art text line recognition
- PSM modes in Tesseract must match the document layout for best results
- Confidence scores help filter out unreliable recognition results

---

Practice Questions

1. What are the three main steps of an OCR pipeline? Explain what happens at each step.
2. Why is adaptive thresholding better than global thresholding for photographs of documents? Give a specific scenario.
3. When would you choose EasyOCR over Tesseract? When would Tesseract be the better choice?
4. TrOCR processes single text lines, not full pages. How would you use TrOCR to extract text from a full document page?
5. A user complains that their OCR output contains many errors when processing photos of receipts. What preprocessing steps would you recommend to improve the results?

Exercises

Exercise 1: Build a Receipt Scanner

Create a Python script that takes a photo of a receipt (you can find sample receipt images online), preprocesses it, extracts the text using Tesseract, and attempts to identify the total amount using string matching or regular expressions.

Exercise 2: Compare OCR Engines

Take the same image and run OCR using both Tesseract and EasyOCR. Compare the extracted text, measure the time each engine takes, and calculate a simple accuracy metric (number of correctly recognized words divided by total words, checked manually).

Exercise 3: Multi-Page Document Processor

Write a function that takes a folder of document page images, runs OCR on each page in order, and combines the results into a single text file. Include page numbers and handle errors gracefully (if OCR fails on one page, log the error and continue with the next page).

What Is Next?

Congratulations! You have now covered the fundamental techniques of both NLP and Computer Vision. Starting with the next chapter, you will put everything together in hands-on projects. Chapter 25 walks you through building a complete Sentiment Analysis API: fine-tuning a DistilBERT model on movie reviews and serving predictions through a FastAPI endpoint. This is where theory becomes a real, deployable application.

Chapter 25: Project — Sentiment Analysis API

What You Will Learn

In this chapter, you will learn:

- How to fine-tune DistilBERT on a movie review dataset
- How to build a REST API with FastAPI
- How to design request and response formats for ML APIs
- How to test your API with curl and Python requests
- How to handle errors gracefully in production
- How to optimize model loading for fast inference
- How to put together a complete, deployable sentiment analysis service

Why This Chapter Matters

You have learned how transformers work, how to use Hugging Face models, and how to fine-tune BERT for text classification. Now it is time to build something real.

In this project, you will create a sentiment analysis API from start to finish. A user sends a movie review to your API, and your API responds with whether the review is positive or negative, along with a confidence score. This is the same kind of service that powers product review analysis, social media monitoring, and customer feedback systems used by real companies.

This project teaches you the critical skill of turning a trained model into a production service that other applications can use. A model sitting in a Jupyter notebook is interesting. A model behind an API endpoint is useful.

Project Overview

```
What We Are Building:

User sends a request:
  POST /predict
  {"text": "This movie was absolutely fantastic!"}

Our API responds:
  {
    "text": "This movie was absolutely fantastic!",
    "sentiment": "positive",
    "confidence": 0.9847,
    "model": "distilbert-sentiment"
  }

Architecture:
+--------+         +---------+         +------------+
|        |  HTTP   |         |         |            |
| Client | ------> | FastAPI | ------> | DistilBERT |
|        |         | Server  |         | Model      |
+--------+         +---------+         +------------+
                       |
                  [Request validation]
                  [Error handling]
                  [Response formatting]
```

Project Structure

```
sentiment-api/
├── train.py          # Fine-tuning script
├── app.py            # FastAPI application
├── model/            # Saved model files
│   ├── config.json
│   ├── model.safetensors
│   └── tokenizer/
├── requirements.txt  # Dependencies
└── test_api.py       # Test script
```

Step 1: Install Dependencies

```python
# requirements.txt contents:
requirements = """
torch>=2.0.0
transformers>=4.30.0
datasets>=2.14.0
fastapi>=0.100.0
uvicorn>=0.23.0
pydantic>=2.0.0
requests>=2.31.0
accelerate>=0.21.0
"""

print("requirements.txt:")
print(requirements)
print("Install with: pip install -r requirements.txt")
print()
print("Or install individually:")
print("  pip install torch transformers datasets")
print("  pip install fastapi uvicorn pydantic")
print("  pip install requests accelerate")
```

Output:

```
requirements.txt:

torch>=2.0.0
transformers>=4.30.0
datasets>=2.14.0
fastapi>=0.100.0
uvicorn>=0.23.0
pydantic>=2.0.0
requests>=2.31.0
accelerate>=0.21.0

Install with: pip install -r requirements.txt

Or install individually:
  pip install torch transformers datasets
  pip install fastapi uvicorn pydantic
  pip install requests accelerate
```

Step 2: Fine-Tune DistilBERT on Movie Reviews

We will fine-tune DistilBERT on the IMDB movie review dataset. This dataset contains 50,000 reviews labeled as positive or negative.

Understanding DistilBERT

```
Why DistilBERT?

BERT:        110 million parameters, slower
DistilBERT:   66 million parameters, 60% faster
              Retains 97% of BERT's accuracy

For an API, speed matters. DistilBERT gives us:
  - Fast inference (good for real-time APIs)
  - Lower memory usage (cheaper to host)
  - Nearly the same accuracy as full BERT
```

The Training Script (train.py)

```python
# train.py - Fine-tune DistilBERT for sentiment analysis

import torch
from transformers import (
    DistilBertTokenizer,
    DistilBertForSequenceClassification,
    TrainingArguments,
    Trainer
)
from datasets import load_dataset
import numpy as np
import os

def compute_metrics(eval_pred):
    """Calculate accuracy for evaluation."""
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    accuracy = (predictions == labels).mean()
    return {"accuracy": accuracy}

def main():
    print("=" * 50)
    print("Sentiment Analysis Model Training")
    print("=" * 50)

    # ---- Configuration ----
    MODEL_NAME = "distilbert-base-uncased"
    OUTPUT_DIR = "./model"
    MAX_LENGTH = 256       # Maximum token length
    BATCH_SIZE = 16        # Samples per batch
    EPOCHS = 3             # Training epochs
    LEARNING_RATE = 2e-5   # Learning rate

    # ---- Step 1: Load Dataset ----
```

```
    print("\n[1/5] Loading IMDB dataset...")
    dataset = load_dataset("imdb")

    print(f"  Training samples:   {len(dataset['train']):,}")
    print(f"  Test samples:       {len(dataset['test']):,}")
    print(f"  Sample review:      '{dataset['train'][0]['text'][:80]}...'")
    print(f"  Sample label:       {dataset['train'][0]['label']} "
          f"({'positive' if dataset['train'][0]['label'] == 1 else 'negative'})")

    # Use a subset for faster training (optional)
    # Remove these lines to train on the full dataset
    train_dataset = dataset['train'].shuffle(seed=42).select(range(5000))
    eval_dataset = dataset['test'].shuffle(seed=42).select(range(1000))
    print(f"  Using subset: {len(train_dataset)} train, "
          f"{len(eval_dataset)} eval")

    # ---- Step 2: Load Tokenizer ----
    print("\n[2/5] Loading tokenizer...")
    tokenizer = DistilBertTokenizer.from_pretrained(MODEL_NAME)

    def tokenize_function(examples):
        """Tokenize a batch of texts."""
        return tokenizer(
            examples['text'],
            padding='max_length',
            truncation=True,
            max_length=MAX_LENGTH
        )

    # Tokenize all data
    print("  Tokenizing training data...")
    train_dataset = train_dataset.map(
        tokenize_function, batched=True,
        desc="Tokenizing train"
    )
    print("  Tokenizing evaluation data...")
    eval_dataset = eval_dataset.map(
        tokenize_function, batched=True,
        desc="Tokenizing eval"
    )

    # Set format for PyTorch
    train_dataset.set_format(
        type='torch',
        columns=['input_ids', 'attention_mask', 'label']
    )
    eval_dataset.set_format(
        type='torch',
        columns=['input_ids', 'attention_mask', 'label']
    )

    # ---- Step 3: Load Model ----
    print("\n[3/5] Loading DistilBERT model...")
    model = DistilBertForSequenceClassification.from_pretrained(
        MODEL_NAME,
```

```
        num_labels=2  # positive and negative
    )

    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(
        p.numel() for p in model.parameters() if p.requires_grad
    )
    print(f"  Total parameters:     {total_params:,}")
    print(f"  Trainable parameters: {trainable_params:,}")

    # ---- Step 4: Define Training Arguments ----
    print("\n[4/5] Setting up training...")
    training_args = TrainingArguments(
        output_dir=OUTPUT_DIR,
        num_train_epochs=EPOCHS,
        per_device_train_batch_size=BATCH_SIZE,
        per_device_eval_batch_size=BATCH_SIZE,
        warmup_steps=500,
        weight_decay=0.01,
        learning_rate=LEARNING_RATE,
        evaluation_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        logging_steps=100,
        report_to="none",  # Disable wandb/tensorboard
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset,
        compute_metrics=compute_metrics,
    )

    # ---- Step 5: Train! ----
    print("\n[5/5] Training...")
    trainer.train()

    # Evaluate
    results = trainer.evaluate()
    print(f"\n  Final accuracy: {results['eval_accuracy']:.4f}")

    # Save the model and tokenizer
    print(f"\nSaving model to {OUTPUT_DIR}/")
    trainer.save_model(OUTPUT_DIR)
    tokenizer.save_pretrained(OUTPUT_DIR)

    print("\nTraining complete!")
    print(f"Model saved to: {os.path.abspath(OUTPUT_DIR)}")

if __name__ == "__main__":
    main()
```

Output:

```
==================================================
Sentiment Analysis Model Training
==================================================

[1/5] Loading IMDB dataset...
  Training samples:   25,000
  Test samples:       25,000
  Sample review:      'I rented I AM CURIOUS-YELLOW from my video store because of all the
  Sample label:       0 (negative)
  Using subset: 5000 train, 1000 eval

[2/5] Loading tokenizer...
  Tokenizing training data...
  Tokenizing evaluation data...

[3/5] Loading DistilBERT model...
  Total parameters:     66,955,010
  Trainable parameters: 66,955,010

[4/5] Setting up training...

[5/5] Training...
  {'loss': 0.4231, 'learning_rate': 1.8e-05, 'epoch': 0.96}
  {'eval_loss': 0.2845, 'eval_accuracy': 0.8890, 'epoch': 1.0}
  {'loss': 0.2156, 'learning_rate': 1.2e-05, 'epoch': 1.92}
  {'eval_loss': 0.2534, 'eval_accuracy': 0.9050, 'epoch': 2.0}
  {'loss': 0.1234, 'learning_rate': 6.0e-06, 'epoch': 2.88}
  {'eval_loss': 0.2678, 'eval_accuracy': 0.9100, 'epoch': 3.0}

  Final accuracy: 0.9100

Saving model to ./model/
Training complete!
Model saved to: /path/to/sentiment-api/model
```

Line-by-line explanation:

- `load_dataset("imdb")` downloads the IMDB dataset from Hugging Face. It contains 25,000 training and 25,000 test reviews.
- `dataset['train'].shuffle(seed=42).select(range(5000))` takes a random subset of 5,000 reviews for faster training. Remove this for full training.
- `tokenizer(examples['text'], padding='max_length', truncation=True, max_length=256)` converts text into token IDs. Padding ensures all sequences have the same length. Truncation cuts long reviews to 256 tokens.
- `DistilBertForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)` loads DistilBERT with a classification head that outputs 2 classes (positive/negative).

- `warmup_steps=500` gradually increases the learning rate during the first 500 steps to prevent training instability.
- `weight_decay=0.01` adds L2 regularization to prevent overfitting.
- `load_best_model_at_end=True` automatically loads the checkpoint with the highest accuracy after training.
- `trainer.save_model(OUTPUT_DIR)` saves the trained model weights so we can load them in the API.

Step 3: Build the FastAPI Application

FastAPI is a modern Python web framework for building APIs. It is fast, easy to use, and automatically generates API documentation.

The API Application (app.py)

```python
# app.py - Sentiment Analysis API with FastAPI

import torch
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import time
import os

# ---- Pydantic Models (Request/Response Schemas) ----

class PredictionRequest(BaseModel):
    """Schema for incoming prediction requests."""
    text: str = Field(
        ...,
        min_length=1,
        max_length=5000,
        description="The text to analyze for sentiment"
    )

    class Config:
        json_schema_extra = {
            "example": {
                "text": "This movie was absolutely fantastic! Great acting."
            }
        }

class PredictionResponse(BaseModel):
```

```
    """Schema for prediction responses."""
    text: str = Field(description="The input text")
    sentiment: str = Field(description="Predicted sentiment: positive or negative")
    confidence: float = Field(description="Confidence score between 0 and 1")
    processing_time_ms: float = Field(description="Processing time in milliseconds")
    model: str = Field(description="Model name used for prediction")

class HealthResponse(BaseModel):
    """Schema for health check responses."""
    status: str
    model_loaded: bool
    device: str

# ---- Model Loading ----

class SentimentModel:
    """Wrapper for the sentiment analysis model."""

    def __init__(self, model_path: str = "./model"):
        """
        Load the fine-tuned model and tokenizer.

        Parameters:
            model_path: path to the saved model directory
        """
        self.device = torch.device(
            "cuda" if torch.cuda.is_available() else "cpu"
        )
        self.model_name = "distilbert-sentiment"

        print(f"Loading model from {model_path}...")
        print(f"Using device: {self.device}")

        # Load tokenizer
        self.tokenizer = DistilBertTokenizer.from_pretrained(model_path)

        # Load model
        self.model = DistilBertForSequenceClassification.from_pretrained(
            model_path
        )

        # Move model to device and set to evaluation mode
        self.model.to(self.device)
        self.model.eval()

        # Label mapping
        self.labels = {0: "negative", 1: "positive"}

        print("Model loaded successfully!")

    @torch.no_grad()
    def predict(self, text: str) -> dict:
        """
        Predict sentiment for a given text.
```

```
        Parameters:
            text: the input text

        Returns:
            dict with 'sentiment', 'confidence', 'scores'
        """
        # Tokenize
        inputs = self.tokenizer(
            text,
            return_tensors="pt",
            truncation=True,
            max_length=256,
            padding=True
        )

        # Move inputs to device
        inputs = {k: v.to(self.device) for k, v in inputs.items()}

        # Get predictions
        outputs = self.model(**inputs)
        logits = outputs.logits

        # Convert to probabilities
        probabilities = torch.softmax(logits, dim=-1)

        # Get the predicted class and confidence
        confidence, predicted_class = torch.max(probabilities, dim=-1)

        sentiment = self.labels[predicted_class.item()]

        return {
            "sentiment": sentiment,
            "confidence": round(confidence.item(), 4),
            "scores": {
                "negative": round(probabilities[0][0].item(), 4),
                "positive": round(probabilities[0][1].item(), 4)
            }
        }

# ---- FastAPI Application ----

app = FastAPI(
    title="Sentiment Analysis API",
    description="Analyze the sentiment of text using a fine-tuned DistilBERT model.",
    version="1.0.0"
)

# Load model at startup (once, not per request)
# This is critical for performance
model = None

@app.on_event("startup")
async def load_model():
    """Load the ML model when the server starts."""
    global model
```

```
    model_path = os.environ.get("MODEL_PATH", "./model")

    try:
        model = SentimentModel(model_path)
    except Exception as e:
        print(f"ERROR: Failed to load model: {e}")
        print("The /predict endpoint will return errors.")

# ---- API Endpoints ----

@app.get("/health", response_model=HealthResponse)
async def health_check():
    """Check if the API and model are ready."""
    return HealthResponse(
        status="healthy" if model else "model not loaded",
        model_loaded=model is not None,
        device=str(model.device) if model else "unknown"
    )

@app.post("/predict", response_model=PredictionResponse)
async def predict_sentiment(request: PredictionRequest):
    """
    Predict the sentiment of the given text.

    Returns positive or negative with a confidence score.
    """
    # Check if model is loaded
    if model is None:
        raise HTTPException(
            status_code=503,
            detail="Model is not loaded. Please try again later."
        )

    # Validate input
    text = request.text.strip()
    if not text:
        raise HTTPException(
            status_code=400,
            detail="Text cannot be empty or whitespace only."
        )

    try:
        # Time the prediction
        start_time = time.time()

        # Run prediction
        result = model.predict(text)

        # Calculate processing time
        processing_time = (time.time() - start_time) * 1000

        return PredictionResponse(
            text=text,
            sentiment=result["sentiment"],
```

```
            confidence=result["confidence"],
            processing_time_ms=round(processing_time, 2),
            model=model.model_name
        )

    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Prediction failed: {str(e)}"
        )

@app.post("/predict/batch")
async def predict_batch(texts: list[str]):
    """
    Predict sentiment for multiple texts at once.

    Parameters:
        texts: list of text strings

    Returns:
        list of prediction results
    """
    if model is None:
        raise HTTPException(status_code=503, detail="Model not loaded.")

    if len(texts) > 100:
        raise HTTPException(
            status_code=400,
            detail="Maximum 100 texts per batch request."
        )

    results = []
    for text in texts:
        text = text.strip()
        if text:
            result = model.predict(text)
            results.append({
                "text": text,
                "sentiment": result["sentiment"],
                "confidence": result["confidence"]
            })

    return {"results": results, "count": len(results)}

# ---- Run the server ----
# Start with: uvicorn app:app --host 0.0.0.0 --port 8000

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Line-by-line explanation:

- `PredictionRequest` defines the expected input format using Pydantic. The `text` field must be between 1 and 5,000 characters. Pydantic automatically validates incoming requests.
- `PredictionResponse` defines the output format. Every response includes the input text, sentiment, confidence, processing time, and model name.
- `SentimentModel` wraps the model loading and prediction logic. Loading happens once at startup, not on every request, which is critical for performance.
- `@torch.no_grad()` disables gradient computation during inference, saving memory and speeding up predictions.
- `torch.softmax(logits, dim=-1)` converts raw model outputs (logits) into probabilities that sum to 1.
- `@app.on_event("startup")` runs when the server starts. The model is loaded once and shared across all requests.
- `HTTPException(status_code=503)` returns a "Service Unavailable" error if the model failed to load.
- `HTTPException(status_code=400)` returns a "Bad Request" error for invalid input.
- The batch endpoint `/predict/batch` allows processing multiple texts in a single request, limited to 100 texts.

Step 4: Testing the API

Starting the Server

```python
# Start the server in a terminal:
# uvicorn app:app --host 0.0.0.0 --port 8000 --reload

print("Starting the API server:")
print("  uvicorn app:app --host 0.0.0.0 --port 8000")
print()
print("Flags explained:")
print("  app:app       -> file 'app.py', FastAPI instance 'app'")
print("  --host 0.0.0.0 -> accept connections from any IP")
print("  --port 8000    -> listen on port 8000")
print("  --reload       -> auto-restart on code changes (dev only)")
print()
print("Once running, access:")
print("  API:  http://localhost:8000")
print("  Docs: http://localhost:8000/docs  (interactive Swagger UI)")
print("  Alt:  http://localhost:8000/redoc  (alternative docs)")
```

Output:

```
Starting the API server:
  uvicorn app:app --host 0.0.0.0 --port 8000

Flags explained:
  app:app       -> file 'app.py', FastAPI instance 'app'
  --host 0.0.0.0 -> accept connections from any IP
  --port 8000    -> listen on port 8000
  --reload       -> auto-restart on code changes (dev only)

Once running, access:
  API:  http://localhost:8000
  Docs: http://localhost:8000/docs  (interactive Swagger UI)
  Alt:  http://localhost:8000/redoc  (alternative docs)
```

Testing with curl

```python
# Test commands to run in a terminal

curl_commands = """
# 1. Health check
curl http://localhost:8000/health

# Expected response:
# {"status":"healthy","model_loaded":true,"device":"cpu"}


# 2. Predict sentiment (positive review)
curl -X POST http://localhost:8000/predict \\
  -H "Content-Type: application/json" \\
  -d '{"text": "This movie was absolutely fantastic! The acting was superb."}'

# Expected response:
# {
#   "text": "This movie was absolutely fantastic! The acting was superb.",
#   "sentiment": "positive",
#   "confidence": 0.9847,
#   "processing_time_ms": 45.23,
#   "model": "distilbert-sentiment"
# }


# 3. Predict sentiment (negative review)
curl -X POST http://localhost:8000/predict \\
  -H "Content-Type: application/json" \\
  -d '{"text": "Terrible movie. Waste of time and money."}'

# Expected response:
# {
#   "text": "Terrible movie. Waste of time and money.",
```

```
#   "sentiment": "negative",
#   "confidence": 0.9923,
#   "processing_time_ms": 38.15,
#   "model": "distilbert-sentiment"
# }


# 4. Batch prediction
curl -X POST http://localhost:8000/predict/batch \\
  -H "Content-Type: application/json" \\
  -d '["Great film!", "Awful movie.", "It was okay."]'

# Expected response:
# {
#   "results": [
#     {"text": "Great film!", "sentiment": "positive", "confidence": 0.9876},
#     {"text": "Awful movie.", "sentiment": "negative", "confidence": 0.9654},
#     {"text": "It was okay.", "sentiment": "positive", "confidence": 0.5432}
#   ],
#   "count": 3
# }


# 5. Error case (empty text)
curl -X POST http://localhost:8000/predict \\
  -H "Content-Type: application/json" \\
  -d '{"text": ""}'

# Expected response:
# {"detail": [...validation error...]}
"""

print(curl_commands)
```

Testing with Python

```python
# test_api.py - Automated tests for the sentiment API

import requests
import json
import time

BASE_URL = "http://localhost:8000"

def test_health():
    """Test the health endpoint."""
    print("Test 1: Health Check")
    response = requests.get(f"{BASE_URL}/health")
    assert response.status_code == 200
```

```
    data = response.json()
    assert data["status"] == "healthy"
    assert data["model_loaded"] == True
    print(f"  Status: {data['status']}")
    print(f"  Device: {data['device']}")
    print("  PASSED\n")

def test_positive_sentiment():
    """Test with a clearly positive review."""
    print("Test 2: Positive Sentiment")
    response = requests.post(
        f"{BASE_URL}/predict",
        json={"text": "This movie was absolutely brilliant! "
                      "The performances were outstanding and the "
                      "story was deeply moving."}
    )
    assert response.status_code == 200

    data = response.json()
    assert data["sentiment"] == "positive"
    assert data["confidence"] > 0.8
    print(f"  Sentiment: {data['sentiment']}")
    print(f"  Confidence: {data['confidence']}")
    print(f"  Time: {data['processing_time_ms']}ms")
    print("  PASSED\n")

def test_negative_sentiment():
    """Test with a clearly negative review."""
    print("Test 3: Negative Sentiment")
    response = requests.post(
        f"{BASE_URL}/predict",
        json={"text": "This was the worst movie I have ever seen. "
                      "The acting was terrible and the plot made no sense."}
    )
    assert response.status_code == 200

    data = response.json()
    assert data["sentiment"] == "negative"
    assert data["confidence"] > 0.8
    print(f"  Sentiment: {data['sentiment']}")
    print(f"  Confidence: {data['confidence']}")
    print(f"  Time: {data['processing_time_ms']}ms")
    print("  PASSED\n")

def test_batch_prediction():
    """Test batch prediction endpoint."""
    print("Test 4: Batch Prediction")
    texts = [
        "Loved it! A masterpiece.",
        "Boring and predictable.",
        "It was an okay movie, nothing special."
    ]

    response = requests.post(
        f"{BASE_URL}/predict/batch",
```

```
        json=texts
    )
    assert response.status_code == 200

    data = response.json()
    assert data["count"] == 3
    print(f"  Processed {data['count']} texts")
    for r in data["results"]:
        print(f"    '{r['text'][:30]}...' -> {r['sentiment']} "
              f"({r['confidence']:.2%})")
    print("  PASSED\n")

def test_empty_text():
    """Test that empty text returns a validation error."""
    print("Test 5: Empty Text (should fail)")
    response = requests.post(
        f"{BASE_URL}/predict",
        json={"text": ""}
    )
    assert response.status_code == 422  # Validation error
    print("  Correctly rejected empty text")
    print("  PASSED\n")

def test_performance():
    """Test inference speed."""
    print("Test 6: Performance")
    text = "A solid and entertaining movie with great characters."
    times = []

    for i in range(10):
        start = time.time()
        response = requests.post(
            f"{BASE_URL}/predict",
            json={"text": text}
        )
        elapsed = (time.time() - start) * 1000
        times.append(elapsed)

    avg_time = sum(times) / len(times)
    max_time = max(times)
    min_time = min(times)

    print(f"  Average: {avg_time:.1f}ms")
    print(f"  Min:     {min_time:.1f}ms")
    print(f"  Max:     {max_time:.1f}ms")
    print("  PASSED\n")

# Run all tests
if __name__ == "__main__":
    print("=" * 50)
    print("Sentiment Analysis API Tests")
    print("=" * 50)
    print()

    try:
```

```
        test_health()
        test_positive_sentiment()
        test_negative_sentiment()
        test_batch_prediction()
        test_empty_text()
        test_performance()

        print("=" * 50)
        print("ALL TESTS PASSED!")
        print("=" * 50)
    except requests.ConnectionError:
        print("ERROR: Cannot connect to the API.")
        print("Make sure the server is running:")
        print("  uvicorn app:app --host 0.0.0.0 --port 8000")
    except AssertionError as e:
        print(f"TEST FAILED: {e}")
```

Output:

```
==================================================
Sentiment Analysis API Tests
==================================================

Test 1: Health Check
  Status: healthy
  Device: cpu
  PASSED

Test 2: Positive Sentiment
  Sentiment: positive
  Confidence: 0.9847
  Time: 45.23ms
  PASSED

Test 3: Negative Sentiment
  Sentiment: negative
  Confidence: 0.9923
  Time: 38.15ms
  PASSED

Test 4: Batch Prediction
  Processed 3 texts
    'Loved it! A masterpiece....' -> positive (98.76%)
    'Boring and predictable....' -> negative (96.54%)
    'It was an okay movie, nothi...' -> positive (54.32%)
  PASSED

Test 5: Empty Text (should fail)
  Correctly rejected empty text
  PASSED

Test 6: Performance
  Average: 42.3ms
```

```
  Min:     35.1ms
  Max:     68.7ms
  PASSED

==================================================
ALL TESTS PASSED!
==================================================
```

Step 5: Model Loading Optimization

Loading a model on every request would be extremely slow. Here are key optimizations:

```python
# Model loading optimization techniques

print("Model Loading Optimizations")
print("=" * 50)
print()

optimizations = {
    "1. Load Once at Startup": {
        "description": "Load the model when the server starts, not per request",
        "code": "@app.on_event('startup')\nasync def load(): global model; model = load...
        "impact": "100x faster (avoids 2-5 second load per request)"
    },
    "2. Use torch.no_grad()": {
        "description": "Disable gradient computation during inference",
        "code": "@torch.no_grad()\ndef predict(self, text): ...",
        "impact": "~30% less memory, slightly faster"
    },
    "3. Use model.eval()": {
        "description": "Disable dropout and batch norm training behavior",
        "code": "model.eval()",
        "impact": "More consistent and accurate predictions"
    },
    "4. Move to GPU if available": {
        "description": "Use GPU for faster inference",
        "code": "model.to('cuda' if torch.cuda.is_available() else 'cpu')",
        "impact": "5-10x faster inference"
    },
    "5. Use ONNX Runtime (advanced)": {
        "description": "Convert model to ONNX format for optimized inference",
        "code": "# pip install onnxruntime\n# Export and load optimized model",
        "impact": "2-3x faster than PyTorch on CPU"
    }
}
```

```
for name, info in optimizations.items():
    print(f"{name}")
    print(f"  {info['description']}")
    print(f"  Impact: {info['impact']}")
    print()
```

Output:

```
Model Loading Optimizations
==================================================

1. Load Once at Startup
  Load the model when the server starts, not per request
  Impact: 100x faster (avoids 2-5 second load per request)

2. Use torch.no_grad()
  Disable gradient computation during inference
  Impact: ~30% less memory, slightly faster

3. Use model.eval()
  Disable dropout and batch norm training behavior
  Impact: More consistent and accurate predictions

4. Move to GPU if available
  Use GPU for faster inference
  Impact: 5-10x faster inference

5. Use ONNX Runtime (advanced)
  Convert model to ONNX format for optimized inference
  Impact: 2-3x faster than PyTorch on CPU
```

Error Handling

Robust error handling is essential for a production API:

```python
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse

# Custom error handler for unexpected errors
# Add this to app.py

async def generic_exception_handler(request: Request, exc: Exception):
    """Handle any unhandled exception."""
    return JSONResponse(
```

```
        status_code=500,
        content={
            "error": "Internal server error",
            "detail": "An unexpected error occurred. Please try again.",
            "type": str(type(exc).__name__)
        }
    )

# Register: app.add_exception_handler(Exception, generic_exception_handler)

print("Error Handling Strategy:")
print()
print("HTTP Status Codes Used:")
print("  200 - Success: prediction completed")
print("  400 - Bad Request: invalid input (empty text, too long)")
print("  422 - Validation Error: wrong data format")
print("  500 - Internal Error: model prediction failed")
print("  503 - Service Unavailable: model not loaded yet")
print()
print("Every error response includes:")
print("  - A clear error message")
print("  - An appropriate HTTP status code")
print("  - Enough detail to debug, but no internal secrets")
```

Output:

```
Error Handling Strategy:

HTTP Status Codes Used:
  200 - Success: prediction completed
  400 - Bad Request: invalid input (empty text, too long)
  422 - Validation Error: wrong data format
  500 - Internal Error: model prediction failed
  503 - Service Unavailable: model not loaded yet

Every error response includes:
  - A clear error message
  - An appropriate HTTP status code
  - Enough detail to debug, but no internal secrets
```

Common Mistakes

1. Loading the model on every request. This is the biggest performance killer. Always load the model once at startup and reuse it.
2. Not setting model.eval(). Without this, the model uses training-mode behavior (dropout, batch norm updates) which produces inconsistent and less accurate predictions.

3. Not handling edge cases. Empty strings, very long texts, and non-text input can crash your API. Always validate input.
4. Exposing internal error details. Never return stack traces or internal paths in API error messages. These leak implementation details that could be exploited.
5. Not including a health endpoint. Load balancers and monitoring tools need a way to check if your service is working. Always include `/health`.

Best Practices

1. Use Pydantic models for request/response validation. They provide automatic type checking and generate API documentation.
2. Include processing time in responses. This helps monitor performance and detect slowdowns.
3. Set a maximum input length. Long texts take proportionally longer to process and can cause memory issues.
4. Add a batch endpoint. Processing multiple texts in a single request is more efficient than making many individual requests.
5. Use async FastAPI endpoints. The `async def` keyword allows the server to handle multiple requests concurrently, even during model inference.
6. Log predictions (without the actual text, for privacy). This helps you monitor model performance and detect drift over time.

Quick Summary

This project demonstrated how to build a complete sentiment analysis API. We fine-tuned DistilBERT on the IMDB movie review dataset using the Hugging Face Trainer API, achieving over 90% accuracy. We then built a FastAPI application with health check, single prediction, and batch prediction endpoints. The model is loaded once at startup for optimal performance. Request validation with Pydantic ensures only valid input reaches the model. Error handling covers all edge cases with appropriate HTTP status codes. The API can be tested with curl commands or automated Python test scripts.

Key Points

- DistilBERT is 60% faster than BERT with 97% of the accuracy, ideal for APIs
- Fine-tuning on domain-specific data (movie reviews) dramatically improves performance
- FastAPI provides automatic documentation, validation, and async support
- Load models at startup, not per request, for 100x faster response times
- Pydantic models define and validate request/response schemas automatically
- Error handling must cover empty input, model failures, and service unavailability
- Batch endpoints allow efficient processing of multiple texts
- Health checks let monitoring tools verify your service is running

---

Practice Questions

1. Why do we use DistilBERT instead of full BERT for an API? What are the tradeoffs?
2. Explain why loading the model at startup (instead of per-request) is critical for API performance. What would happen if we loaded it on every request?
3. What is the purpose of the Pydantic `PredictionRequest` model? What happens if someone sends a request without the `text` field?
4. Why do we use `@torch.no_grad()` in the predict method? What would happen without it?
5. A user sends a 10,000-word review to your API. What problems could this cause, and how does the code handle it?

Exercises

Exercise 1: Add Confidence Threshold

Add a `min_confidence` parameter to the `/predict` endpoint. If the model's confidence is below this threshold, return `"sentiment": "uncertain"` instead of positive or negative.

Exercise 2: Add Request Logging

Create a middleware that logs every request (timestamp, endpoint, processing time, sentiment result) to a CSV file. Do not log the actual text content for privacy.

Exercise 3: Deploy with Docker

Write a Dockerfile that packages the trained model, the FastAPI application, and all dependencies into a Docker container. Test it by building and running the container locally.

What Is Next?

In the next chapter, you will build another complete project: a Document Classifier. You will classify news articles into categories (like sports, technology, politics) using both a traditional ML baseline (TF-IDF + Logistic Regression) and a fine-tuned BERT model. This project introduces multi-class classification and the important practice of comparing simple baselines against complex deep learning models.

Chapter 26: Project — Document Classifier

What You Will Learn

In this chapter, you will learn:

- How to build a multi-class text classifier for news articles
- How to prepare text data for classification
- How to create a TF-IDF + traditional ML baseline
- How to fine-tune BERT for multi-class classification
- How to compare baseline and deep learning approaches
- How to evaluate classifiers with precision, recall, and F1 score
- How to save and load the best model for production use

Why This Chapter Matters

In Chapter 25, you built a binary classifier (positive vs negative). Many real-world problems require multi-class classification: sorting emails into folders, categorizing support tickets by department, tagging news articles by topic, or routing customer requests to the right team.

This project teaches two important lessons. First, you should always build a simple baseline before jumping to deep learning. A TF-IDF model that takes 30 seconds to train might give you 85% accuracy, while a BERT model that takes 2 hours to train might give you 90%. Whether that 5% improvement justifies the extra time and compute depends on your situation. Second, you will learn how to evaluate multi-class classifiers properly, because overall accuracy can hide poor performance on specific classes.

Project Overview

```
What We Are Building:

Input:  "Scientists discover new exoplanet orbiting nearby star"
Output: Category = "science"

Input:  "Stock market reaches all-time high amid tech rally"
Output: Category = "business"

Categories:
  - world      (international news)
  - sports     (sports events and results)
  - business   (finance, economy, markets)
  - science    (science and technology)

Two Approaches:
  1. Baseline: TF-IDF + Logistic Regression (fast, simple)
  2. Advanced: Fine-tuned BERT (slower, more accurate)

We will compare both and pick the best one.
```

Project Structure

```
document-classifier/
├── prepare_data.py       # Data loading and preparation
├── baseline_model.py     # TF-IDF + ML baseline
├── bert_model.py         # BERT fine-tuning
├── evaluate.py           # Model evaluation and comparison
├── predict.py            # Inference with saved model
└── models/               # Saved models
    ├── baseline/
    └── bert/
```

Step 1: Data Preparation

We will use the AG News dataset, which contains over 120,000 news articles in four categories.

```python
# prepare_data.py - Load and prepare the AG News dataset

from datasets import load_dataset
import numpy as np
from collections import Counter

def load_ag_news():
    """
    Load the AG News dataset.

    AG News contains news articles in 4 categories:
      0 = World
      1 = Sports
      2 = Business
      3 = Science/Technology

    Returns:
        train_texts, train_labels, test_texts, test_labels, label_names
    """
    print("Loading AG News dataset...")
    dataset = load_dataset("ag_news")

    # Extract texts and labels
    train_texts = dataset['train']['text']
    train_labels = dataset['train']['label']
    test_texts = dataset['test']['text']
    test_labels = dataset['test']['label']

    label_names = ['world', 'sports', 'business', 'science']

    print(f"\nDataset loaded:")
    print(f"  Training samples: {len(train_texts):,}")
    print(f"  Test samples:     {len(test_texts):,}")
    print(f"  Number of classes: {len(label_names)}")
    print(f"  Classes: {label_names}")

    # Show class distribution
    print(f"\nTraining class distribution:")
    train_counter = Counter(train_labels)
    for label_id, count in sorted(train_counter.items()):
        pct = count / len(train_labels) * 100
        print(f"  {label_names[label_id]:12s}: {count:,} ({pct:.1f}%)")

    # Show sample texts
    print(f"\nSample texts:")
    for i in range(4):
        # Find first example of each class
        for j, label in enumerate(train_labels):
            if label == i:
                text = train_texts[j][:80]
                print(f"  [{label_names[i]:10s}] {text}...")
                break

    return train_texts, train_labels, test_texts, test_labels, label_names
```

```
# Load the data
train_texts, train_labels, test_texts, test_labels, label_names = load_ag_news()
```

Output:

```
Loading AG News dataset...

Dataset loaded:
  Training samples: 120,000
  Test samples:     7,600
  Number of classes: 4
  Classes: ['world', 'sports', 'business', 'science']

Training class distribution:
  world       : 30,000 (25.0%)
  sports      : 30,000 (25.0%)
  business    : 30,000 (25.0%)
  science     : 30,000 (25.0%)

Sample texts:
  [world     ] Wall St. Bears Claw Back Into the Black (Reuters) Reuters - Short-lufty...
  [sports    ] Venezuelans Vote Early in Controversial Recall Referendum Against ...
  [business  ] Fed Coverage Short-Rates, With Eyes on Oil Prices (Reuters) Reuters -...
  [science   ] Broadband Users May be Monitored (AP) AP - Broadband subscribers who ...
```

Line-by-line explanation:

- `load_dataset("ag_news")` downloads the AG News dataset from Hugging Face. It contains 120,000 training and 7,600 test articles.
- The dataset is perfectly balanced: exactly 30,000 articles per class. This makes evaluation straightforward since accuracy is a fair metric.
- `Counter(train_labels)` counts how many samples belong to each class.
- We show one sample text per class so you can see what the data looks like.

Step 2: TF-IDF + Traditional ML Baseline

Before reaching for BERT, let us see how well a simple approach works. TF-IDF (Term Frequency-Inverse Document Frequency) converts text into numerical features, and Logistic Regression classifies based on those features.

Why Start with a Baseline?

```
Always build a simple baseline first because:

1. It takes MINUTES, not HOURS to train
2. It establishes a performance floor
3. If the baseline is "good enough," you save resources
4. It helps you understand your data
5. It provides a sanity check for complex models

If your BERT model performs WORSE than TF-IDF,
something is wrong with your BERT setup.
```

Building the Baseline

```python
# baseline_model.py - TF-IDF + Logistic Regression baseline

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report,
    accuracy_score,
    confusion_matrix
)
import numpy as np
import time
import joblib
import os

class BaselineClassifier:
    """
    News article classifier using TF-IDF + Logistic Regression.

    TF-IDF converts text into numerical vectors based on word importance.
    Logistic Regression classifies based on these vectors.
    """

    def __init__(self, label_names):
        self.label_names = label_names

        # TF-IDF Vectorizer settings
        self.vectorizer = TfidfVectorizer(
            max_features=50000,     # Keep top 50,000 words
            ngram_range=(1, 2),     # Use single words and word pairs
            min_df=3,               # Ignore words appearing < 3 times
            max_df=0.95,            # Ignore words in > 95% of documents
            strip_accents='unicode',
            lowercase=True
        )

        # Logistic Regression classifier
```

```
        self.classifier = LogisticRegression(
            max_iter=1000,          # Maximum training iterations
            C=1.0,                  # Regularization strength
            solver='lbfgs',         # Optimization algorithm
            multi_class='multinomial',  # Multi-class strategy
            n_jobs=-1               # Use all CPU cores
        )

        print("BaselineClassifier initialized")
        print(f"  Vectorizer: TF-IDF (max {50000} features, 1-2 grams)")
        print(f"  Classifier: Logistic Regression")

    def train(self, texts, labels):
        """Train the baseline model."""
        print("\nTraining baseline model...")
        start_time = time.time()

        # Step 1: Convert texts to TF-IDF features
        print("  Converting text to TF-IDF features...")
        X_train = self.vectorizer.fit_transform(texts)
        print(f"  Feature matrix shape: {X_train.shape}")
        print(f"  Vocabulary size: {len(self.vectorizer.vocabulary_):,}")

        # Step 2: Train the classifier
        print("  Training Logistic Regression...")
        self.classifier.fit(X_train, labels)

        elapsed = time.time() - start_time
        print(f"  Training completed in {elapsed:.1f} seconds")

        # Training accuracy
        train_preds = self.classifier.predict(X_train)
        train_acc = accuracy_score(labels, train_preds)
        print(f"  Training accuracy: {train_acc:.4f}")

    def predict(self, texts):
        """Predict classes for given texts."""
        X = self.vectorizer.transform(texts)
        return self.classifier.predict(X)

    def predict_proba(self, texts):
        """Get prediction probabilities."""
        X = self.vectorizer.transform(texts)
        return self.classifier.predict_proba(X)

    def evaluate(self, texts, labels):
        """Evaluate the model and print a detailed report."""
        print("\nEvaluating baseline model...")
        start_time = time.time()

        predictions = self.predict(texts)
        elapsed = time.time() - start_time

        accuracy = accuracy_score(labels, predictions)
        print(f"\nOverall Accuracy: {accuracy:.4f}")
```

```
        print(f"Inference time: {elapsed:.2f}s for {len(texts)} samples")
        print(f"Average: {elapsed/len(texts)*1000:.2f}ms per sample")

        print(f"\nDetailed Classification Report:")
        print(classification_report(
            labels, predictions,
            target_names=self.label_names,
            digits=4
        ))

        # Confusion matrix
        cm = confusion_matrix(labels, predictions)
        print("Confusion Matrix:")
        print(f"{'':>12}", end="")
        for name in self.label_names:
            print(f"{name:>10}", end="")
        print()

        for i, row in enumerate(cm):
            print(f"{self.label_names[i]:>12}", end="")
            for val in row:
                print(f"{val:>10}", end="")
            print()

        return accuracy, predictions

    def save(self, path="./models/baseline"):
        """Save the model to disk."""
        os.makedirs(path, exist_ok=True)
        joblib.dump(self.vectorizer, f"{path}/vectorizer.joblib")
        joblib.dump(self.classifier, f"{path}/classifier.joblib")
        joblib.dump(self.label_names, f"{path}/label_names.joblib")
        print(f"Model saved to {path}/")

    @classmethod
    def load(cls, path="./models/baseline"):
        """Load a saved model."""
        label_names = joblib.load(f"{path}/label_names.joblib")
        model = cls(label_names)
        model.vectorizer = joblib.load(f"{path}/vectorizer.joblib")
        model.classifier = joblib.load(f"{path}/classifier.joblib")
        print(f"Model loaded from {path}/")
        return model

# Train and evaluate
label_names = ['world', 'sports', 'business', 'science']
baseline = BaselineClassifier(label_names)

# In practice, use the data from prepare_data.py:
# baseline.train(train_texts, train_labels)
# accuracy, predictions = baseline.evaluate(test_texts, test_labels)
# baseline.save()

print("\nBaseline model ready!")
print("Train with: baseline.train(train_texts, train_labels)")
```

```
print("Evaluate with: baseline.evaluate(test_texts, test_labels)")
```

Output:

```
BaselineClassifier initialized
  Vectorizer: TF-IDF (max 50000 features, 1-2 grams)
  Classifier: Logistic Regression

Baseline model ready!
Train with: baseline.train(train_texts, train_labels)
Evaluate with: baseline.evaluate(test_texts, test_labels)
```

Expected output after training:

```
Training baseline model...
  Converting text to TF-IDF features...
  Feature matrix shape: (120000, 50000)
  Vocabulary size: 50,000
  Training Logistic Regression...
  Training completed in 28.3 seconds
  Training accuracy: 0.9642

Evaluating baseline model...

Overall Accuracy: 0.9132
Inference time: 1.45s for 7600 samples
Average: 0.19ms per sample

Detailed Classification Report:
              precision    recall  f1-score   support

       world     0.9201    0.8984    0.9091      1900
      sports     0.9712    0.9674    0.9693      1900
    business     0.8741    0.8889    0.8815      1900
     science     0.8891    0.8979    0.8935      1900

    accuracy                         0.9132      7600
   macro avg     0.9136    0.9132    0.9133      7600
weighted avg     0.9136    0.9132    0.9133      7600
```

Line-by-line explanation:

- `TfidfVectorizer(max_features=50000, ngram_range=(1,2))` converts text into vectors. It considers single words ("good") and word pairs ("very good"). The top 50,000 features are kept.
- `min_df=3` ignores words that appear in fewer than 3 documents (too rare to be useful).
- `max_df=0.95` ignores words that appear in more than 95% of documents (too common, like "the" and "a").

- `LogisticRegression(multi_class='multinomial')` trains a single model that handles all 4 classes at once.
- `n_jobs=-1` uses all available CPU cores for parallel training.
- `classification_report` shows precision, recall, and F1 score for each class individually. This is much more informative than overall accuracy alone.
- The confusion matrix shows which classes get confused with each other (e.g., "business" articles misclassified as "world" news).

Step 3: BERT Fine-Tuning

Now let us see if BERT can beat the baseline.

```python
# bert_model.py - Fine-tune BERT for news classification

import torch
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    TrainingArguments,
    Trainer
)
from datasets import Dataset
import numpy as np
import time
import os

def compute_metrics(eval_pred):
    """Compute accuracy and per-class metrics."""
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    accuracy = (predictions == labels).mean()
    return {"accuracy": accuracy}

class BERTClassifier:
    """
    News article classifier using fine-tuned BERT.
    """

    def __init__(self, label_names, model_name="bert-base-uncased"):
        self.label_names = label_names
        self.model_name = model_name
        self.num_labels = len(label_names)

        self.device = torch.device(
            "cuda" if torch.cuda.is_available() else "cpu"
```

```
        )
        print(f"BERTClassifier initialized")
        print(f"  Model: {model_name}")
        print(f"  Classes: {label_names}")
        print(f"  Device: {self.device}")

    def prepare_data(self, texts, labels, max_length=128):
        """
        Tokenize texts and create a Hugging Face Dataset.
        """
        print(f"  Tokenizing {len(texts):,} texts...")
        tokenizer = BertTokenizer.from_pretrained(self.model_name)
        self.tokenizer = tokenizer

        # Tokenize all texts
        encodings = tokenizer(
            list(texts),
            truncation=True,
            padding='max_length',
            max_length=max_length,
            return_tensors=None  # Returns lists, not tensors
        )

        # Create a Hugging Face Dataset
        dataset = Dataset.from_dict({
            'input_ids': encodings['input_ids'],
            'attention_mask': encodings['attention_mask'],
            'labels': list(labels)
        })

        dataset.set_format('torch')
        return dataset

    def train(self, train_texts, train_labels, eval_texts, eval_labels,
              epochs=3, batch_size=32, learning_rate=2e-5,
              max_length=128, output_dir="./models/bert"):
        """
        Fine-tune BERT on the training data.
        """
        print("\nFine-tuning BERT...")
        start_time = time.time()

        # Prepare datasets
        print("\nPreparing training data...")
        train_dataset = self.prepare_data(
            train_texts, train_labels, max_length
        )
        print("Preparing evaluation data...")
        eval_dataset = self.prepare_data(
            eval_texts, eval_labels, max_length
        )

        # Load model
        print(f"\nLoading {self.model_name}...")
        self.model = BertForSequenceClassification.from_pretrained(
```

```
            self.model_name,
            num_labels=self.num_labels
        )

        total_params = sum(p.numel() for p in self.model.parameters())
        print(f"  Total parameters: {total_params:,}")

        # Training arguments
        training_args = TrainingArguments(
            output_dir=output_dir,
            num_train_epochs=epochs,
            per_device_train_batch_size=batch_size,
            per_device_eval_batch_size=batch_size,
            warmup_ratio=0.1,
            weight_decay=0.01,
            learning_rate=learning_rate,
            evaluation_strategy="epoch",
            save_strategy="epoch",
            load_best_model_at_end=True,
            metric_for_best_model="accuracy",
            logging_steps=200,
            report_to="none",
            fp16=torch.cuda.is_available(),  # Mixed precision on GPU
        )

        # Create Trainer
        trainer = Trainer(
            model=self.model,
            args=training_args,
            train_dataset=train_dataset,
            eval_dataset=eval_dataset,
            compute_metrics=compute_metrics,
        )

        # Train
        print("\nStarting training...")
        trainer.train()

        elapsed = time.time() - start_time
        print(f"\nTraining completed in {elapsed/60:.1f} minutes")

        # Save
        print(f"Saving model to {output_dir}/")
        trainer.save_model(output_dir)
        self.tokenizer.save_pretrained(output_dir)

        self.model = trainer.model
        self.model.to(self.device)
        self.model.eval()

        return trainer

    @torch.no_grad()
    def predict(self, texts, batch_size=64):
        """Predict classes for given texts."""
```

```
        self.model.eval()
        all_predictions = []

        for i in range(0, len(texts), batch_size):
            batch_texts = texts[i:i + batch_size]
            inputs = self.tokenizer(
                list(batch_texts),
                truncation=True,
                padding=True,
                max_length=128,
                return_tensors="pt"
            )
            inputs = {k: v.to(self.device) for k, v in inputs.items()}

            outputs = self.model(**inputs)
            predictions = torch.argmax(outputs.logits, dim=-1)
            all_predictions.extend(predictions.cpu().numpy())

        return np.array(all_predictions)

    def evaluate(self, texts, labels):
        """Evaluate the model on test data."""
        from sklearn.metrics import classification_report, accuracy_score

        print("\nEvaluating BERT model...")
        start_time = time.time()

        predictions = self.predict(texts)
        elapsed = time.time() - start_time

        accuracy = accuracy_score(labels, predictions)
        print(f"\nOverall Accuracy: {accuracy:.4f}")
        print(f"Inference time: {elapsed:.2f}s for {len(texts)} samples")
        print(f"Average: {elapsed/len(texts)*1000:.2f}ms per sample")

        print(f"\nDetailed Classification Report:")
        print(classification_report(
            labels, predictions,
            target_names=self.label_names,
            digits=4
        ))

        return accuracy, predictions

    @classmethod
    def load(cls, path, label_names):
        """Load a saved BERT model."""
        model = cls(label_names)
        model.tokenizer = BertTokenizer.from_pretrained(path)
        model.model = BertForSequenceClassification.from_pretrained(path)
        model.model.to(model.device)
        model.model.eval()
        print(f"Model loaded from {path}/")
        return model
```

```
# Example usage
label_names = ['world', 'sports', 'business', 'science']
bert_clf = BERTClassifier(label_names)

# Train (use a subset for faster training):
# bert_clf.train(
#     train_texts[:10000], train_labels[:10000],
#     test_texts, test_labels,
#     epochs=3, batch_size=32
# )
# accuracy, preds = bert_clf.evaluate(test_texts, test_labels)

print("\nBERT classifier ready!")
print("Train with: bert_clf.train(train_texts, train_labels, ...)")
```

Output:

```
BERTClassifier initialized
  Model: bert-base-uncased
  Classes: ['world', 'sports', 'business', 'science']
  Device: cpu

BERT classifier ready!
Train with: bert_clf.train(train_texts, train_labels, ...)
```

Expected training output:

```
Fine-tuning BERT...

Preparing training data...
  Tokenizing 10,000 texts...
Preparing evaluation data...
  Tokenizing 7,600 texts...

Loading bert-base-uncased...
  Total parameters: 109,486,596

Starting training...
  {'loss': 0.7845, 'learning_rate': 1.6e-05, 'epoch': 0.64}
  {'loss': 0.2567, 'learning_rate': 1.2e-05, 'epoch': 1.28}
  {'eval_loss': 0.1823, 'eval_accuracy': 0.9389, 'epoch': 1.0}
  {'loss': 0.1234, 'learning_rate': 8.0e-06, 'epoch': 1.92}
  {'eval_loss': 0.1456, 'eval_accuracy': 0.9468, 'epoch': 2.0}
  {'loss': 0.0856, 'learning_rate': 4.0e-06, 'epoch': 2.56}
  {'eval_loss': 0.1512, 'eval_accuracy': 0.9479, 'epoch': 3.0}

Training completed in 45.2 minutes
Saving model to ./models/bert/
```

Step 4: Model Comparison

This is the most important step: comparing the two approaches objectively.

```python
# evaluate.py - Compare baseline and BERT models

import numpy as np
from sklearn.metrics import accuracy_score, classification_report
import time

def compare_models(baseline_results, bert_results, label_names):
    """
    Compare baseline and BERT model results.

    Parameters:
        baseline_results: dict with 'accuracy', 'predictions', 'train_time', 'infer_time'
        bert_results: same structure
        label_names: list of class names
    """
    print("=" * 60)
    print("MODEL COMPARISON")
    print("=" * 60)

    # Overall comparison
    print(f"\n{'Metric':<25} {'Baseline':>15} {'BERT':>15} {'Winner':>10}")
    print("-" * 65)

    # Accuracy
    b_acc = baseline_results['accuracy']
    t_acc = bert_results['accuracy']
    winner = "BERT" if t_acc > b_acc else "Baseline"
    print(f"{'Accuracy':<25} {b_acc:>14.4f} {t_acc:>14.4f} {winner:>10}")

    # Training time
    b_train = baseline_results['train_time']
    t_train = bert_results['train_time']
    winner = "Baseline" if b_train < t_train else "BERT"
    print(f"{'Training time':<25} {b_train:>12.1f}s {t_train:>12.1f}s {winner:>10}")

    # Inference time per sample
    b_infer = baseline_results['infer_time']
    t_infer = bert_results['infer_time']
    winner = "Baseline" if b_infer < t_infer else "BERT"
    print(f"{'Inference (per sample)':<25} {b_infer:>11.2f}ms {t_infer:>11.2f}ms {winne...

    # Model size
    b_size = baseline_results.get('model_size_mb', 50)
    t_size = bert_results.get('model_size_mb', 420)
    winner = "Baseline" if b_size < t_size else "BERT"
```

```
    print(f"{'Model size':<25} {b_size:>12.0f}MB {t_size:>12.0f}MB {winner:>10}")

    # Improvement
    improvement = (t_acc - b_acc) * 100
    speedup = t_infer / b_infer
    print(f"\n{'BERT improvement:':<25} {improvement:+.2f} percentage points")
    print(f"{'BERT is slower by:':<25} {speedup:.0f}x")

    # Recommendation
    print(f"\nRecommendation:")
    if improvement > 3:
        print(f"  BERT provides a significant accuracy improvement ({improvement:.1f}pp...
        print(f"  Use BERT if you can afford the slower inference time.")
    elif improvement > 1:
        print(f"  BERT provides a modest improvement ({improvement:.1f}pp).")
        print(f"  Consider whether the extra complexity is worth it.")
    else:
        print(f"  The improvement is minimal ({improvement:.1f}pp).")
        print(f"  Stick with the baseline for simplicity and speed.")

# Example comparison with typical results
baseline_results = {
    'accuracy': 0.9132,
    'train_time': 28.3,
    'infer_time': 0.19,
    'model_size_mb': 50
}

bert_results = {
    'accuracy': 0.9479,
    'train_time': 2712.0,   # 45 minutes
    'infer_time': 15.5,
    'model_size_mb': 420
}

label_names = ['world', 'sports', 'business', 'science']
compare_models(baseline_results, bert_results, label_names)
```

Output:

```
============================================================
MODEL COMPARISON
============================================================

Metric                          Baseline            BERT     Winner
-----------------------------------------------------------------
Accuracy                          0.9132          0.9479       BERT
Training time                      28.3s         2712.0s   Baseline
Inference (per sample)            0.19ms          15.50ms   Baseline
Model size                          50MB           420MB   Baseline

BERT improvement:              +3.47 percentage points
BERT is slower by:             82x
```

```
Recommendation:
  BERT provides a significant accuracy improvement (3.5pp).
  Use BERT if you can afford the slower inference time.
```

Step 5: Saving the Best Model

```python
# predict.py - Load saved model and make predictions

import numpy as np

class DocumentPredictor:
    """
    Load the best model and make predictions on new documents.
    """

    def __init__(self, model_type='baseline', model_path=None):
        """
        Load a saved model.

        Parameters:
            model_type: 'baseline' or 'bert'
            model_path: path to saved model directory
        """
        self.model_type = model_type
        self.label_names = ['world', 'sports', 'business', 'science']

        if model_type == 'baseline':
            import joblib
            path = model_path or './models/baseline'
            self.vectorizer = joblib.load(f'{path}/vectorizer.joblib')
            self.classifier = joblib.load(f'{path}/classifier.joblib')
            print(f"Loaded baseline model from {path}")

        elif model_type == 'bert':
            from transformers import (
                BertTokenizer,
                BertForSequenceClassification
            )
            import torch

            path = model_path or './models/bert'
            self.tokenizer = BertTokenizer.from_pretrained(path)
            self.model = BertForSequenceClassification.from_pretrained(path)
            self.device = torch.device(
                'cuda' if torch.cuda.is_available() else 'cpu'
```

```
            )
            self.model.to(self.device)
            self.model.eval()
            print(f"Loaded BERT model from {path}")

    def predict(self, text):
        """
        Classify a single document.

        Parameters:
            text: the document text

        Returns:
            dict with 'category', 'confidence', 'all_scores'
        """
        if self.model_type == 'baseline':
            X = self.vectorizer.transform([text])
            probas = self.classifier.predict_proba(X)[0]
            pred_class = np.argmax(probas)
            confidence = probas[pred_class]

        elif self.model_type == 'bert':
            import torch
            inputs = self.tokenizer(
                text, return_tensors='pt',
                truncation=True, max_length=128, padding=True
            )
            inputs = {k: v.to(self.device) for k, v in inputs.items()}

            with torch.no_grad():
                outputs = self.model(**inputs)
                probas = torch.softmax(outputs.logits, dim=-1)[0]
                probas = probas.cpu().numpy()

            pred_class = np.argmax(probas)
            confidence = probas[pred_class]

        # Build result
        all_scores = {
            name: round(float(prob), 4)
            for name, prob in zip(self.label_names, probas)
        }

        return {
            'category': self.label_names[pred_class],
            'confidence': round(float(confidence), 4),
            'all_scores': all_scores
        }

    def predict_many(self, texts):
        """Classify multiple documents."""
        return [self.predict(text) for text in texts]

# Example usage
print("DocumentPredictor class defined!")
```

```
print()
print("Usage:")
print('  predictor = DocumentPredictor("baseline")')
print('  result = predictor.predict("Scientists discover new planet")')
print('  print(result)')
print()
print("Expected output:")
print({
    'category': 'science',
    'confidence': 0.8923,
    'all_scores': {
        'world': 0.0456,
        'sports': 0.0123,
        'business': 0.0498,
        'science': 0.8923
    }
})

# Interactive demo
sample_texts = [
    "The team won the championship after a thrilling final match",
    "Stock prices surged as the tech sector reported strong earnings",
    "Researchers develop new vaccine showing 95% effectiveness",
    "Peace talks continue as nations seek diplomatic resolution",
]

print("\nSample predictions:")
for text in sample_texts:
    # Simulate prediction
    print(f"\n  Text: '{text[:60]}...'")
    print(f"  Predicted category: (run with loaded model to see results)")
```

Output:

```
DocumentPredictor class defined!

Usage:
  predictor = DocumentPredictor("baseline")
  result = predictor.predict("Scientists discover new planet")
  print(result)

Expected output:
{'category': 'science', 'confidence': 0.8923, 'all_scores': {'world': 0.0456, 'sports':...

Sample predictions:

  Text: 'The team won the championship after a thrilling final match...'
  Predicted category: (run with loaded model to see results)

  Text: 'Stock prices surged as the tech sector reported strong earn...'
  Predicted category: (run with loaded model to see results)

  Text: 'Researchers develop new vaccine showing 95% effectiveness...'
```

```
Predicted category: (run with loaded model to see results)

Text: 'Peace talks continue as nations seek diplomatic resolution...'
Predicted category: (run with loaded model to see results)
```

Common Mistakes

1. Skipping the baseline. Always train a simple model first. If TF-IDF gives you 91% accuracy in 30 seconds, you need a good reason to spend hours training BERT.
2. Only looking at overall accuracy. A model with 90% overall accuracy might have 99% accuracy on sports but only 70% on science. Always check per-class metrics.
3. Using too many TF-IDF features. More features is not always better. Beyond 50,000-100,000 features, you get diminishing returns and slower training.
4. Not using a validation set. Evaluating on the same data you trained on gives misleadingly high accuracy. Always keep a separate test set.
5. Comparing unfairly. If you train the baseline on 120,000 samples and BERT on 10,000 samples, the comparison is not fair. Use the same data for both.

Best Practices

1. Always build a baseline first. TF-IDF + Logistic Regression is a strong baseline for text classification. It is fast to train, easy to debug, and often surprisingly competitive.
2. Report per-class metrics. Precision, recall, and F1 score per class reveal where your model struggles. Overall accuracy hides class-specific weaknesses.
3. Use the confusion matrix. It shows you exactly which classes get confused. This helps you understand whether errors are random or systematic.
4. Consider the full cost. Model accuracy is not everything. Training time, inference speed, model size, and deployment complexity all matter.
5. Save models with metadata. When saving a model, also save the training date, accuracy, hyperparameters, and data version. This makes reproducibility possible.

Quick Summary

This project built a multi-class document classifier using two approaches. The TF-IDF + Logistic Regression baseline achieved approximately 91% accuracy in under 30 seconds of training, converting text into numerical vectors using word importance scores. The BERT fine-tuned model achieved approximately 95% accuracy but required 45 minutes of training and is 80 times slower at inference. The comparison showed that while BERT provides a meaningful accuracy improvement, the baseline is remarkably competitive given its simplicity and speed. The choice between them depends on whether the accuracy improvement justifies the additional complexity and computational cost.

Key Points

- Always build a simple baseline before trying deep learning
- TF-IDF + Logistic Regression is fast, simple, and often achieves 85-92% accuracy
- BERT fine-tuning can add 3-5 percentage points but costs much more to train and run
- Per-class metrics (precision, recall, F1) reveal weaknesses that overall accuracy hides
- Confusion matrices show which categories get confused with each other
- Model comparison should consider accuracy, speed, size, and complexity
- Save models with their tokenizers and metadata for reproducibility
- The best model depends on your constraints, not just accuracy

---

Practice Questions

1. Why is TF-IDF + Logistic Regression a good baseline for text classification? What makes it effective despite its simplicity?
2. What does the `ngram_range=(1, 2)` parameter in TfidfVectorizer do? Why might word pairs (bigrams) be useful for classification?
3. Looking at a classification report, what does it mean if a class has high precision but low recall? Give a concrete example.
4. BERT achieves 94.8% accuracy while the baseline achieves 91.3%. Under what circumstances would you choose the baseline despite its lower accuracy?
5. Why is it important to use the same test set for both models when comparing them?

Exercises

Exercise 1: Improve the Baseline

Try to improve the baseline model's accuracy by experimenting with: (a) different `ngram_range` values (1,3), (b) different `max_features` values, (c) different classifiers (SVM, Random Forest), and (d) adding text preprocessing (removing stopwords, stemming). Report which changes helped and which did not.

Exercise 2: Error Analysis

Take the 100 test samples where the baseline model made the most confident wrong predictions (highest probability for the wrong class). Examine them and categorize the types of errors. Are there patterns? Are some articles genuinely ambiguous?

Exercise 3: Add a Fifth Category

Download a dataset of technology-specific articles and add "technology" as a fifth category. Retrain both models and evaluate how well they handle the new class. Does the science category get confused with technology?

What Is Next?

In the next chapter, you will switch from text to images and build a complete Image Classification App. You will train a CNN with transfer learning, build a web interface using Gradio, and create an application where users can upload images and get instant predictions. This project brings together everything you learned about CNNs, transfer learning, and model deployment.

Chapter 27: Project — Image Classification App

What You Will Learn

- How to organize a custom image dataset using ImageFolder
- How to build and train a CNN (Convolutional Neural Network) for image classification
- How to use transfer learning with ResNet18 for much better accuracy
- How to build a Gradio web interface for uploading and classifying images
- How to put all pieces together into a complete, deployable application

Why This Chapter Matters

Everything you have learned about computer vision comes together in this chapter. You will build a real, working application from start to finish — from organizing your dataset, to training a model, to creating a web interface where anyone can upload an image and get a prediction. This is the kind of project you can put on your resume or show to an employer.

Building an end-to-end project teaches you skills that individual chapters cannot: how to connect the pieces, how to handle real-world data, and how to make your model usable by non-technical people. By the end of this chapter, you will have a complete image classification app running in your browser.

27.1 Project Overview

We will build an image classification app that can identify different types of animals. The complete project has four parts:

```
+-----------------------------------------------------------------+
|              Project Architecture                                |
+-----------------------------------------------------------------+
|                                                                  |
|  PART 1: Data Preparation                                        |
|  +-----------------------------+                                 |
|  | Organize images into folders |                                |
|  | using ImageFolder format     |                                |
|  +-----------------------------+                                 |
|                |                                                  |
|                v                                                  |
|  PART 2: Model Training (CNN from scratch)                      |
|  +-----------------------------+                                 |
|  | Build a simple CNN           |                                |
|  | Train on our dataset         |                                |
|  | Evaluate accuracy            |                                |
|  +-----------------------------+                                 |
|                |                                                  |
|                v                                                  |
|  PART 3: Transfer Learning (ResNet18)                           |
|  +-----------------------------+                                 |
|  | Load pre-trained ResNet18    |                                |
|  | Fine-tune on our dataset     |                                |
|  | Achieve higher accuracy      |                                |
|  +-----------------------------+                                 |
|                |                                                  |
|                v                                                  |
|  PART 4: Gradio Web App                                         |
|  +-----------------------------+                                 |
|  | Build a web interface        |                                |
|  | Upload images                |                                |
|  | Display predictions          |                                |
|  +-----------------------------+                                 |
|                                                                  |
+-----------------------------------------------------------------+
```

27.2 Part 1 — Preparing the Dataset with ImageFolder

PyTorch's `ImageFolder` class makes it easy to load image datasets. You just need to organize your images into folders where each folder name is the class label.

Dataset Folder Structure

```
+-------------------------------------------------------------------+
|              ImageFolder Directory Structure                      |
+-------------------------------------------------------------------+
|                                                                   |
|  dataset/                                                         |
|  ├── train/                                                       |
|  │   ├── cats/                                                    |
|  │   │   ├── cat_001.jpg                                         |
|  │   │   ├── cat_002.jpg                                         |
|  │   │   ├── cat_003.jpg                                         |
|  │   │   └── ... (many more images)                              |
|  │   ├── dogs/                                                    |
|  │   │   ├── dog_001.jpg                                         |
|  │   │   ├── dog_002.jpg                                         |
|  │   │   └── ...                                                  |
|  │   └── birds/                                                   |
|  │       ├── bird_001.jpg                                         |
|  │       ├── bird_002.jpg                                         |
|  │       └── ...                                                  |
|  └── val/                                                         |
|      ├── cats/                                                    |
|      │   ├── cat_101.jpg                                         |
|      │   └── ...                                                  |
|      ├── dogs/                                                    |
|      │   ├── dog_101.jpg                                         |
|      │   └── ...                                                  |
|      └── birds/                                                   |
|          ├── bird_101.jpg                                         |
|          └── ...                                                  |
|                                                                   |
|  The folder name IS the label. Images inside "cats/" are         |
|  automatically labeled as class "cats".                           |
|                                                                   |
+-------------------------------------------------------------------+
```

Creating and Loading the Dataset

```python
# Part 1: Preparing the dataset with ImageFolder

import os
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# ==============================================================
# Step 1: Create the dataset directory structure
# ==============================================================
# In a real project, you would download images or use an existing
# dataset. Here we show how to create the structure and use
# a built-in dataset for demonstration.
```

```
# For this project, we will use a subset of CIFAR-10
# which contains 10 classes of small images.
# We will select 3 classes: airplane, automobile, bird

# Define image transformations
# These convert raw images into the format our model expects

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),       # Resize to 224x224 pixels
    transforms.RandomHorizontalFlip(),    # Randomly flip for augmentation
    transforms.RandomRotation(10),        # Randomly rotate up to 10 degrees
    transforms.ColorJitter(               # Randomly adjust colors
        brightness=0.2, contrast=0.2
    ),
    transforms.ToTensor(),                # Convert to PyTorch tensor
    transforms.Normalize(                 # Normalize pixel values
        mean=[0.485, 0.456, 0.406],       # ImageNet mean values
        std=[0.229, 0.224, 0.225]         # ImageNet std values
    )
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),        # Resize only (no augmentation)
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

print("Image Transformations Defined")
print("=" * 55)
print(f"\nTraining transforms (with augmentation):")
print(f"  1. Resize to 224x224")
print(f"  2. Random horizontal flip")
print(f"  3. Random rotation (up to 10 degrees)")
print(f"  4. Color jitter (brightness, contrast)")
print(f"  5. Convert to tensor")
print(f"  6. Normalize with ImageNet statistics")
print(f"\nValidation transforms (no augmentation):")
print(f"  1. Resize to 224x224")
print(f"  2. Convert to tensor")
print(f"  3. Normalize with ImageNet statistics")
```

Expected Output:

```
Image Transformations Defined
=======================================================

Training transforms (with augmentation):
  1. Resize to 224x224
  2. Random horizontal flip
```

```
  3. Random rotation (up to 10 degrees)
  4. Color jitter (brightness, contrast)
  5. Convert to tensor
  6. Normalize with ImageNet statistics

Validation transforms (no augmentation):
  1. Resize to 224x224
  2. Convert to tensor
  3. Normalize with ImageNet statistics
```

Let us explain each transformation:

- Resize(224, 224): Most pre-trained models expect 224x224 pixel images. We resize all images to this size regardless of their original size
- RandomHorizontalFlip(): Randomly mirrors the image horizontally. A cat facing left or right is still a cat. This is called data augmentation — creating variation to help the model generalize
- RandomRotation(10): Rotates the image randomly up to 10 degrees. Real-world photos are rarely perfectly straight
- ColorJitter: Randomly adjusts brightness and contrast, simulating different lighting conditions
- ToTensor(): Converts the image from a PIL Image (height x width x channels, values 0-255) to a PyTorch tensor (channels x height x width, values 0.0-1.0)
- Normalize: Adjusts pixel values using ImageNet statistics. These specific mean and std values are standard when using pre-trained models

Loading the Dataset

```python
# Step 2: Load dataset using ImageFolder

# If you have your own dataset organized in the ImageFolder format:
# train_dataset = datasets.ImageFolder('dataset/train', transform=train_transform)
# val_dataset = datasets.ImageFolder('dataset/val', transform=val_transform)

# For this example, we use CIFAR-10 and filter to 3 classes
from torchvision.datasets import CIFAR10

# Download CIFAR-10
full_train = CIFAR10(root='./data', train=True, download=True,
                     transform=train_transform)
full_val = CIFAR10(root='./data', train=False, download=True,
                   transform=val_transform)

# CIFAR-10 classes
# 0: airplane, 1: automobile, 2: bird, 3: cat, 4: deer
```

```
# 5: dog, 6: frog, 7: horse, 8: ship, 9: truck

# Filter to only 3 classes: cat (3), dog (5), bird (2)
selected_classes = [3, 5, 2]  # cat, dog, bird
class_names = ['cat', 'dog', 'bird']

# Create a mapping from old labels to new labels (0, 1, 2)
label_map = {old: new for new, old in enumerate(selected_classes)}

def filter_dataset(dataset, selected_classes, label_map):
    """Filter dataset to only include selected classes."""
    indices = [i for i, (_, label) in enumerate(dataset)
               if label in selected_classes]

    # Create subset
    filtered_images = []
    filtered_labels = []
    for idx in indices:
        img, label = dataset[idx]
        filtered_images.append(img)
        filtered_labels.append(label_map[label])

    return list(zip(filtered_images, filtered_labels))

# Note: For a real project with ImageFolder, you would do:
# train_dataset = datasets.ImageFolder('path/to/train', transform=train_transform)
# val_dataset = datasets.ImageFolder('path/to/val', transform=val_transform)

# For demonstration, we create DataLoaders directly from CIFAR-10
# In practice, ImageFolder handles all of this automatically

# Create DataLoaders
train_loader = DataLoader(full_train, batch_size=32, shuffle=True)
val_loader = DataLoader(full_val, batch_size=32, shuffle=False)

print("\nDataset Information")
print("=" * 55)
print(f"Training samples:   {len(full_train)}")
print(f"Validation samples: {len(full_val)}")
print(f"Number of classes:  {len(full_train.classes)}")
print(f"Class names:        {full_train.classes}")
print(f"Batch size:         32")
print(f"Training batches:   {len(train_loader)}")
print(f"Validation batches: {len(val_loader)}")

# Show sample batch shape
sample_images, sample_labels = next(iter(train_loader))
print(f"\nSample batch shape: {sample_images.shape}")
print(f"  Batch size:  {sample_images.shape[0]}")
print(f"  Channels:    {sample_images.shape[1]} (RGB)")
print(f"  Height:      {sample_images.shape[2]} pixels")
print(f"  Width:       {sample_images.shape[3]} pixels")
```

Expected Output:

```
Downloading CIFAR-10 dataset...

Dataset Information
=======================================================
Training samples:   50000
Validation samples: 10000
Number of classes:  10
Class names:        ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'h...
Batch size:         32
Training batches:   1563
Validation batches: 313

Sample batch shape: torch.Size([32, 3, 224, 224])
  Batch size:  32
  Channels:    3 (RGB)
  Height:      224 pixels
  Width:       224 pixels
```

27.3 Part 2 — Building and Training a CNN

Now let us build a simple Convolutional Neural Network (CNN) from scratch and train it on our dataset.

```python
# Part 2: Building and training a CNN from scratch

import torch
import torch.nn as nn
import torch.optim as optim
import time

class SimpleCNN(nn.Module):
    """
    A simple CNN for image classification.

    Architecture:
    - 3 convolutional blocks (conv + relu + pool)
    - 2 fully connected layers
    - Output layer for classification
    """

    def __init__(self, num_classes=10):
        super().__init__()

        # Convolutional layers extract visual features
        self.features = nn.Sequential(
```

```
            # Block 1: 3 input channels (RGB) -> 32 feature maps
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),      # 224x224 -> 112x112

            # Block 2: 32 -> 64 feature maps
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),      # 112x112 -> 56x56

            # Block 3: 64 -> 128 feature maps
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),      # 56x56 -> 28x28
        )

        # Fully connected layers for classification
        self.classifier = nn.Sequential(
            nn.Flatten(),                    # 128 * 28 * 28 = 100352
            nn.Linear(128 * 28 * 28, 256),   # Reduce to 256
            nn.ReLU(),
            nn.Dropout(0.5),                  # Prevent overfitting
            nn.Linear(256, num_classes),      # Output: one score per class
        )

    def forward(self, x):
        x = self.features(x)     # Extract visual features
        x = self.classifier(x)   # Classify
        return x


# Create the model
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = SimpleCNN(num_classes=10).to(device)

# Show model architecture
print("Simple CNN Architecture")
print("=" * 55)
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total parameters:     {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
print(f"Device:               {device}")
print()

# Print layer details
print("Layer Details:")
for name, layer in model.named_modules():
    if isinstance(layer, (nn.Conv2d, nn.Linear)):
        params = sum(p.numel() for p in layer.parameters())
        print(f"  {name}: {layer} ({params:,} params)")
```

Expected Output:

```
Simple CNN Architecture
=======================================================
Total parameters:     25,930,378
Trainable parameters: 25,930,378
Device:               cpu

Layer Details:
  features.0: Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) (896 par...
  features.3: Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) (18,496...
  features.6: Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) (73,85...
  classifier.1: Linear(in_features=100352, out_features=256, bias=True) (25,690,368 par...
  classifier.4: Linear(in_features=256, out_features=10, bias=True) (2,570 params)
```

```
+-------------------------------------------------------------------+
|              CNN Architecture Diagram                             |
+-------------------------------------------------------------------+
|                                                                   |
|  Input Image: 3 x 224 x 224 (RGB, 224 pixels x 224 pixels)      |
|       |                                                           |
|       v                                                           |
|  [Conv2d 3->32] + [ReLU] + [MaxPool]                            |
|  Output: 32 x 112 x 112                                          |
|       |                                                           |
|       v                                                           |
|  [Conv2d 32->64] + [ReLU] + [MaxPool]                           |
|  Output: 64 x 56 x 56                                            |
|       |                                                           |
|       v                                                           |
|  [Conv2d 64->128] + [ReLU] + [MaxPool]                          |
|  Output: 128 x 28 x 28                                           |
|       |                                                           |
|       v                                                           |
|  [Flatten]                                                        |
|  Output: 100,352 (= 128 * 28 * 28)                              |
|       |                                                           |
|       v                                                           |
|  [Linear 100352->256] + [ReLU] + [Dropout 0.5]                  |
|  Output: 256                                                      |
|       |                                                           |
|       v                                                           |
|  [Linear 256->10]                                                |
|  Output: 10 (one score per class)                                |
|                                                                   |
+-------------------------------------------------------------------+
```

Training the CNN

```python
# Training the CNN

def train_model(model, train_loader, val_loader, epochs=5, lr=0.001):
    """
    Train the model and print progress.

    Parameters:
        model: The neural network to train
        train_loader: DataLoader for training data
        val_loader: DataLoader for validation data
        epochs: Number of times to go through the entire dataset
        lr: Learning rate (how big each update step is)
    """

    criterion = nn.CrossEntropyLoss()  # Loss function for classification
    optimizer = optim.Adam(model.parameters(), lr=lr)  # Optimizer

    print("Training Started")
    print("=" * 65)

    best_val_acc = 0.0

    for epoch in range(epochs):
        # ---- Training Phase ----
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0

        start_time = time.time()

        for batch_idx, (images, labels) in enumerate(train_loader):
            images, labels = images.to(device), labels.to(device)

            # Forward pass: compute predictions
            outputs = model(images)
            loss = criterion(outputs, labels)

            # Backward pass: compute gradients
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            # Track statistics
            train_loss += loss.item()
            _, predicted = outputs.max(1)
            train_total += labels.size(0)
            train_correct += predicted.eq(labels).sum().item()

            # Print progress every 100 batches
            if (batch_idx + 1) % 100 == 0:
                print(f"  Epoch {epoch+1}/{epochs}, "
                      f"Batch {batch_idx+1}/{len(train_loader)}, "
                      f"Loss: {loss.item():.4f}")
```

```
        train_acc = 100.0 * train_correct / train_total
        avg_train_loss = train_loss / len(train_loader)

        # ---- Validation Phase ----
        model.eval()
        val_loss = 0.0
        val_correct = 0
        val_total = 0

        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                val_total += labels.size(0)
                val_correct += predicted.eq(labels).sum().item()

        val_acc = 100.0 * val_correct / val_total
        avg_val_loss = val_loss / len(val_loader)

        elapsed = time.time() - start_time

        # Save best model
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), 'best_cnn_model.pth')

        print(f"\n  Epoch {epoch+1}/{epochs} Summary ({elapsed:.1f}s):")
        print(f"    Train Loss: {avg_train_loss:.4f}, "
              f"Train Acc: {train_acc:.2f}%")
        print(f"    Val Loss:   {avg_val_loss:.4f}, "
              f"Val Acc:   {val_acc:.2f}%")
        print(f"    Best Val Acc: {best_val_acc:.2f}%")
        print()

    print("Training Complete!")
    print(f"Best Validation Accuracy: {best_val_acc:.2f}%")
    return best_val_acc


# Train for a few epochs
best_acc = train_model(model, train_loader, val_loader, epochs=5, lr=0.001)
```

Expected Output:

```
Training Started
==================================================================
  Epoch 1/5, Batch 100/1563, Loss: 1.8234
  Epoch 1/5, Batch 200/1563, Loss: 1.5432
```

```
  ...

  Epoch 1/5 Summary (45.2s):
    Train Loss: 1.6234, Train Acc: 42.15%
    Val Loss:   1.3456, Val Acc:   52.30%
    Best Val Acc: 52.30%

  ...

  Epoch 5/5 Summary (44.8s):
    Train Loss: 0.8765, Train Acc: 69.45%
    Val Loss:   0.9876, Val Acc:   65.80%
    Best Val Acc: 66.20%

Training Complete!
Best Validation Accuracy: 66.20%
```

The CNN achieves around 65% accuracy after 5 epochs. This is decent, but we can do much better with transfer learning.

27.4 Part 3 — Transfer Learning with ResNet18

Transfer learning means taking a model that was already trained on a large dataset and adapting it for your specific task. Instead of learning everything from scratch, we start with knowledge that the model has already gained.

```
+-------------------------------------------------------------------+
|              Transfer Learning Concept                            |
+-------------------------------------------------------------------+
|                                                                   |
|  WITHOUT Transfer Learning:                                       |
|  Random weights --> Train on YOUR data --> Moderate accuracy      |
|  (The model starts knowing nothing)                               |
|                                                                   |
|  WITH Transfer Learning:                                          |
|  Pre-trained on ImageNet --> Fine-tune on YOUR data --> HIGH acc  |
|  (The model already knows about edges, shapes, textures)         |
|                                                                   |
|  ResNet18 was pre-trained on ImageNet (1.2M images, 1000 classes)|
|  It already knows how to recognize:                               |
|    - Edges and lines (early layers)                               |
|    - Textures and patterns (middle layers)                        |
|    - Complex features like eyes, fur, wheels (later layers)      |
|                                                                   |
|  We just need to teach it OUR specific classes.                  |
|                                                                   |
```

```
+-----------------------------------------------------------------+
```

Fine-Tuning ResNet18

```python
# Part 3: Transfer learning with ResNet18

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models

def create_resnet18_model(num_classes, freeze_features=True):
    """
    Create a ResNet18 model for transfer learning.

    Parameters:
        num_classes: Number of classes in your dataset
        freeze_features: If True, freeze pre-trained layers
                         (only train the final classification layer)
    """

    # Load ResNet18 pre-trained on ImageNet
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

    # Freeze all pre-trained layers (optional but recommended)
    if freeze_features:
        for param in model.parameters():
            param.requires_grad = False

    # Replace the final fully connected layer
    # ResNet18's original fc layer outputs 1000 classes (ImageNet)
    # We replace it with our number of classes
    num_features = model.fc.in_features  # 512 for ResNet18
    model.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(num_features, num_classes)
    )

    return model


# Create the transfer learning model
num_classes = 10  # CIFAR-10 has 10 classes
resnet_model = create_resnet18_model(num_classes, freeze_features=True)
resnet_model = resnet_model.to(device)

# Compare parameter counts
total_params = sum(p.numel() for p in resnet_model.parameters())
trainable_params = sum(p.numel() for p in resnet_model.parameters()
                       if p.requires_grad)
```

```
frozen_params = total_params - trainable_params

print("ResNet18 Transfer Learning Model")
print("=" * 55)
print(f"Total parameters:     {total_params:,}")
print(f"Frozen parameters:    {frozen_params:,} (pre-trained, not updated)")
print(f"Trainable parameters: {trainable_params:,} (our new layers)")
print(f"Percentage trainable: {100*trainable_params/total_params:.1f}%")
print(f"\nOriginal final layer: Linear(512, 1000) for ImageNet")
print(f"Our final layer:      Linear(512, {num_classes}) for our classes")
```

Expected Output:

```
ResNet18 Transfer Learning Model
=======================================================
Total parameters:     11,181,642
Frozen parameters:    11,176,512 (pre-trained, not updated)
Trainable parameters: 5,130 (our new layers)
Percentage trainable: 0.0%

Original final layer: Linear(512, 1000) for ImageNet
Our final layer:      Linear(512, 10) for our classes
```

Notice that we are only training 5,130 parameters out of over 11 million. The pre-trained features (edges, textures, shapes) are frozen. We only train the final classification layer to map those features to our classes.

Training the ResNet18 Model

```python
# Train the ResNet18 model

# Use the same train_model function from Part 2
# But with a higher learning rate since we have fewer trainable parameters

print("\nTraining ResNet18 (Transfer Learning)")
print("=" * 55)

resnet_best_acc = train_model(
    resnet_model, train_loader, val_loader,
    epochs=5, lr=0.01  # Higher LR since fewer params
)

# Compare results
print("\n" + "=" * 55)
print("Comparison: CNN from Scratch vs Transfer Learning")
print("=" * 55)
print(f"  Simple CNN:          ~65% validation accuracy")
```

```
print(f"  ResNet18 (transfer): ~{resnet_best_acc:.0f}% validation accuracy")
print(f"  Improvement:         ~{resnet_best_acc - 65:.0f} percentage points")
```

Expected Output:

```
Training ResNet18 (Transfer Learning)
=======================================================
  Epoch 1/5, Batch 100/1563, Loss: 0.9876
  ...

  Epoch 1/5 Summary (38.5s):
    Train Loss: 0.8765, Train Acc: 72.30%
    Val Loss:   0.5432, Val Acc:   82.50%
    Best Val Acc: 82.50%

  ...

  Epoch 5/5 Summary (37.9s):
    Train Loss: 0.4321, Train Acc: 86.70%
    Val Loss:   0.3987, Val Acc:   87.40%
    Best Val Acc: 87.80%

Training Complete!
Best Validation Accuracy: 87.80%

=======================================================
Comparison: CNN from Scratch vs Transfer Learning
=======================================================
  Simple CNN:          ~65% validation accuracy
  ResNet18 (transfer): ~88% validation accuracy
  Improvement:         ~23 percentage points
```

Transfer learning gives us a massive accuracy boost — from about 65% to about 88% — while training fewer parameters and converging faster.

Saving the Model for Deployment

```python
# Save the trained model for later use in our app

# Save just the model weights
torch.save(resnet_model.state_dict(), 'image_classifier.pth')

# Also save the class names for our app
import json

class_info = {
    'class_names': ['airplane', 'automobile', 'bird', 'cat', 'deer',
                    'dog', 'frog', 'horse', 'ship', 'truck'],
```

```
    'num_classes': 10,
    'model_name': 'resnet18',
    'image_size': 224,
}

with open('class_info.json', 'w') as f:
    json.dump(class_info, f, indent=2)

print("Model saved!")
print(f"  Model weights: image_classifier.pth")
print(f"  Class info:    class_info.json")
print(f"\nClass names: {class_info['class_names']}")
```

Expected Output:

```
Model saved!
  Model weights: image_classifier.pth
  Class info:    class_info.json

Class names: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', ...
```

27.5 Part 4 — Building the Gradio Web App

Gradio is a Python library that creates web interfaces for machine learning models with just a few lines of code. It allows anyone to interact with your model through a browser — no web development knowledge needed.

First, install Gradio:

```bash
pip install gradio
```

Building the Complete App

```python
# Part 4: Complete Gradio web app for image classification

import torch
import torch.nn as nn
from torchvision import models, transforms
from PIL import Image
```

```
import gradio as gr
import json

# ==============================================================
# Step 1: Load the trained model
# ==============================================================

def load_model(model_path, class_info_path):
    """Load the trained model and class information."""

    # Load class info
    with open(class_info_path, 'r') as f:
        class_info = json.load(f)

    num_classes = class_info['num_classes']
    class_names = class_info['class_names']

    # Recreate the model architecture
    model = models.resnet18(weights=None)  # No pre-trained weights
    num_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(num_features, num_classes)
    )

    # Load trained weights
    model.load_state_dict(torch.load(model_path, map_location='cpu'))
    model.eval()  # Set to evaluation mode

    return model, class_names


# Load model (use pre-trained ResNet18 if saved model not available)
try:
    model, class_names = load_model('image_classifier.pth', 'class_info.json')
    print("Loaded custom trained model")
except FileNotFoundError:
    # Fallback: use pre-trained ImageNet model for demonstration
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    model.eval()

    # Load ImageNet class names
    weights = models.ResNet18_Weights.IMAGENET1K_V1
    class_names = weights.meta["categories"]
    print("Loaded pre-trained ImageNet model (1000 classes)")


# ==============================================================
# Step 2: Define the prediction function
# ==============================================================

# Image preprocessing (same as validation transform)
preprocess = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
```

```
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])


def classify_image(image):
    """
    Classify an uploaded image and return predictions.

    Parameters:
        image: A PIL Image uploaded by the user

    Returns:
        A dictionary mapping class names to confidence scores
    """

    if image is None:
        return {"No image provided": 1.0}

    # Preprocess the image
    input_tensor = preprocess(image)

    # Add batch dimension: (3, 224, 224) -> (1, 3, 224, 224)
    input_batch = input_tensor.unsqueeze(0)

    # Make prediction
    with torch.no_grad():
        outputs = model(input_batch)

    # Convert to probabilities using softmax
    probabilities = torch.nn.functional.softmax(outputs[0], dim=0)

    # Get top 5 predictions
    top5_probs, top5_indices = torch.topk(probabilities, 5)

    # Create results dictionary
    results = {}
    for i in range(5):
        class_idx = top5_indices[i].item()
        prob = top5_probs[i].item()
        class_name = class_names[class_idx]
        results[class_name] = float(prob)

    return results


# ==============================================================
# Step 3: Create the Gradio interface
# ==============================================================

# Build the web interface
demo = gr.Interface(
    fn=classify_image,           # Our prediction function
```

```
    inputs=gr.Image(type="pil"), # Input: image upload widget
    outputs=gr.Label(num_top_classes=5),  # Output: top-5 labels
    title="Image Classification App",
    description=(
        "Upload an image and the model will classify it. "
        "The model uses ResNet18 with transfer learning. "
        "It returns the top 5 predicted classes with confidence scores."
    ),
    examples=[
        # Add example images if you have them
        # ["example_cat.jpg"],
        # ["example_dog.jpg"],
    ],
    theme="default",
)

# ============================================================
# Step 4: Launch the app
# ============================================================

print("\nGradio App Configuration")
print("=" * 55)
print(f"Model:          ResNet18")
print(f"Classes:        {len(class_names)}")
print(f"Input:          Image upload (any size)")
print(f"Output:         Top 5 predictions with confidence")
print(f"Image size:     Resized to 224x224 internally")
print()
print("Starting the web app...")
print("Open your browser to the URL shown below.")
print()

# Launch the app
# share=True creates a public URL (useful for sharing)
demo.launch(share=False)  # Set share=True for a public link
```

Expected Output:

```
Loaded pre-trained ImageNet model (1000 classes)

Gradio App Configuration
=======================================================
Model:          ResNet18
Classes:        1000
Input:          Image upload (any size)
Output:         Top 5 predictions with confidence
Image size:     Resized to 224x224 internally

Starting the web app...
Open your browser to the URL shown below.

Running on local URL:  http://127.0.0.1:7860
```

```
+-----------------------------------------------------------------+
|              Gradio Web Interface                                |
+-----------------------------------------------------------------+
|                                                                  |
|  +-------------------------------------------------------------+  |
|  |          Image Classification App                          |  |
|  +-------------------------------------------------------------+  |
|  |                                                              |  |
|  | Upload an image and the model will classify it.              |  |
|  |                                                              |  |
|  | +-------------------------+  +----------------------------+  |  |
|  | |                         |  |  Top 5 Predictions:        |  |  |
|  | |                         |  |                            |  |  |
|  | |    [Drop image here]    |  |  1. Golden Retriever 89%   |  |  |
|  | |    or click to upload   |  |  2. Labrador       5%      |  |  |
|  | |                         |  |  3. Cocker Spaniel 2%      |  |  |
|  | |                         |  |  4. Irish Setter   1%      |  |  |
|  | +-------------------------+  |  5. Beagle         1%      |  |  |
|  |                               +----------------------------+  |  |
|  |                                                              |  |
|  |  [Submit]  [Clear]                                           |  |
|  +-------------------------------------------------------------+  |
|                                                                  |
+-----------------------------------------------------------------+
```

Putting It All Together — Complete App File

Here is the complete, self-contained application file:

```python
# complete_app.py
# Run with: python complete_app.py

"""
Complete Image Classification App
Uses ResNet18 with transfer learning for image classification.
Provides a Gradio web interface for easy interaction.
"""

import torch
import torch.nn as nn
from torchvision import models, transforms
import gradio as gr

# ---- Model Setup ----
print("Loading model...")
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
model.eval()

weights = models.ResNet18_Weights.IMAGENET1K_V1
class_names = weights.meta["categories"]
```

```
print(f"Model loaded with {len(class_names)} classes")

# ---- Image Preprocessing ----
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# ---- Prediction Function ----
def predict(image):
    """Classify an image and return top 5 predictions."""
    if image is None:
        return {}

    input_tensor = preprocess(image).unsqueeze(0)

    with torch.no_grad():
        output = model(input_tensor)

    probs = torch.nn.functional.softmax(output[0], dim=0)
    top5_probs, top5_ids = torch.topk(probs, 5)

    return {
        class_names[top5_ids[i]]: float(top5_probs[i])
        for i in range(5)
    }

# ---- Gradio Interface ----
demo = gr.Interface(
    fn=predict,
    inputs=gr.Image(type="pil"),
    outputs=gr.Label(num_top_classes=5),
    title="Image Classification App",
    description="Upload any image to classify it using ResNet18.",
)

if __name__ == "__main__":
    demo.launch()
```

To run this app, save it as `complete_app.py` and run `python complete_app.py`. Open the URL shown in the terminal in your browser, upload an image, and see the predictions.

Common Mistakes

1. Not matching transforms between training and inference: The preprocessing applied during inference must match the validation transforms used during training. If you trained with `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`, you must use the same values during inference.
2. Forgetting model.eval(): When using the model for predictions, always call `model.eval()` first. This disables dropout and batch normalization updates, which are only needed during training.
3. Not using torch.no_grad() during inference: Wrap your prediction code with `with torch.no_grad():` to save memory and speed up inference. Without this, PyTorch records operations for gradient computation, which is unnecessary during prediction.
4. Wrong image format for Gradio: Gradio provides images as PIL Images by default (when `type="pil"`). Make sure your preprocessing pipeline starts from a PIL Image.
5. Not saving class names with the model: When deploying a model, always save the class names alongside the weights. Without class names, you can only see class indices (0, 1, 2) instead of meaningful labels.

Best Practices

1. Always use transfer learning: Unless you have millions of training images, transfer learning with a pre-trained model will almost always outperform training from scratch.
2. Start with frozen features: Begin by freezing the pre-trained layers and only training the final classifier. If accuracy is not sufficient, gradually unfreeze later layers and fine-tune with a very small learning rate.
3. Use data augmentation: Random flips, rotations, and color adjustments help prevent overfitting and improve generalization, especially with small datasets.
4. Validate on unseen data: Always evaluate on a separate validation set. Training accuracy can be misleadingly high if the model memorizes the training data.
5. Save models with metadata: Save class names, image size, and model architecture alongside the weights so that anyone can reload and use the model correctly.

Quick Summary

This project brought together dataset preparation, CNN training, transfer learning, and web deployment into a complete application. We used PyTorch's ImageFolder to organize images

by class, built a simple CNN from scratch (achieving about 65% accuracy), then used transfer learning with ResNet18 to jump to about 88% accuracy by leveraging features pre-trained on ImageNet. Finally, we wrapped the model in a Gradio web interface that allows anyone to upload images and get predictions in their browser with just a few lines of code.

Key Points

- ImageFolder organizes images by placing them in class-named subdirectories
- Data augmentation (flips, rotations, color jitter) improves model robustness
- A simple CNN can achieve reasonable accuracy but has limitations
- Transfer learning with pre-trained models dramatically improves accuracy
- ResNet18 pre-trained on ImageNet provides excellent visual features
- Freezing pre-trained layers and training only the classifier is efficient and effective
- Gradio creates web interfaces with just `gr.Interface(fn, inputs, outputs)`
- Always preprocess inference images identically to training validation images
- Save both model weights and class metadata for deployment

Practice Questions

1. Why do we use different transforms for training and validation? Why is data augmentation only applied during training?
2. Explain why transfer learning with ResNet18 achieves much higher accuracy than a CNN trained from scratch, even though the ResNet18 classifier layer has far fewer trainable parameters.
3. What does `model.eval()` do, and why is it important to call it before making predictions?
4. If you wanted to classify medical X-ray images (a very different domain from ImageNet), would transfer learning from ResNet18 still help? Why or why not?
5. How would you modify this project to support 100 classes instead of 10?

Exercises

Exercise 1: Unfreeze and Fine-Tune Modify the ResNet18 training to first train with frozen features for 3 epochs, then unfreeze the last 2 residual blocks and continue training for 3 more epochs with a learning rate 10 times smaller. Compare the accuracy to the fully-frozen approach.

Exercise 2: Add Confidence Threshold Modify the Gradio app to show a warning when the model's top prediction has confidence below 50%. Display a message like "Low confidence prediction — the model is uncertain about this image."

Exercise 3: Multi-Image Classification Extend the Gradio app to accept multiple images at once (using `gr.File` with `file_count="multiple"`). Process each image, and display all results in a summary table showing the filename, top prediction, and confidence for each image.

What Is Next?

You have built a complete image classification app — from data preparation to web deployment. In the next chapter, you will tackle a more advanced computer vision task: object detection. Instead of classifying what is in an image, you will build a system that finds WHERE objects are in an image and draws boxes around them. You will use YOLOv8, one of the fastest object detection models, and create another Gradio app for real-time detection.

Chapter 28: Project — Object Detection System

What You Will Learn

- How to use YOLOv8 for object detection inference on images
- How to draw custom bounding boxes and labels on detected objects
- How to filter detections by confidence score and class
- How to save annotated images with detection results
- How to build a complete Gradio web app for object detection
- How to put together a deployable detection system

Why This Chapter Matters

Image classification tells you WHAT is in an image. Object detection tells you WHAT is in an image AND WHERE it is. This is a much harder and more useful task. Self-driving cars use object detection to find pedestrians, traffic lights, and other cars. Security cameras use it to detect people in restricted areas. Retail stores use it to count products on shelves. Medical imaging uses it to locate tumors in X-rays.

In this final project chapter, you will build a complete object detection system that takes any image, finds all objects in it, draws labeled boxes around them, and presents the results through a clean web interface. This is the capstone of your NLP and Computer Vision journey.

28.1 Project Overview

```
+-------------------------------------------------------------------+
|              Object Detection System Architecture                 |
+-------------------------------------------------------------------+
|                                                                   |
|  PART 1: YOLOv8 Inference                                        |
|  +-----------------------------+                                  |
|  | Load pre-trained YOLOv8     |                                  |
|  | Run detection on images     |                                  |
|  | Parse detection results     |                                  |
|  +-----------------------------+                                  |
|                |                                                  |
|                v                                                  |
|  PART 2: Custom Visualization                                    |
|  +-----------------------------+                                  |
|  | Draw bounding boxes         |                                  |
|  | Add class labels and scores |                                  |
|  | Color-code by class         |                                  |
|  +-----------------------------+                                  |
|                |                                                  |
|                v                                                  |
|  PART 3: Filtering and Control                                   |
|  +-----------------------------+                                  |
|  | Filter by confidence score  |                                  |
|  | Filter by object class      |                                  |
|  | Save annotated results      |                                  |
|  +-----------------------------+                                  |
|                |                                                  |
|                v                                                  |
|  PART 4: Gradio Detection App                                    |
|  +-----------------------------+                                  |
|  | Upload images               |                                  |
|  | Adjust settings             |                                  |
|  | View annotated results      |                                  |
|  +-----------------------------+                                  |
|                                                                   |
+-------------------------------------------------------------------+
```

What Is YOLO?

YOLO stands for You Only Look Once. It is one of the fastest and most popular object detection models. Unlike older detection methods that scan an image multiple times at different locations, YOLO processes the entire image in a single pass — hence "you only look once."

```
+-------------------------------------------------------------------+
|              How YOLO Works (Simplified)                          |
+-------------------------------------------------------------------+
|                                                                   |
|  Input Image                                                      |
|  +-------------------+                                            |
```

```
|  |  +---+            |                                          |
|  |  |cat|   +-----+  |                                          |
|  |  +---+   | dog |  |                                          |
|  |          +-----+  |                                          |
|  +-------------------+                                          |
|           |                                                     |
|           v                                                     |
|  YOLO divides the image into a grid:                            |
|  +-----+-----+-----+                                           |
|  |  .  |  .  |  .  |                                           |
|  +-----+-----+-----+                                           |
|  |  .  |  .  |  .  |                                           |
|  +-----+-----+-----+                                           |
|  |  .  |  .  |  .  |                                           |
|  +-----+-----+-----+                                           |
|                                                                 |
|  Each grid cell predicts:                                       |
|  - Bounding boxes (x, y, width, height)                        |
|  - Confidence scores (how sure it is)                          |
|  - Class probabilities (what object it is)                     |
|                                                                 |
|  All predictions happen in ONE forward pass!                   |
|  That is why YOLO is so fast.                                  |
|                                                                 |
+-----------------------------------------------------------------+
```

YOLOv8 is the latest version, released by Ultralytics. It is faster, more accurate, and easier to use than previous versions.

28.2 Part 1 — YOLOv8 Inference on Images

First, install the Ultralytics library:

```bash
pip install ultralytics
```

Running Detection on an Image

```python
# Part 1: YOLOv8 inference on images

from ultralytics import YOLO
from PIL import Image
import os
```

```
# Load YOLOv8 pre-trained model
# 'yolov8n' is the nano (smallest, fastest) version
# Other options: yolov8s (small), yolov8m (medium),
#                yolov8l (large), yolov8x (extra large)
model = YOLO('yolov8n.pt')

print("YOLOv8 Model Loaded")
print("=" * 55)
print(f"Model: YOLOv8n (nano)")
print(f"Number of classes: {len(model.names)}")
print(f"\nFirst 20 classes:")
for i in range(20):
    print(f"  {i:3d}: {model.names[i]}")

print(f"\nSome other classes:")
for i in [24, 39, 56, 62, 67, 73]:
    print(f"  {i:3d}: {model.names[i]}")
```

Expected Output:

```
YOLOv8 Model Loaded
=======================================================
Model: YOLOv8n (nano)
Number of classes: 80

First 20 classes:
    0: person
    1: bicycle
    2: car
    3: motorcycle
    4: airplane
    5: bus
    6: train
    7: truck
    8: boat
    9: traffic light
   10: fire hydrant
   11: stop sign
   12: parking meter
   13: bench
   14: bird
   15: cat
   16: dog
   17: horse
   18: sheep
   19: cow

Some other classes:
   24: backpack
   39: bottle
   56: chair
   62: tv
```

```
    67: cell phone
    73: book
```

YOLOv8 is trained on the COCO dataset, which contains 80 common object classes including people, vehicles, animals, furniture, and everyday items.

Running Detection and Parsing Results

```python
# Running detection on an image

from ultralytics import YOLO
from PIL import Image

model = YOLO('yolov8n.pt')

# Run detection on an image
# You can use any image file path or URL
results = model('https://ultralytics.com/images/bus.jpg')

# Parse the results
result = results[0]  # First image's results

print("Detection Results")
print("=" * 55)
print(f"Image size: {result.orig_shape}")
print(f"Number of detections: {len(result.boxes)}")
print()

# Extract detection details
print(f"{'#':<4} {'Class':<15} {'Confidence':<12} {'Bounding Box (x1,y1,x2,y2)'}")
print("-" * 65)

for i, box in enumerate(result.boxes):
    # Get class ID and name
    class_id = int(box.cls[0])
    class_name = model.names[class_id]

    # Get confidence score
    confidence = float(box.conf[0])

    # Get bounding box coordinates
    # xyxy format: (x1, y1, x2, y2) = (left, top, right, bottom)
    x1, y1, x2, y2 = box.xyxy[0].tolist()

    print(f"{i+1:<4} {class_name:<15} {confidence:<12.4f} "
          f"({x1:.0f}, {y1:.0f}, {x2:.0f}, {y2:.0f})")

# Summary by class
print(f"\nDetection Summary:")
class_counts = {}
```

```
for box in result.boxes:
    class_name = model.names[int(box.cls[0])]
    class_counts[class_name] = class_counts.get(class_name, 0) + 1

for class_name, count in sorted(class_counts.items()):
    print(f"  {class_name}: {count}")
```

Expected Output:

```
Detection Results
========================================================
Image size: (1080, 810)
Number of detections: 5

#    Class           Confidence   Bounding Box (x1,y1,x2,y2)
-------------------------------------------------------------------
1    person          0.8845       (670, 377, 809, 878)
2    person          0.8714       (48, 398, 245, 903)
3    bus             0.8675       (17, 231, 801, 768)
4    person          0.8423       (222, 407, 343, 859)
5    person          0.5987       (0, 551, 63, 872)

Detection Summary:
  bus: 1
  person: 4
```

Let us explain the key detection fields:

- class_id / class_name: Which object class was detected (person, car, dog, etc.)
- confidence: How confident the model is in this detection (0.0 to 1.0). Higher is more confident
- bounding box (x1, y1, x2, y2): The coordinates of the rectangle around the detected object. (x1, y1) is the top-left corner and (x2, y2) is the bottom-right corner

```
+-------------------------------------------------------------------+
|              Bounding Box Coordinates                              |
+-------------------------------------------------------------------+
|                                                                    |
|  (x1, y1) +-----------+                                          |
|            |           |                                          |
|            |   OBJECT  |    Bounding box format: xyxy             |
|            |           |    (x1, y1) = top-left corner            |
|            +-----------+ (x2, y2)   (x2, y2) = bottom-right corner|
|                                                                    |
|  Width  = x2 - x1                                                 |
|  Height = y2 - y1                                                 |
|                                                                    |
+-------------------------------------------------------------------+
```

28.3 Part 2 — Custom Visualization with Bounding Boxes

While YOLOv8 has built-in plotting, let us build our own visualization to learn how it works and have full control over the appearance.

```python
# Part 2: Custom visualization with bounding boxes

from PIL import Image, ImageDraw, ImageFont
from ultralytics import YOLO
import random

def get_class_colors(num_classes):
    """Generate distinct colors for each class."""
    random.seed(42)  # Fixed seed for consistent colors
    colors = {}
    for i in range(num_classes):
        colors[i] = (
            random.randint(50, 255),
            random.randint(50, 255),
            random.randint(50, 255)
        )
    return colors


def draw_detections(image, detections, class_names, min_confidence=0.5):
    """
    Draw bounding boxes and labels on an image.

    Parameters:
        image: PIL Image to annotate
        detections: YOLO detection results (result.boxes)
        class_names: Dictionary mapping class IDs to names
        min_confidence: Minimum confidence to display a detection

    Returns:
        Annotated PIL Image
    """

    # Create a copy to avoid modifying the original
    annotated = image.copy()
    draw = ImageDraw.Draw(annotated)

    # Generate colors for each class
    colors = get_class_colors(len(class_names))

    # Try to load a nice font, fall back to default
```

```
    try:
        font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf...
        small_font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.tt...
    except (IOError, OSError):
        font = ImageFont.load_default()
        small_font = font

    # Track detection count
    detection_count = 0

    for box in detections:
        # Get detection info
        class_id = int(box.cls[0])
        confidence = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        # Skip low-confidence detections
        if confidence < min_confidence:
            continue

        detection_count += 1
        class_name = class_names[class_id]
        color = colors[class_id]

        # Draw the bounding box (rectangle)
        box_width = 3
        draw.rectangle(
            [x1, y1, x2, y2],
            outline=color,
            width=box_width
        )

        # Create label text
        label = f"{class_name} {confidence:.0%}"

        # Calculate label background size
        bbox = draw.textbbox((0, 0), label, font=font)
        label_width = bbox[2] - bbox[0]
        label_height = bbox[3] - bbox[1]

        # Draw label background
        label_y = max(y1 - label_height - 6, 0)
        draw.rectangle(
            [x1, label_y, x1 + label_width + 8, label_y + label_height + 6],
            fill=color
        )

        # Draw label text (white on colored background)
        draw.text(
            (x1 + 4, label_y + 2),
            label,
            fill=(255, 255, 255),
            font=font
        )
```

```
    # Add detection summary at the top
    summary = f"Detected: {detection_count} objects (min confidence: {min_confidence:.0...
    draw.rectangle([0, 0, len(summary) * 8 + 10, 22], fill=(0, 0, 0, 180))
    draw.text((5, 3), summary, fill=(255, 255, 255), font=small_font)

    return annotated, detection_count


# Run detection and visualize
model = YOLO('yolov8n.pt')
image = Image.open('bus.jpg')  # Use any image

results = model(image)
result = results[0]

# Draw detections
annotated_image, count = draw_detections(
    image, result.boxes, model.names, min_confidence=0.5
)

# Save the result
annotated_image.save('detected_bus.jpg')

print("Custom Visualization Complete")
print("=" * 55)
print(f"Objects detected: {count}")
print(f"Annotated image saved: detected_bus.jpg")
```

Expected Output:

```
Custom Visualization Complete
=======================================================
Objects detected: 5
Annotated image saved: detected_bus.jpg
```

Let us trace through the `draw_detections` function:

- Lines 8-16: We create a copy of the image and an `ImageDraw` object. The draw object lets us draw shapes and text on the image
- Lines 19: We generate a unique color for each object class so different classes are easy to distinguish
- Lines 22-26: We try to load a TrueType font for nice-looking labels. If the font file is not available, we fall back to a basic default font
- Lines 33-37: For each detection, we extract the class ID, confidence, and bounding box coordinates
- Lines 40-41: We skip detections with confidence below our threshold
- Lines 47-52: We draw the bounding box as a colored rectangle using `draw.rectangle`

- Lines 55-74: We draw a label showing the class name and confidence percentage. The label has a colored background for readability

28.4 Part 3 — Confidence Filtering and Class Filtering

In real applications, you often want to filter detections. For example, a security system might only care about "person" detections, or a wildlife camera might only want animals.

```python
# Part 3: Filtering detections by confidence and class

from ultralytics import YOLO
from PIL import Image

model = YOLO('yolov8n.pt')


def detect_and_filter(image_path, confidence_threshold=0.5,
                      target_classes=None):
    """
    Detect objects with filtering options.

    Parameters:
        image_path: Path to the image file
        confidence_threshold: Minimum confidence score (0.0 to 1.0)
        target_classes: List of class names to detect, or None for all

    Returns:
        List of detection dictionaries
    """

    # Run detection
    results = model(image_path, verbose=False)
    result = results[0]

    # Parse and filter detections
    detections = []

    for box in result.boxes:
        class_id = int(box.cls[0])
        class_name = model.names[class_id]
        confidence = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        # Filter by confidence
        if confidence < confidence_threshold:
```

```
            continue

        # Filter by class (if specified)
        if target_classes and class_name not in target_classes:
            continue

        detections.append({
            'class_name': class_name,
            'class_id': class_id,
            'confidence': confidence,
            'bbox': {
                'x1': int(x1), 'y1': int(y1),
                'x2': int(x2), 'y2': int(y2)
            },
            'width': int(x2 - x1),
            'height': int(y2 - y1),
            'area': int((x2 - x1) * (y2 - y1))
        })

    # Sort by confidence (highest first)
    detections.sort(key=lambda d: d['confidence'], reverse=True)

    return detections


# Example 1: All detections above 50% confidence
print("Example 1: All objects (confidence > 50%)")
print("=" * 55)

detections = detect_and_filter('bus.jpg', confidence_threshold=0.5)

for det in detections:
    print(f"  {det['class_name']:<15} "
          f"conf={det['confidence']:.2f}  "
          f"size={det['width']}x{det['height']}  "
          f"area={det['area']} px")

print(f"\n  Total: {len(detections)} objects")


# Example 2: Only persons with high confidence
print(f"\nExample 2: Only persons (confidence > 70%)")
print("=" * 55)

person_detections = detect_and_filter(
    'bus.jpg',
    confidence_threshold=0.7,
    target_classes=['person']
)

for det in person_detections:
    print(f"  {det['class_name']:<15} "
          f"conf={det['confidence']:.2f}  "
          f"bbox=({det['bbox']['x1']},{det['bbox']['y1']},"
          f"{det['bbox']['x2']},{det['bbox']['y2']})")
```

```
print(f"\n  Total persons found: {len(person_detections)}")


# Example 3: Only vehicles
print(f"\nExample 3: Only vehicles (confidence > 50%)")
print("=" * 55)

vehicle_classes = ['car', 'bus', 'truck', 'motorcycle', 'bicycle']
vehicle_detections = detect_and_filter(
    'bus.jpg',
    confidence_threshold=0.5,
    target_classes=vehicle_classes
)

for det in vehicle_detections:
    print(f"  {det['class_name']:<15} conf={det['confidence']:.2f}")

print(f"\n  Total vehicles found: {len(vehicle_detections)}")
```

Expected Output:

```
Example 1: All objects (confidence > 50%)
=======================================================
  person          conf=0.88  size=139x501  area=69639 px
  person          conf=0.87  size=197x505  area=99485 px
  bus             conf=0.87  size=784x537  area=421008 px
  person          conf=0.84  size=121x452  area=54692 px
  person          conf=0.60  size=63x321   area=20223 px

  Total: 5 objects

Example 2: Only persons (confidence > 70%)
=======================================================
  person          conf=0.88  bbox=(670,377,809,878)
  person          conf=0.87  bbox=(48,398,245,903)
  person          conf=0.84  bbox=(222,407,343,859)

  Total persons found: 3

Example 3: Only vehicles (confidence > 50%)
=======================================================
  bus             conf=0.87

  Total vehicles found: 1
```

Saving Annotated Results

```python
# Saving annotated images with detection results

import json
from datetime import datetime

def save_detection_results(image_path, output_dir='detection_results'):
    """
    Run detection and save annotated image + JSON results.
    """

    os.makedirs(output_dir, exist_ok=True)

    # Load image
    image = Image.open(image_path)

    # Run detection
    results = model(image_path, verbose=False)
    result = results[0]

    # Draw annotations
    annotated_image, count = draw_detections(
        image, result.boxes, model.names, min_confidence=0.5
    )

    # Generate filenames
    base_name = os.path.splitext(os.path.basename(image_path))[0]
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

    img_output = os.path.join(output_dir, f'{base_name}_detected_{timestamp}.jpg')
    json_output = os.path.join(output_dir, f'{base_name}_results_{timestamp}.json')

    # Save annotated image
    annotated_image.save(img_output, quality=95)

    # Save detection data as JSON
    detection_data = {
        'source_image': image_path,
        'timestamp': timestamp,
        'image_size': {'width': image.width, 'height': image.height},
        'total_detections': count,
        'detections': []
    }

    for box in result.boxes:
        confidence = float(box.conf[0])
        if confidence < 0.5:
            continue

        class_id = int(box.cls[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        detection_data['detections'].append({
            'class': model.names[class_id],
            'confidence': round(confidence, 4),
```

```
            'bbox': {
                'x1': int(x1), 'y1': int(y1),
                'x2': int(x2), 'y2': int(y2)
            }
        })

    with open(json_output, 'w') as f:
        json.dump(detection_data, f, indent=2)

    print(f"Results saved:")
    print(f"  Annotated image: {img_output}")
    print(f"  Detection data:  {json_output}")
    print(f"  Objects found:   {count}")

    return img_output, json_output


# Save results
save_detection_results('bus.jpg')
```

Expected Output:

```
Results saved:
  Annotated image: detection_results/bus_detected_20240115_143022.jpg
  Detection data:  detection_results/bus_results_20240115_143022.json
  Objects found:   5
```

28.5 Part 4 — Building the Gradio Detection App

Now let us put everything together into a polished Gradio web application:

```python
# Part 4: Complete Gradio object detection app

from ultralytics import YOLO
from PIL import Image, ImageDraw, ImageFont
import gradio as gr
import random
import numpy as np

# ============================================================
# Load the model
# ============================================================

model = YOLO('yolov8n.pt')
```

```
print(f"YOLOv8 model loaded with {len(model.names)} classes")

# Generate consistent colors for each class
random.seed(42)
CLASS_COLORS = {
    i: (random.randint(50, 255), random.randint(50, 255), random.randint(50, 255))
    for i in range(len(model.names))
}


# ==============================================================
# Detection and visualization function
# ==============================================================

def detect_objects(image, confidence_threshold, class_filter):
    """
    Detect objects in an image and return annotated result.

    Parameters:
        image: PIL Image uploaded by the user
        confidence_threshold: Minimum confidence (0.0-1.0)
        class_filter: Comma-separated class names, or "all"

    Returns:
        Annotated image and detection summary text
    """

    if image is None:
        return None, "No image provided."

    # Parse class filter
    target_classes = None
    if class_filter and class_filter.strip().lower() != "all":
        target_classes = [c.strip().lower() for c in class_filter.split(',')]

    # Run YOLOv8 detection
    results = model(image, verbose=False)
    result = results[0]

    # Create annotated image
    annotated = image.copy()
    draw = ImageDraw.Draw(annotated)

    try:
        font = ImageFont.truetype(
            "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 18
        )
    except (IOError, OSError):
        try:
            font = ImageFont.truetype(
                "/System/Library/Fonts/Helvetica.ttc", 18
            )
        except (IOError, OSError):
            font = ImageFont.load_default()
```

```
    # Process detections
    detection_count = 0
    summary_lines = []
    class_counts = {}

    for box in result.boxes:
        class_id = int(box.cls[0])
        class_name = model.names[class_id]
        confidence = float(box.conf[0])

        # Apply confidence filter
        if confidence < confidence_threshold:
            continue

        # Apply class filter
        if target_classes and class_name.lower() not in target_classes:
            continue

        detection_count += 1
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        color = CLASS_COLORS[class_id]

        # Draw bounding box
        draw.rectangle([x1, y1, x2, y2], outline=color, width=3)

        # Draw label
        label = f"{class_name} {confidence:.0%}"
        bbox = draw.textbbox((0, 0), label, font=font)
        lw = bbox[2] - bbox[0]
        lh = bbox[3] - bbox[1]
        ly = max(y1 - lh - 8, 0)

        draw.rectangle(
            [x1, ly, x1 + lw + 10, ly + lh + 8],
            fill=color
        )
        draw.text((x1 + 5, ly + 3), label, fill='white', font=font)

        # Track class counts
        class_counts[class_name] = class_counts.get(class_name, 0) + 1

    # Build summary text
    summary_lines.append(f"Total objects detected: {detection_count}")
    summary_lines.append(f"Confidence threshold: {confidence_threshold:.0%}")

    if class_filter and class_filter.strip().lower() != "all":
        summary_lines.append(f"Class filter: {class_filter}")

    summary_lines.append("")
    summary_lines.append("Detections by class:")

    for class_name, count in sorted(class_counts.items()):
        summary_lines.append(f"  {class_name}: {count}")

    if detection_count == 0:
```

```
        summary_lines.append("  No objects detected with current settings.")
        summary_lines.append("  Try lowering the confidence threshold.")

    summary = "\n".join(summary_lines)

    return annotated, summary


# ==============================================================
# Build the Gradio interface
# ==============================================================

# List some common classes for the description
common_classes = ", ".join(list(model.names.values())[:15]) + ", ..."

demo = gr.Interface(
    fn=detect_objects,
    inputs=[
        gr.Image(type="pil", label="Upload Image"),
        gr.Slider(
            minimum=0.1, maximum=1.0, value=0.5, step=0.05,
            label="Confidence Threshold",
            info="Only show detections above this confidence level"
        ),
        gr.Textbox(
            value="all",
            label="Class Filter",
            info="Enter class names separated by commas (e.g., 'person, car, dog') or '...
            placeholder="all"
        ),
    ],
    outputs=[
        gr.Image(type="pil", label="Detection Results"),
        gr.Textbox(label="Detection Summary", lines=10),
    ],
    title="Object Detection System",
    description=(
        "Upload an image to detect objects using YOLOv8. "
        "Adjust the confidence threshold and filter by class. "
        f"Available classes: {common_classes}"
    ),
    examples=[
        # Add example images if available
        # ["example1.jpg", 0.5, "all"],
        # ["example2.jpg", 0.7, "person, car"],
    ],
    flagging_mode="never",
)


# ==============================================================
# Launch the app
# ==============================================================

print("\nObject Detection App")
```

```
print("=" * 55)
print(f"Model: YOLOv8n ({len(model.names)} classes)")
print(f"Available classes: {len(model.names)}")
print()
print("Starting web app...")

demo.launch(share=False)
```

Expected Output:

```
YOLOv8 model loaded with 80 classes

Object Detection App
=======================================================
Model: YOLOv8n (80 classes)
Available classes: 80

Starting web app...
Running on local URL:  http://127.0.0.1:7860
```

```
+-------------------------------------------------------------------+
|              Object Detection Web Interface                        |
+-------------------------------------------------------------------+
|                                                                    |
|  +-----------------------------------------------------------+   |
|  |          Object Detection System                          |   |
|  +-----------------------------------------------------------+   |
|  |                                                            |   |
|  | Upload an image to detect objects using YOLOv8.            |   |
|  |                                                            |   |
|  | +---------+  +----------+  +---------------------------+   |   |
|  | | Upload  |  |Confidence|  | Class Filter              |   |   |
|  | | Image   |  | [===|==] |  | [all                   ]  |   |   |
|  | |         |  |   0.50   |  |                           |   |   |
|  | +---------+  +----------+  +---------------------------+   |   |
|  |                                                            |   |
|  | +------------------------+  +---------------------------+  |   |
|  | |                        |  | Detection Summary:        |  |   |
|  | |   [Annotated image     |  |                           |  |   |
|  | |    with boxes and      |  | Total detected: 5         |  |   |
|  | |    labels shown here]  |  | Confidence: 50%           |  |   |
|  | |                        |  |                           |  |   |
|  | |                        |  | Detections by class:      |  |   |
|  | |                        |  |   bus: 1                  |  |   |
|  | +------------------------+  |   person: 4              |  |   |
|  |                              +---------------------------+  |   |
|  |  [Submit]  [Clear]                                          |   |
|  +-----------------------------------------------------------+   |
|                                                                    |
+-------------------------------------------------------------------+
```

Complete Standalone Detection App

Here is the complete, self-contained application file that you can run independently:

```python
# detection_app.py
# Run with: python detection_app.py
# Install dependencies: pip install ultralytics gradio pillow

"""
Complete Object Detection App
Uses YOLOv8 for real-time object detection with a Gradio web interface.
"""

from ultralytics import YOLO
from PIL import Image, ImageDraw, ImageFont
import gradio as gr
import random

# Load model
print("Loading YOLOv8 model...")
model = YOLO('yolov8n.pt')
print(f"Model ready with {len(model.names)} object classes")

# Class colors
random.seed(42)
COLORS = {i: (random.randint(50, 255), random.randint(50, 255),
              random.randint(50, 255)) for i in range(80)}


def detect(image, conf_thresh=0.5, classes="all"):
    """Detect objects and return annotated image + summary."""
    if image is None:
        return None, "Please upload an image."

    # Parse class filter
    filter_classes = None
    if classes.strip().lower() != "all":
        filter_classes = [c.strip().lower() for c in classes.split(",")]

    # Run YOLO
    results = model(image, verbose=False)
    boxes = results[0].boxes

    # Draw
    img = image.copy()
    draw = ImageDraw.Draw(img)

    try:
        font = ImageFont.truetype("DejaVuSans-Bold.ttf", 16)
    except IOError:
        font = ImageFont.load_default()

    counts = {}
```

```
    n = 0

    for box in boxes:
        cls_id = int(box.cls[0])
        name = model.names[cls_id]
        conf = float(box.conf[0])

        if conf < conf_thresh:
            continue
        if filter_classes and name.lower() not in filter_classes:
            continue

        n += 1
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        color = COLORS[cls_id]

        draw.rectangle([x1, y1, x2, y2], outline=color, width=3)

        label = f"{name} {conf:.0%}"
        tb = draw.textbbox((0, 0), label, font=font)
        lh = tb[3] - tb[1]
        lw = tb[2] - tb[0]
        ly = max(y1 - lh - 6, 0)
        draw.rectangle([x1, ly, x1 + lw + 8, ly + lh + 6], fill=color)
        draw.text((x1 + 4, ly + 2), label, fill="white", font=font)

        counts[name] = counts.get(name, 0) + 1

    # Summary
    lines = [f"Objects found: {n}", f"Threshold: {conf_thresh:.0%}", ""]
    for name, cnt in sorted(counts.items()):
        lines.append(f"  {name}: {cnt}")
    if n == 0:
        lines.append("  None. Try lowering the confidence threshold.")

    return img, "\n".join(lines)


# Build interface
app = gr.Interface(
    fn=detect,
    inputs=[
        gr.Image(type="pil", label="Upload Image"),
        gr.Slider(0.1, 1.0, 0.5, step=0.05, label="Confidence"),
        gr.Textbox(value="all", label="Classes (comma-separated or 'all')"),
    ],
    outputs=[
        gr.Image(type="pil", label="Detections"),
        gr.Textbox(label="Summary", lines=8),
    ],
    title="Object Detection with YOLOv8",
    description="Upload any image to detect and locate objects.",
)

if __name__ == "__main__":
```

```
    app.launch()
```

Save this as `detection_app.py` and run it with `python detection_app.py`. Open the URL in your browser, upload any image, and see objects detected with bounding boxes, labels, and confidence scores.

Common Mistakes

1. Using too low a confidence threshold: Setting the threshold below 0.3 produces many false positives (detecting objects that are not there). Start with 0.5 and adjust downward only if you are missing important detections.
2. Not handling missing fonts: The `ImageFont.truetype()` call fails if the font file does not exist. Always include a try/except fallback to `ImageFont.load_default()`.
3. Confusing box coordinate formats: YOLO can output boxes in different formats: `xyxy` (x1, y1, x2, y2), `xywh` (center_x, center_y, width, height), and `xyxyn` (normalized). Always use `box.xyxy` for drawing and make sure you know which format you are working with.
4. Running YOLO on very large images without resizing: Very large images (4000x3000 or bigger) use a lot of memory. YOLOv8 handles resizing internally, but if you are processing many images, consider resizing first.
5. Forgetting verbose=False: By default, YOLO prints detection info to the console for every image. Use `verbose=False` in production to suppress this output.

Best Practices

1. Choose the right YOLO model size: Use `yolov8n` (nano) for speed, `yolov8s` (small) for a good balance, and `yolov8m/l/x` for maximum accuracy. Nano is fast enough for most real-time applications.
2. Use confidence thresholds appropriate for your use case: Security applications might use a lower threshold (0.3) to avoid missing threats. Product counting might use a higher threshold (0.7) to avoid false counts.
3. Filter by relevant classes: If you only care about people and vehicles, filter out the other 75+ classes. This reduces noise and speeds up post-processing.
4. Save both images and metadata: Save the annotated image for visual inspection and the JSON metadata for programmatic analysis. This makes your results easy to review

and process.

5. Test with diverse images: Test your detection app with images of different sizes, lighting conditions, and object counts to ensure robustness.

Quick Summary

In this project, we built a complete object detection system using YOLOv8, one of the fastest and most accurate detection models available. We loaded a pre-trained model that recognizes 80 object classes, ran inference on images, parsed detection results (class names, confidence scores, bounding box coordinates), built custom visualizations with labeled bounding boxes, implemented filtering by confidence and class, saved annotated results with metadata, and created a Gradio web interface where users can upload images, adjust detection settings, and see results instantly.

Key Points

- YOLO (You Only Look Once) detects all objects in an image in a single forward pass
- YOLOv8 is pre-trained on COCO dataset with 80 common object classes
- Model sizes range from nano (fast) to extra-large (accurate): yolov8n/s/m/l/x
- Each detection includes: class name, confidence score, and bounding box coordinates
- Bounding boxes in xyxy format: (x1, y1) = top-left, (x2, y2) = bottom-right
- Confidence threshold filters out uncertain detections (default: 0.5)
- Class filtering lets you detect only specific object types
- Custom visualization draws colored boxes and labels using PIL/Pillow
- Gradio creates an interactive web interface with sliders and text inputs
- Save both annotated images and JSON metadata for complete records

Practice Questions

1. What does "You Only Look Once" mean in the context of YOLO? How is this different from older detection methods that scan an image multiple times?

2. Explain the difference between image classification and object detection. Why is object detection a harder problem?
3. If you set the confidence threshold to 0.9, what happens to your detections? What about 0.1? What is the trade-off?
4. Why do we generate different colors for different object classes in our visualization? What would happen if we used the same color for everything?
5. You are building a parking lot monitoring system. Which YOLO model size would you choose, what confidence threshold would you set, and which classes would you filter for? Explain your reasoning.

Exercises

Exercise 1: Detection Statistics Dashboard Extend the Gradio app to include additional statistics: the average confidence score across all detections, the largest and smallest detected objects (by bounding box area), and the percentage of the image covered by detections. Display these in the summary text box.

Exercise 2: Multi-Image Batch Processing Write a function that takes a folder of images, runs YOLOv8 on each one, saves annotated versions, and produces a CSV summary file listing the filename, number of detections, and classes found in each image. Process all images in the folder automatically.

Exercise 3: Detection Comparison Tool Build a Gradio app that lets the user upload one image but run detection with two different YOLO model sizes (e.g., yolov8n and yolov8m). Display both annotated images side by side with their summaries so the user can compare speed vs. accuracy trade-offs.

Congratulations!

You have completed Book 4: NLP and Computer Vision! Let us look at the incredible journey you have been on.

What You Have Accomplished

```
+-----------------------------------------------------------------+
|              Your NLP & Computer Vision Journey                  |
+-----------------------------------------------------------------+
|                                                                  |
|  NLP Foundations (Chapters 1-6):                                 |
|  - Text preprocessing, tokenization, stopwords                  |
|  - Bag of Words, TF-IDF, word embeddings                        |
|  - Text classification and named entity recognition              |
|                                                                  |
|  Advanced NLP (Chapters 7-15):                                   |
|  - Text summarization (extractive and abstractive)               |
|  - Sequence-to-sequence models and attention                     |
|  - BERT (bidirectional understanding)                            |
|  - GPT family (text generation and RLHF)                        |
|  - Hugging Face ecosystem and fine-tuning                        |
|  - Question answering, text generation, multilingual NLP         |
|                                                                  |
|  Computer Vision (Chapters 16-24):                               |
|  - Image fundamentals and preprocessing                          |
|  - CNNs and transfer learning                                    |
|  - Object detection and image segmentation                       |
|  - Face detection and Vision Transformers                        |
|  - OCR (optical character recognition)                           |
|                                                                  |
|  Projects (Chapters 25-28):                                      |
|  - Sentiment analysis API                                        |
|  - Document classifier                                           |
|  - Image classification app with Gradio                          |
|  - Object detection system with YOLOv8                           |
|                                                                  |
+-----------------------------------------------------------------+
```

You now have the knowledge and skills to:

- Process and analyze text data with modern NLP techniques
- Use pre-trained models like BERT and GPT for real-world tasks
- Build image classification and object detection systems
- Create web interfaces for your AI models
- Deploy complete AI applications

What Is Next? — Book 5: Generative AI and LLMs

The next book takes you into the most exciting frontier of AI: Generative AI and Large Language Models (LLMs). Here is what awaits you:

```
+-----------------------------------------------------------------+
|              Book 5 Preview: Generative AI & LLMs                |
+-----------------------------------------------------------------+
|                                                                  |
```

```
|  What You Will Learn:                                                |
|                                                                      |
|  - The Transformer architecture in complete detail                  |
|  - How Large Language Models (LLMs) are built and trained           |
|  - Prompt engineering techniques for getting the best results       |
|  - Fine-tuning LLMs on your own data (LoRA, QLoRA)                 |
|  - Retrieval-Augmented Generation (RAG) for knowledge systems       |
|  - Building AI agents that can use tools and take actions           |
|  - Generative AI for images (Stable Diffusion)                      |
|  - Responsible AI development and safety considerations             |
|  - Building production-ready LLM applications                       |
|                                                                      |
|  Book 5 builds directly on the BERT, GPT, and Transformer           |
|  knowledge from this book. You are perfectly prepared.              |
|                                                                      |
+----------------------------------------------------------------------+
```

The skills you have built in this book — understanding tokenization, attention mechanisms, encoder-decoder architectures, BERT, and GPT — are the exact foundation you need for the generative AI revolution. You are no longer a beginner. You are ready to build the next generation of AI applications.

Keep learning. Keep building. The best is yet to come.

What Is Next?

Continue your AI journey with Book 5: Generative AI and LLMs, where you will dive deep into the technology powering today's most impressive AI systems. You will learn to build RAG systems, fine-tune LLMs, create AI agents, and develop production-ready generative AI applications. Everything you have learned so far has prepared you for this exciting next step.